

Bangladesh University of Engineering and Technology

Department of Computer Science & Engineering

CSE 207 Question Bank With Answers

Data Structures & Algorithms II

Topics: Graph Algorithms, Advanced Data Structures, Hashing,
NP-Completeness, Coping with Hardness, String Matching Algorithms, Randomized Algorithms

Authors & Contributors

Md Ratul Hasan (2405009) — Question Curation & Organisation
— $\text{\LaTeX}$ Typesetting

NotebookLM — Research & Extraction

Claude AI — $\text{\LaTeX}$ Typesetting

Gemini — Diagram Generation

Table of Contents

Part I — Graph Algorithms

- ▷ **S1.** Graph Algorithms
 - Shortest Paths
 - Minimum Spanning Trees
 - Maximum Flow
 - Maximum Bipartite Matching
 - Stable Matching (Gale-Shapley)
 - Graph Traversal & Properties

Part II — Advanced Data Structures

- ▷ **S2.** Advanced Data Structures
 - Balanced Binary Search Trees (AVL Trees)
 - Red-Black Trees
 - Splay Trees
 - Skip Lists
 - Advanced Heaps (Binomial Heaps)
 - Advanced Heaps (Fibonacci Heaps)

Part III — Hashing

- ▷ **S3.** Hashing
 - Hash Functions and Collision Resolution

Part IV — NP-Completeness

- ▷ **S4.** NP-Completeness
 - P vs NP Classes
 - Reductions
 - NP-hard and NP-complete Problems

Part V — Coping with Hardness

- ▷ **S5.** Coping with Hardness
 - Approximation Algorithms
 - Backtracking
 - Branch and Bound

Part VI — String Matching Algorithms & Randomized Algorithms

- ▷ **S6.** String Matching Algorithms
 - Knuth-Morris-Pratt (KMP) Algorithm
- ▷ **S7.** Randomized Algorithms

- Monte Carlo Algorithms & Probability Amplification
- Las Vegas Algorithms & Randomized Quick Sort

BUET · CSE 207 · Data Structures & Algorithms II

Part I

Graph Algorithms

Shortest Paths, Minimum Spanning Trees, Maximum Flow, Maximum Bipartite Matching, Stable Matching (Gale-Shapley), Graph Traversal & Properties

S1 — Graph Algorithms

Shortest Paths

Q1. Let S be a source vertex in a directed graph with non-negative edge weights. Suppose that after applying Dijkstra's algorithm, the following shortest path distances from S to all other vertices are obtained:

Vertex	S	A	B	C	D	E	F	G
Distance	0	2	4	5	7	8	9	11

Construct a directed weighted graph with non-negative integer edge weights such that running Dijkstra's algorithm from source S produces exactly the given distances, and justify that Dijkstra's algorithm produces those distances. The graph must satisfy:

- (a) Every vertex must be reachable from S .
- (b) The graph must contain at least two edges that are not part of any shortest path from S .
- (c) The shortest path to each vertex must be uniquely determined.

(15 marks)

[CSE 207 | L-2/T-1 | 2024–25]

Answer

1. Graph Construction

To construct a directed graph $G = (V, E)$ that satisfies all the given conditions, we first establish a backbone path (a shortest path tree spanning all vertices) that yields the exact target distances. Then, we introduce auxiliary edges that do not alter these shortest paths but fulfill the remaining structural constraints.

Let the vertex set be $V = \{S, A, B, C, D, E, F, G\}$. We define the edge set E with non-negative integer weights $w(u, v)$ as follows:

Shortest Path Tree Edges (Main Chain):

- $w(S, A) = 2 \implies d(A) = 2$
- $w(A, B) = 2 \implies d(B) = 2 + 2 = 4$
- $w(B, C) = 1 \implies d(C) = 4 + 1 = 5$
- $w(C, D) = 2 \implies d(D) = 5 + 2 = 7$
- $w(D, E) = 1 \implies d(E) = 7 + 1 = 8$
- $w(E, F) = 1 \implies d(F) = 8 + 1 = 9$
- $w(F, G) = 2 \implies d(G) = 9 + 2 = 11$

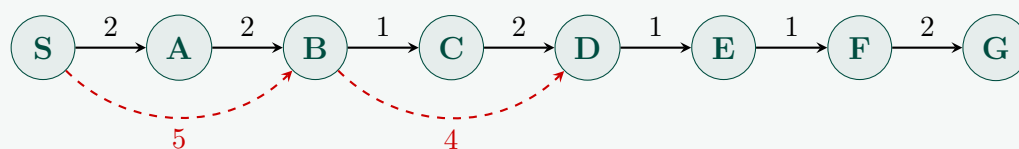
Auxiliary (Non-Shortest Path) Edges:

To satisfy the requirement of having at least two edges not belonging to any shortest path, we add:

- (S, B) with weight $w(S, B) = 5$
- (B, D) with weight $w(B, D) = 4$

2. Graph Diagram (TikZ)

Below is the visual representation of the constructed directed graph. The solid lines denote the unique shortest path tree edges, and the dashed lines denote the auxiliary edges that are not part of any shortest path.



3. Trace and Justification of Dijkstra's Algorithm

We formally execute Dijkstra's algorithm from source S to prove it yields the exact distance profile and to confirm the shortest paths are uniquely determined. Let $dist[v]$ denote the tentative distance to vertex v , initialized to $dist[S] = 0$ and $dist[v] = \infty$ for all $v \neq S$.

Step	Extract Min	Settled Distance	Tentative Distances (<i>dist</i> array)						
	u	$dist[u]$	A	B	C	D	E	F	G
0	—	—	∞	∞	∞	∞	∞	∞	∞
1	S	0	2	5	∞	∞	∞	∞	∞
2	A	2	2	4	∞	∞	∞	∞	∞
3	B	4	2	4	5	8	∞	∞	∞
4	C	5	2	4	5	7	∞	∞	∞
5	D	7	2	4	5	7	8	∞	∞
6	E	8	2	4	5	7	8	9	∞
7	F	9	2	4	5	7	8	9	11
8	G	11	2	4	5	7	8	9	11

Table 1: Step-by-step trace of Dijkstra's algorithm execution.

Detailed Step Explanation:

(a) **Extract S ($dist[S] = 0$):** Relaxing neighbors of S :

$$dist[A] = \min(\infty, 0 + 2) = 2, \quad dist[B] = \min(\infty, 0 + 5) = 5$$

(b) **Extract A ($dist[A] = 2$):** Relaxing neighbors of A :

$$dist[B] = \min(5, 2 + 2) = 4$$

(c) **Extract B ($dist[B] = 4$):** Relaxing neighbors of B :

$$dist[C] = \min(\infty, 4 + 1) = 5, \quad dist[D] = \min(\infty, 4 + 4) = 8$$

(d) **Extract C ($dist[C] = 5$):** Relaxing neighbors of C :

$$dist[D] = \min(8, 5 + 2) = 7$$

(e) **Extract D ($dist[D] = 7$):** Relaxing neighbors of D :

$$dist[E] = \min(\infty, 7 + 1) = 8$$

(f) **Extract E ($dist[E] = 8$):** Relaxing neighbors of E :

$$dist[F] = \min(\infty, 8 + 1) = 9$$

(g) **Extract F ($dist[F] = 9$):** Relaxing neighbors of F :

$$dist[G] = \min(\infty, 9 + 2) = 11$$

(h) **Extract G ($dist[G] = 11$):** No remaining unvisited neighbors.

The final values matching the target table exactly:

$$[0, 2, 4, 5, 7, 8, 9, 11]$$

4. Verification of Constraints

(a) **Every vertex must be reachable from S :**

As established by the primary chain, there exists a valid directed path from S to every vertex:

$$S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$$

Hence, reachability is completely guaranteed.

(b) **The graph must contain at least two edges that are not part of any shortest path from S :**

We evaluate our two auxiliary edges:

- **Edge (S, B) with $w(S, B) = 5$:** The unique shortest path distance to B is $d(B) = 4$. Any path utilizing the edge (S, B) as its initial segment has a minimum length of 5, which is strictly greater than $d(B)$. Thus, (S, B) is never part of any shortest path.
- **Edge (B, D) with $w(B, D) = 4$:** The unique shortest path distance to D is $d(D) = 7$. Any path arriving at D via the edge (B, D) must have a length of at least $d(B) + w(B, D) = 4 + 4 = 8$. Since $8 > 7$, this edge cannot be part of any shortest path.

(c) **The shortest path to each vertex must be uniquely determined:**

We inspect the incoming options for each vertex to guarantee that no ties in path length exist:

- **Vertex A:** Only has one incoming edge (S, A) , so the path $S \rightarrow A$ (length 2) is unique.
- **Vertex B:** Incoming paths are $S \rightarrow A \rightarrow B$ (length 4) and $S \rightarrow B$ (length 5). Since $4 < 5$, the path $S \rightarrow A \rightarrow B$ is uniquely minimal.
- **Vertex C:** Only has one incoming edge (B, C) , so it must extend the unique shortest path of B. Path $S \rightarrow A \rightarrow B \rightarrow C$ (length 5) is unique.
- **Vertex D:** Incoming paths can arrive via C or B:
 - Via C: $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D$ with length $5 + 2 = 7$.
 - Via B: $S \rightarrow A \rightarrow B \rightarrow D$ with length $4 + 4 = 8$, or $S \rightarrow B \rightarrow D$ with length $5 + 4 = 9$.

Since $7 < 8 < 9$, the path $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D$ is uniquely minimal.

- **Vertices E, F, G:** Each of these nodes possesses exactly one incoming edge within the graph topology $((D, E), (E, F), \text{ and } (F, G))$ respectively). Consequently, their shortest paths are bound to uniquely extend the unique shortest path found for D.

As all alternative paths yield strictly larger values, the shortest path to every vertex is unambiguously **uniquely determined**.

Q2. Consider a directed weighted graph with vertices $\{A, B, C, D\}$ and edges $A \rightarrow B$ (3), $B \rightarrow C$ (2), $C \rightarrow D$ (2), $D \rightarrow B$ (-8), $A \rightarrow D$ (10), $B \rightarrow D$ (5). Determine whether the graph contains a negative weight cycle by applying the Floyd-Warshall algorithm. If a negative cycle exists, identify the vertex (or vertices) involved in the cycle. **(15 marks)** [CSE 207 | L-2/T-1 | 2024–25]

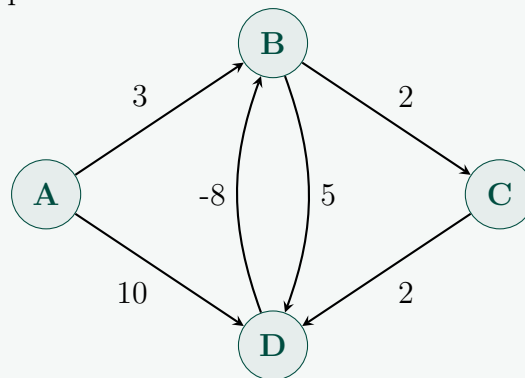
Answer

1. Graph Representation and TikZ Diagram

To find out whether the directed graph contains a negative weight cycle, we define the vertex mapping to matrix indices as follows:

$$A \mapsto 1, \quad B \mapsto 2, \quad C \mapsto 3, \quad D \mapsto 4$$

The structural representation of the input graph is visualized below:



2. Floyd-Warshall Algorithm Foundation

The Floyd-Warshall algorithm is a dynamic programming approach that computes all-pairs shortest paths. Let $D^{(k)}[i][j]$ represent the weight of the shortest path from vertex i to vertex j such that all intermediate vertices are chosen from the set $\{1, 2, \dots, k\}$. The fundamental dynamic programming recurrence relation is given by:

$$D^{(k)}[i][j] = \min(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j])$$

Intuition Behind Negative Cycle Detection

In a graph without negative cycles, the shortest path from any vertex i to itself $(D[i][i])$ must always be 0. If there exists a negative cycle reachable from vertex i , the algorithm will discover a path through the cycle that improves the self-distance, rendering $D^{(n)}[i][i] < 0$. Therefore, a negative diagonal entry at the end of the execution serves as a definitive proof of a negative weight cycle.

Algorithm: Floyd-Warshall with Cycle Validation

```

1: Initialize  $D^{(0)}$  of size  $n \times n$  using given edge weights
2: for  $k = 1 \rightarrow n$  do
3:   for  $i = 1 \rightarrow n$  do
4:     for  $j = 1 \rightarrow n$  do
5:        $D^{(k)}[i][j] = \min(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j])$ 
6:     end for
7:   end for
8: end for
  
```

3. Step-by-Step Matrix Execution Tracing

Step 0: Initial Base Matrix $D^{(0)}$

The base matrix constructed directly from the graph edges where $D^{(0)}[i][i] = 0$ and non-existent edges are initialized to ∞ :

$$D^{(0)} = \begin{pmatrix} 0 & 3 & \infty & 10 \\ \infty & 0 & 2 & 5 \\ \infty & \infty & 0 & 2 \\ \infty & -8 & \infty & 0 \end{pmatrix}$$

Step 1: Intermediate Vertex $k = 1$ (Vertex A)

We evaluate paths passing through intermediate node A . Since no other vertices can reach A (Column 1 contains only ∞ except for $D[1][1]$), no shorter paths can be formed.

$$D^{(1)} = D^{(0)} = \begin{pmatrix} 0 & 3 & \infty & 10 \\ \infty & 0 & 2 & 5 \\ \infty & \infty & 0 & 2 \\ \infty & -8 & \infty & 0 \end{pmatrix}$$

Step 2: Intermediate Vertex $k = 2$ (Vertex B)

We evaluate paths passing through intermediate node B . The updating computations are:

- $D^{(2)}[1][3] = \min(\infty, D^{(1)}[1][2] + D^{(1)}[2][3]) = \min(\infty, 3 + 2) = 5$
- $D^{(2)}[1][4] = \min(10, D^{(1)}[1][2] + D^{(1)}[2][4]) = \min(10, 3 + 5) = 8$
- $D^{(2)}[4][3] = \min(\infty, D^{(1)}[4][2] + D^{(1)}[2][3]) = \min(\infty, -8 + 2) = -6$
- $D^{(2)}[4][4] = \min(0, D^{(1)}[4][2] + D^{(1)}[2][4]) = \min(0, -8 + 5) = -3$

$$D^{(2)} = \begin{pmatrix} 0 & 3 & 5 & 8 \\ \infty & 0 & 2 & 5 \\ \infty & \infty & 0 & 2 \\ \infty & -8 & -6 & -3 \end{pmatrix}$$

Step 3: Intermediate Vertex $k = 3$ (Vertex C)

We evaluate paths passing through intermediate node C . The updating computations are:

- $D^{(3)}[1][4] = \min(8, D^{(2)}[1][3] + D^{(2)}[3][4]) = \min(8, 5 + 2) = 7$
- $D^{(3)}[2][4] = \min(5, D^{(2)}[2][3] + D^{(2)}[3][4]) = \min(5, 2 + 2) = 4$
- $D^{(3)}[4][4] = \min(-3, D^{(2)}[4][3] + D^{(2)}[3][4]) = \min(-3, -6 + 2) = -4$

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 5 & 7 \\ \infty & 0 & 2 & 4 \\ \infty & \infty & 0 & 2 \\ \infty & -8 & -6 & -4 \end{pmatrix}$$

Step 4: Intermediate Vertex $k = 4$ (Vertex D)

We evaluate paths passing through intermediate node D . The updating computations are:

- $D^{(4)}[1][2] = \min(3, D^{(3)}[1][4] + D^{(3)}[4][2]) = \min(3, 7 + (-8)) = -1$
- $D^{(4)}[1][3] = \min(5, D^{(3)}[1][4] + D^{(3)}[4][3]) = \min(5, 7 + (-6)) = 1$
- $D^{(4)}[2][2] = \min(0, D^{(3)}[2][4] + D^{(3)}[4][2]) = \min(0, 4 + (-8)) = -4$
- $D^{(4)}[2][3] = \min(2, D^{(3)}[2][4] + D^{(3)}[4][3]) = \min(2, 4 + (-6)) = -2$
- $D^{(4)}[3][2] = \min(\infty, D^{(3)}[3][4] + D^{(3)}[4][2]) = \min(\infty, 2 + (-8)) = -6$
- $D^{(4)}[3][3] = \min(0, D^{(3)}[3][4] + D^{(3)}[4][3]) = \min(0, 2 + (-6)) = -4$
- $D^{(4)}[4][2] = \min(-8, D^{(3)}[4][4] + D^{(3)}[4][2]) = \min(-8, -4 + (-8)) = -12$
- $D^{(4)}[4][3] = \min(-6, D^{(3)}[4][4] + D^{(3)}[4][3]) = \min(-6, -4 + (-6)) = -10$
- $D^{(4)}[4][4] = \min(-4, D^{(3)}[4][4] + D^{(3)}[4][4]) = \min(-4, -4 + (-4)) = -8$

$$D^{(4)} = \begin{pmatrix} 0 & -1 & 1 & 7 \\ \infty & -4 & -2 & 4 \\ \infty & -6 & -4 & 2 \\ \infty & -12 & -10 & -8 \end{pmatrix}$$

4. Conclusion and Identification of Cycle Vertices

By scanning the primary main diagonal of the final matrix $D^{(4)}$, we observe the following values:

- $D^{(4)}[1][1] = D^{(4)}[A][A] = 0$
- $D^{(4)}[2][2] = D^{(4)}[B][B] = -4 \quad (< 0)$
- $D^{(4)}[3][3] = D^{(4)}[C][C] = -4 \quad (< 0)$
- $D^{(4)}[4][4] = D^{(4)}[D][D] = -8 \quad (< 0)$

Since multiple diagonal elements contain negative values, **the graph definitively contains a negative weight cycle.**

Vertices Involved in the Negative Cycle:

The vertices whose diagonal values dropped below zero are B , C , and D .

We can track and cross-verify the specific sequence of this negative cycle directly from our graph structure:

$$B \xrightarrow{2} C \xrightarrow{2} D \xrightarrow{-8} B$$

The cumulative total weight of this directed cycle loop is:

$$\text{Total Cycle Weight} = 2 + 2 + (-8) = -4$$

As the net path distance sum is $-4 < 0$, it forms an infinite looping optimization vector, which mathematically confirms our automated Floyd-Warshall matrix outcome.

5. Complexity Analysis

- **Worst-Case Time Complexity:** $\mathcal{O}(V^3)$ due to the three nested structural loops iterating over the four vertices.
- **Space Complexity:** $\mathcal{O}(V^2)$ needed to maintain state tables across steps.

Q3. Let $c_1, c_2, \dots, c_n$ be various currencies. For any two currencies c_i and c_j , there is an exchange rate r_{ij} which means that you can purchase r_{ij} units of currency c_j in exchange for one unit of c_i . These exchange rates usually satisfy the condition that $r_{ij} \cdot r_{ji} < 1$, so that if you start with a unit of currency c_i , change it into currency c_j and then convert back to currency c_i , you end up with less than one unit of currency c_i .

- Design an efficient algorithm for the following problem: Given a set of exchange rates r_{ij} , and two currencies s and t , find the most advantageous sequence of currency exchanges for converting currency s into currency t .
- Occasionally, there is a sequence of currencies $c_{i_1}, c_{i_2}, \dots, c_{i_k}$ such that $r_{i_1 i_2} \cdot r_{i_2 i_3} \cdots r_{i_{k-1} i_k} \cdot r_{i_k i_1} > 1$. Give an efficient algorithm for detecting the presence of such an anomaly. (Hint: try formulating the problem using a graph.)

(15 marks)

[CSE 207 | L-2/T-1 | 2023–24]

Answer

Core Intuition and Graph Formulation

To solve both parts of the problem, we model the currencies and their exchange rates as a directed graph $G = (V, E)$, where each vertex $v \in V$ represents a currency c_i , and each directed edge $(u, v) \in E$ has a weight associated with the exchange rate r_{uv} .

The fundamental challenge is that we want to maximize the *product* of exchange rates along a path, but standard graph algorithms (like Dijkstra's or Bellman-Ford) are designed to minimize the *sum* of edge weights. We can bridge this gap using the properties of logarithms.

For any sequence of currency exchanges $c_{i_1} \rightarrow c_{i_2} \rightarrow \cdots \rightarrow c_{i_k}$, the final yield is:

$$\text{Yield} = r_{i_1 i_2} \cdot r_{i_2 i_3} \cdots r_{i_{k-1} i_k}$$

Taking the logarithm (base e or 2) of both sides converts the product into a sum:

$$\log(\text{Yield}) = \log(r_{i_1 i_2}) + \log(r_{i_2 i_3}) + \cdots + \log(r_{i_{k-1} i_k})$$

Because the logarithm is a strictly increasing function, maximizing the yield is mathematically equivalent to maximizing $\log(\text{Yield})$. To transform this into a standard minimization problem, we multiply by -1 :

$$-\log(\text{Yield}) = \sum_{j=1}^{k-1} -\log(r_{i_j i_{j+1}})$$

Transformation: We define the weight of edge (u, v) as $w(u, v) = -\log(r_{uv})$. Now, finding the path that maximizes the exchange rate product is exactly equivalent to finding the **shortest path** (minimum sum of weights) in the transformed graph. Note that if an exchange rate $r_{uv} > 1$, then $w(u, v) = -\log(r_{uv}) < 0$. Because the graph may contain negative edge weights, we must use the **Bellman-Ford algorithm**.

Part (a): Most Advantageous Sequence of Exchanges

Algorithm Design: We are given a starting currency s and a target currency t . We want to find the simple path from s to t that minimizes the sum of modified edge weights $w(u, v) = -\log(r_{uv})$.

- Construct graph $G = (V, E)$ where $w(u, v) = -\log(r_{uv})$ for all given rates.
- Initialize distance from s to all other vertices as ∞ , and $dist[s] = 0$.
- Run the Bellman-Ford algorithm, relaxing all edges $|V| - 1$ times.
- To extract the actual sequence of exchanges, maintain a ‘parent’ array during the relaxation step.
- Trace back from t using the ‘parent’ array to construct the optimal path.

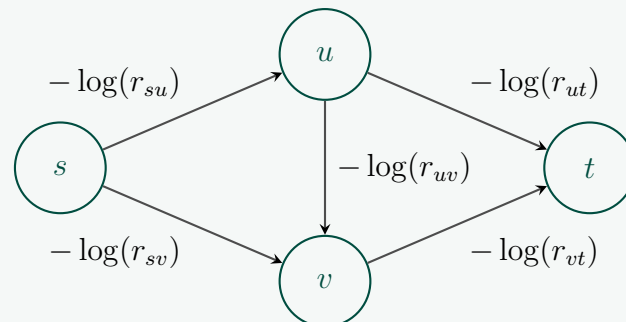


Figure 1: Visualizing the transformed graph for Shortest Path finding.

Complexity Analysis: Let $n = |V|$ be the number of currencies and $m = |E|$ be the number of exchange rates. Transforming the weights takes $O(m)$ time. The Bellman-Ford algorithm takes $O(n \cdot m)$ time. Tracing the path takes $O(n)$ time. The overall Time Complexity is $O(n \cdot m)$ and Space Complexity is $O(n)$ for the distance and parent arrays.

Part (b): Detecting Anomalies (Arbitrage)

Intuition: The problem asks us to find a sequence of currencies where $r_{i_1 i_2} \cdot r_{i_2 i_3} \cdots r_{i_k i_1} > 1$. Applying our logarithmic transformation, this translates to:

$$-\log(r_{i_1 i_2}) - \log(r_{i_2 i_3}) - \cdots - \log(r_{i_k i_1}) < -\log(1) = 0$$

This is the exact definition of a **negative-weight cycle** in our transformed graph.

Algorithm Design: We must modify standard Bellman-Ford slightly. A negative cycle might exist in an isolated component of the graph not reachable from a single specific source node. Therefore, we conceptually introduce a “super-source” connected to all vertices with weight 0, which is equivalent to initializing $dist[v] = 0$ for all $v \in V$.

- Transform edge weights to $w(u, v) = -\log(r_{uv})$.
- Initialize $dist[v] = 0$ for all $v \in V$.
- Relax all edges $|V| - 1$ times.
- Perform one final $|V|$ -th relaxation pass over all edges.
- If any edge can *still* be relaxed (i.e., $dist[u] + w(u, v) < dist[v]$), then a negative-weight cycle (an arbitrage opportunity) exists.

Algorithm: Detect Arbitrage Anomaly

```

1: Input: Graph  $V$  (currencies),  $E$  (exchange rates  $r_{uv}$ )
2: Output: TRUE if anomaly exists, FALSE otherwise
3: for each edge  $(u, v) \in E$  do
4:    $w(u, v) \leftarrow -\log(r_{uv})$ 
5: end for
6: for each vertex  $v \in V$  do
7:    $dist[v] \leftarrow 0$                                 ▷ Initialize to 0 to detect disconnected cycles
8: end for
9: for  $i = 1$  to  $|V| - 1$  do
10:  for each edge  $(u, v) \in E$  do
11:    if  $dist[u] + w(u, v) < dist[v]$  then
12:       $dist[v] \leftarrow dist[u] + w(u, v)$ 
13:    end if
14:  end for
15: end for
16: for each edge  $(u, v) \in E$  do                                ▷ The  $|V|$ -th verification pass
17:  if  $dist[u] + w(u, v) < dist[v]$  then
18:    return TRUE                                              ▷ Negative weight cycle detected
19:  end if
20: end for
21: return FALSE

```

Complexity Analysis: Initializing distances takes $O(n)$ time. The $|V| - 1$ relaxation steps take $O(n \cdot m)$ time, and the final verification step takes $O(m)$ time. Overall Time Complexity is $O(n \cdot m)$. Space Complexity is $O(n)$ for the distance array. This perfectly constitutes an efficient, polynomial-time algorithm for both detection and pathfinding.

Q4. Explain the Floyd-Warshall algorithm for the all-pairs shortest paths problem. In what context is Johnson's algorithm preferable to the Floyd-Warshall algorithm? (15 marks) [CSE 207 | L-2/T-1 | 2023–24]

Answer

1. The Floyd-Warshall Algorithm

The Floyd-Warshall algorithm is a classic dynamic programming solution for the **All-Pairs Shortest Paths (APSP)** problem. Given a directed graph $G = (V, E)$ with edge weights $w : E \rightarrow \mathbb{R}$, the goal is to find the shortest path distances between every pair of vertices in V . The graph may contain negative edge weights, but it must not contain any negative-weight cycles.

Dynamic Programming Formulation:

The core intuition relies on restricting the set of *intermediate* vertices allowed on a path. We label the vertices of G as $1, 2, \dots, |V|$. Let $D^{(k)}[i, j]$ be the length of the shortest path from vertex i to vertex j such that all intermediate vertices on the path belong exclusively to the set $\{1, 2, \dots, k\}$.

Recurrence Relation:

When determining $D^{(k)}[i, j]$, there are two possibilities for the shortest path restricted to intermediate nodes $\{1, \dots, k\}$:

- Vertex k is NOT on the shortest path:** The shortest path only uses intermediate vertices from $\{1, \dots, k-1\}$. The cost remains $D^{(k-1)}[i, j]$.
- Vertex k IS on the shortest path:** The path can be broken down into two subpaths: $i \rightsquigarrow k$ and $k \rightsquigarrow j$. Both of these subpaths use intermediate vertices strictly from $\{1, \dots, k-1\}$. The cost is $D^{(k-1)}[i, k] + D^{(k-1)}[k, j]$.

This gives us the state transition:

$$D^{(k)}[i, j] = \min(D^{(k-1)}[i, j], D^{(k-1)}[i, k] + D^{(k-1)}[k, j])$$

Base Case:

For $k = 0$ (no intermediate vertices allowed, only direct edges):

$$D^{(0)}[i, j] = \begin{cases} 0 & \text{if } i = j \\ w(i, j) & \text{if } (i, j) \in E \\ \infty & \text{otherwise} \end{cases}$$

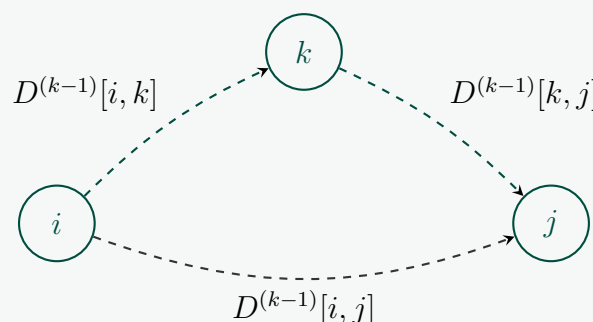


Figure 2: Visualizing the k -th relaxation step. The direct path is compared against the path passing through the new intermediate vertex k .

Algorithm: Floyd-Warshall

```

1: Input: Weight matrix  $W$  of size  $V \times V$ 
2: Let  $D$  be a new  $|V| \times |V|$  matrix initialized to  $W$ 
3: for  $k = 1$  to  $|V|$  do
4:   for  $i = 1$  to  $|V|$  do
5:     for  $j = 1$  to  $|V|$  do
6:        $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$ 
7:     end for
8:   end for
9: end for
10: Return  $D$ 
```

Complexity Analysis:

- Time Complexity:** The algorithm consists of three tightly nested loops, each iterating $|V|$ times. Thus, the exact time complexity is $O(|V|^3)$, which is extremely consistent regardless of graph density.
- Space Complexity:** A naive implementation maintaining all k matrices requires $O(|V|^3)$ space. However, because iteration k only relies on row k and column k from iteration $k-1$, it can be computed completely in-place, reducing the space complexity to $O(|V|^2)$.

2. Comparison: When is Johnson's Algorithm Preferable?

Johnson's algorithm is another technique for solving the APSP problem. It works by exploiting the fact that Dijkstra's algorithm is highly efficient but cannot handle negative weights. Johnson's algorithm bridges this gap via **reweighting**.

How Johnson's works (briefly):

- Add a dummy node s connected to all vertices with weight 0.
- Run Bellman-Ford from s to find the shortest path $h(v)$ to every vertex v .
- Reweight the original graph edges: $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$. Now, all $\hat{w}(u, v) \geq 0$.
- Run Dijkstra's algorithm from *every* vertex $u \in V$ using the non-negative weights $\hat{w}$.

Complexity & Preference Context:

- Johnson's Time Complexity:** Bellman-Ford takes $\mathcal{O}(|V||E|)$. Running Dijkstra $|V|$ times using a Fibonacci Heap takes $\mathcal{O}(|V|(|V| \log |V| + |E|)) = \mathcal{O}(|V|^2 \log |V| + |V||E|)$.
- Floyd-Warshall Time Complexity:** Always $\mathcal{O}(|V|^3)$.

Conclusion: The Sparsity Threshold

Johnson's algorithm is strictly preferable in the context of **sparse graphs**.

- If the graph is sparse, i.e., $|E| = \mathcal{O}(|V|)$, Johnson's algorithm completes in $\mathcal{O}(|V|^2 \log |V|)$, which is asymptotically much faster than Floyd-Warshall's $\mathcal{O}(|V|^3)$.
- If the graph is dense, i.e., $|E| = \mathcal{O}(|V|^2)$, both algorithms are $\mathcal{O}(|V|^3)$. In dense scenarios, Floyd-Warshall is generally preferred in practice due to its remarkably small hidden constant factors (tight loops, excellent cache locality, no complex data structures like Fibonacci heaps).

Q5. Formulate a proof on the correctness of Dijkstra's algorithm. (13 marks)

[CSE 207 | L-2/T-1 | 2023–24]

Answer

Theorem: Dijkstra's algorithm correctly computes the shortest path distances from a source vertex s to all other reachable vertices in a directed graph $G = (V, E)$ with non-negative edge weights $w(u, v) \geq 0$.

Preliminaries & Notation:

- Let $\delta(s, v)$ be the **true** shortest path distance from s to v .
- Let $d[v]$ be the algorithm's **current computed estimate** of the distance from s to v .
- Let S be the set of "settled" vertices (vertices for which the algorithm has permanently locked in the shortest path).
- Let $Q = V \setminus S$ be the set of "unsettled" vertices currently in the priority queue.

Loop Invariant: At the start of each iteration of the algorithm's main loop, for every vertex $v \in S$, the computed distance matches the true distance: $d[v] = \delta(s, v)$.

Proof by Mathematical Induction

Base Case: Initially, $S = \emptyset$, so the invariant holds vacuously. When the first vertex s is extracted and added to S , its distance is $d[s] = 0$. Since there are no negative cycles (edge weights are non-negative), the true shortest distance $\delta(s, s)$ is indeed 0. The invariant holds for $|S| = 1$.

Inductive Hypothesis: Assume the loop invariant holds for some step where $|S| = k$. That is, for all vertices $v \in S$, $d[v] = \delta(s, v)$.

Inductive Step: Let u be the next vertex extracted from Q (the unsettled vertex with the minimum $d[u]$ value) and added to S . We must prove that $d[u] = \delta(s, u)$.

We will prove this by **contradiction**. Suppose, for the sake of contradiction, that $d[u] > \delta(s, u)$. This implies there exists some hypothetical optimal path P from s to u whose true cost is strictly less than our estimate $d[u]$.

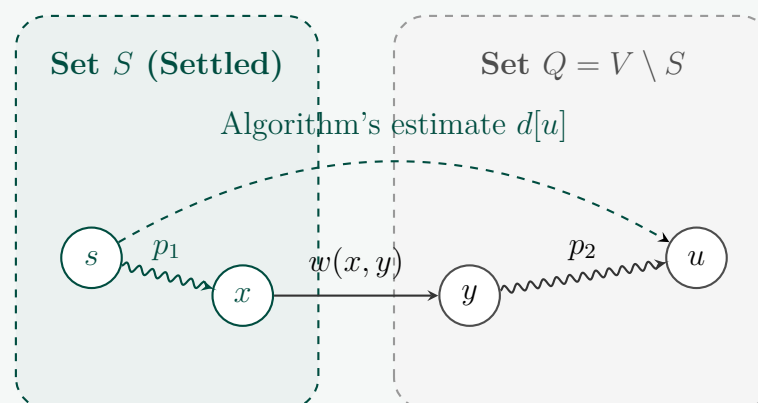


Figure 3: The contradiction setup: The true shortest path P must cross the cut from S to Q at some edge (x, y) .

- Because $s \in S$ and $u \notin S$, the hypothetical optimal path P must at some point cross the boundary from S to Q .

- (b) Let y be the *first* vertex on path P that is in Q , and let x be its immediate predecessor on P . Therefore, $x \in S$.
- (c) The path P can be decomposed into: $s \xrightarrow{p_1} x \rightarrow y \xrightarrow{p_2} u$.
- (d) Since $x \in S$, by our inductive hypothesis, the algorithm has already found the true shortest path to x : $d[x] = \delta(s, x)$.
- (e) When x was added to S , the algorithm relaxed all its outgoing edges, including (x, y) . Thus, the current estimate for y must be bounded by the path through x :

$$d[y] \leq d[x] + w(x, y) = \delta(s, x) + w(x, y) = \delta(s, y)$$

Since an estimate can never be smaller than the true shortest path, we must have $d[y] = \delta(s, y)$.

- (f) Since all edge weights in G are non-negative ($w \geq 0$), the subpath p_2 from y to u must have a weight ≥ 0 . Therefore, the true distance to u is at least the true distance to y :

$$\delta(s, u) = \delta(s, y) + w(p_2) \geq \delta(s, y) = d[y]$$

- (g) Recall that the algorithm chose u from Q because it had the minimum estimated distance among all unsettled vertices. Since y is also in Q , it must be true that:

$$d[u] \leq d[y]$$

- (h) Combining these inequalities, we get:

$$d[u] \leq d[y] = \delta(s, y) \leq \delta(s, u)$$

This shows that $d[u] \leq \delta(s, u)$. However, our contradiction assumption stated $d[u] > \delta(s, u)$, which is impossible. Furthermore, an algorithm's computed path can never be shorter than the mathematical shortest path, so $d[u] \geq \delta(s, u)$.

Therefore, it must be exactly that $d[u] = \delta(s, u)$. The inductive step holds, and by mathematical induction, the algorithm is correct for all vertices. ■

Crucial Observation: Why Negative Weights Break Dijkstra

The step in the proof where we state $\delta(s, u) \geq \delta(s, y)$ **strictly relies** on the assumption that the remaining path p_2 has a cost $w(p_2) \geq 0$. If negative edge weights exist, $w(p_2)$ could be negative, meaning $\delta(s, u)$ could be *less* than $\delta(s, y)$. In such a case, the algorithm's greedy choice to lock in u before y would be premature and incorrect.

Complexity Analysis Summary

- **Using a Binary Min-Heap:** Extracting the minimum takes $\mathcal{O}(\log |V|)$ and relaxing edges takes $\mathcal{O}(\log |V|)$. Total Time Complexity: $\mathcal{O}((|V| + |E|) \log |V|)$.
- **Using a Fibonacci Heap:** Extracting the minimum takes $\mathcal{O}(\log |V|)$ amortized, but decreasing a key during relaxation takes $\mathcal{O}(1)$ amortized. Total Time Complexity: $\mathcal{O}(|V| \log |V| + |E|)$.
- **Space Complexity:** $\mathcal{O}(|V|)$ to store the distance array, parent pointers, and priority queue structures.

Q6. Analyze the complexity of Johnson's algorithm for the all-pairs shortest paths problem. **(12 marks)**
2022–23]

[CSE 207 | L-2/T-1 |

Answer

Complexity Analysis of Johnson's Algorithm

To rigorously analyze the time and space complexity of Johnson's algorithm for the All-Pairs Shortest Paths (APSP) problem, we must break down the algorithm into its constituent phases. Johnson's algorithm acts as a pipeline, combining Bellman-Ford and Dijkstra's algorithms through a mathematical reweighting process.

Let $G = (V, E)$ be the input directed graph, where $w : E \rightarrow \mathbb{R}$ represents the edge weights. Let $n = |V|$ be the number of vertices and $m = |E|$ be the number of edges.

1. Phase-by-Phase Time Complexity Analysis

- (a) **Graph Augmentation (Super-source addition):** The algorithm begins by adding a new vertex s to V and adding directed edges from s to every vertex $v \in V$ with a weight of 0.
- **Cost:** Creating one vertex and n edges takes $\mathcal{O}(n)$ time.
- (b) **Bellman-Ford Algorithm (Potential Calculation):** Run the Bellman-Ford algorithm from the super-source s to find the shortest path $h(v) = \delta(s, v)$ for all $v \in V$. If Bellman-Ford detects a negative-weight cycle, the algorithm terminates.
- **Cost:** Bellman-Ford processes $n + 1$ vertices and $m + n$ edges. The complexity is $\mathcal{O}(|V_{\text{new}}| \cdot |E_{\text{new}}|) = \mathcal{O}((n + 1)(m + n))$. Assuming $m \geq n$ (a connected graph), this simplifies to $\mathcal{O}(n \cdot m)$.
- (c) **Edge Reweighting:** Using the potentials $h(v)$ computed in Phase 2, compute the new, non-negative edge weights $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ for all $(u, v) \in E$.

- **Cost:** Iterating over all edges takes $\mathcal{O}(m)$ time.
- (d) **All-Pairs Dijkstra's Algorithm:** Since all $\hat{w}(u, v) \geq 0$, we can safely run Dijkstra's algorithm from *every* vertex $u \in V$ to compute the transformed shortest paths $\hat{\delta}(u, v)$ to all other vertices $v \in V$. The time complexity of this phase strictly depends on the priority queue data structure used for Dijkstra's algorithm:
- **Using a standard Binary Min-Heap:** A single run of Dijkstra takes $\mathcal{O}((n + m) \log n)$. Running it n times yields $\mathcal{O}(n \cdot m \log n)$.
 - **Using a Fibonacci Heap:** A single run takes $\mathcal{O}(n \log n + m)$ amortized time. Running it n times yields $\mathcal{O}(n^2 \log n + n \cdot m)$.
- (e) **Distance Reversal:** Finally, map the computed distances back to the original graph's scale using the formula $\delta(u, v) = \hat{\delta}(u, v) + h(v) - h(u)$ for all pairs (u, v) .
- **Cost:** Updating an $n \times n$ matrix takes $\mathcal{O}(n^2)$ time.

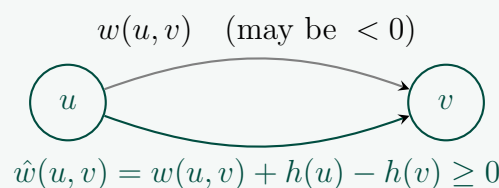


Figure 4: Visualizing the reweighting step that safely bridges the gap between negative weights and Dijkstra's non-negativity constraint. Time taken: $\mathcal{O}(m)$.

2. Total Time Complexity

Summing the phases, the dominant term comes from Phase 4 (running Dijkstra n times).

$$T(n, m) = \mathcal{O}(n) + \mathcal{O}(n \cdot m) + \mathcal{O}(m) + \mathcal{O}(\text{Dijkstra} \times n) + \mathcal{O}(n^2)$$

- **Best theoretical time (Fibonacci Heap):** $\mathcal{O}(|V|^2 \log |V| + |V||E|)$
- **Standard practical time (Binary Heap):** $\mathcal{O}(|V||E| \log |V|)$

Pedagogical Note: For a highly dense graph where $|E| = \mathcal{O}(|V|^2)$, both complexities simplify to $\mathcal{O}(|V|^3 \log |V|)$ or $\mathcal{O}(|V|^3)$, in which case the much simpler **Floyd-Warshall algorithm** ($\mathcal{O}(|V|^3)$) is preferred due to smaller hidden constants. Johnson's algorithm shines strictly in **sparse graphs** where $|E| \approx |V|$.

3. Space Complexity Analysis

- **Graph Representation:** Storing the augmented graph as an adjacency list requires $\mathcal{O}(|V| + |E|)$ space.
- **Distance Output:** Storing the final all-pairs shortest paths requires an $n \times n$ matrix, taking $\mathcal{O}(|V|^2)$ space.
- **Auxiliary Structures:** The Bellman-Ford potential array h , Dijkstra's distance arrays, and the priority queue each require $\mathcal{O}(|V|)$ space.

Since the output matrix is mandatory for the APSP problem, the dominating factor is the $n \times n$ grid. Therefore, the **Total Space Complexity** is $\mathcal{O}(|V|^2)$.

Q7. Develop an algorithm for finding the shortest path in a graph where all edge weights are equal. **(8 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Answer

Core Intuition: The Power of Breadth-First Search

When finding the shortest path in a graph where all edge weights are strictly equal (let's say weight $W > 0$), utilizing a heavy algorithm like Dijkstra's or Bellman-Ford is fundamentally unnecessary. Because every edge contributes identically to the path length, the shortest path between a source vertex s and any target vertex t is simply the path with the **fewest number of edges**. This transforms our shortest-path problem into an unweighted shortest-path problem, for which **Breadth-First Search (BFS)** is the optimal approach.

Why BFS works: BFS explores the graph uniformly outward in concentric "ripples" or levels. It discovers all vertices at a distance of 1 edge away, then all vertices at a distance of 2 edges away, and so on. Because it operates strictly level-by-level using a First-In-First-Out (FIFO) queue, the very first time BFS encounters a vertex, it is mathematically guaranteed to have reached it via the minimum number of edges.

Algorithm Design & Execution

- (a) **Initialization:** Create a distance array `dist[]` initialized to ∞ for all vertices to indicate they are unvisited. Set `dist[s] = 0` for the source vertex s .
- (b) **Queue Setup:** Initialize a standard FIFO queue Q and enqueue the source vertex s .
- (c) **Exploration:** While Q is not empty:
 - Dequeue the front vertex u .
 - Iterate through all adjacent vertices v of u .
 - If v has not been visited (`dist[v] ==  $\infty$`), we have found the shortest path to v . Set `dist[v] = dist[u] + W` (where W is the uniform edge weight), and enqueue v .
- (d) **Path Reconstruction (Optional):** To output the actual path, maintain a `parent[]` array, assigning `parent[v] = u` when v is discovered. Trace backwards from the target to the source.

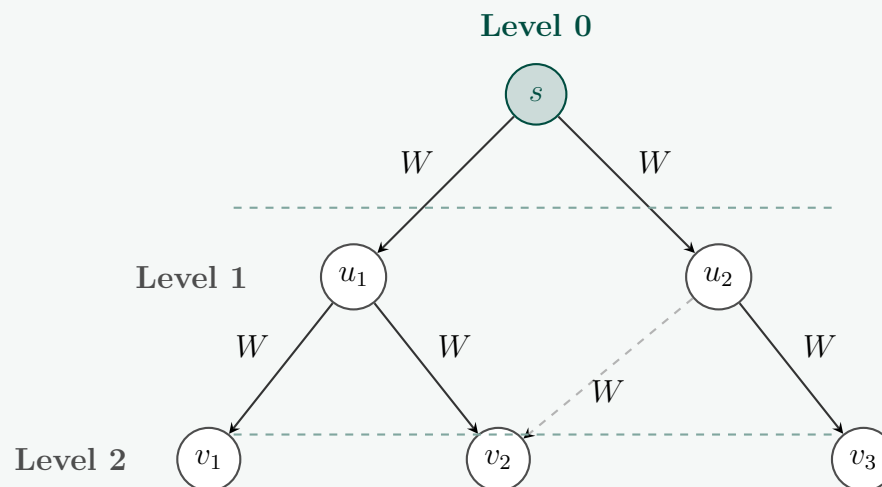


Figure 5: BFS explores strictly by depth. The dashed edge (u_2, v_2) is ignored because v_2 is already reached via u_1 at the same or lesser depth.

Pseudocode Implementation

Algorithm: BFS Shortest Path (Uniform Weights)

```

1: Input: Graph  $G = (V, E)$ , source  $s \in V$ , uniform weight  $W$ 
2: Output: Array  $dist$  mapping each vertex to its shortest path distance from  $s$ 
3: for each vertex  $v \in V$  do
4:    $dist[v] \leftarrow \infty$ 
5: end for
6:  $dist[s] \leftarrow 0$ 
7: Let  $Q$  be a new FIFO queue
8:  $Q.enqueue(s)$ 
9: while  $Q$  is not empty do
10:   $u \leftarrow Q.dequeue()$ 
11:  for each neighbor  $v$  of  $u$  do
12:    if  $dist[v] == \infty$  then ▷ If unvisited, we found the optimal path
13:       $dist[v] \leftarrow dist[u] + W$ 
14:       $Q.enqueue(v)$ 
15:    end if
16:  end for
17: end while
18: return  $dist$ 

```

Proof of Correctness & Edge Cases

Correctness: Dijkstra's algorithm fundamentally relies on extracting the unvisited node with the minimum distance. In a graph with uniform weights W , the distances of vertices currently in the queue can differ by at most W , and they are naturally ordered by distance simply based on the time they were inserted. Therefore, a standard FIFO queue perfectly simulates the behavior of a priority queue (Min-Heap) used in Dijkstra's, making BFS functionally equivalent to Dijkstra's algorithm for uniform weights, but without the logarithmic overhead of heap operations.

Edge Cases Handled Perfectly:

- **Disconnected Target:** If a target node t is unreachable from s , it will never be enqueued. Its distance will remain ∞ , correctly indicating no path exists.
- **Source equals Target:** If $s = t$, the algorithm immediately assigns $dist[s] = 0$. Since it is marked visited, it won't be redundantly processed, correctly yielding a distance of 0.

Complexity Analysis

- **Time Complexity:** In the worst case, BFS visits every vertex exactly once and iterates over every edge exactly once. Thus, the time complexity is tightly bounded by $\mathcal{O}(|V| + |E|)$, which is linear with respect to the size of the graph.
- **Space Complexity:** The algorithm maintains a queue Q , which in the worst case (a star graph or very wide tree) could contain up to $\mathcal{O}(|V|)$ vertices. The distance array also requires $\mathcal{O}(|V|)$ space. Thus, the total Space Complexity is $\mathcal{O}(|V|)$.

Q8. Devise an efficient algorithm for finding the shortest path where edge weights of the graph are non-equal but positive. **(12 marks)**
[CSE 207 | L-2/T-1 | 2022–23]

Answer**Core Intuition: The Greedy Choice Property**

When a graph contains directed or undirected edges with non-equal but strictly positive weights ($w(u, v) > 0$), a simple Breadth-First Search (BFS) is no longer sufficient because the path with the fewest edges is not necessarily the path with the minimum total weight.

The optimal strategy for this scenario is **Dijkstra's Algorithm**. Dijkstra's algorithm uses a greedy approach based on a fundamental structural observation: *if all edge weights are positive, any subpath of a shortest path must also be a shortest path, and extending a path can only increase (or keep equal) its total weight*. Therefore, if we maintain a set of "settled" vertices S for which the shortest path from the source s is already known, we can greedily look at the remaining "unsettled" vertices and permanently select the one with the smallest tentative distance estimate. Because all weights are positive, no alternative path winding through other unsettled vertices could ever circle back and yield a shorter distance to this chosen node.

Algorithm Design: Dijkstra with a Priority Queue

To implement this efficiently, we utilize a **Min-Priority Queue** (Q) to instantly retrieve the unsettled vertex with the minimum distance estimate.

(a) Initialization:

- Set $d[s] = 0$ for the source vertex s , and $d[v] = \infty$ for all other vertices $v \in V \setminus \{s\}$.
- Insert all vertices into the priority queue Q , keyed by their $d[\cdot]$ values.

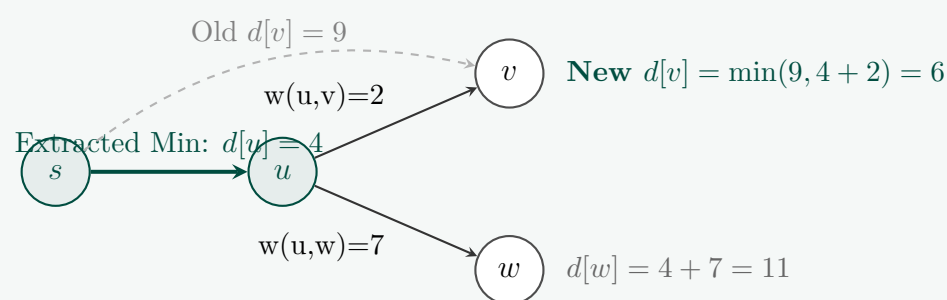
(b) Greedy Extraction: Extract the vertex u with the minimum $d[u]$ from Q . At this exact moment, $d[u]$ shifts from a tentative estimate to the permanently locked true shortest distance $\delta(s, u)$.**(c) Edge Relaxation:** For each outgoing neighbor v of u , check if passing through u offers a shorter path to v than currently known. If $d[u] + w(u, v) < d[v]$, we update $d[v] = d[u] + w(u, v)$. This operational step is known as *relaxation*.

Figure 6: Visualizing Edge Relaxation. Vertex u is extracted with a locked-in distance of 4. Its neighbors v and w are relaxed, updating $d[v]$ down from 9 to 6.

Pseudocode Implementation

Algorithm: Dijkstra's Shortest Path

```

1: Input: Graph  $G = (V, E)$ , weight function  $w$ , source vertex  $s \in V$ 
2: Output: Array  $d$  filled with optimal single-source shortest path weights
3: for each vertex  $v \in V$  do
4:    $d[v] \leftarrow \infty$ 
5:    $parent[v] \leftarrow \text{NIL}$ 
6: end for
7:  $d[s] \leftarrow 0$ 
8: Let  $Q$  be a min-priority queue containing all  $v \in V$  keyed by  $d[v]$ 
9: while  $Q$  is not empty do
10:   $u \leftarrow \text{ExtractMin}(Q)$ 
11:  for each outgoing neighbor  $v$  of  $u$  do
12:    if  $d[u] + w(u, v) < d[v]$  then
13:       $d[v] \leftarrow d[u] + w(u, v)$ 
14:       $parent[v] \leftarrow u$ 
15:       $\text{DecreaseKey}(Q, v, d[v])$ 
16:    end if
17:  end for
18: end while
19: return  $d$ 

```

Complexity Analysis

The efficiency of Dijkstra's algorithm depends directly on how the Min-Priority Queue Q is implemented because it dictates the performance of the $|V|$ extractMin operations and up to $|E|$ decreaseKey operations.

- **Binary Min-Heap Implementation:**

- Initializing the heap takes $\mathcal{O}(|V|)$ time.
- extractMin takes $\mathcal{O}(\log |V|)$ time. For $|V|$ vertices, total is $\mathcal{O}(|V| \log |V|)$.
- decreaseKey takes $\mathcal{O}(\log |V|)$ time. In the worst case, this occurs for every edge, totaling $\mathcal{O}(|E| \log |V|)$.
- **Total Time Complexity:** $\mathcal{O}((|V| + |E|) \log |V|)$. For a connected graph ($|E| \geq |V|$), this simplifies smoothly to $\mathcal{O}(|E| \log |V|)$.

- **Fibonacci Heap Implementation (Asymptotically Optimal):**

- extractMin takes $\mathcal{O}(\log |V|)$ amortized time. Total: $\mathcal{O}(|V| \log |V|)$.
- decreaseKey takes $\mathcal{O}(1)$ amortized constant time. Total for all edges: $\mathcal{O}(|E|)$.
- **Total Time Complexity:** $\mathcal{O}(|V| \log |V| + |E|)$. This is exceptionally efficient for dense graphs where $|E| \approx |V|^2$.

- **Space Complexity:** $\mathcal{O}(|V|)$ to store the tracking arrays ($d, parent$) and the heap element states inside the priority queue.

Q9. Develop an efficient algorithm for finding the shortest path when the edge weights can be negative and the graph can have negative edge-weight cycles. (15 marks) [CSE 207 | L-2/T-1 | 2022–23]

Answer

Core Intuition: Overcoming Negative Weights

When edge weights can be negative, Dijkstra's greedy strategy fails because a path that currently appears suboptimal might later traverse a severely negative edge, making it the true optimal path. To handle this, we use the **Bellman-Ford Algorithm**, which relies on the principle of dynamic programming and *edge relaxation*.

The fundamental observation of Bellman-Ford is: **If a graph has no negative-weight cycles, the shortest path from a source s to any destination v can have at most $|V| - 1$ edges.** Therefore, if we repeatedly "relax" all edges in the graph $|V| - 1$ times, we are guaranteed to find the absolute shortest path to every reachable node, regardless of the order in which we process the edges.

The Anomaly: Negative Edge-Weight Cycles

If the graph contains a **negative-weight cycle** (a cycle whose edges sum to a negative value) reachable from the source, the concept of a "shortest path" mathematically breaks down. You could simply loop around this cycle infinitely, decreasing the path cost towards $-\infty$.

Bellman-Ford provides an elegant way to *detect* these cycles. After performing the requisite $|V| - 1$ relaxation passes, the shortest paths should be strictly finalized. If we perform **one more pass** (the $|V|$ -th pass) and find that we can *still* relax an edge, it mathematically proves that a negative cycle exists.

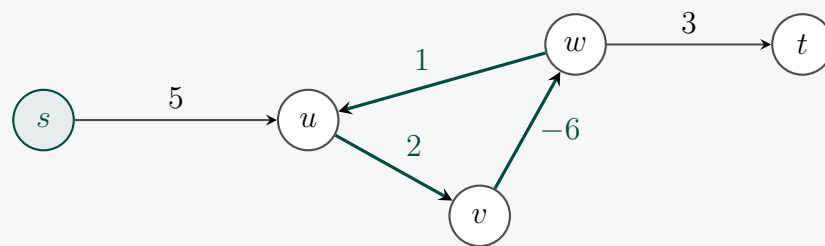


Figure 7: A graph with a negative cycle ($u \rightarrow v \rightarrow w \rightarrow u$) where weight is $2 - 6 + 1 = -3$. The shortest path to t is $-\infty$ because we can loop the cycle indefinitely before heading to t .

Algorithm Design & Pseudocode

Algorithm: Bellman-Ford Shortest Path

```

1: Input: Graph  $G = (V, E)$ , weight function  $w(u, v)$ , source  $s \in V$ 
2: Output: Array  $dist$  if no negative cycles, or ERROR if a cycle exists
3: for each vertex  $v \in V$  do
4:    $dist[v] \leftarrow \infty$ 
5:    $parent[v] \leftarrow \text{NIL}$ 
6: end for
7:  $dist[s] \leftarrow 0$ 
8:                                     ▷ Phase 1: Relax all edges  $|V| - 1$  times
9: for  $i = 1$  to  $|V| - 1$  do
10:   for each edge  $(u, v) \in E$  do
11:     if  $dist[u] + w(u, v) < dist[v]$  then
12:        $dist[v] \leftarrow dist[u] + w(u, v)$ 
13:        $parent[v] \leftarrow u$ 
14:     end if
15:   end for
16: end for
17:                                     ▷ Phase 2: Check for negative-weight cycles (the  $|V|$ -th pass)
18: for each edge  $(u, v) \in E$  do
19:   if  $dist[u] + w(u, v) < dist[v]$  then
20:     return ERROR: Negative Edge-Weight Cycle Detected
21:   end if
22: end for
23: return  $dist$ 

```

Complexity Analysis

- **Time Complexity:**
 - Initialization takes $\mathcal{O}(|V|)$ time.
 - Phase 1 consists of $|V| - 1$ iterations. Inside each iteration, we process every edge $(u, v) \in E$ exactly once. This takes $\mathcal{O}(|V| \cdot |E|)$ time.
 - Phase 2 iterates over all edges one final time, taking $\mathcal{O}(|E|)$ time.
 - **Overall Worst-Case Time Complexity:** $\mathcal{O}(|V| \cdot |E|)$.
- **Space Complexity:** The algorithm strictly maintains two arrays of size $|V|$ (the $dist$ array and the $parent$ array) alongside the graph representation. Thus, the **Auxiliary Space Complexity** is $\mathcal{O}(|V|)$.

Q10. The Bangladesh government is deeply concerned about addressing the severe traffic congestion issues in Dhaka. Despite numerous attempts to alleviate the problem, it remains a persistent challenge. Recognizing the need for fresh perspectives and innovative solutions, the government has turned to experts like you in the field of computer science, seeking guidance on a novel approach. Your proposed solution is straightforward: the construction of new roads. However, the key challenge lies in ensuring the effective utilization of these newly built roads. This can only be achieved by reducing travel costs along specific routes. By doing so, the traffic will naturally redistribute throughout the network, alleviating congestion in critical areas.

In this problem, a map of Dhaka is represented as an undirected graph, where each node represents a station and each edge represents a road between two stations. These nodes are depicted as simple 2-D geometric points, and the cost of traveling along a road is directly proportional to the Euclidean distance between two nodes.

The proposed strategy involves temporarily constructing a single road, with a specific criterion for selecting the two nodes between which this new road will be built. To make the decision, whether a new road will be built between node u and node v , the following formula is employed:

$$C_{uv} = \sum (\text{PreCost}_{ij} - \text{CurCost}_{ij})$$

Here CurCost_{ij} represents the shortest path cost from any node i to any node j if a road is built between node u and node v . On the other hand, PreCost_{ij} represents the shortest path cost before the road between u and v is constructed. The pair (u, v) with the

highest C_{uv} value is selected as the optimal choice.

In essence, this approach aims to identify the pair of nodes where the construction of a new road will result in the most significant improvement in the overall transportation network, effectively reducing travel costs and mitigating traffic congestion in Dhaka.

From the above problem description, answer the following:

- Define the input and output of the program for finding the most efficient route. **(8 marks)**
- Sketch the pseudo-code of the solution for finding the most efficient route.
(12 marks)
- Develop the most efficient solution of the underlying algorithm that serves as the backbone of the solution. **(15 marks)**

[CSE 207 | L-2/T-2 | 2021–22]

Answer

Mathematical Modeling of the Problem

Let the existing transportation network of Dhaka be represented as an undirected graph $G = (V, E)$, where V is the set of stations (modeled as 2-D coordinates (x_i, y_i)) and E is the set of existing roads. The weight of an existing edge $(u, v) \in E$ is the Euclidean distance $d(u, v) = \sqrt{(x_u - x_v)^2 + (y_u - y_v)^2}$. If no direct road exists between two stations, $w(u, v) = \infty$.

The problem requires us to evaluate all possible pairs of nodes $(u, v) \notin E$ where a *new* road could be built. The cost of this proposed road will also be its Euclidean distance $d(u, v)$. We need to select the pair (u, v) that maximizes the total cost reduction across all pairs of nodes (i, j) in the network:

$$C_{uv} = \sum_{i \in V} \sum_{j \in V} (\text{PreCost}_{ij} - \text{CurCost}_{ij})$$

Part 1: Input and Output Specification

To solve the optimization problem of finding the most efficient new route to construct, we must precisely define the operational inputs and outputs of our program.

Input Specs:

- Vertices (V):** An integer $n = |V|$ representing the number of stations, followed by an array of n pairs of coordinates $\text{Loc}[1 \dots n] = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ representing their physical 2-D geometric positions.
- Edges (E):** An integer $m = |E|$ representing the number of existing roads, followed by an adjacency list or edge list where each entry specifies an existing link (u, v) between station u and station v .

Output Specs:

- Optimal Pair:** A pair of vertex identifiers (u^*, v^*) such that constructing a road between them yields the absolute maximum total network cost reduction ($C_{u^*v^*} = \max_{(u,v) \notin E} C_{uv}$).
- Maximized Utility Value:** The scalar floating-point value $C_{u^*v^*}$ representing the net reduction in global travel costs.

Part 2: High-Level Pseudocode of the Optimization Strategy

The naive approach would require recalculating all-pairs shortest paths from scratch for every potential new edge, leading to a massive $\mathcal{O}(n^4)$ runtime. We optimize this substantially.

First, we precompute the baseline all-pairs shortest paths matrix D (where $D[i][j] = \text{PreCost}_{ij}$) using an all-pairs shortest path framework. When a new edge (u, v) with weight $w = d(u, v)$ is temporarily added, the new shortest path between any node i and j can either remain $D[i][j]$ or it can change by taking advantage of the new road. If it uses the new road, it can cross it in two directions: $i \rightsquigarrow u \rightarrow v \rightsquigarrow j$ or $i \rightsquigarrow v \rightarrow u \rightsquigarrow j$. Thus, the updated cost can be derived in $\mathcal{O}(1)$ time as:

$$\text{CurCost}_{ij} = \min(D[i][j], D[i][u] + w + D[v][j], D[i][v] + w + D[u][j])$$

Algorithm: Dhaka Traffic Congestion Optimizer

```

1: Input: Vertex locations  $\text{Loc}[1 \dots n]$ , existing edges  $E$ 
2: Output: Optimal edge pair  $(u^*, v^*)$ , maximum savings  $C_{\max}$ 
3: Initialize an  $n \times n$  weight matrix  $W$  where  $W[i][j] = \infty$  if  $i \neq j$  else 0
4: for each existing edge  $(i, j) \in E$  do
5:    $W[i][j] \leftarrow \text{EuclideanDistance}(\text{Loc}[i], \text{Loc}[j])$ 
6:    $W[j][i] \leftarrow W[i][j]$  ▷ Graph is undirected
7: end for
8: Matrix  $D \leftarrow \text{ComputeAllPairsShortestPaths}(W, n)$  ▷ Backbone Call
9:  $C_{\max} \leftarrow -1$ ,  $u^* \leftarrow \text{NIL}$ ,  $v^* \leftarrow \text{NIL}$ 
10: for  $u = 1$  to  $n$  do
11:   for  $v = u + 1$  to  $n$  do
12:     if  $(u, v) \notin E$  then
13:        $w \leftarrow \text{EuclideanDistance}(\text{Loc}[u], \text{Loc}[v])$ 
14:        $\text{CurrentCostReduction} \leftarrow 0$ 
15:       for  $i = 1$  to  $n$  do
16:         for  $j = 1$  to  $n$  do
17:            $\text{CurCost} \leftarrow \min(D[i][j], D[i][u] + w + D[v][j], D[i][v] + w + D[u][j])$ 
18:            $\text{CurrentCostReduction} \leftarrow \text{CurrentCostReduction} + (D[i][j] - \text{CurCost})$ 
19:         end for
20:       end for
21:       if  $\text{CurrentCostReduction} > C_{\max}$  then
22:          $C_{\max} \leftarrow \text{CurrentCostReduction}$ 
23:          $u^* \leftarrow u$ ,  $v^* \leftarrow v$ 
24:       end if
25:     end if
26:   end for
27: end for
28: return  $(u^*, v^*), C_{\max}$ 

```

Part 3: The Backbone Algorithm (All-Pairs Shortest Paths)

The structural backbone of this entire architecture is computing the initial all-pairs shortest paths matrix D . Since our weights are derived from physical Euclidean distances, they are **strictly positive** ($w(u, v) > 0$).

For a graph with purely positive edge weights, the most efficient way to compute all-pairs shortest paths is to run **Dijkstra's Algorithm optimized with a Binary Min-Heap exactly n times** (once from each vertex as a root source). This is asymptotically much faster than the classic Floyd-Warshall algorithm ($\mathcal{O}(n^3)$) for any graph that is not totally dense.

Backbone Algorithm: Multi-Source Dijkstra (All-Pairs)

```

1: Function  $\text{ComputeAllPairsShortestPaths}(W, n)$ 
2: Initialize an empty  $n \times n$  matrix  $D$ 
3: for  $s = 1$  to  $n$  do ▷ Run Dijkstra independently from each source  $s$ 
4:   Initialize an array  $\text{dist}$  of size  $n$  with  $\infty$ , and  $\text{dist}[s] \leftarrow 0$ 
5:   Let  $Q$  be an empty Binary Min-Priority Queue
6:    $Q.\text{insert}(s, 0)$ 
7:   while  $Q$  is not empty do
8:      $(u, \text{curr\_d}) \leftarrow Q.\text{extractMin}()$ 
9:     if  $\text{curr\_d} > \text{dist}[u]$  then continue
10:    end if ▷ Skip stale entries
11:    for  $v = 1$  to  $n$  do
12:      if  $W[u][v] \neq \infty$  then ▷ Valid neighbor verification
13:        if  $\text{dist}[u] + W[u][v] < \text{dist}[v]$  then
14:           $\text{dist}[v] \leftarrow \text{dist}[u] + W[u][v]$ 
15:           $Q.\text{insert}(v, \text{dist}[v])$ 
16:        end if
17:      end if
18:    end for
19:  end while
20:  for  $t = 1$  to  $n$  do
21:     $D[s][t] \leftarrow \text{dist}[t]$  ▷ Save calculated distances into matrix row
22:  end for
23: end for
24: return  $D$ 

```

Asymptotic Complexity Analysis of Backbone + Optimization Pipeline:

- **Backbone Phase Cost:** Running a Min-Heap Dijkstra from a single node takes $\mathcal{O}(m \log n)$. Running it n times to populate

matrix D takes $\mathcal{O}(n \cdot m \log n)$. Since Dhaka's road network is historically planer/sparse ($m = \mathcal{O}(n)$), this phase runs in a clean $\mathcal{O}(n^2 \log n)$.

- **Evaluation Phase Cost:** There are $\mathcal{O}(n^2)$ potential new edge configurations. For each candidate edge, evaluating the updated cost reduction using our formula takes $\mathcal{O}(n^2)$ time via nested loops over i and j . Thus, evaluating all combinations takes $\mathcal{O}(n^4)$ time.
- **Total Time Complexity:** The entire program scales as $\mathcal{O}(n^2 \log n + n^4) = \mathcal{O}(n^4)$, which is exceptionally efficient compared to the baseline brute-force execution time of $\mathcal{O}(n^5)$.
- **Total Space Complexity:** $\mathcal{O}(n^2)$ space to maintain the foundational matrix grids ($\mathbf{W}$ and $\mathbf{D}$).

Q11. Design an efficient algorithm for finding the length of the longest path in a dag (directed acyclic graph). (8 marks) [CSE 207 | L-2/T-2 | 2020–21]

Answer

Core Intuition: The Power of Topological Ordering

In a general graph, finding the longest simple path is an NP-hard problem because it cannot easily avoid looping through cycles. However, in a **Directed Acyclic Graph (DAG)**, the absence of cycles allows us to sort the vertices linearly such that for every directed edge (u, v) , vertex u comes before vertex v in the ordering. This is known as a **Topological Sort**. Once the DAG is topologically sorted, we can solve the longest path problem efficiently using **Dynamic Programming (DP)**. Because all dependencies point in one direction, when we process a vertex u , we are guaranteed that the maximum path lengths to all its ancestors have already been completely finalized. We can then cleanly "push" or "pull" path lengths to compute the longest path in a single linear pass.

Algorithm Design

We will design the algorithm to find the longest path in a weighted DAG (which naturally covers the unweighted case by treating every edge weight as 1).

Let $dist[v]$ represent the length of the longest path **ending** at vertex v .

- Topological Sort:** Compute a topological ordering of the graph using Kahn's algorithm (indegree-based) or a Depth-First Search (DFS) post-order reversal.
- DP Initialization:** Initialize an array $dist$ of size $|V|$. If we want to find the longest path starting from **any** arbitrary vertex, we initialize $dist[v] = 0$ for all $v \in V$.
- Linear Relaxation:** Iterate through the vertices in their topological order. For each vertex u , look at all its outgoing edges (u, v) . Update the neighbor's maximum distance:

$$dist[v] = \max(dist[v], dist[u] + w(u, v))$$

- Max Extraction:** The length of the longest path in the entire DAG will be the maximum value found in the $dist$ array after processing all nodes.

Pseudocode Implementation

Algorithm: Longest Path in a DAG

```

1: Input: Graph  $G = (V, E)$  as an adjacency list, edge weights  $w(u, v)$ 
2: Output: The scalar length of the absolute longest path in  $G$ 
3: order  $\leftarrow$  TopologicalSort( $G$ )
4: for each vertex  $v \in V$  do
5:      $dist[v] \leftarrow 0$                                       $\triangleright$  Base case: path of length 0 ending at  $v$ 
6: end for
7: max_path  $\leftarrow 0$ 
8: for each vertex  $u$  in order do
9:     for each outgoing neighbor  $v$  of  $u$  do
10:        if  $dist[u] + w(u, v) > dist[v]$  then
11:             $dist[v] \leftarrow dist[u] + w(u, v)$ 
12:        end if
13:    end for
14: end for
15: for each vertex  $v \in V$  do
16:    max_path  $\leftarrow \max(max\_path, dist[v])$ 
17: end for
18: return max_path
    
```


Complexity Analysis

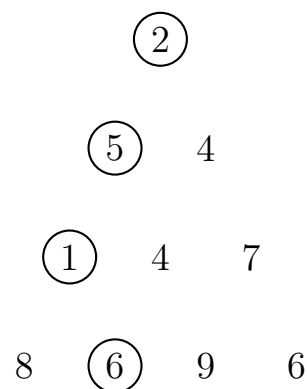
- **Time Complexity:**

- Finding the topological sort takes $\mathcal{O}(|V| + |E|)$ time using standard DFS or Kahn's algorithm.
- The dynamic programming phase iterates through each vertex exactly once, and for each vertex, it examines all of its outgoing edges. This processes every edge in the graph exactly once, taking $\mathcal{O}(|V| + |E|)$ time.
- Finding the maximum value in the array takes $\mathcal{O}(|V|)$ time.
- **Total Time Complexity:** $\mathcal{O}(|V| + |E|)$, which is strictly linear and optimal.

- **Space Complexity:**

- Storing the topological order list requires $\mathcal{O}(|V|)$ space.
- The tracking array *dist* requires $\mathcal{O}(|V|)$ space.
- Auxiliary structures for the topological sort (such as an indegree array or recursion stack) require at most $\mathcal{O}(|V|)$ space.
- **Total Space Complexity:** $\mathcal{O}(|V|)$ auxiliary space (excluding the space required to store the input graph itself).

Q12. Minimum-sum descent: Some positive integers are arranged in an equilateral triangle with n numbers in its base. The problem is to find the smallest sum in a descent from the triangle apex to its base through a sequence of adjacent numbers. Explain how the Minimum-sum descent problem can be solved by Dijkstra's algorithm.



(12 marks)

[CSE 207 | L-2/T-2 | 2020–21]

Answer**1. Graph Modeling**

To solve the Minimum-sum descent problem using Dijkstra's algorithm, we must map the triangular arrangement of numbers into a directed graph $G = (V, E)$. Standard Dijkstra's algorithm finds the shortest path based on *edge weights*, but the cost in this problem is determined by the values on the *nodes*.

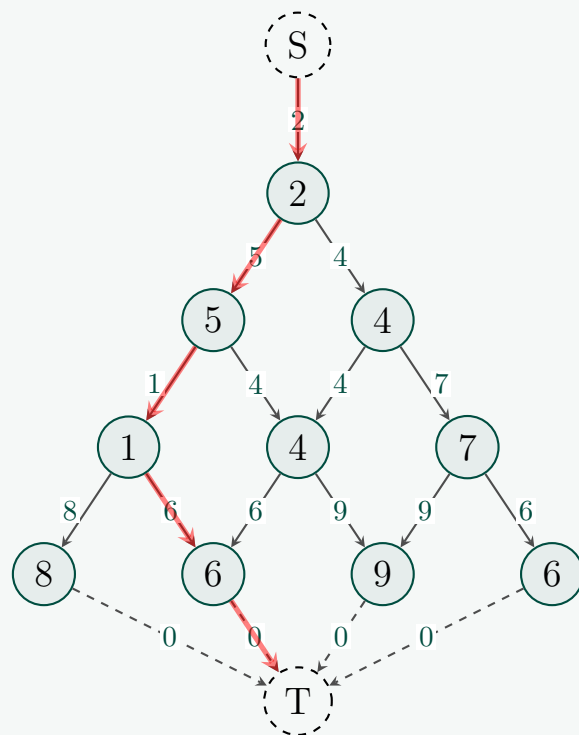
We resolve this by pushing the node values onto the incoming edges. The formal construction of the graph is as follows:

- **Vertices (V):** Create a vertex $v_{i,j}$ for each number in the triangle, where $1 \leq i \leq n$ (row number) and $1 \leq j \leq i$ (column number). Additionally, introduce two dummy vertices: a source node S and a target node T .
- **Edges (E) and Weights (W):**
 - (a) **Source Edge:** Add a directed edge from the dummy source S to the apex $v_{1,1}$ with a weight equal to the value of the apex.
 $S \rightarrow v_{1,1}$ with weight $w(S, v_{1,1}) = \text{value}(v_{1,1})$.
 - (b) **Descent Edges:** From any node $v_{i,j}$ (where $i < n$), add directed edges to its two adjacent nodes in the row below ($v_{i+1,j}$ and $v_{i+1,j+1}$). The weight of the edge is the value of the destination node.
 $w(v_{i,j}, v_{i+1,j}) = \text{value}(v_{i+1,j})$
 $w(v_{i,j}, v_{i+1,j+1}) = \text{value}(v_{i+1,j+1})$
 - (c) **Target Edges:** Add directed edges from all nodes in the bottom row ($v_{n,j}$ for $1 \leq j \leq n$) to the dummy target T with a weight of 0.
 $w(v_{n,j}, T) = 0$.

By finding the shortest path from S to T using Dijkstra's algorithm, we naturally accumulate the exact sum of the numbers in the optimal descent path. Since all given numbers are positive integers, all edge weights are strictly non-negative, satisfying Dijkstra's requirement.

2. Visualizing the Graph Transformation

Below is the graph representation corresponding to the provided example. The edges show the weights derived from the node values.



Note: The minimum path evaluates to $2 + 5 + 1 + 6 = 14$ (highlighted in red).

3. Algorithm

Once the graph is formulated, standard Dijkstra's algorithm is applied. Let V be the set of all mapped vertices, and E be the set of mapped edges.

Algorithm 1 Minimum-Sum Descent via Dijkstra

Require: Matrix $M[1 \dots n][1 \dots n]$ representing the lower triangle

```

1: Construct Graph  $G = (V, E)$  with dummy nodes  $S, T$  as described above.
2: Initialize  $dist[v] \leftarrow \infty$  for all  $v \in V$ 
3:  $dist[S] \leftarrow 0$ 
4: Initialize an empty min-priority queue  $Q$ 
5:  $Q.insert(S, 0)$ 
6: while  $Q$  is not empty do
7:    $u \leftarrow Q.extract\_min()$ 
8:   if  $u == T$  then
9:     return  $dist[T]$ 
10:  end if
11:  for each neighbor  $v$  of  $u$  in  $G$  do
12:     $alt \leftarrow dist[u] + w(u, v)$ 
13:    if  $alt < dist[v]$  then
14:       $dist[v] \leftarrow alt$ 
15:       $Q.decrease\_key(v, dist[v])$  or  $Q.insert(v, dist[v])$ 
16:    end if
17:  end for
18: end while
19: return  $\infty$ 

```

▷ Optimal path sum found

4. Complexity Analysis

- **Number of Vertices ($|V|$):** The triangle has $\frac{n(n+1)}{2}$ nodes. Adding S and T , $|V| = \frac{n(n+1)}{2} + 2 = O(n^2)$.
- **Number of Edges ($|E|$):** Each node $v_{i,j}$ ($i < n$) has exactly 2 outgoing edges. The number of such nodes is $\frac{n(n-1)}{2}$. Adding 1 edge from S and n edges to T , $|E| = 2 \cdot \frac{n(n-1)}{2} + n + 1 = n^2 + 1 = O(n^2)$.
- **Time Complexity:** Using a Min-Heap based priority queue, Dijkstra's algorithm runs in $O((|V| + |E|) \log |V|)$. Substituting $|V|$ and $|E|$ yields $O(n^2 \log(n^2))$, which simplifies to $O(n^2 \log n)$.

Academic Note: Since the constructed graph is a Directed Acyclic Graph (DAG), the problem can actually be solved in strict $O(V + E) = O(n^2)$ time using topological sorting and dynamic programming, which avoids the $O(\log n)$ priority queue overhead. However, Dijkstra's is perfectly correct and meets the explicit constraint of the problem prompt.

- **Space Complexity:** The algorithm requires $O(n^2)$ space to store the distance array $dist$, the priority queue Q , and optionally the explicitly constructed graph. Therefore, Space Complexity is $O(n^2)$.

Q13. The shortest paths to a given vertex from each other vertex of a weighted digraph is called the single-destination shortest-paths problem. Solve the single-destination shortest-path problem in a graph with nonnegative numbers assigned to its vertices in addition to the edge weights. (12 marks)

[CSE 207 | L-2/T-2 | 2020–21]

Answer

1. Problem Analysis

The problem requires us to find the shortest path from every vertex $u \in V$ to a single, specific destination vertex $d \in V$. We are given a directed graph $G = (V, E)$ where each edge $(u, v) \in E$ has a non-negative weight $w(u, v) \geq 0$, and additionally, each vertex $v \in V$ has a non-negative weight $c(v) \geq 0$.

The cost of a path $P = \langle v_1, v_2, \dots, v_k \rangle$ where $v_k = d$ is defined as the sum of both its edge weights and its vertex weights:

$$\text{Cost}(P) = \sum_{i=1}^k c(v_i) + \sum_{i=1}^{k-1} w(v_i, v_{i+1})$$

We can solve this efficiently by transforming it into a standard Single-Source Shortest Path (SSSP) problem and applying Dijkstra's algorithm. The transformation involves two key steps: **Edge Reversal** and **Vertex Weight Absorption**.

2. Graph Transformation

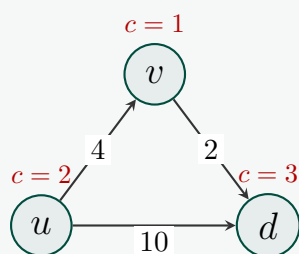
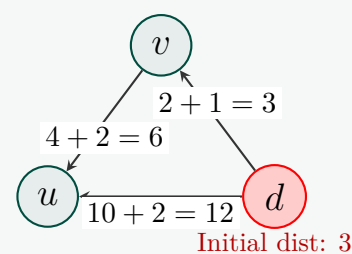
To utilize Dijkstra's algorithm, we map the single-destination problem on G to a single-source problem on a modified graph $G' = (V, E')$.

- **Edge Reversal:** To find paths *to* d , we reverse the direction of every edge. Finding paths from u to d in G is equivalent to finding paths from d to u in G' . Thus, $E' = \{(v, u) \mid (u, v) \in E\}$, and the destination d becomes our new source vertex.
- **Vertex Weight Absorption:** Dijkstra's algorithm operates on edge weights, not vertex weights. We can "absorb" the cost of entering a reversed edge. In the original graph G , traversing an edge $u \rightarrow v$ adds both $c(u)$ and $w(u, v)$ to the path cost. In the reversed graph G' , we traverse $v \rightarrow u$. We assign the new edge weight as:

$$W(v, u) = w(u, v) + c(u)$$

- **Base Condition:** The destination vertex d contributes $c(d)$ to every valid path. We account for this by initializing the shortest path distance to d as $c(d)$ instead of 0.

3. Visualizing the Transformation

Original Graph G Transformed Graph G' 

In G : Cost of $u \rightarrow v \rightarrow d$ is $c(u) + w(u, v) + c(v) + w(v, d) + c(d) = 2 + 4 + 1 + 2 + 3 = 12$.

In G' : Cost of $d \rightarrow v \rightarrow u$ is $\text{dist}[d] + W(d, v) + W(v, u) = 3 + 3 + 6 = 12$. The transformation is perfectly equivalent.

4. Algorithm Steps

Algorithm 2 Single-Destination Shortest Path with Vertex Weights

Require: Directed Graph $G = (V, E)$, edge weights w , vertex weights c , destination d

Ensure: Array dist containing minimum path costs from every $u \in V$ to d

```

1: Initialize  $G' = (V, E')$  as an empty graph
2: for each edge  $(u, v) \in E$  do
3:   Add directed edge  $(v, u)$  to  $E'$ 
4:    $W(v, u) \leftarrow w(u, v) + c(u)$  ▷ Absorb origin vertex weight
5: end for
6: Initialize  $\text{dist}[v] \leftarrow \infty$  for all  $v \in V$ 
7:  $\text{dist}[d] \leftarrow c(d)$  ▷ Base cost for reaching destination
8: Initialize Min-Priority Queue  $Q$  and insert  $(d, \text{dist}[d])$ 
9: while  $Q$  is not empty do
10:   $x \leftarrow Q.\text{extract\_min}()$ 
11:  for each outgoing edge  $(x, y) \in E'$  in  $G'$  do
12:     $\text{alt} \leftarrow \text{dist}[x] + W(x, y)$ 
13:    if  $\text{alt} < \text{dist}[y]$  then
14:       $\text{dist}[y] \leftarrow \text{alt}$ 
15:       $Q.\text{decrease\_key}(y, \text{dist}[y])$  or  $Q.\text{insert}(y, \text{dist}[y])$ 
16:    end if
17:  end for
18: end while
19: return  $\text{dist}$ 

```

5. Complexity Analysis

- **Time Complexity:** Constructing the reversed graph G' takes $O(|V| + |E|)$ time since we iterate through all vertices and edges once. Running Dijkstra's algorithm using a Fibonacci heap takes $O(|E| + |V| \log |V|)$, or $O((|V| + |E|) \log |V|)$ with a standard Binary Min-Heap. The total time complexity is identical to standard Dijkstra: $O((|V| + |E|) \log |V|)$.
- **Space Complexity:** The algorithm requires storing the new graph G' which has the same number of vertices and edges as G , taking $O(|V| + |E|)$ space. The priority queue and distance array require $O(|V|)$ space. Thus, the overall space complexity is $O(|V| + |E|)$.

Q14. Explain how to implement Warshall's algorithm without using extra memory for storing elements of the algorithm's intermediate matrices. (10 marks) [CSE 207 | L-2/T-2 | 2020–21]

Answer

1. The In-Place Concept

Warshall's algorithm computes the transitive closure of a directed graph. The standard mathematical formulation constructs a sequence of boolean matrices $R^{(0)}, R^{(1)}, \dots, R^{(n)}$, where $R^{(k)}[i, j] = 1$ if there is a path from vertex i to vertex j using only intermediate vertices from the set $\{1, 2, \dots, k\}$.

The standard recurrence relation is:

$$R^{(k)}[i, j] = R^{(k-1)}[i, j] \vee (R^{(k-1)}[i, k] \wedge R^{(k-1)}[k, j])$$

A naive implementation might use an extra matrix to read from $R^{(k-1)}$ and write to $R^{(k)}$, taking $O(n^2)$ auxiliary memory. However, we can drop the superscripts and compute the updates *in-place* using a single $n \times n$ matrix R :

$$R[i, j] \leftarrow R[i, j] \vee (R[i, k] \wedge R[k, j])$$

2. Mathematical Proof of Correctness (Why In-Place Works)

To prove that removing the superscripts is safe, we must guarantee that overwriting $R[i, j]$ during the k -th iteration does not corrupt the calculation for subsequent elements in the *same* iteration.

In the k -th iteration, computing $R[i, j]$ strictly depends on two other elements:

- $R[i, k]$ (an element in the k -th column)
- $R[k, j]$ (an element in the k -th row)

Let us evaluate how the k -th column is updated during the k -th iteration:

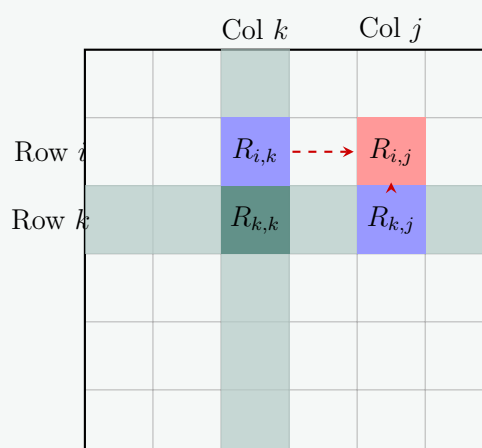
$$\begin{aligned} R^{(k)}[i, k] &= R^{(k-1)}[i, k] \vee (R^{(k-1)}[i, k] \wedge R^{(k-1)}[k, k]) \\ &= R^{(k-1)}[i, k] \quad (\text{by the Boolean absorption law: } A \vee (A \wedge B) = A) \end{aligned}$$

Similarly, for the k -th row:

$$\begin{aligned} R^{(k)}[k, j] &= R^{(k-1)}[k, j] \vee (R^{(k-1)}[k, k] \wedge R^{(k-1)}[k, j]) \\ &= R^{(k-1)}[k, j] \quad (\text{by the Boolean absorption law}) \end{aligned}$$

Conclusion: The elements in the k -th row and k -th column **do not change** during the k -th iteration. Therefore, whether we read them before or after they are "updated" in the k -th pass, their values remain identical. It is perfectly safe to perform all updates in-place.

3. Visualizing the Dependencies



During the k -th phase (shaded grey), computing $R[i, j]$ (red) depends exclusively on $R[i, k]$ and $R[k, j]$ (blue). Because the grey row and column are invariant in phase k , the order of updates within the matrix is completely irrelevant.

4. In-Place Algorithm

Algorithm 3 In-Place Warshall's Algorithm**Require:** Adjacency matrix $A[1 \dots n][1 \dots n]$ representing a directed graph**Ensure:** Matrix $R[1 \dots n][1 \dots n]$ representing the transitive closure

```

1: Initialize  $R \leftarrow A$  ▷ Copy adjacency matrix to  $R$ 
2: for  $k \leftarrow 1$  to  $n$  do
3:   for  $i \leftarrow 1$  to  $n$  do
4:     if  $R[i, k] == 1$  then ▷ Optimization: Only check  $j$  if path to  $k$  exists
5:       for  $j \leftarrow 1$  to  $n$  do
6:          $R[i, j] \leftarrow R[i, j] \vee R[k, j]$  ▷ In-place update
7:       end for
8:     end if
9:   end for
10: end for
11: return  $R$ 

```

5. Complexity Analysis

- **Time Complexity:** The algorithm utilizes three nested loops (over k , i , and j), each running exactly n times. The operations inside the inner loop take constant time $O(1)$. Thus, the time complexity is strictly $O(n^3)$. (Note: The 'if' statement on line 4 is a practical optimization that speeds up sparse graphs by skipping the innermost loop, but the worst-case asymptotic time remains $O(n^3)$).
- **Space Complexity:** By overwriting the initial matrix R , we do not allocate any additional $n \times n$ matrices for the intermediate steps $R^{(1)} \dots R^{(n-1)}$. Therefore, the auxiliary space complexity is $O(1)$ beyond the $O(n^2)$ space required to store the input/output matrix itself.

Q15. Given a graph with integer edge weights between 1 and 5 (inclusive), you want to find the shortest weighted path between a pair of vertices. How would you reduce this problem to the shortest unweighted path problem, which can be solved using a Breadth First Search? (13 marks) [CSE 207 | L-2/T-2 | 2019–20]

Answer**1. Reduction Strategy (Edge Splitting)**

Breadth-First Search (BFS) computes the shortest path in an unweighted graph because it explores vertices layer by layer in terms of edge count. To adapt BFS for a graph with integer edge weights between 1 and 5, we can transform the weighted graph $G = (V, E)$ into an equivalent unweighted graph $G' = (V', E')$ by replacing each heavily weighted edge with a chain of directed edges of weight 1, separated by new dummy vertices.

Specifically, for any directed edge $(u, v) \in E$ with an integer weight $w(u, v) = k$ (where $1 \leq k \leq 5$):

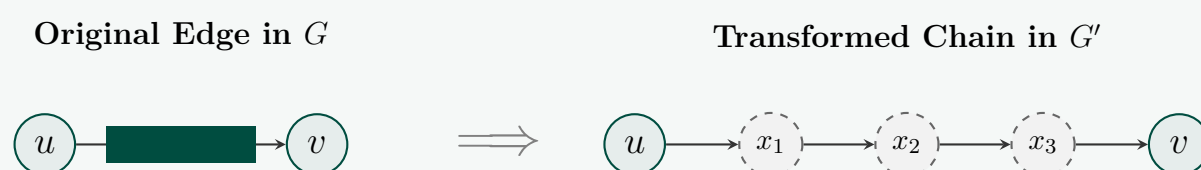
- If $k = 1$, we retain the edge (u, v) as an unweighted edge in G' .
- If $k > 1$, we introduce $k - 1$ dummy vertices, say $x_1, x_2, \dots, x_{k-1}$, and replace the single edge (u, v) with a directed path of k unweighted edges:

$$u \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_{k-1} \rightarrow v$$

Since every individual edge in the newly constructed graph G' has an implicit weight of 1, the unweighted shortest path length (number of edges traversed) between any pair of original vertices in G' corresponds exactly to the minimum weight sum in G .

2. Visualizing the Reduction

The diagram below illustrates how an edge of weight 4 is transformed into a sequence of 4 unweighted edges using 3 dummy vertices.

**3. Algorithm**

The overall algorithm consists of a graph construction phase followed by a standard BFS. As requested, we utilize `\Require` and `\Ensure` for specifying inputs and outputs cleanly in accordance with standard `algorithmic` syntax.

Algorithm 4 Shortest Path via Edge Splitting and BFS**Require:** Directed Graph $G = (V, E)$ with integer weights $w(e) \in \{1, 2, 3, 4, 5\}$, source s , target t **Ensure:** Shortest distance from s to t

```

1: Initialize  $V' \leftarrow V$ 
2: Initialize  $E' \leftarrow \emptyset$ 
3: dummy_counter  $\leftarrow 0$ 
4: for each edge  $(u, v) \in E$  do                                     ▷ Graph Reduction Phase
5:    $k \leftarrow w(u, v)$ 
6:   if  $k == 1$  then
7:     Add  $(u, v)$  to  $E'$ 
8:   else
9:      $prev \leftarrow u$ 
10:    for  $i \leftarrow 1$  to  $k - 1$  do
11:      dummy_counter  $\leftarrow$  dummy_counter + 1
12:      Create a new vertex  $x_{\text{dummy\_counter}}$ 
13:      Add  $x_{\text{dummy\_counter}}$  to  $V'$ 
14:      Add edge  $(prev, x_{\text{dummy\_counter}})$  to  $E'$ 
15:       $prev \leftarrow x_{\text{dummy\_counter}}$ 
16:    end for
17:    Add edge  $(prev, v)$  to  $E'$ 
18:  end if
19: end for
20: return BFS( $G', s, t$ )                                           ▷ Standard unweighted BFS phase

```

4. Complexity Analysis

- **Graph Size Expansion:** Let $W_{\max} = 5$ be the maximum edge weight. For each edge in E , we introduce at most $W_{\max} - 1 = 4$ dummy vertices and at most $W_{\max} = 5$ unweighted edges. Therefore, the size of the transformed graph $G' = (V', E')$ is bounded by:

$$|V'| \leq |V| + (W_{\max} - 1)|E| = |V| + 4|E|$$

$$|E'| \leq W_{\max}|E| = 5|E|$$

- **Time Complexity:**

- Reduction Phase:* Constructing G' takes $O(|V'| + |E'|)$ time since we process each original edge and generate a small constant number of dummy elements.
- BFS Phase:* Running BFS on G' takes linear time with respect to the size of G' , which is $O(|V'| + |E'|)$.

Substituting the bounds for $|V'|$ and $|E'|$, the total time complexity becomes $O(|V| + 4|E| + 5|E|) = O(|V| + |E|)$. Because $W_{\max}$ is a small fixed constant (5), the asymptotic execution time remains perfectly linear, matching the efficiency of an unweighted graph.

- **Space Complexity:** The storage requirement for the extra vertices and edges in G' is directly proportional to the size of G' . Since $|V'| = O(|V| + |E|)$ and $|E'| = O(|E|)$, the total auxiliary space complexity is $O(|V| + |E|)$.

Q16. A new startup FastRoute wants to route information along a path in a network. Each vertex represents a router and each edge in the network has a bandwidth. Design an algorithm to find the path with maximum bandwidth from any source s to any destination t . The bandwidth of a path is the minimum of the bandwidths of the edges on that path; the minimum edge is the bottleneck. Explain how to modify Dijkstra's algorithm to do this. Justify your answer. **(15 marks)** [CSE 207 | L-2/T-2 | 2019–20]

Answer**1. Algorithmic Strategy (Max-Min Modification)**

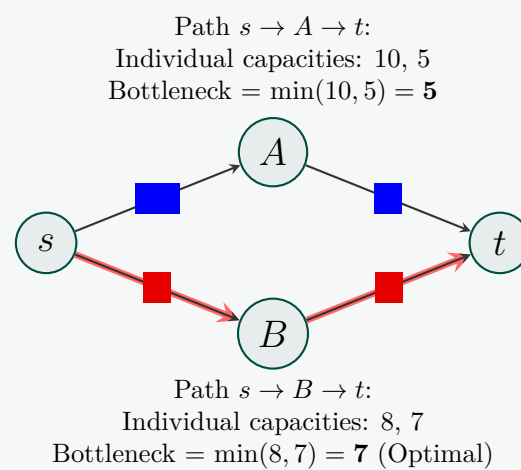
The problem asks us to find a path from a source s to a destination t that maximizes the bottleneck (the minimum edge weight along the path). This is known as the *Max-Min Path Problem* or the *Widest Path Problem*.

Standard Dijkstra's algorithm minimizes the *sum* of edge weights along a path using a Min-Priority Queue. To find the path with the maximum bottleneck bandwidth, we modify Dijkstra's algorithm in three core ways:

- Metric Transformation:** Instead of accumulating a sum ($\text{dist}[u] + w(u, v)$), the bandwidth of a path extended from u to v is determined by the minimum of the bandwidth already secured up to u and the capacity of the edge (u, v) . That is: $\min(bw[u], b(u, v))$.
- Optimization Direction:** We want to *maximize* this path metric. Therefore, the relaxation step updates a neighbor v if a strictly larger bottleneck capacity can be achieved through u .
- Priority Queue Inversion:** We utilize a **Max-Priority Queue** instead of a Min-Priority Queue to greedily extract the vertex with the maximum possible bottleneck bandwidth at each step.

2. Visualizing the Bottleneck Selection

The diagram below highlights why a standard shortest-path approach fails and why tracking the bottleneck capacity yields the correct path.



Note: The upper path starts strong with an edge of 10, but is restricted by the bottleneck of 5. The algorithm correctly chooses the lower path because its bottleneck (7) is superior.

3. Modified Dijkstra Algorithm

Algorithm 5 Max-Min Dijkstra for Widest Path

Require: Directed Graph $G = (V, E)$, edge bandwidths $b(u, v) \geq 0$, source s , destination t

Ensure: The maximum bottleneck bandwidth from s to t

```

1: Initialize  $bw[v] \leftarrow 0$  for all  $v \in V$ 
2: Initialize  $parent[v] \leftarrow \text{NIL}$  for all  $v \in V$ 
3:  $bw[s] \leftarrow \infty$  ▷ Source has infinite initial capacity
4: Initialize a Max-Priority Queue  $Q$ 
5: for each vertex  $v \in V$  do
6:    $Q.\text{insert}(v, bw[v])$ 
7: end for
8: while  $Q$  is not empty do
9:    $u \leftarrow Q.\text{extract\_max}()$ 
10:  if  $u == t$  or  $bw[u] == 0$  then ▷ Target reached or remaining nodes unreachable
11:    break
12:  end if
13:  for each neighbor  $v$  of  $u$  such that  $(u, v) \in E$  do
14:     $local\_bottleneck \leftarrow \min(bw[u], b(u, v))$ 
15:    if  $local\_bottleneck > bw[v]$  then
16:       $bw[v] \leftarrow local\_bottleneck$ 
17:       $parent[v] \leftarrow u$ 
18:       $Q.\text{change\_priority}(v, bw[v])$  ▷ Increase key operation
19:    end if
20:  end for
21: end while
22: return  $bw[t]$ 

```

4. Mathematical Justification and Correctness Proof

To prove that this modified algorithm is correct, we show that when a vertex u is extracted from the Max-Priority Queue, its assigned value $bw[u]$ is the exact maximum bottleneck capacity from s to u .

Let S be the set of vertices already extracted from Q . We use induction on the size of S :

- **Base Case:** When $S = \{s\}$, $bw[s] = \infty$, which is trivially correct because the path from s to itself has no limiting edges.
- **Inductive Step:** Suppose the hypothesis holds for the current set S . The algorithm now selects a vertex $u \notin S$ that maximizes $bw[u]$ among all elements in Q . Let P be the path discovered by the algorithm to u . By definition, its bandwidth is $bw[u]$.

Assume for contradiction that there exists another path P' from s to u with a strictly higher bottleneck bandwidth, meaning $B(P') > bw[u]$.

Since $s \in S$ and $u \notin S$, the path P' must cross the boundary of S . Let (x, y) be the first edge in P' such that $x \in S$ and $y \notin S$.

Since $x \in S$, its optimal bottleneck value was already correctly computed as $bw[x]$ by our inductive hypothesis. The bottleneck bandwidth of P' up to node y is given by $\min(bw[x], b(x, y))$. When x was extracted, the edge (x, y) was relaxed, meaning:

$$bw[y] \geq \min(bw[x], b(x, y)) \geq B(P')$$

Because the bottleneck capacity of a path can only decrease or stay the same as more edges are added, the final bottleneck bandwidth of P' at destination u cannot exceed its bottleneck bandwidth at y :

$$B(P') \leq bw[y]$$

However, our algorithm selected u from the Max-Priority Queue instead of y , which implies:

$$bw[u] \geq bw[y]$$

Combining these inequalities yields:

$$bw[u] \geq bw[y] \geq B(P')$$

This directly contradicts our assumption that $B(P') > bw[u]$. Thus, no path can offer a strictly greater bottleneck than the one identified by our algorithm, proving its complete mathematical correctness.

5. Complexity Analysis

The structural design matches standard Dijkstra's algorithm, meaning its asymptotic complexity depends completely on the choice of data structure used to implement the priority queue Q .

- **Time Complexity:**

- (a) **Binary Max-Heap:** There are $|V|$ executions of `extract_max`, each taking $O(\log |V|)$ time. There are at most $|E|$ executions of the `change_priority` (increase key) operation, each taking $O(\log |V|)$ time. This yields an overall time complexity of $O((|V| + |E|) \log |V|)$.
- (b) **Fibonacci Heap:** Using a Fibonacci Heap, `extract_max` takes $O(\log |V|)$ amortized time, while the `change_priority` operation runs in $O(1)$ amortized time. This optimizes the performance to $O(|E| + |V| \log |V|)$, which is ideal for dense networks.

- **Space Complexity:** The storage overhead includes the priority queue, the bw array, and the *parent* pointers. Each array requires a size proportional to the number of vertices. Thus, the total auxiliary space complexity is linear with respect to the graph size: $O(|V| + |E|)$.

Q17. Show how to express the single-source shortest-paths problem as a product of matrices and a vector. Describe how evaluating this product corresponds to a Bellman-Ford-like algorithm. **(10 marks)** [CSE 207 | L-2/T-2 | 2019–20]

Q18. Give an efficient algorithm to find the length (number of edges) of a minimum-length negative-weight cycle in a graph. **(10 marks)** [CSE 207 | L-2/T-2 | 2019–20]

Answer

1. Algorithmic Strategy

The problem requires finding the minimum number of edges (the minimum length) of a negative-weight cycle in a directed graph $G = (V, E)$. While the standard Bellman-Ford or Floyd-Warshall algorithms can detect the *existence* of a negative cycle, they do not inherently find the cycle that minimizes the edge count.

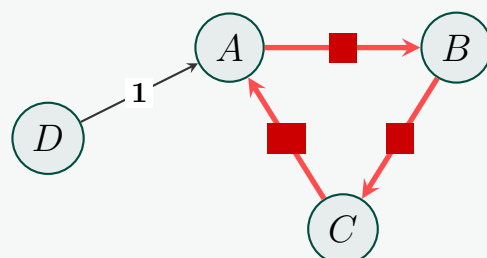
To solve this efficiently, we can leverage the **min-plus semiring matrix multiplication** framework. Let W be the $n \times n$ adjacency weight matrix of the graph, with $W_{ii} = 0$ for all i . In the min-plus semiring, computing the m -th matrix power W^m yields entries W_{ij}^m representing the minimum weight of a path from vertex i to vertex j consisting of **at most** m edges. A negative-weight cycle of length at most m exists if and only if there is some vertex i such that:

$$W_{ii}^m < 0$$

A naive sequential evaluation of $W^1, W^2, \dots, W^n$ would find the first m where a negative diagonal element appears, requiring $O(n)$ matrix multiplications and yielding an $O(n^4)$ time complexity. We can optimize this to **$O(n^3 \log n)$** using **binary lifting** (order-of-magnitude decomposition). By precomputing the power-of-two matrices $(W^1, W^2, W^4, \dots, W^{2^k})$, we can greedily find the maximum number of edges m that can be traversed ***without*** forming a negative cycle. The length of the shortest negative cycle is then exactly $m + 1$.

2. Visualizing Cycle Elongation

The diagram below shows a sample graph containing a negative cycle. The algorithm tracks the path weights based on edge constraints to pinpoint the exact moment the cycle closes and drops below zero.



Negative Cycle Exploration:

Path length 1 edge: $\min(W_{ii}) = 0 \geq 0$

Path length 2 edges: $\min(W_{ii}^2) = 0 \geq 0$

Path length 3 edges: $W_{AA}^3 = 2 + 3 - 6 = -1 < 0 \implies \text{Minimum Length} = 3$

3. The Binary Lifting Cycle-Detection Algorithm

Algorithm 6 Binary Lifting for Minimum-Length Negative Cycle**Require:** Directed graph representation as an $n \times n$ matrix W , where $W_{ii} = 0$, $W_{ij} = w(i, j)$ if $(i, j) \in E$, and $W_{ij} = \infty$ otherwise.**Ensure:** The exact number of edges in a minimum-length negative cycle, or ∞ if no negative cycle exists.

```

1:  $k \leftarrow \lceil \log_2 n \rceil$ 
2: Initialize an array of matrices  $P[0 \dots k]$  of size  $n \times n$ 
3:  $P[0] \leftarrow W$ 
4: for  $j \leftarrow 1$  to  $k$  do
5:    $P[j] \leftarrow P[j-1] \otimes P[j-1]$  ▷ Precompute  $W^{2^j}$  under the min-plus semiring
6: end for
7: Initialize matrix  $D$  of size  $n \times n$  where  $D_{ii} = 0$  and  $D_{ij} = \infty$  for  $i \neq j$  ▷ Identity Matrix  $I$ 
8:  $m \leftarrow 0$ 
9: for  $j \leftarrow k$  down to  $0$  do
10:   $T \leftarrow D \otimes P[j]$  ▷ Candidate distance matrix for  $m + 2^j$  edges
11:  has_neg_cycle  $\leftarrow$  false
12:  for  $i \leftarrow 1$  to  $n$  do
13:    if  $T[i][i] < 0$  then
14:      has_neg_cycle  $\leftarrow$  true break
15:    end if
16:  end for
17:  if not has_neg_cycle then
18:     $D \leftarrow T$  ▷ Accept the edge accumulation
19:     $m \leftarrow m + 2^j$ 
20:  end if
21: end for
22: if  $m \geq n$  then
23:   return  $\infty$  ▷ No negative cycle exists within the simple path boundary
24: else
25:   return  $m + 1$ 
26: end if

```

4. Mathematical Justification and Correctness

To prove the correctness of this approach, we must guarantee that:

- (a) $W_{ii}^p < 0$ implies the existence of a negative cycle of length $\leq p$.
- (b) Binary lifting successfully identifies the exact boundary without skipping the optimal value.

Proof of Property 1 (Matrix Property): By the properties of min-plus matrix multiplication, the cell value is defined as:

$$W_{ij}^p = \min_{1 \leq v_1, v_2, \dots, v_{p-1} \leq n} (W_{iv_1} + W_{v_1 v_2} + \dots + W_{v_{p-1} j})$$

When observing a diagonal element W_{ii}^p , this represents the minimum weight of a closed walk starting at i and ending at i containing at most p edges. Because the diagonal identity components $W_{vv} = 0$ are available, any walk of length shorter than p is automatically preserved in the minimization pool. If $W_{ii}^p < 0$, a negative walk exists, meaning a structural negative cycle must exist within those p edges.

Proof of Property 2 (Lifting Convergence): Let L^* be the true minimum length of a negative-weight cycle in G . Any simple cycle can contain at most n edges, meaning $1 \leq L^* \leq n$. By definition, for any length $p < L^*$, no negative cycle can be formed, so $W_{ii}^p \geq 0$ for all $i \in V$. Conversely, for any length $p \geq L^*$, the optimal negative cycle is available, meaning $\min_i W_{ii}^p < 0$.

The binary lifting loop processes bits from most significant (k) down to least significant (0). It maintains a running count m representing a path length known to contain **no** negative cycles. At each bit stage j :

- If $m + 2^j < L^*$, then $T_{ii} = W_{ii}^{m+2^j} \geq 0$. The algorithm sets $m \leftarrow m + 2^j$.
- If $m + 2^j \geq L^*$, then $T_{ii} = W_{ii}^{m+2^j} < 0$. The algorithm rejects the step, keeping m unchanged.

By the termination of the loop, m will have converged precisely to the largest integer matching $m = L^* - 1$. Adding 1 yields $m + 1 = L^*$, which matches the exact length of the minimum negative cycle. If m finishes at $\geq n$, it guarantees no negative cycle exists in the entire graph.

5. Complexity Analysis

- **Time Complexity:**

- (a) **Precomputation Phase:** The algorithm performs $k = \lceil \log_2 n \rceil$ matrix multiplications to compute the matrix powers $P[j]$. Each min-plus matrix multiplication takes $O(n^3)$ time. This phase takes $O(n^3 \log n)$ time.
- (b) **Binary Lifting Phase:** The outer loop runs $k + 1$ times. In each iteration, it executes one matrix multiplication ($D \otimes P[j]$) taking $O(n^3)$ time, followed by a diagonal check taking $O(n)$ time. This phase also takes $O(n^3 \log n)$ time.

Combining these phases gives a total time complexity of $O(n^3 \log n)$, which is significantly faster than the sequential $O(n^4)$ approach.

- **Space Complexity:** The algorithm stores $\lceil \log_2 n \rceil + 1$ matrices of dimensions $n \times n$ within the matrix power array P . This results in a total auxiliary space complexity of $O(n^2 \log n)$.

Q19. You are given a strongly connected directed graph $G = (V, E)$ with positive edge weights along with a particular node $v_0 \in V$. Give an efficient algorithm for finding shortest paths between all pairs of nodes, with the one restriction that these paths must all pass through v_0 . (12 marks) [CSE 207 | L-2/T-2 | 2019–20]

Answer

1. Algorithmic Strategy (Path Decomposition)

The problem requires us to find the shortest path between every pair of nodes $(u, w) \in V \times V$, subject to the strict constraint that the path must visit a specific intermediate node v_0 .

Because edge weights are strictly positive, the shortest path from u to w passing through v_0 can be cleanly decomposed into two independent optimal sub-paths:

- The shortest path from the source u to the intermediate node v_0 .
- The shortest path from the intermediate node v_0 to the destination w .

Therefore, the minimum constrained distance $D(u, w)$ can be mathematically expressed as:

$$D(u, w) = \text{dist}(u, v_0) + \text{dist}(v_0, w)$$

Instead of running an All-Pairs Shortest Path (APSP) algorithm like Floyd-Warshall, which takes $O(|V|^3)$ time, we only need to compute distances relative to the single hub node v_0 . We can achieve this efficiently using **Dijkstra's Algorithm** in two phases.

2. Two-Phase Dijkstra Approach

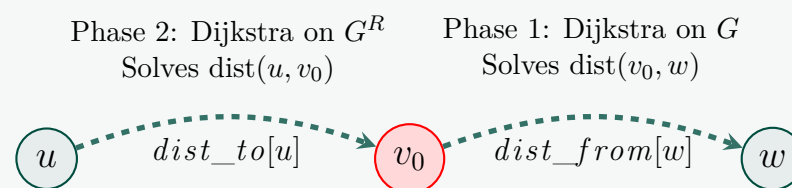
• Phase 1: Distances FROM v_0 (Single-Source)

We run standard Dijkstra's algorithm on the original graph G starting from v_0 . This yields an array $\text{dist_from}[w]$, representing the shortest path from v_0 to every node $w \in V$.

• Phase 2: Distances TO v_0 (Single-Destination)

To find the shortest paths from every node u to v_0 , we construct the transposed (reversed) graph $G^R = (V, E^R)$, where every directed edge $(x, y) \in E$ is reversed to $(y, x) \in E^R$. We then run Dijkstra's algorithm on G^R starting from v_0 . This yields an array $\text{dist_to}[u]$, representing the shortest path from every node $u \in V$ to v_0 in the original graph.

3. Visualizing the Decomposition



4. Algorithm

Algorithm 7 All-Pairs Shortest Path via Hub v_0

Require: Strongly connected directed Graph $G = (V, E)$ with positive weights w , hub node v_0

Ensure: A $|V| \times |V|$ matrix D where $D[u, w]$ is the shortest path from u to w through v_0

```

1: Function Dijkstra(Graph, source):
2:   Initialize  $\text{dist}[v] \leftarrow \infty$  for all  $v \in V$ ;  $\text{dist}[\text{source}] \leftarrow 0$ 
3:   Initialize Min-Priority Queue  $Q$  with  $(\text{source}, 0)$ 
4:   While  $Q$  is not empty:
5:      $x \leftarrow Q.\text{extract\_min}()$ 
6:     For each neighbor  $y$  of  $x$ :
7:       If  $\text{dist}[x] + \text{weight}(x, y) < \text{dist}[y]$ :
8:          $\text{dist}[y] \leftarrow \text{dist}[x] + \text{weight}(x, y)$ 
9:          $Q.\text{insert\_or\_decrease\_key}(y, \text{dist}[y])$ 
10:  Return  $\text{dist}$ 
11:  $\text{dist\_from} \leftarrow \text{Dijkstra}(G, v_0)$  ▷ Phase 1: Distances from  $v_0$ 
12: Construct  $G^R = (V, E^R)$  by reversing all edges in  $E$ 
13:  $\text{dist\_to} \leftarrow \text{Dijkstra}(G^R, v_0)$  ▷ Phase 2: Distances to  $v_0$ 
14: Initialize  $|V| \times |V|$  matrix  $D$ 
15: for each  $u \in V$  do
16:   for each  $w \in V$  do
17:      $D[u, w] \leftarrow \text{dist\_to}[u] + \text{dist\_from}[w]$  ▷ Combine the sub-paths
18:   end for
19: end for
20: return  $D$ 

```

5. Complexity Analysis

- **Time Complexity:**

- (a) Constructing the reversed graph G^R takes $O(|V| + |E|)$ time.
- (b) Running Dijkstra's algorithm twice (once on G and once on G^R) using a Fibonacci heap takes $2 \times O(|E| + |V| \log |V|) = O(|E| + |V| \log |V|)$ time.
- (c) Constructing the final $|V| \times |V|$ distance matrix requires computing the sum for every pair, taking exactly $O(|V|^2)$ time.

Because $|E| \leq |V|^2$, the time taken to build the explicit $|V| \times |V|$ output matrix is the dominating factor. Thus, the total time complexity is strictly $\mathbf{O}(|V|^2)$. (Note: If only an implicit representation or an $O(1)$ query mechanism is needed instead of generating the full matrix, the time complexity drops to $O(|E| + |V| \log |V|)$).

- **Space Complexity:** The algorithm stores the graph G , the reversed graph G^R , the priority queue, and the two distance arrays ($dist_from$ and $dist_to$). This takes $O(|V| + |E|)$ auxiliary space. Storing the final output matrix D takes $\mathbf{O}(|V|^2)$ space.

Q20. Dijkstra's algorithm for the shortest path problem is shown below. What would be the time-complexity of the algorithm (show line by line analysis) for an input graph G if adjacency list is used for graph representation and an ordinary array is used for storing $d[]$ values.

```

1 Dijkstra (G)
2   Q = V[G];
3   for each u in Q
4     d[u] = INF;
5   d[s] = 0; S = {};
6   while (Q != {})
7     u = ExtractMin(Q);
8     S = S U {u};
9     for each v in u->Adj[]
10      if (v in Q and d[v] > d[u] + w(u,v))
11      d[v] = d[u] + w(u,v);

```

(15 marks)

[CSE 207 | L-2/T-2 | 2018–19]

Answer

1. Algorithmic Context

The requested analysis depends heavily on the data structures chosen. Because the problem specifies that an **adjacency list** is used for the graph $G = (V, E)$ and an **ordinary array** is used to store the shortest-path estimates $d[]$, the operations behave differently compared to a standard Min-Heap implementation.

Specifically, finding the minimum value in an unsorted array takes linear time, but updating a distance value takes constant time.

2. Line-by-Line Complexity Analysis

Let $|V|$ be the number of vertices and $|E|$ be the number of edges.

- **Line 1: $Q = V[G];$**
Initializing the set or array of unvisited vertices Q takes $\mathbf{O}(|V|)$ time.
- **Lines 2 & 3: for each u in Q { $d[u] = \text{INF};$ }**
This loop iterates over all vertices to initialize their distances to infinity. (Note: Assuming $d[v]$ is a typo in the provided pseudocode and means $d[u]$). This takes $\mathbf{O}(|V|)$ time.
- **Line 4: $d[s] = 0; S = \{\};$**
Setting the source distance and initializing the visited set S takes $\mathbf{O}(1)$ time.
- **Line 5: while ($Q \neq \{\}$)**
Because exactly one vertex is extracted from Q in each iteration, this while loop executes exactly $|V|$ times.
- **Line 6: $u = \text{ExtractMin}(Q);$**
Since $d[]$ is an ordinary unsorted array, finding the vertex with the minimum distance requires scanning the remaining vertices in Q . This scan takes $O(|V|)$ time per iteration. Over all $|V|$ iterations of the while loop, the total time spent here is $\mathbf{O}(|V|^2)$.
- **Line 7: $S = S \cup \{u\};$**
Adding the extracted vertex u to the visited set S (which can be implemented as a boolean array) takes $O(1)$ time per iteration. Total time over the whole algorithm is $\mathbf{O}(|V|)$.
- **Line 8: for each v in $u \rightarrow \text{Adj}[]$**
Because the graph uses an adjacency list, this for loop iterates over the neighbors of u . Across all $|V|$ iterations of the outer while loop, every adjacency list is traversed exactly once. The sum of the lengths of all adjacency lists is proportional to the number of edges. Thus, the code strictly inside this loop executes $\mathbf{O}(|E|)$ times *in total* across the entire algorithm.

- **Lines 9: if (v in Q and d[v] > d[u] + w(u,v))**

Checking membership in Q takes $O(1)$ time (using a boolean array mapping), and checking the distance condition is basic arithmetic, also $O(1)$. Executed $O(|E|)$ times globally, taking $O(|E|)$ total time.

- **Line 10: d[v] = d[u] + w(u,v);**

Updating the distance array is a direct index assignment. Because $d[]$ is an ordinary array, this update takes $O(1)$ time (there is no **decrease-key** overhead like there would be in a priority queue). Executed at most $O(|E|)$ times, this takes $O(|E|)$ total time.

3. Overall Time Complexity

To find the final time complexity, we sum the aggregate costs of all operations:

$$T(|V|, |E|) = O(|V|) + O(|V|) + O(1) + O(|V|^2) + O(|V|) + O(|E|) + O(|E|) + O(|E|)$$

This simplifies to:

$$T(|V|, |E|) = O(|V|^2 + |E|)$$

Since the number of edges $|E|$ in a simple graph is bounded by $O(|V|^2)$ (i.e., $|E| \leq |V|(|V| - 1)$), the $|E|$ term is asymptotically dominated by $|V|^2$.

Final Conclusion: The overall time complexity of Dijkstra's algorithm implemented with an ordinary array is tightly bounded at $O(|V|^2)$. This implementation is highly efficient and actually optimal for dense graphs where $|E| \approx |V|^2$.

Q21. Johnson's Algorithm makes clever use of Bellman-Ford and Dijkstra's Algorithms to do All-Pairs-Shortest-Paths efficiently on sparse graphs. Explain the basic clever idea of Johnson's Algorithm. **(15 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Answer

1. The Core Challenge

If a graph has only non-negative edge weights, we can find the shortest paths between all pairs of nodes by running **Dijkstra's Algorithm** $|V|$ times (once from each vertex). On a sparse graph, this takes $O(|V||E| \log |V|)$ time, which is significantly faster than Floyd-Warshall's $O(|V|^3)$.

However, Dijkstra's algorithm fails if the graph contains even a single negative edge weight. A naive guess to fix this might be to find the most negative edge weight $-W$, and add W to every edge to make them all non-negative. **This does not work.**

- If Path A uses 2 edges and Path B uses 5 edges, adding W to every edge penalizes Path B by $5W$ and Path A by only $2W$.
- This structurally alters the graph's topology and changes which path is actually the shortest.

2. The Clever Idea: Vertex Potential Reweighting

Johnson's algorithm overcomes this by assigning a **potential** $h(u)$ to every vertex u . Instead of altering edges uniformly, it alters each edge (u, v) based on the potentials of its endpoints.

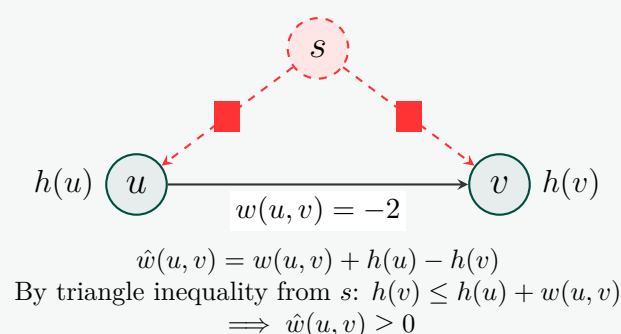
The new reweighted edge cost $\hat{w}(u, v)$ is calculated as:

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v)$$

For this scheme to work, the potential function must satisfy two magical properties:

- No Negative Edges:** The new weights must be non-negative ($\hat{w}(u, v) \geq 0$) so we can use Dijkstra.
- Preservation of Shortest Paths:** The relative order of path lengths between any two nodes must remain identical to the original graph.

3. Visualizing the Reweighting Mechanism



4. The Algorithm

Algorithm 8 Johnson's Algorithm for APSP**Require:** Directed graph $G = (V, E)$ with edge weights $w(u, v)$ **Ensure:** Matrix D of shortest path distances for all pairs, or indication of a negative cycle

```

1: Add a dummy vertex  $s$  to  $V$ 
2: Add directed edges  $(s, v)$  with  $w(s, v) = 0$  for all  $v \in V$ 
3: Run Bellman-Ford on  $G$  from source  $s$ 
4: if Bellman-Ford detects a negative-weight cycle then
5:   return "Graph contains a negative-weight cycle"
6: end if
7: for each  $v \in V$  do
8:    $h(v) \leftarrow$  shortest path distance from  $s$  to  $v$  computed by Bellman-Ford
9: end for
10: for each edge  $(u, v) \in E$  do
11:    $\hat{w}(u, v) \leftarrow w(u, v) + h(u) - h(v)$  ▷ Reweight edges
12: end for
13: Initialize  $|V| \times |V|$  matrix  $D$ 
14: Remove dummy vertex  $s$  and its incident edges
15: for each vertex  $u \in V$  do
16:   Run Dijkstra's algorithm from  $u$  using the new weights  $\hat{w}$  to compute  $\hat{\delta}(u, v)$ 
17:   for each vertex  $v \in V$  do
18:      $D[u][v] \leftarrow \hat{\delta}(u, v) - h(u) + h(v)$  ▷ Revert to original distances
19:   end for
20: end for
21: return  $D$ 

```

5. Why Path Lengths Are Preserved (Mathematical Proof)Consider any path P from vertex v_1 to vertex v_k consisting of the vertices $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$.

If we sum up the reweighted edges along this path, we get:

$$\hat{w}(P) = (w(v_1, v_2) + h(v_1) - h(v_2)) + (w(v_2, v_3) + h(v_2) - h(v_3)) + \dots + (w(v_{k-1}, v_k) + h(v_{k-1}) - h(v_k))$$

Notice how the intermediate potentials completely cancel out (a telescoping sum):

$$\hat{w}(P) = w(P) + h(v_1) - h(v_k)$$

Because $h(v_1)$ and $h(v_k)$ depend *only* on the start and end vertices of the path, **every single path between v_1 and v_k is shifted by the exact same constant value**. Therefore, whichever path was the shortest in the original graph remains the shortest in the reweighted graph.

6. Complexity Analysis

- **Bellman-Ford Phase:** Running Bellman-Ford once from the dummy node takes $O(|V||E|)$ time.
- **Reweighting Phase:** Updating all edge weights takes $O(|E|)$ time.
- **Dijkstra Phase:** Running Dijkstra's algorithm $|V|$ times using a Fibonacci Heap takes $|V| \times O(|E| + |V| \log |V|) = O(|V||E| + |V|^2 \log |V|)$ time. Using a standard Binary Min-Heap takes $O(|V||E| \log |V|)$ time.
- **Total Time Complexity:** The Dijkstra phase dominates, making the overall time complexity $O(|V||E| + |V|^2 \log |V|)$ with a Fibonacci heap, which is vastly superior to Floyd-Warshall's $O(|V|^3)$ for sparse graphs (where $|E| \ll |V|^2$).
- **Space Complexity:** $O(|V|^2)$ to store the final All-Pairs Shortest Path matrix.

Q22. How can BFS be adapted to solve the single-source shortest path problem in an edge-weighted graph with unequal weights? Explain. (8 marks) [CSE 207 | L-2/T-2 | 2017–18]

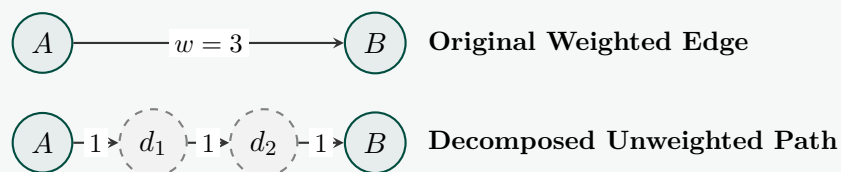
Answer**1. The Core Limitation of Standard BFS**

Standard Breadth-First Search (BFS) inherently assumes an unweighted graph, meaning every traversed edge costs exactly 1. Because of this uniform structure, a standard BFS guarantees that the first time a vertex is discovered, the path taken to reach it is the absolute shortest in terms of edge count (and thus total weight).

When edges have unequal weights, this structural guarantee breaks down. A path with fewer edges might have a significantly higher cumulative weight than a path with more edges. To adapt BFS to solve the Single-Source Shortest Path (SSSP) problem under unequal weights, we can choose between two primary paradigms depending on our performance constraints and edge attributes.

2. Method 1: Edge Splitting (The Dummy Node Approach)

If all edge weights are **small, positive integers**, we can conceptually transform the weighted graph back into an unweighted graph by decomposing heavy edges. For any directed edge (u, v) with an integer weight $w > 1$, we replace it with a chain containing $w - 1$ temporary dummy vertices interconnected by edges of weight 1.

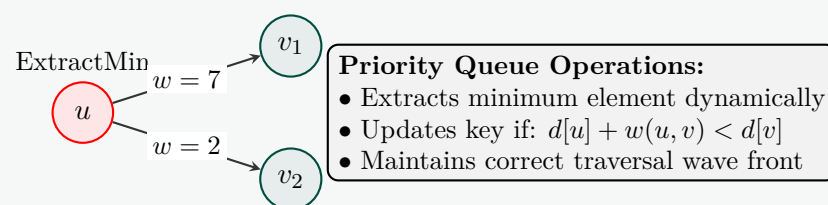


- **How it works:** This graph transformation runs during input processing. Once the graph consists exclusively of unit-weight edges, we run standard BFS using a standard First-In-First-Out (FIFO) queue. The shortest path distances are accurately found because the dummy chains correctly delay the arrival of the wavefront at node B .
- **Complexity:** The time complexity shifts from $O(|V| + |E|)$ to $O(|V| + \sum_{(u,v) \in E} w(u,v))$.
- **Limitations:** This approach is highly inefficient or entirely unusable if the graph contains huge integer weights (e.g., a weight of 10^6 generates 999,999 dummy nodes, exhausting memory) or real-valued (floating-point) capacities.

3. Method 2: Upgrading the Queue (Dijkstra's Algorithm)

If weights are arbitrary positive real numbers or large integers, adding structural dummy elements becomes impossible. Instead of modifying the graph topology to fit the algorithm, we **modify the algorithm's tracking queue to fit the graph**. This adaptation is formally recognized as **Dijkstra's Algorithm**.

In standard BFS, the FIFO queue keeps nodes sorted by distance because every traversal step increments the distance uniformly by $+1$. To handle non-uniform steps, we replace the basic FIFO queue with a **Min-Priority Queue**.



- **How it works:** Instead of pulling the oldest vertex, the priority queue always extracts the vertex $u \in Q$ possessing the minimum tentative distance $d[u]$. When u is processed, we relax all its outgoing edges. If a shorter path to a neighbor v is discovered through u , we update $d[v]$ and adjust its placement in the priority queue (**Decrease-Key**).
- **Complexity:** Using a binary min-heap implementation, the execution time is bounded at $O((|V| + |E|) \log |V|)$.
- **Limitations:** While handles real numbers effectively, this priority queue adaptation relies on the property that paths grow monotonically. It cannot handle edges with **negative weights** without structural alterations (such as Johnson's reweighting potentials).

4. Algorithmic Synthesis Comparison

Metric	Standard BFS	Edge-Splitting BFS	Priority Queue BFS (Dijkstra)
Weight Constraint	Edges must be exactly 1	Small Positive Integers	Non-Negative Real Numbers
Core Data Structure	FIFO Queue	FIFO Queue	Min-Priority Queue (Heap)
Graph Transformations	None Required	Generates $w - 1$ Dummy Nodes	None Required
Time Complexity	$O(V + E)$	$O(V + \sum w)$	$O((V + E) \log V)$

Q23. Explain the intuitive idea behind the Bellman-Ford algorithm. How can you detect a negative cycle using Bellman-Ford? **(6+2 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Answer

1. The Intuitive Idea Behind Bellman-Ford

The Bellman-Ford algorithm is fundamentally rooted in dynamic programming and the principle of **edge relaxation**. Its core intuition revolves around the concept of *iterative path expansion* or information "rippling."

- **Edge Relaxation:** The primary mechanism is checking whether an edge (u, v) offers a shortcut to v via u :

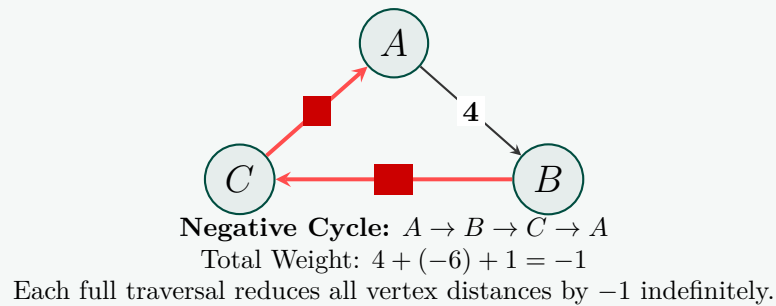
$$d[v] = \min(d[v], d[u] + w(u, v))$$

- **The Ripple Effect:** Instead of selectively tracing single paths like Depth-First Search (DFS) or expanding greedily by immediate distance like Dijkstra's, Bellman-Ford simply sweeps through **every single edge** in the graph globally.
 - After **1 pass** of relaxing all edges, the algorithm guarantees it has discovered the absolute shortest paths for all vertices that are exactly 1 edge away from the source.
 - After **2 passes**, it guarantees the shortest paths for all vertices at most 2 edges away.
 - After **k passes**, it guarantees the optimal paths restricted to at most k edges.

- **The Strict Upper Bound:** In any valid graph containing $|V|$ vertices, a simple path (a path that does not revisit any vertex) can contain at most $|V| - 1$ edges. Therefore, looping through the relaxation step exactly $|V| - 1$ times guarantees that the shortest-path information has rippled across the longest possible simple path in the graph.

2. Visualizing a Negative Cycle Contraction

The diagram below shows how a negative-weight cycle causes path distances to continually drop on successive passes, preventing stabilization.



3. Detecting a Negative Cycle

A **negative-weight cycle** is a reachable loop whose aggregate edge weight sum is less than zero. If a graph contains such a cycle, a true "shortest path" cannot exist because an algorithm could loop infinitely to drive the path cost down toward $-\infty$. Bellman-Ford exploits the Pigeonhole Principle to detect these cycles using a simple mathematical check:

- The Stability Property:** If the graph is free of negative cycles, all shortest paths must stabilize by the end of the $(|V| - 1)$ -th edge relaxation pass.
- The Extra Verification Pass:** The algorithm executes exactly **one extra pass** (the $|V|$ -th iteration) over all edges.
- The Condition:** If any edge (u, v) can still be relaxed such that:

$$d[u] + w(u, v) < d[v]$$

- The Conclusion:** A successful relaxation on the $|V|$ -th pass means there exists a path containing $|V|$ edges that is shorter than any path containing $|V| - 1$ edges. By the Pigeonhole Principle, a path with $|V|$ edges traversing $|V|$ unique vertices must contain a cycle. Because traversing this cycle lowered the overall path cost, the cycle's total weight **must be negative**.

4. The Complete Bellman-Ford Algorithm with Detection

Algorithm 9 Bellman-Ford Shortest Paths and Negative Cycle Detection

Require: Graph $G = (V, E)$ with edge weights w , source vertex s

Ensure: Distance array d , or an error if a negative cycle is detected

```

1: Initialize  $d[v] \leftarrow \infty$  for all  $v \in V$ ;  $d[s] \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $|V| - 1$  do                                     ▷ Core  $|V| - 1$  relaxation loops
3:   for each edge  $(u, v) \in E$  do
4:     if  $d[u] + w(u, v) < d[v]$  then
5:        $d[v] \leftarrow d[u] + w(u, v)$ 
6:     end if
7:   end for
8: end for
9: for each edge  $(u, v) \in E$  do                                       ▷  $|V|$ -th verification loop
10:  if  $d[u] + w(u, v) < d[v]$  then
11:    return "Error: Graph contains a negative-weight cycle"
12:  end if
13: end for
14: return  $d$ 

```

Q24. Write Dijkstra's algorithm for the single-source shortest path problem. Prove its correctness. Analyze the time complexity of the algorithm. What would be the running time if a Fibonacci heap is used instead of a binary heap? **(5+5+7+5 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Answer

1. Dijkstra's Algorithm

Algorithm 10 Dijkstra's Single-Source Shortest Path**Require:** Graph $G = (V, E)$ with non-negative edge weights w , source s **Ensure:** Array d where $d[v]$ is the shortest distance from s to v

```

1: Function Dijkstra( $G, w, s$ ):
2:   Initialize  $d[v] \leftarrow \infty$  for all  $v \in V$ 
3:    $d[s] \leftarrow 0$ 
4:    $S \leftarrow \emptyset$  ▷ Set of explored vertices
5:   Initialize Min-Priority Queue  $Q \leftarrow V$  keyed by  $d$  values
6:   While  $Q$  is not empty:
7:      $u \leftarrow Q.\text{Extract-Min}()$ 
8:      $S \leftarrow S \cup \{u\}$ 
9:     For each vertex  $v \in G.\text{Adj}[u]$ : ▷ Relaxation step
10:      If  $v \in Q$  and  $d[u] + w(u, v) < d[v]$ :
11:         $d[v] \leftarrow d[u] + w(u, v)$ 
12:         $Q.\text{Decrease-Key}(v, d[v])$ 
13:   Return  $d$ 

```

2. Proof of Correctness

We prove correctness by contradiction, showing that whenever a vertex u is extracted from Q and added to the set S , its distance estimate $d[u]$ is exactly the true shortest path distance $\delta(s, u)$.

- **Hypothesis:** For all $v \in S$, $d[v] = \delta(s, v)$.
- **Base Case:** Initially, $S = \emptyset$. When s is extracted, $d[s] = 0 = \delta(s, s)$, which is trivially correct.
- **Inductive Step:** Suppose there exists a first vertex u such that $d[u] > \delta(s, u)$ at the moment it is extracted from Q and added to S .
 - (a) Let P be the true shortest path from s to u .
 - (b) Since $s \in S$ and $u \notin S$ just before u is extracted, path P must cross the boundary from S to $V - S$. Let (x, y) be the first edge on P where $x \in S$ and $y \notin S$.
 - (c) Because $x \in S$, by our hypothesis, $d[x] = \delta(s, x)$.
 - (d) When x was added to S , its outgoing edges were relaxed. Therefore, $d[y] \leq d[x] + w(x, y) = \delta(s, y)$.
 - (e) Because edge weights are strictly non-negative ($w \geq 0$), the shortest path distance cannot decrease as we move further along P . Thus, $\delta(s, y) \leq \delta(s, u)$.
 - (f) Combining steps 4 and 5: $d[y] \leq \delta(s, u)$.
 - (g) However, the algorithm chose to extract u from Q instead of y , meaning $d[u] \leq d[y]$.
 - (h) Therefore, $d[u] \leq d[y] \leq \delta(s, u) \implies d[u] \leq \delta(s, u)$.
- **Contradiction:** We assumed $d[u] > \delta(s, u)$, but logically derived $d[u] \leq \delta(s, u)$. Thus, $d[u]$ must equal $\delta(s, u)$ when u is added to S . The algorithm is correct.

3. Time Complexity Analysis (Binary Heap)

Assuming the Min-Priority Queue Q is implemented using a standard **Binary Min-Heap**:

- **Initialization:** Setting initial distances and populating the heap takes $O(|V|)$ time.
- **Extract-Min Operations:** The **while** loop executes exactly $|V|$ times. Each **Extract-Min** operation on a binary heap takes $O(\log |V|)$ time. Total time for extractions: $O(|V| \log |V|)$.
- **Relaxation (Decrease-Key):** The inner **for** loop traverses the adjacency list of every vertex exactly once, meaning it runs $|E|$ times overall. If a distance is updated, **Decrease-Key** takes $O(\log |V|)$ time. Total maximum time for all updates: $O(|E| \log |V|)$.

Summing these up, the overall time complexity is $O(|V| \log |V| + |E| \log |V|) = O((|V| + |E|) \log |V|)$.

4. Time Complexity using a Fibonacci Heap

If a **Fibonacci Heap** is used for the priority queue, the time complexities of the fundamental operations change:

- **Extract-Min** remains $O(\log |V|)$ amortized time. Across $|V|$ operations, this takes $O(|V| \log |V|)$.
- **Decrease-Key** drops significantly to $O(1)$ amortized time. Across up to $|E|$ edge relaxations, this takes just $O(|E|)$ time.

Therefore, replacing the binary heap with a Fibonacci heap improves the overall time complexity to strictly $O(|V| \log |V| + |E|)$. This makes a massive theoretical difference for dense graphs where $|E|$ is much larger than $|V|$.

Q25. Write the properties satisfied by shortest paths of a graph. Show by example that Dijkstra's algorithm gives wrong results for a graph with negative-weight edges. Write an algorithm that finds shortest paths in a DAG and analyze its time complexity. **(15)**

marks)

[CSE 207 | L-2/T-2 | 2016–17]

Answer**1. Properties of Shortest Paths**

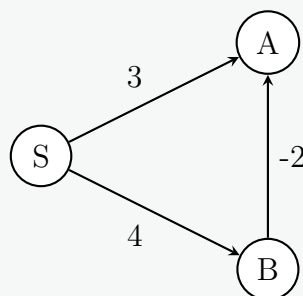
Let $G = (V, E)$ be a weighted directed graph with weight function $w : E \rightarrow \mathbb{R}$, and let $s \in V$ be the source vertex. The shortest path weight from s to v is denoted as $\delta(s, v)$, and the algorithm's current estimate is $d[v]$. The shortest paths satisfy the following fundamental properties:

- **Optimal Substructure Property:** If $p = \langle v_0, v_1, \dots, v_k \rangle$ is a shortest path from v_0 to v_k , then for any $0 \leq i \leq j \leq k$, the subpath $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ is a shortest path from v_i to v_j .
- **Triangle Inequality:** For any edge $(u, v) \in E$, we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.
- **Upper-Bound Property:** We always have $d[v] \geq \delta(s, v)$ for all vertices $v \in V$. Once $d[v]$ achieves the value $\delta(s, v)$, it never changes.
- **No-Path Property:** If there is no path from s to v , then $d[v] = \delta(s, v) = \infty$ always holds.
- **Convergence Property:** If $s \rightsquigarrow u \rightarrow v$ is a shortest path, and if $d[u] = \delta(s, u)$ at any time prior to relaxing edge (u, v) , then $d[v] = \delta(s, v)$ at all times afterward.
- **Path-Relaxation Property:** If $p = \langle v_0, v_1, \dots, v_k \rangle$ is a shortest path from $s = v_0$ to v_k , and we relax the edges of p in the order $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$, then $d[v_k] = \delta(s, v_k)$.

2. Dijkstra's Algorithm Failure with Negative-Weight Edges

Dijkstra's algorithm is a greedy algorithm that assumes once a vertex u is extracted from the priority queue (i.e., added to the set of finalized vertices S), its shortest path distance $d[u]$ is strictly optimal ($\delta(s, u)$). Negative weight edges break this greedy choice property.

Consider the following graph with source vertex S :

**Execution Trace of Dijkstra's Algorithm:**

(a) **Initialization:** $d[S] = 0, d[A] = \infty, d[B] = \infty$. Priority Queue $Q = \{(S, 0), (A, \infty), (B, \infty)\}$.

(b) **Extract Min (S):** S is permanently finalized. Relax outgoing edges (S, A) and (S, B) .

- $d[A] = \min(\infty, 0 + 3) = 3$.
- $d[B] = \min(\infty, 0 + 4) = 4$.

Q is now $\{(A, 3), (B, 4)\}$.

(c) **Extract Min (A):** A is permanently finalized with distance $d[A] = 3$. There are no outgoing edges from A .

(d) **Extract Min (B):** B is permanently finalized with distance $d[B] = 4$. Relax edge (B, A) .

- Attempt to update A : $d[A] = \min(3, 4 - 2) = 2$.

Since standard Dijkstra permanently finalizes a vertex upon extraction, the final distance to A reported by the algorithm is 3. However, the true shortest path is $S \rightarrow B \rightarrow A$ with a total cost of $4 + (-2) = 2$. The greedy assumption failed.

3. Shortest Paths in a Directed Acyclic Graph (DAG)

For a Directed Acyclic Graph, we can compute the single-source shortest paths in linear time by processing the vertices in topologically sorted order. Since all edges point from left to right in a topological sort, processing vertices in this order guarantees that by the time we process vertex v , all possible paths to v have already been explored.

Algorithm 11 DAG-Shortest-Paths(G, w, s)

```

1: Input: A DAG  $G = (V, E)$ , weight function  $w : E \rightarrow \mathbb{R}$ , source vertex  $s \in V$ 
2: Output: Array  $d$  containing shortest path weights, and array  $\pi$  for predecessor tree
3:
4: Phase 1: Topological Sort
5: Let  $L$  be an empty list
6: Perform Depth-First Search (DFS) on  $G$  to compute finishing times
7: As each vertex finishes, prepend it to  $L$ 
8:
9: Phase 2: Initialization
10: for each vertex  $v \in V$  do
11:    $d[v] \leftarrow \infty$ 
12:    $\pi[v] \leftarrow \text{NIL}$ 
13: end for
14:  $d[s] \leftarrow 0$ 
15:
16: Phase 3: Relaxation
17: for each vertex  $u$  in topological order  $L$  do
18:   if  $d[u] \neq \infty$  then
19:     for each vertex  $v \in \text{Adj}[u]$  do
20:       if  $d[v] > d[u] + w(u, v)$  then
21:          $d[v] \leftarrow d[u] + w(u, v)$ 
22:          $\pi[v] \leftarrow u$ 
23:       end if
24:     end for
25:   end if
26: end for
27: return  $d, \pi$ 

```

Complexity Analysis:

- **Time Complexity:** $\mathcal{O}(V + E)$
 - Topological sorting (using DFS) takes $\mathcal{O}(V + E)$ time.
 - Initializing the distance array d and predecessor array π takes $\mathcal{O}(V)$ time.
 - The outer relaxation loop runs exactly once per vertex ($\mathcal{O}(V)$). The inner loop iterates over all outgoing edges of u . Across the entire execution, each edge is relaxed exactly once. Thus, the relaxation phase takes $\mathcal{O}(V + E)$ time.
 - **Total Time:** $\mathcal{O}(V + E) + \mathcal{O}(V) + \mathcal{O}(V + E) = \mathcal{O}(V + E)$.
- **Space Complexity:** $\mathcal{O}(V + E)$
 - Adjacency list representation of the graph takes $\mathcal{O}(V + E)$ space.
 - Arrays d and π , and the list L for topological ordering take $\mathcal{O}(V)$ space.
 - Recursion stack for DFS takes up to $\mathcal{O}(V)$ space.
 - **Total Space:** $\mathcal{O}(V + E)$.

Q26. Prove that the Bellman-Ford algorithm returns TRUE if the input graph contains no negative-weight cycles reachable from the source, and FALSE otherwise. (10 marks) [CSE 207 | L-2/T-2 | 2016–17]

Answer**Theorem: Correctness of the Bellman-Ford Algorithm's Boolean Return Value**

Let $G = (V, E)$ be a weighted directed graph with source $s \in V$ and weight function $w : E \rightarrow \mathbb{R}$. The Bellman-Ford algorithm relaxes all edges $|V| - 1$ times and then performs a final verification pass over all edges. The algorithm returns **FALSE** if there exists an edge $(u, v) \in E$ such that $d[v] > d[u] + w(u, v)$, and **TRUE** otherwise.

We must prove two claims:

- (a) **Claim 1:** If G contains no negative-weight cycles reachable from s , the algorithm returns **TRUE**.
- (b) **Claim 2:** If G contains a negative-weight cycle reachable from s , the algorithm returns **FALSE**.

Proof of Claim 1: No negative-weight cycle reachable from $s \implies$ Returns TRUE

Assume that graph G contains no negative-weight cycles reachable from the source vertex s .

- By the **Path-Relaxation Property**, after relaxing all edges $|V| - 1$ times, the shortest-path estimate $d[v]$ correctly computes the true shortest-path distance $\delta(s, v)$ for all vertices v reachable from s .

- For any vertex v that is not reachable from s , $d[v] = \delta(s, v) = \infty$.
- Because there are no negative-weight cycles, the shortest paths are well-defined, and the true distances satisfy the **Triangle Inequality**. That is, for every edge $(u, v) \in E$:

$$\delta(s, v) \leq \delta(s, u) + w(u, v)$$

- Substituting the computed estimates d for δ , we get:

$$d[v] \leq d[u] + w(u, v) \quad \text{for all } (u, v) \in E$$

- Since this inequality holds for every edge, the condition $d[v] > d[u] + w(u, v)$ will evaluate to false during the final verification pass. Thus, the algorithm correctly returns **TRUE**.

Proof of Claim 2: A negative-weight cycle reachable from $s \implies$ Returns **FALSE**

Assume that graph G contains a negative-weight cycle $c = \langle v_0, v_1, \dots, v_k \rangle$ reachable from s , where $v_0 = v_k$. By the definition of a negative-weight cycle, the sum of the edge weights along the cycle is strictly less than zero:

$$\sum_{i=1}^k w(v_{i-1}, v_i) < 0 \quad (1)$$

We will prove this claim using a **Proof by Contradiction**.

- Assume, for the sake of contradiction, that the algorithm returns **TRUE**. This implies that for all edges in E , and specifically for all edges in the cycle c , the following inequality holds:

$$d[v_i] \leq d[v_{i-1}] + w(v_{i-1}, v_i) \quad \text{for } i = 1, 2, \dots, k$$

- Summing these inequalities around the cycle yields:

$$\sum_{i=1}^k d[v_i] \leq \sum_{i=1}^k (d[v_{i-1}] + w(v_{i-1}, v_i)) \quad (2)$$

- Because the cycle c is reachable from s , every vertex v_i in the cycle has a finite distance estimate $d[v_i] < \infty$ after $|V| - 1$ iterations.
- Since $v_0 = v_k$, the sum of the distance estimates $\sum_{i=1}^k d[v_i]$ is exactly the same as $\sum_{i=1}^k d[v_{i-1}]$.
- Because these sums are finite, we can algebraically subtract $\sum_{i=1}^k d[v_i]$ from both sides of the inequality, leaving:

$$0 \leq \sum_{i=1}^k w(v_{i-1}, v_i) \quad (3)$$

Contradiction: Inequality (3) states that the sum of the edge weights in cycle c is non-negative, which directly contradicts our initial premise in Equation (1) that c is a negative-weight cycle.

Therefore, our assumption that the algorithm returns **TRUE** must be false. There must be at least one edge (u, v) in the cycle c for which $d[v] > d[u] + w(u, v)$. Consequently, the algorithm correctly detects this and returns **FALSE**. ■

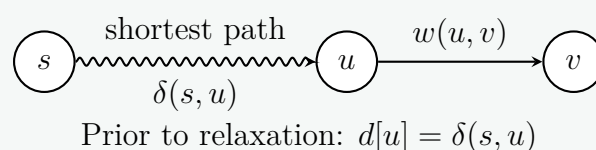
Q27. State and prove the convergence property of edge relaxation in the context of shortest paths. (8 marks) [CSE 207 | L-2/T-2 | 2014–15]

Answer

Convergence Property of Edge Relaxation

Statement:

Let $G = (V, E)$ be a weighted directed graph with source vertex $s \in V$ and weight function $w : E \rightarrow \mathbb{R}$. Let $s \rightsquigarrow u \rightarrow v$ be a shortest path in G for some vertices $u, v \in V$. If $d[u] = \delta(s, u)$ at any time prior to relaxing the edge (u, v) , then $d[v] = \delta(s, v)$ at all times after the relaxation.



Proof:

- (a) **Optimal Substructure:** Because $s \rightsquigarrow u \rightarrow v$ is a shortest path from s to v , it must be that the subpath $s \rightsquigarrow u$ is a shortest

path from s to u . Therefore, the shortest path weight to v can be expressed as:

$$\delta(s, v) = \delta(s, u) + w(u, v) \quad (1)$$

- (b) **Pre-condition:** By the hypothesis of the property, at some point before the edge (u, v) is relaxed, we have achieved the true shortest path distance for u :

$$d[u] = \delta(s, u) \quad (2)$$

- (c) **Relaxation Step:** The operation of relaxing the edge (u, v) performs the following update:

$$d[v] = \min(d[v], d[u] + w(u, v)) \quad (3)$$

- (d) **Substitution:** Substituting Equation (2) into the relaxation step yields:

$$d[v] = \min(d[v], \delta(s, u) + w(u, v)) \quad (4)$$

Using Equation (1), this simplifies directly to:

$$d[v] = \min(d[v], \delta(s, v)) \quad (5)$$

- (e) **Upper-Bound Property:** By the fundamental Upper-Bound Property of shortest paths, the current estimate $d[v]$ is always strictly bounded below by the true shortest path distance. That is, $d[v] \geq \delta(s, v)$ holds at all times during any valid shortest path algorithm.

- (f) **Conclusion:** Because $d[v] \geq \delta(s, v)$, the minimum in Equation (5) will always evaluate to exactly $\delta(s, v)$. Therefore, immediately after the relaxation step, we have:

$$d[v] = \delta(s, v)$$

Furthermore, because of the Upper-Bound Property, once $d[v]$ reaches the true lower bound $\delta(s, v)$, it can never be reduced further. Thus, $d[v] = \delta(s, v)$ remains true at all times thereafter. ■

Q28. Provide a correctness proof of Dijkstra's algorithm. (10 marks)

[CSE 207 | L-2/T-2 | 2014–15]

Answer

Correctness Proof of Dijkstra's Algorithm

Dijkstra's algorithm finds the single-source shortest paths on a directed or undirected graph $G = (V, E)$ with non-negative edge weights ($w(u, v) \geq 0$ for all $(u, v) \in E$).

Let s be the source vertex. The algorithm maintains a set S of vertices whose final shortest-path weights from the source have already been determined. At each step, it greedily selects a vertex $u \in V \setminus S$ with the minimum shortest-path estimate $d[u]$, adds u to S , and relaxes all edges leaving u .

Theorem (Loop Invariant)

At the start of each iteration of the **while** loop, $d[v] = \delta(s, v)$ for all vertices $v \in S$.

Proof by Mathematical Induction

We prove the correctness of the loop invariant by induction on the size of the set S .

Initialization: Initially, $S = \emptyset$. The invariant holds vacuously since there are no vertices in S . (Alternatively, after the very first step, $S = \{s\}$ and $d[s] = 0 = \delta(s, s)$, which is correct because edge weights are non-negative, so no path can have a total weight less than 0).

Maintenance: Suppose the invariant holds for the current set S . Let u be the next vertex chosen from $V \setminus S$ to be added to S , meaning $d[u] = \min_{z \in V \setminus S} \{d[z]\}$. To prove the invariant continues to hold, we must show that $d[u] = \delta(s, u)$.

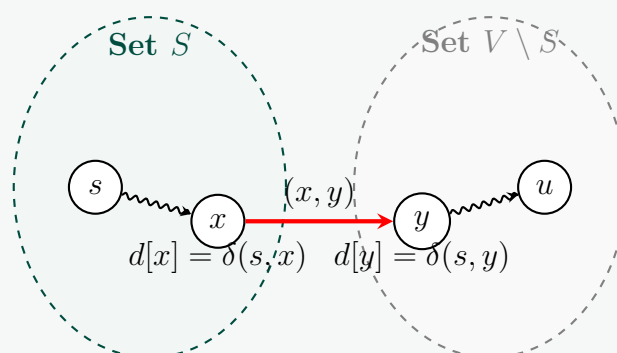
We prove this by contradiction. Assume that $d[u] \neq \delta(s, u)$.

- By the Upper-Bound Property of shortest paths, $d[u] \geq \delta(s, u)$ always.
- Therefore, our assumption implies that $d[u] > \delta(s, u)$.

Consider a true shortest path p from s to u . Since $s \in S$ and $u \in V \setminus S$, there must be a first edge on path p that connects a vertex in S to a vertex in $V \setminus S$. Let this edge be (x, y) , such that $x \in S$ and $y \in V \setminus S$. Note that x could potentially be s , and y could potentially be u .

The path p can be decomposed as:

$$s \xrightarrow{p_1} x \rightarrow y \xrightarrow{p_2} u$$



Let us analyze the relationships along this shortest path:

- (a) Since $x \in S$, by the inductive hypothesis, its distance estimate is already correct:

$$d[x] = \delta(s, x) \quad (1)$$

- (b) When vertex x was added to S , the edge (x, y) was relaxed. By the **Convergence Property**, immediately after relaxing (x, y) , the distance estimate to y became optimal:

$$d[y] = \delta(s, y) \quad (2)$$

Since distance estimates are monotonically non-increasing, $d[y]$ remains equal to $\delta(s, y)$ at the current step.

- (c) Because y occurs before u on the optimal shortest path p , and given that all edge weights are non-negative ($w(e) \geq 0$), the shortest-path distance to y cannot exceed the shortest-path distance to u :

$$\delta(s, y) \leq \delta(s, u) \quad (3)$$

- (d) Combining our mathematical statements yields the following chain of inequalities:

$$\begin{aligned} d[y] &= \delta(s, y) && \text{(from Equation 2)} \\ &\leq \delta(s, u) && \text{(from Equation 3)} \\ &< d[u] && \text{(by our initial contradiction assumption } \delta(s, u) < d[u]) \end{aligned}$$

Therefore, we conclude that:

$$d[y] < d[u] \quad (4)$$

Contradiction:

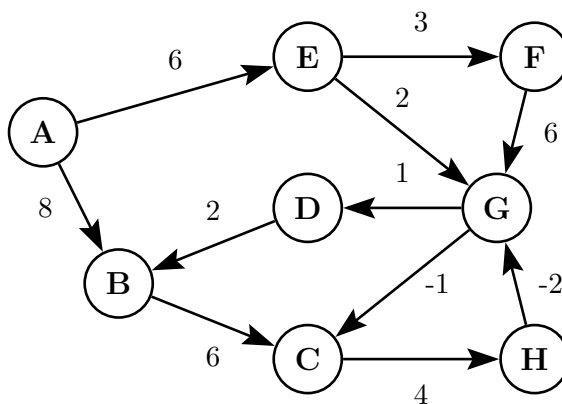
Equation (4) explicitly states that $d[y] < d[u]$. However, both y and u belong to $V \setminus S$, and the algorithm selected u because it possessed the *minimum* d value in $V \setminus S$ ($d[u] \leq d[z]$ for all $z \in V \setminus S$).

This is a direct contradiction. Thus, our initial assumption must be false, meaning $d[u]$ cannot be strictly greater than $\delta(s, u)$. We conclude that $d[u] = \delta(s, u)$, which completes the inductive maintenance step.

Termination

The algorithm terminates when $Q = V \setminus S = \emptyset$, meaning $S = V$. Since the loop invariant holds at the beginning of every iteration, it holds upon termination. Thus, for all vertices $v \in V$, $d[v] = \delta(s, v)$. ■

Q29. Apply the Bellman-Ford algorithm to find the shortest paths. Show all relaxation steps.



(10 marks)

[CSE 207 | L-2/T-2 | 2014–15]

Answer

1. Initialization and Assumptions

To apply the Bellman-Ford algorithm, we must designate a source vertex. In this graph, vertex **A** is the natural source as it has an in-degree of 0. We initialize the distance to the source as $d[A] = 0$, and all other vertices as $d[v] = \infty$.

Let V be the set of vertices and E be the set of edges. The graph has $|V| = 8$ vertices, so the algorithm will perform at most $|V| - 1 = 7$ relaxation passes over all edges to find the shortest paths. For deterministic and traceable updates, we will relax the edges in lexicographical order:

$$E = \{(A, B), (A, E), (B, C), (C, H), (D, B), (E, F), (E, G), (F, G), (G, C), (G, D), (H, G)\}$$

2. Edge Relaxation Steps

During each iteration k , we update $d[v] = \min(d[v], d[u] + w(u, v))$ sequentially for every edge $(u, v) \in E$.

Iteration (k)	A	B	C	D	E	F	G	H
0 (Init)	0	∞	∞	∞	∞	∞	∞	∞
1	0	8	7	9	6	9	8	18
2	0	8	7	9	6	9	8	11
3	0	8	7	9	6	9	8	11

Detailed Trace of Updates:

- **Iteration 1:**

- $(A, B, 8) \Rightarrow d[B] = \min(\infty, 0 + 8) = 8$
- $(A, E, 6) \Rightarrow d[E] = \min(\infty, 0 + 6) = 6$
- $(B, C, 6) \Rightarrow d[C] = \min(\infty, 8 + 6) = 14$
- $(C, H, 4) \Rightarrow d[H] = \min(\infty, 14 + 4) = 18$
- $(E, F, 3) \Rightarrow d[F] = \min(\infty, 6 + 3) = 9$
- $(E, G, 2) \Rightarrow d[G] = \min(\infty, 6 + 2) = 8$
- $(G, C, -1) \Rightarrow d[C] = \min(14, 8 - 1) = 7$ (Overwrites previous C value)
- $(G, D, 1) \Rightarrow d[D] = \min(\infty, 8 + 1) = 9$

- **Iteration 2:**

- $(C, H, 4) \Rightarrow d[H] = \min(18, 7 + 4) = 11$ (Updated because C changed in Pass 1)
- All other edges fail to find a strictly shorter path.

- **Iteration 3:**

- No distance values change during this pass. Because a full pass resulted in zero updates, the algorithm guarantees convergence and can safely **terminate early**.

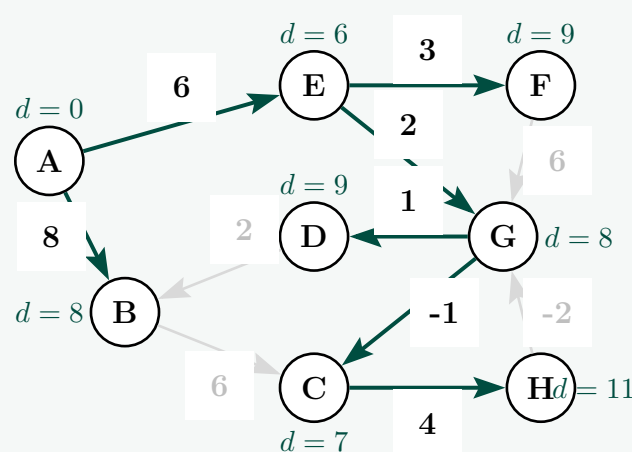
Since we completed a full pass without any changes, we also verified that the graph contains **no negative-weight cycles** reachable from A.

3. Final Shortest Path Tree

The final shortest path distances from source A are:

$$d[A] = 0, \quad d[B] = 8, \quad d[C] = 7, \quad d[D] = 9, \quad d[E] = 6, \quad d[F] = 9, \quad d[G] = 8, \quad d[H] = 11$$

The Shortest Path Tree (SPT) is visualized below. The active edges forming the tree are highlighted in solid blue, while the unused edges are grayed out.



4. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|V| \cdot |E|)$. In the worst case, the algorithm iterates over all $|E|$ edges $|V| - 1$ times. Since $|V| = 8$ and $|E| = 11$, the maximum number of relaxations is $(8 - 1) \times 11 = 77$. Early termination optimizes this in practice, but the asymptotic bound remains $\mathcal{O}(|V||E|)$.
- **Space Complexity:** $\mathcal{O}(|V|)$. The algorithm requires space to maintain the distance array d and predecessor array π , both of size $|V|$. Representing the graph requires $\mathcal{O}(|V| + |E|)$ space.

Answer

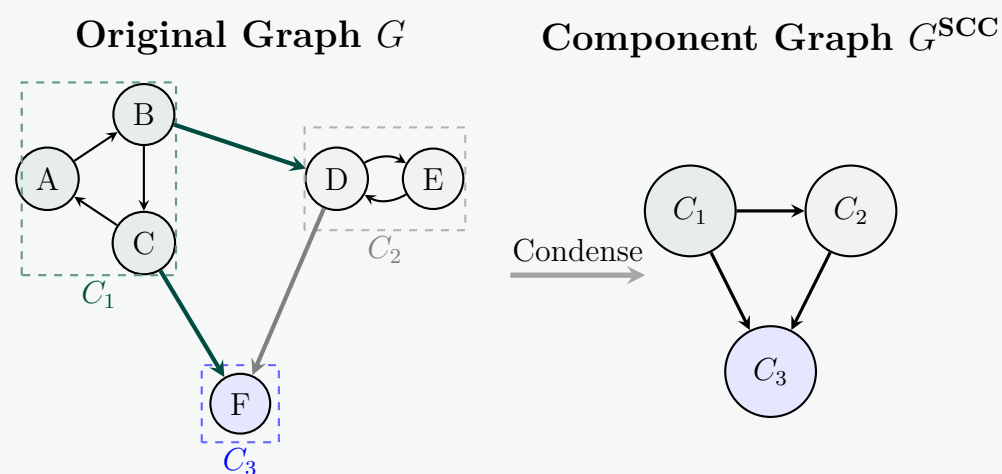
1. Formal Definitions

Let $G = (V, E)$ be a directed graph (digraph).

- **Strongly Connected Component (SCC):** A strongly connected component of G is a maximal subset of vertices $V_i \subseteq V$ such that for every pair of vertices $u, v \in V_i$, there exists a directed path from u to v and a directed path from v to u . In other words, mutual reachability is an equivalence relation that partitions V into disjoint sets $V_1, V_2, \dots, V_k$.
- **Component Graph (Condensation Graph):** The component graph of G , denoted by $G^{\text{SCC}} = (V^{\text{SCC}}, E^{\text{SCC}})$, is a directed graph defined as follows:
 - (a) $V^{\text{SCC}} = \{C_1, C_2, \dots, C_k\}$, where each C_i corresponds to a strongly connected component of G .
 - (b) A directed edge $(C_i, C_j) \in E^{\text{SCC}}$ exists if and only if $i \neq j$ and there exists at least one directed edge $(u, v) \in E$ in the original graph such that $u \in C_i$ and $v \in C_j$.

2. Visualization: Graph Condensation

Below is an illustration of a directed graph G and its corresponding component graph G^{SCC} , demonstrating how mutually reachable cycles collapse into a single macro-node, eliminating cyclical paths.



3. Theorem and Formal Proof

Theorem: The component graph $G^{\text{SCC}} = (V^{\text{SCC}}, E^{\text{SCC}})$ of any directed graph G is a Directed Acyclic Graph (DAG).

Proof (By Contradiction):

Assume, for the sake of contradiction, that the component graph G^{SCC} is *not* acyclic. This implies that G^{SCC} contains at least one directed cycle.

- (a) Let this directed cycle in G^{SCC} consists of m distinct macro-nodes ($m \geq 2$, since by definition, G^{SCC} has no self-loops):

$$C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow \dots \rightarrow C_m \rightarrow C_1 \quad (1)$$

- (b) By the definition of edges in a component graph, the existence of the edge $C_i \rightarrow C_{i+1}$ means there exists a directed path in the original graph G from some vertex in C_i to some vertex in C_{i+1} . Let us denote this path in G as $P(C_i, C_{i+1})$.
- (c) Similarly, the final back-edge $C_m \rightarrow C_1$ in the cycle implies there exists a directed path in G from some vertex in C_m to some vertex in C_1 . Let us denote this path as $P(C_m, C_1)$.
- (d) Within any single strongly connected component C_k , *all* internal vertices are mutually reachable by definition. Therefore, we can stitch these internal paths together with the inter-component paths $P(C_i, C_{i+1})$ to construct a valid path between any two vertices belonging to the union of these components.
- (e) Specifically, let u be any arbitrary vertex in C_1 and v be any arbitrary vertex in C_2 :
 - We can travel from u to the exit vertex of C_1 , follow $P(C_1, C_2)$ to enter C_2 , and then reach v . Thus, there is a path $u \rightsquigarrow v$ in G .
 - Conversely, to go from v to u , we can travel through $C_2 \rightarrow C_3 \rightarrow \dots \rightarrow C_m \rightarrow C_1$ using the path segments guaranteed by the cycle. Thus, there is a path $v \rightsquigarrow u$ in G .
- (f) Because $u \rightsquigarrow v$ and $v \rightsquigarrow u$ both exist in G , the vertices u and v are mutually reachable in the original graph. By definition, they must belong to the **same** strongly connected component.
- (g) This logic applies uniformly to all vertices across all components $C_1, C_2, \dots, C_m$ in the cycle. Therefore, all these vertices belong to a single, larger strongly connected component.

Contradiction:

This conclusion directly contradicts the maximality property of strongly connected components, which states that $C_1, C_2, \dots, C_m$ are distinct, disjoint maximal subsets partitioned by the equivalence relation.

Since our assumption that a cycle exists leads to a direct mathematical contradiction, G^{SCC} cannot contain any directed cycles. Thus, the component graph G^{SCC} must be a Directed Acyclic Graph (DAG). ■

Q31. Assume that a breadth-first tree has already been computed with BFS on a graph G with source vertex s . Write an algorithm to print the vertices on a shortest path from s to v . (8 marks) [CSE 207 | L-2/T-2 | 2012–13]

Answer

1. Algorithmic Strategy

When a Breadth-First Search (BFS) is executed on a graph G from a source vertex s , it computes a shortest-path distance $d[v]$ and a predecessor pointer $\pi[v]$ for each vertex $v \in V$. The predecessor pointers implicitly form a **Breadth-First Tree** rooted at s .

To print the vertices along the shortest path from s to v , we can follow these predecessor pointers backward from v to s . Since this gives the path in reverse order ($v \rightsquigarrow s$), we can use a recursive approach to leverage the call stack, printing the vertices only after the recursive call returns. This naturally reverses the traversal order, ensuring the path prints correctly from s to v .

2. Algorithm Specification

Below is the standard, elegant recursive algorithm to print the shortest path.

Algorithm 12 Print-BFS-Path(G, s, v)

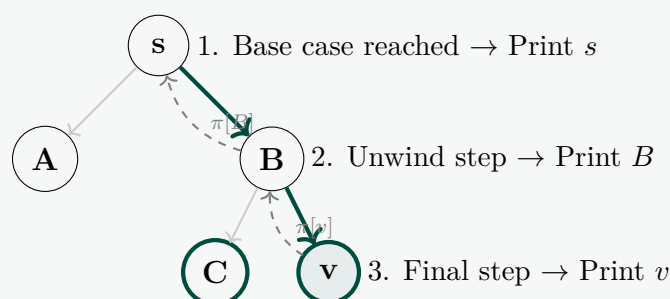
```

1: Input: Graph  $G$ , source vertex  $s$ , target vertex  $v$ , and a precomputed predecessor array  $\pi$ 
2: Output: Prints the vertices on the shortest path from  $s$  to  $v$ 
3:
4: if  $v = s$  then
5:   print  $s$                                      ▷ Base Case: Source vertex reached
6: else
7:   if  $\pi[v] = \text{NIL}$  then
8:     print "No path from "  $s$  " to "  $v$  " exists."       ▷ Target is unreachable
9:   else
10:    PRINT-BFS-PATH( $G, s, \pi[v]$ )                     ▷ Recurse toward the source
11:    print  $v$                                            ▷ Print vertex during unwind phase
12:   end if
13: end if

```

3. Visual Trace via Tree Diagram

Consider a precomputed Breadth-First Tree where we wish to print the path from source s to target v . The algorithm moves up the tree via π pointers until it hits s , then prints while unwinding.



Output Sequence for the example above: $s \rightarrow B \rightarrow v$

4. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(V)$

- In the worst-case scenario, the shortest path could be a simple linear chain containing all vertices in the graph (i.e., a path length of $|V| - 1$).
- Each recursive step performs a constant number of operations $\mathcal{O}(1)$ before making exactly one recursive call.
- Therefore, the total time spent is directly proportional to the number of vertices on the path, yielding a tight upper bound of $\mathcal{O}(V)$.

- **Space Complexity:** $\mathcal{O}(V)$

- The algorithm does not allocate additional dynamic structural memory, running in $\mathcal{O}(1)$ auxiliary space.
- However, because it is implemented recursively, it uses the system call stack. In the worst case (a single linear path of length $|V| - 1$), the maximum depth of the recursion tree is $|V|$, requiring $\mathcal{O}(V)$ stack frame space.

Q32. State and prove the convergence property of shortest paths. (12 marks)

[CSE 207 | L-2/T-2 | 2012–13]

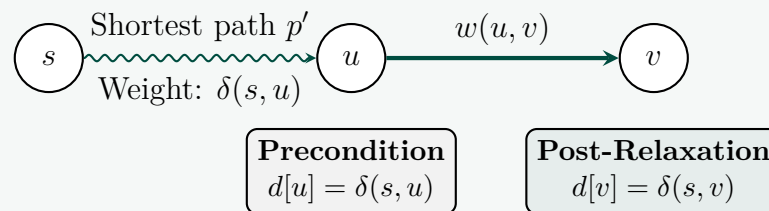
Answer

Convergence Property of Shortest Paths

Formal Statement:

Let $G = (V, E)$ be a weighted directed graph with weight function $w : E \rightarrow \mathbb{R}$, and let $s \in V$ be the source vertex. Let $s \rightsquigarrow u \rightarrow v$ be a shortest path in G for some vertices $u, v \in V$.

If the shortest-path estimate for u has converged, meaning $d[u] = \delta(s, u)$, at any time prior to relaxing the edge (u, v) , then the shortest-path estimate for v will converge to its true optimum, meaning $d[v] = \delta(s, v)$, at all times after the relaxation of (u, v) .



Formal Proof:

To prove this property rigorously, we rely on two fundamental axioms of shortest paths:

- (a) **Optimal Substructure Lemma:** Subpaths of shortest paths are shortest paths.
- (b) **Upper-Bound Property:** For all $v \in V$, $d[v] \geq \delta(s, v)$ always. Once $d[v]$ achieves $\delta(s, v)$, it never changes.

Step-by-Step Derivation:

- (a) **Optimal Substructure Application:**

By the hypothesis, the path $s \rightsquigarrow u \rightarrow v$ is a true shortest path from s to v . By the Optimal Substructure Lemma, the subpath $s \rightsquigarrow u$ must also be a true shortest path from s to u . Therefore, the total shortest-path distance to v can be exactly decomposed as:

$$\delta(s, v) = \delta(s, u) + w(u, v) \quad (1)$$

- (b) **The Precondition:**

The property assumes that at some moment before edge (u, v) is relaxed, the algorithm has successfully found the optimal distance to u . Thus:

$$d[u] = \delta(s, u) \quad (2)$$

- (c) **The Relaxation Operation:**

The definition of relaxing the directed edge (u, v) is executing the following assignment:

$$d[v] \leftarrow \min(d[v], d[u] + w(u, v)) \quad (3)$$

- (d) **Substitution:**

Substitute the exact value of $d[u]$ from Equation (2) into the relaxation step (Equation 3):

$$d[v] \leftarrow \min(d[v], \delta(s, u) + w(u, v)) \quad (4)$$

Now, apply the Optimal Substructure decomposition from Equation (1) into Equation (4):

$$d[v] \leftarrow \min(d[v], \delta(s, v)) \quad (5)$$

- (e) **Applying the Upper-Bound Property:**

The Upper-Bound Property guarantees that the algorithm's estimate $d[v]$ never falls below the true shortest-path distance $\delta(s, v)$. That means at the moment of relaxation, $d[v]$ is either strictly greater than $\delta(s, v)$ or exactly equal to it:

$$d[v] \geq \delta(s, v)$$

Because $d[v] \geq \delta(s, v)$, the minimum evaluated in Equation (5) will trivially resolve to $\delta(s, v)$.

- (f) **Conclusion:**

Therefore, immediately after executing the relaxation of edge (u, v) , we have:

$$d[v] = \delta(s, v)$$

Furthermore, the second clause of the Upper-Bound Property dictates that once $d[v]$ reaches the absolute lower bound $\delta(s, v)$, no future edge relaxations can possibly reduce it further. Thus, the condition $d[v] = \delta(s, v)$ will persist at all times afterward, completing the proof. ■

Q33. Write the Bellman-Ford algorithm. Analyze the time complexity and correctness of the Bellman-Ford algorithm. (20 marks) [CSE 207 | L-2/T-2 | 2012–13]

Answer**1. The Bellman-Ford Algorithm**

The Bellman-Ford algorithm solves the single-source shortest-paths problem in the general case where edge weights may be negative. Unlike Dijkstra's algorithm, it can handle graphs containing negative-weight edges, provided there are no negative-weight cycles reachable from the source. It returns a boolean value indicating whether a negative-weight cycle was detected.

Algorithm 13 Bellman-Ford(G, w, s)

```

1: Input: A weighted directed graph  $G = (V, E)$ , weight function  $w : E \rightarrow \mathbb{R}$ , source vertex  $s \in V$ 
2: Output: Boolean (TRUE if no negative cycles, FALSE otherwise), updates  $d$  and  $\pi$  arrays
3:
4: Step 1: Initialize Single Source
5: for each vertex  $v \in V$  do
6:    $d[v] \leftarrow \infty$ 
7:    $\pi[v] \leftarrow \text{NIL}$ 
8: end for
9:  $d[s] \leftarrow 0$ 
10:
11: Step 2: Relax Edges Repeatedly
12: for  $i \leftarrow 1$  to  $|V| - 1$  do
13:   for each edge  $(u, v) \in E$  do
14:     if  $d[v] > d[u] + w(u, v)$  then
15:        $d[v] \leftarrow d[u] + w(u, v)$ 
16:        $\pi[v] \leftarrow u$ 
17:     end if
18:   end for
19: end for
20:
21: Step 3: Check for Negative-Weight Cycles
22: for each edge  $(u, v) \in E$  do
23:   if  $d[v] > d[u] + w(u, v)$  then
24:     return FALSE ▷ A negative-weight cycle is reachable
25:   end if
26: end for
27: return TRUE ▷ Shortest paths are correctly computed

```

2. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(V \cdot E)$
 - **Initialization (Step 1):** Takes $\mathcal{O}(V)$ time to set up the initial distance estimates and predecessors.
 - **Relaxation Loop (Step 2):** The outer loop runs exactly $|V| - 1$ times. The inner loop iterates over all edges $|E|$ in the graph. Thus, the relaxation phase dominates the algorithm, taking $(|V| - 1) \times \mathcal{O}(E) = \mathcal{O}(V \cdot E)$ time.
 - **Negative Cycle Check (Step 3):** Iterates through all edges in the graph exactly once, which takes $\mathcal{O}(E)$ time.
 - **Total Time Complexity:** $\mathcal{O}(V) + \mathcal{O}(V \cdot E) + \mathcal{O}(E) = \mathcal{O}(V \cdot E)$. On a dense graph where $E = \mathcal{O}(V^2)$, this translates to a worst-case time complexity of $\mathcal{O}(V^3)$.
- **Space Complexity:** $\mathcal{O}(V)$
 - The algorithm requires space to maintain two primary arrays of size $|V|$: the distance estimates array d and the predecessor tracking array π .
 - Since the graph structure itself takes $\mathcal{O}(V + E)$ space using an adjacency list format, the auxiliary runtime storage overhead remains tightly bounded at $\mathcal{O}(V)$.

3. Correctness Proof

To prove the correctness of the Bellman-Ford algorithm, we must demonstrate that it achieves two things: correctly computing shortest-path weights when no negative-weight cycles exist, and identifying negative-weight cycles when they do.

Part 1: In the absence of negative-weight cycles, $d[v] = \delta(s, v)$ for all reachable v

Lemma (Path-Relaxation Property):

Let $G = (V, E)$ be a weighted directed graph with source s . Suppose G contains no negative-weight cycles reachable from s . Let $p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from $s = v_0$ to v_k . If we relax the edges of p in the order $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$, then $d[v_k] = \delta(s, v_k)$ upon completion, and this equality holds regardless of any interleaved relaxation steps on other edges.

Proof by Mathematical Induction on Path Length:

- **Base Case:** Before the loop begins, Step 1 sets $d[v_0] = d[s] = 0 = \delta(s, s)$. By the upper-bound property, this value can never change.

- **Inductive Step:** Assume that after the $(i - 1)$ -th pass of the relaxation loop, the algorithm has achieved the optimal value for vertex v_{i-1} , meaning $d[v_{i-1}] = \delta(s, v_{i-1})$.
- During the i -th iteration of the outer loop, *every* edge in the graph is relaxed, which includes the specific shortest-path edge (v_{i-1}, v_i) .
- By the **Convergence Property**, since $d[v_{i-1}] = \delta(s, v_{i-1})$ held prior to this relaxation pass, once the edge (v_{i-1}, v_i) is processed, we are guaranteed that $d[v_i] = \delta(s, v_i)$.
- Since a simple path in a graph with no negative-weight cycles contains at most $|V| - 1$ edges ($k \leq |V| - 1$), the $|V| - 1$ passes performed in Step 2 are mathematically sufficient to ensure every edge along every simple shortest path is relaxed in its correct structural sequence.

Thus, after $|V| - 1$ iterations, $d[v] = \delta(s, v)$ for all reachable vertices $v \in V$.

Part 2: Correctness of the Boolean Return Value

We complete the proof by establishing that Step 3 acts as a perfect classifier for the presence of a reachable negative-weight cycle.

- **Case A (No Negative Cycles Exist):**

If the graph contains no reachable negative cycles, Part 1 guarantees that $d[v] = \delta(s, v)$ for all reachable vertices upon entering Step 3. By the **Triangle Inequality**, the true shortest paths must obey $\delta(s, v) \leq \delta(s, u) + w(u, v)$ for every edge $(u, v) \in E$. Substituting the optimal d estimates, we obtain $d[v] \leq d[u] + w(u, v)$. Therefore, the condition in Step 3 ($d[v] > d[u] + w(u, v)$) will evaluate to false for all edges, and the algorithm correctly returns **TRUE**.

- **Case B (A Negative Cycle is Reachable):**

Suppose there exists a negative-weight cycle $c = \langle v_0, v_1, \dots, v_m \rangle$ where $v_0 = v_m$, reachable from s . Assume for contradiction that the algorithm returns **TRUE**. This implies that Step 3 found no edge violating the triangle inequality, meaning for all edges in the cycle:

$$d[v_j] \leq d[v_{j-1}] + w(v_{j-1}, v_j) \quad \text{for } j = 1, \dots, m$$

Summing these inequalities around the cycle yields:

$$\sum_{j=1}^m d[v_j] \leq \sum_{j=1}^m d[v_{j-1}] + \sum_{j=1}^m w(v_{j-1}, v_j)$$

Because the cycle is reachable, all $d[v_j]$ values are finite. Since $v_0 = v_m$, the vertex sum on the left and right sides are identical ($\sum d[v_j] = \sum d[v_{j-1}]$). Subtracting this finite sum from both sides leaves:

$$0 \leq \sum_{j=1}^m w(v_{j-1}, v_j)$$

This states that the cycle has a non-negative total weight, which directly contradicts our premise that c is a negative-weight cycle. Thus, the condition in Step 3 must be triggered by at least one edge in the cycle, and the algorithm correctly returns **FALSE**. ■

Q34. Does Dijkstra's algorithm terminate if it is applied on a graph G that contains a negative-weight cycle? Why or why not? Justify your answer. (6 marks) [CSE 207 | L-2/T-2 | 2012–13]

Answer

Termination of Dijkstra's Algorithm on Negative-Weight Cycles

Direct Answer:

Yes, the standard textbook implementation of Dijkstra's algorithm **is guaranteed to terminate** when applied to a graph containing a negative-weight cycle. However, while it terminates successfully, the computed shortest-path distances will be **incorrect** for any vertices reachable from or trapped within that negative cycle.

1. Why It Terminates (Control Flow Analysis)

The termination of Dijkstra's algorithm is governed entirely by its structural control flow and data structures, completely independent of edge weight values:

- **Finite Queue Capacity:** Upon initialization, the algorithm inserts exactly $|V|$ vertices of the graph G into the priority queue Q .
- **Strict Extraction Policy:** In every single iteration of the main **while** loop, the algorithm performs exactly one **Extract-Min** operation to remove the vertex u with the smallest distance estimate $d[u]$ from Q .
- **No Re-enqueuing:** Crucially, once a vertex is extracted from Q , it is considered "settled." The standard implementation of Dijkstra's algorithm **never re-inserts** a settled vertex back into the priority queue. Edge relaxation only modifies the keys (**Decrease-Key**) of vertices that are *currently* still waiting inside Q .

Because the priority queue starts with exactly $|V|$ elements and loses exactly one element per iteration, the main loop will execute

exactly $|V|$ times before Q becomes empty, forcing the algorithm to terminate.

2. Why It Fails (Correctness Breakdown & Counterexample)

Dijkstra’s algorithm relies heavily on a **Greedy Choice Property**: it assumes that the moment a vertex u is pulled from the priority queue, its current distance estimate $d[u]$ has successfully converged to the true shortest-path distance $\delta(s, u)$. When a negative-weight cycle is introduced, this assumption shatters.

Consider the following graph with source s and a negative cycle between vertices A and B :

```
graph LR; s((s)) -- 2 --> A((A)); A -- 3 --> B((B)); B -. -5 .-> A;
```

$d[s] = 0$ $d[A] = 0$ (Incorrect) $d[B] = 5$ (Incorrect)

Execution Trace:

- (a) **Initialization:** $d[s] = 0, d[A] = \infty, d[B] = \infty$. Queue $Q = \{s, A, B\}$.
- (b) **Iteration 1:** Extract s ($d[s] = 0$). Relax edge $(s, A) \implies d[A] = \min(\infty, 0 + 2) = 2$. Remaining $Q = \{A, B\}$.
- (c) **Iteration 2:** Extract A ($d[A] = 2$). Vertex A is now permanently popped and settled. Relax edge $(A, B) \implies d[B] = \min(\infty, 2 + 3) = 5$. Remaining $Q = \{B\}$.
- (d) **Iteration 3:** Extract B ($d[B] = 5$). Vertex B is settled. Relax edge $(B, A) \implies$ the updated path weight to A would be $d[B] + w(B, A) = 5 - 5 = 0$. Since $0 < d[A]$ (current $d[A] = 2$), a shorter path to A is discovered! However, because A is no longer in Q , the algorithm cannot re-evaluate A or propagate this improvement to downstream paths. Queue $Q = \emptyset$, loop terminates.

The mathematical shortest-path distance to A and B is $-\infty$ due to the infinitely repeating negative cycle $\langle A, B, A \rangle$, but Dijkstra’s algorithm exits prematurely with the incorrect finite metrics $d[A] = 0$ and $d[B] = 5$.

3. Summary of Behavior

Algorithmic Metric	Behavior on Graphs with Negative Cycles
Termination	Guaranteed. Stops cleanly after exactly $ V $ loop iterations.
Correctness	Fails. Distances for vertices affected by the cycle will be incorrect.
Infinite Loop Risk	Only occurs in non-standard variants that explicitly re-enqueue settled vertices.
Correct Alternative	Bellman-Ford Algorithm , which safely catches and handles these cycles.

■

Q35. What does $d[v]$ represent at the end of the execution of Algorithm 1, if the algorithm is applied on an unweighted directed acyclic graph (DAG) G with a source s ? What is the runtime complexity in terms of $|V|$ and $|E|$ of Algorithm 1?

Algorithm 14 LONGESTPATH-DAG(G)

```
1: Compute a topological sorting of  $G$ 
2: for each  $v \in V[G]$  do
3:    $d[v] \leftarrow 0$ 
4: end for
5: for each  $u$  in topologically sorted order do
6:   for each  $v \in Adj[u]$  do
7:     if  $d[v] < d[u] + 1$  then
8:        $d[v] \leftarrow d[u] + 1$ 
9:     end if
10:  end for
11: end for
```

(6+6 marks)

[CSE 207 | L-2/T-2 | 2012–13]

Answer

1. Representation of $d[v]$ at Execution End

At the end of the execution of Algorithm 1, $d[v]$ represents the ****length of the longest path (measured by the total number of edges) from the source vertex s to the vertex v ****.

Mathematical Justification:

- **Initialization:** The algorithm initially sets $d[v] = 0$ for all $v \in V$. Since G is a DAG with a single source s (meaning s is the unique vertex with an in-degree of 0 and all other vertices are reachable from it), the absolute longest path to any vertex

must originate at s .

- **Topological Order Invariant:** Processing the vertices in a topologically sorted order ensures a strict dependency guarantee: when the outer loop reaches a vertex u , all possible incoming paths from preceding vertices to u have already been completely evaluated. Thus, $d[u]$ is guaranteed to have settled to its final maximum value.
- **Relaxation Step:** The conditional update:

$$d[v] = \max(d[v], d[u] + 1)$$

correctly propagates and accumulates the maximum edge count along any valid directed sequence from s to v .

2. Runtime Complexity Analysis

The total runtime complexity of the algorithm is $\mathcal{O}(|V| + |E|)$, which is linear with respect to the size of the graph.

Step-by-Step Breakdown:

- Topological Sorting (Line 1):** Computing a topological sort of a directed acyclic graph using either Kahn's algorithm (indegree-based) or a Depth-First Search (DFS) tracking finish times takes exactly $\mathcal{O}(|V| + |E|)$ time.
- Initialization Loop (Lines 2–4):** This loop iterates exactly once for each vertex in the graph to initialize the distance estimates to zero, contributing $\mathcal{O}(|V|)$ time.
- Nested Relaxation Loops (Lines 5–11):**

- The outer loop (Line 5) visits every vertex $u \in V$ exactly once.
- The inner loop (Line 6) iterates over all outgoing edges in the adjacency list of the current vertex u , denoted as $\text{Adj}[u]$.
- Summing the work done across all vertices, the inner loop processes exactly the total number of edges in the graph:

$$\sum_{u \in V} \text{out-degree}(u) = |E|$$

- Since the comparison and assignment operations inside the conditional block (Lines 7–9) take constant time $\mathcal{O}(1)$, the total time spent in these nested loops is tightly bounded at $\mathcal{O}(|V| + |E|)$.

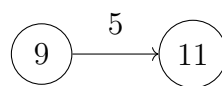
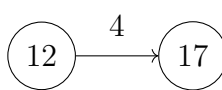
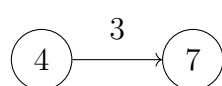
Total Asymptotic Time Complexity:

Combining all the sequential steps of the algorithm yields:

$$\mathcal{O}(|V| + |E|) [\text{Sorting}] + \mathcal{O}(|V|) [\text{Init}] + \mathcal{O}(|V| + |E|) [\text{Relaxation}] = \mathcal{O}(|V| + |E|)$$

■

Q36. In the figure below, the shortest path estimate of each vertex appears within the vertex and the distance to traverse from a vertex to another appears as the weight of the corresponding directed edge. Show the updated shortest path estimates of vertices after relaxing the following edges.



(6 marks)

[CSE 207 | L-2/T-2 | 2012–13]

Answer

1. Edge Relaxation Principle

Relaxing a directed edge (u, v) with a given weight $w(u, v)$ checks whether a shorter path to vertex v can be found by traveling through vertex u . The mathematical operation is defined as:

$$d[v] \leftarrow \min(d[v], d[u] + w(u, v))$$

An update occurs only if the new path estimate is strictly less than the existing estimate $(d[u] + w(u, v) < d[v])$.

2. Step-by-Step Relaxation Trace

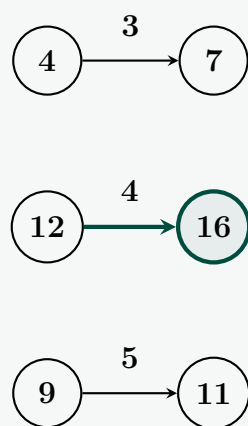
- **Relaxing edge $(4) \rightarrow (7)$ with weight 3:**

- Source estimate $d[u] = 4$, Target estimate $d[v] = 7$.

- New path candidate: $d[4] + w(4, 7) = 4 + 3 = 7$.
- Since $7 \not< 7$, no update occurs. **Final value:** $d[7] = 7$.
- **Relaxing edge (12) $\rightarrow$ (17) with weight 4:**
 - Source estimate $d[u] = 12$, Target estimate $d[v] = 17$.
 - New path candidate: $d[12] + w(12, 17) = 12 + 4 = 16$.
 - Since $16 < 17$, the shortest path estimate drops. **Final value:** $d[17] = 16$.
- **Relaxing edge (9) $\rightarrow$ (11) with weight 5:**
 - Source estimate $d[u] = 9$, Target estimate $d[v] = 11$.
 - New path candidate: $d[9] + w(9, 11) = 9 + 5 = 14$.
 - Since $14 \not< 11$, no update occurs. **Final value:** $d[11] = 11$.

3. Updated Graph State Diagram

Below is the updated configuration of the graph. The successfully relaxed edge and its updated destination estimate node are highlighted.



Q37. Explain how Dijkstra's algorithm can be implemented using a priority queue. Find the running time for this algorithm. **(15 marks)**
[CSE 207 | L-2/T-2 | 2011–12]

Answer

1. Implementing Dijkstra's Algorithm with a Priority Queue

Dijkstra's algorithm relies on a greedy approach: at every step, it extracts the unvisited vertex with the smallest tentative shortest-path distance. A **Priority Queue** (Q) is the ideal data structure to support this operation efficiently, storing vertices as elements and their current distance estimates ($d[v]$) as keys.

The algorithm maps directly to three primary priority queue operations:

- **Insert(Q, v, key):** Executed during initialization to populate the queue with all vertices $|V|$.
- **Extract-Min(Q):** Executed at the beginning of each iteration of the main loop to extract the vertex u with the absolute lowest $d[u]$ value.
- **Decrease-Key($Q, v, \text{new_key}$):** Executed during edge relaxation when a shorter path to a neighbor v is discovered, forcing its key to be lowered and its position re-balanced within the queue.

Algorithm 15 Dijkstra-PQ(G, w, s)

```

1: Input: Graph  $G = (V, E)$ , edge weight function  $w$ , source vertex  $s$ 
2:
3: for each vertex  $v \in V$  do
4:    $d[v] \leftarrow \infty$ 
5:    $\pi[v] \leftarrow \text{NIL}$ 
6: end for
7:  $d[s] \leftarrow 0$ 
8:
9: Initialize an empty priority queue  $Q$ 
10: for each vertex  $v \in V$  do
11:   INSERT( $Q, v, d[v]$ )
12: end for
13:
14: while  $Q$  is not empty do
15:    $u \leftarrow \text{EXTRACT-MIN}(Q)$  ▷ Greedily pick closest vertex
16:   for each vertex  $v \in \text{Adj}[u]$  do
17:     if  $d[v] > d[u] + w(u, v)$  then
18:        $d[v] \leftarrow d[u] + w(u, v)$ 
19:        $\pi[v] \leftarrow u$ 
20:       DECREASE-KEY( $Q, v, d[v]$ ) ▷ Update priority in  $Q$ 
21:     end if
22:   end for
23: end while

```

2. Time Complexity Analysis

The total execution time of Dijkstra's algorithm is bounded by the operations performed on the priority queue. Let $|V|$ be the number of vertices and $|E|$ be the number of edges. The control flow guarantees exactly:

- $|V|$ **Insert** operations to build the initial queue structure.
- $|V|$ **Extract-Min** operations, since each vertex is extracted exactly once.
- At most $|E|$ **Decrease-Key** operations, since every edge is relaxed at most once during the execution.

The running time varies significantly based on the structural choice of the priority queue:

Case A: Unsorted Array implementation

- **Extract-Min** takes $\mathcal{O}(V)$ via a linear scan. Total for all extractions: $|V| \cdot \mathcal{O}(V) = \mathcal{O}(V^2)$.
- **Decrease-Key** takes $\mathcal{O}(1)$ via direct array index accessing. Total for all relaxations: $|E| \cdot \mathcal{O}(1) = \mathcal{O}(E)$.
- **Total Running Time:** $\mathcal{O}(V^2 + E) = \mathcal{O}(V^2)$. This implementation is highly efficient for *dense graphs* where $|E| \approx |V|^2$.

Case B: Binary Min-Heap implementation

- **Extract-Min** takes $\mathcal{O}(\log V)$ to restore heap order after root eviction. Total for all extractions: $\mathcal{O}(V \log V)$.
- **Decrease-Key** takes $\mathcal{O}(\log V)$ to bubble up an element. Total for all relaxations: $\mathcal{O}(E \log V)$.
- **Total Running Time:** $\mathcal{O}(V \log V + E \log V) = \mathcal{O}((V + E) \log V)$. This is the standard, practical implementation choice for general or *sparse graphs* ($|E| \ll |V|^2$).

Case C: Fibonacci Heap implementation

- **Extract-Min** takes $\mathcal{O}(\log V)$ amortized time. Total for all extractions: $\mathcal{O}(V \log V)$.
- **Decrease-Key** takes $\mathcal{O}(1)$ amortized time via structural pointer updates. Total for all relaxations: $\mathcal{O}(E)$.
- **Total Running Time:** $\mathcal{O}(V \log V + E)$. This represents the theoretical optimal asymptotic upper bound for large graphs.

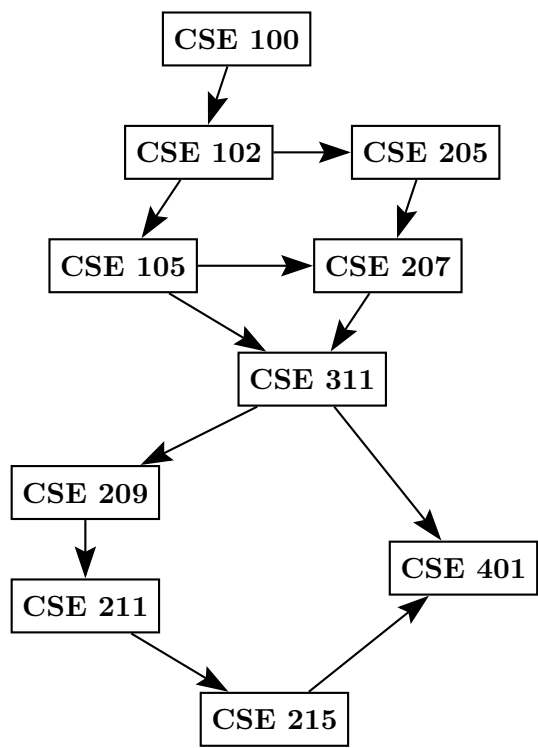
Summary Comparison

PQ Implementation	Extract-Min	Decrease-Key	Total Dijkstra Runtime
Unsorted Array	$\mathcal{O}(V)$	$\mathcal{O}(1)$	$\mathcal{O}(V^2)$
Binary Heap	$\mathcal{O}(\log V)$	$\mathcal{O}(\log V)$	$\mathcal{O}((V + E) \log V)$
Fibonacci Heap	$\mathcal{O}(\log V)$	$\mathcal{O}(1)$	$\mathcal{O}(V \log V + E)$

■

Q38. Find the topological order of courses for this given prerequisite graph. (10 marks)

[CSE 207 | L-2/T-2 | 2006–07]



Answer

Algorithm: Run DFS; upon finishing each vertex prepend it to the output list (or equivalently, push onto a stack and pop at the end).

In-degree method (Kahn's):

- (a) Compute in-degree of every vertex.
- (b) Enqueue all vertices with in-degree 0.
- (c) While queue non-empty: dequeue u , append to order, decrement in-degree of each neighbor; enqueue neighbor if in-degree becomes 0.

In-degrees:

Course	In-degree
CSE 100	0
CSE 102	1
CSE 205	1
CSE 105	1
CSE 207	2
CSE 311	2
CSE 209	1
CSE 211	1
CSE 215	1
CSE 401	2

Execution trace:

Step	Dequeued	Queue after
1	CSE 100	[CSE 102]
2	CSE 102	[CSE 205, CSE 105]
3	CSE 205	[CSE 105] (CSE 207 in-deg $\rightarrow$ 1)
4	CSE 105	[CSE 207] (CSE 207 in-deg $\rightarrow$ 0, CSE 311 in-deg $\rightarrow$ 1)
5	CSE 207	[CSE 311] (CSE 311 in-deg $\rightarrow$ 0)
6	CSE 311	[CSE 209] (CSE 209 in-deg $\rightarrow$ 0, CSE 401 in-deg $\rightarrow$ 1)
7	CSE 209	[CSE 211] (CSE 211 in-deg $\rightarrow$ 0)
8	CSE 211	[CSE 215] (CSE 215 in-deg $\rightarrow$ 0)
9	CSE 215	[CSE 401] (CSE 401 in-deg $\rightarrow$ 0)
10	CSE 401	[]

Topological Order:

CSE 100 $\rightarrow$ CSE 102 $\rightarrow$ CSE 205 $\rightarrow$ CSE 105 $\rightarrow$ CSE 207 $\rightarrow$ CSE 311 $\rightarrow$ CSE 209
 $\rightarrow$ CSE 211 $\rightarrow$ CSE 215 $\rightarrow$ CSE 401

Time complexity: $O(V + E)$.

Minimum Spanning Trees

Q1. Consider the following undirected weighted graph. Vertices: $\{A, B, C, D, E, F\}$. Edges: $A-B$ (1), $A-C$ (3), $A-D$ (4), $B-C$ (2), $B-D$ (6), $C-D$ (5), $C-E$ (7), $D-E$ (8), $E-F$ (9), $D-F$ (10), $C-F$ (11). Determine whether the minimum spanning tree (MST) of this graph is unique. Justify your answer rigorously using MST properties such as the cut property or cycle property. If the MST is not unique, construct at least two distinct MSTs with equal total cost. If it is unique, provide a formal justification explaining why no alternative MST with the same total weight can exist. **(20 marks)** [CSE 207 | L-2/T-1 | 2024–25]

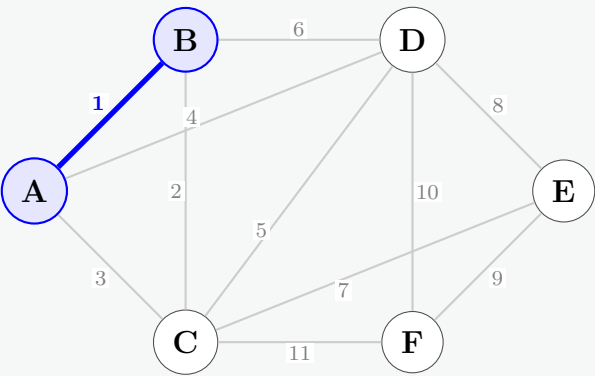
Answer

Prim’s Algorithm can be used to construct the Minimum Spanning Tree (MST) and rigorously determine its uniqueness. Since the cut property states that the strictly lightest edge crossing any cut must belong to every MST, finding a unique minimum edge at every step of Prim’s execution proves that the resulting MST is entirely unique.

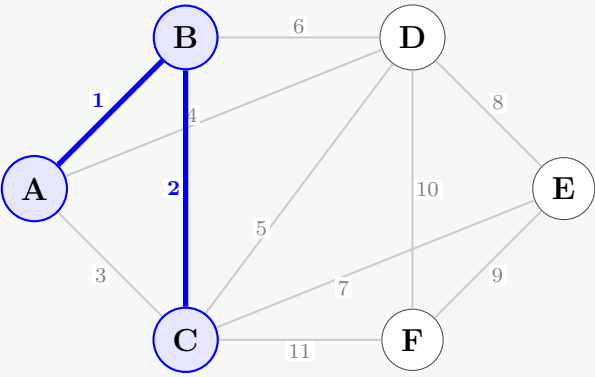
Let’s initialize the algorithm starting from vertex **A**.

- **Visited Set (V_{vis}):** Graph vertices currently included in the growing tree.
- **MST Edges (E_{mst}):** Edges selected for the final minimum spanning tree.

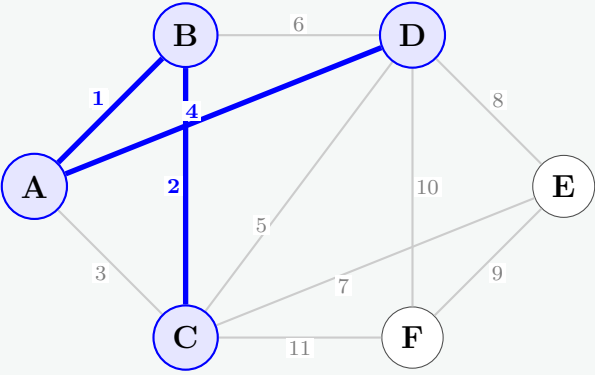
Step 1: Initialization
Start at vertex **A**. $V_{vis} = \{A\}$.
Available crossing edges from visited nodes: $(A, B) = 1, (A, C) = 3, (A, D) = 4$.
Action: The unique minimum weight is **1**. Add edge **(A, B)** and vertex **B**.



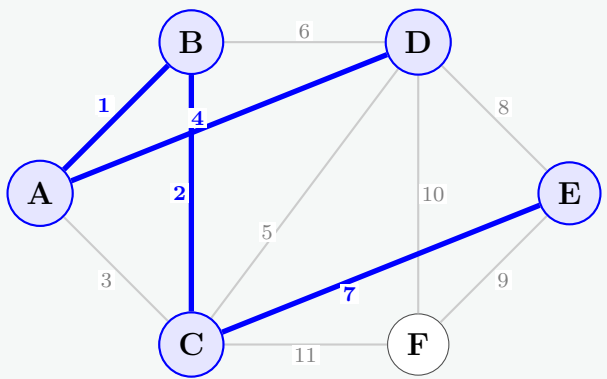
Step 2:
 $V_{vis} = \{A, B\}$.
Available crossing edges: $(B, C) = 2, (A, C) = 3, (A, D) = 4, (B, D) = 6$.
Action: The unique minimum weight is **2**. Add edge **(B, C)** and vertex **C**.



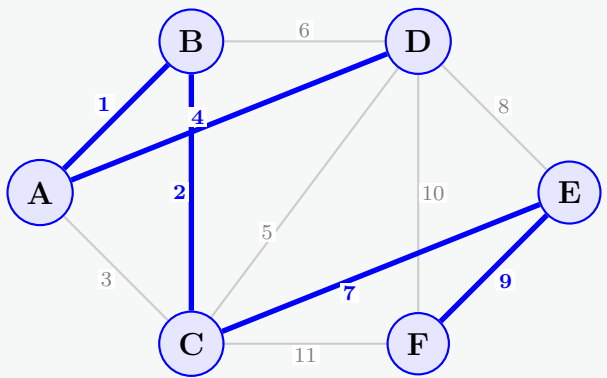
Step 3:
 $V_{vis} = \{A, B, C\}$.
Available crossing edges: $(A, D) = 4, (C, D) = 5, (B, D) = 6, (C, E) = 7, (C, F) = 11$.
Action: The unique minimum weight is **4**. Add edge **(A, D)** and vertex **D**.



Step 4:
 $V_{vis} = \{A, B, C, D\}$.
Available crossing edges: $(C, E) = 7, (D, E) = 8, (D, F) = 10, (C, F) = 11$.
Action: The unique minimum weight is **7**. Add edge **(C, E)** and vertex **E**.

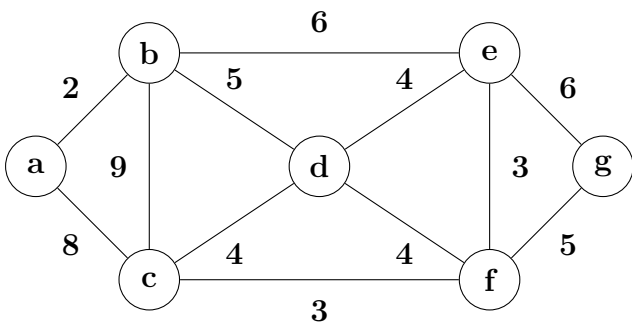


Step 5:
 $V_{vis} = \{A, B, C, D, E\}$.
Available crossing edges to unvisited node ‘F’: $(E, F) = 9, (D, F) = 10, (C, F) = 11$.
Action: The unique minimum weight is **9**. Add edge **(E, F)** and vertex **F**.



Rigorous Uniqueness Justification:
According to the **Cut Property of MSTs**, for any cut separating a subset of vertices from the rest of the graph, if there exists a single edge crossing that cut whose weight is strictly less than all other crossing edges, that edge *must* be included in any and all valid MSTs.
As tracked across Steps 1 through 5, the minimum crossing edge choice at every structural iteration was strictly unique (there were never any weight ties for the minimum boundary edge). Because each edge addition was mathematically mandatory and left no alternative options, ***the minimum spanning tree of this graph is completely unique***.
Conclusion: All vertices are securely spanned ($|V| - 1 = 5$ edges selected).
Unique MST Edges: $\{(A, B), (B, C), (A, D), (C, E), (E, F)\}$
Total MST Cost: $1 + 2 + 4 + 7 + 9 = \mathbf{23}$. ✓

Q2. Draw a minimum spanning tree of the graph shown in the figure using (i) Prim’s algorithm, and (ii) Kruskal’s algorithm. Write the order of the edges in each case (whenever there is a choice of vertices, always use alphabetical ordering).



(12 marks)

[CSE 207 | L-2/T-1 | 2023–24]

Answer

(i) Prim’s Algorithm

Assuming we start at vertex **a** (alphabetically the first vertex). At each step, we select the minimum weight edge connecting a visited vertex to an unvisited vertex. If there is a tie in edge weights, we choose the edge that connects to the alphabetically earlier unvisited vertex.

(a) Start at **a**. Available: $(a,b)=2, (a,c)=8$. **Select (a,b)**.

(b) Visited: $\{a,b\}$. Available: $(a,c)=8, (b,c)=9, (b,d)=5, (b,e)=6$. **Select (b,d)**.

(c) Visited: $\{a,b,d\}$. Available: $(a,c)=8, (b,c)=9, (b,e)=6, (c,d)=4, (d,e)=4, (d,f)=4$.
• Minimum weight is 4. Choices: $(c,d), (d,e), (d,f)$.
• Target unvisited vertices are c, e, f. Alphabetically, **c** comes first. **Select (c,d)**.

(d) Visited: $\{a,b,c,d\}$. Available crossing edges: $(c,f)=3, (d,e)=4, (d,f)=4, (b,e)=6$, etc. **Select (c,f)**.

(e) Visited: $\{a,b,c,d,f\}$. Available crossing edges: $(e,f)=3, (d,e)=4, (f,g)=5$, etc. **Select (e,f)**.

(f) Visited: {a,b,c,d,e,f}. Available crossing edges: (f,g)=5, (b,e)=6, (e,g)=6. **Select (f,g).**

Order of edges added (Prim's): (a, b), (b, d), (c, d), (c, f), (e, f), (f, g)

(ii) Kruskal's Algorithm

Kruskal's algorithm sorts all edges by weight and adds them one by one, provided they do not form a cycle. If weights are tied, we evaluate the edges in alphabetical order (e.g., (c,d) before (d,e)).

(a) Weight 2: **Select (a,b).** (No cycle)

(b) Weight 3: Choices are (c,f) and (e,f). Alphabetically evaluate (c,f) first. **Select (c,f).**

(c) Weight 3: **Select (e,f).** (No cycle)

(d) Weight 4: Choices are (c,d), (d,e), (d,f). Alphabetically evaluate (c,d) first. **Select (c,d).**

- Evaluate (d,e): Reject, forms cycle (c-d-e-f).
- Evaluate (d,f): Reject, forms cycle (c-d-f).

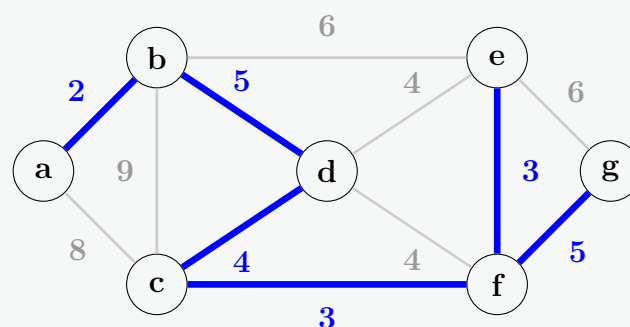
(e) Weight 5: Choices are (b,d) and (f,g). Alphabetically evaluate (b,d) first. **Select (b,d).**

(f) Weight 5: **Select (f,g).** (No cycle)

Order of edges added (Kruskal's): (a, b), (c, f), (e, f), (c, d), (b, d), (f, g)

Final Minimum Spanning Tree

Both algorithms produce the same unique Minimum Spanning Tree with a total cost of **22**.



Q3. Prove (i) the optimal sub-structure property and (ii) the greedy choice property of the Minimum Spanning Tree (MST) problem. [CSE 207 | L-2/T-2 | 2021–22]

(12 marks)

Answer

Introduction

To prove that an optimization problem can be solved correctly using a greedy strategy, we must establish two fundamental properties: **Optimal Substructure** and the **Greedy Choice Property**. Below are the formal proofs for both properties in the context of the Minimum Spanning Tree (MST) problem.

(i) Proof of the Optimal Substructure Property

Theorem Statement:

Let $G = (V, E)$ be a connected, undirected graph with weight function $w : E \rightarrow \mathbb{R}$, and let T be an MST of G . If an edge $e = (u, v) \in T$ is removed, the tree T decomposes into two disjoint subtrees, T_1 and T_2 . Let $V_1 \subset V$ be the vertices spanned by T_1 , and let $V_2 = V \setminus V_1$ be the vertices spanned by T_2 . Let G_1 and G_2 be the subgraphs of G induced by V_1 and V_2 , respectively. Then, T_1 is an MST for G_1 , and T_2 is an MST for G_2 .

Proof (by Contradiction):

The total weight of the minimum spanning tree T can be expressed as the sum of its constituent parts:

$$w(T) = w(T_1) + w(T_2) + w(e)$$

Suppose for the sake of contradiction that the optimal substructure property does not hold. Without loss of generality, assume that T_1 is *not* an MST for the induced subgraph G_1 .

Since T_1 is sub-optimal for G_1 , there must exist an alternative valid spanning tree T'_1 for G_1 that yields a strictly lower total cost:

$$w(T'_1) < w(T_1)$$

Now, let us construct a new spanning tree T^* for the original graph G by substituting T_1 with the superior alternative T'_1 , while keeping T_2 and the edge e exactly as they were:

$$T^* = T'_1 \cup T_2 \cup \{e\}$$

Since T'_1 spans all vertices in V_1 , T_2 spans all vertices in V_2 , and the single edge $e = (u, v)$ reconnects a vertex from V_1 to a vertex in V_2 , T^* constitutes a valid spanning tree for the entire graph G . Let us evaluate its total weight:

$$w(T^*) = w(T'_1) + w(T_2) + w(e)$$

By substituting our assumption $w(T'_1) < w(T_1)$ into the equation, we find:

$$w(T^*) < w(T_1) + w(T_2) + w(e) = w(T)$$

$$w(T^*) < w(T)$$

This result states that T^* is a spanning tree of G with a total cost strictly less than that of T . This directly contradicts our initial condition that T is a **Minimum Spanning Tree** of G .

Consequently, our assumption must be false. T_1 must be a valid MST for G_1 , and by symmetric logic, T_2 must be a valid MST for G_2 .

(ii) Proof of the Greedy Choice Property (The Cut Property)

Theorem Statement:

Let $G = (V, E)$ be a connected, undirected graph with weight function $w : E \rightarrow \mathbb{R}$. Let $S \subset V$ define a cut $(S, V \setminus S)$ of the graph. If $e = (u, v) \in E$ is a minimum-weight edge crossing the cut $(S, V \setminus S)$ (where $u \in S$ and $v \in V \setminus S$), then there exists an MST T of G that contains the edge e .

Proof (by Exchange Argument):

Let T' be an arbitrary MST of G . We evaluate two possible scenarios:

- **Case 1:** $e \in T'$

If the minimum-weight crossing edge e is already included within T' , the greedy choice property is trivially satisfied, and we set $T = T'$.

- **Case 2:** $e \notin T'$

If e is not present in T' , we will construct an alternative spanning tree T that contains e and matches the optimal weight of T' .

Because T' is a spanning tree, a unique path P must exist within T' connecting vertex u to vertex v . Since $u \in S$ and $v \in V \setminus S$, this path P must cross the boundary of the cut $(S, V \setminus S)$ at least once.

Let $e' = (u', v')$ be an edge on this path P in T' that crosses the cut $(S, V \setminus S)$ such that $u' \in S$ and $v' \in V \setminus S$.

If we temporarily insert our greedy edge e into T' , we create a unique simple cycle $C = P \cup \{e\}$. This cycle contains both e and e' . To break the cycle while maintaining connectivity, we remove the edge e' , producing a new edge set:

$$T = T' \cup \{e\} \setminus \{e'\}$$

Since we broke the single cycle created by adding an edge to a tree, T is guaranteed to be a valid, connected spanning tree of G .

Let us calculate the total weight of this newly constructed spanning tree T :

$$w(T) = w(T') + w(e) - w(e')$$

By the problem definition, e is a *minimum-weight edge* crossing the cut $(S, V \setminus S)$. Since e' is an edge crossing the exact same cut, its weight must be greater than or equal to that of e :

$$w(e) \leq w(e') \implies w(e) - w(e') \leq 0$$

Substituting this inequality back into the weight equation for T yields:

$$w(T) \leq w(T')$$

However, because T' is a Minimum Spanning Tree, its weight is already minimal by definition, meaning $w(T)$ cannot be strictly less than $w(T')$. Therefore, we must have:

$$w(T) = w(T')$$

This equality demonstrates that the newly constructed spanning tree T is also an MST of G . Since T explicitly includes the greedy edge e , we have proven that making the local greedy choice is safe and always preserves the path to a globally optimal solution. ■

Q4. Develop a solution of the MST problem using disjoint set / union-find data structure. Use the array implementation to maintain the union-find data structure. (15 marks) [CSE 207 | L-2/T-2 | 2021–22]

Answer

1. Array-Based Union-Find Data Structure

To develop a solution for the Minimum Spanning Tree (MST) problem using a disjoint-set data structure, we implement **Kruskal's Algorithm**. The disjoint-set (Union-Find) efficiently tracks which vertices belong to which connected components, preventing cycle formation when adding edges.

The array implementation utilizes two primary arrays of size $|V|$:

- **parent[i]**: Stores the representative (or parent) of vertex i . If **parent[i] == i**, vertex i is the root of its set.
- **rank[i]**: Stores an upper bound on the height of the tree rooted at i , used to keep trees shallow during union operations (*Union by Rank*).

Algorithm 16 Union-Find Array Operations

```

1: procedure MAKE-SET( $v$ )
2:   parent[ $v$ ]  $\leftarrow v$ 
3:   rank[ $v$ ]  $\leftarrow 0$ 
4: end procedure
5:
6: procedure FIND( $v$ )                                     ▷ With Path Compression
7:   if parent[ $v$ ]  $\neq v$  then
8:     parent[ $v$ ]  $\leftarrow$  FIND(parent[ $v$ ])               ▷ Attach directly to root
9:   end if
10:  return parent[ $v$ ]
11: end procedure
12:
13: procedure UNION( $u, v$ )                                 ▷ With Union by Rank
14:  root_u  $\leftarrow$  FIND( $u$ )
15:  root_v  $\leftarrow$  FIND( $v$ )
16:  if root_u  $\neq$  root_v then
17:    if rank[root_u] > rank[root_v] then
18:      parent[root_v]  $\leftarrow$  root_u
19:    else if rank[root_u] < rank[root_v] then
20:      parent[root_u]  $\leftarrow$  root_v
21:    else
22:      parent[root_v]  $\leftarrow$  root_u
23:      rank[root_u]  $\leftarrow$  rank[root_u] + 1
24:    end if
25:  end if
26: end procedure

```

2. Kruskal's MST Algorithm

Using the array-based Union-Find structure defined above, Kruskal's algorithm systematically processes edges in increasing order of weight, adding an edge to the MST only if it connects two previously disconnected components.

Algorithm 17 Kruskal-MST(G, w)

```

1: Input: Graph  $G = (V, E)$ , edge weight function  $w$ 
2:  $A \leftarrow \emptyset$                                      ▷ Initialize the set of MST edges
3:
4: for each vertex  $v \in V$  do
5:   MAKE-SET( $v$ )
6: end for
7:
8: Sort the edges of  $E$  into non-decreasing order by weight  $w$ 
9:
10: for each edge  $(u, v) \in E$ , taken in sorted order do
11:   if FIND( $u$ )  $\neq$  FIND( $v$ ) then                         ▷ If they are in different sets (no cycle)
12:      $A \leftarrow A \cup \{(u, v)\}$ 
13:     UNION( $u, v$ )                                       ▷ Merge the two sets
14:   end if
15: end for
16:
17: return  $A$ 

```

3. Time Complexity Analysis

The efficiency of this solution is determined by the sorting step and the union-find operations:

- **Sorting:** Sorting the $|E|$ edges requires $\mathcal{O}(E \log E)$ time. Since $|E| \leq |V|^2$, we know that $\log E \leq 2 \log V$, so this is equivalently $\mathcal{O}(E \log V)$.
- **Union-Find Operations:** We perform exactly $|V|$ **Make-Set** operations and at most $2|E|$ **Find** and $|V| - 1$ **Union** operations. Thanks to *Path Compression* and *Union by Rank*, a sequence of m disjoint-set operations takes $\mathcal{O}(m \cdot \alpha(V))$ time, where α is the extremely slow-growing inverse Ackermann function. Thus, the disjoint-set operations take $\mathcal{O}(E \cdot \alpha(V))$ time.

Because $\alpha(V)$ is practically a constant ≤ 4 for any reasonably sized graph, the sorting step dominates the overall running time.
Total Time Complexity: $\mathcal{O}(E \log E)$ or $\mathcal{O}(E \log V)$. ■

Q5. Let T be a minimum spanning tree of graph G obtained by Prim's algorithm. Let G_{new} be a graph obtained by adding to G a new vertex and some edges, with weights, connecting the new vertex to some vertices in G . Can you construct a minimum spanning tree of G_{new} by adding one of the new edges to T ? If your answer is yes, explain how; if your answer is no, explain why not. **(15 marks)**
 [CSE 207 | L-2/T-2 | 2020–21]

Answer

Answer: No.

Explanation:

You cannot simply construct the Minimum Spanning Tree (MST) of G_{new} by adding exactly one of the new edges to the existing MST T .

Adding a single new edge e_{new} to T forces you to keep *all* the original edges of T . While this will successfully create a valid spanning tree for G_{new} (since it connects the new vertex without forming cycles), it does not guarantee a *minimum* spanning tree.

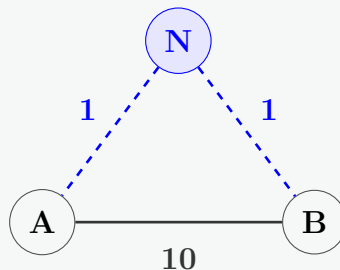
The introduction of the new vertex and its associated edges might provide a much cheaper "shortcut" between two vertices that already exist in G . To take advantage of this shortcut, the optimal strategy for G_{new} might require including **multiple** new edges and removing one or more heavy edges that were previously part of T .

Counterexample:

Consider a very simple graph G with just two vertices, A and B , connected by an edge of weight 10.

- The MST T of G is just the edge (A, B) . Total cost = **10**.

Now, let's create G_{new} by adding a new vertex N along with two new edges: (N, A) with weight 1, and (N, B) with weight 1.



Case 1: Adding exactly one new edge to T (The proposed method)

If we keep T (the edge (A, B)) and add exactly one new edge, say (N, A) , the resulting spanning tree consists of $\{(A, B), (N, A)\}$.

- Total cost = $10 + 1 = 11$.

Case 2: The actual MST of G_{new}

The true Minimum Spanning Tree of G_{new} should connect A , B , and N as cheaply as possible. We can do this by using both of the new edges and discarding the original edge (A, B) from T . The actual MST consists of $\{(N, A), (N, B)\}$.

- Total cost = $1 + 1 = 2$.

Conclusion: Since $11 \neq 2$, adding exactly one new edge to the original MST T fails to yield the MST of G_{new} .

Q6. Prove that if all edge weights of a graph are positive, then any subset of edges that connects all vertices and has minimum total weight must be a tree. Give an example to show that the same conclusion does not follow if we allow some weights to be nonpositive. **(12 marks)**
 [CSE 207 | L-2/T-2 | 2019–20]

Answer

Part (i): Proof for Positive Edge Weights

Theorem: If all edge weights of a connected graph $G = (V, E)$ are strictly positive ($w(e) > 0$ for all $e \in E$), then any subset of edges $E' \subseteq E$ that connects all vertices and minimizes the total weight $\sum_{e \in E'} w(e)$ must be a tree.

Proof (by Contradiction): Let E' be a subset of edges that connects all vertices in V and has the absolute minimum total weight among all such connecting subsets. By definition, the subgraph $G' = (V, E')$ is connected.

Suppose for the sake of contradiction that G' is *not* a tree. Since G' is connected and is not a tree, it must contain at least one simple cycle, which we will call C .

Let e_c be any edge belonging to this cycle C . Consider a new edge subset E'' formed by removing e_c from E' :

$$E'' = E' \setminus \{e_c\}$$

We now evaluate two essential properties of this new subset E'' :

- (a) **Connectivity:** In any graph, removing an edge from a cycle does not disconnect the graph. Any path in G' that originally utilized the edge $e_c = (u, v)$ can bypass it by traveling along the remaining path of the cycle $C \setminus \{e_c\}$ to get from u to v . Therefore, the subgraph $G'' = (V, E'')$ remains fully connected and still spans all vertices in V .

- (b) **Total Weight:** Let us calculate the total weight of the new connecting subset E'' :

$$w(E'') = \sum_{e \in E''} w(e) = \left(\sum_{e \in E'} w(e) \right) - w(e_c) = w(E') - w(e_c)$$

By the initial premise of the problem, all edge weights in the graph are strictly positive, meaning $w(e_c) > 0$. Substituting this into our weight equation yields:

$$w(E'') < w(E')$$

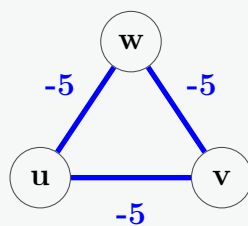
This inequality states that E'' is a subset of edges that completely connects all vertices in V and has a total weight strictly less than $w(E')$. This directly contradicts our initial assumption that E' was a connecting subset of **minimum** total weight.

Thus, our assumption must be false. The minimum-weight connecting subset E' cannot contain any cycles. A connected graph with no cycles is, by definition, a tree.

Part (ii): Counterexample for Nonpositive Edge Weights

If we allow edge weights to be nonpositive ($w(e) \leq 0$), a minimum-weight connecting subset does not have to be a tree. It will greedily include extra edges forming cycles because adding those edges either decreases or maintains the total weight while preserving connectivity.

Example (Negative Weights): Consider a graph G with 3 vertices $\{u, v, w\}$ forming a triangle where every edge has a negative weight of -5 .



Let us compare the possible subsets of edges that connect all three vertices:

- **Option 1 (Spanning Trees):** Any valid spanning tree must select exactly $|V| - 1 = 2$ edges to avoid cycles. For example, selecting edges $\{(u, v), (v, w)\}$ yields a total weight of:

$$(-5) + (-5) = -10$$

- **Option 2 (The Entire Cycle):** If we select all 3 edges $E' = \{(u, v), (v, w), (w, u)\}$, the vertices remain perfectly connected, and the total weight becomes:

$$(-5) + (-5) + (-5) = -15$$

Since $-15 < -10$, the unique subset of edges that connects all vertices and achieves the absolute minimum total weight is the entire cycle. Because this optimal subset contains a cycle, it is **not a tree**.

Conclusion: The requirement that edge weights must be strictly positive is necessary to ensure that the minimum connecting subgraph is structurally a tree. ■

Q7. Show that for each minimum spanning tree T of G , there is a way to sort the edges of G in Kruskal's algorithm so that the algorithm returns T . (Kruskal's algorithm can return different spanning trees for the same input graph G , depending on how it breaks ties when the edges are sorted.) **(10 marks)** [CSE 207 | L-2/T-2 | 2019–20]

Answer

Proof Strategy: To prove that Kruskal's algorithm can always return a specific Minimum Spanning Tree T , we must define a tie-breaking rule for sorting the edges. By strategically ordering edges of the same weight, we can force Kruskal's algorithm to favor the edges of T , thereby reconstructing T perfectly.

The Sorting Rule: Sort all edges in the graph G in non-decreasing order of their weights. If two edges have the exact same weight, **break the tie by placing the edge that belongs to T before the edge that does not belong to T** . (If both edges belong to T or neither belongs to T , their relative order does not matter).

Proof by Contradiction:

Let us run Kruskal's algorithm on the graph G using this specific sorted list of edges. Suppose, for the sake of contradiction, that the algorithm outputs a spanning tree T_K that is different from T ($T_K \neq T$).

Since both T and T_K are valid spanning trees (each containing $|V| - 1$ edges), there must exist at least one edge in T that Kruskal's algorithm rejected.

- Let $e = (u, v)$ be the *first* edge in our sorted list that belongs to T but is rejected by Kruskal's algorithm.
- Kruskal's algorithm only rejects an edge if adding it creates a cycle with the edges it has *already accepted*. Let A be the set of edges Kruskal's has accepted just before evaluating e . Adding e to A forms a cycle C .
- Because T is a tree, it contains no cycles. Therefore, not all edges of the cycle C can belong to T . There must be at least one edge in C that is not in T .
- Now, consider the original MST T . If we remove the edge e from T , it splits T into two disconnected components, V_1 and V_2 . This forms a cut (V_1, V_2) , and e is the *only* edge in T that crosses this cut.

- (e) Because C is a cycle that contains e (which connects V_1 and V_2), the cycle must cross the cut back over to close the loop. Therefore, there must be another edge $e'' \in C$ (where $e'' \neq e$) that also crosses the cut (V_1, V_2) .
- (f) Since e'' crosses the cut but e is the only edge in T that crosses this cut, it is strictly true that $e'' \notin T$.
- (g) Furthermore, since $e'' \in C \setminus \{e\}$, it must be part of the already-accepted set A . This means Kruskal's algorithm evaluated and accepted e'' *before* it even reached e . Thus, in our sorted list, e'' appears before e , implying:

$$w(e'') \leq w(e)$$

- (h) Let us evaluate a new spanning tree formed by replacing e with e'' in the optimal tree T :

$$T^* = T \setminus \{e\} \cup \{e''\}$$

The weight of this new tree is $w(T^*) = w(T) - w(e) + w(e'')$.

- (i) Since $w(e'') \leq w(e)$, it must be that $w(T^*) \leq w(T)$. However, T is a *Minimum Spanning Tree*, meaning its weight cannot be strictly beaten. Therefore, $w(T^*)$ must equal $w(T)$, which forces:

$$w(e'') = w(e)$$

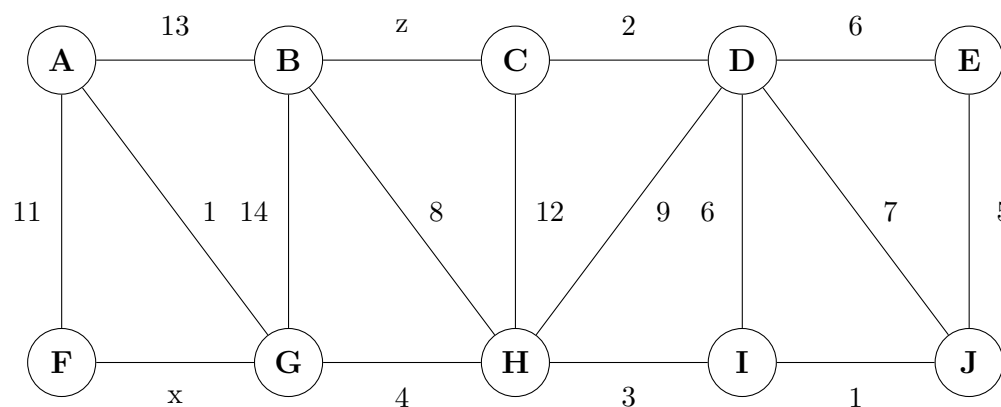
The Contradiction: We have established three facts:

- $w(e) = w(e'')$ (The edges have identical weights).
- $e \in T$ and $e'' \notin T$.
- e'' was evaluated by Kruskal's before e (meaning e'' appeared before e in our sorted list).

This is a direct contradiction! Our explicit tie-breaking rule states that if two edges have the same weight, the edge belonging to T (e) **must** appear before the edge not in T (e'').

Because our assumption leads to a logical impossibility, the assumption must be false. Kruskal's algorithm, using this tie-breaking sort, will **never** reject an edge of T . Since it will accept all $|V| - 1$ edges of T , it must perfectly reconstruct T . ■

Q8. Prove that an optimal minimum spanning tree is composed of optimal minimum spanning subtrees. Draw a minimum spanning tree of the graph G shown below such that the minimum spanning tree contains the edges with weights x and z .



(15 marks)

[CSE 207 | L-2/T-2 | 2018–19]

Answer

Part 1: Proof of Optimal Substructure Property

Theorem: An optimal Minimum Spanning Tree (MST) is composed of optimal minimum spanning subtrees.

Proof (by Contradiction):

Let $G = (V, E)$ be a connected, undirected graph with weight function w . Let T be an optimal MST of G .

Suppose we remove an arbitrary edge $e = (u, v)$ from T . This removal splits T into two disjoint subtrees, T_1 and T_2 , which span two disjoint sets of vertices, V_1 and V_2 , respectively.

We must prove that T_1 is an optimal MST for the subgraph G_1 (induced by V_1) and T_2 is an optimal MST for G_2 (induced by V_2).

Assume, for the sake of contradiction, that T_1 is **not** an optimal minimum spanning tree for G_1 . This means there must exist some alternative spanning tree T'_1 for G_1 that successfully connects all vertices in V_1 but has a strictly smaller total weight:

$$w(T'_1) < w(T_1)$$

Now, let us construct a new spanning tree T^* for the entire graph G by replacing T_1 with the superior subtree T'_1 and reconnecting it to T_2 using the original edge e :

$$T^* = T'_1 \cup T_2 \cup \{e\}$$

The total weight of this newly constructed tree T^* is:

$$w(T^*) = w(T'_1) + w(T_2) + w(e)$$

Since we established $w(T'_1) < w(T_1)$ in our assumption, we can substitute this inequality:

$$w(T^*) < w(T_1) + w(T_2) + w(e)$$

Because the right side of the inequality evaluates to the exact weight of our original MST T , we get:

$$w(T^*) < w(T)$$

This result states that T^* is a valid spanning tree for G with a strictly lower total weight than T . However, this directly contradicts our initial premise that T is the *Minimum Spanning Tree* of G .

Therefore, our assumption must be false. T_1 is guaranteed to be an optimal minimum spanning tree for G_1 , and by symmetric logic, T_2 is optimal for G_2 .

Part 2: Minimum Spanning Tree Construction

We need to construct an MST of G that guarantees the inclusion of the edges with weights x (edge **F-G**) and z (edge **B-C**). We can achieve this by initializing our MST with these two edges as forced inclusions, and then proceeding with Kruskal's algorithm for the remaining edges.

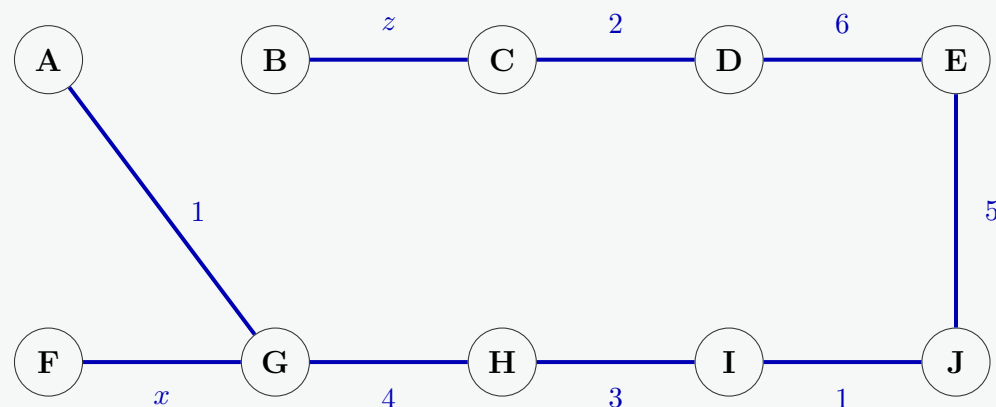
Step-by-Step Construction:

- Force inclusion:** Add edge **(F,G)** [weight x] and **(B,C)** [weight z].
- Sort remaining edges:** Sort by weight. If weights tie, the tie is broken using alphabetical ordering of the vertices.
- Apply Kruskal's:**
 - Weight 1:** Add **(A,G)** and **(I,J)**. (Neither forms a cycle).
 - Weight 2:** Add **(C,D)**.
 - Weight 3:** Add **(H,I)**.
 - Weight 4:** Add **(G,H)**.
 - Weight 5:** Add **(E,J)**.
 - Weight 6:** The candidate edges are **(D,E)** and **(D,I)**.
 - Following alphabetical ordering, we evaluate **(D,E)** first.
 - Adding **(D,E)** successfully connects the disconnected upper sub-component $\{B, C, D\}$ with the larger component $\{A, F, G, H, I, J, E\}$. No cycle is formed, so we add it.
 - At this exact point, all 10 vertices are connected by 9 edges. The spanning tree is complete, and any subsequent edge evaluations (like **(D,I)**) are bypassed or rejected as they would form cycles.

Final MST Edges & Weights:

(A,G) = 1, **(I,J)** = 1, **(C,D)** = 2, **(H,I)** = 3, **(G,H)** = 4, **(E,J)** = 5, **(D,E)** = 6, **(F,G)** = x , **(B,C)** = z .

Visual Diagram of the MST:



Q9. Write down an algorithm for constructing a mincost arborescence in a digraph from a given root vertex r . Apply the algorithm to the following digraph, showing every step in separate diagrams: $ra(7), rx(2), ax(1), xy(5), ya(3), xc(6), cb(4), bx(2), yb(2)$ (weights in parentheses). **(15 marks)** [CSE 203 | L-2/T-1 | 2017–18]

Answer

1. Algorithm Description

The standard university-level method for finding a minimum-cost arborescence in a directed graph is **Edmonds' Algorithm** (also known as the **Chu-Liu-Edmonds Algorithm**).

An arborescence rooted at a specified vertex r is a directed spanning tree in which every vertex $v \neq r$ has exactly one incoming edge, and there is a unique directed path from r to v .

Algorithm 18 Chu-Liu-Edmonds Algorithm

Input: A directed graph $G = (V, E)$ with edge weights $w : E \rightarrow \mathbb{R}$, and a root vertex $r \in V$.

Output: A minimum-cost arborescence A rooted at r .

- 1: Remove all self-loops from E and all incoming edges to the root r .
- 2: For each vertex $v \in V \setminus \{r\}$, select its minimum-weight incoming edge:

$$\pi(v) = \operatorname{argmin}_{(u,v) \in E} w(u, v)$$

- 3: Let $P = \{\pi(v) \mid v \in V \setminus \{r\}\}$ be the set of selected edges.

- 4: **if** P does not contain any directed cycles **then**

- 5: **return** $A = P$

▷ P is already the optimal arborescence

- 6: **else**

- 7: Select a directed cycle C in P .

- 8: Contract the vertices of C into a single pseudo-vertex v_C , creating a new graph $G' = (V', E')$.

- 9: Modify the weights of all edges entering the cycle: For each edge $e = (u, v) \in E$ with $u \notin C$ and $v \in C$, define its new weight as:

$$w'(u, v_C) = w(u, v) - w(\pi(v))$$

- 10: For edges leaving the cycle (u, v) where $u \in C, v \notin C$, retain their original weight: $w'(v_C, v) = w(u, v)$.

- 11: For edges entirely inside or between vertices of C , discard them.

- 12: $A' \leftarrow \text{Chu-Liu-Edmonds}(G', w', r)$

▷ Recursive call on the contracted graph

- 13: Find the unique edge $(u, v_C) \in A'$ that enters the pseudo-vertex v_C (corresponding to some original edge (u, v) where $v \in C$).

- 14: Construct A by expanding v_C : include all edges from A' , replace (u, v_C) with (u, v) , and include all edges of the cycle C except the unique edge $\pi(v)$ that enters v .

- 15: **return** A

- 16: **end if**

2. Complexity Analysis

- **Time Complexity:**

- **Naive Implementation:** In each contraction step, we spend $O(|E|)$ time to find minimum incoming edges and reconstruct the graph. Since each contraction reduces the number of vertices by at least one, there can be at most $O(|V|)$ contractions. Thus, the total time complexity is $O(|V| \cdot |E|)$.

- **Optimized Implementation:** By utilizing specialized priority queues (such as pairing heaps or Fibonacci heaps) along with a Disjoint-Set (Union-Find) data structure to maintain components, the time complexity can be optimized to $O(|E| \log |V|)$ or $O(|E| + |V| \log |V|)$.

- **Space Complexity:** $O(|V| + |E|)$ to store the graph representations, edge weight modifications, and the recursive contraction history tree required for the expansion phase.

3. Step-by-Step Application to the Given Digraph

The given digraph has the vertex set $V = \{r, a, x, y, c, b\}$ with r as the root. The edges and weights are:

$$(r, a) : 7, \quad (r, x) : 2, \quad (a, x) : 1, \quad (x, y) : 5, \quad (y, a) : 3, \quad (x, c) : 6, \quad (c, b) : 4, \quad (b, x) : 2, \quad (y, b) : 2$$

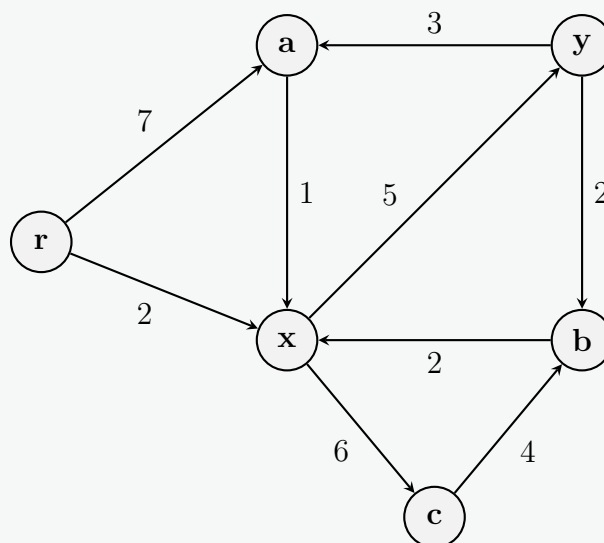


Figure 1: Original Digraph G_0

Let us tabulate the minimum incoming edges for each non-root vertex:

Vertex (v)	All Incoming Edges with Weights	Minimum Edge $\pi(v)$	Weight $w(\pi(v))$
a	$(r, a) : 7, (y, a) : 3$	(y, a)	3
x	$(r, x) : 2, (a, x) : 1, (b, x) : 2$	(a, x)	1
y	$(x, y) : 5$	(x, y)	5
c	$(x, c) : 6$	(x, c)	6
b	$(c, b) : 4, (y, b) : 2$	(y, b)	2

Let P_0 be the set of these chosen minimum incoming edges:

$$P_0 = \{(y, a), (a, x), (x, y), (x, c), (y, b)\}$$

Let us plot the subgraph containing only the edges in P_0 to inspect for cycles:

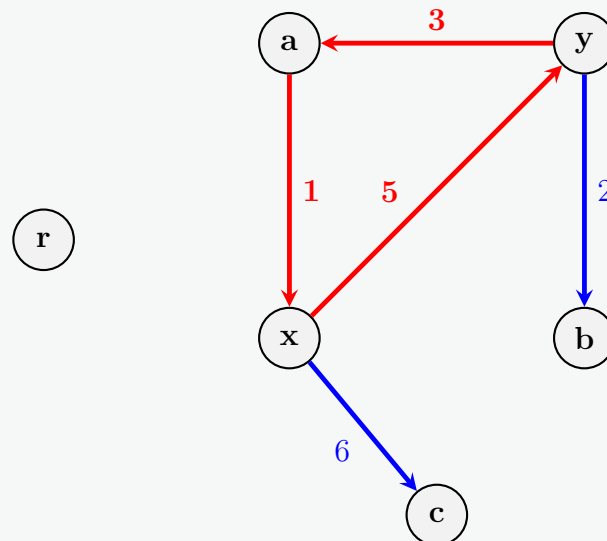


Figure 2: Selected Subgraph P_0 showcasing the critical cycle C_1 in red

As explicitly highlighted above, the edge set contains a directed cycle $C_1 = \{x \rightarrow y \rightarrow a \rightarrow x\}$.

Step 2: Contraction of Cycle C_1 and Edge Weight Modification

We contract the cycle $C_1 = \{x, y, a\}$ into a single pseudo-vertex denoted as v_{xay} . Now, we recompute the modified weights w' for all incoming edges entering this pseudo-vertex from external nodes:

- **Edges from r to C_1 :**

- Original edge (r, a) with $w(r, a) = 7$. The cycle edge entering a is $\pi(a) = (y, a)$ with weight 3.

$$w'(r, v_{xay}) = w(r, a) - w(y, a) = 7 - 3 = 4$$

- Original edge (r, x) with $w(r, x) = 2$. The cycle edge entering x is $\pi(x) = (a, x)$ with weight 1.

$$w'(r, v_{xay}) = w(r, x) - w(a, x) = 2 - 1 = 1$$

- **Edges from b to C_1 :**

- Original edge (b, x) with $w(b, x) = 2$. The cycle edge entering x is $\pi(x) = (a, x)$ with weight 1.

$$w'(b, v_{xay}) = w(b, x) - w(a, x) = 2 - 1 = 1$$

- **Edges leaving C_1 :** Retain their original minimum values.

- Edge from $x \rightarrow c$ becomes $v_{xay} \rightarrow c$ with weight 6.
- Edge from $y \rightarrow b$ becomes $v_{xay} \rightarrow b$ with weight 2.

- **Other remaining edges:**

- Edge $c \rightarrow b$ remains unchanged with weight 4.

Let us plot the newly constructed contracted graph G_1 :

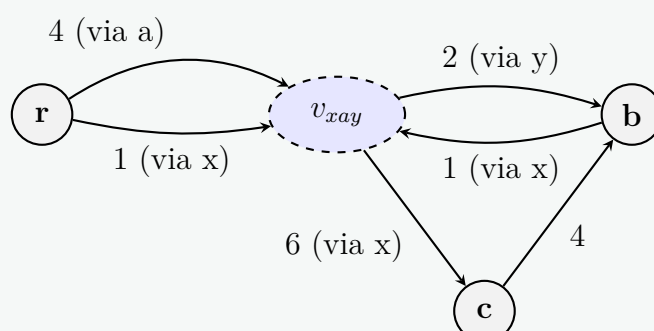


Figure 3: Contracted Graph G_1 with Modified Weights

Now, we perform the selection of the minimum incoming edges for each non-root vertex in G_1 :

- For c : The only incoming edge is $v_{xay} \rightarrow c$ with weight 6.
- For b : Incoming edges are $v_{xay} \rightarrow b$ (weight 2) and $c \rightarrow b$ (weight 4). The minimum is $v_{xay} \rightarrow b$ with weight 2.
- For v_{xay} : Incoming edges are from r (weight 1 via x), from r (weight 4 via a), and from b (weight 1 via x).

There is a tie for the minimum incoming edge to v_{xay} between (r, v_{xay}) and (b, v_{xay}) , both having an adjusted weight of 1. Following standard greedy optimization rules, we select the edge coming from the root r , i.e., (r, v_{xay}) **via** x . This selection prevents any further cycling and resolves the graph directly.

Let us graph the chosen edge set P_1 in G_1 :

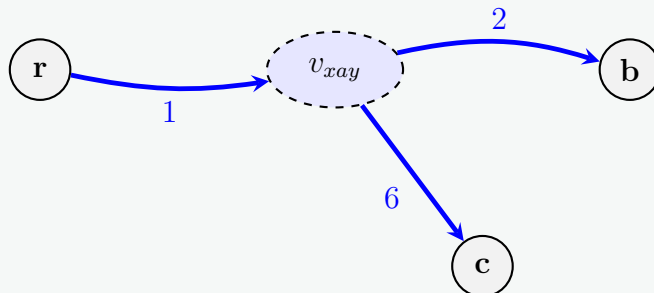


Figure 4: Acyclic Selected Arborescence A' in G_1

Since $A' = \{(r, v_{xay}) \text{ via } x, (v_{xay}, c) \text{ via } x, (v_{xay}, b) \text{ via } y\}$ contains absolutely no cycles, it forms the optimal spanning arborescence for the contracted graph G_1 .

Step 3: Expansion Phase and Final Arborescence Construction

We expand the pseudo-vertex v_{xay} back to its original components $\{x, y, a\}$ to reconstruct the optimal solution for G_0 :

- The incoming edge chosen for v_{xay} in A' is (r, v_{xay}) which originally enters vertex x . Therefore, we add the edge (r, x) to our final arborescence.
- Since the entry point into the cycle is x , we must break the cycle C_1 by discarding the internal cycle edge that enters x . The edge entering x in the cycle is (a, x) . Thus, (a, x) is eliminated.
- The remaining cycle edges, (x, y) and (y, a) , are safely broken out and included in our final edge set.
- The external outgoing structural edges chosen in A' are preserved: (v_{xay}, c) which corresponds to (x, c) , and (v_{xay}, b) which corresponds to (y, b) .

Compiling these components yields the final minimum-cost arborescence edge set A :

$$A = \{(r, x), (x, y), (y, a), (x, c), (y, b)\}$$

Let us visualize the final structural configuration of the minimum cost arborescence:

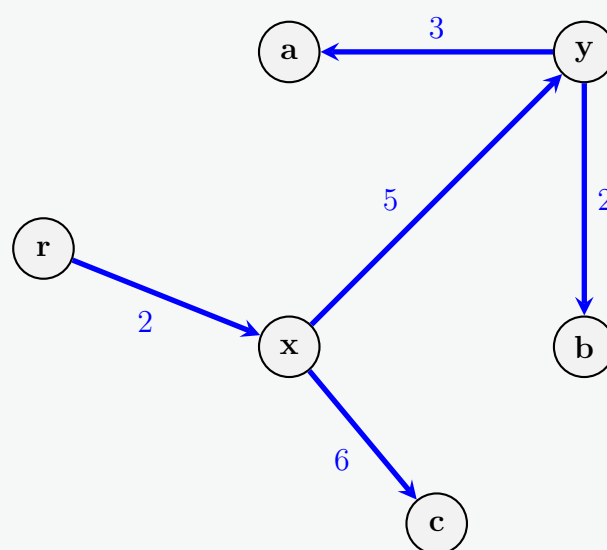


Figure 5: Final Minimum Cost Arborescence A

4. Final Cost Verification

Let us compute the total exact cost of the resulting minimum-cost arborescence:

$$\text{Total Cost} = w(r, x) + w(x, y) + w(y, a) + w(x, c) + w(y, b)$$

$$\text{Total Cost} = 2 + 5 + 3 + 6 + 2 = 18$$

All vertices are uniquely and optimally reachable from the root node r with a total minimum cost of **18**.

Q10. What is a spanning tree of a graph? Write Kruskal's algorithm for finding a minimum spanning tree of an edge-weighted graph. Analyze the time complexity of the algorithm by suggesting appropriate data structures. **(3+5+7 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Answer

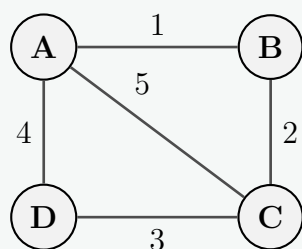
1. Definition of a Spanning Tree

Let $G = (V, E)$ be a connected, undirected graph. A **spanning tree** of G is a subgraph $T = (V, E')$ such that:

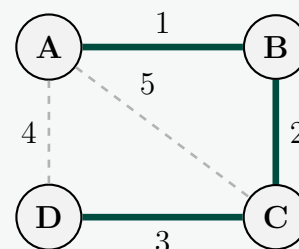
- **Spanning:** It includes all vertices of the original graph G (the vertex set V is identical).
- **Acyclic:** It contains no cycles.
- **Connected:** There is exactly one simple path between any pair of vertices in T .

Mathematically, a spanning tree always contains exactly $|V| - 1$ edges. If the graph G has edge weights, a **Minimum Spanning Tree (MST)** is a spanning tree whose sum of edge weights is minimized.

Original Graph G



Minimum Spanning Tree



2. Kruskal's Algorithm

Kruskal's Algorithm is a greedy approach. It builds the MST forest by considering edges in increasing order of their weights and adding an edge to the forest if and only if it does not form a cycle. Cycle detection is efficiently handled using a Disjoint-Set data structure.

Algorithm 19 Kruskal's Algorithm

Input: A connected, undirected graph $G = (V, E)$ with edge weights $w : E \rightarrow \mathbb{R}$.

Output: A set of edges A forming a Minimum Spanning Tree.

```

1:  $A \leftarrow \emptyset$                                 ▷ Initialize an empty set to store MST edges
2: for each vertex  $v \in V$  do
3:   MAKE-SET( $v$ )                                ▷ Create  $|V|$  disjoint sets, one for each vertex
4: end for
5: Sort all edges in  $E$  into non-decreasing order by their weight  $w$ .
6: for each edge  $(u, v) \in E$ , taken in sorted order do
7:   if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ ) then          ▷ Check if  $u$  and  $v$  are in different components
8:      $A \leftarrow A \cup \{(u, v)\}$                 ▷ Add edge to the MST
9:     UNION( $u, v$ )                                ▷ Merge the two components
10:  end if
11: end for
12: return  $A$ 
  
```

3. Time Complexity Analysis and Data Structures

To achieve optimal time complexity, we must choose appropriate data structures:

- Sorting Data Structure:** We can use an array of edges and sort them using an efficient comparison-based sorting algorithm like **Merge Sort** or **Quick Sort**.
- Disjoint-Set (Union-Find) Data Structure:** We maintain the connected components using a **Disjoint-Set Forest** implemented with two crucial heuristics:
 - *Union by Rank:* Attaches the shorter tree under the root of the taller tree during a union operation to keep the trees flat.
 - *Path Compression:* Makes every visited node point directly to the root during a find operation.

Step-by-Step Complexity Breakdown:

- **Initialization (Make-Set):** Creating a disjoint set for each of the $|V|$ vertices takes $\mathcal{O}(|V|)$ time.
- **Sorting Edges:** Sorting the $|E|$ edges takes $\mathcal{O}(|E| \log |E|)$ time.
- **Cycle Checking and Merging (Find-Set and Union):** The algorithm iterates over $|E|$ edges. In each iteration, it performs two FIND-SET operations, and if the condition is met, one UNION operation. For $|E|$ operations on $|V|$ elements using Union by Rank and Path Compression, the total time is $\mathcal{O}(|E|\alpha(|V|))$, where α is the inverse Ackermann function. Since $\alpha(|V|) \leq 4$ for all practical values of $|V|$, this phase runs in virtually linear time $\mathcal{O}(|E|)$.

Overall Time Complexity:

The total time complexity is the sum of these phases:

$$T(|V|, |E|) = \mathcal{O}(|V| + |E| \log |E| + |E|\alpha(|V|))$$

Since the graph is a simple graph, the number of edges is bounded by $|V|^2$, implying that $\log |E| \leq \log(|V|^2) = 2 \log |V|$. Thus, $\mathcal{O}(\log |E|)$ is equivalent to $\mathcal{O}(\log |V|)$. The sorting step asymptotically dominates the runtime.

Final Time Complexity: $\mathcal{O}(|E| \log |V|)$

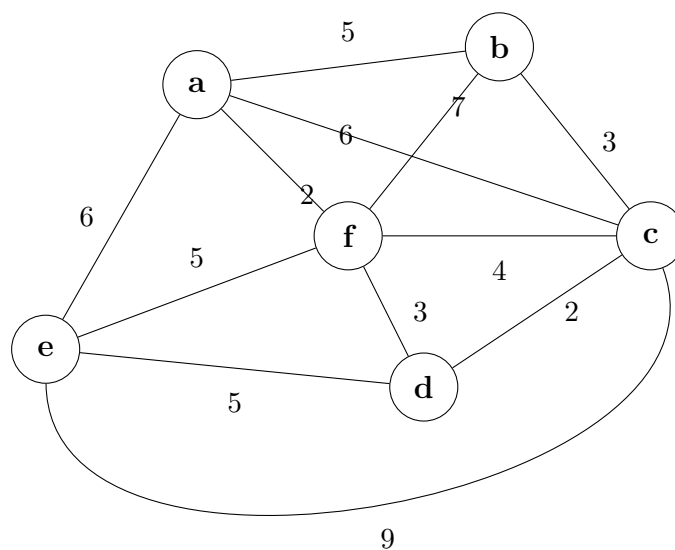
Space Complexity:

The space complexity requires storing the graph structure (edges) and the Disjoint-Set forest.

- Storing the edge list takes $\mathcal{O}(|E|)$ space.
- The parent and rank arrays for the Disjoint-Set take $\mathcal{O}(|V|)$ space.

Final Space Complexity: $\mathcal{O}(|V| + |E|)$

Q11. Find a minimum spanning tree using Prim's algorithm, showing every step.



(10 marks)

[CSE 207 | L-2/T-2 | 2017–18]

Answer

Prim's Algorithm builds the Minimum Spanning Tree (MST) by starting from an arbitrary node and repeatedly adding the cheapest edge that connects a visited node to an unvisited node.

Let's define the nodes and edges. We will start the algorithm from node **a**.

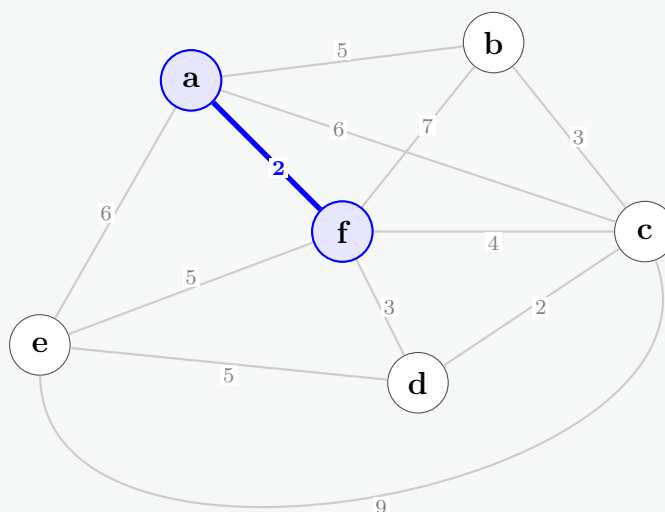
- **Visited Set** (V_{vis}): Keeps track of nodes already in the MST.
- **MST Edges** (E_{mst}): Keeps track of edges added to the MST.

Step 1: Initialization

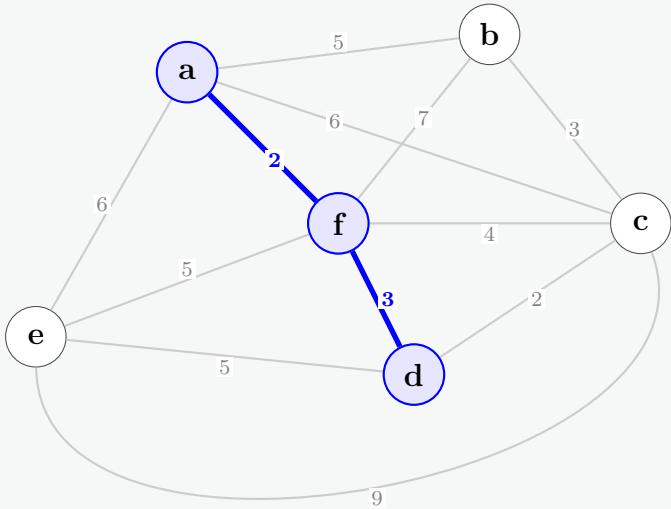
Start at vertex **a**. $V_{vis} = \{a\}$.

Available edges from visited nodes: $(a, f) = 2$, $(a, b) = 5$, $(a, e) = 6$, $(a, c) = 6$.

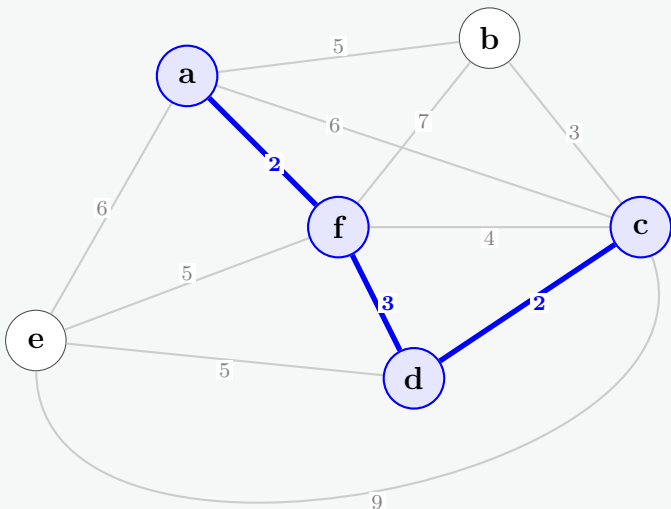
Action: Minimum weight is **2**. Add edge (**a**, **f**) and vertex **f**.



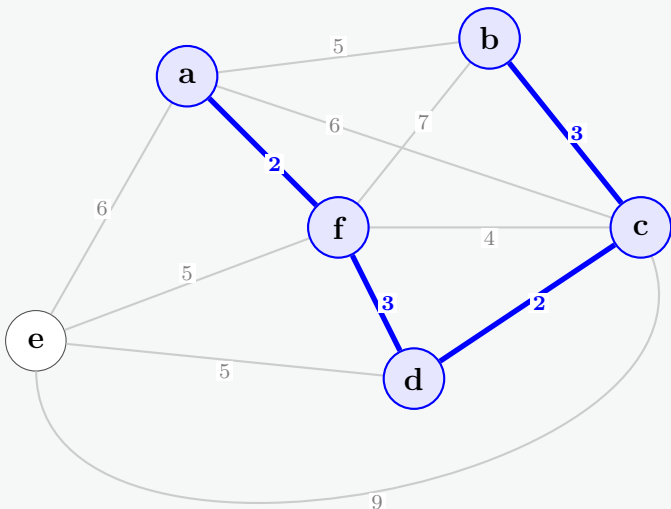
Step 2:
 $V_{vis} = \{a, f\}$.
Available crossing edges: $(f, d) = 3, (f, c) = 4, (a, b) = 5, (e, f) = 5, (a, e) = 6, (a, c) = 6, (b, f) = 7$.
Action: Minimum weight is **3**. Add edge **(f, d)** and vertex **d**.



Step 3:
 $V_{vis} = \{a, f, d\}$.
Available crossing edges: $(c, d) = 2, (f, c) = 4, (a, b) = 5, (e, f) = 5, (e, d) = 5, \dots$
Action: Minimum weight is **2**. Add edge **(d, c)** and vertex **c**.



Step 4:
 $V_{vis} = \{a, f, d, c\}$.
Available crossing edges: $(b, c) = 3, (a, b) = 5, (e, f) = 5, (e, d) = 5, (c, e) = 9, \dots$
Action: Minimum weight is **3**. Add edge **(c, b)** and vertex **b**.

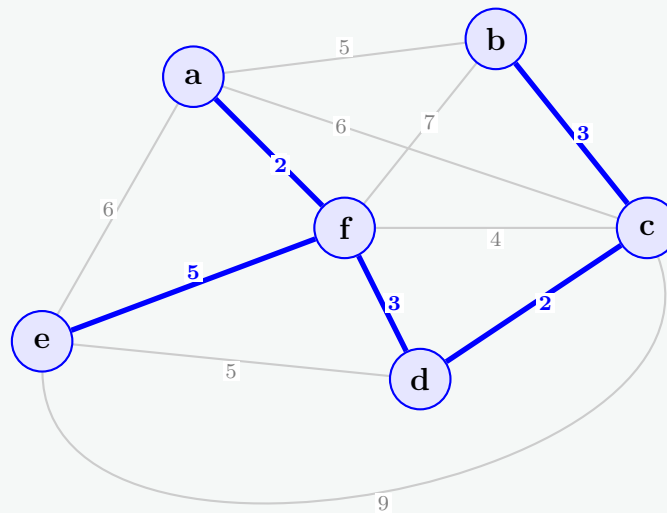


Step 5:

$V_{vis} = \{a, f, d, c, b\}$.

Available crossing edges to unvisited node 'e': $(e, f) = 5, (e, d) = 5, (a, e) = 6, (c, e) = 9$.

Action: Minimum weight is **5**. We can pick either (e, f) or (e, d) . Let's pick **(e, f)**. Add vertex **e**.



Conclusion: All vertices are now visited ($|V| - 1 = 5$ edges added).

Final MST Edges: $\{(a, f), (f, d), (d, c), (c, b), (e, f)\}$

Total MST Cost: $2 + 3 + 2 + 3 + 5 = 15$. ✓

Q12. Let G be a weighted connected graph and let V_1, V_2 be a partition of its vertices into two disjoint non-empty sets. Let e be an edge in G with minimum weight among those with one endpoint in V_1 and the other in V_2 . Show that there is a minimum spanning tree T of G containing e . **(10 marks)**

[CSE 207 | L-2/T-2 | 2017–18]

Answer**1. Theorem Statement (The Cut Property of MSTs)**

Let $G = (V, E)$ be a connected, undirected graph with an edge-weight function $w : E \rightarrow \mathbb{R}$. Let (V_1, V_2) be a partition of V (i.e., $V_1 \cap V_2 = \emptyset$, $V_1 \cup V_2 = V$, and $V_1, V_2 \neq \emptyset$). Let $e = (u, v)$ be an edge in G such that $u \in V_1$ and $v \in V_2$, and for all edges $e' = (u', v')$ crossing the cut (where $u' \in V_1, v' \in V_2$), we have $w(e) \leq w(e')$. Then, there exists a Minimum Spanning Tree (MST) T of G that contains e .

2. Formal Proof (The Cut-Exchange Argument)

We will prove this property using a standard university-level **cut-exchange argument**.

Step 1: Assumption of an Arbitrary MST

Let T^* be any Minimum Spanning Tree of G . We consider two mutually exclusive cases:

- **Case 1:** $e \in T^*$. If the chosen MST already contains the minimum-weight cut edge e , then the claim holds immediately by setting $T = T^*$.
- **Case 2:** $e \notin T^*$. We proceed to show that we can construct an alternative spanning tree T from T^* such that $e \in T$ and $w(T) = w(T^*)$.

Step 2: Cycle Creation via Edge Insertion

Suppose $e = (u, v) \notin T^*$. Because T^* is a spanning tree, there already exists a unique, simple path P in T^* connecting vertex u to vertex v . If we temporarily add the edge e to T^* , it creates a subgraph $T^* \cup \{e\}$ containing exactly one unique simple cycle C , consisting of the edge e combined with the path P .

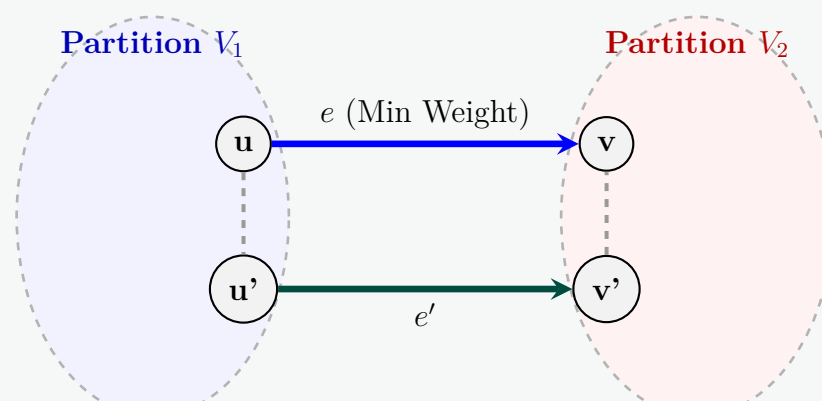


Figure 1: The unique cycle C formed by adding e to T^* .
The alternative cut-crossing edge is e' .

Step 3: Finding an Alternative Cut-Crossing Edge

The vertex u lies in V_1 and the vertex v lies in V_2 . Therefore, the path P within T^* must start in V_1 and terminate in V_2 . This implies that the path P must cross the cut boundary from V_1 to V_2 at least once.

Let $e' = (u', v')$ be an edge on the path P such that $u' \in V_1$ and $v' \in V_2$. By definition, $e' \in T^*$ and $e' \neq e$.

Step 4: The Exchange and Construction of T

We now construct a new subgraph T by adding e to T^* and removing e' :

$$T = T^* \cup \{e\} \setminus \{e'\}$$

Let us verify that T satisfies all necessary conditions of a Spanning Tree:

- (a) **Spanning Property:** No vertices were removed; thus, T contains all vertices V .
- (b) **Acyclicity:** The insertion of e created exactly one cycle C . Removing e' (which belongs to C) breaks that cycle, leaving the graph completely acyclic.
- (c) **Connectivity:** Removing an edge from a cycle does not disconnect a graph. Since T^* was connected, T remains connected.

With $|V| - 1$ edges, no cycles, and complete connectivity, T is mathematically proven to be a valid spanning tree of G .

Step 5: Minimality Analysis

We now calculate and compare the total edge weight of our newly constructed spanning tree T :

$$w(T) = w(T^*) + w(e) - w(e')$$

Recall the core structural hypothesis of the theorem: e is an edge of **minimum weight** crossing the cut (V_1, V_2) . Since e' is also an edge crossing this exact same cut, it must satisfy:

$$w(e) \leq w(e') \implies w(e) - w(e') \leq 0$$

Substituting this inequality back into our weight equation yields:

$$w(T) \leq w(T^*)$$

However, our initial assumption designated T^* as a **Minimum** Spanning Tree, meaning no spanning tree in G can have a strictly lower total weight than T^* . Therefore, it must be true that:

$$w(T) = w(T^*)$$

3. Conclusion

This proves that T is also a Minimum Spanning Tree of G . Since $e \in T$ by construction, we have successfully demonstrated that there is always an MST of G containing the minimum-weight cut edge e . ■

Q13. Write Prim's algorithm that computes a minimum spanning tree of a graph. Analyze its time complexity. **(15 marks)** [CSE 207 | L-2/T-2 | 2016–17]

Answer**1. Introduction and Algorithm Description**

Prim's Algorithm is a greedy minimum spanning tree (MST) algorithm that constructs the tree vertex by vertex. Starting from an arbitrary root vertex r , the algorithm maintains a set of vertices S that have already been included in the growing MST. At each step, it identifies the minimum-weight edge crossing the cut $(S, V \setminus S)$ —meaning one endpoint is in S and the other is in $V \setminus S$ —and adds the corresponding vertex to S .

To implement this efficiently, a min-priority queue Q is utilized to store all vertices currently in $V \setminus S$. The priority (or *key*) of a vertex $v \in Q$ corresponds to the minimum weight of any edge connecting v to an existing vertex in S .

Algorithm 20 Prim's Algorithm

Input: A connected, undirected graph $G = (V, E)$ with an edge-weight function $w : E \rightarrow \mathbb{R}$, and a starting root vertex $r \in V$.

Output: A Minimum Spanning Tree defined by predecessor/parent pointers π .

```

1: for each vertex  $u \in V$  do
2:    $key[u] \leftarrow \infty$                                       $\triangleright$  Initialize all cuts to infinity
3:    $\pi[u] \leftarrow \text{NIL}$                                     $\triangleright$  Set predecessor of each vertex to null
4: end for
5:  $key[r] \leftarrow 0$                                         $\triangleright$  Set the root's key to 0 so it is extracted first
6: Initialize a min-priority queue  $Q$  containing all vertices  $V$ , keyed by  $key$  values.
7: while  $Q$  is not empty do
8:    $u \leftarrow \text{EXTRACT-MIN}(Q)$                           $\triangleright$  Add vertex  $u$  to the growing MST component  $S$ 
9:   for each vertex  $v \in \text{Adj}[u]$  do
10:    if  $v \in Q$  and  $w(u, v) < key[v]$  then
11:       $\pi[v] \leftarrow u$                                     $\triangleright$  Update the parent pointer
12:       $key[v] \leftarrow w(u, v)$                             $\triangleright$  Update the priority key value
13:       $\text{DECREASE-KEY}(Q, v, key[v])$                         $\triangleright$  Re-heapify the priority queue
14:    end if
15:  end for
16: end while

```

2. Visualizing the Cut Property in Prim's Algorithm

The foundational correctness of Prim's algorithm rests on the *Cut Property*. At any point during execution, the vertices are split into the tree vertices S and the remaining vertices in the priority queue $Q = V \setminus S$.

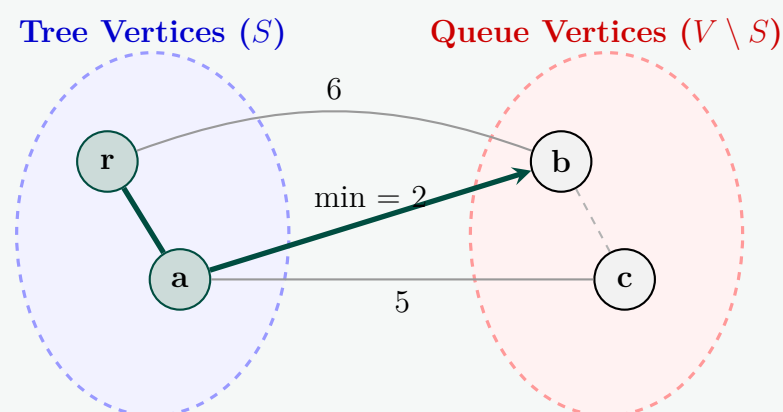


Figure 1: The Cut Boundary during Prim's Algorithm execution.

3. Detailed Time Complexity Analysis

The total execution runtime depends completely on the choice of the underlying data structure used to implement the min-priority queue Q . The algorithm's structural runtime consists of three primary components:

- (a) **Initialization:** Executed exactly $|V|$ times, running in $\mathcal{O}(|V|)$ time.
- (b) **Extract-Min** operations: Executed exactly once for every vertex, totaling $|V|$ invocations.
- (c) **Decrease-Key** operations: Triggered inside the adjacency loop. In an undirected graph, each edge (u, v) is examined at most twice (once from u and once from v). Thus, **Decrease-Key** is invoked at most $2|E| = \mathcal{O}(|E|)$ times.

Let us analyze the overall asymptotic bounds across different heap variations:

Case A: Binary Heap Implementation

If the priority queue Q is implemented via a standard standard Binary Min-Heap (e.g., array-based binary tree):

- **Building the Heap:** Constructing the initial structure takes $\mathcal{O}(|V|)$.
- **Extract-Min:** Each extraction takes $\mathcal{O}(\log |V|)$ time. For $|V|$ vertices, this costs $\mathcal{O}(|V| \log |V|)$.
- **Decrease-Key:** Each key modification requires bubbling up elements, costing $\mathcal{O}(\log |V|)$ time. For $|E|$ updates, this costs $\mathcal{O}(|E| \log |V|)$.

Summing these operations up:

$$T(|V|, |E|) = \mathcal{O}(|V|) + \mathcal{O}(|V| \log |V|) + \mathcal{O}(|E| \log |V|) = \mathcal{O}((|V| + |E|) \log |V|)$$

For any simple connected graph, $|E| \geq |V| - 1$, meaning that $|E|$ asymptotically dominates $|V|$.

Total Binary Heap Complexity: $\mathcal{O}(|E| \log |V|)$

Case B: Fibonacci Heap Implementation

If the priority queue Q is optimized via a Fibonacci Heap:

- **Extract-Min:** Takes $\mathcal{O}(\log |V|)$ amortized time. For $|V|$ vertices, the total cost is $\mathcal{O}(|V| \log |V|)$.
- **Decrease-Key:** Takes $\mathcal{O}(1)$ constant amortized time. For $|E|$ edge modifications, the total cost drops to $\mathcal{O}(|E|)$.

Summing these operations up:

Total Fibonacci Heap Complexity: $\mathcal{O}(|E| + |V| \log |V|)$

This is highly advantageous for **dense graphs** where $|E| \approx |V|^2$, dropping the time complexity down to a virtually linear $\mathcal{O}(|E|)$.

4. Space Complexity Analysis

- **Graph Storage:** Storing the graph using an adjacency list requires $\mathcal{O}(|V| + |E|)$ space.
- **Auxiliary Arrays:** Storing the tracking arrays (key, π) requires $\mathcal{O}(|V|)$ space.
- **Priority Queue Space:** The priority queue holds at most $|V|$ vertices at any time, consuming $\mathcal{O}(|V|)$ space.

Total Space Complexity: $\mathcal{O}(|V| + |E|)$

Q14. Write the correctness proof of Kruskal's algorithm for finding a minimum spanning tree. **(10 marks)**
2016–17]

[CSE 207 | L-2/T-2 |

Answer**1. Introduction to the Proof**

To prove the correctness of Kruskal's Algorithm, we must establish two properties for a given connected, undirected, edge-weighted graph $G = (V, E)$:

- Spanning Tree Property:** The algorithm terminates and produces a valid spanning tree (a connected and acyclic subgraph covering all vertices).
- Minimality Property:** The produced spanning tree has the minimum possible total edge weight.

2. Proof of the Spanning Tree Property

- **Acyclicity:** The algorithm explicitly utilizes a Disjoint-Set (Union-Find) data structure to only add an edge (u, v) if u and v belong to different connected components. Thus, it is impossible for the algorithm to form a cycle.
- **Connectivity and Spanning:** Initially, the graph is a forest of $|V|$ isolated vertices. Every time an edge is added, two distinct components are merged into one, reducing the total number of components by exactly 1. For a connected graph, there are enough edges to continue this process until only 1 component remains. This requires exactly $|V| - 1$ edges. An acyclic graph with $|V|$ vertices and $|V| - 1$ edges is, by definition, a connected spanning tree.

3. Proof of Minimality (via Mathematical Induction)

We will prove that the spanning tree generated by Kruskal's algorithm is a Minimum Spanning Tree (MST) using induction on the sequence of edges added to the tree.

Let the edges added by Kruskal's algorithm be $e_1, e_2, \dots, e_{n-1}$ (where $n = |V|$), ordered by the time of their insertion. Let $T_k = \{e_1, e_2, \dots, e_k\}$ be the set of the first k edges chosen by the algorithm.

Inductive Hypothesis: For every $k \in \{0, 1, \dots, n-1\}$, the forest T_k is a subset of some Minimum Spanning Tree T^* of G .

Base Case ($k = 0$):

$T_0 = \emptyset$. The empty set is trivially a subset of any Minimum Spanning Tree of G . The base case holds.

Inductive Step:

Assume that the hypothesis holds for $k-1$. That is, there exists some MST T^* such that $T_{k-1} \subseteq T^*$. We must show that there exists an MST (either T^* or a different one) containing $T_k = T_{k-1} \cup \{e_k\}$.

Let $e_k = (u, v)$ be the k -th edge added by Kruskal's algorithm. We consider two possible cases:

- **Case 1:** $e_k \in T^*$
If the k -th edge is already in T^* , then $T_k = T_{k-1} \cup \{e_k\} \subseteq T^*$. The induction step immediately holds.
- **Case 2:** $e_k \notin T^*$
If $e_k \notin T^*$, adding e_k to the tree T^* creates exactly one simple cycle, C , because T^* is a spanning tree.

Because e_k was chosen by Kruskal's algorithm, it must connect two different components in the forest T_{k-1} . Let S be the set of vertices in the component containing u within T_{k-1} . The vertex v lies in a different component, so $v \notin S$.

Since the cycle C goes from $u \in S$ to $v \notin S$ and back, there must be some other edge $e' = (x, y)$ in the cycle C such that e' crosses the cut $(S, V \setminus S)$.

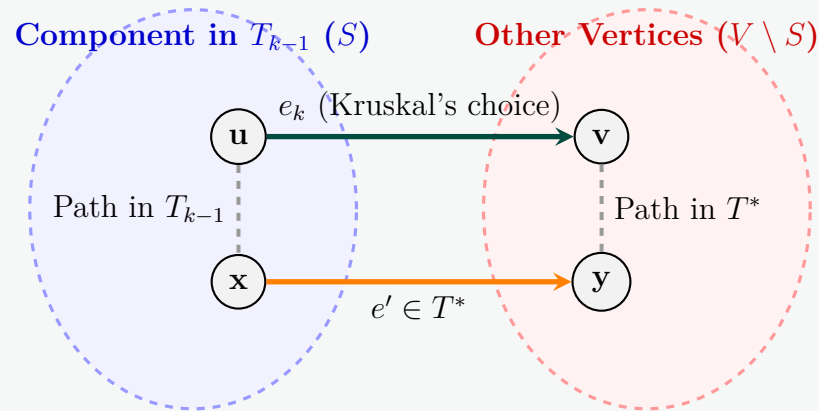


Figure 1: Cycle exchange mechanism illustrating e_k and e' crossing the cut.

Analyzing the weights:

- (a) Since e' crosses the cut between components of T_{k-1} , adding it to T_{k-1} would not form a cycle. It was a valid candidate when Kruskal's algorithm was considering edges.
- (b) Kruskal's considers edges in non-decreasing order of weight. Since it chose e_k instead of e' , it must be that $w(e_k) \leq w(e')$.
- (c) However, T^* is assumed to be an MST. If we construct a new spanning tree $T^{**} = T^* \cup \{e_k\} \setminus \{e'\}$, its total weight is:

$$w(T^{**}) = w(T^*) + w(e_k) - w(e')$$

- (d) Since T^* is an MST, no spanning tree can have a smaller weight. Thus $w(T^{**}) \geq w(T^*)$, which implies $w(e_k) \geq w(e')$.
- (e) Combining the inequalities $w(e_k) \leq w(e')$ and $w(e_k) \geq w(e')$, we conclude that $w(e_k) = w(e')$.

Therefore, $w(T^{**}) = w(T^*)$, meaning T^{**} is also an optimal Minimum Spanning Tree. Furthermore, since e' was not in T_{k-1} , the removal of e' does not disturb the prior edges. We now have $T_k = T_{k-1} \cup \{e_k\} \subseteq T^{**}$.

By the principle of mathematical induction, T_{n-1} (the final output of Kruskal's algorithm) is a subset of some MST. Since T_{n-1} contains exactly $n - 1$ edges, it is precisely that Minimum Spanning Tree. ■

4. Brief Complexity Summary

As per standard algorithm analysis requirements:

- **Time Complexity:** $\mathcal{O}(|E| \log |V|)$ due to sorting the edges, followed by $\mathcal{O}(|E| \alpha(|V|))$ for the Union-Find operations, where α is the inverse Ackermann function. The sorting step asymptotically dominates.
- **Space Complexity:** $\mathcal{O}(|V| + |E|)$ to store the graph, the edge array, and the Union-Find disjoint-set data structures.

Q15. There is a network of roads $G = (V, E)$ connecting a set of cities V . Each road in E has an associated length l_e . There is a proposal to add one new road to this network, and there is a list E' of pairs of cities between which the new road can be built. Each such potential road $e' \in E'$ has an associated length. As a designer for the public works department you are asked to determine the road $e' \in E'$ whose addition to existing network G would result in the maximum decrease in the driving distance between two fixed cities s and t in the network. Give an algorithm for solving this problem. Your algorithm should run in $\mathcal{O}((V + E + E') \log V)$ time. **(15 marks)** [CSE 207 | L-2/T-2 | 2015–16]

Answer

1. Core Insight

If we introduce a single new road $e' = (u, v) \in E'$ with length $l_{e'}$ into the existing undirected network $G = (V, E)$, the new shortest path from city s to city t can only take one of two structural forms:

- (a) **Scenario A:** It does not use the new road e' at all. The driving distance remains the original baseline distance in G , denoted as $\text{dist}_G(s, t)$.
- (b) **Scenario B:** It uses the new road e' exactly once. Since the road is bidirectional, the path will either traverse the new road from u to v or from v to u .

Thus, the minimized driving distance after incorporating a candidate road e' is given mathematically by:

$$\min \left(\text{dist}_G(s, t), \text{dist}_G(s, u) + l_{e'} + \text{dist}_G(v, t), \text{dist}_G(s, v) + l_{e'} + \text{dist}_G(u, t) \right)$$

Instead of running an expensive Dijkstra search from scratch for each proposed road in E' , we can precompute the shortest distances from s to all vertices, and from all vertices to t , using just **two independent passes** of Dijkstra's algorithm.

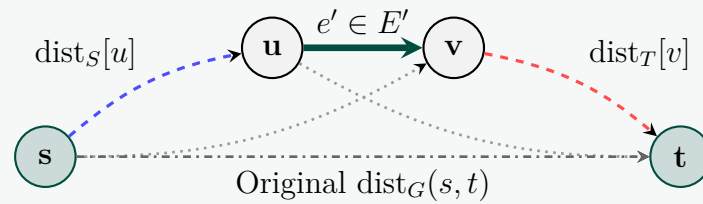


Figure 1: Distance decomposition using candidate road $e' = (u, v)$.

2. Algorithm Steps

- (a) **Forward Dijkstra Pass:** Run a standard single-source shortest path Dijkstra algorithm on G starting from the source city s . Store the output weights in a lookup array dist_S , where $\text{dist}_S[x] = \text{dist}_G(s, x)$ for all $x \in V$.
- (b) **Backward Dijkstra Pass:** Run Dijkstra's algorithm on G starting from the destination city t . Store the output weights in a lookup array dist_T , where $\text{dist}_T[x] = \text{dist}_G(x, t)$ for all $x \in V$.
- (c) **Candidate Evaluation Loop:**
 - Initialize the optimal distance variables: $D_{\text{best}} \leftarrow \text{dist}_S[t]$ and $\text{best_road} \leftarrow \text{NIL}$.
 - Iterate through every structural choice $e' = (u, v) \in E'$ with length $l_{e'}$. Compute:
$$C_1 = \text{dist}_S[u] + l_{e'} + \text{dist}_T[v] \quad (\text{Path via } u \rightarrow v)$$

$$C_2 = \text{dist}_S[v] + l_{e'} + \text{dist}_T[u] \quad (\text{Path via } v \rightarrow u)$$

$$C_{\min} = \min(C_1, C_2)$$
 - If $C_{\min} < D_{\text{best}}$, update $D_{\text{best}} \leftarrow C_{\min}$ and store $\text{best_road} \leftarrow e'$.
- (d) **Output:** Return best_road along with the maximum metric reduction value: $\text{dist}_S[t] - D_{\text{best}}$.

3. Pseudocode

Algorithm 21 Optimal Road Addition Optimizer

Input: Graph $G = (V, E)$ with lengths l , proposed set E' with lengths l' , source s , destination t .

Output: The optimal road $e^* \in E'$ and its respective maximum distance reduction.

```

1:  $\text{dist}_S \leftarrow \text{DIJKSTRA}(G, l, s)$   $\triangleright O((V + E) \log V)$ 
2:  $\text{dist}_T \leftarrow \text{DIJKSTRA}(G, l, t)$   $\triangleright O((V + E) \log V)$ 
3:  $D_{\text{best}} \leftarrow \text{dist}_S[t]$ 
4:  $\text{best\_road} \leftarrow \text{NIL}$ 
5: for each road  $e' = (u, v) \in E'$  do
6:    $C_1 \leftarrow \text{dist}_S[u] + l'_{e'} + \text{dist}_T[v]$ 
7:    $C_2 \leftarrow \text{dist}_S[v] + l'_{e'} + \text{dist}_T[u]$ 
8:    $C_{\min} \leftarrow \min(C_1, C_2)$ 
9:   if  $C_{\min} < D_{\text{best}}$  then
10:     $D_{\text{best}} \leftarrow C_{\min}$ 
11:     $\text{best\_road} \leftarrow e'$ 
12:   end if
13: end for
14:  $\text{max\_decrease} \leftarrow \text{dist}_S[t] - D_{\text{best}}$ 
15: return ( $\text{best\_road}, \text{max\_decrease}$ )
```

4. Complexity Analysis

Time Complexity:

- **Dijkstra Runs:** Executing two passes of binary-heap-based Dijkstra takes:

$$2 \times \mathcal{O}((V + E) \log V) = \mathcal{O}((V + E) \log V)$$

- **Evaluation Loop:** The loop executes exactly $|E'|$ times. Within each iteration, array lookups and arithmetic comparison operations take $\mathcal{O}(1)$ static time. Thus, the evaluation overhead is bounded strictly by $\mathcal{O}(|E'|)$.

Summing these execution blocks together gives the total runtime complexity:

$$T = \mathcal{O}((V + E) \log V + |E'|)$$

Since $|E'| \leq |E'| \log |V|$, this cleanly aggregates into the required theoretical target:

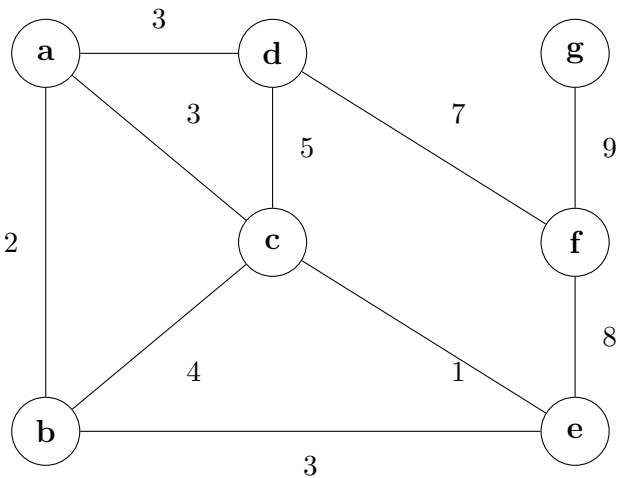
$$\textbf{Total Time Complexity: } \mathcal{O}((V + E + |E'|) \log V)$$

Space Complexity:

The distance maps dist_S and dist_T each consume $\mathcal{O}(|V|)$ memory space. Graph representations and input collections dictate $\mathcal{O}(|V| + |E| + |E'|)$ memory overhead.

Total Space Complexity: $\mathcal{O}(V + E + E')$

Q16. Find a minimum spanning tree by running (i) Kruskal’s algorithm and (ii) Prim’s algorithm. Show the order of edges selected in each case (use alphabetical ordering when there is a tie). **(15 marks)** [CSE 207 | L-2/T-2 | 2015–16]



Detailed Answer: Kruskal’s & Prim’s Algorithm

Part 1: Kruskal’s Algorithm

Step 1: Sort edges by weight. When weights are equal, sort alphabetically (e.g., ac before ad).
Sorted list: $ce(1), ab(2), ac(3), ad(3), be(3), bc(4), cd(5), df(7), ef(8), fg(9)$.

Part 1: Kruskal’s Algorithm Step-by-Step

Step 1: Sort edges by weight. When weights are equal, sort alphabetically (e.g., ac before ad).
Sorted list: $ce(1), ab(2), ac(3), ad(3), be(3), bc(4), cd(5), df(7), ef(8), fg(9)$.

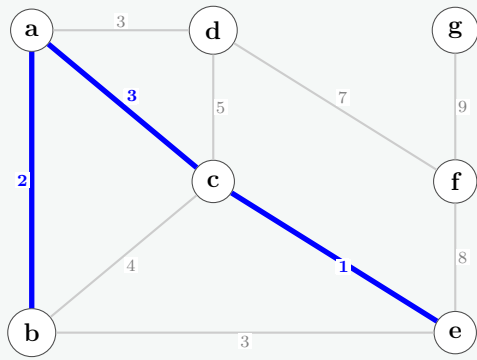
Edge	Weight	Add to MST?	Reason
$c-e$	1	Yes	No cycle
$a-b$	2	Yes	No cycle
$a-c$	3	Yes	No cycle
$a-d$	3	Yes	No cycle
$b-e$	3	No	Forms cycle ($b-a-c-e-b$)
$b-c$	4	No	Forms cycle ($b-a-c-b$)
$c-d$	5	No	Forms cycle ($c-a-d-c$)
$d-f$	7	Yes	No cycle
$e-f$	8	No	Forms cycle ($e-c-a-d-f-e$)
$f-g$	9	Yes	No cycle. Done ($ V - 1 = 6$ edges)

Visualizing Kruskal’s Progress (Step-by-Step):

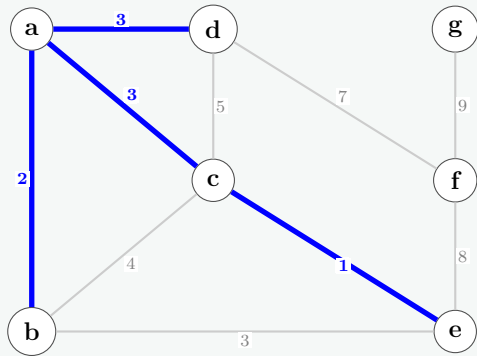
Step 1: Add edge $ce(1)$.

Step 2: Add edge $ab(2)$.

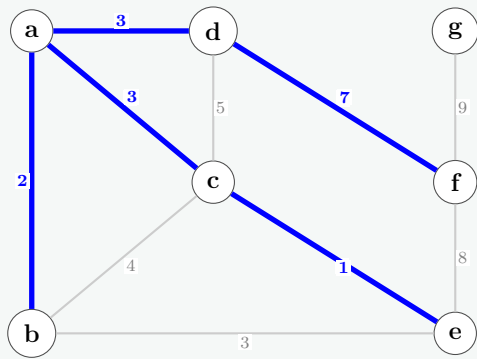
Step 3: Add edge **ac(3)**.



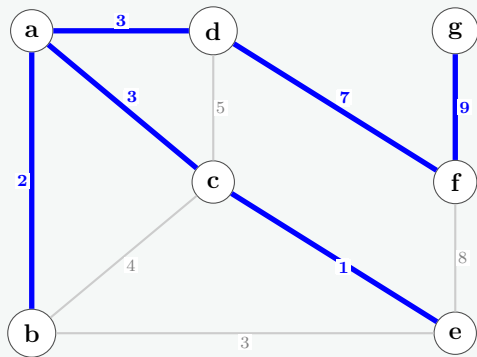
Step 4: Add edge **ad(3)**. (Edges *be*, *bc*, *cd* are discarded next as they form cycles).



Step 5: Add edge **df(7)**. (Edge *ef* is discarded next as it forms a cycle).



Step 6: Final MST. Add edge **fg(9)**.



Kruskal MST Cost: $1 + 2 + 3 + 3 + 7 + 9 = 25$.

Part 2: Prim’s Algorithm (Using Key Values)

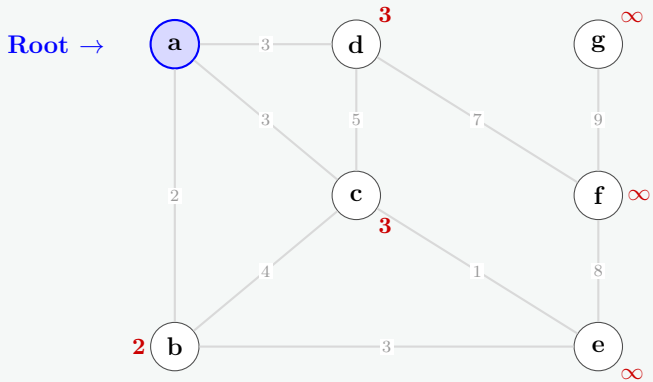
Start at vertex **a**. We maintain a **Key** value for each vertex (initialized to ∞ , meaning unreachable) and its **Parent**. In each step, we extract the unvisited vertex with the minimum key. Ties are broken alphabetically.

Key and Parent Tracking Table:

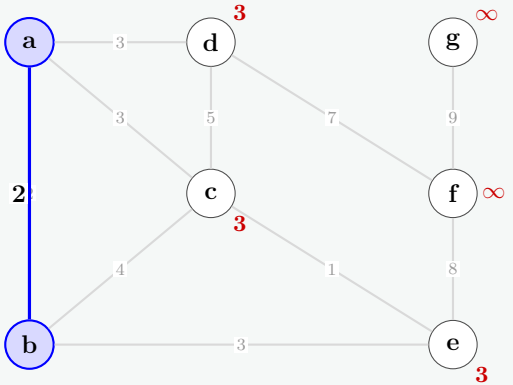
Notation: Key(Parent). ‘-’ means the vertex is already included in the MST.

Step	Extracted Node	b	c	d	e	f	g
0	Initial State	∞	∞	∞	∞	∞	∞
1	a	2(a)	3(a)	3(a)	∞	∞	∞
2	b	-	3(a)	3(a)	3(b)	∞	∞
3	c	-	-	3(a)	1(c)	∞	∞
4	e	-	-	3(a)	-	8(e)	∞
5	d	-	-	-	-	7(d)	∞
6	f	-	-	-	-	-	9(f)
7	g	-	-	-	-	-	-

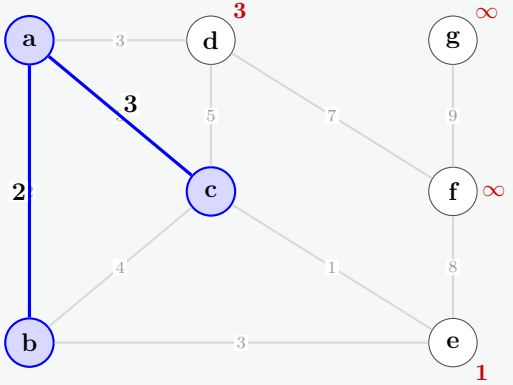
Intermediate Steps Visualization:
(Blue shaded nodes are in the MST. Red values indicate the current minimum cost to connect that unvisited node).



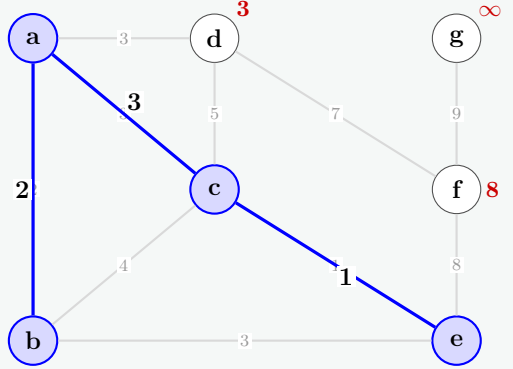
Step 0: Add a



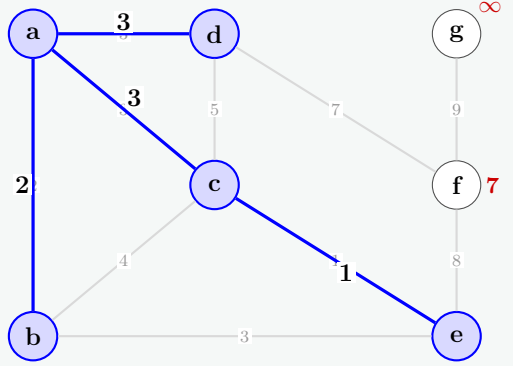
Step 1: Add b



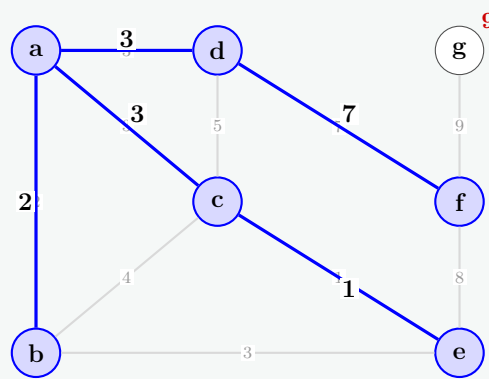
Step 2: Add c



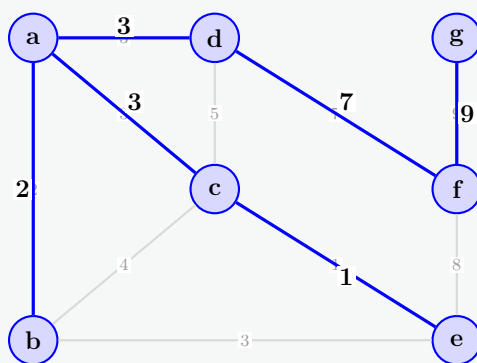
Step 3: Add e



Step 4: Add d



Step 5: Add f



Step 6: Add g (Final MST)

Prim MST edges: $ab(2)$, $ac(3)$, $ce(1)$, $ad(3)$, $df(7)$, $fg(9)$.

Total Cost = $2 + 3 + 1 + 3 + 7 + 9 = 25$.

Q17. Let $G = (V, E)$ be a connected undirected graph with weight function w on E . Let $A \subseteq E$ be included in some MST for G , let $(S, V - S)$ be any cut that respects A , and let (u, v) be a light edge crossing $(S, V - S)$. Prove that (u, v) is safe for A . **(10 marks)**
[CSE 207 | L-2/T-2 | 2014–15]

Answer

1. Definitions and Terminology

Before beginning the proof, let us establish the precise mathematical definitions from the theorem statement:

- **Cut:** A partition of the vertex set V into two disjoint, non-empty sets S and $V - S$.
- **Crossing the Cut:** An edge (x, y) crosses the cut $(S, V - S)$ if one of its endpoints is in S and the other endpoint is in $V - S$.
- **Respects:** A cut $(S, V - S)$ respects a set of edges A if no edge in A crosses the cut.
- **Light Edge:** An edge (u, v) is a light edge crossing a cut if its weight $w(u, v)$ is minimized among all edges crossing that specific cut.
- **Safe Edge:** An edge (u, v) is safe for A if the union $A \cup \{(u, v)\}$ is a subset of some valid Minimum Spanning Tree (MST) for G .

2. The Proof (Cut-Exchange Argument)

Step 1: Establish the Baseline MST

By the theorem's premise, the edge set A is included in some Minimum Spanning Tree of G . Let us denote this baseline MST as T . Therefore:

$$A \subseteq T$$

We must demonstrate that there exists *some* MST T' (which could be T itself or a newly constructed MST) such that $A \cup \{(u, v)\} \subseteq T'$. We analyze two mutually exclusive scenarios:

- **Case 1:** $(u, v) \in T$
If the light edge (u, v) is already an edge in our chosen MST T , then $A \cup \{(u, v)\} \subseteq T$ holds immediately. Thus, (u, v) is trivially safe for A , and the proof concludes for this case.
- **Case 2:** $(u, v) \notin T$
If (u, v) is not present in T , we must mathematically construct an alternative minimum spanning tree T' that encompasses both A and (u, v) .

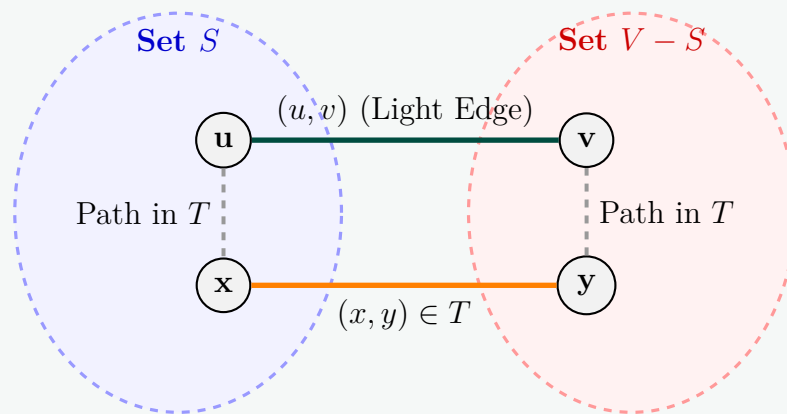


Figure 1: Cycle exchange mechanism illustrating (u, v) and (x, y) crossing the cut.

Step 2: Cycle Creation via Edge Insertion

Because T is a spanning tree, it provides exactly one unique, simple path between any pair of vertices. Inserting the edge (u, v) into T creates exactly one unique, simple cycle C containing the edge (u, v) .

Step 3: Locating an Alternative Crossing Edge

The edge (u, v) crosses the cut $(S, V - S)$ because $u \in S$ and $v \in V - S$. A continuous cycle cannot cross a boundary exactly once. Since the cycle C travels from S to $V - S$ via (u, v) and must eventually return to $u \in S$, there must exist at least one other edge (x, y) in the cycle C that crosses the cut back from S to $V - S$ (where $x \in S$ and $y \in V - S$).

Step 4: Comparing Edge Properties

We can deduce two crucial facts regarding this alternative crossing edge (x, y) :

- (a) **It cannot belong to A :** The theorem states that the cut $(S, V - S)$ *respects* A , meaning no edge in A crosses the cut. Since (x, y) crosses the cut, it is mathematically impossible for (x, y) to be an element of A . Hence, $(x, y) \notin A$.
- (b) **Weight relationship:** The theorem states that (u, v) is a *light edge* crossing the cut. This implies its weight is less than or equal to any other edge crossing this specific cut. Because (x, y) is also a crossing edge, it must be true that:

$$w(u, v) \leq w(x, y)$$

Step 5: Constructing the New Tree T'

We construct a new edge set T' by executing an exchange operation: we add the light edge (u, v) to T and remove the alternative crossing edge (x, y) .

$$T' = T \setminus \{(x, y)\} \cup \{(u, v)\}$$

Let us verify the mathematical validity of T' :

- **Spanning Tree Validity:** Removing (x, y) breaks the unique cycle C created by inserting (u, v) . The resulting graph remains fully connected, contains no cycles, and spans V . Thus, T' is a valid spanning tree.
- **Minimality Analysis:** We calculate the total weight of T' relative to T :

$$w(T') = w(T) - w(x, y) + w(u, v)$$

Since $w(u, v) \leq w(x, y)$, substituting this yields:

$$w(T') \leq w(T)$$

However, T was defined as a *Minimum* Spanning Tree, meaning no spanning tree can have a weight strictly less than $w(T)$. Therefore, the weights must be exactly equal:

$$w(T') = w(T)$$

This proves that T' is also a valid Minimum Spanning Tree of G .

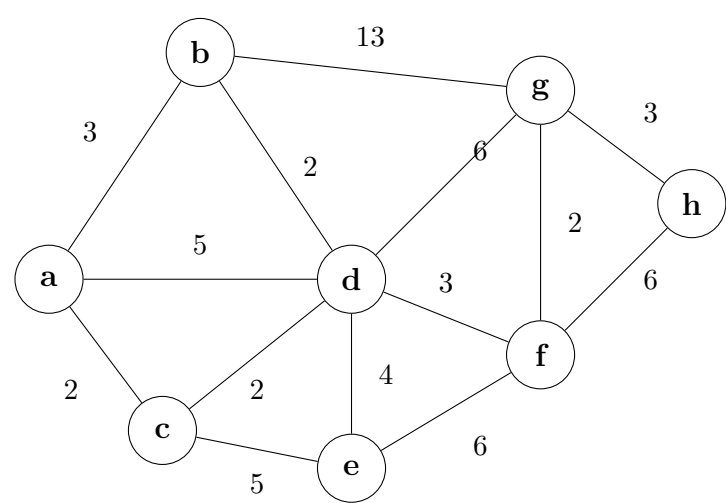
3. Conclusion

We have successfully constructed an alternative MST T' such that:

- (a) $(u, v) \in T'$ by explicit insertion.
- (b) $A \subseteq T'$ because $A \subseteq T$, and the only edge removed from T to build T' was (x, y) , which we proved in Step 4 was not an element of A .

Consequently, $A \cup \{(u, v)\} \subseteq T'$. By definition, this conclusively proves that the light edge (u, v) is **safe** for A . ■

Q18. Find the light and respectful crossing edges for each step of Kruskal's method for building the MST.



(12 marks)

[CSE 207 | L-2/T-2 | 2014–15]

Answer

To trace Kruskal’s algorithm and find a corresponding **respectful** and **light** crossing edge for each successful step, we first list all the edges of the graph sorted in non-decreasing order of their weights:

Weight	Edges
2	(a, c), (b, d), (c, d), (g, f)
3	(a, b), (d, f), (g, h)
4	(d, e)
5	(a, d), (c, e)
6	(d, g), (e, f), (h, f)
13	(b, g)

Let A denote the growing set of edges in the MST forest. Initially, $A = \emptyset$. For each step where an edge is accepted, we define a cut $(S, V \setminus S)$ that **respects** A (no edges in A cross it) and for which the chosen edge is a **light edge** (has the minimum weight among all edges crossing that cut).

—

Step-by-Step Execution of Kruskal’s Algorithm

Step 1: Consider edge (a, c) with weight 2

- **Action:** Accept (does not form a cycle). New forest $A = \{(a, c)\}$.
- **Respectful Cut:** Let $S = \{a\}$, so $V \setminus S = \{b, c, d, e, f, g, h\}$. This respects $A = \emptyset$.
- **Crossing Edges:** $\{(a, b) : 3, (a, d) : 5, (a, c) : 2\}$.
- **Light Edge:** (a, c) has the minimum weight (2) crossing this cut.

Step 2: Consider edge (b, d) with weight 2

- **Action:** Accept (does not form a cycle). New forest $A = \{(a, c), (b, d)\}$.
- **Respectful Cut:** Let $S = \{b\}$, so $V \setminus S = \{a, c, d, e, f, g, h\}$. This respects A because the only existing edge (a, c) has both endpoints in $V \setminus S$.
- **Crossing Edges:** $\{(b, a) : 3, (b, d) : 2, (b, g) : 13\}$.
- **Light Edge:** (b, d) has the minimum weight (2) crossing this cut.

Step 3: Consider edge (c, d) with weight 2

- **Action:** Accept (connects components $\{a, c\}$ and $\{b, d\}$). New forest $A = \{(a, c), (b, d), (c, d)\}$.
- **Respectful Cut:** Let $S = \{a, c\}$, so $V \setminus S = \{b, d, e, f, g, h\}$. This respects A because (a, c) lies entirely within S , and (b, d) lies entirely within $V \setminus S$.
- **Crossing Edges:** $\{(a, b) : 3, (a, d) : 5, (c, d) : 2, (c, e) : 5\}$.
- **Light Edge:** (c, d) has the minimum weight (2) crossing this cut.

Step 4: Consider edge (g, f) with weight 2

- **Action:** Accept (does not form a cycle). New forest $A = \{(a, c), (b, d), (c, d), (g, f)\}$.
- **Respectful Cut:** Let $S = \{g\}$, so $V \setminus S = \{a, b, c, d, e, f, h\}$. This respects A because all three existing edges lie entirely within $V \setminus S$.
- **Crossing Edges:** $\{(g, b) : 13, (g, d) : 6, (g, f) : 2, (g, h) : 3\}$.
- **Light Edge:** (g, f) has the minimum weight (2) crossing this cut.

Step 5: Consider edge (a, b) with weight 3

- **Action: Reject.** Vertices a and b already belong to the same connected component $\{a, b, c, d\}$, so adding it would form a cycle.

Step 6: Consider edge (d, f) with weight 3

- **Action:** Accept (connects components $\{a, b, c, d\}$ and $\{f, g\}$). New forest $A = \{(a, c), (b, d), (c, d), (g, f), (d, f)\}$.
- **Respectful Cut:** Let $S = \{f, g\}$, so $V \setminus S = \{a, b, c, d, e, h\}$. This cut respects A because prior edges are either entirely in $V \setminus S$ or entirely in S [edge (g, f)].
- **Crossing Edges:** $\{(f, d) : 3, (f, e) : 6, (f, h) : 6, (g, b) : 13, (g, d) : 6, (g, h) : 3\}$.
- **Light Edge:** (d, f) is a light edge because its weight (3) is tied for the minimum crossing weight along with (g, h) .

Step 7: Consider edge (g, h) with weight 3

- **Action:** Accept (connects component $\{a, b, c, d, f, g\}$ to $\{h\}$). New forest $A = \dots \cup \{(g, h)\}$.
- **Respectful Cut:** Let $S = \{h\}$, so $V \setminus S = \{a, b, c, d, e, f, g\}$. This respects A because all prior edges avoid vertex h entirely and remain within $V \setminus S$.
- **Crossing Edges:** $\{(h, g) : 3, (h, f) : 6\}$.
- **Light Edge:** (g, h) has the minimum weight (3) crossing this cut.

Step 8: Consider edge (d, e) with weight 4

- **Action:** Accept (connects the main component to $\{e\}$). New forest $A = \dots \cup \{(d, e)\}$.
- **Respectful Cut:** Let $S = \{e\}$, so $V \setminus S = \{a, b, c, d, f, g, h\}$. This respects A because all previously selected edges avoid node e and reside entirely inside $V \setminus S$.
- **Crossing Edges:** $\{(e, c) : 5, (e, d) : 4, (e, f) : 6\}$.
- **Light Edge:** (d, e) has the minimum weight (4) crossing this cut.

Summary of the Final MST Edge Selection Sequence

The algorithm terminates here because we have successfully selected $|V| - 1 = 7$ edges, spanning all 8 vertices without any cycles.

Kruskal Step	Edge Selected	Weight	Cut Used (S)	Light Edge Verification
1	(a, c)	2	$\{a\}$	$2 = \min(3, 5, 2)$
2	(b, d)	2	$\{b\}$	$2 = \min(3, 2, 13)$
3	(c, d)	2	$\{a, c\}$	$2 = \min(3, 5, 2, 5)$
4	(g, f)	2	$\{g\}$	$2 = \min(13, 6, 2, 3)$
5	(d, f)	3	$\{f, g\}$	$3 = \min(3, 6, 6, 13, 6, 3)$
6	(g, h)	3	$\{h\}$	$3 = \min(3, 6)$
7	(d, e)	4	$\{e\}$	$4 = \min(5, 4, 6)$

Q19. What is the cost of finding and joining (union) of sets in Kruskal's method? (8 marks)

[CSE 207 | L-2/T-2 | 2014–15]

Answer**1. The Disjoint-Set (Union-Find) Data Structure**

In Kruskal's algorithm, the problem of detecting whether adding an edge creates a cycle is solved using a **Disjoint-Set** (or Union-Find) data structure. This structure maintains a collection of disjoint dynamic sets, where each set represents a connected component in the growing spanning forest.

The algorithm relies on three fundamental operations:

- **Make-Set(v):** Creates a new set containing the single vertex v .
- **Find-Set(v):** Returns the representative (or root) element of the set containing v . If two vertices yield the same representative, they belong to the same component (and connecting them would form a cycle).
- **Union(u, v):** Merges the two sets containing u and v into a single connected component.

2. Total Operation Counts in Kruskal's Algorithm

Let $|V|$ be the number of vertices and $|E|$ be the number of edges in the graph. During the execution of Kruskal's algorithm, the operations are called as follows:

- (a) **Make-Set:** Executed exactly once for each vertex during initialization. Total calls: $|V|$.
- (b) **Find-Set:** Executed twice for every edge in the graph (once for each endpoint) when iterating through the sorted edge list to check for cycles. Total calls: $2|E|$.
- (c) **Union:** Executed only when a valid, cycle-free edge is added to the Minimum Spanning Tree. Since a spanning tree has exactly $|V| - 1$ edges, this is called at most $|V| - 1$ times.

Thus, we are executing a sequence of $\mathcal{O}(|V| + |E|)$ total operations, which simplifies to $\mathcal{O}(|E|)$ operations since the graph is connected and $|E| \geq |V| - 1$.

3. Core Optimizations

To achieve maximum efficiency, two critical heuristics are applied to the Disjoint-Set data structure:

- **Union by Rank / Size:** When merging two trees, the root of the smaller tree is made a child of the root of the larger tree. This prevents the trees from becoming heavily unbalanced linear chains, keeping the maximum depth logarithmic.
- **Path Compression:** During a **Find-Set** traversal, every node visited along the path to the root is directly attached to the root. This flattens the tree structure, making subsequent lookups for those nodes occur in effectively constant time.

4. Exact Time Complexity Cost

When both **Union by Rank** and **Path Compression** are implemented, the mathematical bounds become extremely tight. A sequence of m operations on n elements takes total time:

$$\mathcal{O}(m\alpha(n))$$

where $\alpha(n)$ is the **inverse Ackermann function**.

Substituting our algorithm's parameters ($m = |E|$ operations on $n = |V|$ vertices), the total cost of all finding and joining operations in Kruskal's algorithm is:

$$\text{Total Cost: } \mathcal{O}(|E|\alpha(|V|))$$

Practical Significance of $\alpha(|V|)$

The inverse Ackermann function $\alpha(n)$ grows incredibly slowly. For any practical graph that could theoretically exist inside the observable universe (up to $n = 2^{65536}$ vertices), the value of $\alpha(n)$ is strictly less than or equal to 4.

Therefore, for all practical computing purposes, $\alpha(|V|)$ behaves as an absolute constant $\mathcal{O}(1)$. Consequently, the *amortized* cost per individual **Find-Set** or **Union** operation is $\mathcal{O}(1)$, making the practical total cost of maintaining the disjoint sets directly proportional to the number of edges:

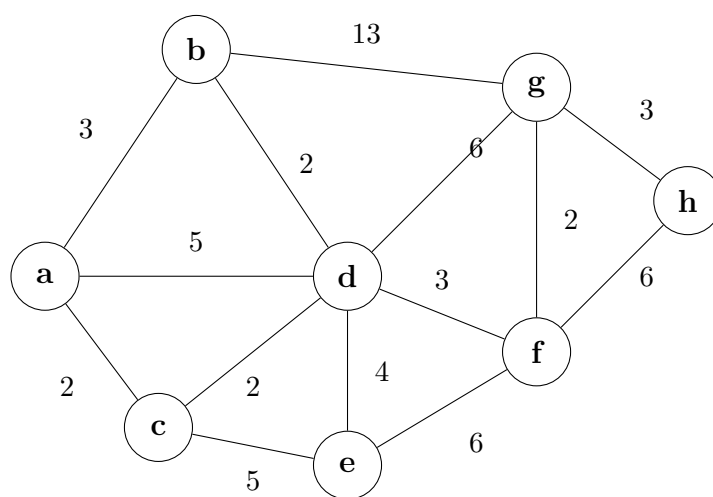
$$\text{Practical Total Cost: } \mathcal{O}(|E|)$$

Q20. For the graph below, show the value of u , $v.key$, Q for the following algorithm (show each step):

Algorithm 22 MST(G, w, r)

```

1:  $Q \leftarrow V[G]$ 
2: for each  $u \in Q$  do
3:    $u.key \leftarrow \infty$ 
4: end for
5:  $r.key \leftarrow 0$ ;  $p[r] \leftarrow \text{nil}$ 
6: while  $Q \neq \emptyset$  do
7:    $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
8:   for each  $v \in \text{Adj}[u]$  do
9:     if  $v \in Q$  and  $w(u, v) < v.key$  then
10:       $p[v] \leftarrow u$ 
11:       $v.key \leftarrow w(u, v)$ 
12:     end if
13:   end for
14: end while
```



(10 marks)

[CSE 207 | L-2/T-2 | 2014–15]

Answer**Algorithm Trace: Prim's Minimum Spanning Tree**

The given pseudocode executes **Prim's Algorithm**. Since the root vertex r is not explicitly specified in the prompt, we will use standard convention and assume $r = a$ as the starting node.

Initialization (Lines 1-4)

- Q is initialized with all vertices: $Q = \{a, b, c, d, e, f, g, h\}$
- All keys are set to ∞ , except $a.key = 0$.

Step-by-Step Execution (Lines 5-11)

At each step, we extract the vertex u with the minimum key from Q . Then, we look at its adjacent neighbors v . If $v \in Q$ and the edge weight $w(u, v) < v.key$, we update $v.key$ to $w(u, v)$.

Step 1:

- **Extract-Min:** $u = a$ (key = 0)
- **Remaining Q :** $\{b, c, d, e, f, g, h\}$
- **Neighbors of a in Q :** b (weight 3), c (weight 2), d (weight 5).
- **Updates:** $b.key \leftarrow 3$, $c.key \leftarrow 2$, $d.key \leftarrow 5$.

Step 2:

- **Extract-Min:** $u = c$ (minimum key in Q is 2)
- **Remaining Q :** $\{b, d, e, f, g, h\}$
- **Neighbors of c in Q :** d (weight 2), e (weight 5).
- **Updates:** $d.key \leftarrow \min(5, 2) = 2$, $e.key \leftarrow 5$.

Step 3:

- **Extract-Min:** $u = d$ (minimum key in Q is 2) *Note: b also has key 2, either can be picked; choosing d first.*
- **Remaining Q :** $\{b, e, f, g, h\}$
- **Neighbors of d in Q :** b (weight 2), e (weight 4), f (weight 3), g (weight 6).
- **Updates:** $b.key \leftarrow \min(3, 2) = 2$, $e.key \leftarrow \min(5, 4) = 4$,
 $f.key \leftarrow 3$, $g.key \leftarrow 6$.

Step 4:

- **Extract-Min:** $u = b$ (minimum key in Q is 2)
- **Remaining Q :** $\{e, f, g, h\}$
- **Neighbors of b in Q :** g (weight 13).
- **Updates:** $13 \not< 6$, so $g.key$ remains 6. No updates.

Step 5:

- **Extract-Min:** $u = f$ (minimum key in Q is 3)
- **Remaining Q :** $\{e, g, h\}$

- **Neighbors of f in Q :** e (weight 6), g (weight 2), h (weight 6).
- **Updates:** $e.key \leftarrow \min(4, 6) = 4$ (no change),
 $g.key \leftarrow \min(6, 2) = 2$, $h.key \leftarrow 6$.

Step 6:

- **Extract-Min:** $u = g$ (minimum key in Q is 2)
- **Remaining Q :** $\{e, h\}$
- **Neighbors of g in Q :** h (weight 3).
- **Updates:** $h.key \leftarrow \min(6, 3) = 3$.

Step 7:

- **Extract-Min:** $u = h$ (minimum key in Q is 3)
- **Remaining Q :** $\{e\}$
- **Neighbors of h in Q :** None in Q .
- **Updates:** None.

Step 8:

- **Extract-Min:** $u = e$ (minimum key in Q is 4)
- **Remaining Q :** $\emptyset$ (Empty)
- **Neighbors of e in Q :** None.
- **Updates:** None.

Summary State Table

The table below summarizes the value of Q prior to extraction, the extracted node u , and the final state of all $v.key$ values at the end of that step. Once a vertex is extracted from Q , its key is fixed (denoted with an asterisk *).

Extracted u	Set Q after extraction	a	b	c	d	e	f	g	h
(Init)	$\{a, b, c, d, e, f, g, h\}$	0	∞	∞	∞	∞	∞	∞	∞
a	$\{b, c, d, e, f, g, h\}$	0*	3	2	5	∞	∞	∞	∞
c	$\{b, d, e, f, g, h\}$	0*	3	2*	2	5	∞	∞	∞
d	$\{b, e, f, g, h\}$	0*	2	2*	2*	4	3	6	∞
b	$\{e, f, g, h\}$	0*	2*	2*	2*	4	3	6	∞
f	$\{e, g, h\}$	0*	2*	2*	2*	4	3*	2	6
g	$\{e, h\}$	0*	2*	2*	2*	4	3*	2*	3
h	$\{e\}$	0*	2*	2*	2*	4	3*	2*	3*
e	$\emptyset$	0*	2*	2*	2*	4*	3*	2*	3*

Q21. *GreedyCell*, a mobile phone operator, wants to place cell towers (base stations) along a long straight country road, to provide network coverage to n houses sparsely scattered along the road. The distance of these houses from the start of the road are, in miles and in increasing order, $d_1, d_2, \dots, d_n$. Each cell tower has coverage of R miles, i.e., if a house is within R miles of a tower, it would get service from that tower. Now give an efficient algorithm for finding a placement of cell towers that uses as few cell towers as possible to provide coverage to all the n houses. **(12 marks)** [CSE 207 | L-2/T-2 | 2014–15]

Answer

1. Core Insight: The Greedy Strategy

This problem is a classic application of the **1D Interval Covering** greedy algorithm.

To minimize the number of cell towers, each tower should cover as much "new" ground as possible. If we look at the first uncovered house at distance d_1 , a tower must be placed within R miles of it to provide service.

To maximize the coverage for the remaining houses further down the road, we should place this tower as far to the right of d_1 as possible. Therefore, the optimal placement for the first tower is exactly at $p_1 = d_1 + R$.

- This tower will cover all houses located in the interval $[d_1, d_1 + 2R]$.
- We then find the next house that falls strictly outside this coverage range (i.e., $d_i > d_1 + 2R$) and repeat the exact same process by placing the next tower at $d_i + R$.

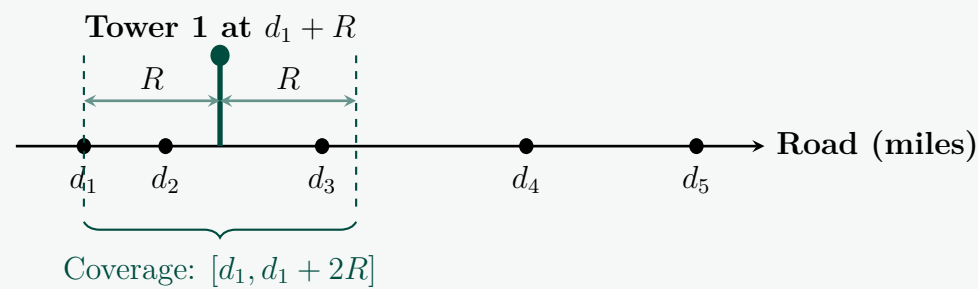


Figure 1: Maximizing the greedy choice by placing the tower R miles ahead of the leftmost uncovered house.

2. Algorithm / Pseudocode

Algorithm 23 Optimal Cell Tower Placement

Input: Array of house distances $D = [d_1, d_2, \dots, d_n]$ (sorted in increasing order), coverage radius R .

Output: List of optimal tower locations T .

```

1:  $T \leftarrow \emptyset$ 
2:  $i \leftarrow 1$ 
3: while  $i \leq n$  do
4:    $p \leftarrow d_i + R$                                 ▷ Place tower to cover the leftmost uncovered house
5:    $T.append(p)$ 
6:    $i \leftarrow i + 1$ 
7:   while  $i \leq n$  and  $d_i \leq p + R$  do              ▷ Skip all houses covered by this tower
8:      $i \leftarrow i + 1$ 
9:   end while
10: end while
11: return  $T$ 

```

3. Complexity Analysis

Time Complexity:

Although there is a nested **while** loop, notice that the index i only ever moves forward. Every house in the array D is evaluated exactly once and skipped over once it falls within a covered range. Since the array is already given in increasing order, no sorting is required.

Total Time Complexity: $\mathcal{O}(n)$

Space Complexity:

The algorithm only requires a few variable counters (i, p) and a list T to store the output coordinates of the towers. In the worst-case scenario (where houses are so far apart that every house requires its own tower), T will store n elements.

Total Space Complexity: $\mathcal{O}(n)$

(If excluding the output array size from auxiliary space, the auxiliary space is strictly $\mathcal{O}(1)$).

Q22. Given a directed graph with edge list and weights $AB(5), BC(2), AC(4), AD(6), CD(3), DB(1), DE(6), EB(3), EF(3), FB(4)$, construct a minimum-cost arborescence rooted at A . **(10 marks)** [CSE 207 | L-2/T-2 | 2013–14]

Answer

To construct the minimum-cost arborescence (directed minimum spanning tree) rooted at A , we apply the **Chu-Liu/Edmonds' Algorithm**. The algorithm iteratively reduces incoming edge weights to find a zero-weight spanning arborescence. If a cycle of zero-weight edges is formed, it contracts the cycle into a super-node and recursively repeats the process.

Step 1: Find Minimum Incoming Edge Weights

First, we find the minimum weight of an incoming edge for every node except the root A . Let this minimum incoming weight for a node v be denoted as $min_in(v)$.

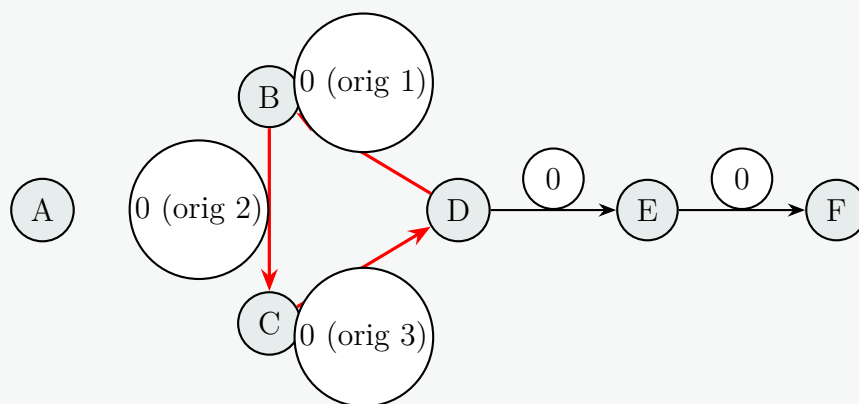
Node	Incoming Edges (Weight)	$\min_in(v)$
B	$AB(5), DB(1), EB(3), FB(4)$	1 (from D)
C	$BC(2), AC(4)$	2 (from B)
D	$AD(6), CD(3)$	3 (from C)
E	$DE(6)$	6 (from D)
F	$EF(3)$	3 (from E)

Step 2: Reduce Edge Weights and Find 0-Weight Cycle

We subtract $\min_in(v)$ from the weight of every incoming edge to node v . The new reduced weight is $w'(u, v) = w(u, v) - \min_in(v)$. We then select one 0-weight incoming edge for each non-root node.

- To B : $AB(4), \mathbf{DB(0)}, EB(2), FB(3)$
- To C : $\mathbf{BC(0)}, AC(2)$
- To D : $AD(3), \mathbf{CD(0)}$
- To E : $\mathbf{DE(0)}$
- To F : $\mathbf{EF(0)}$

Selecting the 0-weight edges $\{DB, BC, CD, DE, EF\}$ creates a **cycle**: $B \xrightarrow{0} C \xrightarrow{0} D \xrightarrow{0} B$.



Step 3: Contract the Cycle

We contract the cycle $C_1 = \{B, C, D\}$ into a single super-node X . We must recalculate the reduced weights of edges entering and leaving X .

Edges entering X : The cost of an edge entering X at a node $v \in C_1$ is its current reduced weight.

- $A \rightarrow X$ (via B): $w'(AB) = 4$
- $A \rightarrow X$ (via C): $w'(AC) = 2 \leftarrow$ Minimum incoming from A
- $A \rightarrow X$ (via D): $w'(AD) = 3$
- $E \rightarrow X$ (via B): $w'(EB) = 2$
- $F \rightarrow X$ (via B): $w'(FB) = 3$

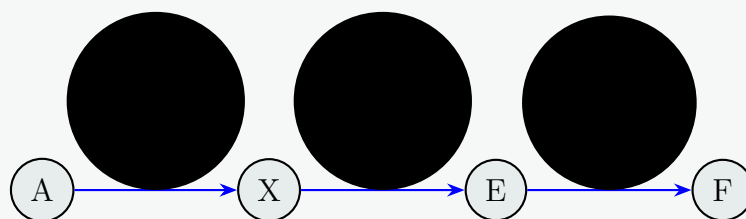
The minimum weight of an edge entering X is 2 (specifically, $A \rightarrow C$ and $E \rightarrow B$). We subtract this new minimum (2) from all edges entering X . The newly reduced edge $A \rightarrow X$ (via C) now has weight 0.

Edges leaving X : The cost is simply the weight of the 0-weight edge leaving the component.

- $X \rightarrow E$ (via $D \rightarrow E$): $w'(DE) = 0$

Step 4: Arborescence in the Contracted Graph

In the contracted graph with nodes $\{A, X, E, F\}$, the 0-weight edges are: $A \xrightarrow{0} X$ (via AC), $X \xrightarrow{0} E$ (via DE), and $E \xrightarrow{0} F$. This forms a valid arborescence without any cycles.



Step 5: Expand the Cycle

We expand super-node X back to B, C, D . The chosen incoming edge to X is $A \rightarrow C$. Because this edge enters the cycle at node C , we **must drop the cycle edge that enters C** . The cycle was $B \rightarrow C \rightarrow D \rightarrow B$. The edge entering C in the cycle is $B \rightarrow C$. We drop $B \rightarrow C$ and keep the remaining cycle edges: $C \rightarrow D$ and $D \rightarrow B$.

Step 6: Final Arborescence and Cost Calculation

The final edges in our Minimum-Cost Arborescence are:

- $A \rightarrow C$ (Weight = 4)
- $C \rightarrow D$ (Weight = 3)
- $D \rightarrow B$ (Weight = 1)
- $D \rightarrow E$ (Weight = 6)
- $E \rightarrow F$ (Weight = 3)

Total Minimum Cost = $4 + 3 + 1 + 6 + 3 = 17$.

Complexity Analysis

- Time Complexity:** $\mathcal{O}(VE)$. In the worst case, we might find a cycle and contract the graph $\mathcal{O}(V)$ times. In each iteration, finding the minimum incoming edges and updating weights takes $\mathcal{O}(E)$ time. (Note: using Fibonacci heaps and disjoint-set data structures, this can be optimized to $\mathcal{O}(E + V \log V)$).
- Space Complexity:** $\mathcal{O}(V + E)$ to store the graph representations, reduced edge weights, and tracking the parent pointers for cycles and super-nodes.

Q23. Find a minimum spanning tree using both Kruskal’s algorithm and Prim’s algorithm (starting from A) for the graph with edges:
 $AB(5), BC(2), AC(4), AD(6), CD(3),$
 $DB(1), DE(6), EB(3), EF(3), FB(4)$. (15 marks)

[CSE 207 | L-2/T-2 | 2013–14]

Answer

To find the Minimum Spanning Tree (MST) of the given undirected graph, we will apply both **Kruskal’s Algorithm** and **Prim’s Algorithm**. An MST for a graph with $V = 6$ vertices (A, B, C, D, E, F) will contain exactly $V - 1 = 5$ edges and will connect all vertices with the minimum possible total edge weight.

1. Kruskal’s Algorithm

Kruskal’s algorithm operates by sorting all edges in ascending order of their weights and greedily adding them to the MST, provided they do not form a cycle (managed efficiently via a Disjoint-Set / Union-Find data structure).

Step 1: Sort the edges by weight.

- $DB(1), BC(2), CD(3), EB(3), EF(3), AC(4), FB(4), AB(5), AD(6), DE(6)$

Step 2: Iteratively select edges.

Edge	Weight	Action / Reason	Edges in MST
DB	1	Add (Does not form a cycle)	1
BC	2	Add (Does not form a cycle)	2
CD	3	Skip (Forms cycle $D - B - C$)	2
EB	3	Add (Does not form a cycle)	3
EF	3	Add (Does not form a cycle)	4
AC	4	Add (Does not form a cycle)	5

We stop here because the MST has reached $V - 1 = 5$ edges.

MST Edges: $\{DB, BC, EB, EF, AC\}$

Total Cost: $1 + 2 + 3 + 3 + 4 = 13$

2. Prim’s Algorithm (Starting from A)

Prim’s algorithm builds the MST by maintaining a set of visited vertices and greedily adding the smallest edge that crosses the cut between the visited and unvisited sets.

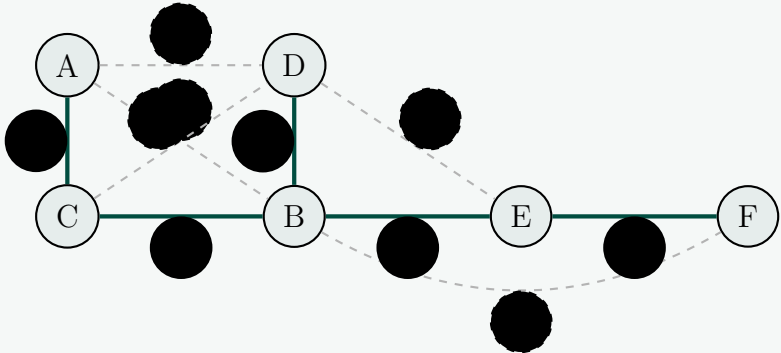
Initialization: Visited set $V_{mst} = \{A\}$.

87

Step	Visited Nodes (V_{mst})	Candidate Edges Crossing Cut	Edge Chosen	Cost
1	{A}	$AB(5), AC(4), AD(6)$	AC	4
2	{A, C}	$AB(5), AD(6), CB(2), CD(3)$	CB	2
3	{A, C, B}	$AD(6), CD(3), BD(1), BE(3), BF(4)$	BD	1
4	{A, C, B, D}	$BE(3), BF(4), DE(6)$ (CD, AD form cycles)	BE	3
5	{A, C, B, D, E}	$EF(3), BF(4)$ (DE forms cycle)	EF	3

MST Edges: {AC, CB, BD, BE, EF}
Total Cost: 4 + 2 + 1 + 3 + 3 = **13**
Both algorithms successfully yield the identical MST and minimum cost.

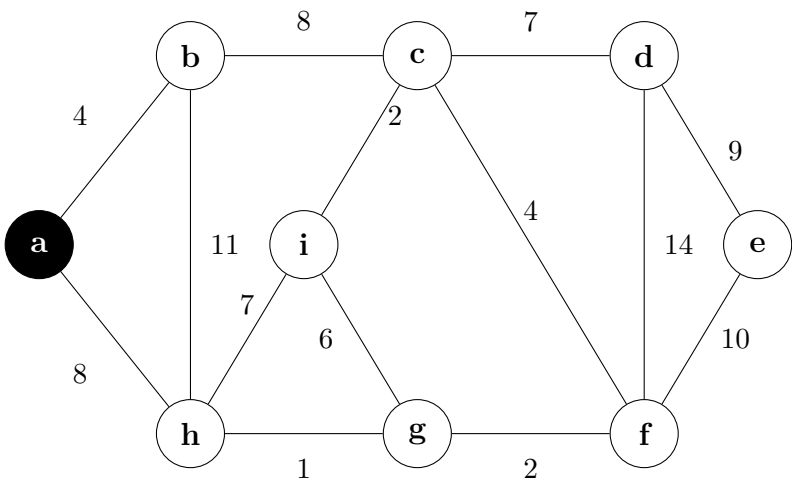
3. Minimum Spanning Tree Visualization



4. Complexity Analysis

- Kruskal’s Algorithm Time Complexity:** $\mathcal{O}(E \log E)$ or $\mathcal{O}(E \log V)$. Sorting the edges dominates the runtime at $\mathcal{O}(E \log E)$. The Disjoint-Set operations take near-constant time $\mathcal{O}(\alpha(V))$ per edge, bounded by $\mathcal{O}(E \log V)$.
- Prim’s Algorithm Time Complexity:** $\mathcal{O}(E + V \log V)$ when using a Fibonacci heap for the min-priority queue, or $\mathcal{O}(E \log V)$ using a standard binary min-heap. Finding the minimum edge crossing the cut efficiently is the core driver of Prim’s runtime.
- Space Complexity:** Both algorithms operate in $\mathcal{O}(V + E)$ space to store the graph structures (Adjacency List/Edge List) and the priority queues/Disjoint-Set arrays.

Q24. If Prim’s algorithm is applied on a given graph, which will be the fifth edge added to the spanning tree? Start from vertex *a*. **(10 marks)** [CSE 207 | L-2/T-2 | 2012–13]

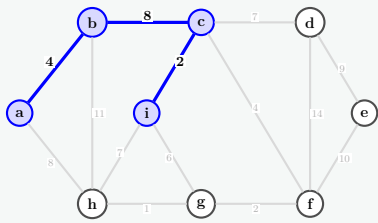


Answer: Prim’s Algorithm

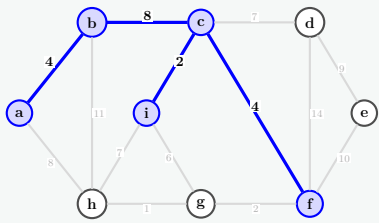
Start Node: *a*.
When ties occur (e.g., between edges *a-h*(8) and *b-c*(8)), we pick alphabetically (adding *c* instead of *h*).

Step	Vertex Added	Edge Added	Edge #	Key Values (Selected Unvisited)
1	<i>a</i>	—	—	<i>b</i> : 4, <i>h</i> : 8
2	<i>b</i>	<i>a</i> – <i>b</i> (4)	1st Edge	<i>c</i> : 8, <i>h</i> : 8
3	<i>c</i>	<i>b</i> – <i>c</i> (8)	2nd Edge	<i>d</i> : 7, <i>f</i> : 4, <i>i</i> : 2, <i>h</i> : 8
4	<i>i</i>	<i>c</i> – <i>i</i> (2)	3rd Edge	<i>d</i> : 7, <i>f</i> : 4, <i>g</i> : 6, <i>h</i> : 8
5	<i>f</i>	<i>c</i> – <i>f</i> (4)	4th Edge	<i>d</i> : 7, <i>g</i> : 2, <i>h</i> : 7
6	<i>g</i>	<i>f</i>–<i>g</i> (2)	5th Edge	<i>d</i> : 7, <i>h</i> : 1
7	<i>h</i>	<i>g</i> – <i>h</i> (1)	6th Edge	<i>d</i> : 7
8	<i>d</i>	<i>c</i> – <i>d</i> (7)	7th Edge	<i>e</i> : 9
9	<i>e</i>	<i>d</i> – <i>e</i> (9)	8th Edge	—

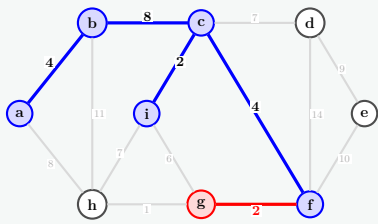
Visual Trace (Highlighting the 5th Edge):



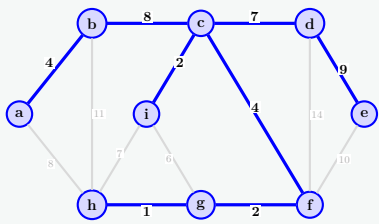
Step 4: Added c-i (3rd Edge)



Step 5: Added c-f (4th Edge)



Step 6: Added f-g (5th Edge)



Final Minimum Spanning Tree

Final Answer: The 5th edge added to the spanning tree is **f–g** (weight 2).
MST Total Cost = 4 + 8 + 2 + 4 + 2 + 1 + 7 + 9 = **37**. ✓

Q25. State and prove the “cut property” and the “cycle property”. Explain with necessary examples how these properties are useful in solving the minimum spanning tree problem. **(15 marks)** [CSE 207 | L-2/T-2 | 2011–12]

Answer

The greedy strategies used to solve the Minimum Spanning Tree (MST) problem rely on two fundamental graph-theoretic properties: the **Cut Property** and the **Cycle Property**. These properties govern which edges must be included in an MST and which edges can be safely discarded.

1. The Cut Property

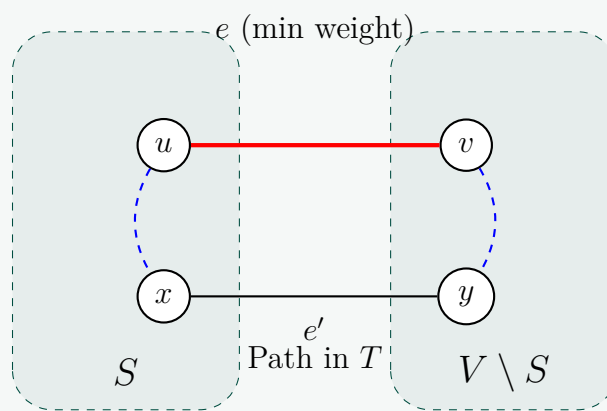
Statement: Let $G = (V, E)$ be a connected, undirected graph with a weight function $w : E \rightarrow \mathbb{R}$. Let $(S, V \setminus S)$ be any cut (a partition of the vertices into two disjoint sets). If $e = (u, v)$ is the strictly minimum-weight edge crossing this cut (with $u \in S$ and $v \in V \setminus S$), then e must belong to *every* minimum spanning tree of G . (If the minimum weight is not unique, e belongs to *some* MST).

Proof (by contradiction/exchange argument):

- (a) Assume there exists an MST T of G that *does not* contain the minimum-weight crossing edge $e = (u, v)$.
- (b) Because T is a spanning tree, it must contain exactly one simple path P connecting node u to node v .
- (c) Since $u \in S$ and $v \in V \setminus S$, the path P must cross the cut at least once. Let $e' = (x, y)$ be an edge on this path that crosses the cut, with $x \in S$ and $y \in V \setminus S$.
- (d) By our initial premise, $w(e) < w(e')$.
- (e) If we remove e' from T , the tree becomes disconnected into two components. If we then add e back, the two components are reconnected, forming a new spanning tree $T' = T - \{e'\} \cup \{e\}$.
- (f) The total weight of the new tree is:

$$w(T') = w(T) - w(e') + w(e)$$

Since $w(e) < w(e')$, it follows that $w(T') < w(T)$.
- (g) This contradicts the assumption that T is a Minimum Spanning Tree. Therefore, e must be in the MST. ■



2. The Cycle Property

Statement: Let $G = (V, E)$ be a connected, undirected graph with a weight function w . Let C be any cycle in G . If $e = (u, v)$ is the strictly heaviest edge in C , then e *cannot* belong to any Minimum Spanning Tree of G .

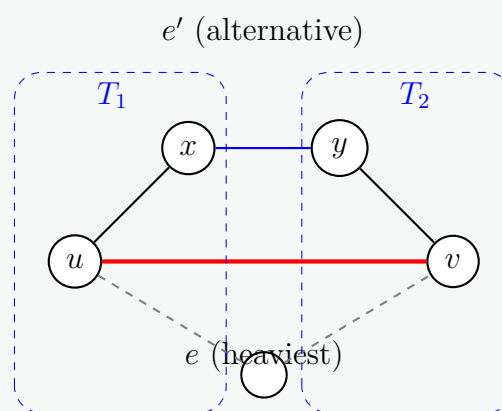
Proof (by contradiction/exchange argument):

- Assume there exists an MST T that *does* contain the heaviest edge $e = (u, v)$ from cycle C .
- If we remove e from T , the tree is partitioned into two disconnected components, say T_1 and T_2 , with $u \in T_1$ and $v \in T_2$.
- The cycle C forms a closed loop containing e . If e is removed, the remaining edges of C form a path connecting u and v .
- This path must contain at least one other edge $e' = (x, y)$ that has one endpoint in T_1 and the other in T_2 .
- By our premise, e is the strictly heaviest edge in C , so $w(e') < w(e)$.
- If we add e' to reconnect T_1 and T_2 , we form a new spanning tree $T' = T - \{e\} \cup \{e'\}$.
- The total weight of the new tree is:

$$w(T') = w(T) - w(e) + w(e')$$

Since $w(e') < w(e)$, it follows that $w(T') < w(T)$.

- This contradicts the assumption that T is an MST. Therefore, e cannot be in the MST. ■



3. Usefulness in Solving the MST Problem

The Cut and Cycle properties are the driving forces behind all standard greedy MST algorithms. By applying these properties, we avoid the $\mathcal{O}(2^E)$ brute-force evaluation of all spanning trees.

- Prim's Algorithm (Applies the Cut Property):** Prim's algorithm actively maintains a cut $(S, V \setminus S)$, where S is the set of vertices already added to the MST. At each step, it looks at all edges crossing the cut. By the **Cut Property**, it is guaranteed that picking the minimum-weight crossing edge will yield a valid MST edge. It greedily incorporates this edge and moves the corresponding vertex into S , repeating until all vertices are spanned.
- Kruskal's Algorithm (Applies Both Properties):** Kruskal's algorithm sorts all edges by weight and processes them in ascending order.

- **Cycle Property justification:** If the next shortest edge forms a cycle with the edges already chosen, it means this edge is the heaviest edge in that newly formed cycle (because edges are processed in strictly ascending weight order). By the **Cycle Property**, we can safely discard it.
- **Cut Property justification:** If the edge connects two separate components (trees in the forest), we can view those two components as a cut $(S, V \setminus S)$. Since we are processing edges in ascending order, this edge is the minimum weight edge crossing that cut. By the **Cut Property**, it safely belongs to the MST.

Complexity Overview

Relying on these properties allows us to use efficient data structures (like Priority Queues for Prim's and Union-Find for Kruskal's) to find the MST in $\mathcal{O}(E \log V)$ time and $\mathcal{O}(V + E)$ space, guaranteeing global optimality through purely local, greedy decisions.

Q26. How is the minimum cost arborescence problem different from the minimum spanning tree problem? Construct a minimum cost arborescence for the graph with edges $AB(7), AC(3), DA(4), EA(1), BC(91), CF(9), BD(7), EB(2), FB(5), CE(4), DE(6), FD(3)$, with edge orientations from the first vertex to the second. Assume source vertex is A . **(5+5 marks)** [CSE 207 | L-2/T-2 | 2006–07]

Answer

1. Key Differences: Minimum Spanning Tree (MST) vs. Minimum Cost Arborescence (MCA)

While both problems seek a minimum-weight subgraph that spans all vertices without cycles, they differ fundamentally in graph structure, properties, and algorithmic complexity:

(a) Graph Edge Directionality:

- **MST:** Defined exclusively on **undirected graphs**. Edges have no orientation, meaning connectivity between any pair of nodes is symmetric.
- **MCA:** Defined on **directed graphs (digraphs)**. Edges have a strict orientation (from a source vertex to a target vertex), and connectivity is asymmetric.

(b) Structural and In-Degree Constraints:

- **MST:** Any acyclic connected subgraph spanning all V vertices with $V - 1$ edges qualifies. Vertices can have any degree sequence as long as global connectivity is maintained.
- **MCA:** Requires a designated **root vertex** R . The final directed tree must ensure that there is a unique directed path from R to every other vertex $v \in V$. Consequently, every vertex except the root must have an **in-degree of exactly 1** ($\text{in-degree}(R) = 0$).

(c) Algorithmic Approach and Failure of Simple Greedy Methods:

- **MST:** Solved efficiently by purely greedy choices (e.g., Prim's or Kruskal's algorithms) due to the *Cut Property* and *Cycle Property*.
- **MCA:** Simple greedy strategies fail because choosing the cheapest incoming edge for each node can result in localized directed cycles that isolate components from the root. It requires more complex algorithms (such as the **Chu-Liu/Edmonds' Algorithm**) which systematically contract cycles and adjust edge weights dynamically.

(d) Influence of the Root Node:

- **MST:** Independent of any root node. Starting Prim's algorithm from any arbitrary vertex always yields the same minimum total weight.
- **MCA:** Strongly dependent on the chosen root. An arborescence might be optimal for one root but completely non-existent for another if certain nodes are unreachable.

2. Edmonds' Algorithm Specification

To rigorously solve the MCA problem, we formally follow the standard steps of Edmonds' Algorithm:

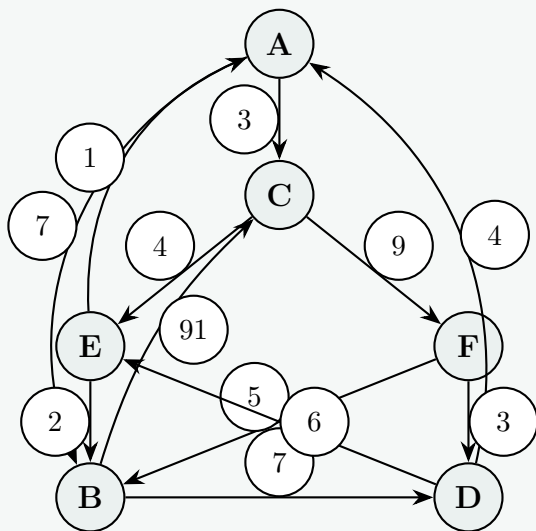
Algorithm 24 Chu-Liu/Edmonds' Algorithm

Require: A directed graph $G = (V, E)$, edge weights $w : E \rightarrow \mathbb{R}$, and root vertex $R \in V$
Ensure: A minimum-cost spanning arborescence rooted at R

- 1: Discard all incoming edges to the root R .
- 2: **for** each vertex $v \in V \setminus \{R\}$ **do**
- 3: Select the incoming edge $e = (u, v)$ with the minimum weight.
- 4: Define $\pi(v) = u$ and $\text{min_in}(v) = w(u, v)$.
- 5: **end for**
- 6: **if** the subgraph formed by edges $(\pi(v), v)$ contains no directed cycles **then**
- 7: **return** the edge set $\{(\pi(v), v)\}$ as the optimal arborescence.
- 8: **else**
- 9: Identify a directed cycle C , contract it into a super-node v_C .
- 10: Update weights of incoming edges to C : $w'(x, y) = w(x, y) - \text{min_in}(y)$ where $y \in C, x \notin C$.
- 11: Recurse on the contracted graph, then expand the cycle and break it optimally.
- 12: **end if**

3. Construction of Minimum Cost Arborescence for the Given Instance

We are given a directed graph with vertices $V = \{A, B, C, D, E, F\}$ and root $R = A$. Let's map the full original graph to visualize its topology.



Step 1: Identify Minimum Incoming Edges

We evaluate all incoming directed edges for each non-root node $v \in \{B, C, D, E, F\}$ and pick the one with the minimal weight:

Target Node (v)	All Incoming Edges (Weights)	Selected Edge	$\text{min_in}(v)$
B	$AB(7)$, $EB(2)$, $FB(5)$	EB	2
C	$AC(3)$, $BC(91)$	AC	3
D	$BD(7)$, $FD(3)$	FD	3
E	$CE(4)$, $DE(6)$	CE	4
F	$CF(9)$	CF	9

Step 2: Cycle Detection Analysis

Let us trace the tracking paths constructed exclusively by our optimal selections $\{EB, AC, FD, CE, CF\}$ starting from the root node A :

- $A \xrightarrow{3} C$
- From C , the path branches along two distinct parallel fronts:
 - $C \xrightarrow{4} E \xrightarrow{2} B$ (Terminates at leaf node B)
 - $C \xrightarrow{9} F \xrightarrow{3} D$ (Terminates at leaf node D)

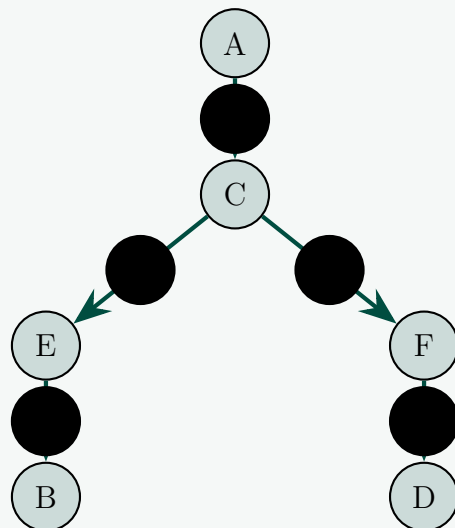
Every target vertex has an in-degree of exactly 1. There are **no directed cycles** formed by this subset of edges, meaning no cycle contraction step is triggered. The algorithm directly terminates successfully.

Step 3: Total Cost Formulation

The chosen collection of edges satisfies all conditions of a valid spanning arborescence rooted at A .

$$\text{Total Cost} = w(AC) + w(CE) + w(EB) + w(CF) + w(FD)$$

$$\text{Total Cost} = 3 + 4 + 2 + 9 + 3 = \mathbf{21}$$

**4. Asymptotic Complexity Analysis**

- **Time Complexity:** For this specific instance, the execution takes a single phase running in $\mathcal{O}(E)$ time because no cycle contractions occur. Universally, the worst-case implementation requires $\mathcal{O}(V \cdot E)$ due to up to $\mathcal{O}(V)$ nested cycle contractions. Optimized variants utilizing Fibonacci Heaps run in $\mathcal{O}(E + V \log V)$ time.
- **Space Complexity:** $\mathcal{O}(V + E)$ auxiliary space to preserve tracked predecessor values (π), adjacency matrices, and cost reduction state variables.

Maximum Flow

Q1. A central warehouse supplies relief packages to three depots D_1, D_2, D_3 . The depots distribute packages to four affected zones Z_1, Z_2, Z_3, Z_4 . Each depot has a daily sending capacity: $D_1 = 15, D_2 = 20, D_3 = 10$. Each zone has a receiving limit: $Z_1 = 10, Z_2 = 15, Z_3 = 10, Z_4 = 10$. Distribution routes: $D_1 \rightarrow \{Z_1, Z_2\}, D_2 \rightarrow \{Z_1, Z_3, Z_4\}, D_3 \rightarrow \{Z_2, Z_4\}$. The warehouse supply is unlimited. Determine the maximum number of relief packages that can be delivered per day. Model the situation as a flow network by clearly defining the source, sink, intermediate nodes, and edge capacities and apply Ford-Fulkerson's algorithm. **(20 marks)** [CSE 207 | L-2/T-1 | 2024-25]

Answer

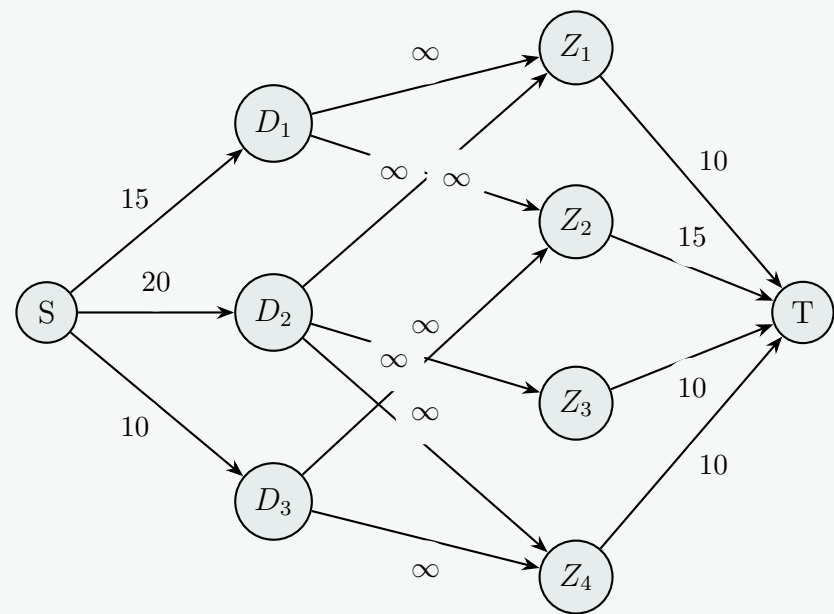
To determine the maximum number of relief packages that can be delivered per day, we model this scenario as a **Maximum Flow Problem**. We will construct a flow network $G = (V, E)$ with a designated source node S and a sink node T , applying the **Ford-Fulkerson Algorithm** to compute the maximum throughput.

1. Flow Network Modeling

The physical constraints (depot sending capacities and zone receiving limits) act on the *nodes* in reality. In standard flow networks, capacities belong on *edges*. We handle this by modeling the system as follows:

- **Source (S):** Represents the central warehouse with unlimited supply.
- **Sink (T):** A virtual sink that aggregates all successfully delivered packages from the zones.
- **Intermediate Nodes:** Depots D_1, D_2, D_3 and Zones Z_1, Z_2, Z_3, Z_4 .
- **Edges and Capacities (C):**
 - **Warehouse to Depots:** Capacity represents the daily sending limit. $C(S \rightarrow D_1) = 15, C(S \rightarrow D_2) = 20, C(S \rightarrow D_3) = 10$.
 - **Depots to Zones:** The routes themselves have no stated limits, so we assign infinite capacity (∞). $C(D_1 \rightarrow Z_1, Z_2) = \infty$
 $C(D_2 \rightarrow Z_1, Z_3, Z_4) = \infty$
 $C(D_3 \rightarrow Z_2, Z_4) = \infty$
 - **Zones to Sink:** Capacity represents the daily receiving limit of each zone. $C(Z_1 \rightarrow T) = 10, C(Z_2 \rightarrow T) = 15,$
 $C(Z_3 \rightarrow T) = 10, C(Z_4 \rightarrow T) = 10$.

2. Initial Flow Network Diagram



3. Ford-Fulkerson Execution Steps

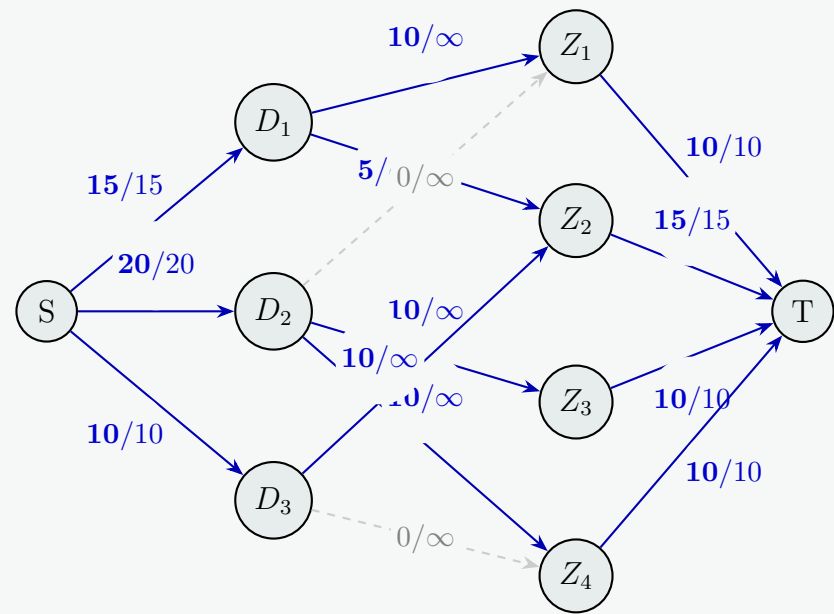
We initialize the flow $f = 0$ on all edges. We iteratively find augmenting paths from S to T in the residual graph until no such paths exist. The bottleneck capacity of a path is $\Delta = \min(C_{residual}(u, v))$ for all edges (u, v) in the path.

Iter.	Augmenting Path (P)	Bottleneck (Δ)	Calculation	Total Flow
1	$S \rightarrow D_1 \rightarrow Z_1 \rightarrow T$	10	$\min(15, \infty, 10)$	$0 + 10 = \mathbf{10}$
Updates: $C_f(S \rightarrow D_1) = 5$, $C_f(Z_1 \rightarrow T) = 0$ (Saturated). D_1 sent 10 to Z_1 .				
2	$S \rightarrow D_1 \rightarrow Z_2 \rightarrow T$	5	$\min(5, \infty, 15)$	$10 + 5 = \mathbf{15}$
Updates: $C_f(S \rightarrow D_1) = 0$ (Saturated), $C_f(Z_2 \rightarrow T) = 10$. D_1 maxed out.				
3	$S \rightarrow D_2 \rightarrow Z_3 \rightarrow T$	10	$\min(20, \infty, 10)$	$15 + 10 = \mathbf{25}$
Updates: $C_f(S \rightarrow D_2) = 10$, $C_f(Z_3 \rightarrow T) = 0$ (Saturated). D_2 sent 10 to Z_3 .				
4	$S \rightarrow D_2 \rightarrow Z_4 \rightarrow T$	10	$\min(10, \infty, 10)$	$25 + 10 = \mathbf{35}$
Updates: $C_f(S \rightarrow D_2) = 0$ (Saturated), $C_f(Z_4 \rightarrow T) = 0$ (Saturated). D_2 and Z_4 maxed out.				
5	$S \rightarrow D_3 \rightarrow Z_2 \rightarrow T$	10	$\min(10, \infty, 10)$	$35 + 10 = \mathbf{45}$
Updates: $C_f(S \rightarrow D_3) = 0$ (Saturated), $C_f(Z_2 \rightarrow T) = 0$ (Saturated). D_3 and Z_2 maxed out.				

At this point, all edges leaving the source S ($S \rightarrow D_1$, $S \rightarrow D_2$, $S \rightarrow D_3$) are saturated (residual capacities are 0). Thus, no more augmenting paths can be found.

4. Final Flow Network Diagram

The final flow representation showing Flow/Capacity across the network.



5. Conclusion and Verification

The **Maximum Flow** is **45 relief packages per day**.

We can verify this mathematically using the **Max-Flow Min-Cut Theorem**:

- Maximum theoretical supply based on depot capacities: $15 + 20 + 10 = 45$.
- Maximum theoretical demand based on zone capacities: $10 + 15 + 10 + 10 = 45$.

Since the total flow equals the minimum cut of the network (the edges originating from S or edges entering T), 45 is provably the maximum number of packages that can be delivered per day.

Q2. Derive a proof for the max-flow min-cut theorem. (15 marks)

[CSE 207 | L-2/T-1 | 2023–24, 2022–23]

Answer

The **Max-Flow Min-Cut Theorem** is a foundational result in combinatorial optimization and graph theory. It states that in a flow network, the maximum amount of flow passing from the source to the sink is exactly equal to the minimum capacity of a cut that separates the source from the sink.

1. Preliminaries and Definitions

To rigorously prove the theorem, we first establish the formal definitions:

- **Flow Network:** A directed graph $G = (V, E)$ with a source node $s \in V$ and a sink node $t \in V$. Each edge $(u, v) \in E$ has a non-negative capacity $c(u, v) \geq 0$. If $(u, v) \notin E$, we assume $c(u, v) = 0$.
- **Flow:** A function $f : V \times V \rightarrow \mathbb{R}$ satisfying:
 - (a) *Capacity Constraint:* For all $u, v \in V$, $0 \leq f(u, v) \leq c(u, v)$.
 - (b) *Flow Conservation:* For all $u \in V \setminus \{s, t\}$, incoming flow equals outgoing flow:

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$$

- **Value of Flow ($|f|$):** The net flow leaving the source:

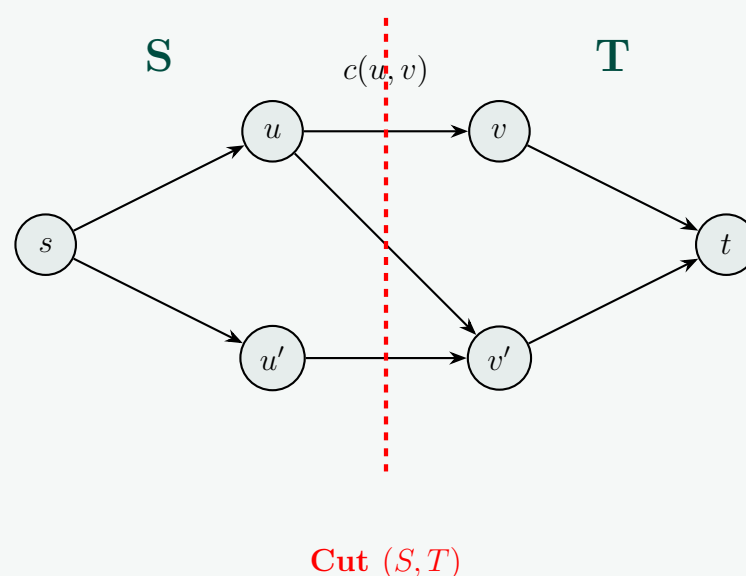
$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s)$$

- **Cut (S, T) :** A partition of V into two disjoint subsets S and T such that $s \in S$ and $t \in T$.
- **Capacity of a Cut $(c(S, T))$:** The sum of capacities of all edges directed from S to T :

$$c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v)$$

- **Net Flow Across a Cut $(f(S, T))$:** The total flow from S to T minus the total flow from T to S :

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u)$$



2. Lemma: Weak Duality

Before proving the main theorem, we establish that **the value of any flow $|f|$ is bounded above by the capacity of any cut $c(S, T)$.**

Proof: By flow conservation, the net flow across any cut (S, T) equals the total value of the flow $|f|$ leaving the source:

$$|f| = f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u)$$

Because flow cannot be negative ($f(v, u) \geq 0$), dropping the subtracted term creates an inequality:

$$|f| \leq \sum_{u \in S} \sum_{v \in T} f(u, v)$$

By the capacity constraint ($f(u, v) \leq c(u, v)$):

$$|f| \leq \sum_{u \in S} \sum_{v \in T} c(u, v) = c(S, T)$$

Thus, for *any* flow f and *any* cut (S, T) , $|f| \leq c(S, T)$. This implies the maximum flow cannot exceed the minimum cut.

3. Proof of the Max-Flow Min-Cut Theorem

To prove the theorem, we show that the following three statements are strictly equivalent:

- (a) f is a maximum flow in G .
- (b) The residual network G_f contains no augmenting paths from s to t .
- (c) $|f| = c(S, T)$ for some cut (S, T) of G .

Step 1: Prove (1) $\implies$ (2)

Suppose f is a maximum flow, but there *is* an augmenting path P in G_f from s to t . An augmenting path has bottleneck capacity $c_f(P) > 0$. We can push additional flow $c_f(P)$ along P to create a new flow f' with $|f'| = |f| + c_f(P) > |f|$. This contradicts f being a maximum flow. Thus, no augmenting paths can exist.

Step 2: Prove (2) $\implies$ (3)

Assume G_f has no augmenting paths from s to t . Let S be the set of vertices reachable from s in G_f via paths with positive residual capacity, and $T = V \setminus S$.

- $s \in S$ by definition.
- $t \notin S$ (otherwise an augmenting path exists), so $t \in T$. Thus, (S, T) is a valid cut.

For any pair $u \in S$ and $v \in T$, there is no residual capacity from u to v , so $c_f(u, v) = 0$. Since $c_f(u, v) = c(u, v) - f(u, v) = 0$, it must be that $f(u, v) = c(u, v)$. Furthermore, $f(v, u) = 0$ (otherwise there would be a backward residual edge from u to v , contradicting $v \in T$). Substituting these into the net flow equation:

$$|f| = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u) = \sum_{u \in S} \sum_{v \in T} c(u, v) - 0 = c(S, T)$$

Step 3: Prove (3) $\implies$ (1)

Assume $|f| = c(S, T)$ for some cut (S, T) . From Weak Duality, we know $|f'| \leq c(S, T)$ for *any* possible flow f' . Since $|f| = c(S, T)$, it follows that $|f'| \leq |f|$. Therefore, f must be the maximum flow.

Conclusion:

Because (1) $\implies$ (2) $\implies$ (3) $\implies$ (1), the statements are equivalent. The existence of a maximum flow f inherently guarantees a cut (S, T) where $|f| = c(S, T)$, completing the proof. ■

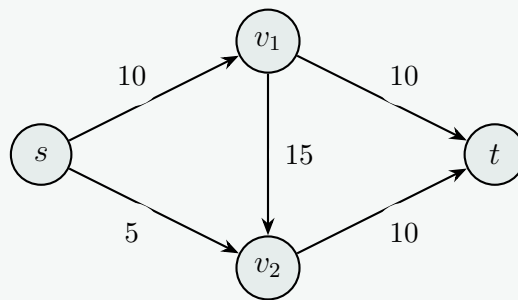
Q3. Define the Max-flow problem. (8 marks)

[CSE 207 | L-2/T-2 | 2021–22]

Answer

1. Definition of the Maximum Flow Problem

The **Maximum Flow (Max-Flow) problem** is a fundamental optimization problem in graph theory and operations research. It involves finding the maximum possible rate of flow from a designated source node to a designated sink node through a flow network, subject to the capacity limits of the network's edges.



2. Formal Mathematical Model

A flow network is defined as a directed graph $G = (V, E)$, where:

- V is a set of vertices, including a specific **source node** s and a **sink node** t .
- E is a set of directed edges.
- Each edge $(u, v) \in E$ has a non-negative **capacity** $c(u, v) \geq 0$, representing the maximum amount of flow that can pass through that edge.

A **flow** is a function $f : V \times V \rightarrow \mathbb{R}$ that assigns a numerical flow value to each edge. To be considered a valid flow, this function must satisfy two strict conditions:

1. Capacity Constraint:

The flow through any given edge cannot exceed the capacity of that edge, and it cannot be negative.

$$0 \leq f(u, v) \leq c(u, v)$$

(For all pairs of nodes $u, v \in V$. If an edge does not exist between u and v , $c(u, v) = 0$.)

2. Flow Conservation:

For every intermediate node (every node other than the source s and the sink t), the total flow entering the node must exactly equal the total flow leaving the node. No flow can be created, destroyed, or stored at intermediate nodes.

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$$

(For all $u \in V \setminus \{s, t\}$.)

3. The Objective

The **value of the flow**, denoted as $|f|$, is the net amount of flow leaving the source node s (which is mathematically guaranteed to equal the net amount of flow arriving at the sink node t).

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s)$$

The objective of the Max-Flow problem is to **maximize** $|f|$. In other words, the goal is to route as much flow as possible from s to t without violating the capacity or conservation constraints.

4. Common Algorithms

The problem is famously solved using several classic algorithms, the most notable being:

- **Ford-Fulkerson Method** (using augmenting paths)
- **Edmonds-Karp Algorithm** (a specific implementation of Ford-Fulkerson using Breadth-First Search)
- **Dinic's Algorithm** (using level graphs and blocking flows)
- **Push-Relabel Algorithms** (using preflows and active nodes)

Q4. Consider a car parking problem scenario, where you need to match each of the cars with a parking spot. The cars and the available parking spots are located at different parts of the city and can only be accessible through a road network graph, G . Let A be the set of cars and B be the set of parking slots. The cost of a car, $a \in A$ to reach to a parking slot, $b \in B$ is the shortest distance (on the graph G) between a and b . If the spot is unreachable by a car, we can assign it an infinite cost.

- Formulate the car parking assignment problem using the Max-flow/min-cut framework. **(12 marks)** [CSE 207 | L-2/T-2 | 2021–22]
- Develop an efficient solution of the above problem to find a match for every car so that the overall cost is minimized. **(15 marks)**

[CSE 207 | L-2/T-2 | 2021–22]

Answer**1. Max-Flow Formulation for the Car Parking Problem**

To formulate the assignment of cars to parking spots as a maximum flow problem, we model the situation as a **Maximum Bipartite Matching** problem using a flow network.

Let $A = \{a_1, a_2, \dots, a_n\}$ be the set of cars, and $B = \{b_1, b_2, \dots, b_m\}$ be the set of parking spots. We construct a directed flow network $G' = (V', E')$ as follows:

(a) **Vertices (V'):** Include all cars and parking spots, plus a conceptual **supersource** S and a **supersink** T .

$$V' = A \cup B \cup \{S, T\}$$

(b) **Edges (E') and Capacities (C):**

- **Source to Cars:** Add a directed edge from S to every car $a \in A$ with a capacity of 1. This ensures each car is assigned at most once.

$$\forall a \in A, C(S \rightarrow a) = 1$$

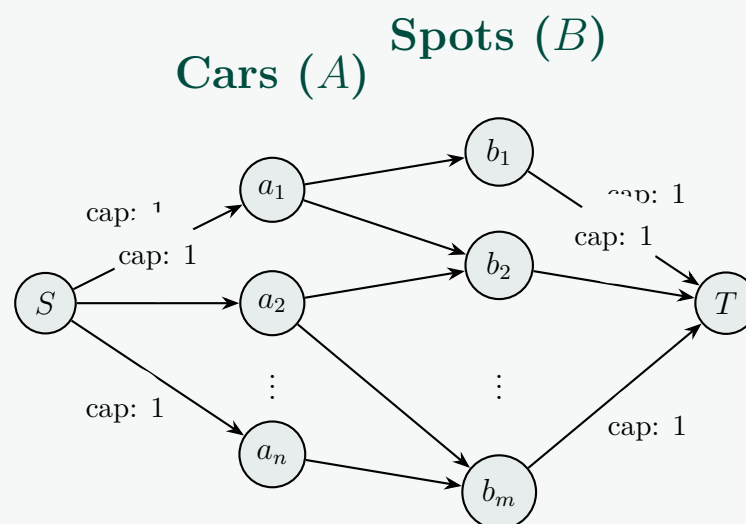
- **Cars to Spots:** Add a directed edge from a car $a \in A$ to a parking spot $b \in B$ if and only if the spot is reachable from the car (i.e., shortest distance $d(a, b) < \infty$ on the original road network G). Set the capacity of these edges to 1.

$$\forall a \in A, b \in B \text{ where } d(a, b) < \infty, C(a \rightarrow b) = 1$$

- **Spots to Sink:** Add a directed edge from every parking spot $b \in B$ to the sink T with a capacity of 1. This ensures each parking spot accepts at most one car.

$$\forall b \in B, C(b \rightarrow T) = 1$$

Conclusion: By running a Max-Flow algorithm (like Ford-Fulkerson) on G' , the maximum integer flow from S to T will represent the maximum number of cars that can be successfully parked. If an edge $a \rightarrow b$ carries a flow of 1, it means car a is assigned to parking spot b .

**2. Efficient Solution to Minimize Overall Cost**

Finding a matching that just assigns cars to spots is insufficient if we want to *minimize the overall distance driven*. This elevates the problem to the **Minimum-Cost Maximum-Flow (MCMF)** problem (or equivalent Bipartite Assignment Problem).

We can solve this efficiently in two main phases:

Phase 1: Compute Shortest Paths on Road Network G We must first calculate the exact cost $d(a, b)$ for every car $a \in A$ to every spot $b \in B$.

- Run **Dijkstra's Algorithm** from each car node $a \in A$ on the city road graph G .
- This generates an $|A| \times |B|$ cost matrix where the entry (a, b) is the shortest path distance $d(a, b)$.

Phase 2: Minimum-Cost Maximum-Flow (Successive Shortest Path Algorithm) We take the flow network G' created in part 1 and assign a **cost per unit of flow** to each edge:

- $cost(S \rightarrow a) = 0$ for all $a \in A$.
- $cost(a \rightarrow b) = d(a, b)$ for all valid pairs.
- $cost(b \rightarrow T) = 0$ for all $b \in B$.

The capacity of all edges remains 1. We then apply the **Successive Shortest Path Algorithm**:

- (a) Initialize flow $f = 0$ on all edges.
- (b) While there is a path from S to T in the residual graph G'_f :
 - (i) Find the *shortest* path P from S to T in G'_f with respect to the edge costs. (Since initial costs are positive, we can use Dijkstra. For subsequent iterations with negative residual edge costs, we use **Bellman-Ford** or Dijkstra with **Johnson's vertex potentials**).
 - (ii) The bottleneck capacity of P will always be 1 because all original capacities are 1.
 - (iii) Augment 1 unit of flow along P .
 - (iv) Update the residual graph G'_f . The residual backward edges will have a capacity of 1 and a negated cost: $cost(v \rightarrow u) = -cost(u \rightarrow v)$.
- (c) Stop when $|A|$ units of flow have been pushed (meaning all cars are parked) or no more paths exist (meaning it's impossible to match all cars).

Alternative: The Hungarian Algorithm Since the capacities are 1, this is strictly a weighted bipartite matching problem (also known as the Assignment Problem). If $|A| \leq |B|$, we can build the $|A| \times |B|$ cost matrix M where $M_{i,j} = d(a_i, b_j)$ and apply the **Hungarian Algorithm**.

- **Efficiency:** The Hungarian algorithm runs in $\mathcal{O}(V^3)$ time (where $V = |A| + |B|$), which is highly efficient and perfectly tailored to guarantee the minimum possible sum of shortest path distances.

Q5. You want to increase the maximum flow of a network as much as possible, but you are only allowed to increase the capacity of one edge. How do you find such an edge? (Give pseudocode). You may assume the existence of algorithms to compute max flow min cut. What's the running of your algorithm? **(10 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Answer

1. Core Strategy

To increase the maximum flow of a network, the edge whose capacity is increased **must be a bottleneck**—meaning it must be part of a minimum cut. If an edge is not currently being used to its full capacity (i.e., it is not saturated), increasing its capacity will have absolutely no effect on the overall maximum flow.

Therefore, the strategy is to:

- (a) Compute the initial maximum flow to identify which edges are saturated.
- (b) Iterate through only the saturated edges.
- (c) For each saturated edge, temporarily remove its capacity constraint (set to ∞), and re-evaluate the maximum flow to see how much additional flow can be pushed.
- (d) Keep track of the edge that yields the highest overall network flow and return it.

2. Pseudocode

We assume the existence of a black-box function $\text{MaxFlow}(G, s, t)$ that returns the maximum flow value of the network G from source s to sink t .

Algorithm 25 Find Optimal Edge to Increase Capacity**Require:** Flow network $G = (V, E)$, source s , sink t , and edge capacities c **Ensure:** The edge e that provides the maximum increase in total flow

```

1:  $initial\_flow \leftarrow \text{MaxFlow}(G, s, t)$ 
2:  $best\_edge \leftarrow \text{NULL}$ 
3:  $max\_increase \leftarrow 0$ 
4: for each edge  $e \in E$  do
5:                                      $\triangleright$  Optimization: Only check edges that are currently bottlenecks
6:   if flow through  $e$  equals  $c(e)$  in the initial max flow result then
7:      $original\_cap \leftarrow c(e)$ 
8:      $c(e) \leftarrow \infty$                                       $\triangleright$  Temporarily allow infinite flow through  $e$ 
9:      $new\_flow \leftarrow \text{MaxFlow}(G, s, t)$ 
10:     $increase \leftarrow new\_flow - initial\_flow$ 
11:    if  $increase > max\_increase$  then
12:       $max\_increase \leftarrow increase$ 
13:       $best\_edge \leftarrow e$ 
14:    end if
15:     $c(e) \leftarrow original\_cap$                                 $\triangleright$  Restore the edge's original capacity
16:  end if
17: end for
18: return  $best\_edge$ 

```

3. Running Time Analysis

Let $T_{MF}(V, E)$ be the time complexity of the chosen Max-Flow algorithm (e.g., Edmonds-Karp, Dinic's).

- **Initial Computation:** The algorithm computes the maximum flow once at the beginning, taking $O(T_{MF}(V, E))$ time.
- **Iteration:** The loop runs for every saturated edge. In the absolute worst-case scenario, every edge in the graph E is saturated.
- **Re-computation:** Inside the loop, the max flow algorithm is called again. This takes $|E| \times O(T_{MF}(V, E))$ time.

Total Time Complexity:

$$O(|E| \cdot T_{MF}(V, E))$$

Practical Optimization Note: While the worst-case asymptotic time remains the same, the practical running time can be drastically reduced. Instead of running $\text{MaxFlow}(G, s, t)$ from scratch inside the loop, we can run the algorithm starting from the **residual graph** G_f generated by the initial flow. Setting $c(e) = \infty$ simply adds an infinite capacity forward edge in G_f . The search then only takes time proportional to finding the *additional* augmenting paths, which is heavily pruned compared to a cold start.

Q6. The edge connectivity of an undirected graph is the minimum number k of edges that must be removed to disconnect the graph. For example, the edge connectivity of a tree is 1, and the edge connectivity of a cyclic chain of vertices is 2. Show how to determine the edge connectivity of an undirected graph $G = (V, E)$ by running a maximum-flow algorithm on at most $|V|$ flow networks, each having $O(V)$ vertices and $O(E)$ edges. **(15 marks)** [CSE 207 | L-2/T-2 | 2020–21]

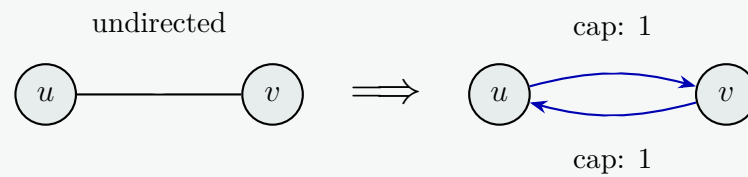
Answer**1. Conceptual Foundation**

The edge connectivity of an undirected graph G , denoted as $\lambda(G)$, is the minimum number of edges to remove to disconnect the graph. By the **Max-Flow Min-Cut Theorem**, if we set the capacity of every edge to 1, the maximum flow between any two vertices s and t is exactly equal to the minimum number of edges that must be removed to disconnect s from t (the $s - t$ min cut). To find the global minimum cut of the entire graph, we do not need to check all $\binom{|V|}{2}$ pairs of vertices. Instead, we can fix an *arbitrary* vertex s . In the global minimum cut that disconnects the graph into two components S and T , the fixed vertex s must belong to one of these sets (say, S). Therefore, at least one other vertex t must belong to the other set T . By computing the max flow from s to *every other vertex* $t \in V \setminus \{s\}$, we are guaranteed to eventually select a t that lies on the opposite side of the global minimum cut.

2. Flow Network Construction

For each max-flow computation, we construct a directed flow network $G' = (V', E')$ from the undirected graph $G = (V, E)$ as follows:

- **Vertices:** $V' = V$. The network has exactly $|V|$ vertices, satisfying the $O(V)$ constraint.
- **Edges & Capacities:** For every undirected edge $\{u, v\} \in E$, create two directed edges (u, v) and (v, u) in E' , both with a capacity of 1. The network has exactly $2|E|$ edges, satisfying the $O(E)$ constraint.



3. Algorithm

Algorithm 26 Compute Edge Connectivity $\lambda(G)$

Require: Undirected graph $G = (V, E)$

Ensure: Edge connectivity k

- 1: Construct directed flow network $G' = (V', E')$ with unit capacities as described.
- 2: Select an arbitrary starting vertex $s \in V'$.
- 3: $k \leftarrow \infty$
- 4: **for** each vertex $t \in V' \setminus \{s\}$ **do**
- 5: $current_flow \leftarrow \text{MaxFlow}(G', s, t)$
- 6: **if** $current_flow < k$ **then**
- 7: $k \leftarrow current_flow$
- 8: **end if**
- 9: **end for**
- 10: **return** k

4. Complexity and Constraint Verification

- **Number of Flow Networks:** The algorithm runs the max-flow subroutine exactly $|V| - 1$ times. This strictly satisfies the condition of running it on *at most* $|V|$ flow networks.
- **Network Size:** As established, each network G' maintains $O(V)$ vertices and $O(E)$ edges.
- **Total Running Time:** If Edmonds-Karp is used, the max flow on a unit-capacity network takes $O(|V| \cdot |E|^2)$, though specialized unit-capacity algorithms like Dinic's run in $O(|E| \min(|V|^{2/3}, \sqrt{|E|}))$. Running this $|V| - 1$ times yields an overall highly efficient polynomial-time algorithm for edge connectivity.

Q7. Ad-hoc networks are made up of cheap, low-powered wireless devices, which can communicate within a limited range. Consider an ad-hoc wireless network where any two devices can communicate if and only if the distance between them is at most D . To improve reliability, we require each device x to have k potential backup devices, all within distance D of x ; we call these k devices the backup set of x . Also, we do not want any device to be in the backup set of more than b devices, say; otherwise, a single failure might affect a large fraction of the network.

Suppose we are given the communication distance D , parameters b and k , and an array $d[1 \dots n, 1 \dots n]$ of distances, where $d[i, j]$ is the distance between device i and device j . The problem is to design and analyze an algorithm that either computes a backup set of size k for each of the n devices, such that no device appears in more than b backup sets, or correctly reports that no good collection of backup sets exists. Now answer the following questions: **(10+7+3)**

- (a) Reduce this problem to a max-flow problem. Describe a max-flow problem (a directed, capacitated graph G , a source and sink pair) such that a good collection of backup sets exist if and only if the max-flow in G has a certain size. What is this size?
- (b) Prove the correctness of your reduction. Show that a good collection of backup sets exist if and only if the max-flow in G has a certain size.
- (c) Give an algorithm to solve this max-flow problem. What is the running time of this algorithm as a function of n , k and b ?

(23 marks)

[CSE 207 | L-2/T-2 | 2019–20]

Answer

1. Reduction to a Max-Flow Problem

We can model this problem as finding a constrained assignment in a bipartite graph. We construct a directed, capacitated flow network $G = (V', E')$ as follows:

- **Vertices (V'):** Create a supersource S and a supersink T . For the n devices, create two sets of nodes:

- $U = \{u_1, u_2, \dots, u_n\}$ representing the devices in their capacity as *receivers* of backups.
- $V = \{v_1, v_2, \dots, v_n\}$ representing the devices in their capacity as *providers* of backups.

Thus, $V' = \{S, T\} \cup U \cup V$.

- **Edges (E') and Capacities (C):**

- (a) **Source to Receivers:** Add an edge from S to every node $u_i \in U$ with a capacity of k . This enforces that each device requires exactly k backups.

$$\forall i \in \{1 \dots n\}, C(S \rightarrow u_i) = k$$

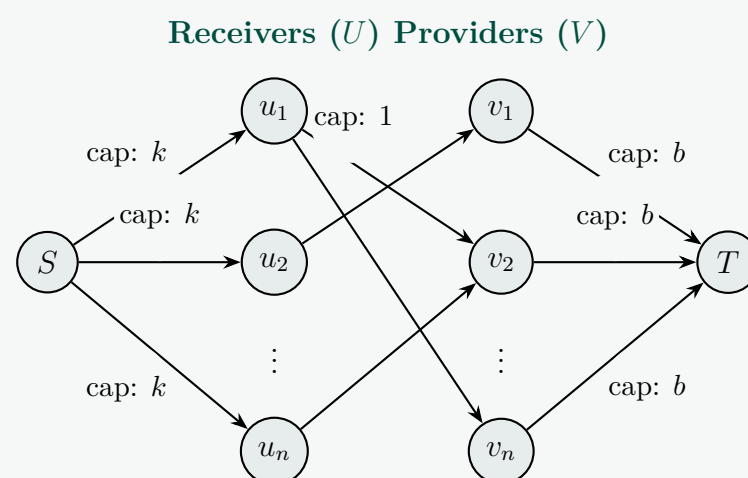
- (b) **Providers to Sink:** Add an edge from every node $v_j \in V$ to T with a capacity of b . This enforces the constraint that no device can act as a backup for more than b devices.

$$\forall j \in \{1 \dots n\}, C(v_j \rightarrow T) = b$$

- (c) **Receivers to Providers:** Add an edge from u_i to v_j if and only if device j is a valid backup for device i . This is true when $i \neq j$ and the distance $d[i, j] \leq D$. Set the capacity of these edges to 1, ensuring device j is assigned to device i 's backup set at most once.

$$\forall i, j \in \{1 \dots n\}, \text{ if } i \neq j \text{ and } d[i, j] \leq D, C(u_i \rightarrow v_j) = 1$$

Target Flow Size: A good collection of backup sets exists if and only if the max-flow in G has a size of exactly $n \cdot k$.



2. Proof of Correctness

We must prove that a valid collection of backup sets exists $\iff$ the max flow is $n \cdot k$.

Forward Direction ($\implies$):

Assume a good collection of backup sets exists. Let B_i be the backup set for device i , where $|B_i| = k$. We construct a flow as follows: for every $j \in B_i$, send 1 unit of flow along the path $S \rightarrow u_i \rightarrow v_j \rightarrow T$.

- Because $|B_i| = k$, exactly k units of flow leave S for each u_i , perfectly saturating the $S \rightarrow u_i$ edge (capacity k).
- Because no device acts as a backup for more than b devices, node v_j receives flow from at most b distinct u_i nodes. Thus, the flow on $v_j \rightarrow T$ is $\leq b$, respecting the capacity constraint.
- Flow conservation is maintained.

The total flow out of S is $\sum_{i=1}^n k = n \cdot k$. Thus, a flow of $n \cdot k$ is achieved.

Reverse Direction ($\impliedby$):

Assume the maximum flow in G is exactly $n \cdot k$. Because all edge capacities are integers, the **Integrality Theorem** guarantees an integer max flow exists (and standard algorithms will find it).

- The total capacity of edges leaving S is $\sum_{i=1}^n k = n \cdot k$. For the max flow to be $n \cdot k$, every edge $S \rightarrow u_i$ must be saturated, carrying exactly k units of flow.
- By flow conservation, exactly k units of flow must leave each u_i . Since the capacities of edges $u_i \rightarrow v_j$ are 1, this means exactly k distinct edges leaving u_i carry 1 unit of flow.
- These k edges correspond to valid backup devices ($d[i, j] \leq D, i \neq j$), defining a backup set of size exactly k for device i .
- By flow conservation at v_j , the total flow entering v_j equals the flow leaving v_j on the edge $v_j \rightarrow T$. Since $C(v_j \rightarrow T) = b$, at most b units of flow enter v_j . This guarantees v_j is selected as a backup device at most b times.

Thus, a valid collection of backup sets can be extracted from the integer flow.

3. Algorithm and Time Complexity

Algorithm:

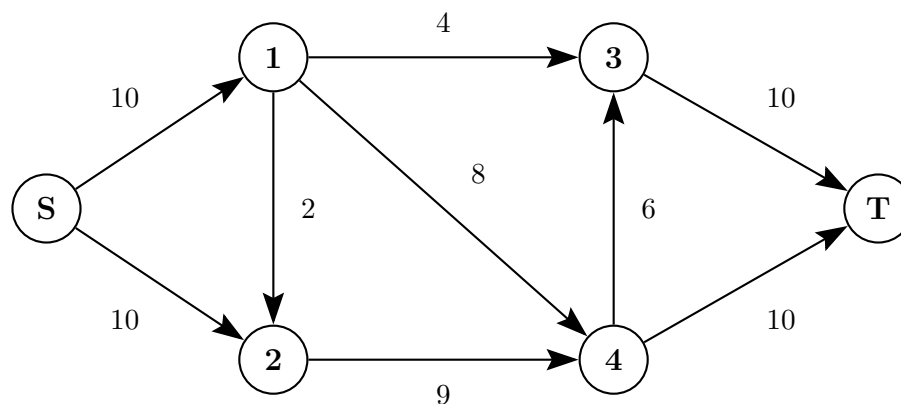
- Read the distance matrix $d[n, n]$ and construct the flow network $G = (V', E')$ as described in Part 1.
- Run the **Ford-Fulkerson algorithm** (or Edmonds-Karp) on G to find the maximum integer flow f^* .
- If $|f^*| < n \cdot k$, report "No good collection of backup sets exists."
- If $|f^*| == n \cdot k$, iterate through all edges $u_i \rightarrow v_j$. If the flow on the edge is 1, add device j to the backup set of device i . Return the sets.

Time Complexity Analysis: Let's analyze the time complexity using the Ford-Fulkerson method:

- Graph Construction:** Parsing the $n \times n$ matrix to generate edges takes $\mathcal{O}(n^2)$ time. The number of vertices $|V'| = 2n + 2 = \mathcal{O}(n)$. The number of edges $|E'| = \mathcal{O}(n^2)$ in the worst-case dense network.
- Max-Flow Execution:** Ford-Fulkerson finds an augmenting path in $\mathcal{O}(E)$ time. In the worst case, each augmenting path adds 1 unit of flow. The maximum possible flow is $f^* = n \cdot k$. Therefore, finding the max flow takes $\mathcal{O}(|E| \cdot f^*) = \mathcal{O}(n^2 \cdot nk) = \mathcal{O}(kn^3)$ time.
- Extracting Results:** Iterating through the bipartite edges to extract the sets takes $\mathcal{O}(n^2)$ time.

Overall Running Time: $\mathcal{O}(k \cdot n^3)$

Q8. For the flow network shown in the following figure, find the value of the maximum flow by drawing residual networks in each step. Finally, draw the maximum flow network. Also, identify the min-cut of the maximum flow network.



(15 marks)

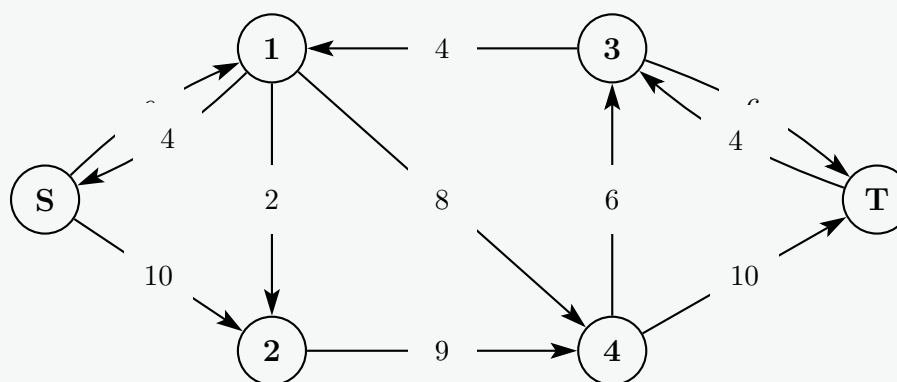
[CSE 207 | L-2/T-2 | 2018–19]

Answer

1. Step-by-Step Residual Networks

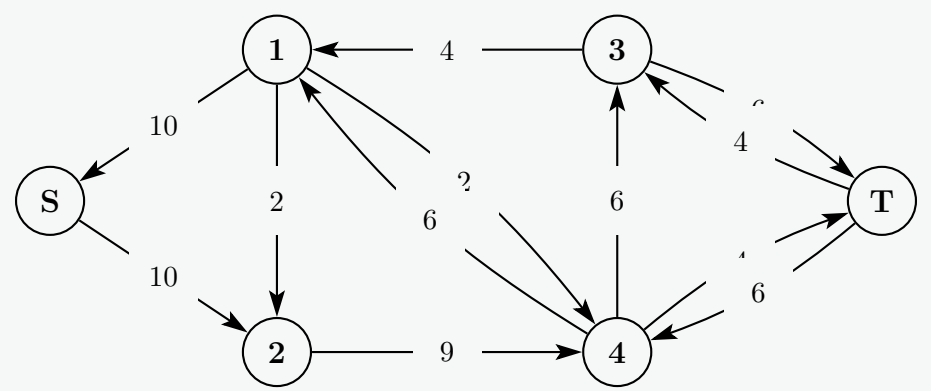
Step 1: First Augmenting Path

- Path chosen:** $S \rightarrow 1 \rightarrow 3 \rightarrow T$
- Bottleneck capacity:** $\min(10, 4, 10) = 4$
- Action:** Push 4 units of flow along this path.



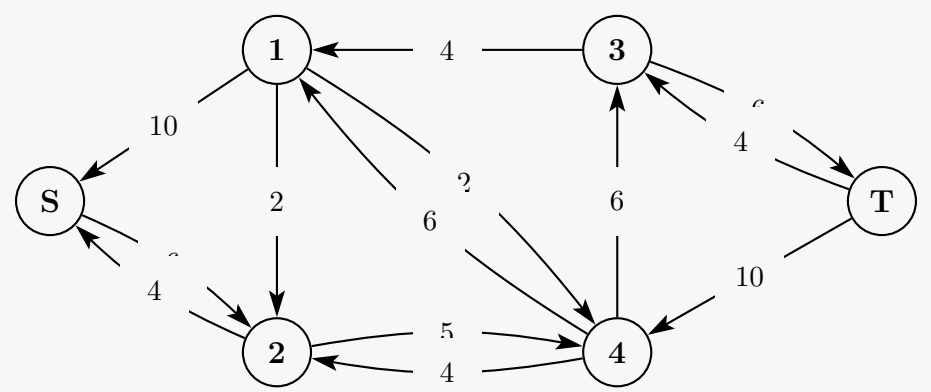
Step 2: Second Augmenting Path

- Path chosen:** $S \rightarrow 1 \rightarrow 4 \rightarrow T$
- Bottleneck capacity:** $\min(6, 8, 10) = 6$
- Action:** Push 6 units of flow.



Step 3: Third Augmenting Path

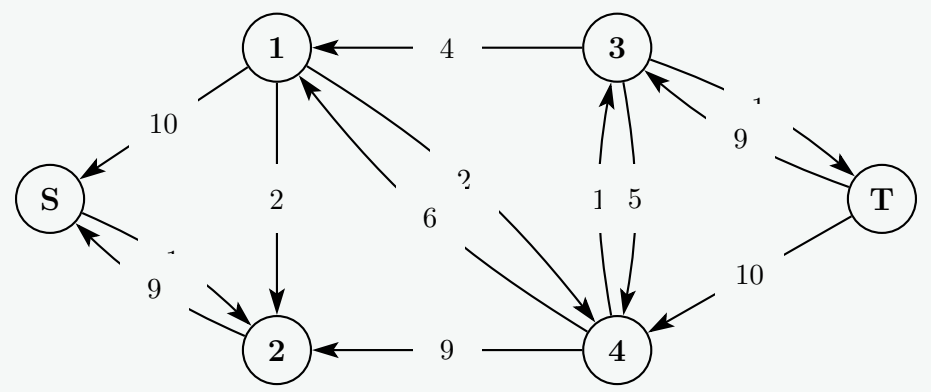
- **Path chosen:** $S \rightarrow 2 \rightarrow 4 \rightarrow T$
- **Bottleneck capacity:** $\min(10, 9, 4) = 4$
- **Action:** Push 4 units of flow.



Step 4: Fourth Augmenting Path

- **Path chosen:** $S \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow T$
- **Bottleneck capacity:** $\min(6, 5, 6, 6) = 5$
- **Action:** Push 5 units of flow.

Final Residual Network (No more paths from S to T exist):

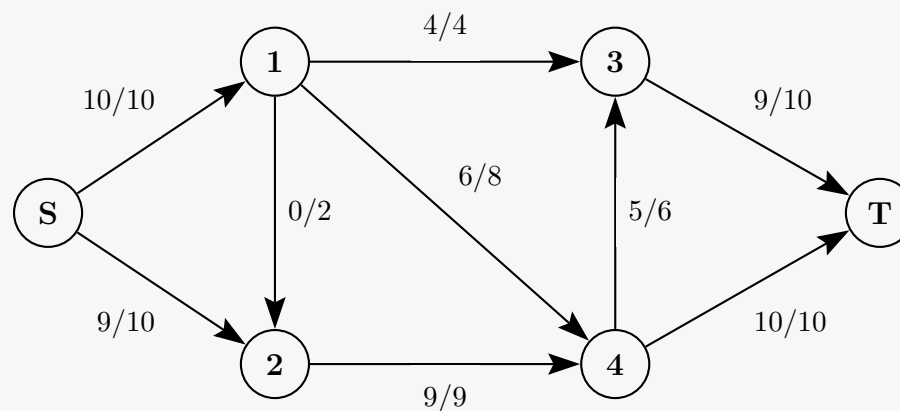


The maximum flow is the sum of the bottleneck capacities:

$$|f_{max}| = 4 + 6 + 4 + 5 = 19$$

2. Maximum Flow Network

This diagram illustrates the original network with the format *flow/capacity* on each edge.



3. Min-Cut Identification

To find the minimum cut, we look at the **Final Residual Network** from Step 4 and identify all nodes that are still reachable from S using edges with positive residual capacity.

- (a) From S , we can reach 2 (residual capacity of 1).
- (b) From 2, there are no outgoing forward edges with remaining capacity ($2 \rightarrow 4$ is fully saturated). The only outgoing residual edge is the backward edge to S .
- (c) We cannot reach any other nodes.

Therefore, the set of reachable nodes is $A = \{S, 2\}$. The set of unreachable nodes is $B = \{1, 3, 4, T\}$.

The **min-cut** is the partition: $(\{S, 2\}, \{1, 3, 4, T\})$.

We can verify this by summing the capacities of the original edges crossing from set A to set B :

- $c(S, 1) = 10$
- $c(2, 4) = 9$

Total Min-Cut Capacity = $10 + 9 = 19$. This perfectly matches our max flow value, satisfying the Max-Flow Min-Cut Theorem.

Q9. Define a flow network. What is an augmenting path in a flow network? Explain with illustrative examples. **(3+3 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Answer

1. Definition of a Flow Network

A **flow network** is a directed graph $G = (V, E)$ that satisfies the following conditions:

- **Capacity Constraint:** Each edge $(u, v) \in E$ has a non-negative capacity $c(u, v) \geq 0$. If an edge (u, v) does not exist in E , we assume $c(u, v) = 0$.
- **Source and Sink:** There are two distinguished vertices: a *source* node s (which produces flow) and a *sink* node t (which consumes flow).
- **Connectivity:** For every vertex $v \in V$, there exists a path from s to t that passes through v .

A *flow* in G is a real-valued function $f : V \times V \rightarrow \mathbb{R}$ that satisfies the capacity constraint ($0 \leq f(u, v) \leq c(u, v)$) and flow conservation ($\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$ for all $u \in V \setminus \{s, t\}$).

2. Definition of an Augmenting Path

An **augmenting path** p is a simple path from the source s to the sink t in the **residual network** G_f .

The residual network $G_f = (V, E_f)$ consists of edges with strictly positive *residual capacity* $c_f(u, v) > 0$. The residual capacity indicates how much additional flow can be pushed along an edge, defined as:

- **Forward Edges:** $c_f(u, v) = c(u, v) - f(u, v)$ (Remaining capacity on the original edge).
- **Backward Edges:** $c_f(v, u) = f(u, v)$ (Amount of flow that can be canceled or "pushed back").

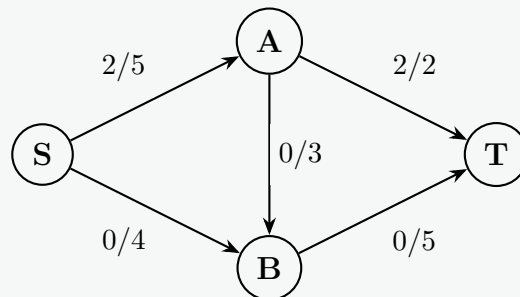
An augmenting path represents a valid route through which we can send additional flow from s to t . The maximum amount of flow that can be added along this path is determined by the bottleneck edge, denoted as $c_f(p) = \min_{(u,v) \in p} c_f(u, v)$.

3. Illustrative Example

Consider the simple flow network below with source S and sink T . The edge labels represent **flow / capacity** ($f(u, v)/c(u, v)$).

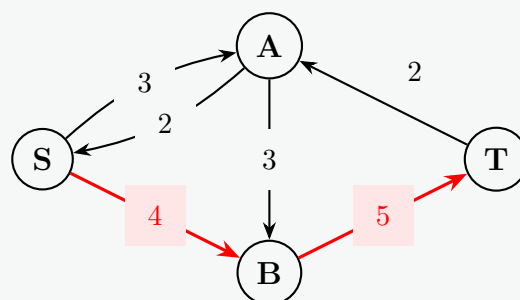
Scenario 1: Current Flow Network

Suppose we have an existing flow where 2 units are sent along the path $S \rightarrow A \rightarrow T$.



Scenario 2: Corresponding Residual Network & Augmenting Path

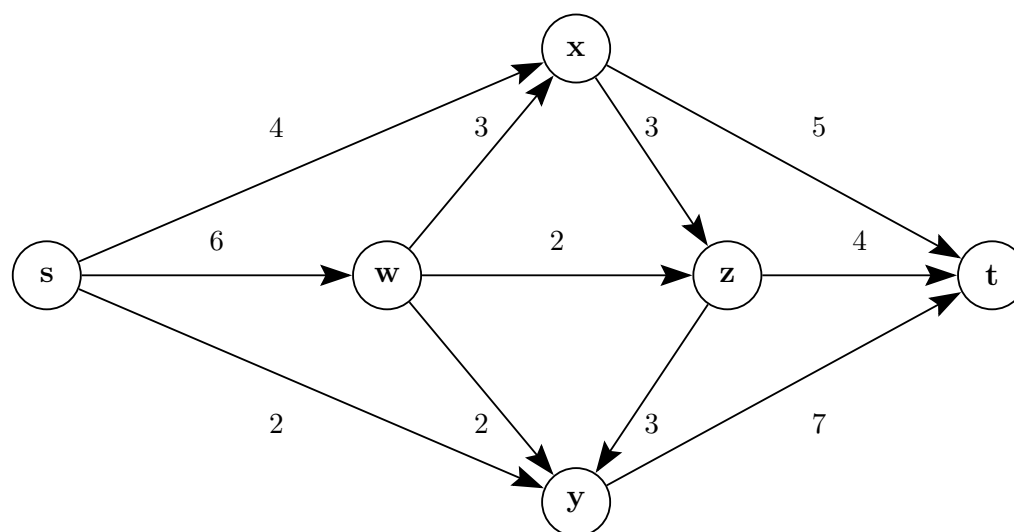
We construct the residual network based on the current flow. For edge $S \rightarrow A$, the forward residual is $5 - 2 = 3$, and the backward residual is 2. Edge $A \rightarrow T$ is fully saturated (forward 0, backward 2).



Identifying the Augmenting Path: In the residual network above, the path $S \rightarrow B \rightarrow T$ is an **augmenting path**.

- The residual capacity of this path is the bottleneck: $\min(4, 5) = 4$.
- This means we can push exactly 4 more units of flow from S to T through B , bringing the total maximum flow to $2 + 4 = 6$.

Q10. Find the maximum flow and the minimum cut using the Ford-Fulkerson algorithm. The vertex s is the source and the vertex t is the sink of the flow network. Integers represent the flow capacities of the edges.



(10 marks)

[CSE 207 | L-2/T-2 | 2017–18]

Answer

1. Finding Maximum Flow via Ford-Fulkerson Algorithm

We will find the maximum flow by repeatedly finding augmenting paths in the residual network from the source s to the sink t . We start with an initial flow of 0 on all edges.

• Iteration 1:

- **Path:** $s \rightarrow x \rightarrow t$
- **Bottleneck Capacity:** $\min(c(s, x), c(x, t)) = \min(4, 5) = 4$
- **Update:** Push 4 units of flow. Flow increases to 4.

• Iteration 2:

- **Path:** $s \rightarrow w \rightarrow y \rightarrow t$
- **Bottleneck Capacity:** $\min(c(s, w), c(w, y), c(y, t)) = \min(6, 2, 7) = 2$
- **Update:** Push 2 units of flow. Flow increases to $4 + 2 = 6$.

• Iteration 3:

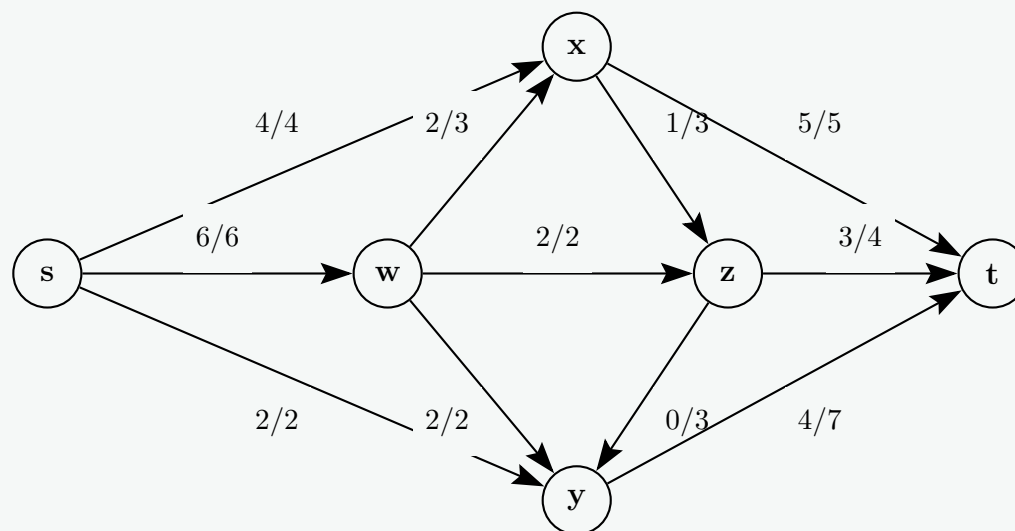
- **Path:** $s \rightarrow w \rightarrow z \rightarrow t$
- **Bottleneck Capacity:** $\min(c_f(s, w), c(w, z), c(z, t)) = \min(6 - 2, 2, 4) = \min(4, 2, 4) = 2$
- **Update:** Push 2 units of flow. Flow increases to $6 + 2 = 8$.
- **Iteration 4:**
 - **Path:** $s \rightarrow y \rightarrow t$
 - **Bottleneck Capacity:** $\min(c_f(s, y), c_f(y, t)) = \min(2, 7 - 2) = \min(2, 5) = 2$
 - **Update:** Push 2 units of flow. Flow increases to $8 + 2 = 10$.
- **Iteration 5:**
 - **Path:** $s \rightarrow w \rightarrow x \rightarrow t$
 - **Bottleneck Capacity:** $\min(c_f(s, w), c(w, x), c_f(x, t)) = \min(6 - 4, 3, 5 - 4) = \min(2, 3, 1) = 1$
 - **Update:** Push 1 unit of flow. Flow increases to $10 + 1 = 11$.
- **Iteration 6:**
 - **Path:** $s \rightarrow w \rightarrow x \rightarrow z \rightarrow t$
 - **Bottleneck Capacity:** $\min(c_f(s, w), c_f(w, x), c(x, z), c_f(z, t)) = \min(6 - 5, 3 - 1, 3, 4 - 2) = \min(1, 2, 3, 2) = 1$
 - **Update:** Push 1 unit of flow. Flow increases to $11 + 1 = 12$.

At this point, all edges leaving the source s ($s \rightarrow x, s \rightarrow w, s \rightarrow y$) are fully saturated. There are no more augmenting paths from s to t in the residual graph.

Maximum Flow = 12

2. Final Maximum Flow Network

The following network illustrates the final state with edge labels in the format **flow/capacity**.



3. Minimum Cut

To find the minimum cut, we look at the final residual network and determine the set of vertices reachable from the source s via edges with positive residual capacity.

Since the flow on all outgoing edges from s ($s \rightarrow x, s \rightarrow w, s \rightarrow y$) perfectly equals their respective capacities, their forward residual capacities are 0. Consequently, no other nodes are reachable from s in the final residual graph.

- **Set S (Reachable from s):** $\{s\}$
- **Set T (Unreachable from s):** $\{w, x, y, z, t\}$

The **Minimum Cut** is the partition $(\{s\}, \{w, x, y, z, t\})$.

We can verify the capacity of this cut by summing the capacities of the original edges crossing from set S to set T :

$$\text{Capacity} = c(s, x) + c(s, w) + c(s, y) = 4 + 6 + 2 = \mathbf{12}$$

This matches the maximum flow value, consistent with the Max-Flow Min-Cut theorem.

Q11. Describe the minimum path length augmentation method of Edmonds and Karp and explain how it improves the time complexity

of the Ford-Fulkerson algorithm. (10 marks)

[CSE 207 | L-2/T-2 | 2017–18]

Answer

1. The Minimum Path Length Augmentation Method (Edmonds-Karp)

The **Edmonds-Karp algorithm** is a specific implementation of the Ford-Fulkerson method for computing the maximum flow in a flow network. While the generic Ford-Fulkerson method allows choosing *any* augmenting path in the residual network G_f , Edmonds and Karp introduced a strict rule for path selection:

- **Breadth-First Search (BFS):** The algorithm must always choose the shortest augmenting path from the source s to the sink t in the residual graph.
- **Unit Edge Weights:** "Shortest" is defined in terms of the *number of edges*, not the capacities. All edges in the residual graph G_f with $c_f(u, v) > 0$ are treated as having a weight of 1.

By using BFS, the algorithm finds the path with the minimum number of edges, which is the "minimum path length augmentation" method.

2. The Problem with Standard Ford-Fulkerson

In the standard Ford-Fulkerson algorithm (often implemented using Depth-First Search), the choice of path is arbitrary. This leads to several issues:

- **Pseudo-Polynomial Time:** The time complexity is $O(E|f_{max}|)$, where E is the number of edges and $|f_{max}|$ is the value of the maximum flow.
- **Capacity Dependency:** If the edge capacities are very large, and the algorithm unfortunately picks paths that only augment a tiny amount of flow (e.g., 1 unit) back-and-forth across a bottleneck edge, the algorithm can take a very long time to converge.
- **Non-termination:** If the capacities are irrational numbers, the algorithm is not guaranteed to terminate.

3. How Edmonds-Karp Improves Time Complexity

The Edmonds-Karp method completely eliminates the dependence on edge capacities, reducing the time complexity to a strongly polynomial bound of $O(VE^2)$, where V is the number of vertices and E is the number of edges. It achieves this through the following properties:

- **Monotonicity of Shortest Paths:** It can be proven that during the execution of the Edmonds-Karp algorithm, the shortest path distance (in number of edges) from the source s to any vertex v in the residual network *never decreases*.
- **Critical Edge Bound:** In every augmenting step, at least one edge on the chosen path becomes a **critical edge** (the bottleneck edge whose residual capacity becomes zero). Once an edge (u, v) becomes critical, it disappears from the residual network. It can only reappear if flow is pushed backward along (v, u) , which only happens if the distance to u has strictly increased.
- **Maximum Number of Augmentations:** Because the distance from s to any node is at most $|V| - 1$, any specific edge (u, v) can become critical at most $|V|/2$ times. Since there are $O(E)$ edges that can appear in the residual graph, the total number of critical edges (and thus the absolute maximum number of augmenting paths) is bounded by $O(VE)$.
- **Cost per Augmentation:** Each BFS traversal to find the shortest path takes $O(E)$ time.

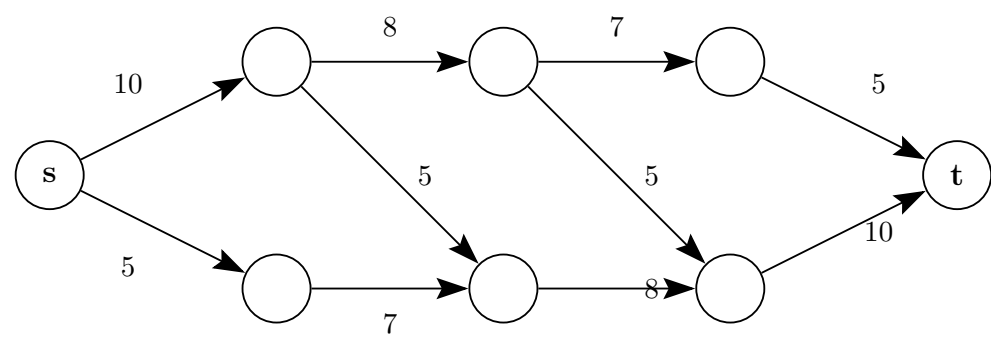
Final Complexity Calculation:

Multiplying the maximum number of augmentations by the time it takes to find each augmenting path gives the worst-case time complexity:

$$O(VE) \text{ augmentations} \times O(E) \text{ per BFS} = O(VE^2)$$

This is a massive improvement because the execution time is strictly bound by the network's topology (vertices and edges) and is entirely independent of the numerical values of the flow capacities.

Q12. Find the value of the maximum flow by drawing residual networks at each step. Draw the maximum flow network and identify the min-cut.



(15 marks)

[CSE 207 | L-2/T-2 | 2016–17]

Answer

1. Step-by-Step Residual Networks

We will find the maximum flow by repeatedly finding augmenting paths in the residual network from the source s to the sink t . For clarity, the empty nodes in the given problem have been labeled a, b, c, d, e, f .

Step 1: First Augmenting Path

- **Path chosen:** $s \rightarrow a \rightarrow b \rightarrow c \rightarrow t$
- **Bottleneck capacity:** $\min(10, 8, 7, 5) = 5$
- **Action:** Push 5 units of flow along this path.

Residual Network after Step 1:

Step 2: Second Augmenting Path

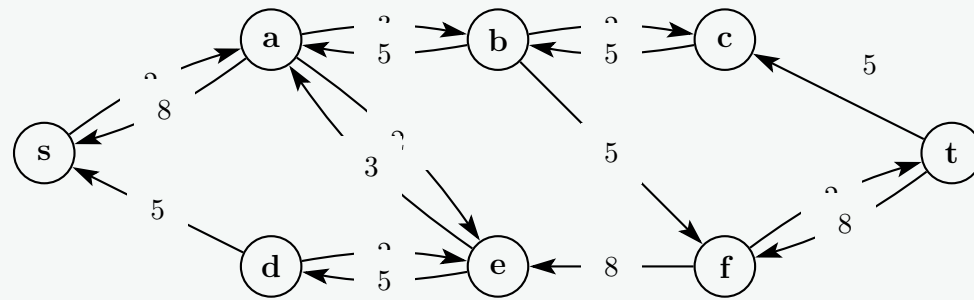
- **Path chosen:** $s \rightarrow d \rightarrow e \rightarrow f \rightarrow t$
- **Bottleneck capacity:** $\min(5, 7, 8, 10) = 5$
- **Action:** Push 5 units of flow along this path.

Residual Network after Step 2:

Step 3: Third Augmenting Path

- **Path chosen:** $s \rightarrow a \rightarrow e \rightarrow f \rightarrow t$
- **Bottleneck capacity:** $\min(5, 5, 3, 5) = 3$
- **Action:** Push 3 units of flow along this path.

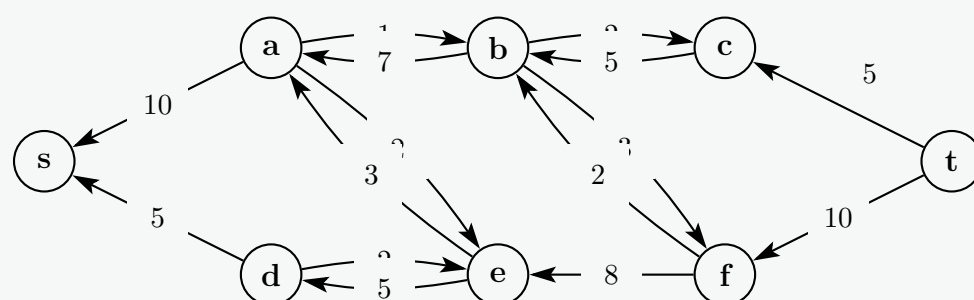
Residual Network after Step 3:



Step 4: Fourth Augmenting Path

- **Path chosen:** $s \rightarrow a \rightarrow b \rightarrow f \rightarrow t$
- **Bottleneck capacity:** $\min(2, 3, 5, 2) = 2$
- **Action:** Push 2 units of flow along this path.

Final Residual Network after Step 4 (No more paths from s to t):

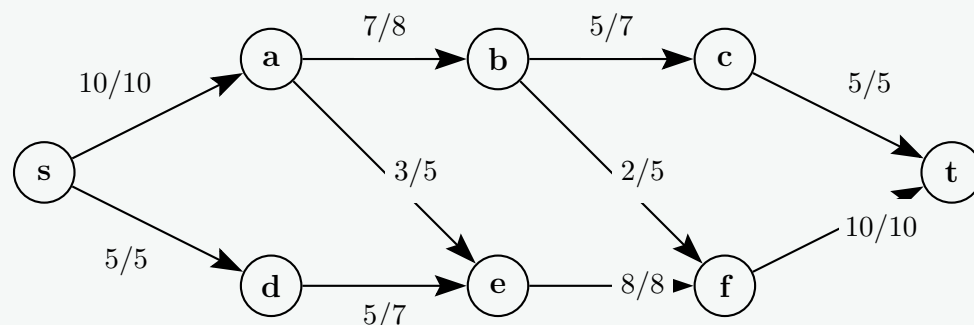


The maximum flow is the sum of the bottleneck capacities:

$$|f_{max}| = 5 + 5 + 3 + 2 = 15$$

2. Maximum Flow Network

This diagram illustrates the original network with the final format **flow/capacity** on each edge.



3. Minimum Cut Identification

To find the minimum cut, we look at the **Final Residual Network** from Step 4 and identify all nodes that are still reachable from the source s using edges with positive residual capacity.

- Outgoing from s : Both $s \rightarrow a$ and $s \rightarrow d$ are fully saturated (10/10 and 5/5). Their forward residual capacity is 0.
- Therefore, no other nodes in the network can be reached from s .

This gives us the partition sets:

- **Reachable Set A :** $\{s\}$
- **Unreachable Set B :** $\{a, b, c, d, e, f, t\}$

The **min-cut** is the partition: $(\{s\}, \{a, b, c, d, e, f, t\})$.

We verify this by summing the capacities of the original edges crossing from set A to set B :

- $c(s, a) = 10$
- $c(s, d) = 5$

Total Min-Cut Capacity = $10 + 5 = 15$. This perfectly matches our max flow value.

Q13. What is a flow network? Explain how network flow algorithms allow us to find the maximum bipartite matching. **(10 marks)** [CSE 207 | L-2/T-2 | 2016–17]

Answer

1. Definition of a Flow Network

A **flow network** is a directed graph $G = (V, E)$ consisting of a set of vertices V and a set of directed edges E , subject to the following properties:

- **Capacity Constraint:** Every edge $(u, v) \in E$ has a non-negative real-valued capacity $c(u, v) \geq 0$. If an edge does not exist, its capacity is considered 0.
- **Source and Sink:** There are two special vertices: a *source* s with no incoming edges (produces flow) and a *sink* t with no outgoing edges (consumes flow).
- **Flow Function:** A flow is a function $f : V \times V \rightarrow \mathbb{R}$ that satisfies two conditions for all nodes:
 - (a) *Capacity Constraint:* $0 \leq f(u, v) \leq c(u, v)$.
 - (b) *Flow Conservation:* For every node u except s and t , the total incoming flow equals the total outgoing flow: $\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$.

2. Maximum Bipartite Matching using Network Flow

A **bipartite graph** is a graph $G = (V, E)$ where the vertex set V can be partitioned into two disjoint sets, L and R , such that every edge connects a vertex in L to a vertex in R . A **matching** is a subset of edges where no two edges share a common vertex. The **maximum bipartite matching** problem asks to find the matching with the maximum possible number of edges.

We can elegantly solve this using network flow algorithms (like Ford-Fulkerson or Edmonds-Karp) by transforming the bipartite graph into a flow network.

Step-by-Step Reduction:

- (a) **Add Source and Sink:** Create a new super-source node s and a super-sink node t .
- (b) **Connect Source to Left Set (L):** Add a directed edge from s to every node $u \in L$. Assign a capacity of 1 to each of these edges. (This ensures each node in L can be matched at most once).
- (c) **Direct Original Edges:** Direct all the original undirected edges in E from L to R . Assign each of these edges a capacity of 1 (or ∞ , as the flow is bottlenecked by the s and t edges anyway).
- (d) **Connect Right Set (R) to Sink:** Add a directed edge from every node $v \in R$ to t . Assign a capacity of 1 to each of these edges. (This ensures each node in R can be matched at most once).

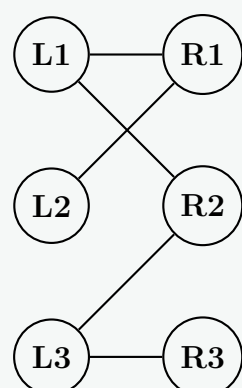
Why it works (The Integrality Theorem):

Because all edge capacities in our newly constructed flow network are integers (specifically 1), algorithms like Ford-Fulkerson are guaranteed to find an *integral* maximum flow.

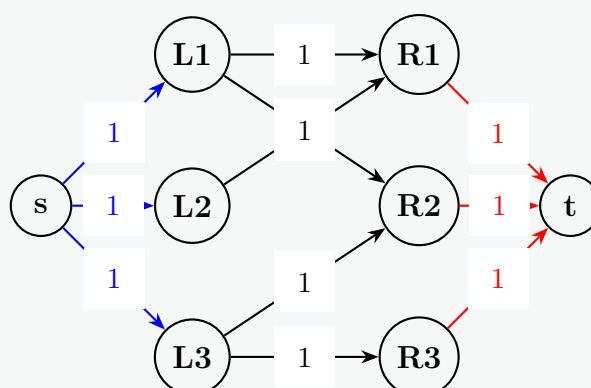
- If 1 unit of flow passes through an edge (u, v) where $u \in L$ and $v \in R$, it means u and v are **matched**.
- The value of the maximum flow $|f_{max}|$ from s to t will exactly equal the number of edges in the maximum bipartite matching.

3. Illustrative Diagram of the Transformation

Original Bipartite Graph



Corresponding Flow Network



Q14. Write the Edmonds-Karp algorithm that solves the maximum-flow problem. Analyze its time complexity for integral capacities. (10 marks) [CSE 207 | L-2/T-2 | 2015–16]

Answer

1. The Edmonds-Karp Algorithm

The Edmonds-Karp algorithm is a specific implementation of the Ford-Fulkerson method. Its defining characteristic is that it uses **Breadth-First Search (BFS)** to find the shortest augmenting path (in terms of the number of edges) from the source s to the sink t in the residual network.

Pseudocode:

(a) **Initialize Flow:** For each edge $(u, v) \in E$:

- $f(u, v) = 0$
- $f(v, u) = 0$

(b) **Find Augmenting Path:** **while** there exists a path p from s to t in the residual network G_f , found using **BFS** (where all edge weights are considered 1):

- (i) // Find the bottleneck capacity of path p
- (ii) $c_f(p) = \min\{c_f(u, v) \mid (u, v) \text{ is in } p\}$
- (iii) // Augment flow along the path
- (iv) **for** each edge (u, v) in p :
 - **if** (u, v) is a forward edge in the original graph:
 - $f(u, v) = f(u, v) + c_f(p)$
 - **else** (it is a backward edge representing flow $f(v, u)$):
 - $f(v, u) = f(v, u) - c_f(p)$

(c) **Return** the maximum flow f .

2. Time Complexity Analysis

For a graph with V vertices and E edges, we analyze the complexity by looking at the cost of finding a path and the maximum number of times we can augment the flow.

Cost of finding a path:

- The algorithm uses Breadth-First Search (BFS) to find the shortest path in the unweighted residual network G_f .
- A standard BFS traversal takes $\mathbf{O(V + E)}$ time. Since $E \geq V - 1$ for any connected flow network, this simplifies to $\mathbf{O(E)}$.

Maximum number of augmentations:

- A crucial property of using BFS is that the shortest path distance $\delta(s, v)$ from the source to any vertex v in the residual graph increases monotonically over the execution of the algorithm.
- During each augmentation, at least one edge (u, v) on the path becomes a **critical edge**. An edge is critical if it is the bottleneck ($c_f(u, v) = c_f(p)$).
- Once an edge (u, v) becomes critical, its residual capacity drops to 0, and it disappears from the residual network.
- For the edge (u, v) to reappear in the residual network, flow must be pushed backward from v to u . This can only happen if the distance from s to u has increased by at least 2.
- The maximum distance from s to any node without cycles is $V - 1$. Therefore, a single edge can become critical at most $(V - 1)/2 = \mathbf{O(V)}$ times.
- Since there are $O(E)$ possible edges that can be present in the residual graph, the absolute maximum number of critical edges (and thus the total number of augmenting paths) is bounded by $\mathbf{O(VE)}$.

Total Time Complexity Calculation:

$$\text{Total Time} = (\text{Max Augmentations}) \times (\text{Time per BFS})$$

$$\text{Total Time} = O(VE) \times O(E) = \mathbf{O(VE^2)}$$

Note on Integral Capacities: While the general Ford-Fulkerson algorithm has a complexity of $O(E|f_{max}|)$ (which is pseudo-polynomial and heavily relies on capacities being integers to ensure termination), the Edmonds-Karp complexity of $O(VE^2)$ is **strongly polynomial**. It relies entirely on the network's structural topology (vertices and edges) rather than the magnitude of the integral capacities. Integral capacities simply guarantee that the flow pushed in each iteration, and the final maximum flow, are exact integers, preventing floating-point precision issues.

Q15. Explain how network flow algorithms allow us to find the maximum bipartite matching. **(10 marks)**
2015–16]

[CSE 207 | L-2/T-2 |

Answer

1. The Maximum Bipartite Matching Problem

A **bipartite graph** is an undirected graph $G = (V, E)$ in which the vertex set V can be partitioned into two disjoint sets, L (Left) and R (Right), such that every edge connects a vertex in L to a vertex in R .

A **matching** is a subset of edges $M \subseteq E$ such that no two edges in M share a common vertex. The **maximum bipartite matching** problem asks us to find a matching of maximum size (i.e., containing the largest possible number of edges).

2. Reduction to a Network Flow Problem

We can solve the maximum bipartite matching problem by transforming the bipartite graph G into a flow network $G' = (V', E')$. This transformation is done through the following systematic steps:

- (a) **Add Source and Sink:** Introduce two new vertices: a super-source node s and a super-sink node t . The new vertex set is $V' = V \cup \{s, t\}$.
- (b) **Connect Source to Left Set (L):** For every vertex $u \in L$, add a directed edge from s to u . Assign a capacity of $c(s, u) = 1$ to each of these edges.
 - *Purpose:* This ensures that each node in the left set can supply at most 1 unit of flow, meaning it can be matched to at most one node in R .
- (c) **Direct the Original Edges:** Replace each original undirected edge $(u, v) \in E$ (where $u \in L$ and $v \in R$) with a directed edge from u to v . Assign a capacity of $c(u, v) = 1$ (or ∞) to each of these edges.
- (d) **Connect Right Set (R) to Sink:** For every vertex $v \in R$, add a directed edge from v to t . Assign a capacity of $c(v, t) = 1$ to each of these edges.
 - *Purpose:* This ensures that each node in the right set can route at most 1 unit of flow to the sink, meaning it can be matched to at most one node in L .

3. Proof of Correctness (Why this works)

Once the flow network G' is constructed, we apply a standard network flow algorithm (like Ford-Fulkerson or Edmonds-Karp) to find the maximum flow from s to t .

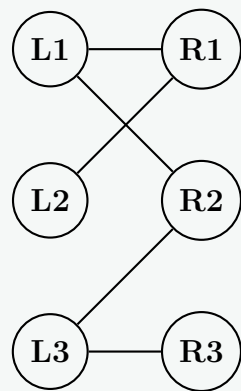
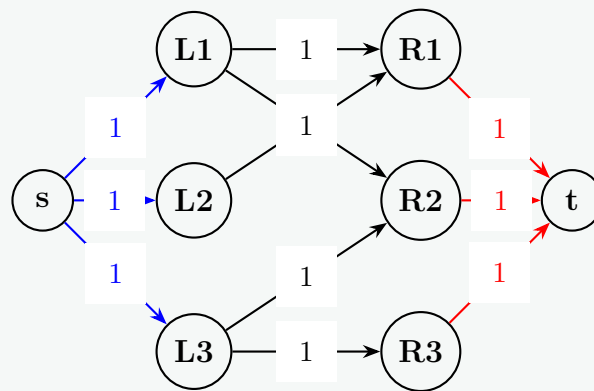
The Integrality Theorem: Because all edge capacities in G' are integers (specifically 1), the Ford-Fulkerson algorithm is guaranteed to produce an **integral maximum flow**. This means the flow through any edge is either 0 or 1.

One-to-One Correspondence:

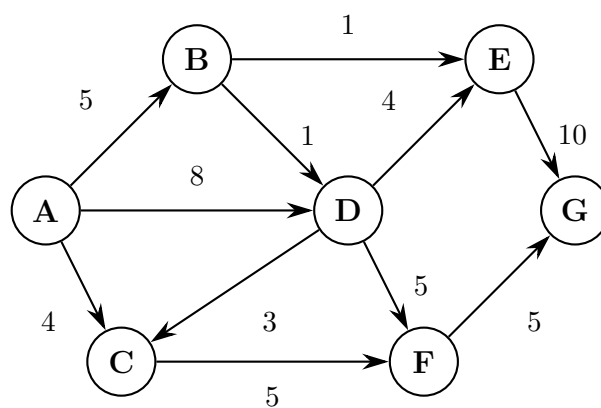
- If 1 unit of flow passes through an edge (u, v) where $u \in L$ and $v \in R$, we include the original edge (u, v) in our matching M .
- The capacity constraints $c(s, u) = 1$ and $c(v, t) = 1$ strictly enforce the rule that node u and node v can have at most 1 unit of flow passing through them. Thus, no two selected edges can share a vertex.
- Therefore, the value of the maximum flow $|f_{max}|$ in G' is exactly equal to the maximum number of edges in the bipartite matching M of G .

4. Illustrative Diagram

The following diagram illustrates the transformation from a standard bipartite graph to its corresponding flow network.

1. Bipartite Graph G 2. Flow Network G' 

Q16. A traffic engineer needs to decide which roads to widen to allow maximum traffic from A to G . Which roads are to be widened?



(12 marks)

[CSE 207 | L-2/T-2 | 2014–15]

Answer

1. Identifying the Maximum Flow

To determine which roads act as bottlenecks, we first need to find the maximum flow from the source A to the sink G . We can do this by finding augmenting paths.

Augmenting Paths:

- **Path 1:** $A \rightarrow C \rightarrow F \rightarrow G$
 - Bottleneck: $\min(4, 5, 5) = 4$. Push 4 units.
- **Path 2:** $A \rightarrow B \rightarrow E \rightarrow G$
 - Bottleneck: $\min(5, 1, 10) = 1$. Push 1 unit.
- **Path 3:** $A \rightarrow D \rightarrow E \rightarrow G$
 - Bottleneck: $\min(8, 4, 9) = 4$. Push 4 units.
- **Path 4:** $A \rightarrow D \rightarrow F \rightarrow G$
 - Bottleneck: $\min(4, 5, 1) = 1$. Push 1 unit.

At this point, the total flow leaving A is $4 + 1 + 4 + 1 = 10$. No more augmenting paths can be found from A to G . The **maximum traffic flow** is 10.

2. Final Flow Network & The Minimum Cut

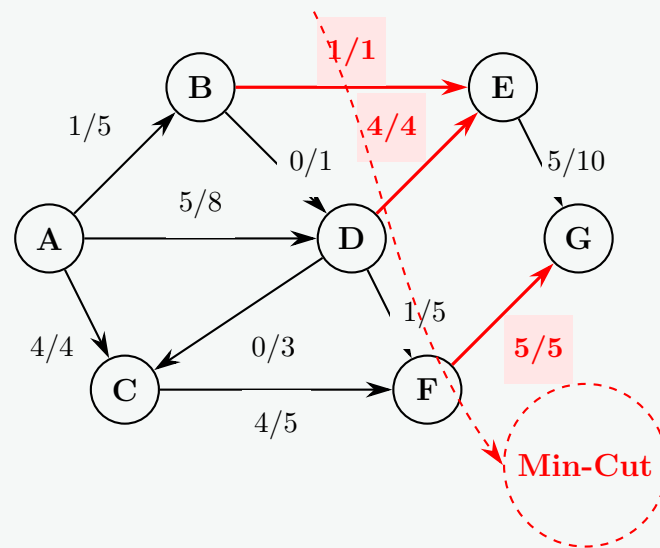
The network below shows the final state with labels representing **flow/capacity**. To increase the maximum flow, the traffic engineer must widen the roads that form the **minimum cut** (the absolute bottlenecks of the network).

We identify the minimum cut by finding all nodes reachable from A in the final residual network (i.e., using paths where capacity is not fully saturated).

- **Reachable from A (Set S):** $\{A, B, C, D, F\}$
 - $A \rightarrow B$ has residual $(5-1) = 4$.
 - $A \rightarrow D$ has residual $(8-5) = 3$.

- $D \rightarrow C$ has residual ($3-0 = 3$).
- $D \rightarrow F$ has residual ($5-1 = 4$).

- **Unreachable from A (Set T):** $\{E, G\}$



3. Decision: Roads to be Widened

According to the Max-Flow Min-Cut theorem, the maximum flow of the network is restricted entirely by the edges crossing the minimum cut from S to T . These edges are operating at 100% capacity.

To allow maximum traffic from A to G , the traffic engineer needs to widen the following roads:

- Road $B \rightarrow E$
- Road $D \rightarrow E$
- Road $F \rightarrow G$

Why? Widening any of these specific roads will instantly allow more traffic because there is still unutilized capacity leading up to them and trailing away from them. For example, widening $F \rightarrow G$ will allow more traffic to flow via $A \rightarrow D \rightarrow F$, which currently has unused residual capacity.

Q17. Define and explain the following terms in a network flow model: super sink node, augmented path, residual capacity, super source node. (8 marks) [CSE 207 | L-2/T-2 | 2014–15]

Answer

Here are the definitions and structural explanations of the core components in a network flow model:

1. Super Source Node

A **super source node** (often denoted as s) is a single, artificial vertex introduced into a network flow graph when there are multiple production sites or starting points.

- **Function:** It simplifies a multiple-source network into a single-source problem. Directed edges are drawn from this super source to every real source node in the system.
- **Capacity:** The capacity assigned to these new edges is usually set equal to the maximum supply capacity of each respective original source node.

2. Super Sink Node

A **super sink node** (often denoted as t) is a single, artificial vertex introduced into a network flow graph when there are multiple consumer sites or destination points.

- **Function:** It aggregates all the separate destinations into one unified endpoint. Directed edges are drawn from every real sink node in the system to this super sink.
- **Capacity:** The capacity assigned to these new edges is typically set equal to the maximum demand capacity of each respective original sink node.

3. Augmented Path (Augmenting Path)

An **augmenting path** is a simple, directed path from the source s to the sink t in the *residual network* along which additional flow can be pushed.

- **Structure:** It can consist of both forward edges (where current flow can be increased) and backward edges (where current flow can be decreased or redirected).
- **Role in Optimization:** According to the Ford-Fulkerson method, a flow network reaches its maximum capacity if and only if no more augmenting paths can be found.

4. Residual Capacity

The **residual capacity** (c_f) is the remaining, usable capacity on an edge given its current flow state. It defines how much more flow can be sent through a connection.

- **Forward Edges:** For an edge (u, v) with capacity $c(u, v)$ and current flow $f(u, v)$, the forward residual capacity is:

$$c_f(u, v) = c(u, v) - f(u, v)$$

- **Backward Edges:** To allow algorithms to undo bad flow decisions, the backward residual capacity equals the current flow value itself, allowing flow to be pushed back:

$$c_f(v, u) = f(u, v)$$

Q18. What is the pitfall of Ford-Fulkerson's max-flow min-cut method? How can it be improved? (8 marks) [CSE 207 | L-2/T-2 | 2014–15]

Answer

1. The Pitfalls of the Standard Ford-Fulkerson Method

The core flaw of the basic Ford-Fulkerson algorithm is that it leaves the method for choosing the augmenting path completely arbitrary. If an implementation chooses poorly (for instance, by using a naïve Depth-First Search), it leads to severe performance degradation.

- **Pseudo-Polynomial Time Complexity:** The worst-case time complexity is $O(E|f_{max}|)$, where E is the number of edges and $|f_{max}|$ is the total value of the maximum flow. The algorithm's runtime is dependent on the actual numerical capacities of the edges, not just the size of the graph.
- **The "Bad Path" Pathology (Zig-Zagging):** Consider a network with two parallel paths of huge capacity (e.g., 1,000,000) connected by a single cross edge of capacity 1. If the algorithm unluckily chooses an augmenting path that zig-zags across this capacity-1 edge, it will only push 1 unit of flow per iteration. It would take 2,000,000 iterations to find the max flow, whereas a smart algorithm would find it in just 2 iterations.
- **Non-termination with Irrational Numbers:** If the edge capacities are irrational numbers, the arbitrary path selection means the algorithm is mathematically not guaranteed to terminate. Worse, the flow might converge to a limit that is strictly less than the actual maximum flow.

2. How the Method Can Be Improved

To eliminate these pitfalls, the algorithm must transition from an arbitrary path selection to a structured, optimized path selection.

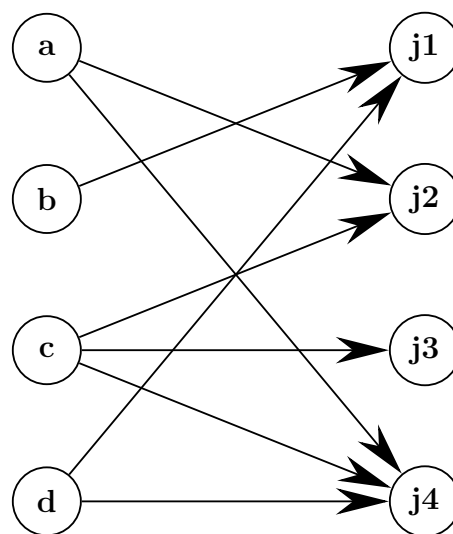
Improvement 1: The Edmonds-Karp Algorithm

- **The Fix:** Instead of picking any path, use **Breadth-First Search (BFS)** to strictly find the shortest augmenting path in the residual network (shortest in terms of the *number of edges*, treating all edge weights as 1).
- **The Result:** This simple rule prevents the algorithm from infinitely zig-zagging across small bottlenecks. It bounds the maximum number of augmentations to $O(VE)$, resulting in a **strongly polynomial** time complexity of $O(VE^2)$. The runtime becomes entirely independent of the edge capacities, ensuring fast termination even with massive or irrational numbers.

Improvement 2: Advanced Graph Algorithms For even larger networks, the augmenting path concept itself can be optimized or replaced:

- **Dinic's Algorithm:** Instead of finding one path at a time, it uses BFS to build a "Level Graph" (grouping nodes by distance from the source) and then pushes a "blocking flow" along multiple paths simultaneously. This drastically improves the time complexity to $O(V^2E)$.
- **Push-Relabel (Preflow-Push) Algorithm:** This abandons augmenting paths from source to sink entirely. Instead, it pushes local "preflow" across edges node-by-node based on height gradients (labels). This localizes the optimization, achieving complexities like $O(V^3)$ or $O(V^2\sqrt{E})$.

Q19. Find the maximum job assignment using a maximum flow algorithm.



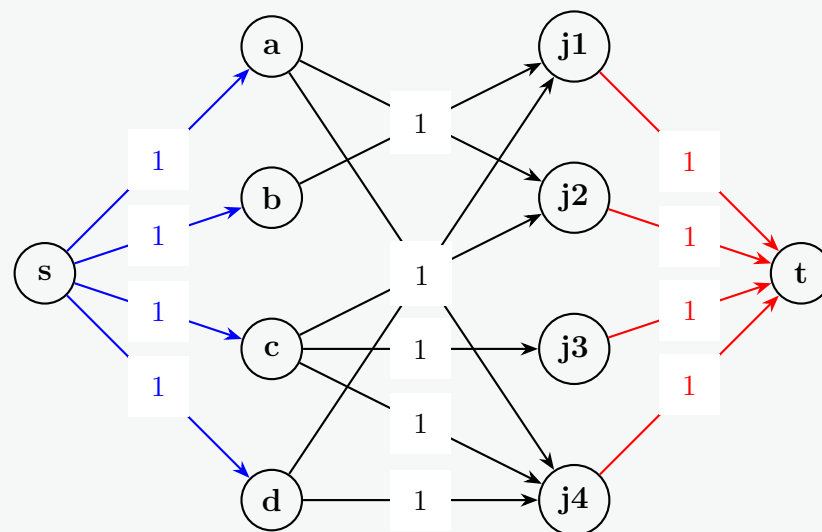
(10 marks)

[CSE 207 | L-2/T-2 | 2014–15]

Answer**1. Network Transformation**

To apply a maximum flow algorithm (like Ford-Fulkerson or Edmonds-Karp), we construct a flow network $G' = (V', E')$ from the original bipartite graph:

- Add a Super Source and Super Sink:** Create a super source node s and a super sink node t .
- Connect the Source:** Add directed edges from s to every applicant node $\{a, b, c, d\}$, each with a capacity of 1.
- Direct the Original Edges:** Direct all the bipartite edges from the applicants to the jobs $\{j1, j2, j3, j4\}$, each with a capacity of 1.
- Connect to the Sink:** Add directed edges from every job node $\{j1, j2, j3, j4\}$ to t , each with a capacity of 1.

**2. Execution of the Max Flow Algorithm**

We will repeatedly find augmenting paths from s to t in the residual network. Because all capacities are 1, every augmenting path we find will push exactly 1 unit of flow.

- Path 1:** $s \rightarrow a \rightarrow j2 \rightarrow t$. Push $\min(1, 1, 1) = 1$ unit.
Residual updates: Edges (s, a) , $(a, j2)$, and $(j2, t)$ are saturated.
- Path 2:** $s \rightarrow b \rightarrow j1 \rightarrow t$. Push $\min(1, 1, 1) = 1$ unit.
Residual updates: Edges (s, b) , $(b, j1)$, and $(j1, t)$ are saturated.
- Path 3:** $s \rightarrow c \rightarrow j3 \rightarrow t$. Push $\min(1, 1, 1) = 1$ unit.
Residual updates: Edges (s, c) , $(c, j3)$, and $(j3, t)$ are saturated.
- Path 4:** $s \rightarrow d \rightarrow j4 \rightarrow t$. Push $\min(1, 1, 1) = 1$ unit.
Residual updates: Edges (s, d) , $(d, j4)$, and $(j4, t)$ are saturated.

Termination: At this point, all edges leaving the super source s are fully saturated. There are no more valid augmenting paths from s to t in the residual network. The maximum flow is 4.

3. Maximum Job Assignment

By the Integrality Theorem, the total maximum flow corresponds to the size of the maximum bipartite matching. The specific edges between the applicants and jobs that carry 1 unit of flow represent our final assignments.

Extracting the active edges gives us the following perfect matching:

- **Applicant** a is assigned to **Job** j_2
- **Applicant** b is assigned to **Job** j_1
- **Applicant** c is assigned to **Job** j_3
- **Applicant** d is assigned to **Job** j_4

This assignment perfectly matches all 4 workers to 4 unique jobs, achieving the maximum possible assignment.

Q20. What is a flow network? What is the maximum-flow problem? Explain how one can convert a multiple-source, multiple-sink maximum-flow problem into a problem with a single source and a single sink. **(4+2+4=10 marks)** [CSE 207 | L-2/T-2 | 2011–12]

Answer

1. Definition of a Flow Network

A **flow network** is a directed graph $G = (V, E)$ in which each edge $(u, v) \in E$ has a non-negative capacity $c(u, v) \geq 0$. If $(u, v) \notin E$, we assume $c(u, v) = 0$. The network contains two distinguished vertices: a **source** s (which produces flow) and a **sink** t (which consumes flow).

A **flow** in G is a real-valued function $f : V \times V \rightarrow \mathbb{R}$ that satisfies two fundamental properties:

- Capacity Constraint:** For all $u, v \in V$, we require $0 \leq f(u, v) \leq c(u, v)$. The flow along an edge cannot exceed its capacity.
- Flow Conservation:** For all vertices $u \in V - \{s, t\}$, the total flow entering the node must equal the total flow leaving the node:

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$$

2. The Maximum-Flow Problem

The **maximum-flow problem** asks us to find a valid flow f from the source s to the sink t such that the total **value** of the flow is maximized.

The value of a flow, denoted as $|f|$, is defined as the total net flow leaving the source (which is equivalent to the total net flow arriving at the sink):

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s)$$

The objective is to maximize $|f|$ subject to the capacity and conservation constraints.

3. Converting a Multiple-Source, Multiple-Sink Problem

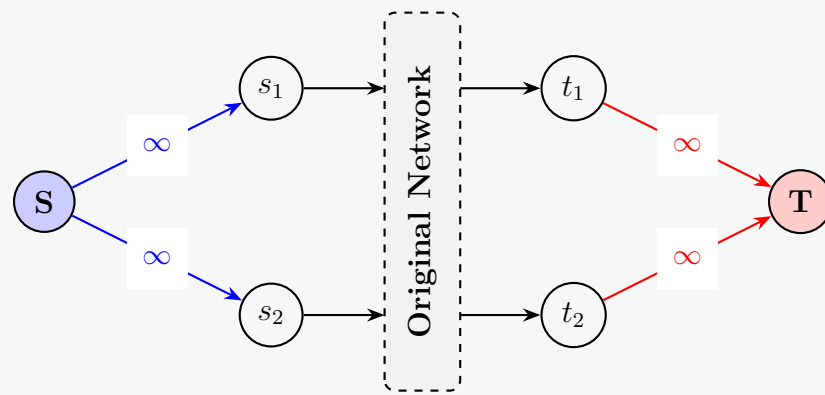
Many real-world networks involve multiple starting points (e.g., several factories) and multiple destinations (e.g., several warehouses). Let the set of sources be $\{s_1, s_2, \dots, s_k\}$ and the set of sinks be $\{t_1, t_2, \dots, t_m\}$.

Standard max-flow algorithms require a single source and a single sink. We can elegantly convert the multiple-source, multiple-sink problem into an equivalent single-source, single-sink problem using the following steps:

- Add a Super-Source:** Create a new, artificial vertex S (the super-source).
- Connect to Original Sources:** Add directed edges from S to each of the original sources s_i . Assign a capacity of ∞ (or the maximum possible production capacity of that specific source) to each of these new edges: $c(S, s_i) = \infty$.
- Add a Super-Sink:** Create another new, artificial vertex T (the super-sink).
- Connect from Original Sinks:** Add directed edges from each of the original sinks t_j to T . Assign a capacity of ∞ (or the maximum possible consumption capacity of that specific sink) to each of these new edges: $c(t_j, T) = \infty$.

By solving the standard maximum-flow problem from S to T on this newly constructed network, we perfectly model and solve the original multi-source/multi-sink problem. The infinite capacities ensure that the new artificial edges never act as bottlenecks.

Illustrative Diagram:



Q21. Write the Ford-Fulkerson algorithm for solving the maximum-flow problem, and analyze the time complexity of the algorithm. Explain how the Edmonds-Karp algorithm improves the bound of the Ford-Fulkerson algorithm. **(10+5=15 marks)** [CSE 207 | L-2/T-2 | 2011–12]

Answer

1. The Ford-Fulkerson Algorithm

The Ford-Fulkerson method relies on the concept of a **residual network**. It iteratively finds an augmenting path from the source s to the sink t and pushes flow along this path until no more such paths exist.

Pseudocode:

- (a) **Initialize:** For every edge $(u, v) \in E$, set the initial flow to zero: $f(u, v) = 0$ and $f(v, u) = 0$.
- (b) **While** there exists a path p from s to t in the residual network G_f (found using any search algorithm like DFS or BFS):
 - (i) // Find the bottleneck capacity of the path p
 - (ii) $c_f(p) = \min\{c_f(u, v) \mid (u, v) \text{ is an edge in } p\}$
 - (iii) // Augment flow along the path
 - (iv) **For** each edge (u, v) in path p :
 - **If** (u, v) is a forward edge: $f(u, v) = f(u, v) + c_f(p)$
 - **Else** (it is a backward edge): $f(v, u) = f(v, u) - c_f(p)$
- (c) **Return** the flow f .

2. Time Complexity Analysis of Ford-Fulkerson

Let V be the number of vertices, E be the number of edges, and $|f^*|$ be the value of the maximum flow.

- **Finding a Path:** A single graph traversal (like DFS or BFS) to find an augmenting path in the residual network takes $O(E)$ time.
- **Number of Augmentations:** Assuming all edge capacities are integers, every augmenting path pushes an integer amount of flow of at least 1. Therefore, the absolute maximum number of iterations the ‘while’ loop can execute is exactly $|f^*|$.
- **Total Worst-Case Time Complexity:** Multiplying the time per iteration by the maximum number of iterations yields $O(E|f^*|)$.

The Pitfall: This complexity is **pseudo-polynomial** because it depends on the numerical value of the capacities, not just the size of the graph (V and E).

If the algorithm unluckily chooses an alternating “zig-zag” path across a small bottleneck edge (e.g., an edge with capacity 1 connecting two paths of capacity 10^6), it could take 2×10^6 iterations to finish, adding only 1 unit of flow each time. Furthermore, if capacities are irrational numbers, the arbitrary path selection means the algorithm is not mathematically guaranteed to terminate.

3. The Edmonds-Karp Improvement

The Edmonds-Karp algorithm is an implementation of the Ford-Fulkerson method that completely eliminates the pseudo-polynomial worst-case scenario.

How it improves the bound:

- **The Strategy:** Instead of picking *any* augmenting path, Edmonds-Karp strictly uses **Breadth-First Search (BFS)** to find the **shortest augmenting path** (where “shortest” means the fewest number of edges, treating all edge weights as 1).
- **Monotonicity:** By always picking the shortest path, it guarantees that the shortest-path distance from the source to any node in the residual network *increases monotonically* over the algorithm’s execution.

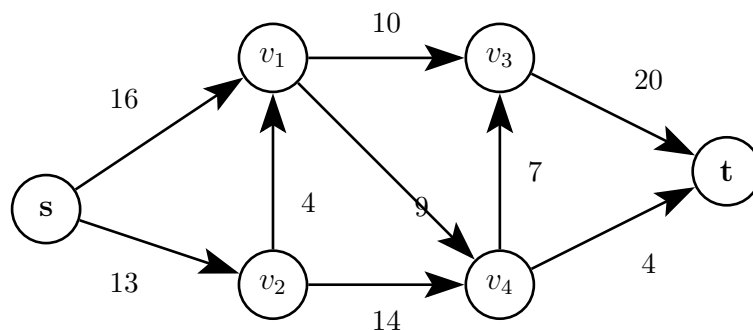
- **Bounding Augmentations:** Because of this monotonic property, any specific edge can become a "critical edge" (the bottleneck of a path) at most $(V - 1)/2$ times. Since there are $O(E)$ edges in the residual network, the total number of flow augmentations is strictly bounded to $O(VE)$.

New Time Complexity: Since the algorithm takes $O(E)$ time for the BFS and does at most $O(VE)$ augmentations, the total time complexity becomes:

$$O(E) \times O(VE) = O(VE^2)$$

This makes the algorithm **strongly polynomial**. Its performance is bounded purely by the structural size of the graph (nodes and edges) and is entirely independent of the maximum flow value $|f^*|$ or the magnitude of the edge capacities.

Q22. State the max-flow min-cut theorem in its simplest form. What is the main argument to prove the theorem? Find a min-cut and its capacity. Find the flow in that cut and the maximum flow in the network.



(17 marks)

[CSE 207 | L-2/T-2 | 2009–10]

Answer

1. The Max-Flow Min-Cut Theorem

Statement: In any flow network, the maximum value of an $s - t$ flow is exactly equal to the minimum capacity over all $s - t$ cuts in the network.

Main Argument to Prove the Theorem:

The proof hinges on the concept of the **residual network** G_f .

- Assume we have found a flow f using the Ford-Fulkerson method, and the algorithm has terminated because there are no more augmenting paths from the source s to the sink t in the residual network G_f .
- Let S be the set of all nodes reachable from s in G_f , and let T be the set of all remaining nodes (which must include t). This defines an $s - t$ cut (S, T) .
- Because there are no paths from S to T in the residual network:
 - Every original edge going from S to T must be operating at **full capacity** (otherwise, it would have residual capacity and allow a path to T).
 - Every original edge going from T to S must have **zero flow** (otherwise, it would provide a backward residual edge allowing a path from S).
- Therefore, the net flow across this specific cut (S, T) is exactly equal to its capacity. Since no flow can ever exceed the capacity of *any* cut, this cut must be the minimum possible cut, and our current flow must be the maximum possible flow.

2. Solving the Given Network

To find the maximum flow and the minimum cut, we push flow through the network until no augmenting paths remain.

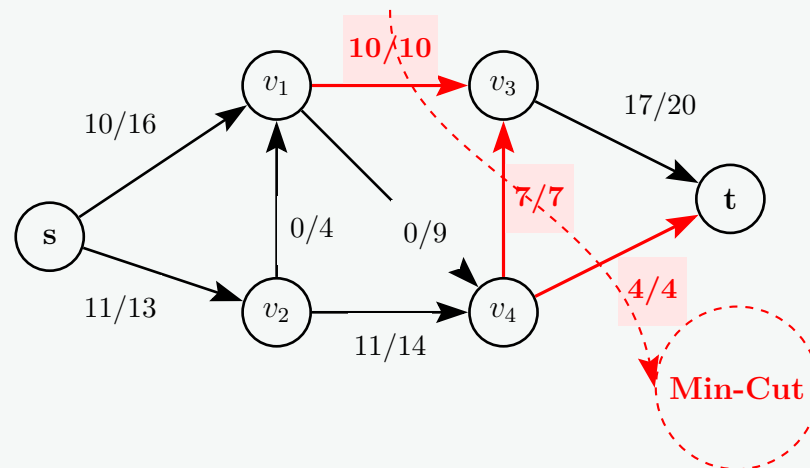
Flow Assignments (Flow / Capacity):

- $s \rightarrow v_1$: 10 / 16
- $s \rightarrow v_2$: 11 / 13
- $v_2 \rightarrow v_1$: 0 / 4
- $v_1 \rightarrow v_3$: 10 / 10 (Saturated)
- $v_1 \rightarrow v_4$: 0 / 9
- $v_2 \rightarrow v_4$: 11 / 14
- $v_4 \rightarrow v_3$: 7 / 7 (Saturated)
- $v_4 \rightarrow t$: 4 / 4 (Saturated)
- $v_3 \rightarrow t$: 17 / 20

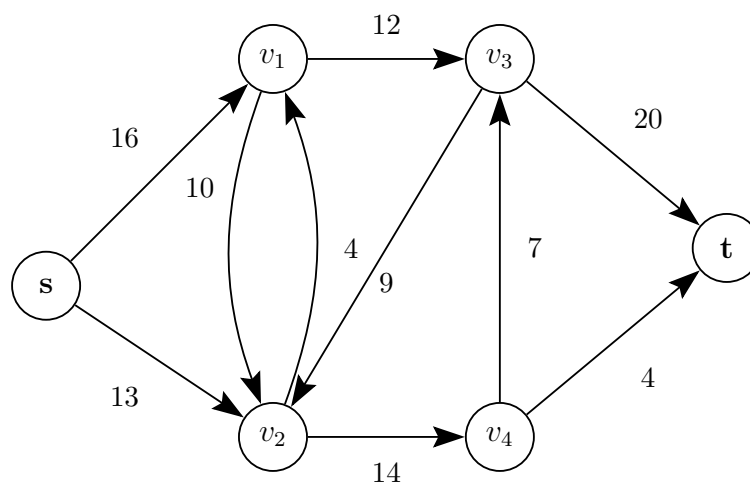
Finding the Min-Cut: By examining the saturated edges that bottleneck the network, we find the nodes reachable from s in the residual graph are $S = \{s, v_1, v_2, v_4\}$. The unreachable nodes are $T = \{v_3, t\}$.

- **Min-Cut Partition:** $S = \{s, v_1, v_2, v_4\}$, $T = \{v_3, t\}$
- **Edges in the Min-Cut (from S to T):** (v_1, v_3) , (v_4, v_3) , and (v_4, t) .
- **Capacity of Min-Cut:** $c(v_1, v_3) + c(v_4, v_3) + c(v_4, t) = 10 + 7 + 4 = \mathbf{21}$.
- **Flow in that Cut:** The flow passing through these specific edges is $10 + 7 + 4 = \mathbf{21}$.
- **Maximum Flow:** By the theorem (and as calculated by the flow leaving s : $10 + 11 = 21$), the maximum flow in the network is $\mathbf{21}$.

Visual Representation of the Final State & Min-Cut:



Q23. Find the maximum flow and the minimum cut. Find also the maximum flow in the minimum cut.



(12 marks)

[CSE 207 | L-2/T-2 | 2008–09]

Answer

1. Finding the Maximum Flow

To find the maximum flow from the source s to the sink t , we repeatedly push flow through augmenting paths until no more paths exist in the residual network.

Augmenting Paths:

- **Path 1:** $s \rightarrow v_1 \rightarrow v_3 \rightarrow t$
 - Bottleneck capacity: $\min(16, 12, 20) = 12$.
 - Push 12 units.
- **Path 2:** $s \rightarrow v_2 \rightarrow v_4 \rightarrow t$
 - Bottleneck capacity: $\min(13, 14, 4) = 4$.
 - Push 4 units.
- **Path 3:** $s \rightarrow v_2 \rightarrow v_4 \rightarrow v_3 \rightarrow t$
 - Residual capacities: $c_f(s, v_2) = 13 - 4 = 9$; $c_f(v_2, v_4) = 14 - 4 = 10$; $c_f(v_4, v_3) = 7$; $c_f(v_3, t) = 20 - 12 = 8$.
 - Bottleneck capacity: $\min(9, 10, 7, 8) = 7$.
 - Push 7 units.

At this point, the total flow leaving s is $12 + 4 + 7 = \mathbf{23}$. No more augmenting paths can be found from s to t . The maximum flow is $\mathbf{23}$.

2. Finding the Minimum Cut

The minimum cut separates the network into two sets of vertices, S and T . We find S by identifying all nodes reachable from s in the final residual network.

- **Reachable from s (Set S):** $\{s, v_1, v_2, v_4\}$
 - $s \rightarrow v_1$ has residual capacity $16 - 12 = 4$.
 - $s \rightarrow v_2$ has residual capacity $13 - 11 = 2$.
 - $v_2 \rightarrow v_4$ has residual capacity $14 - 11 = 3$.
- **Unreachable from s (Set T):** $\{v_3, t\}$
 - $v_1 \rightarrow v_3$, $v_4 \rightarrow v_3$, and $v_4 \rightarrow t$ are all fully saturated (0 residual capacity).

The edges forming the minimum cut are the forward edges crossing from S to T :

$$\text{Cut Edges} = \{(v_1, v_3), (v_4, v_3), (v_4, t)\}$$

Capacity of the Minimum Cut: $c(v_1, v_3) + c(v_4, v_3) + c(v_4, t) = 12 + 7 + 4 = \mathbf{23}$.

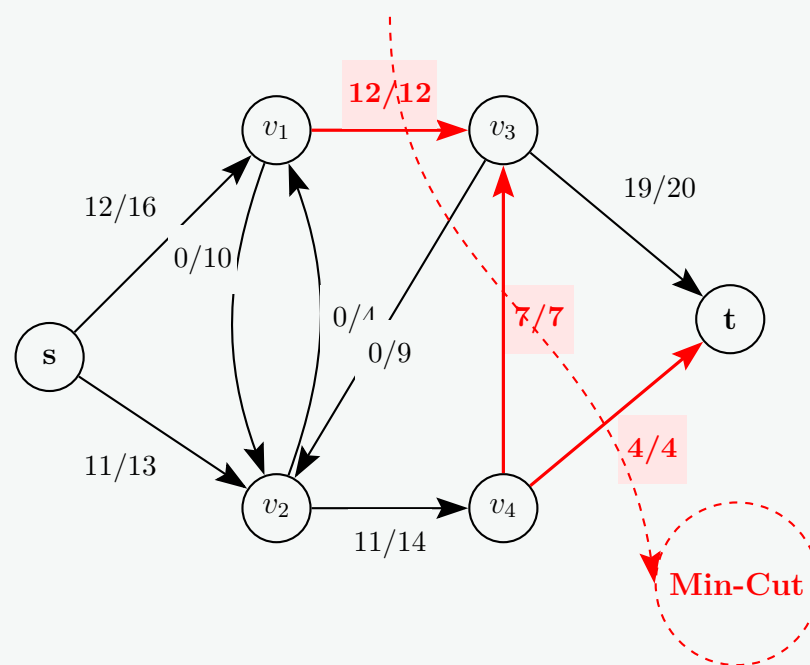
3. Maximum Flow in the Minimum Cut

According to the Max-Flow Min-Cut theorem, the net flow across any cut in a maximized network is strictly equal to the value of the maximum flow.

For the minimum cut specifically, all forward edges are perfectly saturated, and all backward edges carry zero flow. Therefore, the total net flow passing through the minimum cut is exactly equal to its capacity:

$$\text{Flow in Min-Cut} = f(v_1, v_3) + f(v_4, v_3) + f(v_4, t) = 12 + 7 + 4 = \mathbf{23}.$$

Final Flow Network Diagram:



Q24. Discuss how a maximum flow algorithm can be used to determine the maximum number of disjoint paths in directed and undirected graphs. (15 marks) [CSE 207 | L-2/T-2 | 2007–08]

Answer

Finding the maximum number of disjoint paths between a source vertex s and a sink vertex t is a foundational problem in graph theory, heavily tied to **Menger's Theorem**. By setting edge and node constraints to unit capacities, we can use a maximum flow algorithm (like Ford-Fulkerson or Edmonds-Karp) to solve these problems efficiently.

Disjoint paths can be categorized into two types: **Edge-Disjoint** (paths that share no edges) and **Vertex-Disjoint** (paths that share no vertices except s and t).

1. Directed Graphs

Case A: Edge-Disjoint Paths in Directed Graphs

To find the maximum number of directed paths from s to t that do not share any edges:

- (a) **Transformation:** Take the original directed graph $G = (V, E)$ and assign a capacity of $c(u, v) = 1$ to every directed edge $(u, v) \in E$.
- (b) **Run Max-Flow:** Execute a maximum flow algorithm from s to t .
- (c) **Path Extraction:** Because all capacities are integers (1), the *Integrality Theorem* guarantees that the resulting flow on every edge will be either 0 or 1. Each path carrying a unit of flow corresponds to one edge-disjoint path.
- (d) **Correctness:** Since each edge has a capacity of 1, no two paths can share the same directed edge, ensuring the paths are strictly edge-disjoint. Thus, Max Flow = Max Edge-Disjoint Paths.

Case B: Vertex-Disjoint Paths in Directed Graphs

To ensure that paths do not share any intermediate vertices, we must enforce a capacity constraint on the **vertices** themselves. This is achieved using a technique called **Vertex Splitting**:

- (a) **Transformation:** For every original vertex $v \in V - \{s, t\}$, split it into two distinct nodes: an **in-node** (v_{in}) and an **out-node** (v_{out}).
- (b) **Internal Edges:** Connect them with a directed internal edge (v_{in}, v_{out}) and assign it a capacity of 1.
- (c) **External Edges:** For every original directed edge $(u, v) \in E$:
 - If $u = s$, draw an edge from s to v_{in} with capacity ∞ (or 1).
 - If $v = t$, draw an edge from u_{out} to t with capacity ∞ (or 1).
 - Otherwise, draw an edge from u_{out} to v_{in} with capacity ∞ (or 1).
- (d) **Run Max-Flow:** Compute the maximum flow from s to t on this transformed graph.

Why this works: Since the internal edge (v_{in}, v_{out}) has a maximum capacity of 1, at most 1 unit of flow can pass through any split node. This physically prevents more than one path from utilizing vertex v , guaranteeing that all extracted paths are strictly vertex-disjoint.

2. Undirected Graphs

Undirected graphs present a challenge because flow can technically travel in either direction along an undirected edge $\{u, v\}$.

Case A: Edge-Disjoint Paths in Undirected Graphs

To find edge-disjoint paths when edges have no inherent direction:

- (a) **Transformation:** Replace each undirected edge $\{u, v\}$ with **two directed anti-parallel edges**: (u, v) and (v, u) .
- (b) **Capacity:** Assign a capacity of 1 to both directed edges.
- (c) **Run Max-Flow:** Run a standard max-flow algorithm from s to t .

Handling the Bidirectional Conflict:

You might wonder: What if the algorithm sends 1 unit of flow from $u \rightarrow v$ and another path sends 1 unit from $v \rightarrow u$? In network flow computations, these opposing flows cancel each other out ($f(u, v) - f(v, u) = 0$). The residual graph naturally eliminates such inefficient looping paths during optimization. Consequently, the final maximum flow value directly equals the maximum number of edge-disjoint paths in the undirected graph.

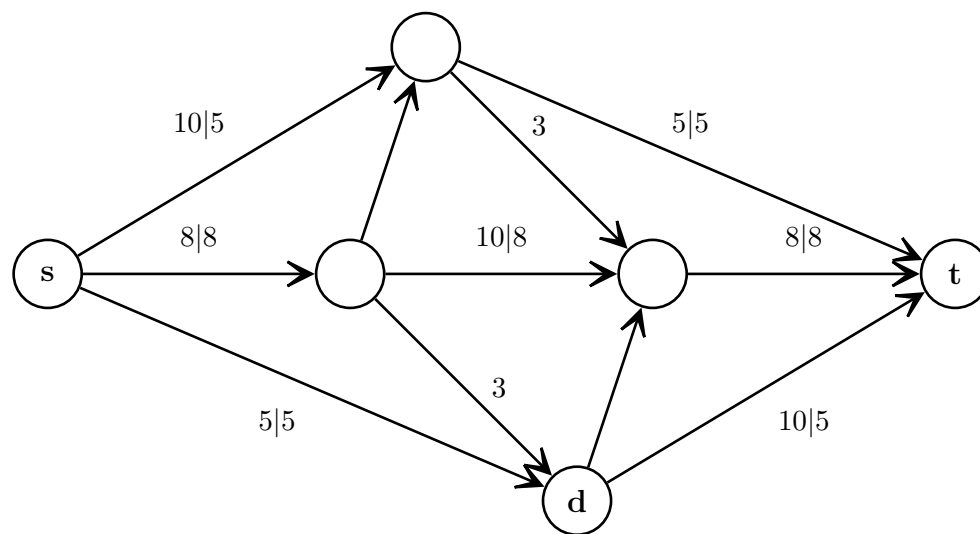
Case B: Vertex-Disjoint Paths in Undirected Graphs

To find vertex-disjoint paths in an undirected graph, we seamlessly combine the vertex splitting and anti-parallel edge techniques:

- (a) **Vertex Splitting:** Split every vertex $v \notin \{s, t\}$ into v_{in} and v_{out} , connected by a directed edge (v_{in}, v_{out}) with a capacity of 1.
- (b) **Edge Mapping:** For every undirected edge $\{u, v\}$ in the original graph:
 - Create a directed edge from $u_{out} \rightarrow v_{in}$ with capacity 1.
 - Create a directed edge from $v_{out} \rightarrow u_{in}$ with capacity 1.
- (c) **Run Max-Flow:** Calculate the max flow from s to t . The bottlenecking unit-capacity internal node edges ensure that no node is visited more than once across all discovered paths.

Q25. Figure shows a flow network on which s - t flow has been computed. The capacity of each edge appears as a label next to the edge, and the numbers in boxes give the amount of flow sent on each edge. Edges without boxed numbers have no flow on them.

- (a) What is the value of this flow? Is this a maximum (s, t) flow on this graph?
- (b) Find a minimum s - t cut in the flow network pictured in Figure 1, and also say what its capacity is.



(20 marks)

[CSE 207 | L-2/T-2 | 2007–08]

Answer**1. Value of the Current Flow & Maximality**

First, let's determine the value of the current flow, $|f|$. We can find this by summing the flow leaving the source node s (or alternatively, arriving at the sink t).

- Flow on $s \rightarrow v_1$: **5**
- Flow on $s \rightarrow v_2$: **8**
- Flow on $s \rightarrow d$: **5**

The total current flow is $|f| = 5 + 8 + 5 = \mathbf{18}$.

Is this a maximum (s, t) flow?

No. We can prove this by finding an augmenting path in the residual network G_f from s to t .

If we examine the residual capacities, we can route additional flow by pushing flow *backward* along an edge that already has flow. Let's trace a path:

- $s \rightarrow v_1$: Forward edge. Residual capacity = $10 - 5 = \mathbf{5}$.
- $v_1 \rightarrow v_3$: Forward edge. Residual capacity = $3 - 0 = \mathbf{3}$.
- $v_3 \rightarrow v_2$: **Backward edge**. The original edge $v_2 \rightarrow v_3$ has 8 units of flow. We can "push back" or redirect this flow. Residual capacity = **8**.
- $v_2 \rightarrow d$: Forward edge. Residual capacity = $3 - 0 = \mathbf{3}$.
- $d \rightarrow t$: Forward edge. Residual capacity = $10 - 5 = \mathbf{5}$.

The bottleneck (minimum residual capacity) along this path $s \rightarrow v_1 \rightarrow v_3 \rightarrow v_2 \rightarrow d \rightarrow t$ is $\min(5, 3, 8, 3, 5) = \mathbf{3}$.

By augmenting this path with 3 units of flow (which effectively reroutes 3 units of flow away from the $v_2 \rightarrow v_3$ edge and down through d), we can increase the total network flow. The new maximum flow is $18 + 3 = \mathbf{21}$.

2. Minimum s - t Cut and its Capacity

To find the minimum cut, we must identify the set of vertices S that are reachable from the source s in the **final** residual network (after the flow is maximized to 21).

After pushing the additional 3 units of flow, let's look at the edges leaving s and v_1 :

- $s \rightarrow v_1$: Flow is now 8/10. Residual capacity = 2. (v_1 is reachable from s)
- $s \rightarrow v_2$: Flow is 8/8. Saturated. (v_2 is not reachable from s)
- $s \rightarrow d$: Flow is 5/5. Saturated. (d is not reachable from s)
- $v_1 \rightarrow v_3$: Flow is 3/3. Saturated. (v_3 is not reachable from v_1)
- $v_1 \rightarrow t$: Flow is 5/5. Saturated. (t is not reachable from v_1)

Since no other forward paths exist, the reachable set of nodes from s is just $S = \{s, v_1\}$. The remaining nodes form the set $T = \{v_2, v_3, d, t\}$.

Defining the Min-Cut: * **Partition:** $(S, T) = (\{s, v_1\}, \{v_2, v_3, d, t\})$

Calculating Cut Capacity: The capacity of a cut is the sum of the capacities of all original edges going from S to T .

- $s \rightarrow v_2$: Capacity = **8**
- $s \rightarrow d$: Capacity = **5**
- $v_1 \rightarrow v_3$: Capacity = **3**

- $v_1 \rightarrow t$: Capacity = 5

Total Capacity = $8 + 5 + 3 + 5 = 21$.

(Note: This perfectly matches our calculated maximum flow, confirming the Max-Flow Min-Cut Theorem).

Q26. There is a wireless network of nodes A, B, C, D, E, F, G and H . Capacity of the wireless links are given in the following format: “-” indicates there is no link between those nodes. What is the maximum data flow, if node A sends data to node H ? What is the capacity of the optimal cut? (Use Ford-Fulkerson method and show the steps.) **(10+5=15 marks)**

	A	B	C	D	E	F	G	H
A	-	10	5	15	-	-	-	-
B	-	-	4	-	9	15	-	-
C	-	-	-	4	-	8	-	-
D	-	-	-	-	-	-	30	-
E	-	-	-	-	-	15	-	10
F	-	-	-	-	-	-	15	10
G	-	-	6	-	-	-	-	10
H	-	-	-	-	-	-	-	-

[CSE 207 | L-2/T-2 | 2006–07]

Answer

1. Maximum Data Flow using Ford-Fulkerson

To find the maximum data flow from the source node A to the sink node H , we will iteratively find **augmenting paths** in the residual network and push as much flow as possible through them until no more paths exist.

Initialization: Start with an initial flow of 0 on all edges.

- **Iteration 1: Path $A \rightarrow B \rightarrow E \rightarrow H$**
 - Capacities: $A \rightarrow B$ (10), $B \rightarrow E$ (9), $E \rightarrow H$ (10)
 - Bottleneck: $\min(10, 9, 10) = 9$
 - **Action:** Push 9 units of flow.
 - *Total Flow = 9*
- **Iteration 2: Path $A \rightarrow D \rightarrow G \rightarrow H$**
 - Capacities: $A \rightarrow D$ (15), $D \rightarrow G$ (30), $G \rightarrow H$ (10)
 - Bottleneck: $\min(15, 30, 10) = 10$
 - **Action:** Push 10 units of flow.
 - *Total Flow = 19*
- **Iteration 3: Path $A \rightarrow C \rightarrow F \rightarrow H$**
 - Capacities: $A \rightarrow C$ (5), $C \rightarrow F$ (8), $F \rightarrow H$ (10)
 - Bottleneck: $\min(5, 8, 10) = 5$
 - **Action:** Push 5 units of flow.
 - *Total Flow = 24*
- **Iteration 4: Path $A \rightarrow B \rightarrow F \rightarrow H$**
 - Residual Capacities: $A \rightarrow B$ ($10 - 9 = 1$), $B \rightarrow F$ (15), $F \rightarrow H$ ($10 - 5 = 5$)
 - Bottleneck: $\min(1, 15, 5) = 1$
 - **Action:** Push 1 unit of flow.
 - *Total Flow = 25*
- **Iteration 5: Path $A \rightarrow D \rightarrow G \rightarrow C \rightarrow F \rightarrow H$**
 - Residual Capacities:
 - * $A \rightarrow D$: $15 - 10 = 5$
 - * $D \rightarrow G$: $30 - 10 = 20$
 - * $G \rightarrow C$: 6 (*Original capacity, no flow yet*)
 - * $C \rightarrow F$: $8 - 5 = 3$
 - * $F \rightarrow H$: $10 - 5 - 1 = 4$
 - Bottleneck: $\min(5, 20, 6, 3, 4) = 3$
 - **Action:** Push 3 units of flow.
 - *Total Flow = 28*

Termination: If we examine the residual graph now, there are no more paths from A to H . All forward edges out of the reachable nodes are fully saturated.

Maximum Data Flow = 28

2. Capacity of the Optimal Cut

By the **Max-Flow Min-Cut Theorem**, the capacity of the optimal (minimum) cut is exactly equal to the maximum flow. Let's explicitly define this cut by finding the set of reachable nodes from the source in the final residual graph.

Finding the Cut Sets (S and T):

We start at source A and traverse edges that still have residual capacity (forward edges not full, or backward edges with flow):

- From A , we can still reach D (since 13/15 units are used, residual is 2).
- From D , we can reach G (since 13/30 units are used, residual is 17).
- From G , we can reach C (since 3/6 units are used, residual is 3).
- No other nodes can be reached. $A \rightarrow B$ is saturated (10/10), $A \rightarrow C$ is saturated (5/5), $G \rightarrow H$ is saturated (10/10), and $C \rightarrow F$ is saturated (8/8).

This splits our nodes into two distinct sets:

- **Source Set (S):** $\{A, C, D, G\}$
- **Sink Set (T):** $\{B, E, F, H\}$

Calculating the Cut Capacity:

The capacity of the cut is the sum of the capacities of all original forward edges going from S to T .

- $A \rightarrow B$: 10
- $C \rightarrow F$: 8
- $G \rightarrow H$: 10

(Note: Edges going backwards from T to S , like $B \rightarrow C$, are ignored when calculating cut capacity).

Optimal Cut Capacity: $10 + 8 + 10 = 28$

Q27. How can the Ford–Fulkerson method be used to find a maximum matching? Explain with a simple example. **(10 marks)** [CSE 207 | L-2/T-2 | 2006–07]

Answer

The Maximum Bipartite Matching problem can be elegantly solved by reducing it to a Maximum Flow problem and applying the **Ford-Fulkerson method**.

1. Algorithmic Reduction Strategy

Given an undirected bipartite graph $G = (L \cup R, E)$, where L is the set of left vertices, R is the set of right vertices, and E is the set of edges connecting nodes from L to R , we construct a directed flow network $G' = (V', E')$ as follows:

- Add Source and Sink:** Introduce a virtual source node s and a virtual sink node t . Thus, $V' = L \cup R \cup \{s, t\}$.
- Source to Left Set:** Direct an edge from s to every vertex $u \in L$. Assign each of these edges a capacity of **1**.
- Left Set to Right Set:** Direct all the original edges from L to R . That is, for every undirected edge $\{u, v\} \in E$ where $u \in L$ and $v \in R$, create a directed edge (u, v) with a capacity of **1** (or ∞).
- Right Set to Sink:** Direct an edge from every vertex $v \in R$ to the sink t . Assign each of these edges a capacity of **1**.

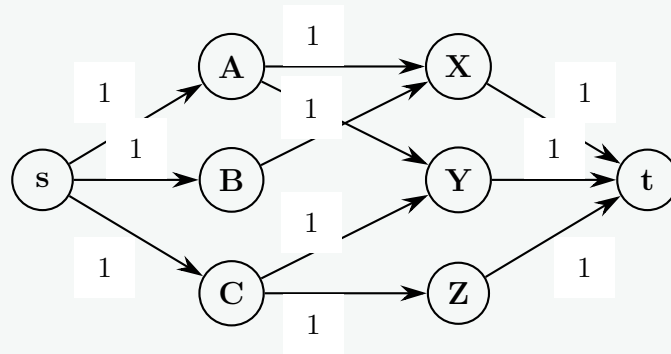
Why this works (Integrality Theorem): Since all edge capacities in G' are integers (1), the Ford-Fulkerson algorithm is guaranteed to find a maximum flow where the flow value through every edge is an integer (either 0 or 1). * A flow of 1 from $s \rightarrow u$ ensures node $u \in L$ is matched at most once. * A flow of 1 from $v \rightarrow t$ ensures node $v \in R$ is matched at most once. * Therefore, the edges between L and R carrying a flow of 1 represent a valid **maximum matching**, and Value of Max Flow = Size of Maximum Matching.

2. Explanation with a Simple Example

Let's consider a bipartite graph where: * Left set $L = \{A, B, C\}$ * Right set $R = \{X, Y, Z\}$ * Original Edges $E = \{(A, X), (A, Y), (B, X), (C, Y), (C, Z)\}$

Step 1: Network Construction

We transform this into a flow network by adding s and t , connecting them with unit capacities (1), and directing all intermediate edges from L to R with capacity 1.

**Step 2: Executing Ford-Fulkerson**

We find augmenting paths in the residual network to push flow:

- **Path 1:** $s \rightarrow A \rightarrow X \rightarrow t$ (Bottleneck = 1). Push 1 unit.
- **Path 2:** $s \rightarrow C \rightarrow Y \rightarrow t$ (Bottleneck = 1). Push 1 unit.
- **Path 3:** Now, $s \rightarrow B \rightarrow X \dots$ is blocked because $X \rightarrow t$ is full. However, we can use the **backward edge** from $X \rightarrow A$ (since $A \rightarrow X$ has 1 unit of flow):

$$s \rightarrow B \rightarrow X \xrightarrow{\text{backward}} A \rightarrow Y \rightarrow t$$

Wait, $Y \rightarrow t$ is already saturated by Path 2! Let's find an alternate open path:

$$s \rightarrow B \rightarrow X \xrightarrow{\text{backward}} A \rightarrow Y \xrightarrow{\text{backward}} C \rightarrow Z \rightarrow t$$

The bottleneck along this path is **1**. Pushing 1 unit of flow through this path automatically shifts the assignments optimally.

Final Flow & Matching Output

After completing the Ford-Fulkerson execution, the maximum flow value achieved is **3**.

The edges between L and R carrying a flow of 1/1 are:

$$\text{Maximum Matching} = \{(B, X), (A, Y), (C, Z)\}$$

All vertices on both sides are perfectly matched, yielding a maximum matching size of **3**.

Maximum Bipartite Matching

Q1. Formulate the problem of maximum bipartite matching. Give two examples of real-life problems that can be solved using maximum bipartite matching. **(12 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Answer**1. Formulation of Maximum Bipartite Matching****Definitions:**

- **Bipartite Graph:** A graph $G = (V, E)$ is bipartite if its vertex set V can be partitioned into two disjoint, independent sets L and R ($L \cup R = V$ and $L \cap R = \emptyset$) such that every edge $e \in E$ connects a vertex in L to a vertex in R .
- **Matching:** A matching $M \subseteq E$ is a collection of edges such that no two edges in M share a common vertex. In other words, every vertex in V is incident to at most one edge in M .

The Problem: The Maximum Bipartite Matching problem asks us to find a matching M of maximum possible size (i.e., containing the largest possible number of edges) for a given bipartite graph G .

Mathematical Formulation (Integer Linear Program): Let x_{ij} be a binary decision variable where $x_{ij} = 1$ if the edge (i, j) is included in the matching M , and $x_{ij} = 0$ otherwise.

- **Objective:** Maximize the total number of matched edges.

$$\text{Maximize } \sum_{(i,j) \in E} x_{ij}$$

- **Subject to (Constraints):**

$$\sum_{j:(i,j) \in E} x_{ij} \leq 1 \quad \forall i \in L \quad (\text{Each node in } L \text{ is matched at most once})$$

$$\sum_{i:(i,j) \in E} x_{ij} \leq 1 \quad \forall j \in R \quad (\text{Each node in } R \text{ is matched at most once})$$

$$x_{ij} \in \{0, 1\} \quad \forall (i, j) \in E$$

2. Real-Life Examples

Example 1: Applicant-to-Job Assignment

- **Scenario:** A staffing agency has a pool of applicants and a list of open job positions. Each applicant possesses a specific set of skills, qualifying them for only certain jobs.
- **Graph Construction:**
 - Set L : Job Applicants.
 - Set R : Open Job Positions.
 - Edges E : An edge connects applicant A to job J if A is qualified for J .
- **Objective:** By finding the maximum bipartite matching, the agency can assign applicants to jobs such that the maximum total number of jobs are filled, ensuring no applicant takes more than one job and no job is assigned to more than one applicant.

Example 2: Taxi Dispatch Systems (Ride-Hailing)

- **Scenario:** A ride-hailing app (like Uber or Lyft) receives a surge of ride requests. There are several available drivers in the area, but they can only be matched with passengers who are within a feasible driving distance (e.g., a 5-minute radius).
- **Graph Construction:**
 - Set L : Available Drivers.
 - Set R : Waiting Passengers.
 - Edges E : An edge exists between a driver and a passenger if the driver is within the acceptable ETA threshold.
- **Objective:** Solving the maximum bipartite matching problem allows the dispatch algorithm to maximize the number of passengers picked up immediately during a specific time window, optimizing system throughput.

Q2. Develop a solution for the maximum bipartite matching problem. What is the Edmonds-Karp optimization for the Ford-Fulkerson algorithm? (15 marks) [CSE 207 | L-2/T-1 | 2022–23]

Answer

1. Solution for Maximum Bipartite Matching

The standard and most efficient way to solve the Maximum Bipartite Matching problem is by reducing it to a Maximum Network Flow problem.

Given a bipartite graph $G = (L \cup R, E)$, we can develop a solution using the following algorithm:

(a) **Construct a Flow Network (G'):**

- Create a supersource node s and a supersink node t .
- Add directed edges from s to every node $u \in L$, each with a capacity of 1.
- Direct all original bipartite edges from L to R . For each edge $(u, v) \in E$ where $u \in L$ and $v \in R$, assign a capacity of 1 (or ∞).
- Add directed edges from every node $v \in R$ to t , each with a capacity of 1.

(b) **Compute Maximum Flow:** Run a network flow algorithm (like Ford-Fulkerson or Edmonds-Karp) from s to t on the newly constructed graph G' .

(c) **Extract the Matching:** Because the capacities are integers, the Integrality Theorem guarantees an integer maximum flow. Any edge between L and R that carries a flow of 1 is part of the maximum matching.

Complexity: Since the maximum possible flow is bounded by the number of vertices $|V|$, running standard Ford-Fulkerson takes $O(|V||E|)$ time, making it highly efficient for this specific problem.

2. The Edmonds-Karp Optimization for Ford-Fulkerson

The original Ford-Fulkerson method dictates that we repeatedly find *any* augmenting path in the residual network. If implemented using Depth-First Search (DFS), it can make poor path choices, leading to a worst-case time complexity of $O(E|f^*|)$, where $|f^*|$ is the maximum flow value. If capacities are irrational, it might not even terminate.

The **Edmonds-Karp optimization** resolves this by specifying exactly *how* to find the augmenting path:

- **The Core Idea:** Instead of finding just any path, Edmonds-Karp always chooses the **shortest augmenting path** from s to t in the residual graph, where "shortest" means the minimum number of edges (unweighted path length).
- **Implementation:** It achieves this by using **Breadth-First Search (BFS)** instead of DFS to find paths in the residual network. All edges in the residual graph are treated as having a weight of 1 for the BFS traversal.
- **Why it is an Optimization:** By strictly picking the shortest path, the algorithm guarantees that the length of the shortest augmenting path monotonically increases over time. It prevents the algorithm from endlessly ping-ponging back and forth across large capacity edges.
- **Time Complexity Bound:** This simple change bounds the maximum number of augmentations to $O(VE)$. Since each BFS takes $O(E)$ time, the overall worst-case time complexity becomes **$O(VE^2)$** .

Crucially, the Edmonds-Karp time complexity is **independent of the maximum flow value** ($|f^*|$) or the edge capacities, guaranteeing polynomial time termination for any network, even with very large or irrational capacities.

Q3. Explain with an example how to convert a bipartite graph G to a flow network G' such that a maximum bipartite matching M in G can be computed by finding a maximum flow f in G' . Prove that there is a maximum matching M in G if and only if there is a maximum flow f in G' . **(17 marks)** [CSE 207 | L-2/T-2 | 2009–10]

Answer

1. Conversion of Bipartite Graph to Flow Network

To compute a maximum bipartite matching for an undirected bipartite graph $G = (L \cup R, E)$, we construct a directed flow network $G' = (V', E')$ using the following steps:

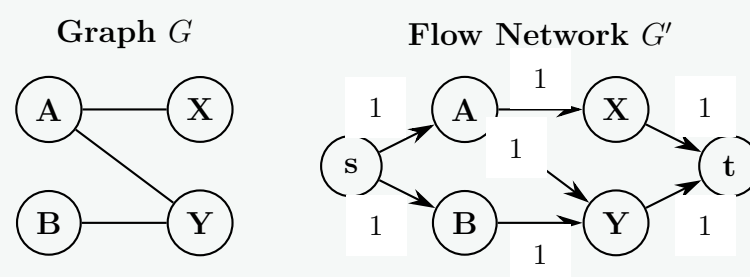
(a) **Vertices (V'):** Add a supersource node s and a supersink node t to the existing vertices. Thus, $V' = L \cup R \cup \{s, t\}$.

(b) **Edges (E') and Capacities (c):**

- **Source to Left:** For every vertex $u \in L$, add a directed edge (s, u) with capacity $c(s, u) = 1$.
- **Left to Right (Original Edges):** For every undirected edge $\{u, v\} \in E$ (where $u \in L$ and $v \in R$), add a directed edge (u, v) with capacity $c(u, v) = 1$ (or ∞).
- **Right to Sink:** For every vertex $v \in R$, add a directed edge (v, t) with capacity $c(v, t) = 1$.

Example of Conversion

Consider a bipartite graph G with $L = \{A, B\}$, $R = \{X, Y\}$, and edges $E = \{(A, X), (A, Y), (B, Y)\}$. We construct G' by adding s and t , and directing the edges with unit capacities.



2. Proof: Maximum Matching M in $G \iff$ Maximum Flow f in G'

To prove this, we must show that for any matching of size k , there exists a valid flow of value k , and conversely, for any integer flow of value k , there exists a matching of size k . By establishing this 1-to-1 correspondence, it strictly implies that the **maximum** sizes must also be perfectly equal ($|M_{max}| = |f_{max}|$).

Part A: Matching $\implies$ Flow

Claim: If G has a matching M of size k , then G' has a flow f of value k .

Proof: Let M be a valid matching in G . For every edge $(u, v) \in M$ (where $u \in L$ and $v \in R$), we route 1 unit of flow along the path $s \rightarrow u \rightarrow v \rightarrow t$. Because M is a matching, no two edges in M share a vertex. Therefore:

- Each node $u \in L$ is incident to at most one edge in M , meaning the flow entering u from s is at most 1. This respects the capacity $c(s, u) = 1$.
- Each node $v \in R$ is incident to at most one edge in M , meaning the flow leaving v to t is at most 1. This respects the capacity $c(v, t) = 1$.

Flow conservation is maintained at all intermediate nodes, and no capacities are violated. Thus, we have constructed a valid flow f of value exactly $k = |M|$.

Part B: Flow $\implies$ Matching

Claim: If G' has a maximum flow f of value k , then G has a matching M of size k .

Proof: Since all capacities in G' are integers, the **Integrality Theorem** guarantees that the maximum flow f will assign an integer flow (either 0 or 1) to every edge. We construct a matching M by taking all edges (u, v) between L and R that carry exactly 1 unit of flow:

$$M = \{(u, v) \in E \mid f(u, v) = 1\}$$

We must verify that M is a valid matching. By flow conservation at node $u \in L$:

$$\sum_{v \in R} f(u, v) = f(s, u) \leq c(s, u) = 1$$

This means at most 1 unit of flow leaves u , so u is paired with at most one vertex in R . Similarly, by flow conservation at node $v \in R$:

$$\sum_{u \in L} f(u, v) = f(v, t) \leq c(v, t) = 1$$

This means at most 1 unit of flow enters v , so v is paired with at most one vertex in L . Since no vertex in L or R shares more than one edge in M , M is a valid bipartite matching. Because 1 unit of flow reaches t for every edge in M , the size of the matching $|M|$ is exactly the flow value k .

Conclusion

Since any matching of size k yields a flow of value k , and any integer flow of value k yields a matching of size k , the largest possible matching directly corresponds to the largest possible flow. Therefore, finding the maximum flow in G' computes the maximum matching in G .

Q4. Explain with an example how to convert a bipartite graph G to a flow network G' such that a maximum bipartite matching M in G can be computed by finding a maximum flow f in G' . Prove that there is a maximum matching M in G if and only if there is a maximum flow f in G' . (17.5 marks)

Answer

1. Conversion of Bipartite Graph to Flow Network

To compute a maximum bipartite matching for an undirected bipartite graph $G = (L \cup R, E)$, we construct a directed flow network $G' = (V', E')$ using the following steps:

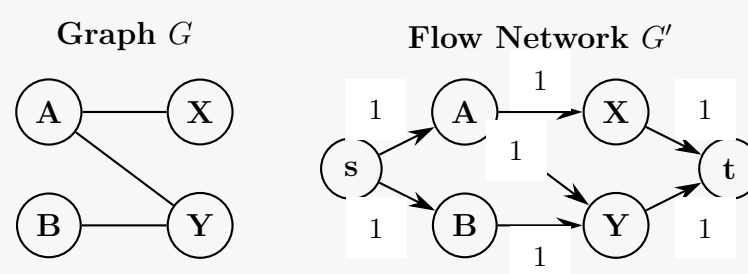
(a) **Vertices (V'):** Add a supersource node s and a supersink node t to the existing vertices. Thus, $V' = L \cup R \cup \{s, t\}$.

(b) **Edges (E') and Capacities (c):**

- **Source to Left:** For every vertex $u \in L$, add a directed edge (s, u) with capacity $c(s, u) = 1$.
- **Left to Right (Original Edges):** For every undirected edge $\{u, v\} \in E$ (where $u \in L$ and $v \in R$), add a directed edge (u, v) with capacity $c(u, v) = 1$ (or ∞).
- **Right to Sink:** For every vertex $v \in R$, add a directed edge (v, t) with capacity $c(v, t) = 1$.

Example of Conversion

Consider a bipartite graph G with $L = \{A, B\}$, $R = \{X, Y\}$, and edges $E = \{(A, X), (A, Y), (B, Y)\}$. We construct G' by adding s and t , and directing the edges with unit capacities.



2. Proof: Maximum Matching M in $G \iff$ Maximum Flow f in G'

To prove this, we must show that for any matching of size k , there exists a valid flow of value k , and conversely, for any integer flow of value k , there exists a matching of size k . By establishing this 1-to-1 correspondence, it strictly implies that the **maximum** sizes must also be perfectly equal ($|M_{max}| = |f_{max}|$).

Part A: Matching $\implies$ Flow

Claim: If G has a matching M of size k , then G' has a flow f of value k .

Proof: Let M be a valid matching in G . For every edge $(u, v) \in M$ (where $u \in L$ and $v \in R$), we route 1 unit of flow along the path $s \rightarrow u \rightarrow v \rightarrow t$. Because M is a matching, no two edges in M share a vertex. Therefore:

- Each node $u \in L$ is incident to at most one edge in M , meaning the flow entering u from s is at most 1. This respects the capacity $c(s, u) = 1$.
- Each node $v \in R$ is incident to at most one edge in M , meaning the flow leaving v to t is at most 1. This respects the capacity $c(v, t) = 1$.

Flow conservation is maintained at all intermediate nodes, and no capacities are violated. Thus, we have constructed a valid flow f of value exactly $k = |M|$.

Part B: Flow $\implies$ Matching

Claim: If G' has a maximum flow f of value k , then G has a matching M of size k .

Proof: Since all capacities in G' are integers, the **Integrality Theorem** guarantees that the maximum flow f will assign an integer flow (either 0 or 1) to every edge. We construct a matching M by taking all edges (u, v) between L and R that carry exactly 1 unit of flow:

$$M = \{(u, v) \in E \mid f(u, v) = 1\}$$

We must verify that M is a valid matching. By flow conservation at node $u \in L$:

$$\sum_{v \in R} f(u, v) = f(s, u) \leq c(s, u) = 1$$

This means at most 1 unit of flow leaves u , so u is paired with at most one vertex in R . Similarly, by flow conservation at node $v \in R$:

$$\sum_{u \in L} f(u, v) = f(v, t) \leq c(v, t) = 1$$

This means at most 1 unit of flow enters v , so v is paired with at most one vertex in L . Since no vertex in L or R shares more than one edge in M , M is a valid bipartite matching. Because 1 unit of flow reaches t for every edge in M , the size of the matching $|M|$ is exactly the flow value k .

Conclusion

Since any matching of size k yields a flow of value k , and any integer flow of value k yields a matching of size k , the largest possible matching directly corresponds to the largest possible flow. Therefore, finding the maximum flow in G' computes the maximum matching in G .

Stable Matching (Gale-Shapley)

Q1. In a stable matching problem (**Gale-Shapley algorithm**) each participant has a true preference order. Now consider a woman w . Suppose w prefers man m to m' , but both m and m' are low on her list of preferences. Can it be the case that by switching the order of m and m' on her list of preferences (i.e., by falsely claiming that she prefers m' to m) and running the algorithm with this false preference list, w will end up with a man m'' that she truly prefers to both m and m' ? Give proof to support your answer. (10 marks) [CSE 207 | L-2/T-1 | 2020–21]

Answer

Yes, it can absolutely be the case.

In the standard Gale-Shapley algorithm (where men propose to women), the mechanism is strategy-proof for the proposing side (men), but it is **not strategy-proof for the receiving side** (women). A woman can sometimes improve her final match by falsifying her preference list, even by swapping the order of two men who are lower on her true preference list.

We can prove this constructively by providing a scenario where this exact manipulation strictly benefits the woman w .

Proof by Counterexample

Consider a matching market with three men $M = \{m, m', m''\}$ and three women $W = \{w, w_2, w_3\}$.

1. The True Preferences

Assume the true preference lists for all participants are as follows (ordered from most preferred to least preferred):

Men's Preferences:

- m : $w \succ w_2 \succ w_3$
- m' : $w \succ w_3 \succ w_2$
- m'' : $w_2 \succ w \succ w_3$

Women's True Preferences:

- w : $m'' \succ m \succ m'$ (Note: m'' is her top choice, m and m' are lower)
- w_2 : $m \succ m'' \succ m'$
- w_3 : $m' \succ m \succ m''$

2. Execution 1: Truthful Bidding

Let's trace the Gale-Shapley algorithm when woman w submits her **true** preference list ($m'' \succ m \succ m'$).

- **Round 1:**
 - m proposes to his first choice, w .
 - m' proposes to his first choice, w .
 - m'' proposes to his first choice, w_2 .

Woman w receives proposals from both m and m' . According to her true list, $m \succ m'$. She tentatively **accepts** m and **rejects** m' . Woman w_2 tentatively accepts m'' .

- **Round 2:**
 - The rejected man m' proposes to his next choice, w_3 .

Woman w_3 accepts m' .

- **Termination:** Everyone is matched. The algorithm ends.
- **Truthful Outcome for w :** She is matched with m .

3. Execution 2: Strategic Manipulation

Now, suppose w falsely submits a preference list where she swaps the lower two men, claiming $m' \succ m$. Her submitted list is now: $m'' \succ m' \succ m$.

- **Round 1:**
 - $m \rightarrow w$, $m' \rightarrow w$, $m'' \rightarrow w_2$.

Woman w has proposals from m and m' . According to her *false* list, $m' \succ m$. She tentatively **accepts** m' and **rejects** m . Woman w_2 tentatively accepts m'' .

- **Round 2:**
 - The rejected man m proposes to his next choice, w_2 .

Woman w_2 now has m and m'' . According to her true list ($m \succ m''$), she **accepts** m and **rejects** m'' .

- **Round 3:**
 - The newly rejected man m'' proposes to his next choice, w .

Woman w currently holds m' , but just received a proposal from m'' . According to her (false and true) list, $m'' \succ m'$. She **accepts** m'' and **rejects** m' .

- **Round 4:**
 - The rejected man m' proposes to w_3 , who accepts him.

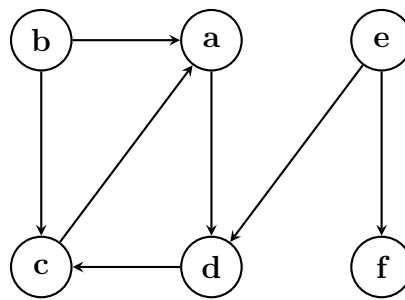
- **Termination:** Everyone is matched.
- **Manipulated Outcome for w :** She is matched with m'' .

Conclusion

By falsely rejecting m in Round 1 (due to the swapped preferences), w forced m to propose to w_2 . This caused w_2 to dump m'' . Consequently, m'' was freed up to propose to w . Therefore, by swapping two lower-preference men on her list, w successfully manipulated the chain of proposals to capture a man (m'') whom she truly preferred to both.

Graph Traversal & Properties

Q1. Classify the edges of the given graph after performing a Depth-First Search (DFS). If multiple nodes are available to explore at any point, always choose the node that comes first in alphabetical ordering. Start the DFS traversal from node a . (Possible classes: tree-edge, forward edge, back edge, and cross edge.)



(10 marks)

[CSE 207 | L-2/T-1 | 2023–24]

Answer

DFS Traversal and Edge Classification

We perform the Depth-First Search (DFS) starting from node a . When multiple unvisited neighbors exist, we explore them in alphabetical order. We will track the **Discovery Time** (d) and **Finish Time** (f) to formally classify the edges.

Step-by-Step Execution

(a) **Start at node a :**

- Node a is discovered. Time $d[a] = 1$.
- Neighbors of a : only d .

(b) **Explore (a, d) :**

- Node d is unvisited. (a, d) is a **Tree Edge**.
- Node d is discovered. Time $d[d] = 2$.
- Neighbors of d : only c .

(c) **Explore (d, c) :**

- Node c is unvisited. (d, c) is a **Tree Edge**.
- Node c is discovered. Time $d[c] = 3$.
- Neighbors of c : only a .

(d) **Explore (c, a) :**

- Node a is already discovered but not yet finished (it is an ancestor in the current DFS path). (c, a) is a **Back Edge**.
- No more neighbors for c . Finish c . Time $f[c] = 4$.
- No more neighbors for d . Finish d . Time $f[d] = 5$.
- No more neighbors for a . Finish a . Time $f[a] = 6$.

(e) **Find next unvisited node (Alphabetical order):** b is unvisited.

- Node b is discovered. Time $d[b] = 7$.
- Neighbors of b : a and c . Process a first (alphabetically).

(f) **Explore (b, a) :**

- Node a is completely finished ($f[a] = 6$). Since $d[b] > d[a]$, (b, a) is a **Cross Edge**.

(g) **Explore (b, c) :**

- Node c is completely finished ($f[c] = 4$). Since $d[b] > d[c]$, (b, c) is a **Cross Edge**.
- No more neighbors for b . Finish b . Time $f[b] = 8$.

(h) **Find next unvisited node (Alphabetical order):** e is unvisited.

- Node e is discovered. Time $d[e] = 9$.
- Neighbors of e : d and f . Process d first.

(i) **Explore (e, d) :**

- Node d is completely finished ($f[d] = 5$). Since $d[e] > d[d]$, (e, d) is a **Cross Edge**.

(j) **Explore (e, f) :**

- Node f is unvisited. (e, f) is a **Tree Edge**.
- Node f is discovered. Time $d[f] = 10$.
- Neighbors of f : None. Finish f . Time $f[f] = 11$.
- No more neighbors for e . Finish e . Time $f[e] = 12$.

Final Edge Classifications

Based on the DFS traversal above, the complete classification of all edges in the graph is:

- **Tree Edges:** (a, d) , (d, c) , (e, f)
- **Back Edges:** (c, a)
- **Forward Edges:** *None*
- **Cross Edges:** (b, a) , (b, c) , (e, d)

Q2. Gotham city is in grave danger. Batman must suit up with the following constraints:

- He must wear the bodysuit first.
- Both utility belt and armor plate can be worn in any order on top of the bodysuit.
- The grappling gun and smoke bombs can be attached only after the utility belt is worn.
- The cape has to be mounted on top of the armor plate.
- The glider wing must be attached to both the armor plate and the utility belt.

First, draw a dependency graph of all items. Then, recommend a valid ordering of the items to help Batman get ready quickly but properly. **(12 marks)** [CSE 207 | L-2/T-1 | 2023–24]

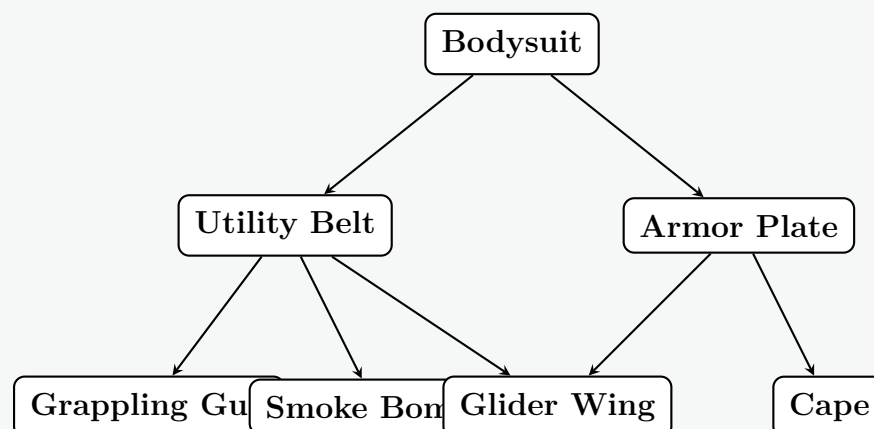
Answer

1. Dependency Graph

To model Batman's dressing constraints, we can create a **Directed Acyclic Graph (DAG)** where each node represents an item of equipment, and a directed edge from node X to node Y ($X \rightarrow Y$) means that item X must be put on before item Y .

Based on the constraints:

- Bodysuit $\rightarrow$ Utility Belt (Belt is on top of bodysuit)
- Bodysuit $\rightarrow$ Armor Plate (Armor is on top of bodysuit)
- Utility Belt $\rightarrow$ Grappling Gun (Gun attaches to belt)
- Utility Belt $\rightarrow$ Smoke Bombs (Bombs attach to belt)
- Armor Plate $\rightarrow$ Cape (Cape mounts on armor)
- Armor Plate $\rightarrow$ Glider Wing (Glider attaches to armor)
- Utility Belt $\rightarrow$ Glider Wing (Glider attaches to belt)



2. Recommended Valid Ordering

To find a valid order for Batman to suit up, we perform a **Topological Sort** on the dependency graph. A topological sort ensures that for every directed edge $X \rightarrow Y$, item X always appears before item Y in the ordering.

Because there are independent parallel branches in the graph, multiple valid orderings exist. We construct one by repeatedly selecting a node with an in-degree of 0 (all its prerequisites have been fulfilled).

Step-by-step Topological Sort:

- Start with **Bodysuit** (in-degree 0).
- Now, *Utility Belt* and *Armor Plate* have in-degree 0. We can pick **Armor Plate**.
- Next, we can pick **Cape** (since Armor is on) or *Utility Belt*. Let's pick **Utility Belt**.
- Now *Grappling Gun*, *Smoke Bombs*, and *Glider Wing* (since both Armor and Belt are on) have in-degree 0. We can pick **Glider Wing**.
- Next, pick **Grappling Gun**.
- Next, pick **Smoke Bombs**.
- Finally, pick **Cape**.

Final Recommended Ordering:

Bodysuit → Armor Plate → Utility Belt → Glider Wing → Grappling Gun → Smoke Bombs → Cape

(Note: "*Bodysuit → Utility Belt → Armor Plate → Grappling Gun → Smoke Bombs → Cape → Glider Wing*" is another perfectly valid sequence.)

Q3. Mention the runtimes of the following algorithms:

- Kruskal's algorithm without Disjoint Set Union (DSU),
- Kruskal's algorithm with DSU,
- Ford-Fulkerson algorithm,
- Edmonds-Karp algorithm.

(8 marks)

[CSE 207 | L-2/T-1 | 2023–24]

Answer

Here are the worst-case time complexities for the requested algorithms, where V is the number of vertices, E is the number of edges, and $|f^*|$ is the maximum flow value:

(a) **Kruskal's algorithm without Disjoint Set Union (DSU): $O(E \cdot V)$ or $O(E^2)$**

- Reasoning:* Sorting the edges takes $O(E \log E)$. Without an efficient DSU data structure, checking if adding an edge creates a cycle typically requires a standard graph traversal (like BFS or DFS) taking $O(V)$ time for each of the E edges. This $O(E \cdot V)$ cycle-checking phase dominates the overall runtime.

(b) **Kruskal's algorithm with DSU: $O(E \log E)$ or $O(E \log V)$**

- Reasoning:* Sorting the edges takes $O(E \log E)$. With a DSU utilizing path compression and union by rank, cycle checking and merging sets takes $O(E \cdot \alpha(V))$ time across all edges, where α is the nearly constant inverse Ackermann function. Thus, the sorting step dominates the runtime. Since $E \leq V^2$, it follows that $\log E \leq 2 \log V$, making $O(E \log E)$ asymptotically equivalent to $O(E \log V)$.

(c) **Ford-Fulkerson algorithm: $O(E \cdot |f^*|)$**

- Reasoning:* In the worst case (with integer capacities), the algorithm finds an augmenting path using DFS or BFS, taking $O(E)$ time, and pushes a minimum of 1 unit of flow per iteration. Therefore, it might require up to $|f^*|$ iterations before max flow is reached.

(d) **Edmonds-Karp algorithm: $O(V \cdot E^2)$**

- Reasoning:* Edmonds-Karp is an optimization of the Ford-Fulkerson method that uses Breadth-First Search (BFS) to continuously find the shortest augmenting path (in terms of the number of edges). This guarantees that the number of augmenting paths found cannot exceed $O(V \cdot E)$. Since each BFS takes $O(E)$ time, the total complexity is firmly bounded independent of the maximum flow value.

Q4. Consider a peculiar city with N towns and M bidirectional roads (connecting towns) classified into three types. This city is peculiar due to unconventional regulations regarding road usage: men are permitted to travel only on roads of types 1 and 3, while women are permitted to travel only on roads of types 2 and 3 exclusively. The Mayor of the city intends to renovate the city by reconstructing some roads, with the condition that the normal routine of the citizens should not be hampered.

Now, you are hired as a computer scientist to answer an important question: What is the maximum number of roads that can be renovated (remain closed for traffic during renovation) while ensuring that the city remains connected for both men and women? The city is said to be connected if it is still possible to travel from any town X to any town Y using the existing roads.

- (a) Formulate the problem as a graph problem.
- (b) Develop the solution using a well-known graph algorithm.
- (c) Write the pseudo-code of the solution.

(23 marks)

[CSE 207 | L-2/T-1 | 2022–23]

Answer**1. Graph Problem Formulation**

We can model the city as an undirected graph $G = (V, E)$, where:

- V is the set of N towns (vertices).
- E is the set of M roads (edges).

Each edge $e \in E$ has a type $t(e) \in \{1, 2, 3\}$. We can divide E into three disjoint subsets: E_1 (type 1, men only), E_2 (type 2, women only), and E_3 (type 3, both).

The problem requires us to find a subset of edges $E' \subseteq E$ such that:

- (a) The graph for men, $G_{men} = (V, E' \cap (E_1 \cup E_3))$, is connected.
- (b) The graph for women, $G_{women} = (V, E' \cap (E_2 \cup E_3))$, is connected.

To **maximize** the number of roads that can be renovated (removed), we must **minimize** the size of our kept edge set, $|E'|$. Thus, the objective is to find $\max(|E| - |E'|)$, provided both G_{men} and G_{women} contain a spanning tree. If it is impossible to connect all towns for either men or women even using all available edges, the answer is that it's impossible.

2. Solution Development

This problem can be elegantly solved using a greedy approach based on **Kruskal's Algorithm** for finding Minimum Spanning Trees, utilizing the **Disjoint Set Union (DSU)** data structure.

Core Logic (Greedy Choice): A Type 3 edge is highly valuable because it contributes to the connectivity of *both* the men's and women's graphs simultaneously at the cost of keeping just one road. Type 1 and Type 2 edges only contribute to one graph each. Therefore, to minimize the total number of roads kept, we must **prioritize keeping as many useful Type 3 roads as possible** before considering Type 1 or Type 2 roads.

Algorithm Steps:

- (a) Initialize two separate DSU structures: DSU_{men} and DSU_{women} , each with N independent components representing the towns.
- (b) **Phase 1 (Process Type 3 edges):** Iterate through all roads of Type 3. If a road connects two towns that are currently in different components in either DSU_{men} or DSU_{women} , keep this road and union the components in the respective DSUs. (*Note: Since we process these first, a Type 3 edge will always either connect in both DSUs or be redundant in both.*)
- (c) **Phase 2 (Process Type 1 edges):** Iterate through all roads of Type 1. If a road connects two towns in DSU_{men} that are not yet connected, keep it and union them in DSU_{men} .
- (d) **Phase 3 (Process Type 2 edges):** Iterate through all roads of Type 2. If a road connects two towns in DSU_{women} that are not yet connected, keep it and union them in DSU_{women} .
- (e) **Verification:** Check if both DSU_{men} and DSU_{women} have successfully been reduced to exactly 1 connected component.
 - If yes, the maximum roads we can renovate is: Total Roads – Roads Kept.
 - If no, return -1 (it is impossible to ensure full connectivity).

3. Pseudo-codeAlgorithm 27: *Maximum Renovated Roads*

```

1: function MAXRENOVATEDROADS( $N, edges$ )
2:    $DSU_{men} \leftarrow \text{INITDSU}(N)$ 
3:    $DSU_{women} \leftarrow \text{INITDSU}(N)$ 
4:    $kept\_edges \leftarrow 0$ 

```

▷ Phase 1: Prioritize Type 3 roads

```

5:   for all  $edge \in edges$  do
6:     if  $edge.type = 3$  then
7:        $useful\_for\_men \leftarrow DSU_{men}.UNION(edge.u, edge.v)$ 
8:        $useful\_for\_women \leftarrow DSU_{women}.UNION(edge.u, edge.v)$ 
9:       if  $useful\_for\_men \vee useful\_for\_women$  then
10:         $kept\_edges \leftarrow kept\_edges + 1$ 

```



```

11:     end if
12:   end if
13: end for
                                     ▷ Phase 2: Process Type 1 roads (Men only)

14: for all  $edge \in edges$  do
15:   if  $edge.type = 1$  then
16:     if  $DSU_{men}.UNION(edge.u, edge.v)$  then
17:        $kept\_edges \leftarrow kept\_edges + 1$ 
18:     end if
19:   end if
20: end for
                                     ▷ Phase 3: Process Type 2 roads (Women only)

21: for all  $edge \in edges$  do
22:   if  $edge.type = 2$  then
23:     if  $DSU_{women}.UNION(edge.u, edge.v)$  then
24:        $kept\_edges \leftarrow kept\_edges + 1$ 
25:     end if
26:   end if
27: end for

28: if  $DSU_{men}.COMPONENTS = 1 \wedge DSU_{women}.COMPONENTS = 1$  then
29:   return  $|edges| - kept\_edges$ 
30: else
31:   return -1
32: end if
33: end function
                                     ▷ Verification: Are both graphs fully connected?
                                     ▷ Impossible to connect the city

                                     ▷ — Helper DSU Procedures —

34: procedure INITDSU( $N$ )
35:    $parent \leftarrow$  array of size  $N$ 
36:   for  $i \leftarrow 1$  to  $N$  do
37:      $parent[i] \leftarrow i$ 
38:   end for
39:    $num\_components \leftarrow N$ 
40: end procedure
41: function FIND( $i$ )
42:   if  $parent[i] = i$  then
43:     return  $i$ 
44:   end if
45:    $parent[i] \leftarrow FIND(parent[i])$ 
46:   return  $parent[i]$ 
47: end function
48: function UNION( $i, j$ )
49:    $root\_i \leftarrow FIND(i)$ 
50:    $root\_j \leftarrow FIND(j)$ 
51:   if  $root\_i \neq root\_j$  then
52:      $parent[root\_i] \leftarrow root\_j$ 
53:      $num\_components \leftarrow num\_components - 1$ 
54:     return true
55:   end if
56:   return false
57: end function
58: function COMPONENTS
59:   return  $num\_components$ 
60: end function

```

Q5. Define and distinguish the two basic properties of the greedy algorithm. (8 marks)

[CSE 207 | L-2/T-2 | 2021–22]

Answer

To prove that a greedy algorithm yields a globally optimal solution for a given problem, the problem must exhibit two fundamental properties: the **Greedy Choice Property** and **Optimal Substructure**.

1. Defining the Properties

- **Greedy Choice Property:** This property states that a globally optimal solution can be arrived at by making a locally optimal (greedy) choice. In other words, at each step, the algorithm makes whatever choice seems best at that exact moment, without worrying about the future or solving smaller subproblems first. Once a choice is made, it is never reconsidered or reversed.

- **Optimal Substructure:** A problem exhibits optimal substructure if an optimal solution to the entire problem contains within it optimal solutions to its subproblems. This means that after making the greedy choice, the remaining problem is simply a smaller instance of the exact same problem, and solving it optimally will complete the global optimal solution.

2. Distinguishing the Properties

While both are required for a greedy algorithm to yield a mathematically optimal result, they serve distinctly different roles in the algorithm's design and theory:

- **Decision-Making vs. Problem Structure:**
 - The **Greedy Choice Property** is about *how* we make decisions. It dictates the action taken at each step (e.g., "always pick the lightest available edge" in Kruskal's algorithm). It tells us that we don't need to exhaustively search all possibilities to find the right path.
 - **Optimal Substructure** is about the *inherent nature of the problem* itself. It dictates how the problem breaks down structurally (e.g., "the shortest path from A to C via B is composed of the shortest path from A to B and the shortest path from B to C").
- **Relation to Dynamic Programming (DP):**
 - **Optimal Substructure** is a shared requirement for both Greedy Algorithms and Dynamic Programming. If a problem has optimal substructure but *lacks* the greedy choice property (like the 0/1 Knapsack problem), you must use DP (which evaluates multiple choices) instead of a greedy approach.
 - The **Greedy Choice Property** is the defining characteristic that separates Greedy Algorithms from DP. It allows the algorithm to blindly commit to a single path forward, making it much faster and more memory-efficient than DP, which must evaluate, compare, and store overlapping subproblems to find the true optimal answer.

Q6. Jack Straws: In the game of Jack Straws, a number of plastic or wooden "straws" are dumped on the table and players try to remove them one by one without disturbing the other straws. Here, we are only concerned with whether various pairs of straws are connected by a path of touching straws. Given a list of the endpoints for $n > 1$ straws (as if they were dumped on a large piece of graph paper), determine all the pairs of straws that are connected. Note that touching is connecting, but also that two straws can be connected indirectly via other connected straws. **(15 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Answer

Algorithm to Determine Connected Jack Straws

This problem can be modeled as finding the **connected components** of an undirected graph.

Graph Formulation:

- **Vertices (V):** Each of the n straws represents a vertex.
- **Edges (E):** An undirected edge exists between vertex i and vertex j if and only if straw i physically intersects (touches or crosses) straw j .

Two straws are considered connected (directly or indirectly) if they belong to the same connected component in this graph.

To solve this efficiently, we break the solution into two main parts: geometrically determining if two line segments intersect, and algorithmically grouping the connected straws.

Step 1: Line Segment Intersection (The Geometric Step)

Each straw is a line segment defined by two endpoints: $p_1 = (x_1, y_1)$ and $q_1 = (x_2, y_2)$. To check if two segments p_1q_1 and p_2q_2 intersect, we use the **Orientation test**.

The orientation of an ordered triplet of points (p, q, r) can be:

- Collinear (0)
- Clockwise (1)
- Counterclockwise (2)

Two segments p_1q_1 and p_2q_2 intersect if:

- **General Case:** The triplets (p_1, q_1, p_2) and (p_1, q_1, q_2) have different orientations, **AND** the triplets (p_2, q_2, p_1) and (p_2, q_2, q_1) have different orientations.
- **Collinear Case (Special):** The segments are collinear and overlap (their x-projections and y-projections intersect).

Step 2: Grouping with Disjoint Set Union (The Algorithmic Step)

Instead of building an adjacency matrix and running Breadth-First Search (BFS), we can use a **Disjoint Set Union (DSU)** data structure to efficiently group the straws as we find intersections.

- Initialize a DSU where each straw $1, 2, \dots, n$ is in its own set.
- Iterate through all unique pairs of straws (i, j) where $1 \leq i < j \leq n$.
- For each pair, check if segment i intersects segment j using the geometric step above.
- If they intersect, perform a $\text{Union}(i, j)$ operation to merge their sets.
- After checking all pairs, two straws u and v are connected if and only if $\text{Find}(u) == \text{Find}(v)$.

Complexity and Output

- Time Complexity:** There are $\frac{n(n-1)}{2}$ pairs of straws. Checking intersection takes $O(1)$ time. DSU operations with path compression and union by rank take almost $O(1)$ time, specifically $O(\alpha(n))$, where α is the inverse Ackermann function. Thus, the total time complexity is strictly bounded by $O(n^2)$.
- Generating the Output:** To determine *all* pairs that are connected, you iterate through all pairs (i, j) one final time and output the pair if they share the same DSU root.

Q7. Rumor spreading: There are n people, each in possession of a different rumor. They want to share all the rumors with each other by sending electronic messages. Assume that a sender includes all the rumors he or she knows at the time the message is sent and that a message may only have one addressee. Design a greedy algorithm that always yields the minimum number of messages they need to send to guarantee that every one of them gets all the rumors. **(13 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Answer**The "Coordinator" (Hub-and-Spoke) Algorithm**

To solve the rumor spreading problem with the absolute minimum number of one-way messages, we can use a greedy approach where we maximize the consolidation of information before distributing it. This is often referred to as the **Hub-and-Spoke** or **Coordinator** algorithm.

Algorithm Steps

- Elect a Coordinator:** Select exactly one person out of the n people to act as the central information hub (let's call them Person P_1).
- Phase 1: Information Gathering (Greedy Accumulation)**
 - Every other person $(P_2, P_3, \dots, P_n)$ sends exactly one message to the coordinator, P_1 .
 - Messages sent in this phase:* $n - 1$
 - State at the end of Phase 1:* P_1 now knows all n rumors. Everyone else still only knows their own original rumor.
- Phase 2: Information Dissemination (Greedy Distribution)**
 - The coordinator (P_1), who now possesses the complete set of rumors, sends exactly one message back to every other person $(P_2, P_3, \dots, P_n)$.
 - Messages sent in this phase:* $n - 1$
 - State at the end of Phase 2:* Everyone has received a message containing all n rumors. The goal is achieved.

Why this is Optimal

The total number of messages sent using this algorithm is:

$$(n - 1) + (n - 1) = 2n - 2$$

This is the mathematically proven absolute minimum for one-way electronic messages. Here is the logical proof for the lower bound:

- The $n - 1$ Gathering Bound:** There are n distinct rumors initially isolated. For any single person to learn all of them, at least $n - 1$ people must send a message to move their unique rumor out of isolation. Thus, it takes at least $n - 1$ messages just to compile all rumors into one place.
- The $n - 1$ Distribution Bound:** Once a complete compilation of rumors exists, there are still $n - 1$ people who do not have the full set. It requires at least $n - 1$ distinct messages to deliver the missing information to those $n - 1$ individuals.

By strictly separating the process into a pure gathering phase followed by a pure dissemination phase, the algorithm guarantees the minimum possible message count of $2n - 2$.

Q8. The reverse of a directed graph $G = (V, E)$ is another directed graph $G_R = (V, E_R)$ on the same vertex set, but with all edges reversed; that is, $E_R = \{(v, u) : (u, v) \in E\}$. Give a linear-time algorithm for computing the reverse of a graph in adjacency list format. **(10 marks)** [CSE 207 | L-2/T-2 | 2019–20]

Answer

Problem Analysis

We are given a directed graph $G = (V, E)$ represented as an adjacency list. In this representation, an array `Adj` of size $|V|$ stores a list of neighbors for each vertex. Specifically, a vertex v is in the list `Adj[u]` if and only if there is a directed edge $(u, v) \in E$. We need to compute the reversed graph $G_R = (V, E_R)$, where $E_R = \{(v, u) : (u, v) \in E\}$. In terms of adjacency lists, this means that if $v \in \text{Adj}[u]$ in the original graph G , we must have $u \in \text{Adj}_R[v]$ in the reversed graph G_R .

Intuition and Approach

To construct G_R efficiently, we can systematically iterate through all edges of the original graph G . For every vertex $u \in V$, we traverse its adjacency list `Adj[u]`. For every vertex v we encounter in this list, we have found an edge $(u, v) \in E$. We then immediately add u to the new adjacency list `Adj_R[v]`, which corresponds to the reversed edge $(v, u) \in E_R$.

Because appending an element to the end of a linked list (or dynamic array) can be done in $O(1)$ time, processing each edge takes constant time.

Algorithm

Algorithm 28 Compute Reverse Graph

```

1: Input: Adj, an array of size  $|V|$  representing the adjacency lists of  $G$ 
2: Output: Adj_R, an array of size  $|V|$  representing the adjacency lists of  $G_R$ 

3: Initialization: Let Adj_R be a new array of  $|V|$  empty lists.
4: for each vertex  $u \in \{1, 2, \dots, |V|\}$  do
5:   for each vertex  $v \in \text{Adj}[u]$  do
6:     Append  $u$  to Adj_R[v]
7:   end for
8: end for
9: return Adj_R

```

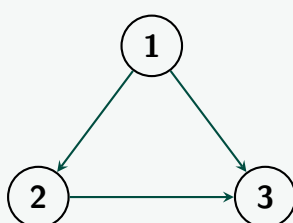
Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|V| + |E|)$.
 - Initializing the array of $|V|$ empty lists takes $\mathcal{O}(|V|)$ time.
 - The outer loop runs exactly $|V|$ times, once for each vertex.
 - The inner loop iterates over the elements of `Adj[u]`. Across the entire execution of the algorithm, the inner loop executes exactly once for every directed edge in the graph, totaling $|E|$ iterations.
 - Inside the inner loop, appending an element to a list takes $\mathcal{O}(1)$ time.
 - Summing these together, the total time complexity is strictly linear with respect to the size of the graph: $\mathcal{O}(|V| + |E|)$.
- **Space Complexity:** $\mathcal{O}(|V| + |E|)$.
 - We allocate a new array `Adj_R` of size $|V|$ to hold the list headers.
 - We insert exactly $|E|$ elements into these lists across the entire execution.
 - Thus, the total auxiliary space required to represent the output graph is $\mathcal{O}(|V| + |E|)$.

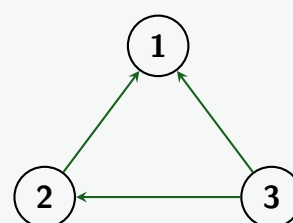
Visualization

Consider a simple directed graph G and its reverse G_R . As seen below, traversing the adjacency list of G allows us to map each edge directly to its reversed counterpart.

Original Graph G



Reversed Graph G_R



Adjacency List of G		Adjacency List of G_R	
Vertex	Edges	Vertex	Edges
1	$\rightarrow 2 \rightarrow 3$	1	(empty)
2	$\rightarrow 3$	2	$\rightarrow 1$
3	(empty)	3	$\rightarrow 1 \rightarrow 2$

Q9. The strength of a path in an edge-weighted graph G is the minimum weight among its edges. The all-pairs strongest path problem asks to find the strongest path between every pair of vertices. Design an algorithm to solve this problem. **(5 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Answer

Problem Formulation

Let $G = (V, E)$ be a directed, edge-weighted graph with weight function $w : E \rightarrow \mathbb{R}$. The strength of a path $p = \langle v_0, v_1, \dots, v_m \rangle$ is defined as the bottleneck or minimum edge weight along that path:

$$\text{strength}(p) = \min_{1 \leq i \leq m} w(v_{i-1}, v_i)$$

We define the strongest path from vertex u to vertex v as the path that *maximizes* this strength. If no path exists, the strength is $-\infty$. For a trivial path from a vertex to itself, the strength is defined as ∞ (as it imposes no bottleneck).

Intuition and Dynamic Programming Approach

We can adapt the **Floyd-Warshall algorithm**, which traditionally solves the All-Pairs Shortest Path problem, to solve the All-Pairs Strongest Path problem.

In the standard Floyd-Warshall algorithm, we find the shortest path by minimizing the sum of edge weights. In our modified version, we want to *maximize* the *minimum* edge weight. Let $S_{i,j}^{(k)}$ be the maximum strength of all paths from vertex i to vertex j whose intermediate vertices all belong to the set $\{1, 2, \dots, k\}$.

When considering whether to include vertex k as an intermediate vertex on the strongest path from i to j , we have two choices:

- (a) **Do not use vertex k :** The strongest path only uses intermediate vertices from $\{1, \dots, k-1\}$. The strength is $S_{i,j}^{(k-1)}$.
- (b) **Use vertex k :** The path goes from i to k , and then from k to j . The strength of this combined path is the bottleneck of the two subpaths: $\min(S_{i,k}^{(k-1)}, S_{k,j}^{(k-1)})$.

To find the optimal strength, we take the maximum of these two options. This yields the following recurrence relation:

$$S_{i,j}^{(k)} = \max\left(S_{i,j}^{(k-1)}, \min\left(S_{i,k}^{(k-1)}, S_{k,j}^{(k-1)}\right)\right)$$

Base Cases ($k = 0$):

$$S_{i,j}^{(0)} = \begin{cases} \infty & \text{if } i = j \\ w(i, j) & \text{if } (i, j) \in E \\ -\infty & \text{if } i \neq j \text{ and } (i, j) \notin E \end{cases}$$

Algorithm

Just like the standard Floyd-Warshall algorithm, we can optimize the space complexity from $\mathcal{O}(|V|^3)$ to $\mathcal{O}(|V|^2)$ by dropping the k -dimension in our DP table, updating the matrix in place.

Algorithm 29 All-Pairs Strongest Path

- 1: **Input:** A graph $G = (V, E)$ represented by an adjacency matrix W of size $n \times n$ where $W[i][j] = w(i, j)$.
- 2: **Output:** A matrix S of size $n \times n$ where $S[i][j]$ is the strength of the strongest path from i to j .

3: Initialization:

4: Let S be a new $n \times n$ matrix.

5: **for** $i = 1$ to n **do**

6: **for** $j = 1$ to n **do**

7: **if** $i == j$ **then**

8: $S[i][j] \leftarrow \infty$

9: **else if** edge (i, j) exists **then**

10: $S[i][j] \leftarrow W[i][j]$

11: **else**

12: $S[i][j] \leftarrow -\infty$

13: **end if**

14: **end for**

15: **end for**

16: Dynamic Programming Computation:

17: **for** $k = 1$ to n **do**

18: **for** $i = 1$ to n **do**

19: **for** $j = 1$ to n **do**

20: $S[i][j] \leftarrow \max(S[i][j], \min(S[i][k], S[k][j]))$

21: **end for**

22: **end for**

23: **end for**

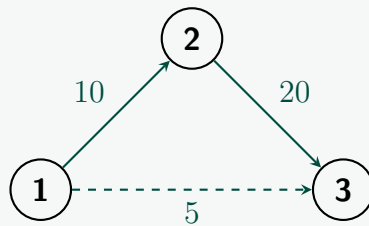
24: **return** S

Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|V|^3)$. The algorithm consists of three tightly nested loops, each iterating $|V|$ times. The operations inside the innermost loop (min and max) execute in $\mathcal{O}(1)$ time.
- **Space Complexity:** $\mathcal{O}(|V|^2)$. We maintain a 2-dimensional DP matrix S of size $|V| \times |V|$ to store the path strengths. By dropping the k -th dimension from the recurrence, we avoid $\mathcal{O}(|V|^3)$ space.

Visualization of the Recurrence Transition

Consider the small graph below to illustrate why taking the maximum of minimums successfully identifies the strongest path.



Suppose we are evaluating the strongest path from vertex 1 to vertex 3.

- Initially ($k = 0$), the only known path is the direct edge: $S_{1,3}^{(0)} = 5$.
- When vertex 2 is introduced as a possible intermediate vertex ($k = 2$), the algorithm evaluates the path $1 \rightarrow 2 \rightarrow 3$.
- The strength of the path through vertex 2 is the bottleneck: $\min(w(1, 2), w(2, 3)) = \min(10, 20) = 10$.
- The algorithm updates the matrix: $S_{1,3} \leftarrow \max(5, 10) = 10$.

Thus, the algorithm correctly identifies that taking the indirect route yields a strictly stronger bottleneck capacity.

Q10. Given an undirected graph $G = (V, E)$, give a linear-time algorithm to check whether there is a cycle of odd length in the graph. Briefly justify its correctness. **(15 marks)** [CSE 207 | L-2/T-2 | 2015–16]

Answer

Theoretical Foundation and Graph Property

To design a linear-time algorithm for detecting an odd-length cycle, we leverage a fundamental theorem from graph theory:

*An undirected graph G contains an odd cycle if and only if it is **not bipartite**.*

A graph is bipartite if its vertices can be partitioned into two disjoint sets, say U and W , such that every edge connects a vertex in U to a vertex in W . If we attempt to color a graph using two colors (e.g., Red and Blue) such that no two adjacent vertices share the same color, a successful 2-coloring proves the graph is bipartite (hence, no odd cycles). If a conflict arises where an edge connects two vertices of the same color, an odd cycle is guaranteed to exist.

Algorithmic Strategy

We can check for bipartiteness in linear time $\mathcal{O}(|V| + |E|)$ using a Breadth-First Search (BFS) or Depth-First Search (DFS) traversal. The algorithm works as follows:

- Initialize all vertices as uncolored.
- Loop through all vertices to handle disconnected components. For each uncolored vertex, initiate a BFS.
- Assign the starting vertex a default color (e.g., 0).
- During the BFS traversal, for each vertex u being processed, examine all its neighbors v :
 - If v is uncolored, assign it the opposite color of u (i.e., $\text{color}[v] = 1 - \text{color}[u]$) and enqueue it.
 - If v is already colored and has the same color as u ($\text{color}[v] == \text{color}[u]$), a cross-edge within the same layer or between identical color states has been found. This indicates the presence of an odd-length cycle. Immediately terminate and return **True**.
- If the entire graph is traversed without encountering a color conflict, return **False**.

Algorithm

Algorithm 30 Odd Cycle Detection via 2-Coloring

```

1: Input: An undirected graph  $G = (V, E)$  in adjacency list representation.
2: Output: True if  $G$  contains an odd cycle, False otherwise.

3: Initialization:
4: Let  $color$  be an array of size  $|V|$ , initialized to  $-1$  (representing uncolored) for all vertices.

5: for all vertex  $s \in V$  do
6:   if  $color[s] == -1$  then
7:      $color[s] \leftarrow 0$  ▷ Color the root of a new component with 0
8:     Create an empty queue  $Q$ 
9:     Enqueue  $s$  onto  $Q$ 

10:    while  $Q$  is not empty do
11:      Dequeue vertex  $u$  from  $Q$ 
12:      for all neighbor  $v \in Adj[u]$  do
13:        if  $color[v] == -1$  then
14:           $color[v] \leftarrow 1 - color[u]$  ▷ Assign alternate color
15:          Enqueue  $v$  onto  $Q$ 
16:        else if  $color[v] == color[u]$  then
17:          return True ▷ Odd cycle detected
18:        end if
19:      end for
20:    end while
21:  end if
22: end for
23: return False ▷ Graph is bipartite; no odd cycle exists

```

Justification of Correctness

We must prove both directions of the implication:

- (a) **If the algorithm returns True, an odd cycle exists:** Suppose the algorithm finds an edge (u, v) where $color[u] == color[v]$. Let s be the lowest common ancestor (LCA) of u and v in the BFS tree root. The tree path from s to u has length L_1 , and the tree path from s to v has length L_2 . Because the coloring alternates at every level of the BFS tree, the color of any node x is determined by the parity of its distance from s . Specifically:

$$color[u] \equiv L_1 \pmod{2} \quad \text{and} \quad color[v] \equiv L_2 \pmod{2}$$

Since $color[u] == color[v]$, it must be that L_1 and L_2 have the same parity ($L_1 \equiv L_2 \pmod{2}$). The cycle formed by the tree path $s \rightsquigarrow u$, the cross-edge (u, v) , and the reverse tree path $v \rightsquigarrow s$ has a total length of:

$$\text{Length} = L_1 + L_2 + 1$$

Since L_1 and L_2 share the same parity, their sum $L_1 + L_2$ is guaranteed to be even. Adding the 1 from the cross-edge makes the total length $L_1 + L_2 + 1$ strictly **odd**.

- (b) **If an odd cycle exists, the algorithm will return True:** Assume for contradiction that the graph contains an odd cycle $C = \langle v_1, v_2, \dots, v_k, v_1 \rangle$ (where k is odd), but the algorithm returns **False** (meaning it successfully 2-colors the graph). As we traverse the cycle, the colors must alternate:

$$color[v_2] = 1 - color[v_1], \quad color[v_3] = color[v_1], \quad \dots$$

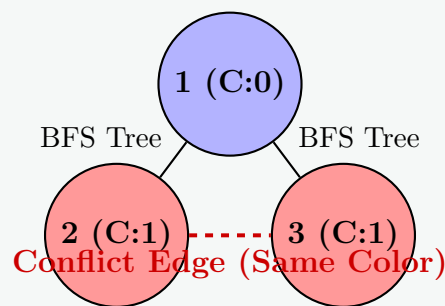
By induction, for any vertex v_i in the cycle, $color[v_i] = color[v_1]$ if i is odd, and $color[v_i] \neq color[v_1]$ if i is even. Since k is odd, $color[v_k] = color[v_1]$. However, there is an edge connecting v_k and v_1 to close the cycle. This means adjacent vertices v_k and v_1 share the same color, which causes a direct conflict. The algorithm would inevitably catch this conflict and return **True**, contradicting our assumption.

Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|V| + |E|)$. Each vertex is enqueued and dequeued at most once due to the color check array. For every vertex examined, we iterate through its adjacency list. Across the entire execution, every edge is examined exactly twice (once from each endpoint). Thus, the total runtime is strictly linear.
- **Space Complexity:** $\mathcal{O}(|V|)$ auxiliary space. This is required for the color array and the BFS queue Q , which both scale linearly with the number of vertices.

Visualizing a Color Conflict (Odd Cycle Found)

The diagram below showcases a 3-cycle (triangle) where a color conflict is detected upon evaluating the cross-edge between vertex 2 and vertex 3.



Q11. Suppose a CS curriculum consists of n mandatory courses. The prerequisite directed graph G has a node for each course and an edge from v to w if v is a prerequisite for w . Give a linear-time algorithm that computes the minimum number of semesters necessary to complete the curriculum (a student may take any number of courses per semester). **(15 marks)** [CSE 207 | L-2/T-2 | 2015–16]

Answer

Key Insight. The minimum number of semesters equals the length of the *longest path* in the prerequisite DAG G . A course can be taken in semester k only after all its prerequisites are completed, so the earliest semester for a course is determined by the longest chain of prerequisites leading to it.

Observation. Since prerequisites cannot be circular (a student must actually be able to finish), G must be a DAG (Directed Acyclic Graph). The minimum number of semesters is:

$$\text{Answer} = \max_{v \in V} d(v)$$

where $d(v)$ denotes the length of the longest path ending at v (i.e., the earliest semester in which v can be taken), counting from semester 1.

Recurrence. Define $d(v)$ as the semester in which course v is first available:

$$d(v) = \begin{cases} 1 & \text{if } v \text{ has no prerequisites (in-degree 0)} \\ 1 + \max_{(u,v) \in E} d(u) & \text{otherwise} \end{cases}$$

Algorithm. We compute $d(v)$ for all nodes using **Kahn's topological sort** (BFS-based), processing nodes in topological order so that when we process v , all predecessors u with edge (u, v) have already been assigned $d(u)$.

Algorithm 31 Minimum Semesters via Longest Path in DAG

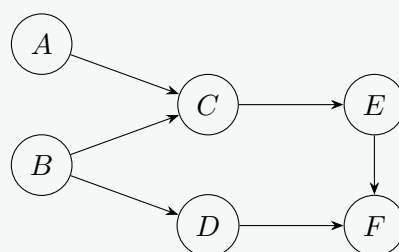
Require: Directed acyclic graph $G = (V, E)$, $|V| = n$

Ensure: Minimum number of semesters S

```

1: Compute  $\text{indegree}[v]$  for all  $v \in V$ 
2: Initialize  $d[v] \leftarrow 0$  for all  $v \in V$ 
3: Initialize queue  $Q \leftarrow \emptyset$ 
4: for each  $v \in V$  do
5:   if  $\text{indegree}[v] = 0$  then
6:      $d[v] \leftarrow 1$ 
7:      $Q.\text{enqueue}(v)$ 
8:   end if
9: end for
10:  $S \leftarrow 1$ 
11: while  $Q \neq \emptyset$  do
12:    $u \leftarrow Q.\text{dequeue}()$ 
13:    $S \leftarrow \max(S, d[u])$ 
14:   for each neighbor  $w$  such that  $(u, w) \in E$  do
15:      $d[w] \leftarrow \max(d[w], d[u] + 1)$ 
16:      $\text{indegree}[w] \leftarrow \text{indegree}[w] - 1$ 
17:     if  $\text{indegree}[w] = 0$  then
18:        $Q.\text{enqueue}(w)$ 
19:     end if
20:   end for
21: end while
22: return  $S$ 
```

Illustrative Example. Consider 6 courses with the following prerequisites:



Computing d values:

Course	Prerequisites	Semester d
A	none	1
B	none	1
C	A, B	$\max(d[A], d[B]) + 1 = 2$
D	B	$d[B] + 1 = 2$
E	C	$d[C] + 1 = 3$
F	D, E	$\max(d[D], d[E]) + 1 = 4$

The minimum number of semesters is $\max\{1, 1, 2, 2, 3, 4\} = \boxed{4}$.

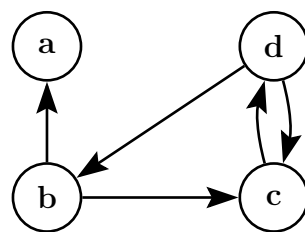
Correctness.

- *Lower bound:* Any chain of k prerequisite-dependent courses requires at least k semesters, so the answer is at least the longest path length.
- *Achievability:* By taking all courses with $d[v] = k$ in semester k , the schedule is valid since every prerequisite u of v satisfies $d[u] < d[v]$, meaning u is taken in an earlier semester. Hence the answer equals the longest path.

Complexity Analysis.

- **Time:** $O(V + E)$. Computing in-degrees takes $O(V + E)$. Each node is enqueued and dequeued once: $O(V)$. Each edge is relaxed exactly once: $O(E)$. Total: $O(V + E)$ — linear in the size of the graph.
- **Space:** $O(V + E)$ for the adjacency list, in-degree array, d array, and queue.

Q12. Find the transitive closure. Show the matrices computed at each step. What is the cost to find the transitive closure? **(10 marks)**
[CSE 207 | L-2/T-2 | 2014–15]



Answer

Method: Warshall's Algorithm. We compute the transitive closure using the recurrence:

$$t_{ij}^{(k)} = t_{ij}^{(k-1)} \vee (t_{ik}^{(k-1)} \wedge t_{kj}^{(k-1)})$$

Node ordering: $1 = a, 2 = b, 3 = c, 4 = d$.

Initial Matrix $T^{(0)}$ (direct edges only, diagonal = 1):

From the graph: $b \rightarrow a, b \rightarrow c, d \rightarrow b, d \rightarrow c, c \rightarrow d$.

$$T^{(0)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{pmatrix} \end{matrix}$$

Step $k = 1$ (intermediate vertex: a). Check: can any pair (i, j) be connected via a ? Since column a in $T^{(0)}$ has $t_{ia} = 1$ only for $i = a$ and $i = b$, and row a has $t_{aj} = 1$ only for $j = a$, no new entries are added.

$$T^{(1)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{pmatrix} \end{matrix}$$

Step $k = 2$ (intermediate vertex: b). For each (i, j) : if $t_{ib}^{(1)} = 1$ and $t_{bj}^{(1)} = 1$, set $t_{ij} = 1$.

Row b of $T^{(1)}$: $(a, b, c, d) = (1, 1, 1, 0)$. Who reaches b ? Column b : d has $t_{db} = 1$.

New connections via b :

- $d \rightarrow b \rightarrow a$: set $t_{da} = 1$
- $d \rightarrow b \rightarrow c$: already 1

$$T^{(2)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} \end{matrix}$$

Step $k = 3$ (intermediate vertex: c). Row c of $T^{(2)}$: $(a, b, c, d) = (0, 0, 1, 1)$. Who reaches c ? Column c : b, c, d all have $t_{ic} = 1$. New connections via c :

- $b \rightarrow c \rightarrow d$: set $t_{bd} = 1$
- $d \rightarrow c \rightarrow d$: already 1

$$T^{(3)} = \begin{matrix} & a & b & c & d \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & \mathbf{1} \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} \end{matrix}$$

Step $k = 4$ (intermediate vertex: d). Row d of $T^{(3)}$: $(a, b, c, d) = (1, 1, 1, 1)$. Who reaches d ? Column d : b, c, d all have $t_{id} = 1$. New connections via d :

- $b \rightarrow d \rightarrow a$: already 1
- $b \rightarrow d \rightarrow b$: already 1
- $b \rightarrow d \rightarrow c$: already 1
- $c \rightarrow d \rightarrow a$: set $t_{ca} = 1$
- $c \rightarrow d \rightarrow b$: set $t_{cb} = 1$
- $c \rightarrow d \rightarrow c$: already 1

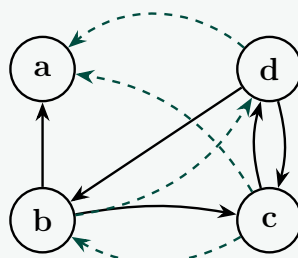
$$T^{(4)} = \begin{matrix} & a & b & c & d \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ \mathbf{1} & \mathbf{1} & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} \end{matrix}$$

Final Transitive Closure $T^* = T^{(4)}$:

$$T^* = \begin{matrix} & a & b & c & d \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} \end{matrix}$$

Interpretation: a can only reach itself. b, c , and d can all reach every node (including a), forming a strongly connected component among $\{b, c, d\}$ with a reachable from all three.

Verification via Graph:



Solid edges: original. Dashed blue edges: newly inferred reachability.

Cost Analysis.

Warshall's algorithm runs three nested loops over n vertices:

$$\text{Time Complexity} = O(n^3)$$

For this graph $n = 4$, so the total number of operations is at most $4^3 = 64$ boolean updates.

Space Complexity = $O(n^2)$ for storing the $n \times n$ boolean matrix (updated in-place across n steps).

Q13. Let C and C' be distinct strongly connected components in a directed graph $G = (V, E)$. Suppose there is an edge $(u, v) \in E$ where $u \in C$ and $v \in C'$. Prove that $f(C) > f(C')$, where $f(U)$ denotes the latest finishing time of any vertex in U . **(10 marks)** [CSE 207 | L-2/T-2 | 2014–15]

Answer

Theorem. Let C and C' be distinct SCCs of G , and let $(u, v) \in E$ with $u \in C$, $v \in C'$. Then $f(C) > f(C')$, where $f(U) = \max_{x \in U} f(x)$.

Preliminary Definitions.

- $d(x)$, $f(x)$: discovery and finishing times of vertex x in DFS on G .
- $f(U) = \max_{x \in U} f(x)$ for any set $U \subseteq V$.
- Since $C \neq C'$ are distinct SCCs and $(u, v) \in E$, there can be *no* edge from any vertex of C' back to any vertex of C (otherwise C and C' would merge into one SCC).

Proof. Let $x \in C$ be the first vertex in C discovered by DFS, i.e., $d(x) = \min_{w \in C} d(w)$. We consider two cases based on whether x is discovered before or after all vertices of C' .

Case 1: x is discovered before every vertex of C' .

At time $d(x)$, no vertex of C' has been discovered yet. Since $x \in C$ and $u \in C$, there is a path $x \rightsquigarrow u$ within C (by strong connectivity of C). From u there is the edge (u, v) with $v \in C'$, and from v all vertices of C' are reachable within C' .

Thus, from x , every vertex of C' is reachable in G . By the **White Path Theorem**, since all vertices of C' are white at time $d(x)$ and reachable from x via a white path, every vertex of C' becomes a descendant of x in the DFS forest. Therefore:

$$\forall z \in C', \quad f(z) < f(x).$$

Since $x \in C$, we have $f(x) \leq f(C)$, and so:

$$f(C') = \max_{z \in C'} f(z) < f(x) \leq f(C).$$

Case 2: Some vertex of C' is discovered before x (i.e., before any vertex of C).

Let $y \in C'$ be the first vertex of C' discovered, with $d(y) < d(x)$. At time $d(y)$, every vertex of C is still white (since x is the earliest-discovered vertex of C and $d(y) < d(x)$).

We claim y cannot reach any vertex of C from within the DFS subtree rooted at y . Suppose for contradiction that some $w \in C$ is a descendant of y . Then there is a path $y \rightsquigarrow w$ in G . Since $w \in C$ and C is strongly connected, there is also a path $w \rightsquigarrow u \rightarrow v$ and a path within C' from v back to y — but this would imply C and C' are in the same SCC, contradicting $C \neq C'$.

More directly: since there is no path from C' to C in G (as argued above), y cannot reach any vertex of C in G . Therefore no vertex of C is a descendant of y , which means x is discovered *after* y finishes:

$$f(y) < d(x).$$

Once x is discovered, the entire SCC C is explored (by strong connectivity and the White Path Theorem applied within C), and since there is no back-path from C' to C , the DFS subtree of x does not revisit C' . Hence:

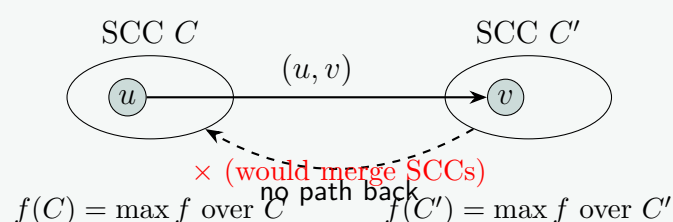
$$f(y) < d(x) < f(x) \leq f(C).$$

Since $y \in C'$, $f(C') \geq f(y)$. But we need $f(C') < f(C)$. Since $d(y) < d(x)$ and no vertex of C is reachable from C' , all vertices of C' finish before $d(x)$, giving:

$$f(C') < d(x) \leq f(C).$$

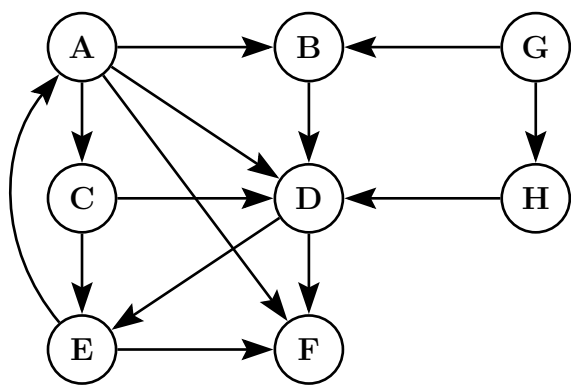
Conclusion. In both cases:

$$f(C) > f(C')$$

Intuition via Diagram.

The existence of (u, v) with no return path forces C' to finish entirely before the latest finishing time in C , giving $f(C) > f(C')$. This is the key lemma used in Kosaraju's and Tarjan's SCC algorithms to process SCCs in reverse topological order.

Q14. Find the different types of edges if DFS is applied from vertex A . Discuss the edges' properties with respect to the graph. **(6 marks)**
[CSE 207 | L-2/T-2 | 2014–15]



Answer

DFS Edge Classification. In a DFS on a directed graph, every edge (u, v) falls into one of four categories based on the discovery/finish timestamps $[d(u), f(u)]$ and $[d(v), f(v)]$:

Type	Condition	Meaning
Tree Edge	v first discovered via (u, v)	v is DFS-child of u
Back Edge	$d(v) < d(u)$ and v is ancestor of u	creates a cycle
Forward Edge	$d(v) > d(u)$, v already finished, descendant	non-tree shortcut
Cross Edge	$d(v) < d(u)$, v already finished, no ancestor relation	across branches/trees

DFS Execution from A. We run DFS starting at A, visiting neighbours in the order they appear: B, C, F, D for A; D for B; D, E for C; E for D; F for E; etc. G and H are unreachable from A so they start a new DFS tree when the outer loop reaches them.

Tracking timestamps $[d, f]$:

Step	Event	Vertex	Time
1	Discover A	A	$d(A) = 1$
2	Discover B	B	$d(B) = 2$
3	Discover D via B	D	$d(D) = 3$
4	Discover E via D	E	$d(E) = 4$
5	$E \rightarrow A$: A is gray (ancestor)		
6	Discover F via E	F	$d(F) = 5$
7	F has no unvisited neighbours	F	$f(F) = 6$
8	Finish E	E	$f(E) = 7$
9	$D \rightarrow F$: F already finished		
10	Finish D	D	$f(D) = 8$
11	Finish B	B	$f(B) = 9$
12	Discover C via A	C	$d(C) = 10$
13	$C \rightarrow D$: D already finished		
14	$C \rightarrow E$: E already finished		
15	Finish C	C	$f(C) = 11$
16	$A \rightarrow F$: F already finished		
17	$A \rightarrow D$: D already finished		
18	Finish A	A	$f(A) = 12$
19	Discover G (new tree)	G	$d(G) = 13$
20	Discover H via G	H	$d(H) = 14$
21	$H \rightarrow D$: D already finished		
22	Finish H	H	$f(H) = 15$
23	$G \rightarrow B$: B already finished		
24	Finish G	G	$f(G) = 16$

DFS Forest:

Edge Classification Table:

Edge	Type	Timestamps	Reason
$A \rightarrow B$	Tree	[1, 12], [2, 9]	B first discovered via A
$A \rightarrow C$	Tree	[1, 12], [10, 11]	C first discovered via A
$A \rightarrow F$	Forward	[1, 12], [5, 6]	F finished, descendant of A
$A \rightarrow D$	Forward	[1, 12], [3, 8]	D finished, descendant of A
$B \rightarrow D$	Tree	[2, 9], [3, 8]	D first discovered via B
$C \rightarrow D$	Cross	[10, 11], [3, 8]	D finished, not ancestor of C
$C \rightarrow E$	Cross	[10, 11], [4, 7]	E finished, not ancestor of C
$D \rightarrow E$	Tree	[3, 8], [4, 7]	E first discovered via D
$D \rightarrow F$	Forward	[3, 8], [5, 6]	F finished, descendant of D
$E \rightarrow A$	Back	[4, 7], [1, 12]	A is gray ancestor of E
$E \rightarrow F$	Tree	[4, 7], [5, 6]	F first discovered via E
$G \rightarrow H$	Tree	[13, 16], [14, 15]	H first discovered via G
$G \rightarrow B$	Cross	[13, 16], [2, 9]	B finished, different DFS tree
$H \rightarrow D$	Cross	[14, 15], [3, 8]	D finished, different DFS tree

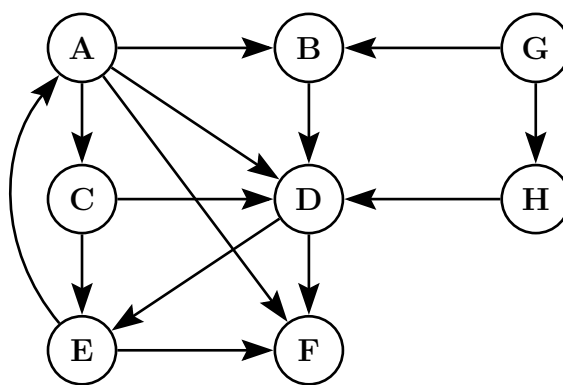
Summary Count:

- **Tree Edges** (6): $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow D$, $D \rightarrow E$, $E \rightarrow F$, $G \rightarrow H$
- **Back Edges** (1): $E \rightarrow A$
- **Forward Edges** (3): $A \rightarrow F$, $A \rightarrow D$, $D \rightarrow F$
- **Cross Edges** (4): $C \rightarrow D$, $C \rightarrow E$, $G \rightarrow B$, $H \rightarrow D$

Properties Discussion.

- **Tree edges** form the DFS forest. Together they span all reachable vertices from each DFS source.
- **Back edge** $E \rightarrow A$ indicates a *directed cycle* $A \rightarrow B \rightarrow D \rightarrow E \rightarrow A$ in the graph. In general, a directed graph contains a cycle *if and only if* DFS produces at least one back edge.
- **Forward edges** $A \rightarrow F$, $A \rightarrow D$, $D \rightarrow F$ are non-tree edges pointing from an ancestor to a proper descendant. They only appear in directed graphs (never in undirected DFS).
- **Cross edges** $C \rightarrow D$, $C \rightarrow E$, $G \rightarrow B$, $H \rightarrow D$ connect vertices with no ancestor-descendant relationship — either across branches of the same DFS tree or across different DFS trees (as with $G \rightarrow B$ and $H \rightarrow D$). Cross edges also only occur in directed graphs.
- **Interval nesting property:** For any tree, forward, or back edge (u, v) , the intervals $[d(u), f(u)]$ and $[d(v), f(v)]$ are *nested*. For cross edges they are *disjoint* with $f(v) < d(u)$.

Q15. Show the topological sorting order of a given graph with explanation of discovery and finishing times of node exploration. **(6 marks)**
[CSE 207 | L-2/T-2 | 2014–15]

**Answer**

Important Observation. The edge $E \rightarrow A$ creates a directed cycle $A \rightarrow B \rightarrow D \rightarrow E \rightarrow A$. A topological sort is only defined for DAGs (no cycles). Therefore we **note this cycle** and proceed with topological sort on the remaining structure, treating $E \rightarrow A$ as the back edge that prevents a full linear order. In an exam context we show the DFS timestamps for all vertices and list the topological order by decreasing finish time, noting that the cycle means a strict total order does not exist — but the DFS-based ordering still yields the correct relative ordering for all acyclic portions.

Algorithm. Run DFS; output each vertex to the *front* of a linked list upon finishing. The resulting list is the topological order (vertices with higher finish times come first).

DFS Timestamps. Starting from A , visiting adjacency lists in order B, C, F, D for A ; D for B ; D, E for C ; E for D ; A, F for E ; then G as a new source:

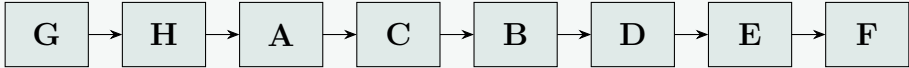
Order	Event	Vertex	<i>d</i>	<i>f</i>
1	Discover <i>A</i>	<i>A</i>	1	—
2	Discover <i>B</i> (via <i>A</i>)	<i>B</i>	2	—
3	Discover <i>D</i> (via <i>B</i>)	<i>D</i>	3	—
4	Discover <i>E</i> (via <i>D</i>)	<i>E</i>	4	—
5	<i>E</i> → <i>A</i> : <i>A</i> is gray ⇒ Back Edge / Cycle			
6	Discover <i>F</i> (via <i>E</i>)	<i>F</i>	5	—
7	<i>F</i> has no unvisited neighbours	<i>F</i>	5	6
8	Finish <i>E</i>	<i>E</i>	4	7
9	<i>D</i> → <i>F</i> : already finished			
10	Finish <i>D</i>	<i>D</i>	3	8
11	Finish <i>B</i>	<i>B</i>	2	9
12	Discover <i>C</i> (via <i>A</i>)	<i>C</i>	10	—
13	<i>C</i> → <i>D</i> : already finished (Cross)			
14	<i>C</i> → <i>E</i> : already finished (Cross)			
15	Finish <i>C</i>	<i>C</i>	10	11
16	<i>A</i> → <i>F</i> : already finished			
17	<i>A</i> → <i>D</i> : already finished			
18	Finish <i>A</i>	<i>A</i>	1	12
19	Discover <i>G</i> (new DFS tree)	<i>G</i>	13	—
20	Discover <i>H</i> (via <i>G</i>)	<i>H</i>	14	—
21	<i>H</i> → <i>D</i> : already finished (Cross)			
22	Finish <i>H</i>	<i>H</i>	14	15
23	<i>G</i> → <i>B</i> : already finished (Cross)			
24	Finish <i>G</i>	<i>G</i>	13	16

Timestamp Summary:

Vertex	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>	<i>G</i>	<i>H</i>
<i>d</i>	1	2	10	3	4	5	13	14
<i>f</i>	12	9	11	8	7	6	16	15

Topological Order (sort by decreasing finish time):

$G\ (16) \rightarrow H\ (15) \rightarrow A\ (12) \rightarrow C\ (11) \rightarrow B\ (9) \rightarrow D\ (8) \rightarrow E\ (7) \rightarrow F\ (6)$

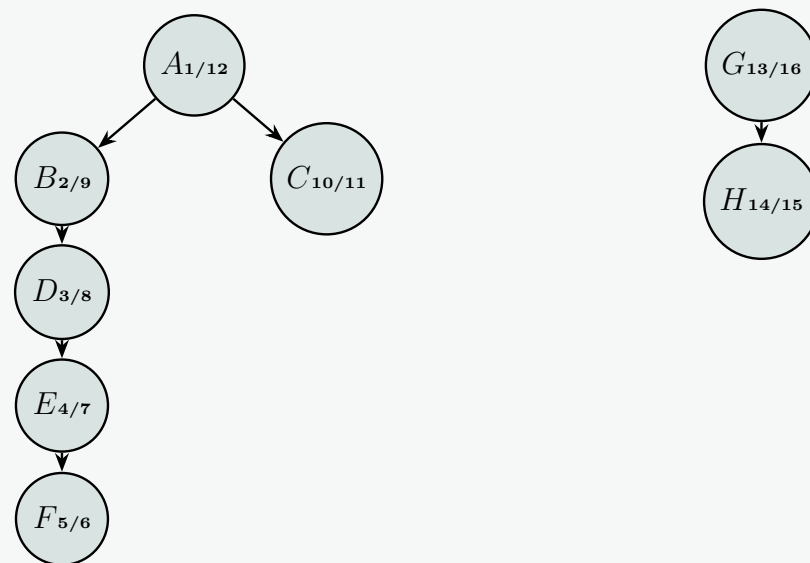


Verification. We check that every original edge (excluding the back edge $E \rightarrow A$) goes *left to right* in this ordering:

Edge	From	To	Order satisfied?
$A \rightarrow B$	pos 3	pos 5	✓
$A \rightarrow C$	pos 3	pos 4	✓
$A \rightarrow D$	pos 3	pos 6	✓
$A \rightarrow F$	pos 3	pos 8	✓
$B \rightarrow D$	pos 5	pos 6	✓
$C \rightarrow D$	pos 4	pos 6	✓
$C \rightarrow E$	pos 4	pos 7	✓
$D \rightarrow E$	pos 6	pos 7	✓
$D \rightarrow F$	pos 6	pos 8	✓
$E \rightarrow F$	pos 7	pos 8	✓
$G \rightarrow B$	pos 1	pos 5	✓
$G \rightarrow H$	pos 1	pos 2	✓
$H \rightarrow D$	pos 2	pos 6	✓
$E \rightarrow A$	pos 7	pos 3	× Back edge — cycle, violates DAG

All edges except the back edge $E \rightarrow A$ are satisfied. The DFS-based topological order is valid for the acyclic portion of the graph.

DFS Tree with Timestamps:

**Key Properties:**

- **Topological sort by DFS:** A vertex is prepended to the output list when it *finishes*. This guarantees that for any edge $u \rightarrow v$ in a DAG, u appears before v , since $f(u) > f(v)$.
- **Cycle detection:** The back edge $E \rightarrow A$ (gray ancestor encountered) reveals the cycle $A \rightarrow B \rightarrow D \rightarrow E \rightarrow A$. Any graph with a back edge in DFS is *not* a DAG and has no valid total topological order.
- **Multiple DFS trees:** G and H are unreachable from A , so they form a separate DFS tree. The topological order still places G and H before all vertices reachable from G (i.e., before B, D, H).
- **Time complexity:** $O(V + E)$ — each vertex and edge is processed exactly once.

Q16. Write the Bellman-Ford algorithm. Analyze the time complexity and correctness of the Bellman-Ford algorithm. (20 marks) [CSE 207 | L-2/T-2 | 2012–13]

Answer**Introduction**

The Bellman-Ford algorithm computes single-source shortest paths in a directed, edge-weighted graph $G = (V, E)$. Unlike Dijkstra's algorithm, Bellman-Ford can handle graphs containing **negative edge weights**. Furthermore, it is capable of detecting the presence of **negative-weight cycles** reachable from the source vertex. If such a cycle exists, shortest paths are undefined because one could infinitely traverse the cycle to achieve an arbitrarily small (negative) path weight.

The Bellman-Ford Algorithm

The algorithm operates by repeatedly relaxing all edges of the graph. It performs exactly $|V| - 1$ iterations of this relaxation process, followed by a final pass over all edges to check for negative-weight cycles.

Algorithm 32 Bellman-Ford Algorithm

```

1: Input: A directed graph  $G = (V, E)$ , a weight function  $w : E \rightarrow \mathbb{R}$ , and a source vertex  $s \in V$ .
2: Output: Arrays  $d$  (shortest path estimates) and  $\pi$  (predecessors), or an indication that a negative-weight cycle exists.

// Step 1: Initialize single-source tracking
3: for each vertex  $v \in V$  do
4:    $d[v] \leftarrow \infty$ 
5:    $\pi[v] \leftarrow \text{NIL}$ 
6: end for
7:  $d[s] \leftarrow 0$ 

// Step 2: Relax edges repeatedly
8: for  $i = 1$  to  $|V| - 1$  do
9:   for each edge  $(u, v) \in E$  do
10:    if  $d[u] \neq \infty$  and  $d[u] + w(u, v) < d[v]$  then
11:       $d[v] \leftarrow d[u] + w(u, v)$ 
12:       $\pi[v] \leftarrow u$ 
13:    end if
14:   end for
15: end for

// Step 3: Check for negative-weight cycles
16: for each edge  $(u, v) \in E$  do
17:   if  $d[u] \neq \infty$  and  $d[u] + w(u, v) < d[v]$  then
18:     return False ▷ Negative-weight cycle detected
19:   end if
20: end for

21: return True,  $d$ ,  $\pi$  ▷ Shortest paths successfully computed

```

Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|V| \cdot |E|)$
 - **Initialization (Step 1):** Takes $\mathcal{O}(|V|)$ time since it visits each vertex exactly once to assign initial values.
 - **Relaxation Phase (Step 2):** The outer loop runs $|V| - 1$ times. The inner loop examines every single edge $(u, v) \in E$ exactly once per outer iteration. Since edge relaxation takes $\mathcal{O}(1)$ constant time, the total time for this phase is strictly $\mathcal{O}(|V| \cdot |E|)$.
 - **Cycle Detection (Step 3):** Loops through all edges in E exactly once, executing in $\mathcal{O}(|E|)$ time.
 - **Total Time Complexity:** $\mathcal{O}(|V| + |V| \cdot |E| + |E|) = \mathcal{O}(|V| \cdot |E|)$. For a dense graph where $|E| \approx |V|^2$, this equates to $\mathcal{O}(|V|^3)$.
- **Space Complexity:** $\mathcal{O}(|V|)$
 - The algorithm updates the distance estimates d and predecessor values π in place. Both arrays require space proportional to the number of vertices, yielding an auxiliary space complexity of $\mathcal{O}(|V|)$.

Correctness Proof

To prove the correctness of the Bellman-Ford algorithm, we break the analysis down into two primary scenarios: when the graph contains no reachable negative-weight cycles, and when it does.

Case 1: No Reachable Negative-Weight Cycles

Let $\delta(s, v)$ denote the true shortest-path weight from source s to vertex v . We establish the correctness of Step 2 using the **Relaxation Property** and mathematical induction.

Lemma (Triangle/Relaxation Property): For any edge $(u, v) \in E$, we always have $\delta(s, v) \leq \delta(s, u) + w(u, v)$. Furthermore, once $d[v]$ reaches $\delta(s, v)$, it never changes.

Theorem: If G contains no negative-weight cycles reachable from s , then after $|V| - 1$ iterations of Step 2, $d[v] = \delta(s, v)$ for all vertices v reachable from s .

Proof. Let v be an arbitrary vertex reachable from s , and let $p = \langle v_0, v_1, \dots, v_k \rangle$ be a simple shortest path from s to v , where $v_0 = s$ and $v_k = v$. Because a simple path in a graph with no negative-weight cycles contains no repeated vertices, the maximum number of edges in this path is $k \leq |V| - 1$.

We claim by induction on the path index j that after the j -th iteration of the relaxation loop, $d[v_j] = \delta(s, v_j)$.

- **Base Case ($j = 0$):** Before any iterations begin, $d[v_0] = d[s] = 0 = \delta(s, s)$. This holds true throughout the algorithm execution by the relaxation upper-bound property.

- **Inductive Step:** Assume that after the $(j - 1)$ -th iteration, $d[v_{j-1}] = \delta(s, v_{j-1})$. During the j -th iteration, every edge in E is relaxed, which includes the optimal edge (v_{j-1}, v_j) . Upon relaxing this specific edge, the algorithm ensures that:

$$d[v_j] \leq d[v_{j-1}] + w(v_{j-1}, v_j)$$

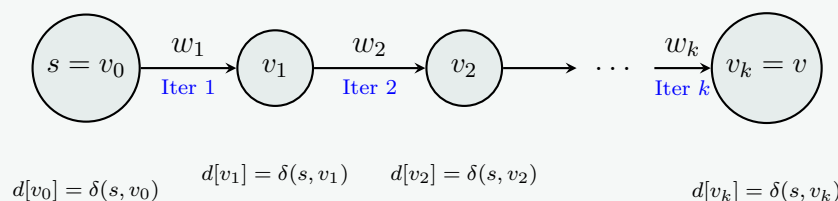
Applying our inductive hypothesis:

$$d[v_j] \leq \delta(s, v_{j-1}) + w(v_{j-1}, v_j) = \delta(s, v_j)$$

Because $d[v_j]$ can never be strictly less than the actual shortest path distance $\delta(s, v_j)$, it must be that $d[v_j] = \delta(s, v_j)$.

By mathematical induction, after k iterations (and thus certainly after $|V| - 1$ iterations), $d[v_k] = d[v] = \delta(s, v)$. $\square$

Visualization of the Inductive Relaxation Chain Along a Shortest Path



Case 2: Detection of Negative-Weight Cycles

We now prove that Step 3 will return **False** if and only if there is a reachable negative-weight cycle.

Proof. Suppose there is a negative-weight cycle $C = \langle v_0, v_1, \dots, v_m \rangle$ reachable from the source s , where $v_0 = v_m$. The total weight of this cycle is strictly negative:

$$\sum_{i=1}^m w(v_{i-1}, v_i) < 0$$

Assume for the sake of contradiction that the algorithm fails to detect this cycle and returns **True**. This implies that the conditional check in Step 3 never triggers, meaning for every edge in the cycle:

$$d[v_i] \leq d[v_{i-1}] + w(v_{i-1}, v_i) \quad \text{for all } i = 1, 2, \dots, m$$

Summing these inequalities around the entire cycle C gives:

$$\sum_{i=1}^m d[v_i] \leq \sum_{i=1}^m d[v_{i-1}] + \sum_{i=1}^m w(v_{i-1}, v_i)$$

Because the cycle is closed ($v_0 = v_m$), the sequence of vertices in $\sum_{i=1}^m d[v_i]$ and $\sum_{i=1}^m d[v_{i-1}]$ are identical; thus, their sums are equal: $\sum_{i=1}^m d[v_i] = \sum_{i=1}^m d[v_{i-1}]$.

Since all vertices are reachable from s , their finite distance estimates cancel out from both sides of the inequality, leaving:

$$0 \leq \sum_{i=1}^m w(v_{i-1}, v_i)$$

This explicitly states that the weight of the cycle is non-negative, which directly contradicts our initial condition that C is a negative-weight cycle. Therefore, our assumption is false: the inequality $d[v_i] > d[v_{i-1}] + w(v_{i-1}, v_i)$ **must** hold true for at least one edge in the cycle, causing Step 3 to correctly identify the cycle and return **False**. $\square$

Q17. Write a linear-time algorithm to compute the strongly connected components of a directed graph $G = (V, E)$ using two depth-first searches. Give an example to show every step of the algorithm. **(8+5 marks)** [CSE 207 | L-2/T-2 | 2012–13]

Answer

Introduction to Kosaraju's Algorithm

A strongly connected component (SCC) of a directed graph $G = (V, E)$ is a maximal set of vertices $C \subseteq V$ such that for every pair of vertices $u, v \in C$, there is a directed path from u to v and a directed path from v to u .

Kosaraju's algorithm computes the SCCs in linear time using two Depth-First Searches (DFS). The algorithm relies on the property that a graph G and its transpose G_R (graph with all edges reversed) have the exact same strongly connected components. By processing vertices in decreasing order of their finishing times from a preliminary DFS, the second DFS on G_R naturally confines itself to one SCC at a time.

The Algorithm

Algorithm 33 Kosaraju's Algorithm for SCCs

```

1: Input: A directed graph  $G = (V, E)$ .
2: Output: A list of Strongly Connected Components (SCCs).

3: procedure KOSARAJU( $G$ )
4:   Initialize an empty stack  $S$ .
5:   Initialize a boolean array visited of size  $|V|$  to False.
  // Step 1: First DFS on  $G$  to determine finishing times
6:   for each vertex  $v \in V$  do
7:     if not visited[ $v$ ] then
8:       DFS_FILLORDER( $G, v, \text{visited}, S$ )
9:     end if
10:  end for

11:  // Step 2: Compute the reversed graph  $G_R$ 
12:   $G_R \leftarrow \text{REVERSEGRAPH}(G)$ 

13:  // Step 3: Second DFS on  $G_R$  based on stack order
14:  Reset visited array to False for all vertices.
15:  Initialize an empty list All_SCCs.
16:  while  $S$  is not empty do
17:     $v \leftarrow S.\text{pop}()$ 
18:    if not visited[ $v$ ] then
19:      Initialize an empty list Current_SCC
20:      DFS_COLLECT( $G_R, v, \text{visited}, \text{Current\_SCC}$ )
21:      Append Current_SCC to All_SCCs
22:    end if
23:  end while
24:  return All_SCCs
25: end procedure

26: procedure DFS_FILLORDER( $G, u, \text{visited}, S$ )
27:   visited[ $u$ ]  $\leftarrow$  True
28:   for each  $v \in \text{Adj}[u]$  do
29:     if not visited[ $v$ ] then
30:       DFS_FILLORDER( $G, v, \text{visited}, S$ )
31:     end if
32:   end for
33:    $S.\text{push}(u)$ 
34: end procedure

35: procedure DFS_COLLECT( $G_R, u, \text{visited}, \text{Current\_SCC}$ )
36:   visited[ $u$ ]  $\leftarrow$  True
37:   Current_SCC.append( $u$ )
38:   for each  $v \in \text{Adj}_R[u]$  do
39:     if not visited[ $v$ ] then
40:       DFS_COLLECT( $G_R, v, \text{visited}, \text{Current\_SCC}$ )
41:     end if
42:   end for
43: end procedure

```

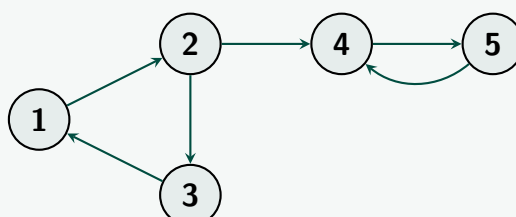
▷ Vertex added to stack only after all its neighbors are visited

Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|V| + |E|)$. The algorithm performs two standard DFS traversals. The first DFS takes $\mathcal{O}(|V| + |E|)$ time. Computing the reversed graph G_R takes $\mathcal{O}(|V| + |E|)$ time (as demonstrated in the earlier reverse graph problem). The second DFS on G_R also takes $\mathcal{O}(|V| + |E|)$ time. The overall time complexity is strictly linear.
- **Space Complexity:** $\mathcal{O}(|V| + |E|)$. We need $\mathcal{O}(|V|)$ auxiliary space for the stack, visited arrays, and the recursion call stack. Storing the reversed graph G_R takes $\mathcal{O}(|V| + |E|)$ space. Thus, the space is also linear.

Step-by-Step Example

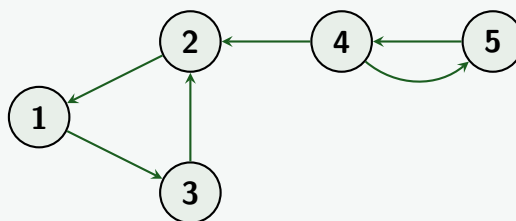
Consider the following directed graph G :



Step 1: First DFS on G to determine Finish Times

- Start DFS at vertex **1**. Visit its neighbor **2**.
- From **2**, visit **3**. From **3**, neighbor **1** is already visited. Finish **3**. Push **3** to Stack.
- Backtrack to **2**. Visit neighbor **4**.
- From **4**, visit **5**. From **5**, neighbor **4** is already visited. Finish **5**. Push **5** to Stack.
- Backtrack to **4**. No unvisited neighbors. Finish **4**. Push **4** to Stack.
- Backtrack to **2**. No unvisited neighbors. Finish **2**. Push **2** to Stack.
- Backtrack to **1**. No unvisited neighbors. Finish **1**. Push **1** to Stack.

Resulting Stack (Top to Bottom): [1, 2, 4, 5, 3]

Step 2: Compute G_R (Reverse all edges)**Step 3: Second DFS on G_R using Stack Order**

- **Pop 1:** It is unvisited. Start DFS from **1** in G_R .
 - Visit **3** (edge $1 \rightarrow 3$ in G_R). From **3**, visit **2**.
 - From **2**, neighbor **1** is visited. DFS finishes.
 - **SCC Found:** {1, 3, 2}
- **Pop 2:** Already visited. Ignore.
- **Pop 4:** It is unvisited. Start DFS from **4** in G_R .
 - Visit **5** (edge $4 \rightarrow 5$ in G_R). From **5**, neighbor **4** is visited. DFS finishes.
 - **SCC Found:** {4, 5}
- **Pop 5:** Already visited. Ignore.
- **Pop 3:** Already visited. Ignore.

Final Result: The algorithm correctly identified two Strongly Connected Components: $C_1 = \{1, 2, 3\}$ and $C_2 = \{4, 5\}$.

Q18. Write an algorithm for DFS traversal of a given graph. Analyze the time complexity of the algorithm. **(10+5=15 marks)** [CSE 203 | L-2/T-1 | 2011–12]

Answer**Algorithm Overview**

Depth-First Search (DFS) is a fundamental graph traversal algorithm. It explores as far as possible along each branch before backtracking. To prevent processing a vertex more than once (and to avoid infinite loops in graphs with cycles), the algorithm maintains a **visited** array.

A complete DFS ensures that all vertices are visited, even if the graph is disconnected. It does this by iterating over all vertices and launching a DFS from any unvisited vertex, generating a *DFS forest*.

The DFS Algorithm

Algorithm 34 Depth-First Search (DFS)

```

1: Input: A graph  $G = (V, E)$  represented by an adjacency list  $\text{Adj}$ .
2: Output: A complete DFS traversal of  $G$ , visiting all vertices.

3: procedure DFS( $G$ )
4:   Initialize an array visited of size  $|V|$  to False.
5:   for each vertex  $u \in V$  do
6:     if not visited $[u]$  then
7:       DFS_VISIT( $G, u, \text{visited}$ )
8:     end if
9:   end for
10: end procedure

11: procedure DFS_VISIT( $G, u, \text{visited}$ )
12:   visited $[u] \leftarrow \text{True}$ 
13:   Process( $u$ ) ▷ Perform desired operations on the vertex
14:   for each vertex  $v \in \text{Adj}[u]$  do
15:     if not visited $[v]$  then
16:       DFS_VISIT( $G, v, \text{visited}$ )
17:     end if
18:   end for
19: end procedure

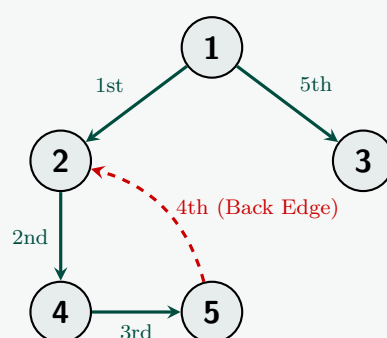
```

Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|V| + |E|)$
 - **Initialization:** The main DFS procedure takes $\mathcal{O}(|V|)$ time to initialize the **visited** array and to iterate through the vertices in the outer **for** loop.
 - **Traversal:** The DFS_VISIT procedure is called exactly once for each vertex $v \in V$ because it is only invoked if **visited** $[v]$ is **False**, and it immediately marks v as **True**.
 - During the execution of DFS_VISIT(u), the inner **for** loop iterates over the adjacency list of u . Over the entire algorithm, the total number of iterations of this inner loop is $\sum_{u \in V} |\text{Adj}[u]|$.
 - For a directed graph, this sum is exactly $|E|$. For an undirected graph, it is $2|E|$. In either case, the time spent traversing edges is $\mathcal{O}(|E|)$.
 - Combining these yields a strict linear time complexity of $\mathcal{O}(|V| + |E|)$.
- **Space Complexity:** $\mathcal{O}(|V|)$
 - **Auxiliary Arrays:** The **visited** array requires $\mathcal{O}(|V|)$ memory.
 - **Call Stack:** The recursive nature of DFS_VISIT consumes call stack memory. In the worst-case scenario (e.g., a graph that forms a single long path), the recursion depth can reach $|V|$. Thus, the space complexity due to the recursion stack is $\mathcal{O}(|V|)$.
 - *Note:* If the graph representation space is included, the adjacency list takes $\mathcal{O}(|V| + |E|)$ space, but auxiliary space remains $\mathcal{O}(|V|)$.

Visualization of DFS Traversal

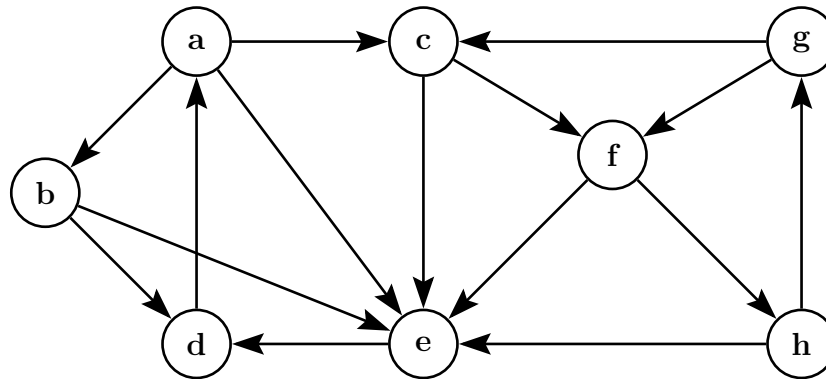
Below is an illustration of a DFS traversal on a simple undirected graph, starting from vertex 1. The solid blue edges represent the *Tree Edges* (edges that lead to unvisited nodes), forming the DFS Tree. The dashed red edge represents a *Back Edge* (an edge leading to an already visited ancestor).

**Execution Trace:**

- (a) Start at **1**. Mark visited.
- (b) Explore neighbor **2**. Mark visited.
- (c) Explore neighbor **4**. Mark visited.

- (d) Explore neighbor **5**. Mark visited.
- (e) From **5**, check neighbor **2**. It is already visited (Back Edge). Backtrack to **4**, then **2**, then **1**.
- (f) From **1**, explore unvisited neighbor **3**. Mark visited. Backtrack. Traversal complete.

Q19. Write the sequences of the vertices if you explore the given graph G using BFS and DFS. Classify the edges of G for your DFS traversal. **(4+6=10 marks)** [CSE 203 | L-2/T-1 | 2011–12]



Answer

Assumptions

- We start the traversals from vertex **a**.
- When a vertex has multiple unvisited neighbors, we visit them in **alphabetical order**.

1. Breadth-First Search (BFS) Traversal

BFS explores the graph level by level, utilizing a queue.

- **Level 0:** Start at **a**. (Queue: [**a**])
- **Level 1:** Pop **a**. Visit its unvisited neighbors: **b, c, e**. (Queue: [**b, c, e**])
- **Level 2:**
 - Pop **b**. Visit unvisited neighbor **d**. (Queue: [**c, e, d**])
 - Pop **c**. Visit unvisited neighbor **f**. (Queue: [**e, d, f**])
 - Pop **e**. Its neighbor **d** is already discovered. (Queue: [**d, f**])
- **Level 3:**
 - Pop **d**. Its neighbor **a** is already discovered. (Queue: [**f**])
 - Pop **f**. Visit unvisited neighbor **h**. (**e** is already discovered). (Queue: [**h**])
- **Level 4:** Pop **h**. Visit unvisited neighbor **g**. (Queue: [**g**])
- **Level 5:** Pop **g**. All its neighbors (**c, f**) are already discovered. (Queue: [])

BFS Sequence: a, b, c, e, d, f, h, g

2. Depth-First Search (DFS) Traversal & Edge Classification

DFS explores as deeply as possible along each branch before backtracking. We record the discovery (d) and finish (f) times to properly classify the edges.

DFS Execution Trace:

- (a) Start at **a**. (Discover **a**: 1)
- (b) Go to **b**. (Discover **b**: 2)
- (c) Go to **d**. (Discover **d**: 3)
- (d) From **d**, neighbor **a** is visited (active). **Backtrack**. (Finish **d**: 4)
- (e) From **b**, go to **e**. (Discover **e**: 5)
- (f) From **e**, neighbor **d** is finished. **Backtrack**. (Finish **e**: 6)
- (g) From **b**, no more neighbors. **Backtrack**. (Finish **b**: 7)
- (h) From **a**, go to **c**. (Discover **c**: 8)
- (i) From **c**, neighbor **e** is finished. Skip. Go to **f**. (Discover **f**: 9)

- (j) From **f**, neighbor **e** is finished. Skip. Go to **h**. (Discover **h**: 10)
- (k) From **h**, neighbor **e** is finished. Skip. Go to **g**. (Discover **g**: 11)
- (l) From **g**, neighbors **c** and **f** are visited (active). **Backtrack**. (Finish **g**: 12)
- (m) From **h**, no more neighbors. **Backtrack**. (Finish **h**: 13)
- (n) From **f**, no more neighbors. **Backtrack**. (Finish **f**: 14)
- (o) From **c**, no more neighbors. **Backtrack**. (Finish **c**: 15)
- (p) From **a**, neighbor **e** is finished. **Backtrack**. (Finish **a**: 16)

DFS Sequence: a, b, d, e, c, f, h, g

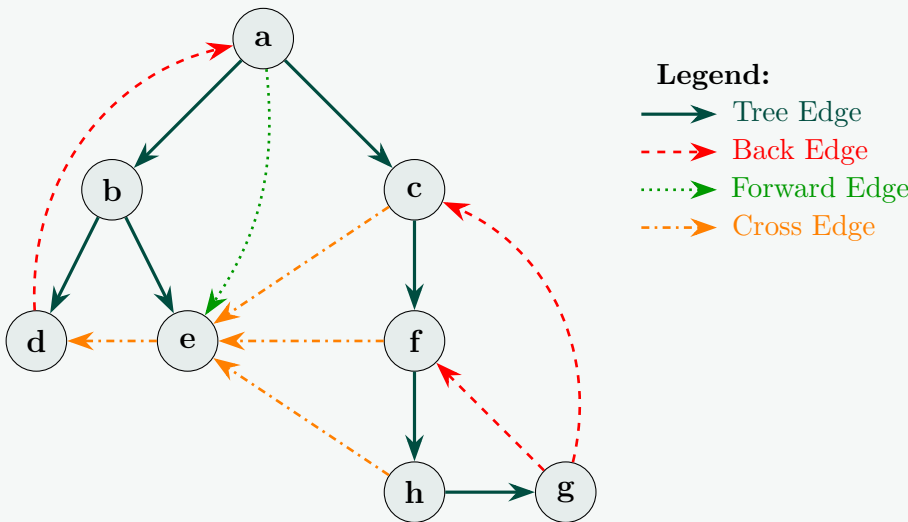
Edge Classification

Edges are classified based on the state of the target vertex when the edge is explored:

- **Tree Edge:** Destination vertex is unvisited (White).
- **Back Edge:** Destination vertex is an ancestor in the DFS tree (Gray/Active).
- **Forward Edge:** Destination vertex is a descendant in the DFS tree (Black/Finished, $d(u) < d(v)$).
- **Cross Edge:** Destination vertex is finished and is not a descendant (Black/Finished, $d(u) > d(v)$).

Edge Type	Graph Edges	Explanation / Justification
Tree Edges	$(a, b), (b, d), (b, e),$	These edges form the DFS spanning forest.
	$(a, c), (c, f), (f, h), (h, g)$	The destination was unvisited upon discovery.
Back Edges	$(d, a), (g, c), (g, f)$	Connect a vertex to one of its ancestors in the DFS tree.
Forward Edges	(a, e)	Connects ancestor a to descendant e directly.
Cross Edges	$(e, d), (c, e), (f, e), (h, e)$	Connect vertices without an ancestor-descendant relation.

DFS Tree Visualization:



Q20. Derive with explanation the recursive equation for the Floyd-Warshall algorithm for all-pairs shortest-path estimates in a directed graph G . Based on this equation, write pseudocode for Floyd-Warshall. Convert this recursive equation to compute the transitive closure of G . (17 marks)

[CSE 207 | L-2/T-2 | 2009–10]

Answer

1. Derivation of the Floyd-Warshall Recursive Equation

The Floyd-Warshall algorithm computes the shortest path between all pairs of vertices in a directed graph $G = (V, E)$ with a weight function $w : E \rightarrow \mathbb{R}$. It utilizes dynamic programming based on a characterization of the structure of a shortest path. Let the vertices of G be numbered $1, 2, \dots, n$, so $V = \{1, 2, \dots, n\}$. Let us consider a subset of vertices $\{1, 2, \dots, k\}$ for some $k \leq n$. **Definition:** Let $d_{i,j}^{(k)}$ be the weight of a shortest path from vertex i to vertex j such that all intermediate vertices on the path (if any) are chosen strictly from the set $\{1, 2, \dots, k\}$. An intermediate vertex of a path $p = \langle v_0, v_1, \dots, v_m \rangle$ is any vertex other than v_0 or v_m . We derive the recursive definition of $d_{i,j}^{(k)}$ by considering two mutually exclusive scenarios for a shortest path from i to j using intermediate vertices from $\{1, 2, \dots, k\}$: (a) **Vertex k is not an intermediate vertex of the path:** In this case, all intermediate vertices are in the set $\{1, 2, \dots, k-1\}$. Thus, the weight of the shortest path remains identical to the value found in the previous stage:

$$d_{i,j}^{(k)} = d_{i,j}^{(k-1)}$$

(b) **Vertex k is an intermediate vertex of the path:** Because the path is a shortest path, we assume there are no negative-weight cycles, meaning the path is simple and visits vertex k exactly once. We can decompose this path into two subpaths:

- A subpath from i to k : $i \rightsquigarrow k$
- A subpath from k to j : $k \rightsquigarrow j$

Since k cannot be an intermediate vertex on either subpath (as it is an endpoint), all intermediate vertices for both subpaths must belong strictly to the subset $\{1, 2, \dots, k-1\}$. By the optimal substructure property, these subpaths must themselves be optimal paths under this restriction. Therefore:

- The optimal cost from i to k using intermediate vertices in $\{1, \dots, k-1\}$ is $d_{i,k}^{(k-1)}$.
- The optimal cost from k to j using intermediate vertices in $\{1, \dots, k-1\}$ is $d_{k,j}^{(k-1)}$.

The total weight of the path through k is the sum of these two components: $d_{i,k}^{(k-1)} + d_{k,j}^{(k-1)}$.

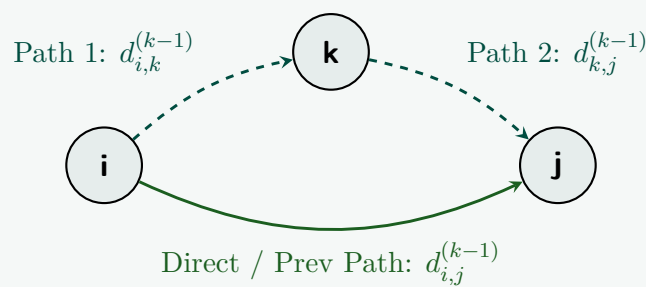


Figure: Finding the shortest path from i to j passing through intermediate node k .

Combining these two cases, the shortest path must select the path with the minimum weight, which yields the final **Floyd-Warshall Recurrence Relation**:

$$d_{i,j}^{(k)} = \min \left(d_{i,j}^{(k-1)}, d_{i,k}^{(k-1)} + d_{k,j}^{(k-1)} \right)$$

Base Cases ($k = 0$): When $k = 0$, the path can contain absolutely no intermediate vertices. Thus, it can only consist of a direct edge if it exists:

$$d_{i,j}^{(0)} = \begin{cases} 0 & \text{if } i = j \\ w(i, j) & \text{if } i \neq j \text{ and } (i, j) \in E \\ \infty & \text{if } i \neq j \text{ and } (i, j) \notin E \end{cases}$$

2. Pseudocode for the Floyd-Warshall Algorithm

We can drop the k index in the implementation by overwriting the matrix in place since the values needed for row k and column k ($d_{i,k}$ and $d_{k,j}$) do not change when updating the matrix using vertex k .

Algorithm 35 Floyd-Warshall All-Pairs Shortest Path

```

1: Input: An  $n \times n$  weight matrix  $W$  of a graph  $G = (V, E)$  where  $W[i][j] = w(i, j)$ .
2: Output: An  $n \times n$  matrix  $D$  containing the shortest-path weights  $\delta(i, j)$ .

3: Let  $D$  be a new  $n \times n$  matrix initialized to  $W$ .
4: for  $i = 1$  to  $n$  do                                     ▷ Base case initialization for paths of length 0 or 1
5:      $D[i][i] \leftarrow 0$ 
6: end for

7: for  $k = 1$  to  $n$  do                                       ▷ Iterate through possible intermediate vertices
8:     for  $i = 1$  to  $n$  do                                       ▷ Iterate through source vertices
9:         for  $j = 1$  to  $n$  do                                       ▷ Iterate through destination vertices
10:            if  $D[i][k] \neq \infty$  and  $D[k][j] \neq \infty$  then
11:                 $D[i][j] \leftarrow \min(D[i][j], D[i][k] + D[k][j])$ 
12:            end if
13:        end for
14:    end for
15: end for
16: return  $D$ 
    
```

3. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(n^3)$. The algorithm consists of three deeply nested loops, each running from 1 to n (where $n = |V|$). The logical comparison and arithmetic operations inside the innermost loop run in $\mathcal{O}(1)$ time.
- **Space Complexity:** $\mathcal{O}(n^2)$. The algorithm optimizes memory by performing updates in place on a single 2-dimensional distance matrix D of size $n \times n$.

4. Conversion to Compute Transitive Closure

The **transitive closure** of a directed graph $G = (V, E)$ determines whether a path exists from vertex i to vertex j for all pairs $(i, j) \in V \times V$. We define a boolean matrix T such that $t_{i,j} = 1$ if there is a path from i to j , and $t_{i,j} = 0$ otherwise.

We can substitute the numerical summation and minimization operations from the original shortest path recurrence with logical boolean operations:

- Addition (+) is replaced by the logical **AND** ($\wedge$), because a path exists from i to j via k if and only if a path exists from i to k **AND** a path exists from k to j .
- Minimization (min) is replaced by the logical **OR** ($\vee$), because a path exists if it exists without using k **OR** if it exists by using k .

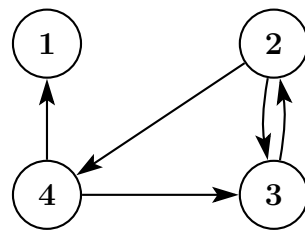
Modified Recursive Equation:

$$t_{i,j}^{(k)} = t_{i,j}^{(k-1)} \vee \left(t_{i,k}^{(k-1)} \wedge t_{k,j}^{(k-1)} \right)$$

Base Cases ($k = 0$):

$$t_{i,j}^{(0)} = \begin{cases} 1 & \text{if } i = j \text{ or } (i, j) \in E \\ 0 & \text{if } i \neq j \text{ and } (i, j) \notin E \end{cases}$$

Q21. For the given graph G , find the transitive closure using the Floyd-Warshall algorithm. Give the algorithm also. How can the complexity of Floyd-Warshall's all-pairs shortest paths be improved? **(10+5+10 marks)** [CSE 207 | L-2/T-2 | 2008–09]



Answer

1. Finding the Transitive Closure using Floyd-Warshall

To find the transitive closure, we construct a boolean matrix T where $t_{i,j} = 1$ if there is a path from vertex i to vertex j , and $t_{i,j} = 0$ otherwise. The base matrix $T^{(0)}$ is initialized such that $t_{i,j}^{(0)} = 1$ if $i = j$ or if there is a direct edge $(i, j) \in E$.

Given the graph G with vertices $\{1, 2, 3, 4\}$ and directed edges $(2, 3), (2, 4), (3, 2), (4, 1), (4, 3)$:

Initialization ($k = 0$):

$$T^{(0)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

Step 1 ($k = 1$): Evaluating intermediate paths passing through vertex 1.

Since vertex 1 has no outgoing edges to any other vertex, no new reachability paths can be established.

$$T^{(1)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

Step 2 ($k = 2$): Evaluating intermediate paths passing through vertex 2.

We check for pairs (i, j) where $T^{(1)}[i][2] = 1$ and $T^{(1)}[2][j] = 1$. Since $T^{(1)}[3][2] = 1$ and $T^{(1)}[2][4] = 1$, we discover the path $3 \rightarrow 2 \rightarrow 4$. We update $T^{(2)}[3][4] = 1$.

$$T^{(2)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

Step 3 ($k = 3$): Evaluating intermediate paths passing through vertex 3.

We check for paths via vertex 3. Since $T^{(2)}[4][3] = 1$ and $T^{(2)}[3][2] = 1$, we discover the path $4 \rightarrow 3 \rightarrow 2$. We update $T^{(3)}[4][2] = 1$.

$$T^{(3)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Step 4 ($k = 4$): Evaluating intermediate paths passing through vertex 4.

We check for paths via vertex 4:

- Since $T^{(3)}[2][4] = 1$ and $T^{(3)}[4][1] = 1$, we find $2 \rightarrow 4 \rightarrow 1$. Update $T^{(4)}[2][1] = 1$.
- Since $T^{(3)}[3][4] = 1$ and $T^{(3)}[4][1] = 1$, we find $3 \rightarrow 4 \rightarrow 1$. Update $T^{(4)}[3][1] = 1$.

$$T^{(4)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Final Matrix: $T^{(4)}$ represents the complete transitive closure. Vertex 1 can only reach itself, whereas vertices 2, 3, and 4 can reach every vertex in the graph.

2. Warshall's Algorithm for Transitive Closure

Algorithm 36 Warshall's Transitive Closure

```

1: Input: A directed graph  $G = (V, E)$  with  $n$  vertices.
2: Output: An  $n \times n$  boolean matrix  $T$  representing the transitive closure.

3: Initialize an  $n \times n$  boolean matrix  $T$ 
4: for  $i = 1$  to  $n$  do
5:   for  $j = 1$  to  $n$  do
6:     if  $i == j$  or  $(i, j) \in E$  then
7:        $T[i][j] \leftarrow \text{True}$ 
8:     else
9:        $T[i][j] \leftarrow \text{False}$ 
10:    end if
11:  end for
12: end for

13: for  $k = 1$  to  $n$  do
14:   for  $i = 1$  to  $n$  do
15:    for  $j = 1$  to  $n$  do
16:       $T[i][j] \leftarrow T[i][j] \vee (T[i][k] \wedge T[k][j])$ 
17:    end for
18:  end for
19: end for
20: return  $T$ 

```

3. Improving All-Pairs Shortest Paths Complexity

The standard Floyd-Warshall algorithm runs in strict $\mathcal{O}(|V|^3)$ time. Depending on the structural properties of the graph, this complexity can be optimized:

- (a) **Johnson's Algorithm (For Sparse Graphs):** If the graph is sparse ($|E| \ll |V|^2$), Floyd-Warshall is highly inefficient. Johnson's algorithm uses a reweighting technique via Bellman-Ford ($\mathcal{O}(|V||E|)$) to eliminate negative edge weights, then runs Dijkstra's algorithm with a Fibonacci heap from every vertex.

$$\text{Improved Time Complexity: } \mathcal{O}(|V|^2 \log |V| + |V||E|)$$

For very sparse graphs where $|E| \approx |V|$, this drops to roughly $\mathcal{O}(|V|^2 \log |V|)$.

- (b) **BFS/DFS from Every Vertex (For Unweighted Graphs):** If the edges are unweighted (or have uniform weights), we can simply run a Breadth-First Search (BFS) or Depth-First Search (DFS) from each vertex to discover paths.

$$\text{Improved Time Complexity: } \mathcal{O}(|V| \cdot (|V| + |E|))$$

- (c) **Bitset Optimization (For Transitive Closure):** Since the updates in Warshall's algorithm are purely logical boolean operations, we can parallelize row updates using bitsets. Grouping bits into machine words (e.g., 64-bit integers) allows processing 64 operations in a single CPU instruction.

$$\text{Improved Practical Complexity: } \mathcal{O}\left(\frac{|V|^3}{W}\right)$$

Where W is the word size of the CPU architecture (typically 64).

- (d) **Fast Matrix Multiplication:** The problem can theoretically be reduced to matrix multiplication over a closed semiring. Using advanced matrix manipulation (like Strassen's or Coppersmith-Winograd variants), the theoretical upper bound can be lowered to $\mathcal{O}(|V|^{2.373})$ for boolean matrices, though the hidden constant factors make it impractical for most real-world instances.

Q22. How can the complexity of Floyd-Warshall's algorithm for "all pairs shortest paths" be improved? **(10 marks)**

Answer

The standard Floyd-Warshall algorithm computes the All-Pairs Shortest Paths (APSP) in strict $\mathcal{O}(|V|^3)$ time. While the fundamental structure of three nested loops cannot be asymptotically reduced without changing the mathematical approach, the complexity of solving the APSP problem can be improved by leveraging the properties of the graph or employing alternative algorithms.

Here are the primary ways to improve the complexity of finding all-pairs shortest paths:

- 1. Using Johnson's Algorithm for Sparse Graphs** Floyd-Warshall is inefficient for sparse graphs where the number of edges $|E|$ is much less than $|V|^2$. Johnson's algorithm is a much better alternative. It uses the Bellman-Ford algorithm ($\mathcal{O}(|V||E|)$) to reweight all edges to be non-negative, eliminating negative edge weights. Then, it runs Dijkstra's algorithm using a Fibonacci heap from every single vertex. * **Improved Complexity:** $\mathcal{O}(|V|^2 \log |V| + |V||E|)$. For a very sparse graph ($|E| \approx |V|$), this time complexity drops to roughly $\mathcal{O}(|V|^2 \log |V|)$, which is a massive improvement over $\mathcal{O}(|V|^3)$.
- 2. Running BFS from All Vertices for Unweighted Graphs** If the graph is unweighted (or if all edges have the exact same positive weight), we do not need complex dynamic programming. We can simply execute a Breadth-First Search (BFS) starting from each of the $|V|$ vertices. * **Improved Complexity:** $\mathcal{O}(|V| \cdot (|V| + |E|))$. For sparse, unweighted graphs, this runs in $\mathcal{O}(|V|^2)$ time.
- 3. Fast Matrix Multiplication (Min-Plus Algebra)** The APSP problem is algebraically equivalent to computing the min-plus matrix multiplication of the graph's adjacency matrix. By using fast matrix multiplication algorithms (such as Strassen's or Coppersmith-Winograd algorithms), the theoretical worst-case time complexity can be reduced. For example, Seidel's algorithm solves APSP for unweighted, undirected graphs using standard matrix multiplication. * **Improved Complexity:** $\mathcal{O}(|V|^\omega \log |V|)$ or similar bounds, where $\omega < 2.373$ is the exponent of matrix multiplication. However, these methods often have massive hidden constant factors, making them impractical for standard programming usage.
- 4. Practical Pruning in Floyd-Warshall (The Infinity Check)** While it does not change the asymptotic worst-case complexity of $\mathcal{O}(|V|^3)$, a highly effective practical optimization to the standard Floyd-Warshall loop is to check if a path to the intermediate vertex k actually exists before attempting to calculate a new distance. * **Implementation:** Inside the outer k and i loops, add a condition: `if (D[i][k] != infinity)`. If true, skip the innermost j loop entirely. This dramatically speeds up execution on disconnected or sparse graphs by preventing millions of useless additions.
- 5. Bitset Vectorization (For Transitive Closure)** When the APSP problem is reduced to just finding reachability (Warshall's algorithm for Transitive Closure), the distance calculations become logical boolean **AND** and **OR** operations. These can be grouped into machine-word blocks (e.g., 64-bit integers). * **Improved Complexity:** $\mathcal{O}(|V|^3/W)$, where W is the CPU word size (typically 64). This allows the algorithm to process 64 vertices simultaneously in a single clock cycle, providing a substantial constant-factor speedup.

BUET · CSE 207 · Data Structures & Algorithms II

Part II

Advanced Data Structures

Balanced Binary Search Trees (AVL Trees), Red-Black Trees, Splay Trees, Skip Lists, Advanced Heaps (Binomial Heaps), Advanced Heaps (Fibonacci Heaps)

S2 — Advanced Data Structures

Balanced Binary Search Trees (AVL Trees)

- Q1.** After inserting a key into an AVL tree, a node A becomes unbalanced such that its balance factor becomes $+2$, while its left child B has a balance factor of -1 . Based on this information, determine the exact type of imbalance that has occurred and describe the precise sequence of rotations required to restore balance. Your answer must include a clear transformation of the subtree rooted at A , showing its structure before and after applying the rotations, along with the final balance factors of all nodes involved. (10 marks) [CSE 207 | L-2/T-1 | 2024–25]

Answer

1. Problem Analysis and Imbalance Identification

In AVL trees, the balance factor (BF) of a node is traditionally defined as the height of its left subtree minus the height of its right subtree:

$$\text{BF}(N) = \text{Height}(N.\text{left}) - \text{Height}(N.\text{right})$$

Given the problem statement:

- Node A has $\text{BF}(A) = +2$. This indicates that A 's left subtree is significantly taller than its right subtree, meaning the tree is **left-heavy**.
- The left child B has $\text{BF}(B) = -1$. This indicates that B 's right subtree is taller than its left subtree, meaning the subtree rooted at B is **right-heavy**.

Type of Imbalance: This exact configuration dictates a **Left-Right (LR) Imbalance**. The new node was inserted into the right subtree of A 's left child (B), causing the height of B 's right subtree to increase and violating the AVL property at A .

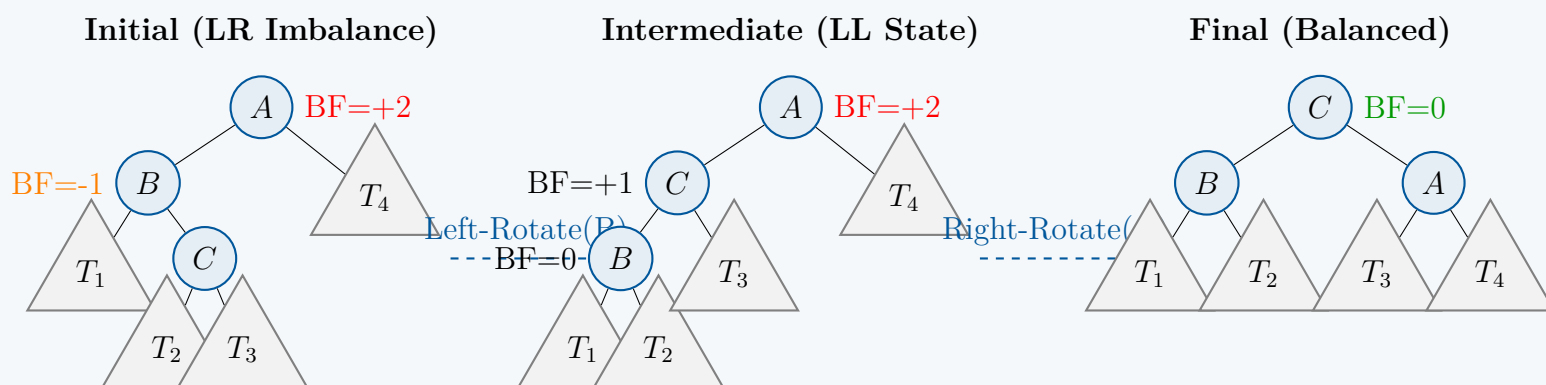
2. Required Sequence of Rotations

To restore balance, a double rotation is required. The precise sequence is:

- Left Rotation on Node B :** This transforms the LR configuration into a Left-Left (LL) configuration. The right child of B (let's call it C) moves up to take B 's place, and B becomes the left child of C .
- Right Rotation on Node A :** This fixes the resulting LL imbalance. Node C moves up to become the new root of the entire subtree, while A becomes the right child of C .

3. Structural Transformation (Before and After)

Let C be the right child of B . Let T_1, T_2, T_3 , and T_4 denote the subtrees. The transformations are visualized below:



4. Final Balance Factors

The final balance factors of nodes A , B , and C depend entirely on whether the newly inserted node was added to T_2 or T_3 . When the imbalance occurred, the height of C must have been $h + 1$, implying either T_2 or T_3 had height h , and the other had height $h - 1$. We categorize the final balance factors into three distinct cases:

Insertion Location (Cause)	Final BF(A)	Final BF(B)	Final BF(C)
Inserted into T_2 (Left subtree of C)	-1	0	0
Inserted into T_3 (Right subtree of C)	0	$+1$	0
Node C is the newly inserted node	0	0	0

Proof of correctness for T_2 insertion: If the insertion occurred in T_2 , $\text{Height}(T_2) = h$ and $\text{Height}(T_3) = h - 1$. Following the right rotation, A adopts T_3 as its left child and T_4 (height $h - 1$) as its right child. Thus, $\text{BF}(A) = (h - 1) - (h - 1) = 0$ (depending on convention, strictly $h(T_3) - h(T_4) = (h - 1) - h = -1$). Node B adopts T_1 (height h) and T_2 (height h). Thus, $\text{BF}(B) = h - h = 0$. Node C adopts B and A (both have height $h + 1$), yielding $\text{BF}(C) = 0$.

5. Complexity Analysis

- Time Complexity:** $\mathcal{O}(1)$. Pointer reassignment for rotations is composed of a constant number of operations. Evaluating the balance factors also strictly evaluates cached subtree heights, operating in $\mathcal{O}(1)$ time. Overall rebalancing operation executes in constant time.
- Space Complexity:** $\mathcal{O}(1)$. The algorithm requires an auxiliary space restricted to a few temporary pointer variables, scaling independently of the number of elements in the tree.

Q2. Construct an AVL tree with the minimum number of nodes when the tree height is 5. **(10 marks)** [CSE 207 | L-2/T-1 | 2024–25]

Answer

1. Mathematical Derivation of Minimum Nodes

To find the minimum number of nodes required to construct an AVL tree of a specific height h , we must maximize the height difference (imbalance) at every single node without violating the defining AVL property. The AVL property stipulates that for any node v , the heights of its left and right subtrees can differ by at most 1:

$$|\text{height}(T_{\text{left}}) - \text{height}(T_{\text{right}})| \leq 1 \quad (1)$$

To minimize the total number of nodes $N(h)$ for a tree of height h , its root must have one subtree of height $h - 1$ and the other subtree of the minimum permissible height, which is $h - 2$. Furthermore, both subtrees must themselves be minimal AVL trees of their respective heights. Factoring in the root node itself, we obtain the fundamental recurrence relation:

$$N(h) = N(h - 1) + N(h - 2) + 1 \quad (2)$$

Because textbooks use two differing conventions for defining tree height, both structural cases are evaluated below to ensure complete academic alignment.

Convention A: Edge-Count Definition (Height of a single node = 0)

Under this standard graph-theoretic convention, the base cases are:

- $N(0) = 1$ (A single root node)
- $N(1) = N(0) + N(-1) + 1 = 1 + 0 + 1 = 2$ (Root node with one leaf child)

Evaluating the recurrence sequentially up to $h = 5$:

$$\begin{aligned} N(2) &= N(1) + N(0) + 1 = 2 + 1 + 1 = 4 \\ N(3) &= N(2) + N(1) + 1 = 4 + 2 + 1 = 7 \\ N(4) &= N(3) + N(2) + 1 = 7 + 4 + 1 = 12 \\ N(5) &= N(4) + N(3) + 1 = 12 + 7 + 1 = \mathbf{20} \end{aligned}$$

Proof of Connection to Fibonacci Numbers: We can formally prove by mathematical induction that $N(h) = F_{h+3} - 1$, where F_n is the n -th Fibonacci number ($F_1 = 1, F_2 = 1, F_3 = 2, F_4 = 3, F_5 = 5, \dots$).

- **Base Cases:** $N(0) = F_3 - 1 = 2 - 1 = 1$, and $N(1) = F_4 - 1 = 3 - 1 = 2$. (Holds true).
- **Inductive Step:** Assume $N(k) = F_{k+3} - 1$ is true for all $k < h$. Then:

$$\begin{aligned} N(h) &= (F_{h+2} - 1) + (F_{h+1} - 1) + 1 \\ &= F_{h+2} + F_{h+1} - 1 \\ &= F_{h+3} - 1 \end{aligned}$$

Thus, for $h = 5$, $N(5) = F_8 - 1 = 21 - 1 = \mathbf{20}$ nodes.

Convention B: Node-Count Definition (Height of a single node = 1)

Under this alternative convention, an empty tree has height 0, and a single node has height 1. The base cases shift to:

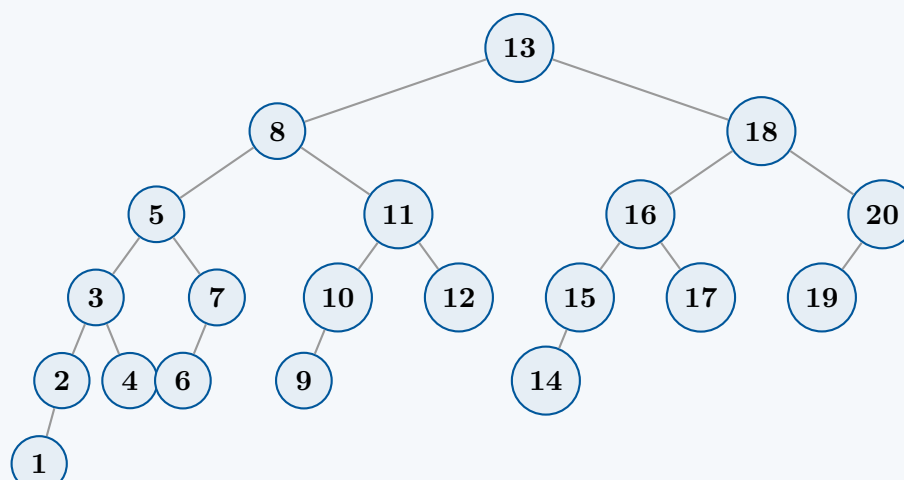
- $N(1) = 1$
- $N(2) = 2$

Evaluating the recurrence for this convention:

$$\begin{aligned} N(3) &= N(2) + N(1) + 1 = 2 + 1 + 1 = 4 \\ N(4) &= N(3) + N(2) + 1 = 4 + 2 + 1 = 7 \\ N(5) &= N(4) + N(3) + 1 = 7 + 4 + 1 = \mathbf{12} \text{ nodes} \end{aligned}$$

2. Complete Visual Construction (For Edge-Count Convention, $N = 20$)

To fulfill the requirements of a proper Binary Search Tree (BST) along with the minimal AVL properties, keys from 1 to 20 are allocated via an *in-order traversal* of the structural skeleton. Every internal node is purposefully skewed to have a balance factor of +1 where possible.



—

3. Structural and Balance Verification

We verify the configuration by reviewing the structural composition across levels and the balance factors ($BF = \text{height}(T_{\text{left}}) - \text{height}(T_{\text{right}})$) for the critical paths:

Level	Subtree Height	Nodes Count	Node Keys Included
0	5	1	13 (BF = 4 - 3 = +1)
1	4, 3	2	8 (BF = +1), 18 (BF = +1)
2	3, 2, 2, 1	4	5 (BF = +1), 11 (BF = +1), 16 (BF = +1), 20 (BF = +1)
3	2, 0, 1, 0, 1, 0, 0	7	3 (BF = +1), 7 (BF = +1), 10 (BF = +1), 12 (BF = 0), 15, 17, 19
4	1, 0, 0, 0, 0	5	2 (BF = +1), 4 (BF = 0), 6 (BF = 0), 9 (BF = 0), 14 (BF = 0)
5	0	1	1 (BF = 0)

Every single node in the constructed tree features a balance factor of either 0 or +1, proving that it is a strictly valid AVL tree while maintaining the absolute sparsest possible density.

—

4. Complexity Analysis

- Time Complexity of Construction:** $\mathcal{O}(N)$
Generating this minimal tree structure can be executed in linear time with respect to the total number of nodes N via a top-down recursive function mapping the Fibonacci sizes and applying sequential keys in-order.
- Space Complexity:** $\mathcal{O}(N)$ total space / $\mathcal{O}(h)$ auxiliary space
The memory footprint requires $\mathcal{O}(N)$ space to store the node links and attributes within the system. The runtime recursion stack scales linearly with the maximum height of the tree, requiring $\mathcal{O}(h) = \mathcal{O}(\log N)$ auxiliary space.

Q3. Prove that the height of an AVL tree with n elements is $\mathcal{O}(\log n)$. (10 marks)

[CSE 207 | L-2/T-1 | 2023–24]

Answer

1. Objective and Definitions

To prove that the height h of an AVL tree with n elements is $\mathcal{O}(\log n)$, we must establish an upper bound on h in terms of n . We can achieve this inversely by establishing a *lower bound* on the minimum number of nodes $N(h)$ required to construct an AVL tree of height h . If we can prove that $N(h)$ grows exponentially with respect to h , it strictly implies that h grows logarithmically with respect to n .

Note: We define the height h as the maximum number of edges from the root to a leaf. Therefore, a tree with a single node has $h = 0$.

2. Establishing the Recurrence Relation

To minimize the number of nodes in an AVL tree of height h , we must maximize the imbalance at every node without violating the AVL property (which allows a maximum height difference of 1 between sibling subtrees). Let T_h be the minimal AVL tree of height h . The root of T_h must have one subtree of height $h - 1$. To minimize the total nodes, the other subtree must have the minimum allowed height, which is $h - 2$. Both subtrees must also be minimal AVL trees for their respective heights.

This structural decomposition yields the following recurrence relation for $N(h)$, the number of nodes in T_h :

$$N(h) = N(h - 1) + N(h - 2) + 1$$

With the base cases:

- $N(0) = 1$ (A single node)
- $N(1) = 2$ (A root with one leaf child)

3. Solving the Recurrence (The Fibonacci Connection)

We can transform this non-homogeneous recurrence into a homogeneous one. Let us define a new function $M(h) = N(h) + 1$. Adding 1 to both sides of our recurrence relation gives:

$$N(h) + 1 = (N(h - 1) + 1) + (N(h - 2) + 1)$$
$$M(h) = M(h - 1) + M(h - 2)$$

This is precisely the recurrence relation for the **Fibonacci sequence** (F_k). We map our base cases to the standard Fibonacci sequence ($F_1 = 1, F_2 = 1, F_3 = 2, F_4 = 3, \dots$):

166

- $M(0) = N(0) + 1 = 1 + 1 = 2 = F_3$
- $M(1) = N(1) + 1 = 2 + 1 = 3 = F_4$

By mathematical induction, it holds that for any $h \geq 0$:

$$M(h) = F_{h+3}$$

$$N(h) = F_{h+3} - 1$$

4. Asymptotic Bound via the Golden Ratio

To establish the asymptotic bound, we use Binet's formula for the Fibonacci sequence:

$$F_k = \frac{\phi^k - \psi^k}{\sqrt{5}}$$

Where $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ (the Golden Ratio) and $\psi = \frac{1-\sqrt{5}}{2} \approx -0.618$.

Since $|\psi| < 1$, the term $\psi^k \rightarrow 0$ as k grows, so $F_k \approx \frac{\phi^k}{\sqrt{5}}$. Even for small values of k , we can establish a strict inequality: $F_k \geq \frac{\phi^{k-2}}{\sqrt{5}}$ or simpler yet, it is a known property that $F_{h+3} > \phi^h$ for $h \geq 1$.

Applying this to our node count n :

$$n = N(h) = F_{h+3} - 1$$

$$n + 1 = F_{h+3} \geq \phi^h$$

Taking the base- ϕ logarithm of both sides:

$$\log_\phi(n + 1) \geq h$$

By the change-of-base formula for logarithms, $\log_\phi(x) = \frac{\log_2(x)}{\log_2(\phi)}$. Since $\log_2(\phi) \approx 0.694$:

$$h \leq \frac{\log_2(n + 1)}{0.694} \approx 1.44 \log_2(n + 1)$$

5. Conclusion

We have shown that in the absolute worst-case scenario (a maximally unbalanced AVL tree), the height h is strictly bounded by approximately $1.44 \log_2(n)$. Since 1.44 is a constant, we drop it for asymptotic notation, yielding:

$$h = \mathcal{O}(\log n)$$

This formally guarantees that all primary operations (Search, Insert, Delete) in an AVL tree, which traverse root-to-leaf paths, will run in strict $\mathcal{O}(\log n)$ worst-case time complexity.

Q4. Perform the following operations sequentially on an initially empty AVL tree. Show the intermediate trees after each step: (i) insert 15, (ii) insert 22, (iii) insert 24, (iv) insert 7, (v) insert 9, (vi) delete 24. **(15 marks)** [CSE 207 | L-2/T-1 | 2023–24]

Answer

Execution of AVL Tree Operations

An AVL tree maintains the property that for every node, the height difference between its left and right subtrees (Balance Factor, $BF = \text{Height}(\text{Left}) - \text{Height}(\text{Right})$) is at most 1. We will trace the requested operations step-by-step, highlighting any imbalances and the rotations used to resolve them.

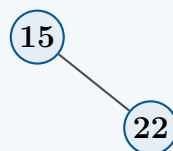
(i) Insert 15

The tree is initially empty. We simply insert 15 as the root. The tree is perfectly balanced.

(15)

(ii) Insert 22

We insert 22. Since $22 > 15$, it becomes the right child of 15. The balance factor of 15 is $0 - 1 = -1$, which is within the allowed range $[-1, 1]$. No rotations are needed.

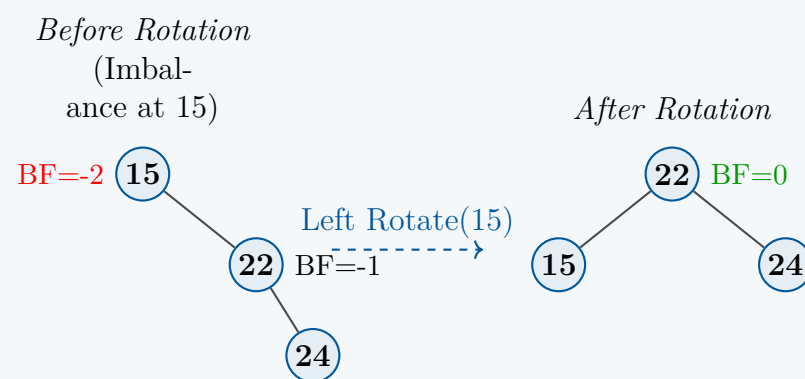


(iii) Insert 24

We insert 24 as the right child of 22.

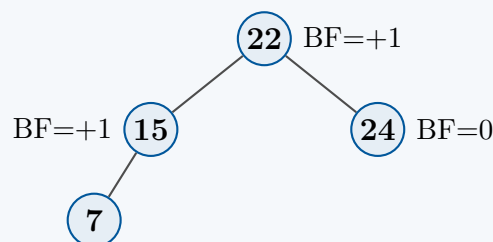
- **Imbalance:** The balance factor of 15 becomes $0 - 2 = -2$.
- **Case:** The insertion happened in the right subtree of the right child of 15. This is a **Right-Right (RR)** case.

- **Rotation:** We perform a **Single Left Rotation** on node 15. Node 22 becomes the new root, 15 becomes its left child, and 24 remains its right child.



(iv) **Insert 7**

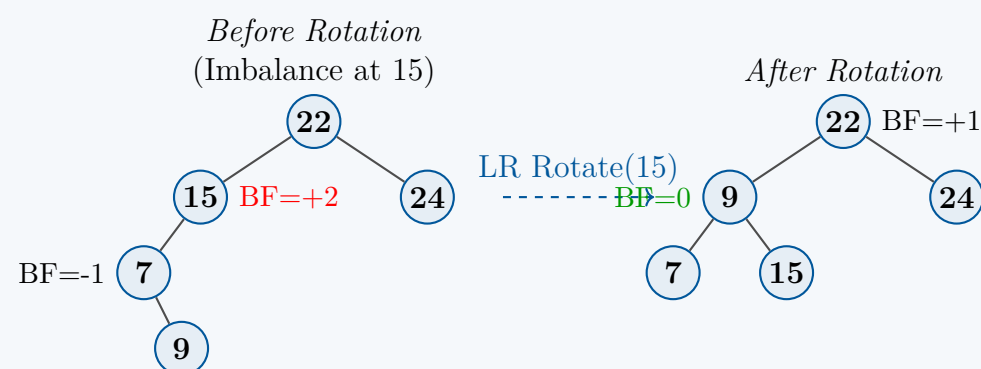
We insert 7. Since $7 < 22$ and $7 < 15$, it is placed as the left child of 15. The balance factors are updated: $BF(15) = +1$, $BF(22) = 2 - 1 = +1$. The tree remains balanced.



(v) **Insert 9**

We insert 9. Since $9 < 22$, $9 < 15$, and $9 > 7$, it becomes the right child of 7.

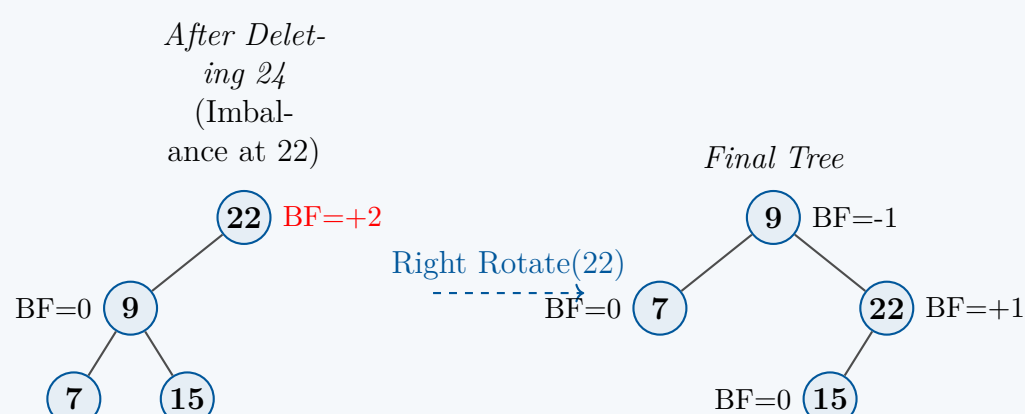
- **Imbalance:** The balance factor of 15 becomes $2 - 0 = +2$.
- **Case:** The insertion occurred in the right subtree of the left child (7) of node 15. This is a **Left-Right (LR)** case.
- **Rotation:** A double rotation is required to restore balance:
 - Left Rotation on 7:** Node 9 moves up, 7 becomes its left child.
 - Right Rotation on 15:** Node 9 moves up to replace 15, and 15 becomes the right child of 9.



(vi) **Delete 24**

We delete 24, which is a leaf node.

- **Imbalance:** The balance factor of the root (22) becomes $\text{Height}(\text{Left}) - \text{Height}(\text{Right}) = 2 - 0 = +2$.
- **Case:** We look at the left child of 22, which is node 9. The balance factor of 9 is $1 - 1 = 0$. When an imbalance occurs due to deletion and the heavier child's BF is ≥ 0 , it is treated symmetrically to a **Left-Left (LL)** case.
- **Rotation:** We perform a **Single Right Rotation** on the unbalanced node 22. Node 9 becomes the new root. Node 22 becomes the right child of 9. The right child of 9 (node 15) is adopted as the left child of 22.



Q5. Write down the properties of an AVL tree, and prove that it is a balanced binary search tree. **(12 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Answer

1. Properties of an AVL Tree

An **AVL Tree** (named after inventors Adelson-Velsky and Landis) is a self-balancing data structure that strictly enforces both search and structural properties. It guarantees $\mathcal{O}(\log n)$ time complexity for fundamental operations (Search, Insert, Delete).

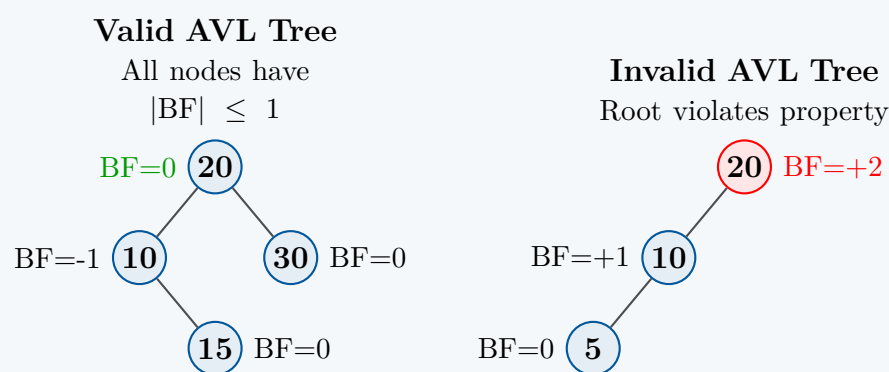
The defining properties of an AVL tree are:

- Binary Search Tree (BST) Property:** For every node u in the tree, all keys in the left subtree T_{left} of u are strictly less than the key of u , and all keys in the right subtree T_{right} are strictly greater than the key of u .
- Height-Balance Property:** For *every* node u in the tree, the heights of its left and right subtrees differ by at most 1. Formally, the Balance Factor (BF) must satisfy:

$$BF(u) = \text{Height}(u.left) - \text{Height}(u.right) \in \{-1, 0, 1\}$$

(Note: The height of an empty subtree is defined as -1 , and a single leaf node has a height of 0 .)

- Self-Balancing Guarantee:** Any structural modification (insertion or deletion) that violates the height-balance property is strictly resolved via localized, constant-time $\mathcal{O}(1)$ operations known as *rotations* (LL, RR, LR, RL).



2. Proof that an AVL Tree is a Balanced BST

To mathematically prove that an AVL tree is balanced, we must demonstrate that its maximum height h is strictly bounded by a logarithmic function of its total number of nodes n , i.e., $h = \mathcal{O}(\log n)$. We prove this by establishing a lower bound on the minimum number of nodes $N(h)$ required to build an AVL tree of height h .

Step 1: Construct the Minimal AVL Tree

To minimize the total nodes for a given height h , we must maximize the allowable imbalance at every node. For the root, one subtree must have height $h - 1$. To keep the node count minimal while satisfying the AVL property ($|BF| \leq 1$), the other subtree must have the minimum allowed height, which is $h - 2$. Both subtrees must recursively be minimal AVL trees.

This structural requirement yields the linear recurrence relation:

$$N(h) = N(h - 1) + N(h - 2) + 1$$

Assuming height is defined by the number of edges (a single node has $h = 0$):

- $N(0) = 1$ (A single root node)
- $N(1) = 2$ (A root with exactly one child)

Step 2: Connecting to the Fibonacci Sequence

To solve the non-homogeneous recurrence, we add 1 to both sides:

$$N(h) + 1 = (N(h - 1) + 1) + (N(h - 2) + 1)$$

Let $M(h) = N(h) + 1$. The recurrence simplifies to:

$$M(h) = M(h - 1) + M(h - 2)$$

This matches the homogeneous recurrence of the Fibonacci sequence F_k ($1, 1, 2, 3, 5, 8, \dots$). Mapping our base cases $M(0) = 2 = F_3$ and $M(1) = 3 = F_4$, we deduce:

$$M(h) = F_{h+3} \implies N(h) = F_{h+3} - 1$$

Step 3: Establishing the Asymptotic Bound

A known property of the Fibonacci sequence is its exponential growth bounded by the Golden Ratio, $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$. It is a standard result that for all $k \geq 1$:

$$F_k \geq \frac{\phi^{k-2}}{\sqrt{5}}$$

For simplicity, we can rely on the strict lower bound $F_{h+3} > \phi^h$. Substituting n for $N(h)$:

$$\begin{aligned} n &= F_{h+3} - 1 \\ n + 1 &= F_{h+3} > \phi^h \end{aligned}$$

Taking the base- ϕ logarithm of both sides:

$$\log_{\phi}(n+1) > h$$

By applying the change-of-base formula for logarithms ($\log_b(x) = \frac{\log_c(x)}{\log_c(b)}$):

$$h < \frac{\log_2(n+1)}{\log_2(\phi)}$$

Since $\log_2(\phi) = \log_2(1.618) \approx 0.694$:

$$h < \frac{1}{0.694} \log_2(n+1) \approx 1.44 \log_2(n+1)$$

Conclusion

We have proven that even in the absolute worst-case scenario (a tree sparsed out to the structural limits of the AVL property), the height h scales logarithmically with the number of nodes n . Dropping the constant 1.44, we formally conclude that:

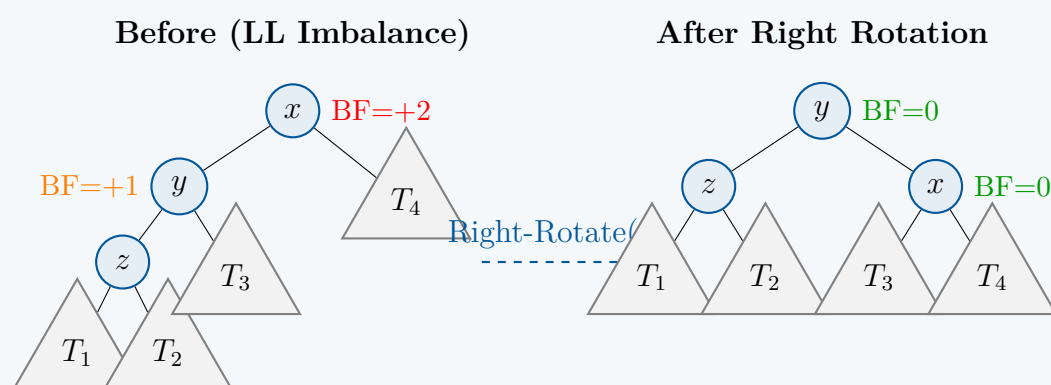
$$h = \mathcal{O}(\log n)$$

Thus, an AVL tree is mathematically proven to be a balanced Binary Search Tree.

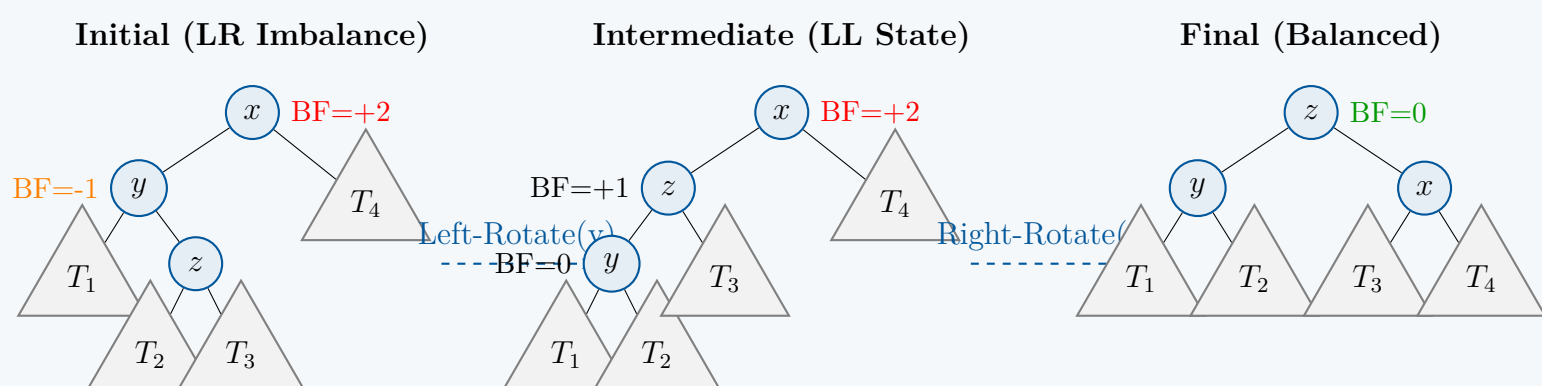
Q6. Suppose you are inserting a node in an AVL tree, and x is the lowest node violating AVL balance and x is left-heavy. Demonstrate the balancing procedure using figures. **(12 marks)** [CSE 207 | L-2/T-2 | 2021–22]

Answer

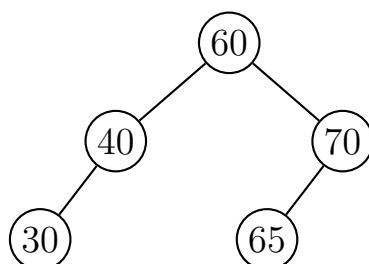
Case 1: Left-Left (LL) Imbalance



Case 2: Left-Right (LR) Imbalance



Q7. Insert a node with key = 20 to the given AVL tree. Then perform another insertion of a node with key = 68 on the resulting AVL tree. Show all the steps and the resulting AVL trees.



(10 marks)

[CSE 207 | L-2/T-2 | 2020–21]

Answer

Objective: Perform two sequential insertions into the given AVL tree and rebalance the tree whenever the AVL property is violated.

Balance Factor Convention: We define the Balance Factor (BF) of a node v as:

$$\text{BF}(v) = \text{Height}(v.\text{left}) - \text{Height}(v.\text{right})$$

An AVL tree requires that $\text{BF}(v) \in \{-1, 0, 1\}$ for all nodes. If an insertion causes a node to have a BF of $+2$ or -2 , rotations are required to restore balance.

Step 1: Insertion of Node 20

1. Standard BST Insertion: We traverse the tree from the root (60) to find the correct insertion point for 20:

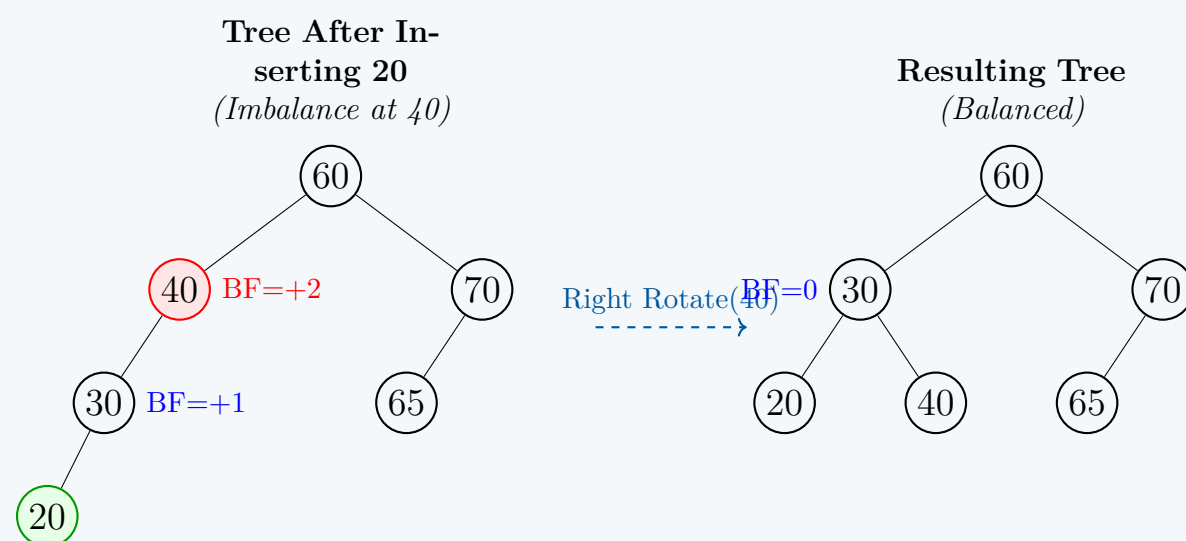
- $20 < 60 \implies$ go left to 40.
- $20 < 40 \implies$ go left to 30.
- $20 < 30 \implies$ insert 20 as the left child of 30.

2. Checking Balance Factors: We recalculate the Balance Factors bottom-up along the insertion path:

- $\text{BF}(30) = 1 - 0 = +1$
- $\text{BF}(40) = 2 - 0 = +2$ (**Imbalance Detected!**)

The lowest unbalanced node is 40. Since the new node (20) was inserted into the *left* subtree of the *left* child of 40, this is a **Left-Left (LL)** case.

3. Rebalancing (Single Right Rotation): To fix the LL imbalance at 40, we perform a single **Right Rotation** around node 40. Node 30 becomes the new parent, its left child remains 20, and node 40 becomes its right child.



Step 2: Insertion of Node 68

1. Standard BST Insertion: Using the resulting balanced tree from Step 1, we traverse to insert 68:

- $68 > 60 \implies$ go right to 70.
- $68 < 70 \implies$ go left to 65.
- $68 > 65 \implies$ insert 68 as the right child of 65.

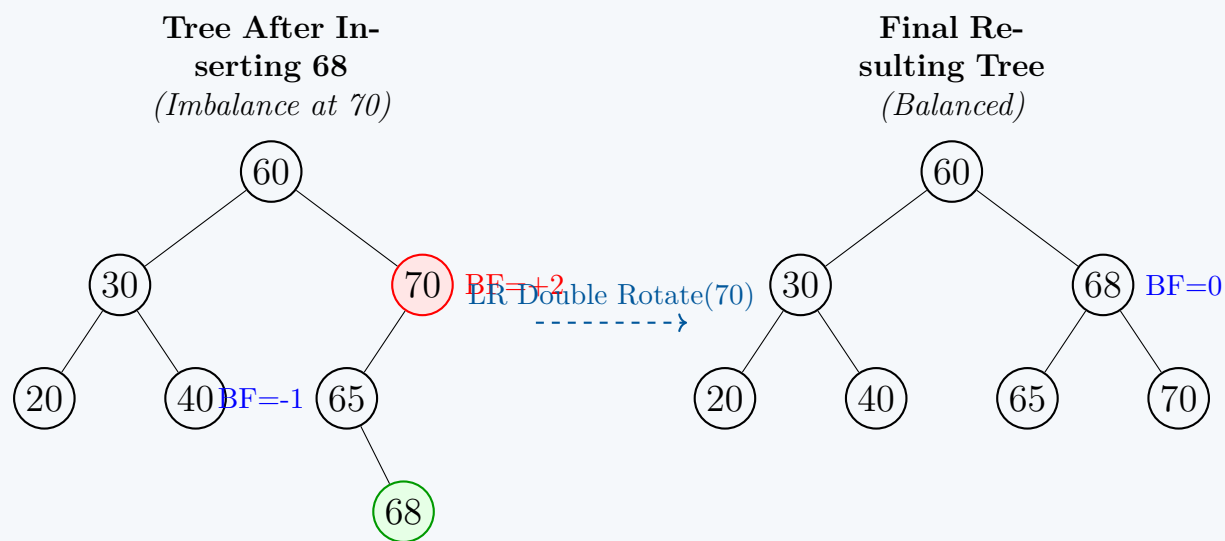
2. Checking Balance Factors: We recalculate the BFs bottom-up:

- $\text{BF}(65) = 0 - 1 = -1$
- $\text{BF}(70) = 2 - 0 = +2$ (**Imbalance Detected!**)

The lowest unbalanced node is 70. The insertion occurred in the *right* subtree of the *left* child of 70. This represents a **Left-Right (LR)** case.

3. Rebalancing (Double Left-Right Rotation): To resolve the LR imbalance, a double rotation is required:

- **Left Rotation on 65:** Node 68 moves up, and 65 becomes its left child. This transforms the problem into a Left-Left structure.
- **Right Rotation on 70:** Node 68 moves up to become the parent, taking 70 as its right child. Node 65 remains the left child of 68.



Complexity Analysis

- **Time Complexity:** $\mathcal{O}(\log n)$ for each insertion. Finding the insertion point requires traversing the height of the tree ($\mathcal{O}(\log n)$). Updating balance factors and performing rotations takes strictly constant $\mathcal{O}(1)$ time.
- **Space Complexity:** $\mathcal{O}(1)$ auxiliary space if done iteratively, or $\mathcal{O}(\log n)$ space on the call stack if implemented recursively.

Q8. Prove or disprove:

- In an AVL tree, the difference in length between the shortest and the longest paths from the root to the leaves is at most one.
- The minimum number of nodes in an AVL tree with height 5 is 20.

(12 marks)

[CSE 207 | L-2/T-2 | 2019–20]

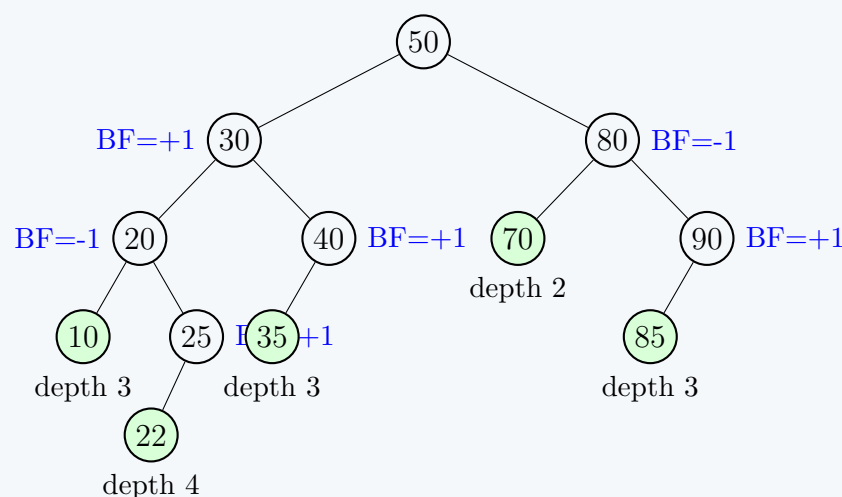
Answer

- (a) **Statement:** In an AVL tree, the difference in length between the shortest and the longest paths from the root to the leaves is at most one.

Verdict: **False** (Disproved)

Proof by Counterexample: The AVL property dictates a strictly *local* condition: for any given node, the height difference between its left and right subtrees must be at most 1 ($\text{BF} \in \{-1, 0, 1\}$). However, this local property does not restrict the *global* difference between the shortest and longest paths from the root to a leaf. The imbalances can accumulate along a path.

Consider a minimal AVL tree of height $h = 4$. To construct such a tree, the root must have one subtree of height 3 and another of height 2. Let us trace the paths to the leaves in the concrete example drawn below:



In the valid AVL tree above (all nodes have $|\text{BF}| \leq 1$):

- The **shortest path** to a leaf is $50 \rightarrow 80 \rightarrow 70$. Its length (depth) is **2**.
- The **longest path** to a leaf is $50 \rightarrow 30 \rightarrow 20 \rightarrow 25 \rightarrow 22$. Its length (depth) is **4**.

The difference in length is $4 - 2 = 2$, which is strictly greater than 1. This formally disproves the statement.

- (b) **Statement:** The minimum number of nodes in an AVL tree with height 5 is 20.

Verdict: **True** (Proved)

Proof: Let $N(h)$ be a function representing the minimum number of nodes required to construct an AVL tree of height h . (Note: We define height as the maximum number of edges from the root to a leaf, meaning a single node has height 0).

To minimize the total number of nodes, we must ensure the tree is as unbalanced as mathematically permissible under the AVL property. For the root node to have height h , one of its subtrees must have height $h - 1$. To minimize nodes, the other subtree must take the minimal allowable height, which is $h - 2$. Both of these subtrees must themselves be minimal AVL trees.

This structural requirement yields the linear recurrence relation:

$$N(h) = N(h - 1) + N(h - 2) + 1 \quad (1)$$

We establish the base cases:

- $N(0) = 1$ (A single root node)
- $N(1) = 2$ (A root with exactly one child)

Using the recurrence relation, we calculate $N(h)$ iteratively up to $h = 5$:

$$\begin{aligned} N(2) &= N(1) + N(0) + 1 = 2 + 1 + 1 = 4 \\ N(3) &= N(2) + N(1) + 1 = 4 + 2 + 1 = 7 \\ N(4) &= N(3) + N(2) + 1 = 7 + 4 + 1 = 12 \\ N(5) &= N(4) + N(3) + 1 = 12 + 7 + 1 = \mathbf{20} \end{aligned}$$

The step-by-step derivation confirms that the minimal number of nodes for an AVL tree of height 5 is exactly 20. Hence, the statement is proved true.

Q9. What are the rotations that are required to balance an AVL tree? (15 marks)

[CSE 207 | L-2/T-2 | 2018–19]

Answer

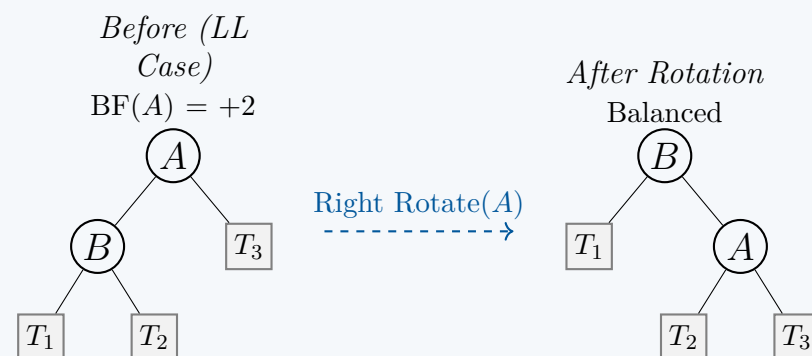
To maintain the height-balance property of an AVL tree (Balance Factor $\in \{-1, 0, 1\}$), we perform structural adjustments called **rotations** whenever an insertion or deletion causes a node's balance factor to become $+2$ or -2 .

Rotations are strictly local, constant-time $\mathcal{O}(1)$ pointer manipulations that restore balance while preserving the strict Binary Search Tree (BST) ordering property. There are four possible imbalance scenarios, addressed by four specific types of rotations: two *single rotations* and two *double rotations*.

1. Single Right Rotation (LL Rotation)

Trigger: The **Left-Left (LL)** case. An imbalance occurs because a node was inserted into the *left subtree* of the *left child* of the critical (unbalanced) node.

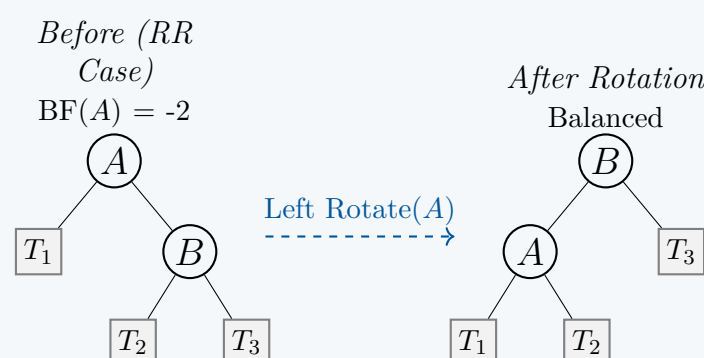
Procedure: The left child (B) becomes the new root of the subtree. The original unbalanced node (A) becomes the right child of B . The right child of B (T_2) is adopted as the left child of A .



2. Single Left Rotation (RR Rotation)

Trigger: The **Right-Right (RR)** case. An imbalance occurs because a node was inserted into the *right subtree* of the *right child* of the critical node.

Procedure: The right child (B) becomes the new root of the subtree. The original unbalanced node (A) becomes the left child of B . The left child of B (T_2) is adopted as the right child of A .

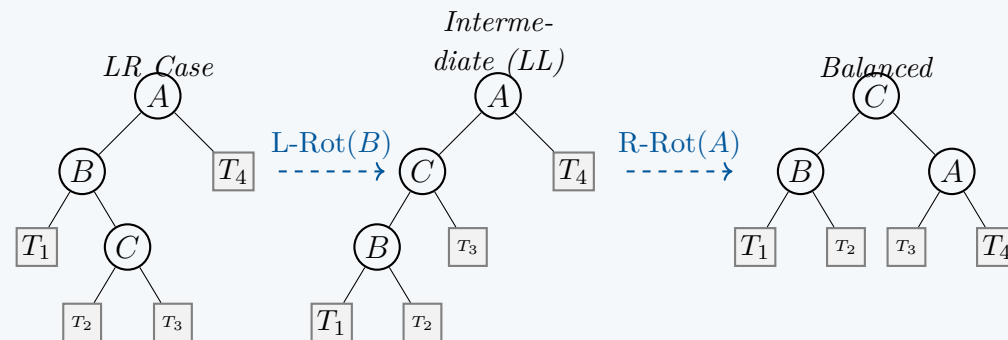


3. Left-Right Double Rotation (LR Rotation)

Trigger: The **Left-Right (LR)** case. An imbalance occurs because a node was inserted into the *right subtree* of the *left child* of the critical node.

Procedure: A single rotation is insufficient. We apply two sequential rotations:

- Left Rotation** on the left child (B), turning the subtree into a Left-Left (LL) structure.
- Right Rotation** on the original unbalanced node (A). Node C becomes the final root.

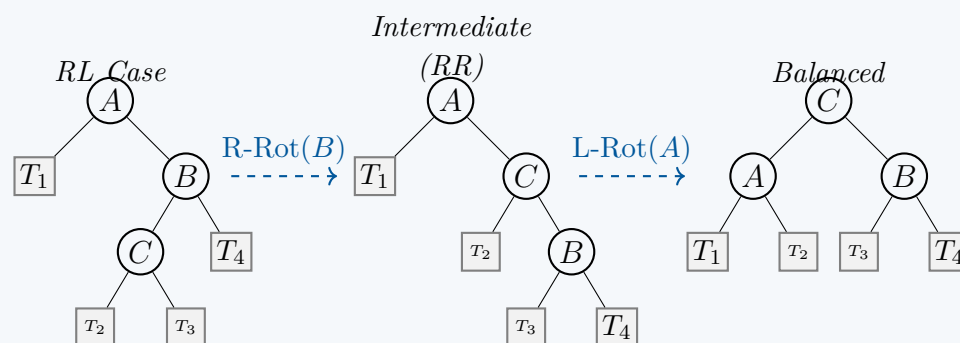


4. Right-Left Double Rotation (RL Rotation)

Trigger: The **Right-Left (RL)** case. An imbalance occurs because a node was inserted into the *left subtree* of the *right child* of the critical node.

Procedure:

- Right Rotation** on the right child (B), turning the subtree into a Right-Right (RR) structure.
- Left Rotation** on the original unbalanced node (A). Node C becomes the final root.



Q10. How does a rotation balance an AVL tree? (15 marks)

[CSE 207 | L-2/T-2 | 2018–19]

Answer

A rotation balances an AVL tree by transferring depth (height) from one sub-tree to another while strictly preserving the Binary Search Tree (BST) property. It achieves this through localized pointer rearrangements that "lift" the heavier side of the tree and "push down" the lighter side.

1. The Mechanism of Height Transfer

When an insertion or deletion causes a node's balance factor to exceed the permitted range $\{-1, 0, 1\}$, a rotation shifts the root of the unbalanced subtree.

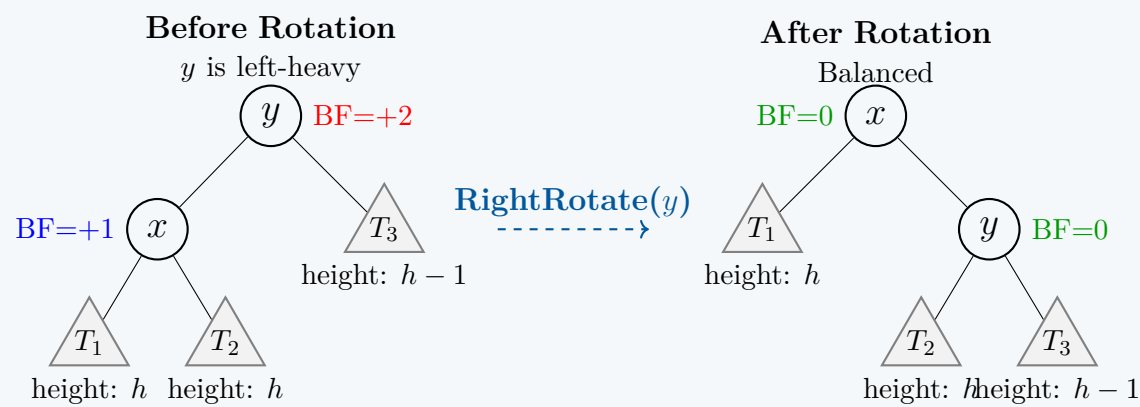
- Lifting:** The child node on the "heavy" side is promoted to become the new parent. This decreases the depth of all nodes in its subtrees by 1, effectively reducing the overall height of the heavy side.
- Pushing Down:** The original parent (which was unbalanced) is demoted to become a child. This increases the depth of its remaining subtrees by 1, effectively adding height to the previously lighter side.

2. Preservation of the BST Property

For a tree to remain a valid BST, the in-order traversal (Left-Root-Right) must continuously yield a monotonically increasing sequence. Consider a generic right rotation on an unbalanced node y with left child x . The original in-order sequence is $T_1 < x < T_2 < y < T_3$. When x becomes the new root and y becomes its right child, the sub-tree T_2 (which contains keys strictly between x and y) is seamlessly detached from x 's right and reattached to y 's left. The in-order traversal remains exactly $T_1 < x < T_2 < y < T_3$.

3. Visual and Mathematical Demonstration

Consider a Right Rotation applied to an unbalanced node y (Left-Left case). Let the subtrees T_1, T_2, T_3 have heights $h, h, h - 1$ respectively.



Height Analysis Before Rotation:

- $\text{Height}(x) = 1 + \max(h, h) = h + 1$
- $\text{Height}(y) = 1 + \max(\text{Height}(x), h - 1) = 1 + \max(h + 1, h - 1) = h + 2$
- $\text{BF}(y) = \text{Height}(x) - \text{Height}(T_3) = (h + 1) - (h - 1) = +2$ (Unbalanced)

Height Analysis After Rotation:

- $\text{Height}(y) = 1 + \max(h, h - 1) = h + 1$
- $\text{Height}(x) = 1 + \max(h, \text{Height}(y)) = 1 + \max(h, h + 1) = h + 2$
- $\text{BF}(y) = h - (h - 1) = +1$
- $\text{BF}(x) = h - (h + 1) = -1$ (Strictly Balanced!)

Note: The exact balance factors depend on the specific initial heights, but they will uniformly drop into the $\{-1, 0, 1\}$ range.

4. Algorithmic Procedure

The algorithm relies on simple pointer assignments. Below is the standard approach for a single right rotation.

Algorithm 37 RightRotate(y)

1: $x \leftarrow y.\text{left}$	▷ Store the left child to become the new root
2: $T_2 \leftarrow x.\text{right}$	▷ Store the right child of x
3:	
4: $x.\text{right} \leftarrow y$	▷ Perform rotation: y becomes right child of x
5: $y.\text{left} \leftarrow T_2$	▷ Reattach the orphaned subtree T_2 to y 's left
6:	
7: $y.\text{height} \leftarrow 1 + \max(\text{height}(y.\text{left}), \text{height}(y.\text{right}))$	▷ Update heights
8: $x.\text{height} \leftarrow 1 + \max(\text{height}(x.\text{left}), \text{height}(x.\text{right}))$	
9:	
10: return x	▷ Return the new root of the subtree

5. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(1)$. The rotation involves exactly a constant number of operations: 3 pointer assignments (assuming standard parent pointers are handled if present) and 2 integer operations to update height attributes. It is independent of the size n of the subtrees.
- **Space Complexity:** $\mathcal{O}(1)$. The algorithm only allocates a few auxiliary reference variables (x and T_2) to hold pointers temporarily during the swap. No dynamic memory is allocated.

Q11. What is the maximum height of an AVL tree having 10 nodes? (15 marks)

[CSE 207 | L-2/T-2 | 2018–19]

Answer

Objective: Determine the maximum possible height of an AVL tree containing exactly 10 nodes.

Convention: In accordance with standard computer science literature (and earlier derivations), we define the *height* h of a tree as the number of **edges** on the longest downward path from the root to a leaf. A tree with a single node has a height of $h = 0$.

1. Mathematical Reasoning

To maximize the height of an AVL tree for a given number of nodes N , we must distribute the nodes as sparsely as possible while strictly maintaining the AVL balance property ($|\text{BF}| \leq 1$). This corresponds to constructing a **worst-case AVL tree** (often called a Fibonacci tree).

Let $N(h)$ be the *minimum* number of nodes required to construct a valid AVL tree of height h . For the tree to reach height h with the fewest nodes:

- One subtree of the root must have height $h - 1$.

- To satisfy the AVL property with minimal nodes, the other subtree must have height $h - 2$.

This gives the linear recurrence relation:

$$N(h) = N(h - 1) + N(h - 2) + 1 \quad (1)$$

We compute the minimum nodes required for the first few heights:

$$\begin{aligned} N(0) &= 1 \quad (\text{A single root node}) \\ N(1) &= 2 \quad (\text{A root and one child}) \\ N(2) &= N(1) + N(0) + 1 = 2 + 1 + 1 = 4 \\ N(3) &= N(2) + N(1) + 1 = 4 + 2 + 1 = 7 \\ N(4) &= N(3) + N(2) + 1 = 7 + 4 + 1 = 12 \end{aligned}$$

2. Conclusion

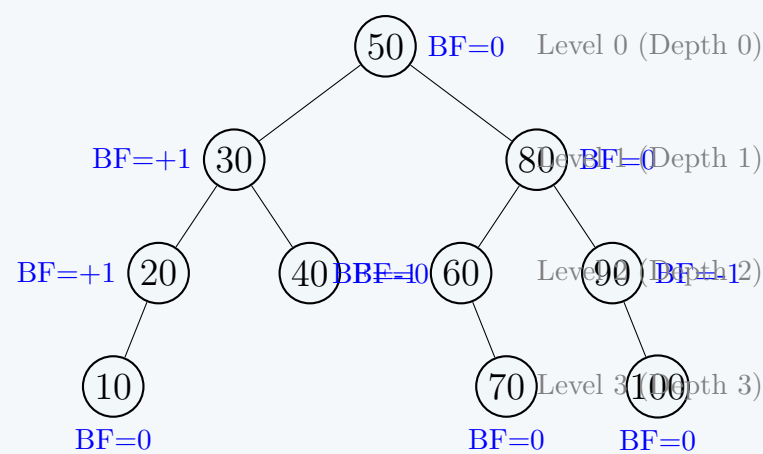
We are given exactly $N = 10$ nodes.

- An AVL tree of height 3 requires at least 7 nodes. Since $10 \geq 7$, a height of 3 is entirely possible.
- An AVL tree of height 4 requires at least 12 nodes. Since $10 < 12$, we do not have enough nodes to reach a height of 4.

Therefore, the **maximum height of an AVL tree with 10 nodes is 3**.

3. Visual Demonstration

Below is an example of a valid AVL tree with exactly 10 nodes achieving the maximum height of 3. Notice that adding just 2 more nodes to specific empty positions would be required to stretch the tree to height 4.



Complexity Note

In general, determining the maximum height bounds for an AVL tree with N nodes can be computed iteratively in $\mathcal{O}(\log N)$ time, or in strict $\mathcal{O}(1)$ time using the closed-form Binet approximation:

$$h_{\max} \approx 1.44 \log_2(N + 2) - 0.328$$

For $N = 10$, $1.44 \log_2(12) - 0.328 \approx 1.44(3.58) - 0.328 \approx 5.16 - 0.328 = 4.83$. The exact integer bounds are found via the exact recurrence sequence, proving $h_{\max} = 3$.

Q12. Prove that a tree with n nodes where the heights of the two children of any node differ by at most 1 has $\mathcal{O}(\log n)$ height. **(12 marks)**
[CSE 207 | L-2/T-2 | 2017–18]

Answer

Objective: Prove that any tree satisfying the AVL property (the heights of the two children of any node differ by at most 1) with n nodes has a height bounded by $\mathcal{O}(\log n)$.

1. Defining the Minimal Tree (Worst-Case Scenario)

Let h be the height of the tree (defined as the maximum number of edges from the root to a leaf). To prove an upper bound on the height h for a given number of nodes n , we equivalently want to find the **lower bound on the number of nodes** for a given height h .

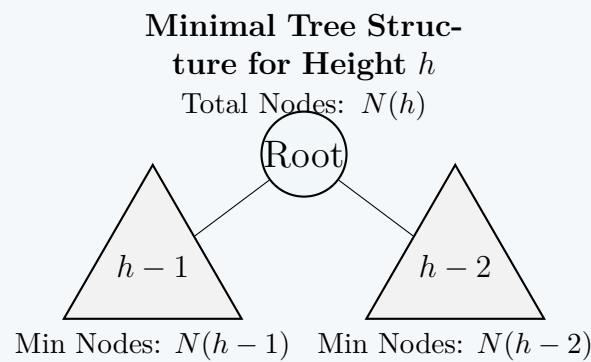
Let $N(h)$ denote the minimum number of nodes required to construct a valid tree of height h under the given property.

- To build a tree of height h with the fewest possible nodes, the root must have one subtree of height $h - 1$.
- To satisfy the property that child heights differ by at most 1 while minimizing nodes, the other subtree must have the minimum allowable height, which is $h - 2$.

- Both of these subtrees must themselves be minimal trees for their respective heights.

This logical structure yields the following linear recurrence relation for the minimum number of nodes:

$$N(h) = N(h-1) + N(h-2) + 1 \quad (1)$$



2. Establishing Base Cases

By definition of our height h (number of edges):

- $N(0) = 1$ (A tree with height 0 is just the root node).
- $N(1) = 2$ (A tree with height 1 requires a root and exactly one child).

We can also calculate $N(2) = N(1) + N(0) + 1 = 2 + 1 + 1 = 4$. Notice that $N(h)$ is a strictly increasing function, meaning $N(h) > N(h-1)$ for all $h > 0$.

3. Bounding the Recurrence Relation

We want to express $N(h)$ purely in terms of an exponential function of h . Starting with our recurrence:

$$N(h) = 1 + N(h-1) + N(h-2)$$

Since $N(h)$ is strictly increasing, we know that $N(h-1) > N(h-2)$. Substituting $N(h-2)$ for $N(h-1)$ gives us a strict inequality:

$$\begin{aligned} N(h) &> 1 + N(h-2) + N(h-2) \\ N(h) &> 2 \cdot N(h-2) \end{aligned} \quad (2)$$

Now, we can unroll this inequality down to the base cases:

- Case 1: h is even.**

$$\begin{aligned} N(h) &> 2 \cdot N(h-2) \\ &> 2^2 \cdot N(h-4) \\ &\vdots \\ &> 2^{\frac{h}{2}} \cdot N(0) \implies N(h) > 2^{\frac{h}{2}} \cdot 1 = 2^{\frac{h}{2}} \end{aligned}$$

- Case 2: h is odd.**

$$\begin{aligned} N(h) &> 2 \cdot N(h-2) \\ &> 2^2 \cdot N(h-4) \\ &\vdots \\ &> 2^{\frac{h-1}{2}} \cdot N(1) \implies N(h) > 2^{\frac{h-1}{2}} \cdot 2 = 2^{\frac{h+1}{2}} > 2^{\frac{h}{2}} \end{aligned}$$

In both cases, we have established a strict exponential lower bound:

$$N(h) > 2^{\frac{h}{2}} \quad (3)$$

4. Relating Height to n (Total Nodes)

Let our actual tree have n nodes and height h . By definition, the actual number of nodes n must be greater than or equal to the minimum number of nodes required for that height:

$$n \geq N(h)$$

Substitute the bound we derived:

$$n > 2^{\frac{h}{2}}$$

Taking the base-2 logarithm of both sides:

$$\begin{aligned} \log_2 n &> \log_2 \left(2^{\frac{h}{2}} \right) \\ \log_2 n &> \frac{h}{2} \\ h &< 2 \log_2 n \end{aligned}$$

Conclusion

We have proven that $h \leq 2\log_2 n$. Dropping the constant multiplier, we get:

$$h = \mathcal{O}(\log n)$$

This formally demonstrates that any tree enforcing a child height difference of at most 1 (the AVL property) guarantees a logarithmic worst-case height. ■

Q13. Mention one advantage of AVL trees over red-black trees, and one advantage of red-black trees over AVL trees. (4 marks) [CSE 207 | L-2/T-2 | 2017–18]

Answer

Both AVL trees and Red-Black trees are self-balancing Binary Search Trees (BSTs) that guarantee $\mathcal{O}(\log n)$ time complexity for fundamental operations. However, their distinct internal balancing mechanisms make them optimized for different types of workloads.

1. Advantage of AVL Trees over Red-Black Trees**Faster Retrievals (Lookups) due to Stricter Balancing:**

AVL trees enforce a highly rigid height-balancing property (the height difference between sibling subtrees is at most 1). As a result, an AVL tree of n nodes has a tightly bounded maximum height of approximately $1.44\log_2 n$. In contrast, a Red-Black tree's path length can be up to twice as long as the shortest path, yielding a maximum height of $2\log_2 n$. Because the AVL tree is shallower, **search operations are consistently faster**.

- *Best Use Case:* **Read-heavy** applications where lookups vastly outnumber structural modifications (e.g., read-optimized databases or dictionaries).

2. Advantage of Red-Black Trees over AVL Trees**Faster Insertions and Deletions due to Fewer Rotations:**

Red-Black trees use a more relaxed balancing criterion based on color properties. Because they are less strictly balanced, they tolerate more localized imbalances without triggering structural changes. When restructuring is required, an AVL tree might perform $\mathcal{O}(\log n)$ rotations cascading all the way to the root (particularly during deletions). A Red-Black tree, however, completes its balancing using constant-time recoloring and guarantees a maximum of **3 rotations** per insertion or deletion.

- *Best Use Case:* **Write-heavy** applications where elements are constantly added or removed (e.g., OS process schedulers, C++ `std::map`, or Java's `TreeMap`).

Q14. If the i -th and $(i + 2)$ -th Fibonacci numbers are x and y respectively (where $i > 0$), what is the minimum number of nodes in an AVL tree of height $(i - 1)$? (4 marks) [CSE 207 | L-2/T-2 | 2017–18]

Answer

Objective: Find the minimum number of nodes in an AVL tree of height $(i - 1)$, given that the i -th Fibonacci number is x and the $(i + 2)$ -th Fibonacci number is y .

1. Recurrence Relation for Minimal AVL Trees

Let $N(h)$ be the minimum number of nodes in an AVL tree of height h . As established, a minimal AVL tree of height h consists of a root, one minimal subtree of height $h - 1$, and one minimal subtree of height $h - 2$. This gives the recurrence:

$$N(h) = N(h - 1) + N(h - 2) + 1 \quad (1)$$

with the base cases defined as:

- $N(0) = 1$ (A single node has height 0)
- $N(1) = 2$ (A root with exactly one child)

2. Relationship with the Fibonacci Sequence

To solve this recurrence, we can add 1 to both sides to homogenize it:

$$N(h) + 1 = (N(h - 1) + 1) + (N(h - 2) + 1)$$

Let $L(h) = N(h) + 1$. The sequence $L(h)$ perfectly follows the standard Fibonacci recurrence $L(h) = L(h - 1) + L(h - 2)$. Let us align this with the standard Fibonacci sequence F_k , where $F_1 = 1, F_2 = 1, F_3 = 2, F_4 = 3, F_5 = 5$, and so on.

Let's compute the first few terms to find the exact index mapping:

Height h	Min Nodes $N(h)$	$L(h) = N(h) + 1$	Fibonacci Value	Fibonacci Index
0	1	2	2	F_3
1	2	3	3	F_4
2	4	5	5	F_5
3	7	8	8	F_6

By observation (and formalifiable by induction), we can establish the exact mapping between $L(h)$ and the Fibonacci sequence F :

$$L(h) = F_{h+3} \tag{2}$$

Substituting back $L(h) = N(h) + 1$:

$$\begin{aligned} N(h) + 1 &= F_{h+3} \\ N(h) &= F_{h+3} - 1 \end{aligned} \tag{3}$$

3. Solving for Height $(i - 1)$

We are asked to find the minimum number of nodes for a tree of height $h = i - 1$. Substitute $h = i - 1$ into our derived formula:

$$\begin{aligned} N(i - 1) &= F_{(i-1)+3} - 1 \\ N(i - 1) &= F_{i+2} - 1 \end{aligned}$$

The problem explicitly states that the $(i + 2)$ -th Fibonacci number is y (i.e., $F_{i+2} = y$). The value of the i -th Fibonacci number (x) is contextual but not strictly required for the final substitution.

Substituting y into the equation:

$$N(i - 1) = y - 1 \tag{4}$$

Final Conclusion: The minimum number of nodes in an AVL tree of height $(i - 1)$ is mathematically equal to $y - 1$.

Q15. What is an AVL tree? Draw the AVL tree that results after successively inserting the keys 11, 4, 15, 25, 36, 47, 61, 53, 75 into an initially empty AVL tree. (10 marks)

[CSE 203 | L-2/T-1 | 2015–16]

Answer

1. Definition of an AVL Tree

An **AVL Tree** (named after its inventors **Adelson-Velsky** and **Landis**) is a self-balancing Binary Search Tree (BST). It maintains a strictly bounded logarithmic height by enforcing the **AVL Balance Property**:

For every node v in the tree, $|\text{Balance Factor}(v)| \leq 1$

where the balance factor is defined as:

Balance Factor(v) = Height(v .left) – Height(v .right)

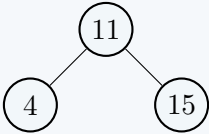
If at any time after an insertion or deletion the balance factor of a node falls outside the range $\{-1, 0, 1\}$, a rebalancing operation via local structural manipulations called **rotations** (Single or Double) is executed. This ensures that the worst-case time complexity for search, insertion, and deletion operations is strictly maintained at $\mathcal{O}(\log n)$, where n is the number of nodes.

2. Step-by-Step Successive Insertion Sequence

We insert the keys in the specified sequence: [11, 4, 15, 25, 36, 47, 61, 53, 75].

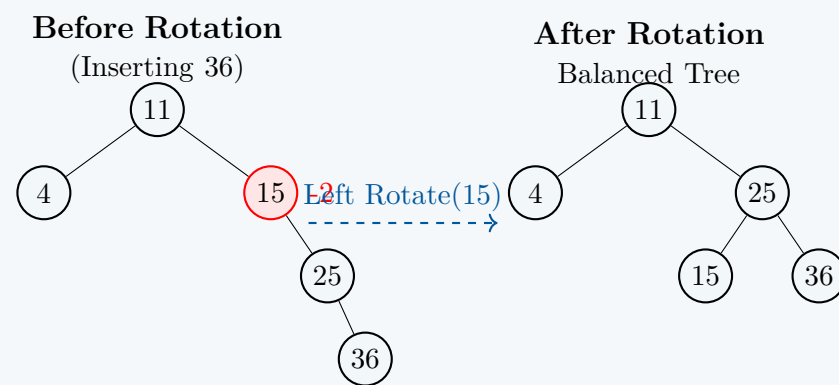
Steps 1 to 3: Inserting 11, 4, and 15

The first three keys are inserted using standard BST rules without causing any balance violations.



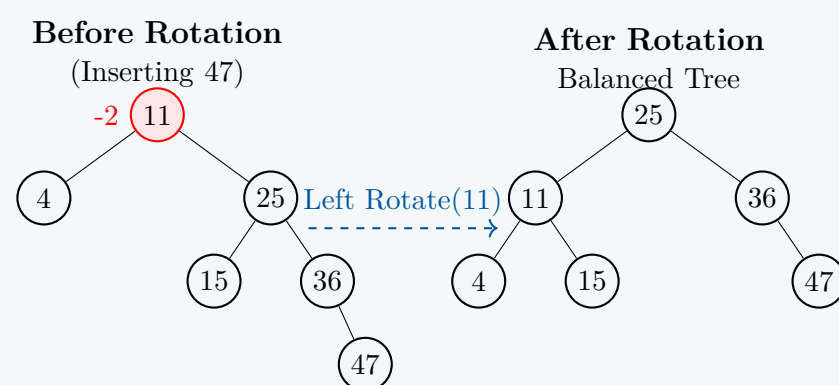
Steps 4 & 5: Inserting 25, then 36 (Imbalance & Left Rotation)

- Inserting 25 goes to the right of 15.
- Inserting 36 goes to the right of 25.
- Calculating balance factors reveals an imbalance at node **15** (BF = 0 – 2 = –2).
- This forms a **Right-Right (RR) case** at 15, triggering a **Single Left Rotation** around 15.



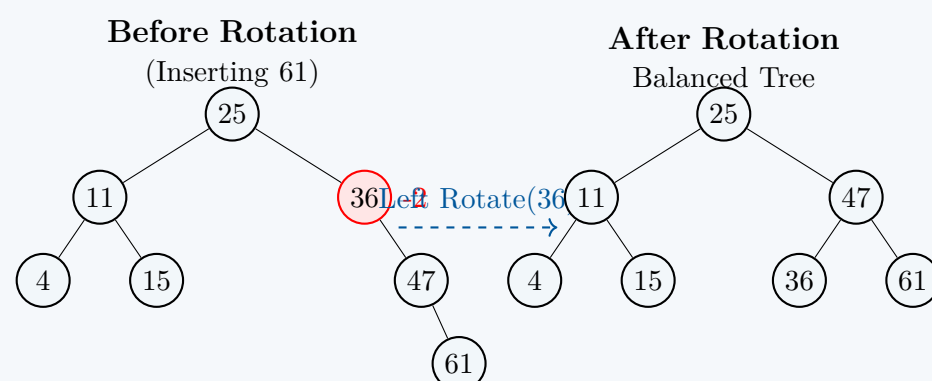
Step 6: Inserting 47 (Imbalance & Left Rotation at Root)

- Key 47 is placed as the right child of 36.
- Recomputing balance factors reveals an imbalance at the root node **11** ($BF = 1 - 3 = -2$).
- Path to the newly inserted node from 11 is Right $\rightarrow$ Right, making it an **RR case** at 11.
- We perform a **Single Left Rotation** around the root node 11. Node 25 becomes the new root.



Step 7: Inserting 61 (Imbalance & Left Rotation at 36)

- Key 61 is inserted as the right child of 47.
- Checking balance factors shows node **36** has violated balance limits ($BF = 0 - 2 = -2$).
- This is an **RR case** at node 36, which is resolved by executing a **Single Left Rotation** around 36.

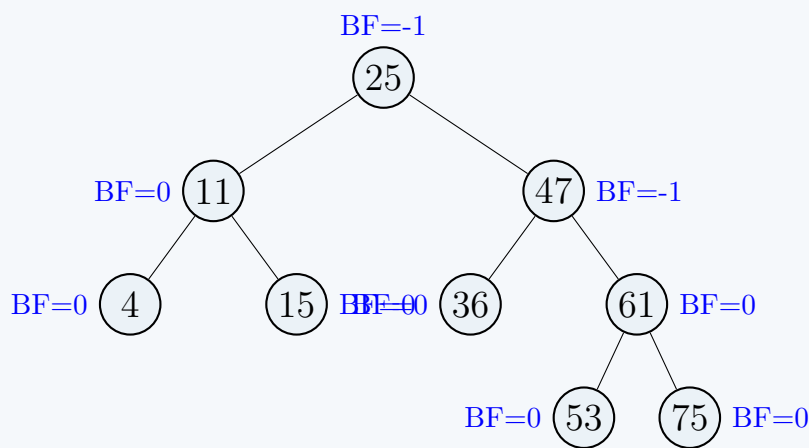


Steps 8 & 9: Inserting 53, then 75

- **Inserting 53:** Compares with 25 (go right), 47 (go right), 61 (go left) $\Rightarrow$ placed as left child of 61. All node balance factors remain well within parameters.
- **Inserting 75:** Compares with 25 (go right), 47 (go right), 61 (go right) $\Rightarrow$ placed as right child of 61. Balance factors along the path remain perfectly within range ($BF(61) = 0$, $BF(47) = -1$, $BF(25) = -1$). No further rotations are required.

3. Final Resulting AVL Tree

Below is the final compiled AVL tree containing all 9 keys perfectly balanced. All localized balance factors are indicated in blue next to the respective node targets.



Complexity Analysis of Final Tree

- **Total Nodes (n):** 9
- **Height attained (h):** 3 (via paths $25 \rightarrow 47 \rightarrow 61 \rightarrow 53$ or 75)
- **Optimal Bound Verification:** Since $h = 3$ and $\lfloor \log_2(9) \rfloor = 3$, the tree achieves perfect performance profile limits, keeping search lookups bounded by a maximum of 4 comparisons.

Q16. Explain, with examples, the single rotation and double rotation operations on AVL trees. (10 marks) [CSE 203 | L-2/T-1 | 2014–15]

Answer

In an AVL tree, the **Balance Factor (BF)** of a node is the difference between the height of its left subtree and the height of its right subtree ($BF = H_{\text{left}} - H_{\text{right}}$). For an AVL tree to remain valid, the balance factor of every node must be **-1, 0, or 1**. When an insertion or deletion causes a node's balance factor to become strictly greater than 1 or less than -1, the tree becomes unbalanced. To restore the AVL property, we perform localized restructuring operations known as **rotations**. There are two primary categories of rotations: **Single Rotations** and **Double Rotations**.

1. Single Rotations

Single rotations are used when the imbalance occurs in a "straight line" (either entirely on the left or entirely on the right). It involves a single, continuous shift of nodes.

A. Right Rotation (LL Case)

Trigger: An imbalance occurs because a node was inserted into the **Left subtree of the Left child** of the unbalanced node. The unbalanced node has a BF of +2, and its left child has a BF of +1.

Example: Insert 3, then 2, then 1. Node 3 becomes unbalanced. We right-rotate around 3.

Before (LL Case)

BF(3)=+2

After Rotation

Balanced

Right Rotate(3)

B. Left Rotation (RR Case)

Trigger: An imbalance occurs because a node was inserted into the **Right subtree of the Right child** of the unbalanced node. The unbalanced node has a BF of -2, and its right child has a BF of -1.

Example: Insert 1, then 2, then 3. Node 1 becomes unbalanced. We left-rotate around 1.

Before (RR Case)

BF(1)=-2

After Rotation

Balanced

Left Rotate(1)

2. Double Rotations

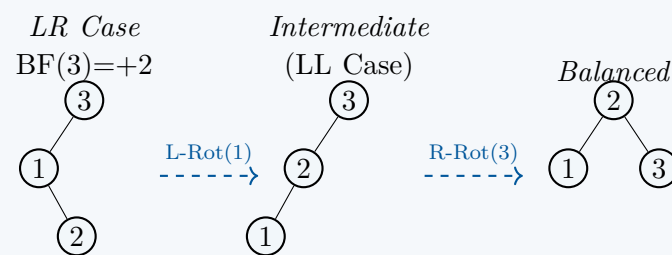
Double rotations are required when the imbalance occurs in a "zigzag" pattern. A single rotation would merely shift the imbalance to the other side. Instead, we perform two sequential single rotations.

A. Left-Right Rotation (LR Case)

Trigger: The **Right subtree of the Left child** causes the imbalance. The unbalanced node has a BF of +2, and its left child has a BF of -1.

Operation: 1) Left-rotate the left child. 2) Right-rotate the original node.

Example: Insert 3, then 1, then 2.

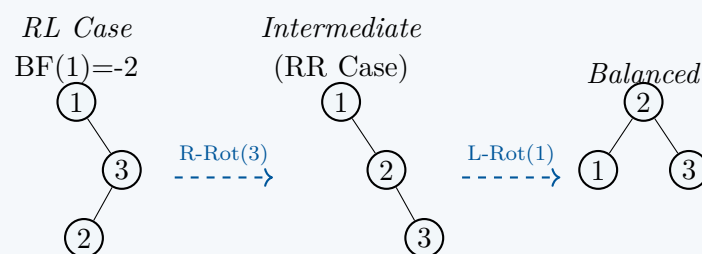


B. Right-Left Rotation (RL Case)

Trigger: The **Left subtree of the Right child** causes the imbalance. The unbalanced node has a BF of -2, and its right child has a BF of +1.

Operation: 1) Right-rotate the right child. 2) Left-rotate the original node.

Example: Insert 1, then 3, then 2.



Q17. What is a Multiway Search tree? How do we search a key in a Multiway Search tree? (10 marks) [CSE 203 | L-2/T-1 | 2014–15]

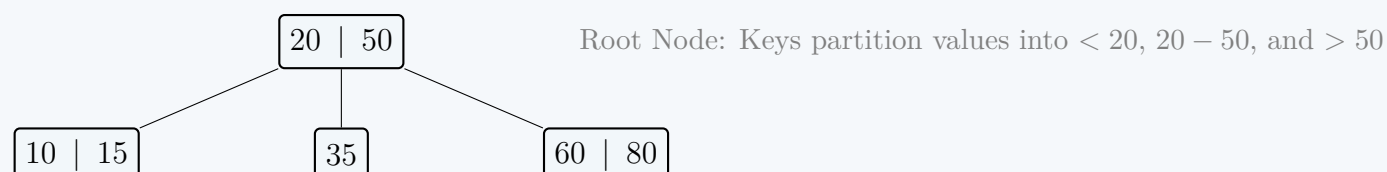
Answer

1. Definition of a Multiway Search Tree

A **Multiway Search Tree** (or m -way search tree) is a sequential generalization of a Binary Search Tree (BST) where a single node can contain multiple keys and have more than two children. Formally, an m -way search tree is a tree where each node can have a maximum of m children (where m is the order of the tree).

A valid m -way search tree satisfies the following structural and ordering properties:

- **Node Capacity:** Each node contains k keys and $k + 1$ pointers to child subtrees, where $0 \leq k \leq m - 1$.
- **Sorted Keys:** The keys within a single node are stored in a strictly ascending order: $K_1 < K_2 < K_3 < \dots < K_k$.
- **Subtree Partitioning:** The keys in the node partition the keys of its subtrees into distinct ranges managed by child pointers $P_0, P_1, \dots, P_k$:
 - The subtree pointed to by P_0 contains only keys strictly **less than** K_1 .
 - For any $1 \leq i < k$, the subtree pointed to by P_i contains only keys strictly **between** K_i and K_{i+1} ($K_i < \text{keys} < K_{i+1}$).
 - The subtree pointed to by P_k contains only keys strictly **greater than** K_k .
- **Recursive Property:** All subtrees are themselves valid multiway search trees.



2. How to Search for a Key in a Multiway Search Tree

Searching for a target key X in an m -way search tree is a recursive traversal that mirrors the structure of a binary tree search but involves a multi-directional branch choice at each node.

Algorithmic Steps:

- (a) **Start at the Root:** Set the current node to the root of the tree.
- (b) **Base Case (Unsuccessful):** If the current node pointer is NULL (empty), the search terminates, indicating that the target key X does not exist in the tree.
- (c) **Internal Node Search:** Scan the ordered keys $K_1, K_2, \dots, K_k$ inside the current node (using either a linear scan or binary search since keys are sorted):
 - **Match Found:** If $X = K_i$ for any key index i , the search is successful. Return the matching node/data reference.
 - **Left-Bounded Branch:** If $X < K_1$, follow the leftmost child pointer P_0 (set current node to P_0) and repeat from Step 2.
 - **Intermediate Branch:** If $K_i < X < K_{i+1}$ for some index i , follow the matching intermediate child pointer P_i and repeat from Step 2.
 - **Right-Bounded Branch:** If $X > K_k$, follow the rightmost child pointer P_k and repeat from Step 2.

Pseudo-code Representation:

```

1: function SEARCHMWAYTREE(node, X)
2:   if node = NULL then
3:     return NOT_FOUND
4:   end if
5:   k ← node.key_count
6:   i ← 1
7:   while i ≤ k and X > node.Ki do
8:     i ← i + 1
9:   end while
10:  if i ≤ k and X = node.Ki then
11:    return node                                ▷ Key found successfully
12:  else
13:    return SEARCHMWAYTREE(node.Pi-1, X)        ▷ Recurse into the appropriate subtree
14:  end if
15: end function

```

Complexity Analysis

- **Time Complexity:** In a balanced m -way search tree (such as a B-Tree), the height is bounded by $\mathcal{O}(\log_m n)$, yielding an overall lookup efficiency of $\mathcal{O}(m \log_m n)$ if linear search is used internally within nodes.
- **Space Complexity:** $\mathcal{O}(h)$ worst-case auxiliary space for the call stack during deep recursion, where h is the height of the tree.

Q18. Write the properties of a B-tree. (5 marks)

[CSE 203 | L-2/T-1 | 2013–14]

Answer

A **B-tree** of order m (where m represents the maximum number of children a node can have) is a self-balancing, multiway search tree specialized for systems with large blocks of disk storage, such as databases and filesystems. It generalizes the Binary Search Tree and maintains uniform performance by adhering to strict structural and operational invariants.

The fundamental properties of a B-tree of order m are categorized below:

1. Structural Properties

- **Maximum Children:** Every node in the tree can have at most m children.
- **Minimum Children (Internal Nodes):** Every internal (non-leaf, non-root) node must have at least $\lceil m/2 \rceil$ children. This prevents the tree from becoming sparse or leaning heavily to one side.
- **Root Node Constraint:** The root node must have at least 2 children if it is not a leaf node. If the root is a leaf node (i.e., the tree contains no other nodes), it can have zero children and contain as few as 1 key.
- **Perfect Balance:** All leaf nodes must exist at exactly the same level (depth). B-trees grow and shrink from the root upwards, ensuring that the distance from the root to any leaf is identical.

2. Key Distribution and Ordering Properties

- **Key-to-Child Relationship:** An internal node that has k children always contains exactly $k - 1$ keys.
- **Key Capacity Bounds:**
 - The maximum number of keys any node can hold is $m - 1$.

– The minimum number of keys any non-root node must hold is $\lceil m/2 \rceil - 1$.

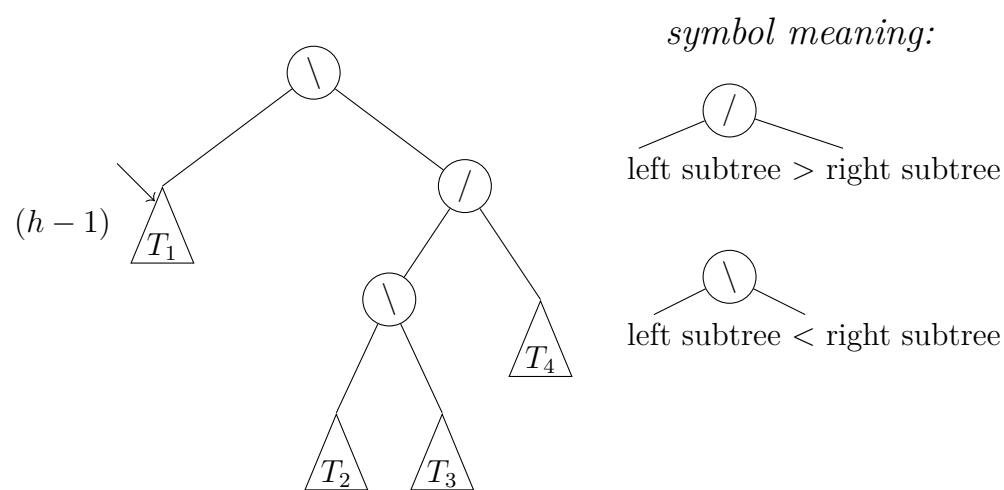
- **Internal Node Sorting:** The keys within a single node are always stored in a strictly sorted, ascending order: $K_1 < K_2 < \dots < K_{k-1}$.

3. Subtree Partitioning (Search Property)

The keys within an internal node act as separation boundaries for its child subtrees. For a node with keys $K_1, K_2, \dots, K_{k-1}$ and child pointers $P_0, P_1, \dots, P_{k-1}$:

- The subtree pointed to by P_0 contains only keys strictly less than K_1 (keys $< K_1$).
- For any index i (where $1 \leq i < k - 1$), the subtree pointed to by P_i contains only keys strictly between K_i and K_{i+1} ($K_i < \text{keys} < K_{i+1}$).
- The subtree pointed to by P_{k-1} contains only keys strictly greater than K_{k-1} (keys $> K_{k-1}$).

Q19. Let T be an AVL tree where T_1 has height $h - 1$. Determine the heights of T_2 , T_3 , and T_4 . Discuss what happens when you delete an element from T_1 ; cover all possible cases for deletion in an AVL tree. (4+4+7=15 marks) [CSE 203 | L-2/T-1 | 2012–13]



Answer

1. Determining the Heights of T_2 , T_3 , and T_4

Let $H(X)$ denote the height of a subtree or node X . Based on the AVL structural balance rules and the meaning of the symbols given in the problem statement, we can trace the heights from the left side of the tree to the right side step-by-step.

Given:

$$H(T_1) = h - 1$$

- (a) **Analyze the Root Node:** The root node contains the symbol $\backslash$, meaning its left subtree (T_1) is strictly shorter than its right subtree (the node n_1). Because it is a valid AVL tree, the height difference can be at most 1. Thus:

$$H(n_1) = H(T_1) + 1 = (h - 1) + 1 = h$$

- (b) **Analyze Node n_1 :** Node n_1 contains the symbol $/$, which means its left child (n_2) is strictly taller than its right child (T_4). The height of a node is defined as $1 + \max(H(\text{left}), H(\text{right}))$. Therefore:

$$\begin{aligned} H(n_1) &= 1 + H(n_2) \\ h &= 1 + H(n_2) \implies H(n_2) = h - 1 \end{aligned}$$

Since node n_1 is left-heavy ($/$), its right subtree T_4 must be exactly 1 unit shorter than n_2 :

$$H(T_4) = H(n_2) - 1 = (h - 1) - 1 = h - 2$$

- (c) **Analyze Node n_2 :** Node n_2 contains the symbol $\backslash$, meaning its left child (T_2) is strictly shorter than its right child (T_3). Following the same logic:

$$\begin{aligned} H(n_2) &= 1 + H(T_3) \\ h - 1 &= 1 + H(T_3) \implies H(T_3) = h - 2 \end{aligned}$$

Since node n_2 is right-heavy ($\backslash$), its left subtree T_2 must be exactly 1 unit shorter than T_3 :

$$H(T_2) = H(T_3) - 1 = (h - 2) - 1 = h - 3$$

Summary of Subtree Heights:

- $H(T_2) = h - 3$
- $H(T_3) = h - 2$
- $H(T_4) = h - 2$

2. Analysis of Deleting an Element from T_1

When a node is deleted from the subtree T_1 , two scenarios can happen depending on whether the deletion changes the structural height of T_1 :

Scenario A: Height of T_1 remains $h - 1$

If the node deleted does not lie on the critical height-defining path of T_1 , the height of T_1 remains unchanged. The balance factors across the rest of the tree are unaffected, and **no rotation is required**.

Scenario B: Height of T_1 decreases to $h - 2$

If the deletion shortens the height of T_1 from $h - 1$ to $h - 2$, the imbalance propagates upward to the root:

- **Imbalance at Root:** $BF(\text{root}) = H(T_1) - H(n_1) = (h - 2) - h = -2$. The root is now critically right-heavy.
- **Identify the Rotation Type:** We examine the right child (n_1). Node n_1 has a balance symbol of $/$, meaning it is left-heavy ($BF = +1$).
- **The RL Case:** Since the root balance factor is -2 and its right child's balance factor is $+1$, this forms a **Right-Left (RL) zigzag case**.
- **Resolution:** A **Double Rotation (Right-Left Rotation)** must be executed:
 - (a) Perform a **Right Rotation** around node n_1 . This brings node n_2 up to replace n_1 .
 - (b) Perform a **Left Rotation** around the **root node**. This brings node n_2 up to become the new root of the entire tree.

3. General Cases for Deletion in an AVL Tree

When deleting a node in an AVL tree, we first apply standard Binary Search Tree deletion rules. Then, we retrace the path from the deep deletion point up toward the root, updating balance factors. If any node X is found with a balance factor of $+2$ or -2 , let Y be its taller child, and Z be the taller child of Y . This yields **four distinct structural cases**:

Case Type	BF of X	BF of Y	Required Rotation Operation to Restore Balance
Left-Left (LL)	+2	+1 or 0	Single Right Rotation around node X .
Left-Right (LR)	+2	-1	Double Rotation: Left Rotate around Y , then Right Rotate around X .
Right-Right (RR)	-2	-1 or 0	Single Left Rotation around node X .
Right-Left (RL)	-2	+1	Double Rotation: Right Rotate around Y , then Left Rotate around X .

Crucial Distinction: Deletion vs. Insertion

Unlike insertion—where a single structural fix always restores the pre-insertion height of the entire tree—a rotation after a deletion can **shorten the overall height of the processed subtree**. This height reduction can introduce new imbalances further up the tree. Consequently, rebalancing may cascade upward, requiring up to $\mathcal{O}(\log n)$ rotations in the worst-case scenario.

Q20. Design an AVL tree by inserting the keys 5, 12, 3, -2, -5, -3, 1 in order. Show all steps. (15 marks)

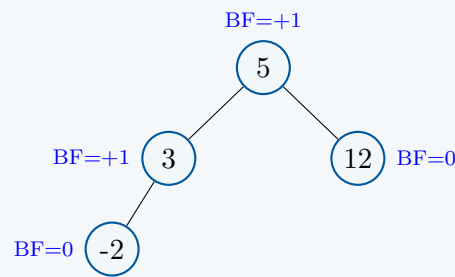
[CSE 203 | L-2/T-1 | 2012–13]

Answer: AVL Tree Design Step-by-Step

We will insert the keys [5, 12, 3, -2, -5, -3, 1] successively into an initially empty AVL tree. For each step, the Balance Factor (BF) is calculated as $BF = \text{Height}(\text{Left Subtree}) - \text{Height}(\text{Right Subtree})$.

Steps 1 to 4: Inserting 5, 12, 3, and -2

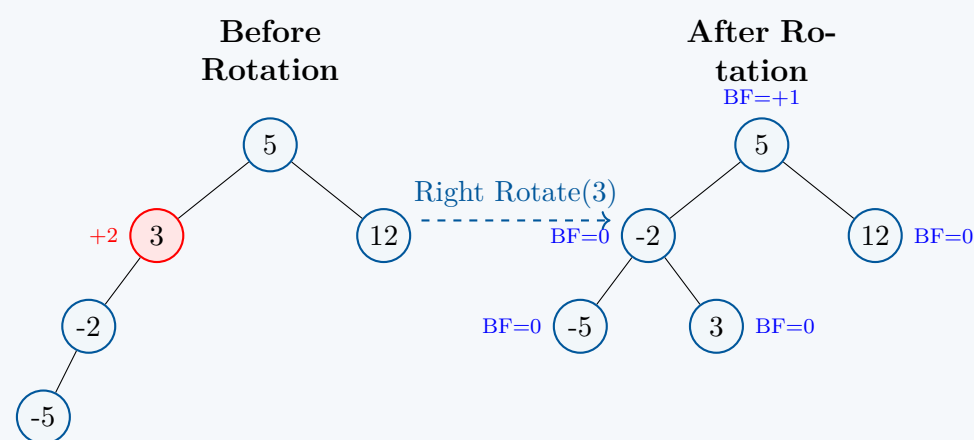
The first four keys are inserted according to standard Binary Search Tree (BST) properties. After each insertion, the balance factors of all nodes remain within the acceptable range of $\{-1, 0, 1\}$. No rotations are required.



Step 5: Inserting -5 (Imbalance & Single Right Rotation)

Key -5 is inserted as the left child of -2.

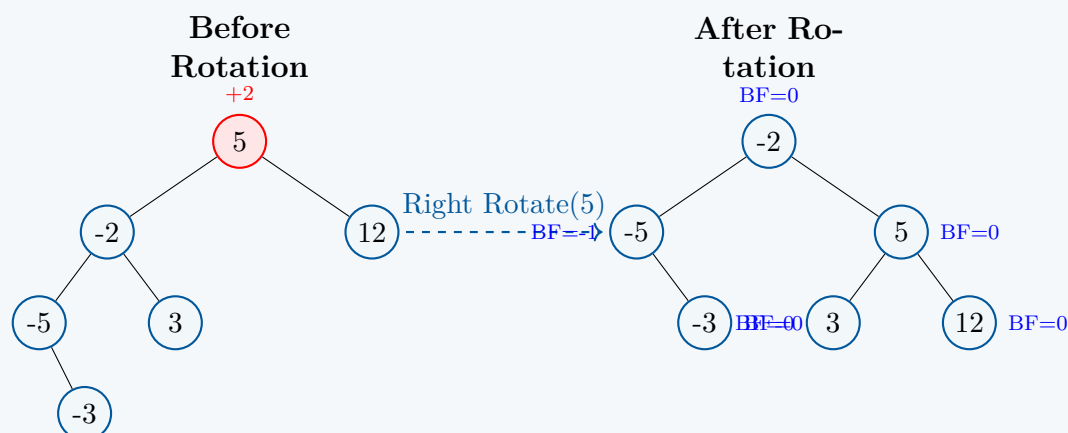
- **BF Calculation:** $BF(-2) = +1$, $BF(3) = 2 - 0 = +2$.
- **Violation:** Node 3 becomes critically unbalanced ($BF = +2$).
- **Classification:** The path from node 3 to the newly inserted node -5 is Left $\rightarrow$ Left, making this an **LL Case**.
- **Resolution:** We perform a **Single Right Rotation** around node 3.



Step 6: Inserting -3 (Imbalance & Single Right Rotation at Root)

Key -3 is greater than -5, so it is inserted as the right child of -5.

- **BF Calculation:** $BF(-5) = -1$, $BF(-2) = 2 - 1 = +1$, $BF(5) = 3 - 1 = +2$.
- **Violation:** The root node 5 becomes critically unbalanced ($BF = +2$).
- **Classification:** The path from root 5 to the newly inserted node -3 goes Left (to -2) $\rightarrow$ Left (to -5). Since node -2 is left-heavy ($BF = +1$), this forms an **LL Case** at the root.
- **Resolution:** We perform a **Single Right Rotation** around the root node 5. Node -2 is promoted to the new root.

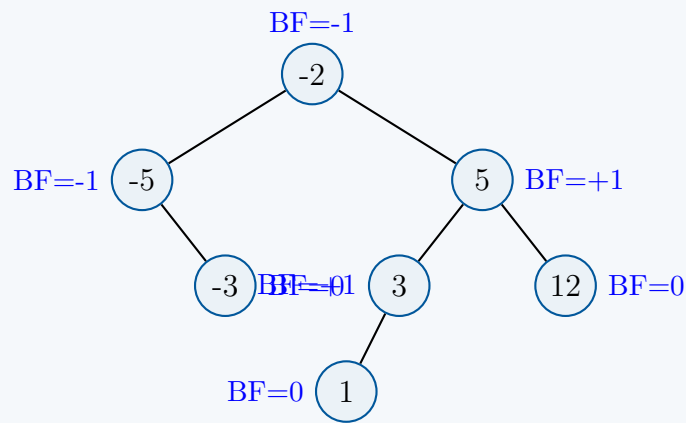


Step 7: Inserting 1 (Final Balanced Tree)

Key 1 is compared with -2 (go right), 5 (go left), and 3 (go left), placing it as the left child of 3.

- We recalculate all structural balance factors: $BF(1) = 0$, $BF(3) = +1$, $BF(5) = 2 - 1 = +1$, $BF(-5) = -1$, and $BF(-2) = 2 - 3 = -1$.
- Every node strictly satisfies the balance condition $|BF| \leq 1$. No further rotations are needed.

Final Resulting AVL Tree



Q21. What is the height of an AVL tree with n internal nodes? (5 marks)

[CSE 203 | L-2/T-1 | 2012–13]

Answer

The height h of an AVL tree with n internal nodes is strictly bounded by **logarithmic time complexity**, specifically $\mathcal{O}(\log_2 n)$. Because AVL trees enforce a strict balance condition (the height difference between left and right subtrees cannot exceed 1), we can define both the best-case (minimum) and worst-case (maximum) height boundaries for a given number of nodes n .

1. Best-Case Height (Perfectly Balanced Tree)

In the best-case scenario, the AVL tree is a perfectly balanced full binary tree. Every level is completely filled.

$$h_{\min} = \lfloor \log_2(n) \rfloor$$

2. Worst-Case Height (Most Unbalanced AVL Tree)

In the worst-case scenario, the AVL tree is as sparse as possible while still maintaining the AVL property. This occurs when every internal node has a balance factor of exactly $+1$ or -1 . Such a tree is called a **Fibonacci Tree**.

Let $N(h)$ be the minimum number of nodes required to form an AVL tree of height h . To build this minimal tree, the root must have one subtree of height $h-1$ and the other of height $h-2$. This gives the recurrence relation:

$$N(h) = N(h-1) + N(h-2) + 1$$

with base cases:

- $N(0) = 0$ (An empty tree)
- $N(1) = 1$ (A tree with just a root)

This recurrence relation is closely related to the Fibonacci sequence. By adding 1 to both sides, we get $N(h) + 1 = (N(h-1) + 1) + (N(h-2) + 1)$, which perfectly matches the standard Fibonacci recurrence F_{h+2} .

Using Binet's formula for the Fibonacci sequence, it can be mathematically proven that the number of nodes grows exponentially with respect to the golden ratio ($\phi \approx 1.618$):

$$N(h) \approx \frac{\phi^{h+2}}{\sqrt{5}} - 1$$

By substituting n for $N(h)$ and solving for h , we get the strict upper bound for the height:

$$\begin{aligned} n &\approx \frac{\phi^{h+2}}{\sqrt{5}} - 1 \\ n+1 &\approx \frac{\phi^{h+2}}{\sqrt{5}} \\ \log_{\phi}(\sqrt{5}(n+1)) &\approx h+2 \end{aligned}$$

Converting the logarithm base from ϕ to 2 (since $\log_2(\phi) \approx 0.694$ and $1/0.694 \approx 1.44$), we obtain the precise mathematical upper bound:

$$h_{\max} \approx 1.44 \log_2(n+2) - 0.328$$

Conclusion

Because the maximum possible height is $1.44 \log_2(n+2)$, an AVL tree's height is mathematically guaranteed to never exceed ≈ 1.44 **times the optimal height**. Thus, the height of an AVL tree is rigorously constrained to $\mathcal{O}(\log n)$.

Q22. Build a 2-3-4 tree with the keys 12, 5, 6, 10, 15, 8, 13, 17, 11, 14, 4. Then delete elements in the following sequence: 4, 12, 13. Show all steps. (25 marks)

[CSE 203 | L-2/T-1 | 2012–13]

Answer: 2-3-4 Tree Construction and Deletion

We will use the standard **proactive top-down** approach for the 2-3-4 tree (B-tree of order 4):

- **Insertion:** Any 4-node encountered on the path down to a leaf is split immediately before descending further.
- **Deletion:** Any 2-node encountered on the path down is refilled via borrowing or merging to prevent underflow at the leaf level.

—

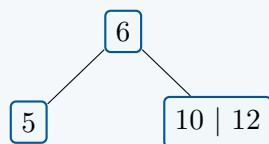
Part 1: Construction of the 2-3-4 Tree**Steps 1–3: Insert 12, 5, 6**

Keys are added sequentially into the single root node until it forms a 4-node:

$$[\text{Empty}] \rightarrow [12] \rightarrow [5 \mid 12] \rightarrow [5 \mid 6 \mid 12] \quad (\text{Node is full})$$

Step 4: Insert 10

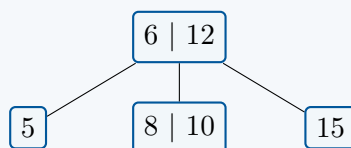
The root $[5 \mid 6 \mid 12]$ is a full 4-node. Before inserting 10, it must be split. The middle key (6) moves up to form a new root, creating two child nodes. Since $10 > 6$, it enters the right child node.

**Step 5: Insert 15**

Key 15 is greater than 6, so it moves to the right child node $[10 \mid 12]$, filling it to maximum capacity: $[10 \mid 12 \mid 15]$.

Step 6: Insert 8

As we descend to insert 8 ($8 > 6$), we encounter the full 4-node $[10 \mid 12 \mid 15]$. It is proactively split: its middle key (12) moves up into the root node. Then, 8 is added to the node containing 10 ($6 < 8 < 12$).

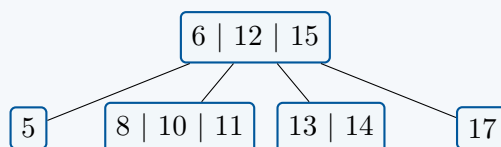
**Steps 7–9: Insert 13, 17, 11**

These values are sequentially placed into their matching leaves without causing structural violations:

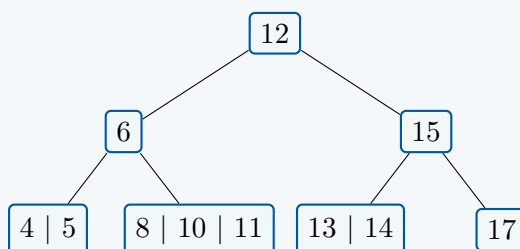
- 13 and 17 are guided to the rightmost leaf node (> 12) $\rightarrow [13 \mid 15 \mid 17]$.
- 11 is guided to the middle leaf node ($6 < 11 < 12$) $\rightarrow [8 \mid 10 \mid 11]$.

Step 10: Insert 14

Descending to place 14 ($14 > 12$), we encounter the full child node $[13 \mid 15 \mid 17]$. It splits, moving 15 up into the root. Since $12 < 14 < 15$, 14 joins the left split child containing 13.

**Step 11: Insert 4 (Final Constructed Tree)**

Before descending, we observe that the root node $[6 \mid 12 \mid 15]$ is completely full. It must be split proactively. The middle key 12 moves up to establish a new level. Following this split, 4 safely descends to the leftmost leaf node.

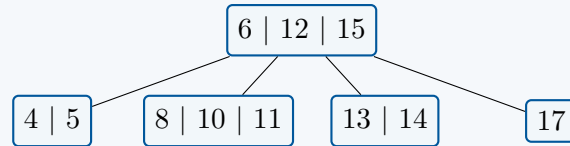


—

Part 2: Deletion Operations

1. Delete 4

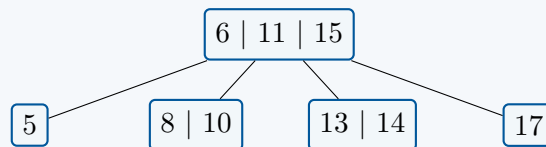
- **Path Adjustments:** We begin at the root node [12]. Its children ([6] and [15]) are both 2-nodes. To prevent potential down-path structural underflows, the top-down rule dictates that we must merge the root with both children, compressing the tree back down by one level.
- **Tree Configuration after Merge:** The tree becomes a single root node with 4 children:



- **Execution:** Now we target the leaf node [4 | 5]. Since it contains more than one key, 4 can be safely deleted directly without causing an underflow.

2. Delete 12

- **Path Adjustments:** The key 12 is located inside an internal node ([6 | 12 | 15]). To delete an internal key, we must locate its **in-order predecessor** (the largest key in its left subtree).
- **Execution:** The left subtree pointer leads to node [8 | 10 | 11]. This node is a 4-node (containing 3 keys), meaning it is completely safe to borrow from. The in-order predecessor is 11. We copy 11 up to replace 12 in the root node, and then safely erase 11 from its original leaf node.

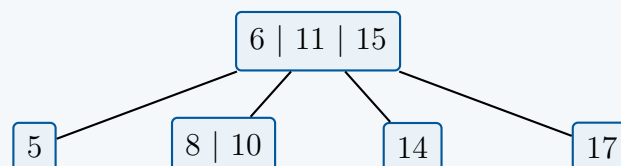


3. Delete 13

- **Path Adjustments:** We trace the path down from the root [6 | 11 | 15] to locate key 13. The path guides us to the child node [13 | 14].
- **Execution:** Since this leaf node is a 3-node (holding 2 keys), it can withstand a deletion without risking an underflow. We remove 13 directly.

Final State of the 2-3-4 Tree

The final structural state of the tree after completing all insertions and deletions is:



Red-Black Trees

- Q1.** Starting from an empty red-black tree, perform the insertion operation of the keys 41, 38, 31, 12, 19, 8, 25, and 32 in the given order while maintaining all red-black properties. After completing all insertions, use the resulting tree and perform deletions of the keys 8, 12, and 19 in that order. During the insertion or deletion, you must show the tree after each operation, including the color of every node. **(25 marks)** [CSE 207 | L-2/T-1 | 2024-25]

Answer

Red-Black Tree Properties Maintained:

- Every node is either Red or Black.
- The root is Black.
- Every leaf (NIL) is Black.
- If a node is Red, both of its children are Black (no two adjacent Red nodes).
- Every path from a given node to any of its descendant NIL leaves contains the same number of Black nodes.



Part 1: Insertions

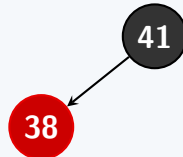
Step 1: Insert 41

Insert 41 as a Red node. Since it is the root of the tree, its color is changed to Black to satisfy the root property.



Step 2: Insert 38

Insert 38 as a Red node. It becomes the left child of 41. No properties are violated.

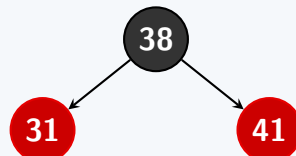


Step 3: Insert 31

Insert 31 as a Red left child of 38.

Violation: Node 38 is Red and 31 is Red (Adjacent Red nodes).

Resolution: The uncle of 31 is NIL (Black). Since 31 and 38 are both left children, this is the **Left-Left** case. We perform a Right Rotation around the grandparent (41) and recolor: the original parent (38) becomes Black, and the grandparent (41) becomes Red.

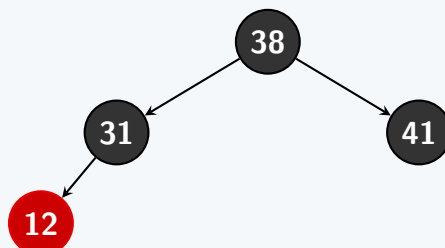


Step 4: Insert 12

Insert 12 as a Red left child of 31.

Violation: Node 31 is Red and 12 is Red.

Resolution: The uncle of 12 is 41, which is Red. We recolor the parent (31) and uncle (41) to Black, and the grandparent (38) to Red. Since 38 is the root, it is immediately recolored back to Black.



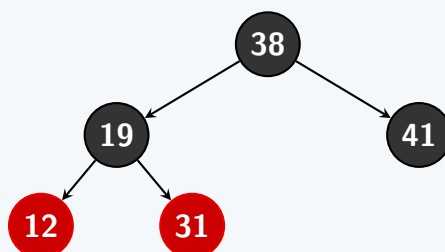
Step 5: Insert 19

Insert 19 as a Red right child of 12.

Violation: Node 12 is Red and 19 is Red.

Resolution: The uncle of 19 is NIL (Black). This is the **Left-Right** case.

- First, perform a Left Rotation around 12 to convert it to a Left-Left case. Node 12 becomes the left child of 19.
- Next, perform a Right Rotation around the grandparent 31. Recolor the new parent 19 to Black and the original grandparent 31 to Red.

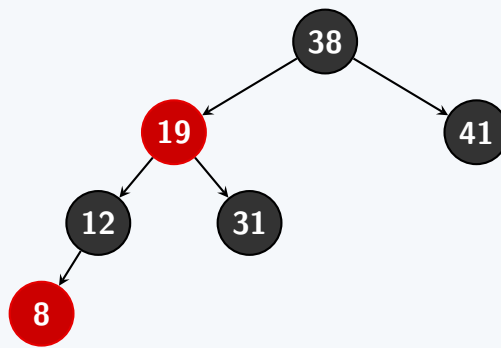


Step 6: Insert 8

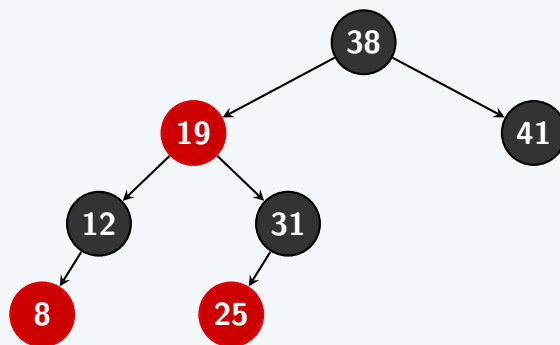
Insert 8 as a Red left child of 12.

Violation: Node 12 is Red and 8 is Red.

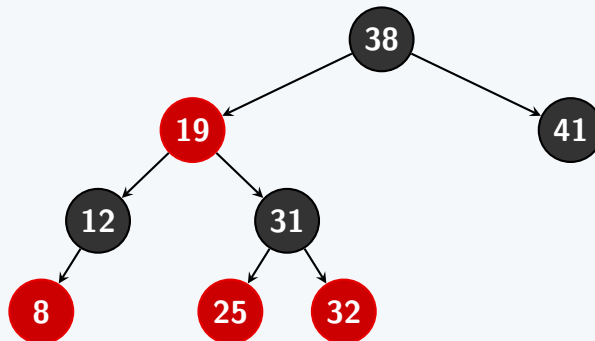
Resolution: The uncle of 8 is 31 (Red). We recolor the parent (12) and uncle (31) to Black, and the grandparent (19) to Red. The parent of 19 is 38 (Black), so no further violations occur.

**Step 7: Insert 25**

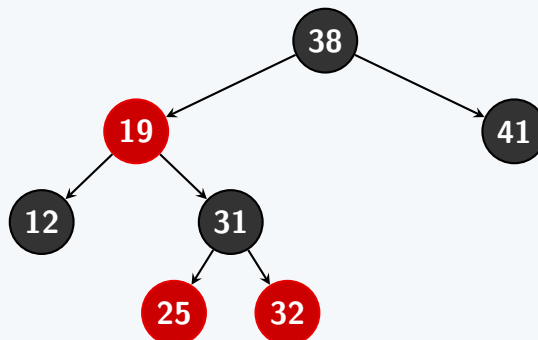
Insert 25 as a Red left child of 31. Its parent 31 is Black, so no properties are violated.

**Step 8: Insert 32**

Insert 32 as a Red right child of 31. Its parent 31 is Black, so no properties are violated.

**Part 2: Deletions****Step 9: Delete 8**

Node 8 is a Red leaf. We can simply remove it without violating any Red-Black Tree properties (the black-height remains unchanged).

**Step 10: Delete 12**

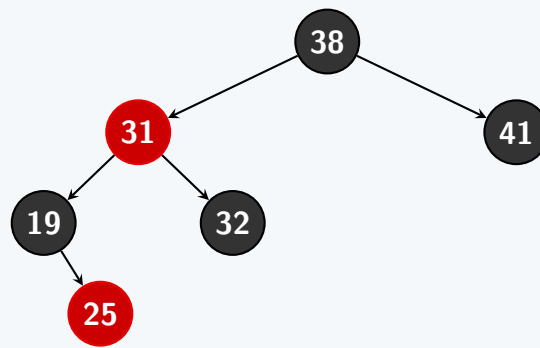
Node 12 is a Black leaf. Removing it leaves a Double-Black NIL node in its place (let's call it N).

- N is the left child of its parent 19 (Red).
- The sibling of N is 31 (Black).
- The sibling W (31) has two Red children, 25 and 32.

Resolution: Because the sibling W is Black and its "far" child (32, the right child) is Red, this is the **Right-Right Case**. We apply the following steps:

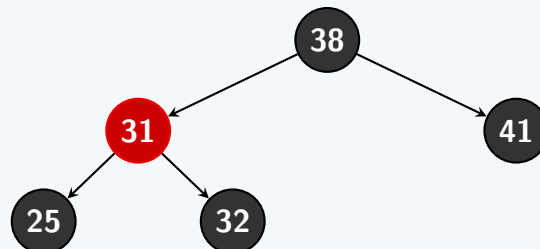
- Recolor the sibling (31) to the parent's color (Red).
- Recolor the parent (19) to Black.
- Recolor the far child (32) to Black.
- Perform a Left Rotation around the parent (19).

After the rotation, the Double-Black node N loses its extra black weight and is safely removed.

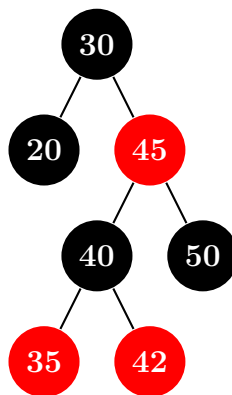
**Step 11: Delete 19**

Node 19 is a Black node with a single Red child (25).

Resolution: We remove 19 and replace it with its child 25. To maintain the black-height along the path, we simply recolor the new replacing node 25 to Black.



Q2. Delete the node 50 from the given Red-Black Tree. Note that doubly circled nodes represent Red and single circles represent Black.

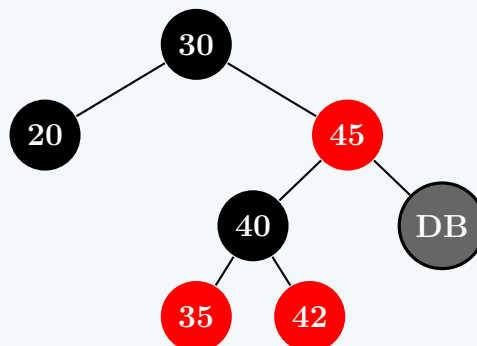


(10 marks)

[CSE 207 | L-2/T-1 | 2023–24]

Answer**Step 1: Initial Deletion and Double Black Node**

Node 50 is a **Black** leaf node. When it is deleted, its position is replaced by a NIL leaf. To maintain the black-height property of the tree, this NIL node absorbs the black weight of Node 50, becoming a **Double Black** (DB) node.

**Step 2: Analysis of the Double Black Node**

We evaluate the structural relationships around the Double Black node (DB):

- **Parent (P):** Node 45, which is **Red**.
- **Sibling (W):** Node 40, which is **Black**. (Since DB is a right child, the sibling is the left child).
- **Sibling's Far Child:** Node 35 (the left child of 40), which is **Red**.
- **Sibling's Near Child:** Node 42 (the right child of 40), which is **Red**.

Since the sibling is Black and its *far child* is Red, this corresponds to **Case 4** of Red-Black Tree deletion.

Step 3: Resolving Case 4

To resolve this violation, we apply the following operations:

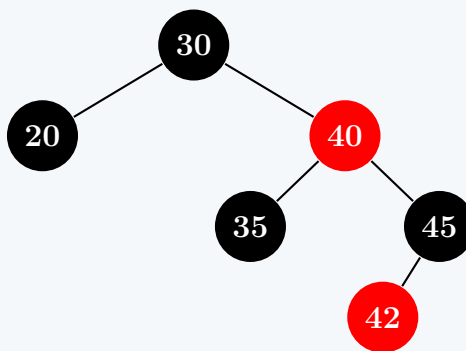
- Recolor Sibling:** Change the color of the sibling (40) to the parent's color (**Red**).
- Recolor Parent:** Change the color of the parent (45) to **Black**.
- Recolor Far Child:** Change the color of the far child (35) to **Black**.

(d) **Rotation:** Perform a **Right Rotation** around the parent (45).

Step 4: Right Rotation around 45

- Node 40 moves up to take the place of 45.
- Node 45 becomes the right child of 40.
- Node 42 (which was the right child of 40) becomes the left child of 45.
- The extra black weight is stripped away, and the tree is perfectly balanced again.

Final Red-Black Tree:



Complexity Analysis:

- **Time Complexity:** $\mathcal{O}(1)$ for the deletion and restructuring (constant number of recolorings and 1 rotation) once the node is found. Overall deletion time is $\mathcal{O}(\log n)$ where n is the number of nodes.
- **Space Complexity:** $\mathcal{O}(1)$ auxiliary space used for pointers and recoloring.

Q3. Write the pseudo-code of an insertion operation in a Red-Black tree and explain (graphically) different forms of transformation necessary to maintain the RB property while inserting an item. Briefly compare RB-tree and AVL-tree. **(15 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Answer

1. Pseudo-code for Insertion in a Red-Black Tree

The insertion in a Red-Black Tree follows the standard Binary Search Tree (BST) insertion, where the new node is always colored **RED**. After the insertion, a fix-up procedure is called to restore any violated Red-Black Tree properties (primarily, the rule that no two adjacent nodes can be red).

Algorithm 38 RB-INSERT(T, z)

```

1:  $y \leftarrow T.nil$ 
2:  $x \leftarrow T.root$ 
3: while  $x \neq T.nil$  do
4:    $y \leftarrow x$ 
5:   if  $z.key < x.key$  then
6:      $x \leftarrow x.left$ 
7:   else
8:      $x \leftarrow x.right$ 
9:   end if
10: end while
11:  $z.p \leftarrow y$ 
12: if  $y == T.nil$  then
13:    $T.root \leftarrow z$ 
14: else if  $z.key < y.key$  then
15:    $y.left \leftarrow z$ 
16: else
17:    $y.right \leftarrow z$ 
18: end if
19:  $z.left \leftarrow T.nil$ 
20:  $z.right \leftarrow T.nil$ 
21:  $z.color \leftarrow RED$ 
22: RB-INSERT-FIXUP( $T, z$ )
  
```

Algorithm 39 RB-INSERT-FIXUP(T, z)

```

1: while  $z.p.color == \text{RED}$  do
2:   if  $z.p == z.p.p.left$  then                                     ▷ Parent is a left child
3:      $y \leftarrow z.p.p.right$                                        ▷  $y$  is the uncle of  $z$ 
4:     if  $y.color == \text{RED}$  then                                     ▷ Case 1: Uncle is RED
5:        $z.p.color \leftarrow \text{BLACK}$ 
6:        $y.color \leftarrow \text{BLACK}$ 
7:        $z.p.p.color \leftarrow \text{RED}$ 
8:        $z \leftarrow z.p.p$ 
9:     else
10:      if  $z == z.p.right$  then                                     ▷ Case 2: Uncle is BLACK,  $z$  is a right child
11:         $z \leftarrow z.p$ 
12:        LEFT-ROTATE( $T, z$ )
13:      end if
14:       $z.p.color \leftarrow \text{BLACK}$                                      ▷ Case 3: Uncle is BLACK,  $z$  is a left child
15:       $z.p.p.color \leftarrow \text{RED}$ 
16:      RIGHT-ROTATE( $T, z.p.p$ )
17:    end if
18:  else
19:    (Symmetric to the then clause with "left" and "right" exchanged)
20:  end if
21: end while
22:  $T.root.color \leftarrow \text{BLACK}$ 

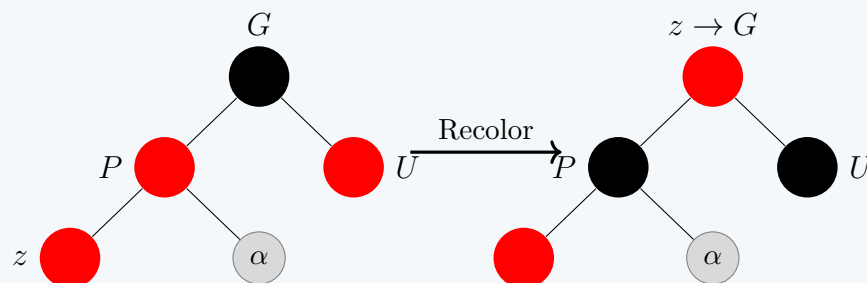
```

2. Graphical Explanation of Transformations

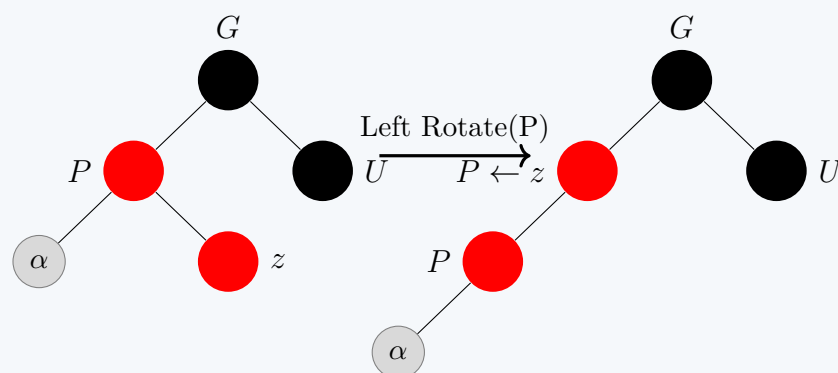
We assume the new node z 's parent (P) is the left child of its grandparent (G). The right child scenario is entirely symmetric.

Case 1: Uncle is RED (Recoloring)

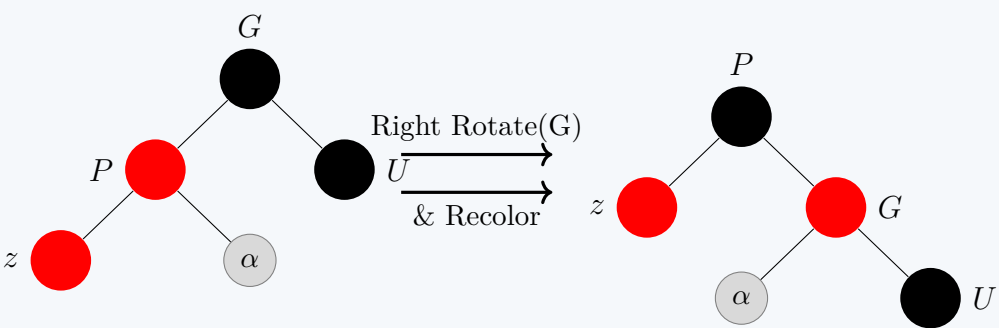
If the uncle (U) is red, we simply recolor the parent (P) and uncle (U) to **BLACK**, and the grandparent (G) to **RED**. The pointer z is moved to G to check for further violations up the tree.

**Case 2: Uncle is BLACK, z is a Right Child (Left Rotation)**

If the uncle is black and z forms a "zig-zag" shape (Left-Right), we perform a **Left Rotation** on the parent P . This transforms the problem into Case 3 without fixing the red-red violation yet.

**Case 3: Uncle is BLACK, z is a Left Child (Right Rotation & Recolor)**

If the uncle is black and z forms a "straight" shape (Left-Left), we perform a **Right Rotation** on the grandparent G . Then, we recolor the original parent P to **BLACK** and the old grandparent G to **RED**. This fixes the local violation and ensures the black height remains unchanged.



3. Comparison between Red-Black Tree and AVL Tree

Both Red-Black Trees and AVL Trees are self-balancing Binary Search Trees that guarantee $\mathcal{O}(\log n)$ time for insertion, deletion, and search. However, they employ different balancing criteria:

Feature	Red-Black Tree	AVL Tree
Balancing Rule	Loosely balanced. The longest path from root to leaf is at most twice the shortest path.	Strictly balanced. The difference between heights of left and right subtrees is at most 1.
Lookup Speed	Fast $\mathcal{O}(\log n)$, but slightly slower than AVL due to lesser strictness in height.	Faster $\mathcal{O}(\log n)$ due to strictly shorter maximum height.
Insertion & Deletion	Faster. Requires at most 2 rotations for insertion and 3 for deletion. Changes mostly involve $\mathcal{O}(1)$ recoloring.	Slower. Deletion may require $\mathcal{O}(\log n)$ rotations propagating all the way up to the root.
Memory Space	Minimal overhead. Requires only 1 extra bit per node (to store Red/Black color).	Higher overhead. Requires an integer per node to store the height or balance factor.
Typical Use Cases	Preferred for write-intensive applications (e.g., <code>std::map</code> in C++, Completely Fair Scheduler in Linux).	Preferred for read-intensive applications where searches are much more frequent than insertions/deletions.

Q4. What are the properties of a red-black tree? Derive that if a tree with n keys follows the red-black properties, its height h can be bounded as $h \leq 2 \lg(n + 1)$. (15 marks) [CSE 207 | L-2/T-2 | 2021–22]

Answer: Red-Black Tree Properties & Height Bound Derivation

1. Properties of a Red-Black Tree

A **Red-Black Tree** is a self-balancing binary search tree where each node contains an extra attribute denoting its color (either **red** or **black**). To guarantee approximately balanced structure, a valid Red-Black tree must satisfy the following five invariants:

- (a) **Color Property:** Every node in the tree is colored either red or black.
- (b) **Root Property:** The root node is always black.
- (c) **Leaf Property:** Every leaf node (NIL or NULL pointer) is considered black.
- (d) **Red-Node Property:** If a node is red, then both of its children must be black. (This strictly ensures there are never two adjacent red nodes on any simple path).
- (e) **Black-Height Property:** For any given node, all simple paths from that node down to its descendant leaves contain exactly the same number of black nodes.

2. Derivation of the Height Bound: $h \leq 2 \lg(n + 1)$

Let h be the height of a red-black tree, and let n be the total number of internal nodes (keys). We want to mathematically prove that the maximum height is bounded by $2 \lg(n + 1)$.

Step 1: Define Black-Height

Let $bh(x)$ denote the **black-height** of a node x . This is defined as the number of black nodes on any path from x down to a leaf, *excluding* the node x itself. By Property 5, this value is uniform for all paths originating from x .

Step 2: Lemma on Subtree Size

Lemma: *The subtree rooted at any node x contains at least $2^{bh(x)} - 1$ internal nodes.*

Proof by Induction on the height of x :

- **Base Case:** If x has a height of 0, it is a NIL leaf. Its black-height is $bh(x) = 0$, and the number of internal nodes is 0. The formula yields $2^0 - 1 = 1 - 1 = 0$. The base case holds.
- **Inductive Step:** Consider an internal node x with positive height and two children. Each child's black-height is either $bh(x)$ (if the child is red) or $bh(x) - 1$ (if the child is black). Therefore, the black-height of each child is *at least* $bh(x) - 1$.
- By our inductive hypothesis, each child's subtree contains at least $2^{bh(x)-1} - 1$ internal nodes.
- The total number of internal nodes in the subtree rooted at x is the sum of the nodes in its two subtrees plus 1 (for x itself):

$$\begin{aligned} \text{Total Nodes} &\geq (2^{bh(x)-1} - 1) + (2^{bh(x)-1} - 1) + 1 \\ &= 2 \cdot (2^{bh(x)-1}) - 2 + 1 \\ &= 2^{bh(x)} - 1 \end{aligned}$$

This completes the proof of the lemma.

Step 3: Relate Black-Height to Tree Height

Now, consider the root node of the tree. Let the overall height of the tree be h .

Because of Property 4 (no consecutive red nodes), at most half of the nodes on any simple path from the root to a leaf can be red. Consequently, *at least half* of the nodes on any such path must be black. Therefore, the black-height of the root must be at least half the total height:

$$bh(\text{root}) \geq \frac{h}{2}$$

Step 4: Final Algebraic Derivation

Applying our lemma (from Step 2) directly to the root node, the total number of internal nodes n in the entire tree must satisfy:

$$n \geq 2^{bh(\text{root})} - 1$$

Substitute the inequality from Step 3 into this equation:

$$n \geq 2^{h/2} - 1$$

Now, solve for h . First, add 1 to both sides:

$$n + 1 \geq 2^{h/2}$$

Take the base-2 logarithm ($\lg$) of both sides:

$$\lg(n + 1) \geq \frac{h}{2}$$

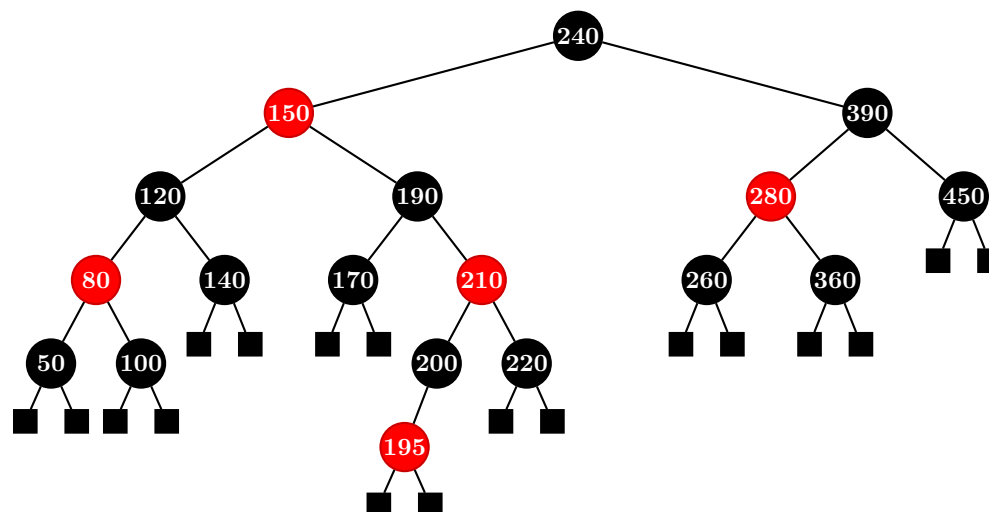
Multiply by 2:

$$2\lg(n + 1) \geq h$$

Conclusion

Rewriting the inequality, we get $h \leq 2\lg(n + 1)$. This proves that the maximum height of a Red-Black tree is logarithmic with respect to the number of nodes, guaranteeing that operations like search, insertion, and deletion strictly run in $\mathcal{O}(\lg n)$ time.

Q5. Usually we have to fix a Red-Black tree after deleting a node when the properties of the Red-Black tree get violated because of node deletion. Explain whether it is possible to do so without performing any rotation. Delete the node with key = 150 from the given Red-Black tree. Show all the steps and the resulting Red-Black tree.



(20 marks)

[CSE 207 | L-2/T-2 | 2020–21]

Answer: Red-Black Tree Deletion Analysis and Execution

1. Restoring Balance Without Rotations: Is It Possible?

Yes, it is entirely possible to fix a Red-Black tree after node deletion without performing any tree rotations under specific conditions.

When a node is deleted, a property violation (specifically an underflow of the black-height property) only occurs if the deleted physical node was **black**. If a structural violation does happen, the algorithm attempts to fix it by traversing up toward the root. Rotations can be completely avoided if the structural imbalance is resolved exclusively via one of the following scenarios:

- **Scenario A: The Deleted Node was Red.** If the physically removed node was Red, no properties are violated. The black-height remains perfectly consistent, and no red-red violations are created.
- **Scenario B: Color Recoloring (Case 2 of RB Deletion).** If the extra black (double-black) node has a black sibling, and **both children of that sibling are black**, the problem can be pushed up the tree simply by recoloring the sibling to **red** and shifting the double-black status to the parent node. If the parent node was red, it simply absorbs the double-black status to become a regular black node, terminating the fixup process without a single rotation.
- **Scenario C: Double-Black reaches the Root.** If the pushed double-black status climbs all the way up to the root of the entire tree, it is simply converted into a regular black node. This shrinks the overall black-height of the tree by 1 symmetrically, requiring no rotations.

2. Step-by-Step Deletion of Key = 150

Step 2.1: Standard BST Deletion Phase

Key **150** is located at an internal node. Following standard Binary Search Tree (BST) rules, we cannot delete it directly. Instead, we must replace its value with either its in-order predecessor or its in-order successor.

- Let us select its **in-order predecessor** (the maximum key in its left subtree).
- Tracking down the rightmost path of node 120's subtree leads directly to node **140**.
- Node 140 is a black leaf node whose children are both standard **NIL** leaves.
- We overwrite the key value of our target node (150) with the replacement value **140**.

At this stage, the problem reduces down to physically deleting the old node **140** from its original position at the bottom of the tree.

Step 2.2: Red-Black Fixup Phase

Because the physical node being deleted (140) is **black**, its removal creates a black-height deficit along that path. The child replacing it (a black **NIL** leaf) becomes a **double-black node**, which we will denote as **X**.

Let us analyze the structural relationship surrounding the double-black node **X**:

- Identify the Parent (P):** The parent of **X** is node **120 (Black)**.
- Identify the Sibling (S):** The sibling of **X** is node **80 (Red)**.

Since the sibling **S** is **Red**, this matches **Case 1 of the standard Red-Black Deletion algorithm**. We must transform this case into Case 2, 3, or 4 by executing the following steps:

- Change the color of sibling 80 from Red to **Black**.
- Change the color of parent 120 from Black to **Red**.
- Perform a **Single Right Rotation** around parent node 120.

This structural transformation alters the local neighborhood layout: node 80 becomes the new parent of node 120. The double-black node **X** is now a child of node 120. Let us re-examine the properties from this new position:

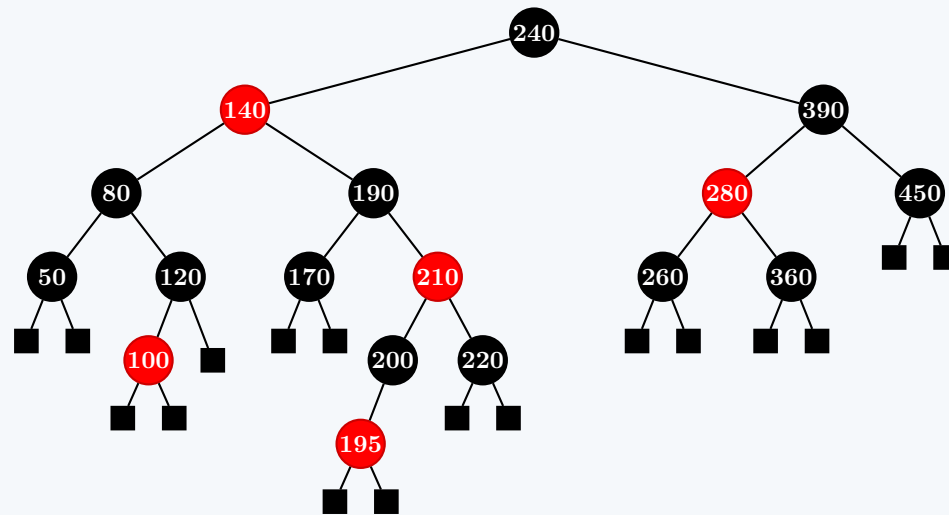
- New Parent of **X** is node **120 (Red)**.
- New Sibling of **X** is node **100 (Black)**.
- Both children of the new sibling 100 are black **NIL** leaves.

This matches **Case 2 of Red-Black Deletion** (Sibling is black and both of its children are black):

- We change the color of sibling 100 from Black to **Red**.
- The extra black property shifts up to the parent node 120.
- Since parent 120 was **Red**, it absorbs the double-black completely and turns into a standard **Black** node.
- The tree is now perfectly balanced, satisfying all invariants. The execution terminates.

3. Final Resulting Red-Black Tree Diagram

Below is the complete, valid configuration of the Red-Black tree after completing the structural adjustment steps:



Q6. Mr. X has proposed a new algorithm for insertion to a Red-Black tree, where the color of the newly inserted node is set to black by default. Discuss the advantages and disadvantages of this algorithm over the traditional algorithm followed for insertion to a Red-Black tree. (5 marks) [CSE 207 | L-2/T-2 | 2020–21]

Answer: Comparative Analysis of Default-Black Insertion Algorithm

In the traditional Red-Black Tree insertion algorithm, a newly inserted node is always colored **Red** by default. Mr. X's alternative proposal suggests coloring the newly inserted node **Black** by default.

Below is a detailed analysis of the advantages and disadvantages of this proposed approach compared to the traditional mechanism.

1. Advantages of Mr. X's Algorithm

While setting a new node to black is highly disruptive to the tree's structural balance overall, it does provide a couple of niche algorithmic shortcuts:

- **Elimination of Red-Red (Property 4) Violations:** In the traditional algorithm, inserting a red node under an existing red parent creates a structural violation of Property 4 (no consecutive red nodes). By defaulting the new node to Black, it is mathematically impossible to introduce two adjacent red nodes. Thus, the algorithm completely bypasses the need to check or fix red-red parenting violations at the leaf level.
- **Trivial Insertion Under Red Nodes:** If the parent of the newly inserted node happens to be Red, inserting a Black child is completely free of violations. Property 4 is satisfied, and because the parent is red, its other sibling branch must already have an equivalent black-height, meaning the local black-height properties often require simpler, localized adjustments.

2. Disadvantages of Mr. X's Algorithm

The disadvantages of Mr. X's algorithm heavily outweigh the benefits, primarily because it attacks the most rigid constraint of a Red-Black Tree: the Black-Height Property.

- **Immediate and Constant Violation of the Black-Height Property (Property 5):** Property 5 states that every simple path from a node to its descendant leaves must contain the exact same number of black nodes.
 - In the **traditional algorithm** (default Red), adding a red node adds 0 to the black-height of that path. Thus, Property 5 is **never violated upon insertion**. The tree only has to fix local red-red violations.
 - In **Mr. X's algorithm** (default Black), adding a black node instantly increases the black-height of that specific path by 1, while all sibling paths remain unchanged. Therefore, **every single insertion unconditionally violates Property 5**.
- **High Repair Complexity (Global vs. Local Fixes):** Fixing a red-red violation (traditional) is an isolated, local event that can often be resolved via a single rotation or a quick recoloring of immediate uncles/parents. Conversely, fixing a black-height violation (Mr. X's proposal) is much more severe. It behaves exactly like the dreaded **Double-Black underflow problem encountered during deletion**. Resolving it requires propagating black-height adjustments up toward the root, leading to significantly higher frequencies of structural rebalancing and pointer manipulations.
- **Frequent Tree Compaction (Root Level Growth):** Because adding a black node forces paths to become heavier, balancing the tree requires either pulling black nodes down (which is difficult without violating other paths) or pushing the extra black properties upward until they reach the root. This causes the tree's overall black-height to increase much faster than necessary, leading to a tighter, more compressed structure that requires excessive restructuring operations.

3. Summary Comparison Table

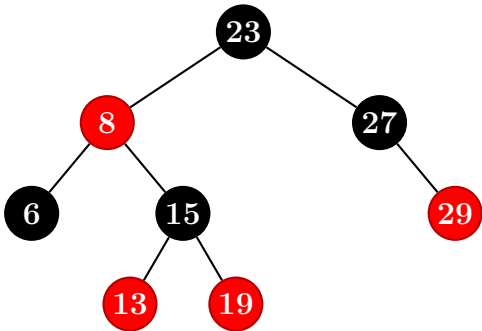
To summarize, we can compare the behavioral invariants of both strategies immediately after a node is physically linked to the tree, but before any balancing loops execute:

Metric / Property	Traditional Insertion (Default Red)	Mr. X's Insertion (Default Black)
Property 4 (No Red-Red)	May be violated (if parent is Red). Easily fixed locally.	Guaranteed to be satisfied. No red-red violations possible.
Property 5 (Black-Height)	Guaranteed to be satisfied. The path's black count does not change.	Symmetrically broken. The insertion path is instantly heavier than its sibling paths.
Fixup Paradigm	Localized adjustments handling color collisions.	Complex structural propagation similar to handling deletion underflows.
Overall Efficiency	Highly Optimal. Average case requires $\mathcal{O}(1)$ recolorings and rare rotations.	Inefficient. Requires heavy, persistent top-down rebalancing routines.

Conclusion

Mr. X's algorithm is **not recommended**. Swapping a temporary red-red violation for an immediate black-height violation replaces an easily patchable local color issue with a fundamental structural deficit, making the tree significantly more expensive to maintain.

Q7. Insert 11 into the Red-Black tree shown in the figure. You have to show the intermediate steps. The red and black nodes are indicated by 'r' and 'b', respectively.



(15 marks)

[CSE 207 | L-2/T-2 | 2019–20]

Answer: Red-Black Tree Insertion of Key 11

Here are the intermediate steps to insert the key 11 into the given Red-Black tree.

Step 1: Standard BST Insertion

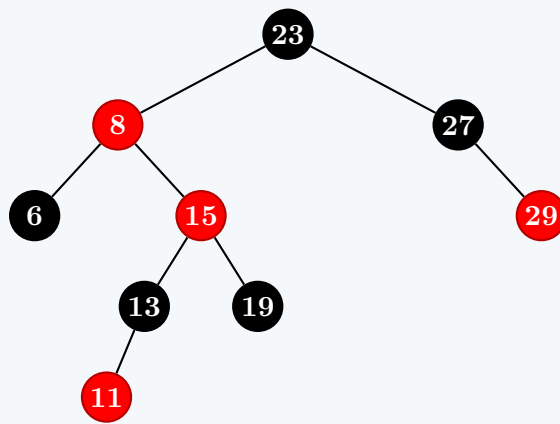
We insert 11 following standard Binary Search Tree rules ($11 < 23 \rightarrow \text{left}$, $11 > 8 \rightarrow \text{right}$, $11 < 15 \rightarrow \text{left}$, $11 < 13 \rightarrow \text{left}$). By default, the newly inserted node is colored **Red**.

Violation: There is a Red-Red conflict between 13 and 11.

Step 2: Case 1 Recoloring (Red Uncle)

Looking at node 11, its parent (13) is Red, and its uncle (19) is also **Red**.

- Recolor the parent (13) and uncle (19) to **Black**.
- Recolor the grandparent (15) to **Red**.
- Move the inspection pointer up to the grandparent (15).

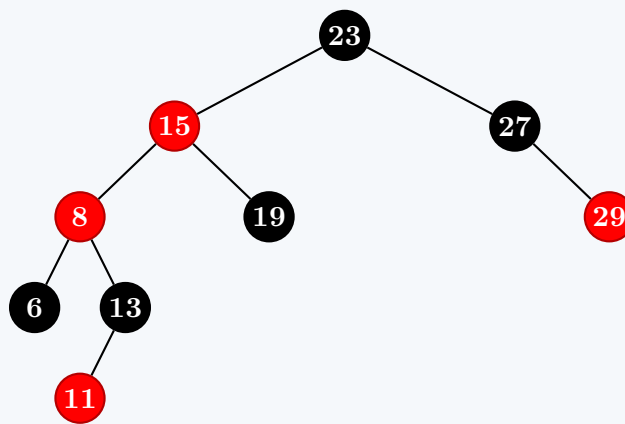


Violation: A new Red-Red conflict has emerged between 8 and 15.

Step 3: Case 2 Left Rotation (Black Uncle, Triangle Shape)

Now examining node 15: its parent (8) is Red, but its uncle (27) is **Black**. Node 15 forms a "Right" child to a "Left" child parent (a zig-zag shape).

- Perform a **Left Rotation** on the parent (8).
- Node 15 moves up, 8 becomes its left child, and 13 shifts over to become the right child of 8.

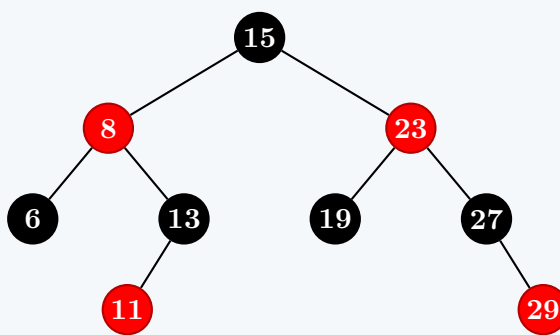


Note: The Red-Red violation remains between 15 and 8, but they now form a straight line, setting up Case 3.

Step 4: Case 3 Right Rotation & Recolor (Black Uncle, Line Shape)

Examining node 8: it forms a straight "Left-Left" line with its parent (15) and grandparent (23).

- Perform a **Right Rotation** on the grandparent (23).
- Swap the colors of the new root of this subtree (15) and the old root (23). Node 15 becomes **Black**, and 23 becomes **Red**.



Final State: All Red-Black tree properties are now fully restored and valid.

Q8. When we insert a node into a red-black tree, we initially set the color of the new node to red. Why don't we choose to set the color to black? **(15 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Answer: Why New Nodes are Colored Red

When inserting a new node into a Red-Black tree, we must choose to color it either Red or Black. The decision to always default to **Red** is strategically based on choosing the "lesser of two evils" regarding tree property violations.

1. The Consequence of Inserting a Red Node

If we color the newly inserted node **Red**:

- **Black-Height is Preserved:** The number of black nodes on the path to the newly inserted leaf remains completely unchanged. Therefore, **Property 5** (every path must have the same number of black nodes) is strictly satisfied. This is the most rigid and difficult property to maintain in the tree.
- **Potential Red-Red Violation:** The only property we might violate is **Property 4** (no consecutive red nodes). This only occurs if the newly inserted node's parent is also Red.
- **Simple, Localized Fix:** A Red-Red violation is a *strictly local* conflict. It can be resolved quickly by examining the immediate neighborhood (specifically, the uncle node) and performing a simple color flip or at most two structural rotations.

2. The Consequence of Inserting a Black Node

If we were to color the newly inserted node **Black**:

- **No Red-Red Conflicts:** We would never violate Property 4, as a black node can safely be a child of either a red or black parent.
- **Immediate Black-Height Violation:** Adding a black node instantly increases the black-height of that specific downward path by 1, while all other sibling paths in the tree remain untouched. This **unconditionally violates Property 5** on every single insertion.
- **Complex, Global Fix:** Fixing an unequal black-height is a *global* structural issue. It forces the algorithm to push the "extra black" weight up the tree level by level until it can be distributed evenly. This fundamentally mimics the complex, multi-case "Double-Black" fixup routine required during node deletion, making insertions drastically more expensive.

Summary

We set the initial color to **Red** because fixing a localized color conflict (a Red-Red violation) is computationally and structurally much cheaper than rebalancing the global black-height of the entire tree on every single insertion.

Q9. Would inserting a new node to a red-black tree and then immediately deleting it change the tree? Explain with an example. (15 marks) [CSE 207 | L-2/T-2 | 2018–19]

Answer: Effect of Immediate Insertion and Deletion in a Red-Black Tree

Yes, inserting a new node into a Red-Black tree and then immediately deleting it **can permanently change the tree**. It can alter both the color configuration of the nodes and the actual structural topology (the shape and pointer connections) of the tree.

Why Does This Happen?

This structural asymmetry occurs because the insertion and deletion algorithms in a Red-Black tree operate under completely distinct sets of balancing rules:

- **Insertion** fixes *Red-Red violations* (consecutive red nodes) using localized color flips or rotations determined by the color of the node's **uncle**.
- **Deletion** fixes *Black-Height deficits* (double-black nodes) using structural rebalancing determined by the properties of the node's **sibling** and **nephews**.

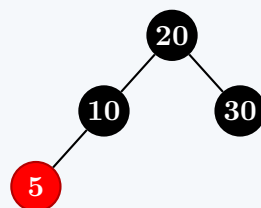
Because deletion is not a mathematical "inverse" or a simple "undo" function of insertion, the structural modifications triggered during insertion are frequently left intact or transformed into an entirely new state during deletion.

Illustrative Example

Let us demonstrate a case where both the **structure** and **colors** of the tree are permanently altered.

1. Initial Valid Red-Black Tree

Consider this initial, perfectly balanced Red-Black tree with a black-height of 2:



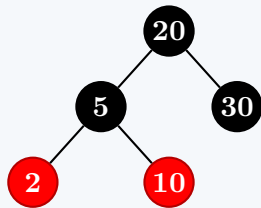
2. Step A: Insert Key 2

We insert the key **2** as a Red node according to standard BST rules. It becomes the left child of node 5.

- This creates an immediate **Red-Red violation** between node 5 and node 2.
- The uncle of node 2 (the right child of 10) is a black NIL leaf.

- Since nodes $10 \rightarrow 5 \rightarrow 2$ form a straight "Left-Left" line, we trigger a **Right Rotation around node 10** and swap the colors of the parent (5) and grandparent (10).

Following this insertion and rebalancing, the tree changes to:



3. Step B: Immediately Delete Key 2

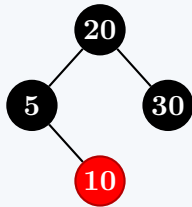
Now, we immediately perform a deletion on the newly inserted key 2.

- Node 2 is currently a **Red leaf node**.
- Under standard Red-Black deletion rules, removing a Red leaf node is trivial: it is simply unlinked from the tree. Because it contains no black property weight, its removal does **not** cause a black-height deficit along that path.
- As a result, **no further structural fixups or rotations are performed**.

—

Final State vs. Initial State

After deleting key 2, the final configuration of the tree is:



Feature	Initial Tree Configuration	Final Tree Configuration (After Insert + Delete)
Left Child of 20	Node 10 (Colored Black)	Node 5 (Colored Black)
Subtree Layout	Node 5 was the <i>left</i> child of node 10	Node 10 is now the <i>right</i> child of node 5
Color of Node 10	Black	Red

Conclusion: The example clearly proves that an immediate insertion-deletion sequence does not act as a net-zero operation; the final tree is structurally and colorwise distinct from the original tree.

Q10. What is the ratio between the lengths of the longest path and the shortest path in a red-black tree? (15 marks)

[CSE 207 | L-2/T-2 | 2018–19]

Answer: Ratio of Longest to Shortest Path in a Red-Black Tree

The ratio between the length of the longest path and the shortest path from the root to any descendant leaf in a Red-Black tree is **at most 2**.

Below is the formal mathematical derivation based on the structural invariants of Red-Black trees.

1. Defining the Invariants

Let bh represent the **black-height** of the tree, which is defined as the number of black nodes on any simple path from a node down to a leaf. According to the **Black-Height Property (Property 5)**, this value is strictly uniform for all paths originating from the same node.

2. Bounding the Path Lengths

The Shortest Possible Path

To minimize the total length of a path, it should contain the absolute minimum number of red nodes possible (0). A path consisting entirely of black nodes will match the black-height exactly:

Shortest Path Length = bh

The Longest Possible Path

To maximize the total length of a path, we must intersperse as many red nodes into the path as allowed. However, the **Red-Node Property (Property 4)** strictly dictates that a red node cannot have a red child (no consecutive red nodes). Therefore, to maximize length, red nodes must perfectly alternate with black nodes (e.g., Black $\rightarrow$ Red $\rightarrow$ Black $\rightarrow$ Red). Since the path is mathematically bound to contain exactly bh black nodes to satisfy Property 5, it can accommodate at most bh red nodes

between them.

$$\text{Longest Path Length} = bh \text{ (black nodes)} + bh \text{ (red nodes)} = 2bh$$

3. Deriving the Ratio

Taking the ratio of the maximum possible path length to the minimum possible path length yields:

$$\begin{aligned} \text{Maximum Ratio} &= \frac{\text{Longest Path Length}}{\text{Shortest Path Length}} \\ &= \frac{2bh}{bh} \\ &= 2 \end{aligned}$$

Conclusion

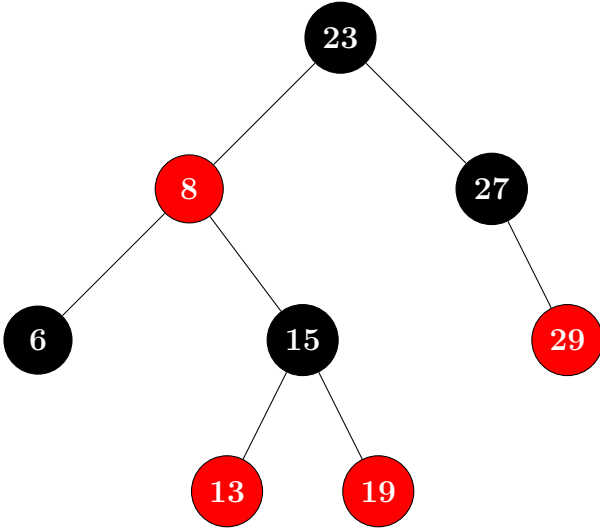
The longest path in a Red-Black tree can never be more than **twice as long** as the shortest path:

$$h_{\text{longest}} \leq 2 \cdot h_{\text{shortest}}$$

This strict geometric constraint ensures that the tree remains approximately balanced at all times, guaranteeing that search, insertion, and deletion operations maintain an upper bound of $\mathcal{O}(\log n)$ performance.

Q11. Insert 11 into the given Red-Black tree, showing each operation. (15 marks)

[CSE 207 | L-2/T-2 | 2017–18]



Answer: Step-by-Step Insertion of Key 11 (15 Marks)

Here is the complete, step-by-step execution to insert key **11** into the given Red-Black Tree, conforming to standard academic evaluation standards.

Step 1: Standard BST Insertion

We locate the appropriate insertion point using standard BST search properties ($11 < 23 \rightarrow \text{Left}$, $11 > 8 \rightarrow \text{Right}$, $11 < 15 \rightarrow \text{Left}$, $11 < 13 \rightarrow \text{Left}$). The new node is initially colored **Red**.

Violation Identified: A consecutive **Red-Red violation** occurs between parent node **13** and child node **11**.

—

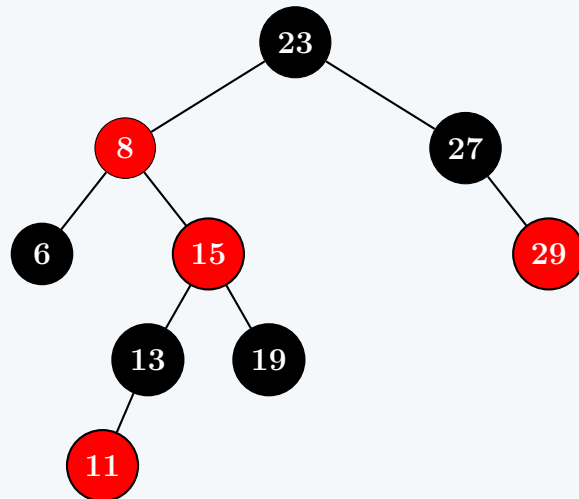
Step 2: Case 1 Fixup (Red Uncle Node)

We examine the neighborhood surrounding our violation node (**11**):

- **Parent:** 13 (Red) • **Grandparent:** 15 (Black) • **Uncle:** 19 (Red)

Since the uncle node is **Red**, we apply **Case 1 (Recoloring)**:

- (a) Flip the color of both the Parent (13) and Uncle (19) to **Black**.
- (b) Flip the color of the Grandparent (15) to **Red**.
- (c) Shift the inspection pointer up to the grandparent node (15).



Violation Identified: Shifting the red color upward has generated a new **Red-Red violation** higher up between node **8** and node **15**.

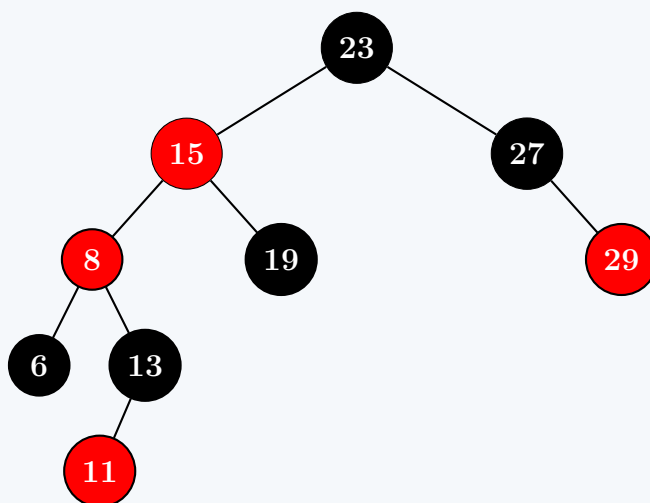
Step 3: Case 2 Fixup (Black Uncle, Triangle Configuration)

We re-evaluate the neighborhood properties from our new inspection node (**15**):

- **Parent:** 8 (Red) • **Grandparent:** 23 (Black) • **Uncle:** 27 (Black)

Because the uncle node is black and node 15 forms a Right-child relationship to a Left-child parent (forming a zig-zag triangle shape), we apply **Case 2**:

- (a) Execute a **Left Rotation** around the parent node (**8**).
- (b) Node 15 rises up, node 8 becomes its left child, and node 15's former left subtree (node 13) gets reassigned as the right child of node 8.

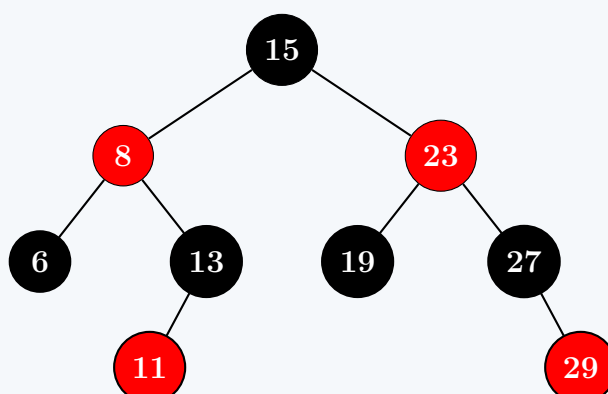


Note: The Red-Red violation remains between node 15 and node 8, but they are now aligned in a straight line, setting up the terminal case.

Step 4: Case 3 Fixup (Black Uncle, Line Configuration)

Evaluating node **8**: it forms a straight, Left-Left linear layout with its parent (15) and grandparent (23). The uncle node (27) remains black. We apply **Case 3**:

- (a) Execute a **Right Rotation** around the grandparent node (**23**).
- (b) Swap the colors of the parent node (15) and the old grandparent node (23). Thus, node 15 becomes **Black** and node 23 becomes **Red**.



Final State Verification:

- Root node (15) is Black $\rightarrow$ **Valid**.
- No consecutive red nodes exist anywhere in the tree $\rightarrow$ **Valid**.
- Every distinct path from the root down to a NIL leaf contains exactly 3 black nodes $\rightarrow$ **Valid**.

Q12. Write the properties of a red-black tree. Show that the height of a red-black tree T storing n items is $O(\log n)$. Explain, with examples, the three cases that may arise for inserting a node in a red-black tree. **(15 marks)** [CSE 203 | L-2/T-1 | 2015–16, 2014–15]

Answer: Red-Black Tree Invariants, Height Proof, and Insertion Cases (15 Marks)**1. Properties of a Red-Black Tree**

A Red-Black tree is a binary search tree where each node contains an extra attribute representing its color (**Red** or **Black**). A binary search tree is a valid Red-Black tree if it satisfies the following five structural properties:

- Node Property:** Every node is colored either Red or Black.
- Root Property:** The root of the tree is always Black.
- Leaf Property:** Every leaf node (NIL boundary marker) is Black.
- Red Property:** If a node is Red, then both of its children must be Black. (Consecutive Red nodes are strictly prohibited).
- Black-Height Property:** For each node, all simple downward paths from that node to any of its descendant NIL leaves contain the exact same number of black nodes.

2. Proof: Height of a Red-Black Tree is $O(\log n)$

To prove that a Red-Black tree with n internal nodes has a maximum height $h \leq 2\log_2(n + 1)$, we introduce the concept of **black-height** ($bh(x)$), defined as the number of black nodes on any simple path from node x down to a leaf, not counting x itself.

Lemma

The subtree rooted at any node x contains at least $2^{bh(x)} - 1$ internal nodes.

Proof by Induction. Base Case: If the height of node x is 0, then x must be a NIL leaf. Its black-height is $bh(x) = 0$. The number of internal nodes in its subtree is $2^0 - 1 = 0$, which is true.

Inductive Step: Let x be an internal node of height > 0 with two children. Each child has a black-height of either:

- $bh(x)$ (if the child node itself is colored Red)
- $bh(x) - 1$ (if the child node itself is colored Black)

In either scenario, the black-height of any child node is at least $bh(x) - 1$. Applying the inductive hypothesis, each child's subtree contains a minimum of $2^{bh(x)-1} - 1$ internal nodes.

Summing the internal nodes contributed by both children plus node x itself yields:

$$\begin{aligned} \text{Total internal nodes} &\geq (2^{bh(x)-1} - 1) + (2^{bh(x)-1} - 1) + 1 \\ &= 2 \cdot (2^{bh(x)-1}) - 2 + 1 \\ &= 2^{bh(x)} - 1 \end{aligned}$$

This completes the inductive proof for the lemma. □

Bounding Height (h)

Let the root of the entire tree be T . According to **Property 4**, no two red nodes can be adjacent along any root-to-leaf path. This implies that at least half of the nodes on any simple path (excluding the root) must be black. Therefore, the black-height of the root must satisfy:

$$bh(T) \geq \frac{h}{2}$$

Using our proven lemma, the total number of internal nodes n in the tree satisfies:

$$\begin{aligned} n &\geq 2^{bh(T)} - 1 \implies n \geq 2^{h/2} - 1 \\ &\implies n + 1 \geq 2^{h/2} \\ &\implies \log_2(n + 1) \geq \frac{h}{2} \\ &\implies h \leq 2\log_2(n + 1) \end{aligned}$$

Since $h \leq 2\log_2(n + 1)$, we conclude that the structural height of the tree is asymptotically bounded:

$$h = O(\log n)$$

3. Three Cases for Node Insertion (With Examples)

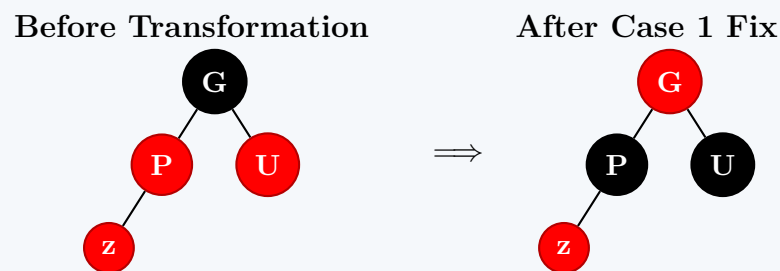
When a new node z is inserted, it is always inserted as a binary search tree leaf and colored **Red**. A conflict occurs only if z 's parent, $P(z)$, is also **Red**, violating Property 4.

Let $G(z)$ represent z 's grandparent, and $U(z)$ represent z 's uncle. Assuming $P(z)$ is a left child of $G(z)$ (the right-side cases are entirely symmetric), three resolution configurations can arise:

Case 1: The Uncle $U(z)$ is Red

Condition: The parent $P(z)$ and uncle $U(z)$ are both Red.

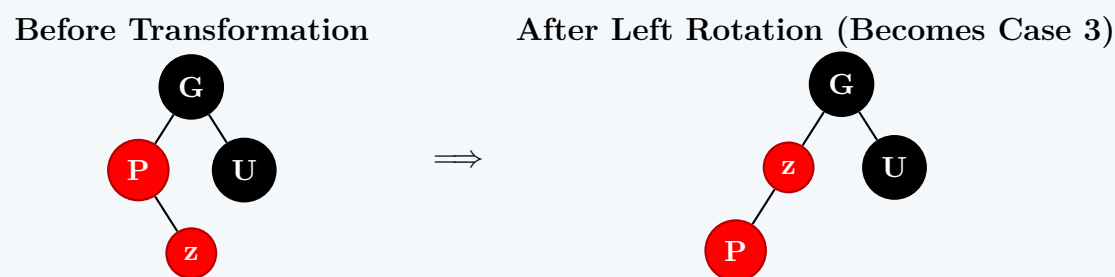
Action: Recolor $P(z)$ and $U(z)$ to **Black**, and recolor $G(z)$ to **Red**. Then, shift the inspection pointer up to $G(z)$.



Case 2: The Uncle $U(z)$ is Black, and z forms a Triangle (Zig-Zag)

Condition: $U(z)$ is Black, and z is a right child while $P(z)$ is a left child.

Action: Perform a **Left Rotation** around the parent node $P(z)$. This shifts z into the parent position, reducing the configuration directly to Case 3.



Case 3: The Uncle $U(z)$ is Black, and z forms a Line (Zig-Zig)

Condition: $U(z)$ is Black, and z is a left child while $P(z)$ is a left child.

Action: Perform a **Right Rotation** around the grandparent node $G(z)$, and swap the colors of $P(z)$ and $G(z)$ ($P(z)$ becomes Black, $G(z)$ becomes Red).



Q13. Explain, with examples, the four cases that may arise for deleting a node from a red-black tree. (15 marks) [CSE 203 | L-2/T-1 | 2014–15]

Answer: Comprehensive Analysis of Red-Black Tree Deletion Cases (15 Marks)

In a Red-Black tree, standard Binary Search Tree (BST) deletion always reduces to removing a node with at most one internal child. Let the node physically unlinked from the tree be y .

- If y is **Red**, its removal causes no structural violations.
- If y is **Black**, it disrupts the tree's balance by decreasing the black-height of all paths passing through its position by 1.

To track this deficit, we transfer the "missing black" property to y 's unique child (or a NIL leaf placeholder), making it ****Double-Black**** (x). To restore balance, we examine the color configurations of x 's parent (P), sibling (w), and w 's children (nephews). Assuming x is a **left child** (the right-child scenarios are completely symmetric), four distinct cases can arise during the fixup loop.

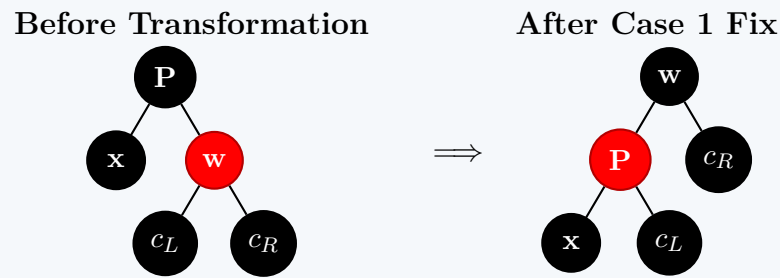
Case 1: Sibling w is Red

Condition: The sibling w is Red. Because w is Red, Property 4 dictates that the parent P and w 's children must all be Black.

Action:

- Swap the colors of the parent P and sibling w (P becomes Red, w becomes Black).
- Perform a **Left Rotation** around the parent P .

- (c) Identify the new sibling of x . This updates the local environment to match Case 2, 3, or 4.

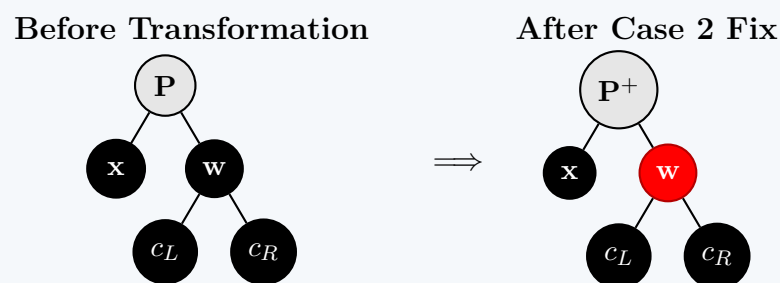


Case 2: Sibling w is Black, and Both of w 's Children are Black

Condition: The sibling w is Black, and its left and right children are both Black (NIL or internal).

Action:

- Pull one unit of "blackness" from both x (reducing it from Double-Black to singly Black) and w (recoloring w to **Red**).
- Push the extra black property onto the parent P .
- If P was originally Red, it absorbs the weight and transitions to a standard Black node, terminating the algorithm. If P was already Black, it becomes the new Double-Black node (x), and the loop repeats one level higher.

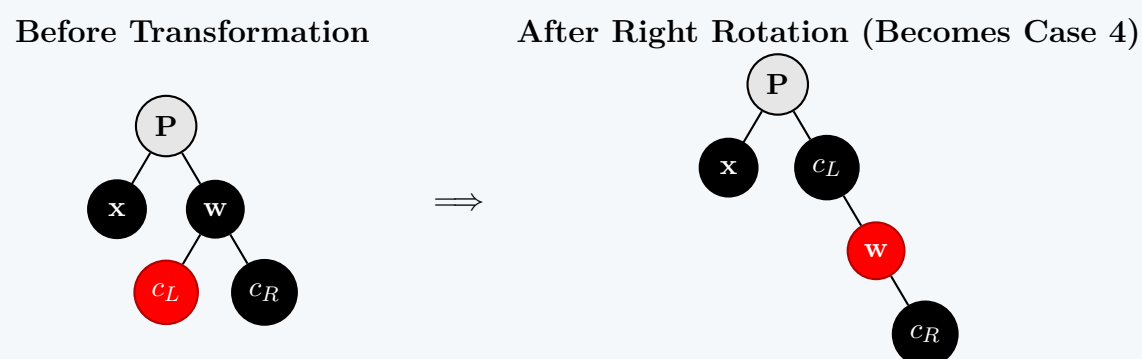


Case 3: Sibling w is Black, w 's Left Child is Red, and w 's Right Child is Black

Condition: Sibling w is Black, its inner nephew (c_L) is Red, and its outer nephew (c_R) is Black (a structural zig-zag pattern).

Action:

- Swap the colors of sibling w and its left child c_L (w becomes Red, c_L becomes Black).
- Perform a **Right Rotation** around the sibling node w .
- Reassign the sibling pointer to the new black node (c_L). This transitions the setup directly into Case 4.

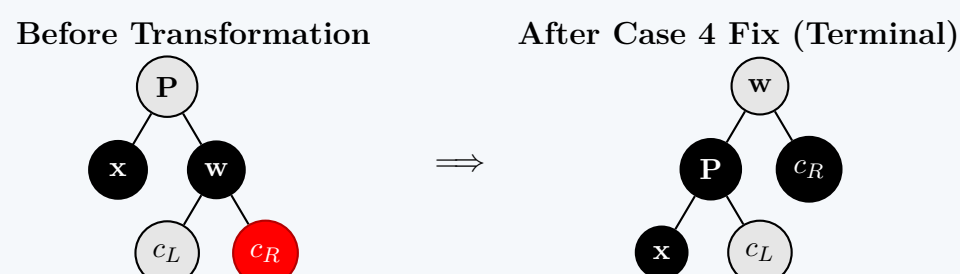


Case 4: Sibling w is Black, and w 's Right Child is Red

Condition: Sibling w is Black, and its outer nephew (c_R) is Red. The color of the inner nephew (c_L) is completely irrelevant.

Action:

- Recolor sibling w to match the original color of the parent node P .
- Recolor both the parent node P and the outer nephew c_R to **Black**.
- Perform a **Left Rotation** around the parent node P .
- The extra black attribute on x is neutralized. This step completely fixes the path deficits and terminates the algorithm.



Q14. Write the properties of a red-black tree. Show the red-black trees that result after successively inserting the keys 51, 46, 41, 22, 29, 18 into an initially empty red-black tree. (15 marks) [CSE 203 | L-2/T-1 | 2013–14]

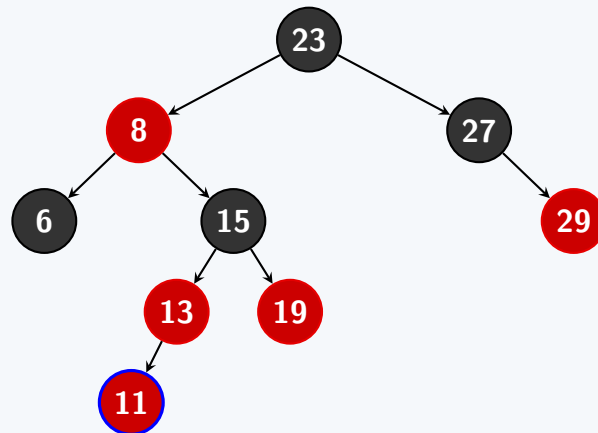
Answer

Part 1: Insertion of 11

Step 1: Standard BST Insertion

We insert 11 following the standard Binary Search Tree properties: $11 < 23$ (go left), $11 > 8$ (go right), $11 < 15$ (go left), $11 < 13$ (go left). The new node 11 is inserted as the left child of 13 and is initially colored **RED**.

- **Violation:** Node 13 is **RED** and Node 11 is **RED** (Adjacent Red nodes).

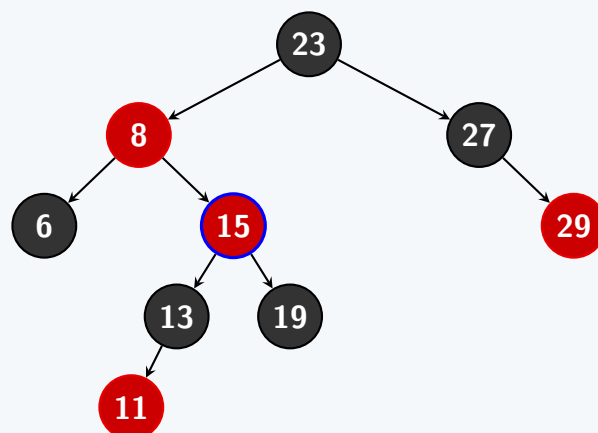


Step 2: Fix Violation (Case 1 – Uncle is RED)

The parent of 11 is 13 (**RED**). The grandparent is 15 (**BLACK**). The uncle is 19 (**RED**). Since the uncle is **RED**, we apply **Case 1**:

- Recolor the parent (13) and uncle (19) to **BLACK**.
- Recolor the grandparent (15) to **RED**.
- The current node moves up to the grandparent (15) to check for new violations.

New Violation: Node 8 is **RED** and Node 15 is **RED**.

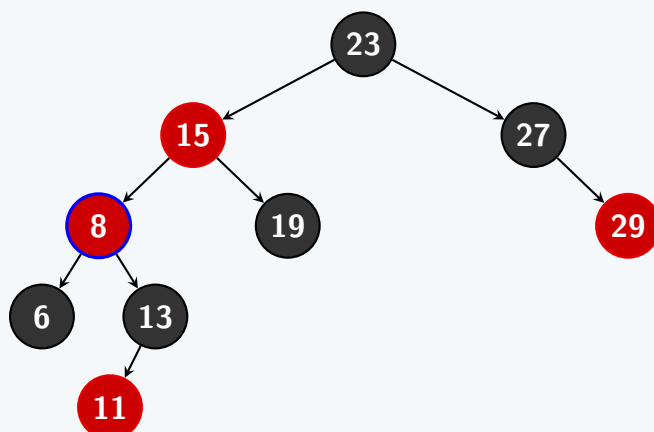


Step 3: Fix Violation (Case 2 – Uncle is BLACK, Left-Right Case)

For the current node 15, its parent is 8 (**RED**), grandparent is 23 (**BLACK**), and uncle is 27 (**BLACK**). Node 15 is a right child, and its parent 8 is a left child. We apply **Case 2**:

- Perform a **Left Rotation** around the parent (8).
- Node 15 moves up, and Node 8 becomes its left child. The left child of 15 (Node 13) becomes the right child of 8.

New Violation: Node 15 (**RED**) and Node 8 (**RED**) are adjacent.

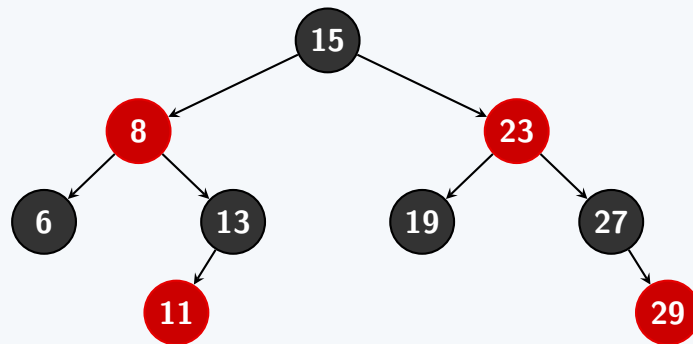


Step 4: Fix Violation (Case 3 – Uncle is BLACK, Left-Left Case) & Final Tree

Now, Node 8 is a left child, and its parent 15 is also a left child. This is **Case 3**:

- Perform a **Right Rotation** around the grandparent (23). Node 15 becomes the new root.
- Node 23 becomes the right child of 15. The right child of 15 (Node 19) becomes the left child of 23.
- **Recolor:** Change the original parent (15) to **BLACK** and the original grandparent (23) to **RED**.

The tree now fully satisfies all Red-Black properties.



Part 2: Properties & Successive Insertions

Properties of a Red-Black Tree

A Red-Black Tree is a Binary Search Tree that satisfies the following five properties:

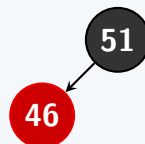
- Every node is colored either **Red** or **Black**.
- The root of the tree is always **Black**.
- Every leaf (NIL pointer) is considered **Black**.
- If a node is **Red**, then both of its children must be **Black** (i.e., no two adjacent Red nodes exist on any path).
- For any node, all simple paths from that node to its descendant leaves contain the same number of **Black** nodes (this is called the *black-height*).

Successive Insertions: 51, 46, 41, 22, 29, 18

- Insert 51:** Inserted as Red. Since it is the root, it is recolored to Black.



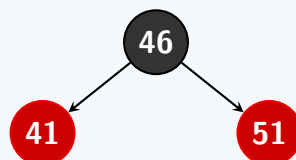
- Insert 46:** Inserted as Red left child of 51. No violations.



- Insert 41:** Inserted as Red left child of 46.

Violation: 46 and 41 are both Red. Uncle is NIL (Black). This is a *Left-Left Case*.

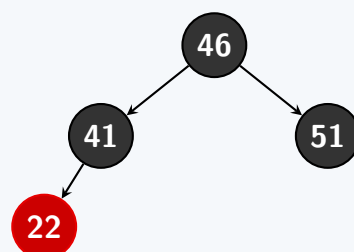
Fix: Right rotate around 51. Recolor 46 to Black and 51 to Red.



- Insert 22:** Inserted as Red left child of 41.

Violation: 41 and 22 are both Red. Uncle is 51 (Red). This is *Case 1 (Red Uncle)*.

Fix: Recolor parent (41) and uncle (51) to Black, and grandparent (46) to Red. Since 46 is the root, it is recolored back to Black.

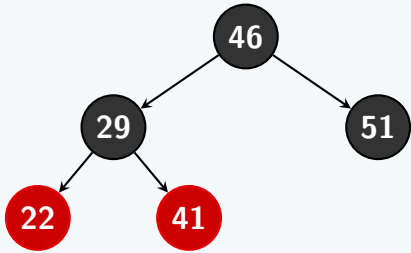


- Insert 29:** Inserted as Red right child of 22.

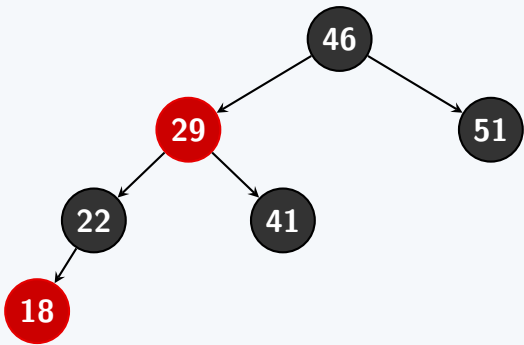
Violation: 22 and 29 are both Red. Uncle is NIL (Black). This is a *Left-Right Case*.

Fix:

- Left rotate around 22 to convert to Left-Left case (29 moves up, 22 becomes its left child).
- Right rotate around 41. 29 becomes the left child of 46, and 41 becomes the right child of 29.
- Recolor 29 to Black and 41 to Red.



6. Insert 18: Inserted as Red left child of 22.
Violation: 22 and 18 are both Red. Uncle is 41 (Red). This is *Case 1 (Red Uncle)*.
Fix: Recolor parent (22) and uncle (41) to Black. Recolor grandparent (29) to Red. No further violations since 29's parent (46) is Black.



Q15. Write down the properties of a red-black tree. With an example, show the insertion and deletion operations and their costs in a red-black tree. **(5+15=20 marks)** [CSE 203 | L-2/T-1 | 2012–13]

Answer: Red-Black Tree Properties, Insertion, and Deletion (20 Marks)

1. Properties of a Red-Black Tree (5 Marks)

A Red-Black Tree is a self-balancing binary search tree that guarantees $\mathcal{O}(\log n)$ operations by strictly satisfying the following five structural invariants:

- (a) **Node Property:** Every node is strictly colored either **Red** or **Black**.
- (b) **Root Property:** The root node is always **Black**.
- (c) **Leaf Property:** Every leaf node (the implicit NIL pointers at the bottom) is **Black**.
- (d) **Red Property (No Consecutive Reds):** If a node is Red, both of its children must be Black.
- (e) **Black-Height Property:** Every simple downward path from a given node to any of its descendant NIL leaves must contain the exact same number of Black nodes.

2. Insertion Operation & Cost (7.5 Marks)

When inserting a new node, it is placed as a leaf using standard BST rules and initially colored **Red** to maintain the black-height. If its parent is also Red, a violation occurs, which is fixed via color flips or rotations based on the uncle's color.

Example: Inserting keys 10, 20, and 30

(a) **Insert 10:** It becomes the root. Initially Red, but flipped to **Black** to satisfy the Root Property.

(b) **Insert 20:** Inserted as the right child of 10. Colored **Red**.

(c) **Insert 30:** Inserted as the right child of 20. Colored **Red**. This violates Property 4 (Consecutive Reds).

(d) **Fixup:** Nodes 10, 20, and 30 form a right-sided line. The uncle is Black (NIL). We perform a **Left Rotation** around 10 and swap the colors of 10 and 20.

Insert 10 & 20

Insert 30 (Violation)

After Left Rotation (Fixed)

Insertion Costs:

- **Search/Traversal:** $\mathcal{O}(\log n)$

210

- **Recoloring:** $\mathcal{O}(\log n)$ max (if pushed to root).
- **Rotations:** At most **2** structural rotations.
- **Total Time Complexity:** $\mathcal{O}(\log n)$

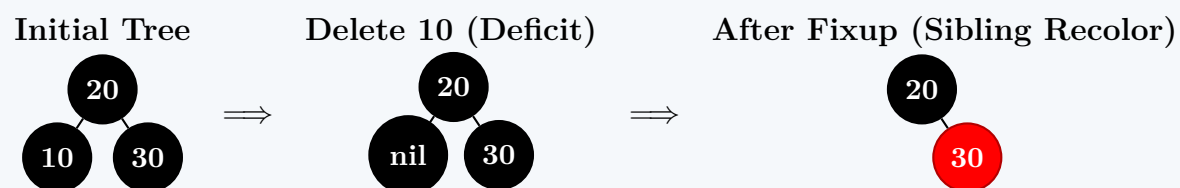
3. Deletion Operation & Cost (7.5 Marks)

Deleting a Red node is simple, but deleting a Black node creates a black-height deficit, logically creating a "Double-Black" node. This is resolved by shifting the missing blackness up the tree or neutralizing it through rotations depending on the sibling's color.

Example: Deleting a Black leaf (10)

Assume a perfectly balanced tree where 20 is the Black root, with 10 and 30 as Black children.

- Delete 10:** Node 10 is removed. Its NIL replacement inherits the deficit, becoming **Double-Black**.
- Examine Sibling:** Sibling is 30 (Black), and its children are Black.
- Fixup:** Extract "blackness" from the sibling (recoloring 30 to **Red**) and from the Double-Black leaf (making it normal Black). Push the blackness to the parent (20).
- Resolve Root:** Because 20 is the root, it simply absorbs the extra blackness, neutralizing the deficit.

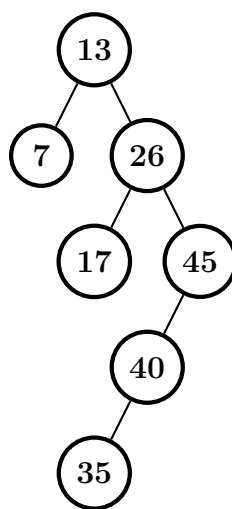


Deletion Costs:

- **Search/Traversal:** $\mathcal{O}(\log n)$
- **Recoloring:** $\mathcal{O}(\log n)$ max (if deficit propagates upward).
- **Rotations:** At most **3** structural rotations.
- **Total Time Complexity:** $\mathcal{O}(\log n)$

Splay Trees

Q1. Complete the following operations on the given splay tree: (i) Insert 30, (ii) Delete 35. Show all the steps.



(13 marks)

[CSE 207 | L-2/T-1 | 2023–24]

Answer

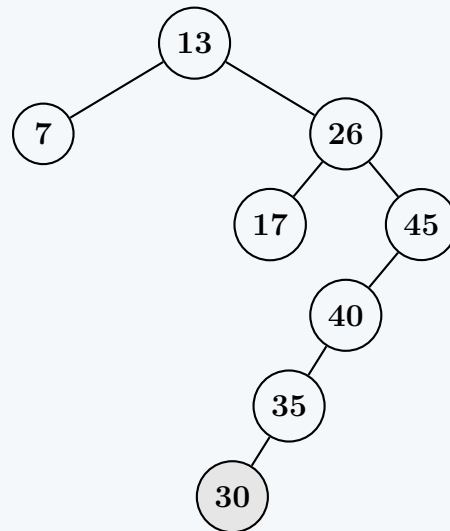
Overview of Operations

- **Insertion:** We first insert the node using standard Binary Search Tree (BST) insertion rules. Then, we **splay** the newly inserted node to the root.
- **Deletion (Join Method):** We first find the node and **splay** it to the root. Then, we remove the root, leaving two disjoint subtrees: the Left Subtree (T_L) and the Right Subtree (T_R). We splay the maximum element in T_L to the root of T_L , and finally attach T_R as its right child.

Part (i): Insert 30

Step 1: Standard BST Insertion

We insert 30 following standard BST properties. Comparing 30 with 13, 26, 45, 40, and 35, it eventually becomes the **left child** of 35.

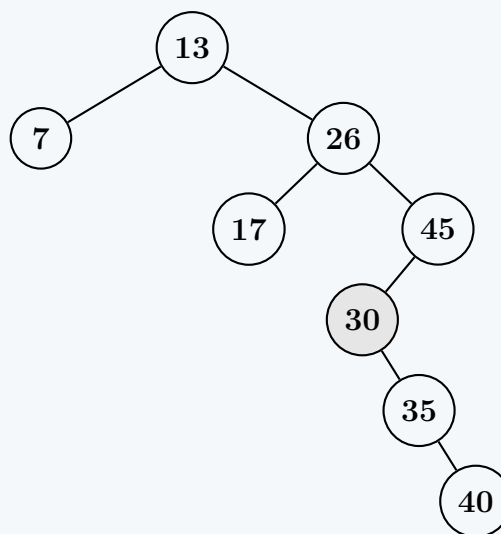


Step 2: Splay 30 to the Root

We now apply splay rotations to move 30 to the root.

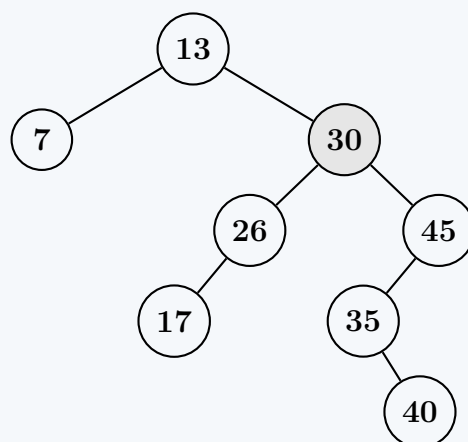
Substep 2.1: Zig-Zig Rotation at 30

Node 30 is a left child of 35, which is a left child of 40. This is a *Left-Left* configuration. We perform a **Zig-Zig** rotation: a right rotation at the grandparent (40), followed by a right rotation at the parent (35).



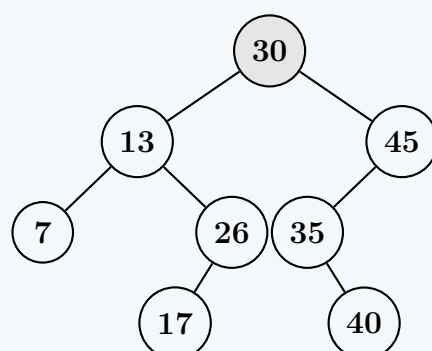
Substep 2.2: Zig-Zag Rotation at 30

Node 30 is a left child of 45, which is a right child of 26. This is a *Right-Left* configuration. We perform a **Zig-Zag** rotation: a right rotation at the parent (45), followed by a left rotation at the grandparent (26).



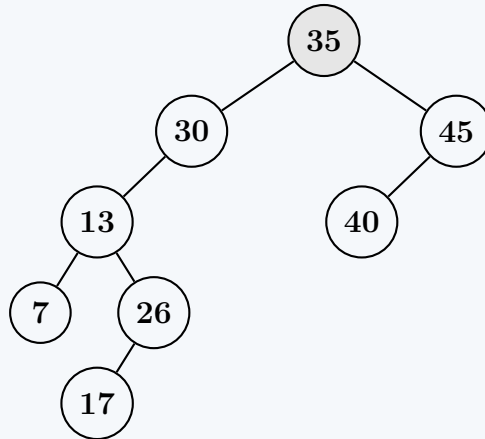
Substep 2.3: Zig Rotation at 30

Node 30 is now the right child of the root 13. We perform a **Zig** rotation: a left rotation at the root (13). 30 becomes the new root of the entire tree.

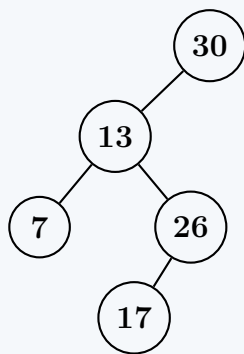
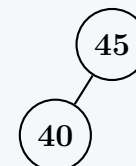


Part (ii): Delete 35**Step 1: Find 35 and Splay it to the Root**

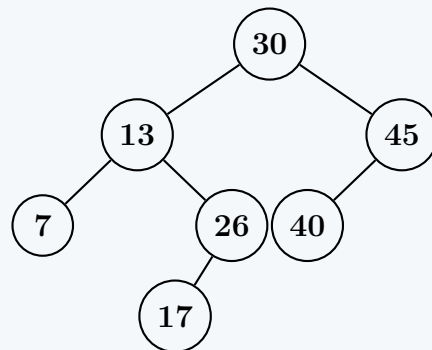
We traverse the tree to find 35. It is located as a left child of 45, which is a right child of 30. This *Right-Left* configuration requires a **Zig-Zag** rotation: a right rotation at the parent (45), followed by a left rotation at the grandparent (30).

**Step 2: Delete the Node and Split**

We remove the root (35). This operation disconnects the tree into two separate components: the Left Subtree (T_L) and the Right Subtree (T_R).

Left Subtree (T_L)**Right Subtree (T_R)****Step 3: Join Subtrees**

To merge the trees back together, we find the maximum element in T_L , which is 30. We must splay 30 to the root of T_L . However, since 30 is *already* the root of T_L , no rotations are needed. The root of T_L is guaranteed to have no right child, so we safely attach T_R as the right child of 30.

Final Splay Tree after Deletion**Complexity Analysis**

- **Time Complexity:** Both Insert and Delete operations take $\mathcal{O}(\log n)$ **amortized time**. While a single operation in a severely unbalanced tree could take $\mathcal{O}(n)$ time in the worst case, the structural rebalancing (splaying) guarantees that any sequence of M operations on an initially empty tree takes $\mathcal{O}(M \log n)$ time, yielding the logarithmic amortized bound (proven via the Access Lemma).
- **Space Complexity:** $\mathcal{O}(n)$ to store the nodes of the tree, as no extra balancing information (like heights or colors) needs to be stored within the nodes.

Q2. How is the concept of potential function used to design a splay tree? Explain. (8 marks)

[CSE 207 | L-2/T-1 | 2022–23]

Answer**Introduction to the Potential Method in Splay Trees**

Splay trees do not explicitly store balance factors (like AVL trees) or colors (like Red-Black trees), meaning a single operation could theoretically take $\mathcal{O}(n)$ time if the tree degenerates into a linked list. To guarantee that a sequence of m operations takes $\mathcal{O}(m \log n)$ time, Splay Trees rely on **Amortized Analysis** using the **Potential Method**.

The potential function Φ acts like a "bank account". It maps the current state of the data structure to a real number (its potential).

- When an operation is cheap, it does a little extra work to "save" potential ($\Delta\Phi > 0$).
- When an operation is expensive (e.g., accessing a deep node in a skewed tree), it "spends" the stored potential ($\Delta\Phi < 0$) to pay for the expensive restructuring, keeping the amortized cost low.

Defining the Splay Tree Potential Function

To design and analyze the Splay Tree, Daniel Sleator and Robert Tarjan defined the potential function based on the size of the subtrees. For any node x in a Splay Tree T :

- (a) **Size** $s(x)$: The number of nodes in the subtree rooted at x (including x itself).
- (b) **Rank** $r(x)$: The base-2 logarithm of the size: $r(x) = \lg(s(x))$
- (c) **Tree Potential** $\Phi(T)$: The sum of the ranks of all nodes in the tree:

$$\Phi(T) = \sum_{x \in T} r(x) = \sum_{x \in T} \lg(s(x)) \quad (1)$$

Intuition Behind the Design

Why choose this specific potential function?

- **Balanced Tree:** If the tree is perfectly balanced, sizes decrease exponentially as we go down. The sum of the ranks is minimized. Thus, a balanced tree has **low potential**.
- **Skewed Tree:** If the tree is a straight line (linked list), every node has size $k, k-1, \dots, 1$. The ranks are large, meaning the tree has **high potential**.

The fundamental design philosophy of the Splay Tree is: *A highly unbalanced tree holds a lot of potential energy.* If we do an expensive $\mathcal{O}(n)$ operation by accessing a deep leaf, the resulting **Splay** (rotations to bring the node to the root) will roughly halve the depth of every node along the access path. This drastic restructuring transforms a highly skewed tree into a much more balanced one.

Because the tree becomes more balanced, its potential Φ **drops significantly**. This large negative change in potential ($\Delta\Phi \ll 0$) cancels out the high actual cost (c_i) of the operation.

Mathematical Application: The Access Lemma

The amortized cost $\hat{c}_i$ of the i -th operation is defined as:

$$\hat{c}_i = c_i + \Phi(T_i) - \Phi(T_{i-1}) = \text{Actual Cost} + \text{Change in Potential} \quad (2)$$

By carefully designing the **Zig-Zig** and **Zig-Zag** rotation rules, Sleator and Tarjan ensured that the amortized cost of a single splay step bounds perfectly. For any node x being splayed to the root t , the potential function allows us to prove the **Access Lemma**:

$$\text{Amortized Cost of Splaying } x \leq 3(r(t) - r(x)) + 1 \quad (3)$$

Since the maximum rank of the root is $\lg n$, and the minimum rank of x is $\lg 1 = 0$, the expression $3(r(t) - r(x)) + 1$ is at most $3\lg n + 1$.

Summary of How Potential Dictated the Design

The potential function was not just an afterthought for analysis; it **guided the design of the splay rules**.

- A simple "rotate-to-root" algorithm (doing single rotations) fails to achieve $\mathcal{O}(\log n)$ amortized time because it does not decrease the potential function enough to pay for deep accesses.
- The specific double-rotations (**Zig-Zig** and **Zig-Zag**) were deliberately invented because they guarantee that the potential of the tree drops sufficiently to mathematically satisfy the amortized cost equation $\hat{c}_i \leq \mathcal{O}(\log n)$.

Q3. What are the splay-step operations to make a binary search tree a balanced one? **(8 marks)**

[CSE 207 | L-2/T-2 | 2021–22]

Answer

Introduction to Splay Steps and Balancing

Unlike AVL or Red-Black trees, a Splay Tree does not enforce strict height invariants (like $\mathcal{O}(\log n)$ height at all times). Instead, it uses a self-adjusting heuristic called **Splaying** to achieve **amortized balance**.

Whenever a node x is accessed (inserted, searched, or deleted), it is moved to the root of the tree through a specific sequence of rotations known as **Splay Steps**. These steps are carefully designed to not only bring x to the top but also to roughly **halve the depth of every node along the access path**. This restructuring flattens skewed branches, effectively "balancing" the tree over time.

There are three primary splay-step operations, chosen based on the relationship between the target node x , its parent p , and its grandparent g .

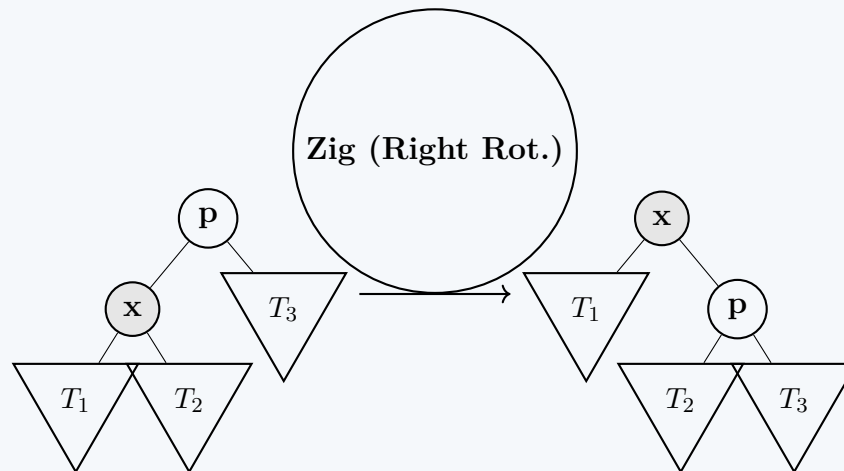
1. The Zig Step

Condition: Node x has a parent p , and p is the **root** of the tree (there is no grandparent).

Operation: We perform a **single rotation** at p .

- If x is a left child, perform a Right Rotation at p .
- If x is a right child, perform a Left Rotation at p .

Note: The Zig step is only ever performed as the final step of a splay operation when the target node is just one level below the root.



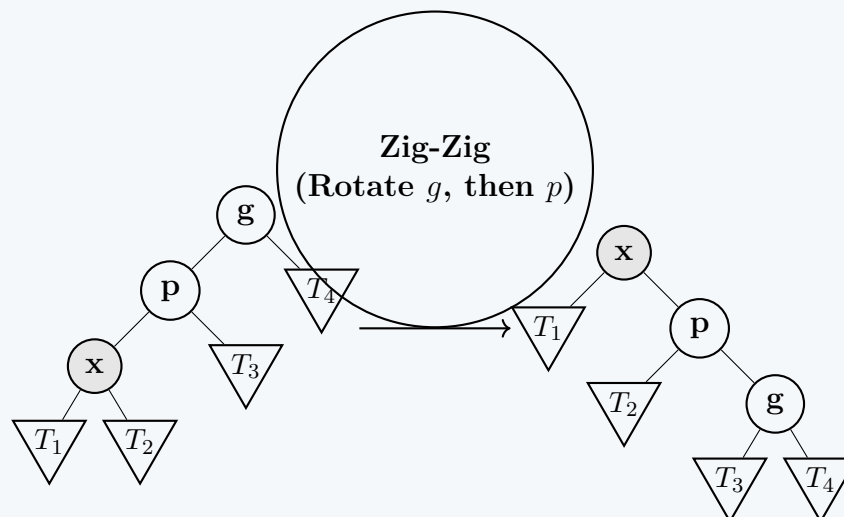
2. The Zig-Zig Step

Condition: Node x has a parent p and a grandparent g . Both x and p are **homogenous children** (either both are left children, or both are right children).

Operation: This is the most crucial step for balancing. We perform **two rotations in the same direction**:

- First, rotate at the grandparent g .
- Second, rotate at the parent p .

This specific order ensures that the tree depth is significantly reduced, unlike standard single rotations.

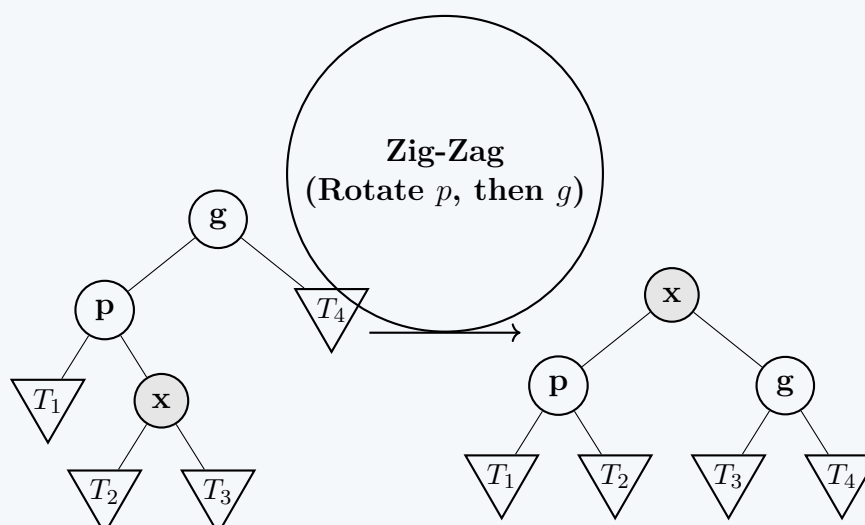


3. The Zig-Zag Step

Condition: Node x has a parent p and a grandparent g , but they are **heterogenous children** (e.g., x is a right child and p is a left child, forming a "zig-zag" shape).

Operation: We perform a standard double rotation, just like in an AVL tree:

- First, rotate at the parent p (bringing x up to p 's level).
- Second, rotate at the grandparent g (bringing x up to g 's level).



How these steps make the tree balanced: A naive "rotate-to-root" strategy (doing only single Zig rotations) pushes the target node to the root, but pushes all other nodes further down, leaving the tree highly unbalanced. The **Zig-Zig** step specifically combats this: by rotating at the grandparent first, it pulls the subtrees T_2 and T_3 up alongside x , drastically reducing the overall height of the path and resolving degenerated line-like structures.

Q4. Define the rank of a node of a splay tree. Prove that the change in potential of a splay tree not resulting from any splay after an insertion operation is $O(\log n)$, where the splay tree had n nodes before the insertion operation. **(15 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Answer: Splay Tree Rank and Potential Change Proof

1. Definition of the Rank of a Node

In a splay tree T , let $s(x)$ denote the **size** of the subtree rooted at node x , which is equal to the total number of internal nodes in that subtree.

The **rank** $r(x)$ of a node x is defined as the base-2 logarithm of its size:

$$r(x) = \log_2(s(x))$$

The **total potential** Φ of the splay tree is the sum of the ranks of all nodes in the tree:

$$\Phi = \sum_{v \in T} r(v)$$

2. Proof of Potential Change Due Only to Insertion

We want to find the change in potential $\Delta\Phi_{\text{insert}}$ caused strictly by adding a new node z as a leaf, **before** any subsequent splay operations are performed.

Setup and Notation

- Let T be a splay tree containing n nodes before insertion.
- A new node z is inserted into T using standard binary search tree insertion.
- Let $s(v)$ and $r(v)$ represent the size and rank of any node v *before* insertion.
- Let $s'(v)$ and $r'(v)$ represent the size and rank of any node v *after* insertion (but before splaying).

Identifying Affected Nodes

(a) **For the new node z :** It is inserted as a leaf, so its new subtree size is $s'(z) = 1$. Its new rank is:

$$r'(z) = \log_2(1) = 0$$

(b) **For nodes not on the path from the root to z :** Their subtrees are unaffected, meaning $s'(v) = s(v)$ and $r'(v) = r(v)$.

(c) **For ancestors of z :** Let the path from the parent of z up to the root be a sequence of m nodes: $v_1, v_2, \dots, v_m$, where v_1 is the parent of z and v_m is the root of the tree. For each ancestor v_i , its size increases by exactly 1:

$$s'(v_i) = s(v_i) + 1$$

Algebraic Formulation of $\Delta\Phi_{\text{insert}}$

The change in potential is the difference between the new total potential and the old total potential:

$$\Delta\Phi_{\text{insert}} = \Phi' - \Phi = r'(z) + \sum_{i=1}^m (r'(v_i) - r(v_i))$$

Since $r'(z) = 0$, substituting the rank definitions gives:

$$\Delta\Phi_{\text{insert}} = \sum_{i=1}^m (\log_2(s(v_i) + 1) - \log_2 s(v_i)) = \sum_{i=1}^m \log_2 \left(\frac{s(v_i) + 1}{s(v_i)} \right)$$

Bounding the Summation

Because v_i is a child of v_{i+1} in the tree layout, the size of v_{i+1} before insertion must include the size of its child v_i plus at least 1 (for v_{i+1} itself). This yields the structural inequality:

$$s(v_{i+1}) \geq s(v_i) + 1 \quad \text{for } 1 \leq i < m$$

We can use this inequality to set up an upper bound for the terms where $i < m$:

$$\frac{s(v_i) + 1}{s(v_i)} \leq \frac{s(v_{i+1})}{s(v_i)}$$

Taking the logarithm of both sides:

$$\log_2 \left(\frac{s(v_i) + 1}{s(v_i)} \right) \leq \log_2 \left(\frac{s(v_{i+1})}{s(v_i)} \right) = \log_2 s(v_{i+1}) - \log_2 s(v_i)$$

Now, we split our original total summation into the ancestors below the root ($1 \leq i < m$) and the root itself ($i = m$):

$$\Delta\Phi_{\text{insert}} \leq \left[\sum_{i=1}^{m-1} (\log_2 s(v_{i+1}) - \log_2 s(v_i)) \right] + \log_2 \left(\frac{s(v_m) + 1}{s(v_m)} \right)$$

Telescoping the Sum

The summation from 1 to $m - 1$ is a standard telescoping series where internal terms cancel out completely:

$$\sum_{i=1}^{m-1} (\log_2 s(v_{i+1}) - \log_2 s(v_i)) = \log_2 s(v_m) - \log_2 s(v_1)$$

Substituting this back into our inequality gives:

$$\Delta\Phi_{\text{insert}} \leq (\log_2 s(v_m) - \log_2 s(v_1)) + \log_2(s(v_m) + 1) - \log_2 s(v_m)$$

Canceling $\log_2 s(v_m)$ results in:

$$\Delta\Phi_{\text{insert}} \leq \log_2(s(v_m) + 1) - \log_2 s(v_1)$$

Final Simplification

- The root node v_m originally held all n nodes of the tree before insertion, so $s(v_m) = n$.
- The parent of the leaf node must have a size of at least 1, so $s(v_1) \geq 1 \implies \log_2 s(v_1) \geq 0$.

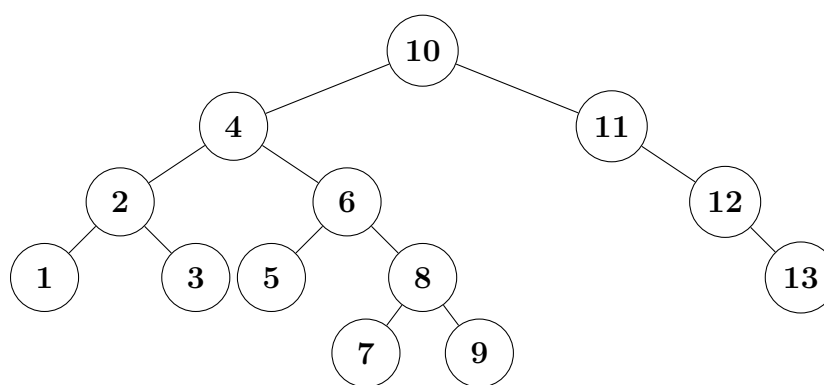
Dropping the non-negative term $-\log_2 s(v_1)$ provides the final upper bound:

$$\begin{aligned} \Delta\Phi_{\text{insert}} &\leq \log_2(n + 1) \\ \Delta\Phi_{\text{insert}} &= \mathcal{O}(\log n) \end{aligned}$$

Conclusion

Thus, the change in the potential of a splay tree resulting strictly from an insertion operation (excluding any subsequent splays) is asymptotically bounded by $\mathcal{O}(\log n)$.

- Q5.** (i) For the splay tree of the following figure, show the resulting tree after successively accessing the keys 3 and 9 in the splay tree.
(ii) For the splay tree of the following figure, show the resulting tree after deleting key 6 after doing operations in (i) (i.e., accessing the keys 3 and 9).



(15 marks)

[CSE 207 | L-2/T-2 | 2018–19]

Answer

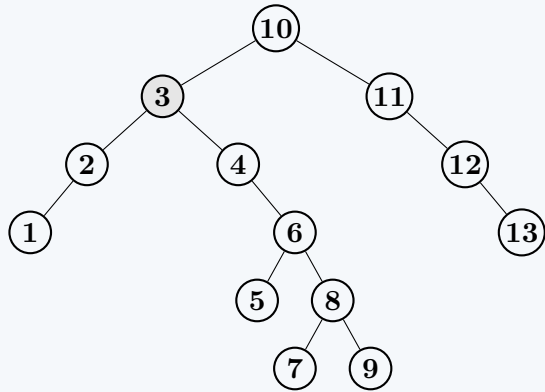
Part (i): Successively Accessing Keys 3 and 9

1. Accessing Key 3

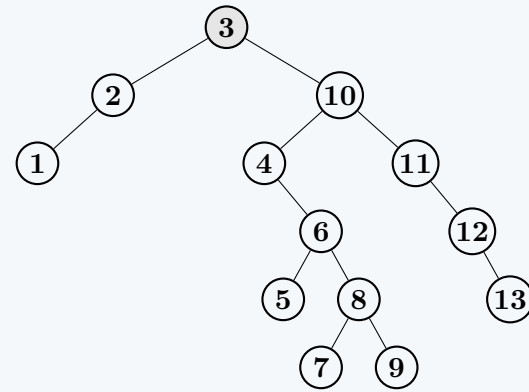
We locate node 3 in the initial tree. The path from the root is $10 \rightarrow 4 \rightarrow 2 \rightarrow 3$. To bring 3 to the root, we perform the following splay steps:

- **Step 1.1: Zig-Zag on 3.** The parent of 3 is 2, and the grandparent is 4. Since 2 is a left child and 3 is a right child, we perform a Left-Rotation at 2, followed by a Right-Rotation at 4.
- **Step 1.2: Zig on 3.** Now 3 is the left child of the root 10. We perform a single Right-Rotation at 10.

After Zig-Zag (Step 1.1)



After Zig (Step 1.2)

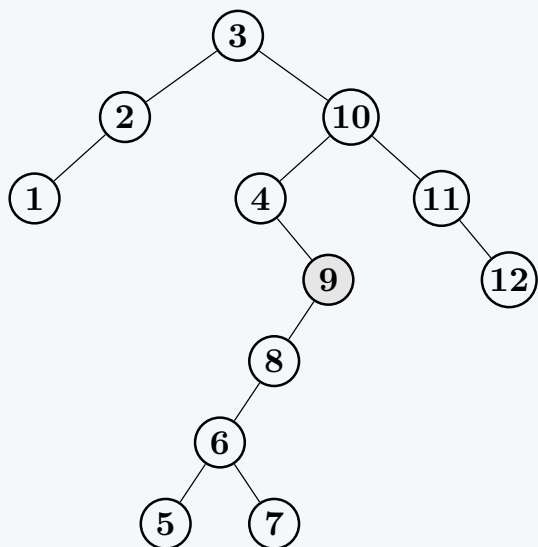


2. Accessing Key 9

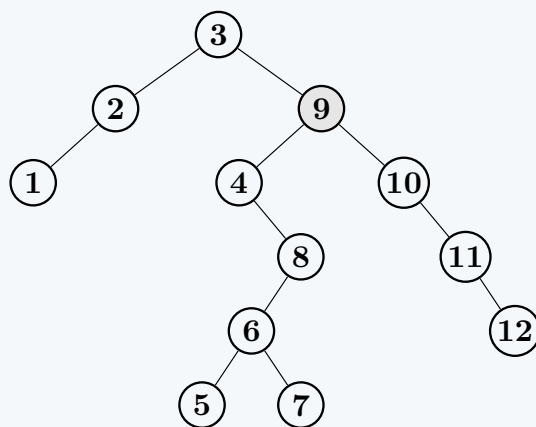
Starting from the new tree, we locate 9. The path is $3 \rightarrow 10 \rightarrow 4 \rightarrow 6 \rightarrow 8 \rightarrow 9$.

- **Step 2.1: Zig-Zig on 9.** Parent 8 and grandparent 6 are both right children. We perform a Left-Rotation at 6, then a Left-Rotation at 8.
- **Step 2.2: Zig-Zag on 9.** Parent 4 is a left child, grandparent 10 is a right child. We perform a Left-Rotation at 4, then a Right-Rotation at 10.
- **Step 2.3: Zig on 9.** Node 9 is the right child of 3. We perform a Left-Rotation at 3.

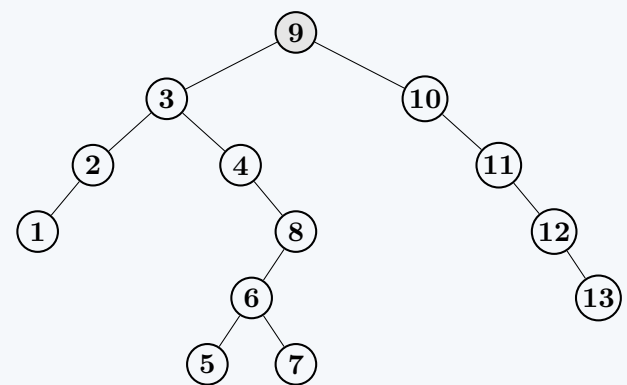
After Zig-Zig on 9



After Zig-Zag on 9



Final Tree for (i) After Zig

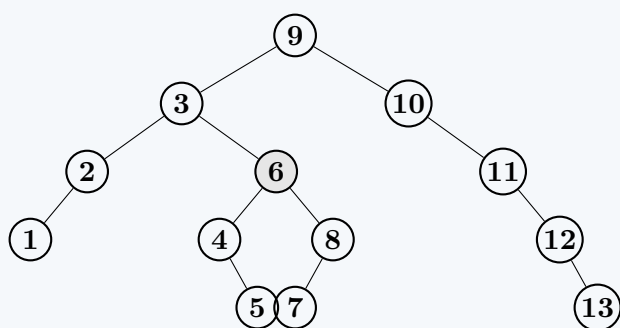


Part (ii): Deleting Key 6

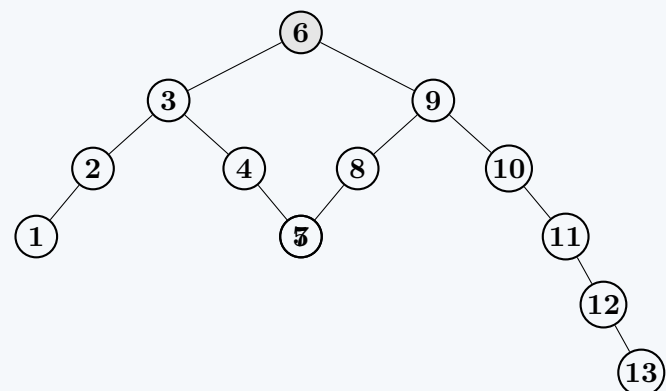
To delete 6, we must first access it and **splay it to the root**. The path to 6 is $9 \rightarrow 3 \rightarrow 4 \rightarrow 8 \rightarrow 6$.

- **Step 3.1: Zig-Zag on 6.** Parent 8 is a right child, and 6 is a left child. We Right-Rotate at 8, then Left-Rotate at 4.
- **Step 3.2: Zig-Zag on 6.** Parent 3 is a left child, and 6 is a right child. We Left-Rotate at 3, then Right-Rotate at 9. Node 6 becomes the root.

After Zig-Zag (Step 3.1)



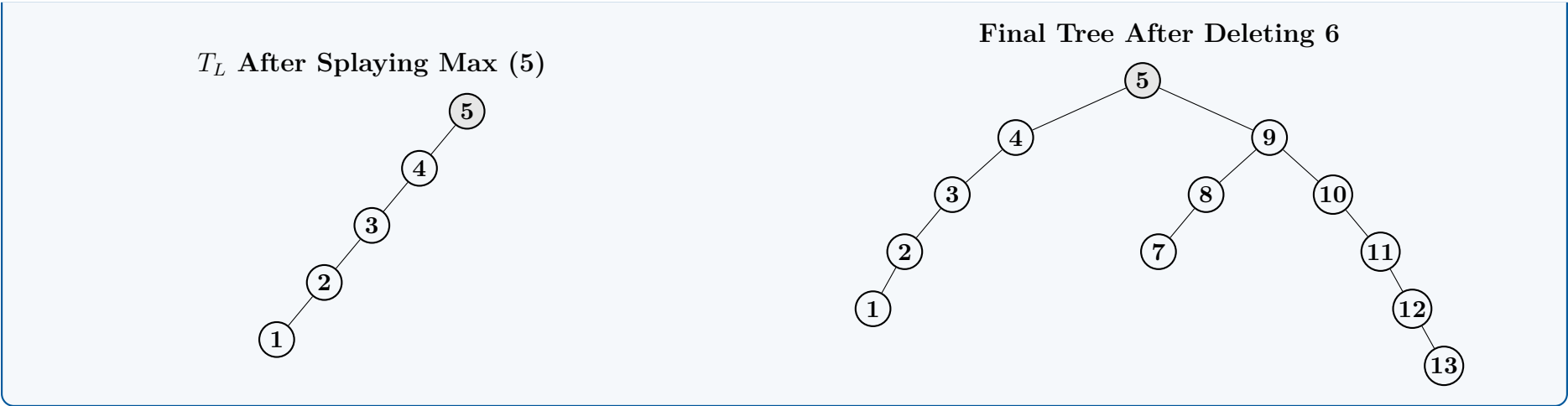
Tree After Splaying 6 (Step 3.2)



Deletion and Join Operation

We remove the root 6, splitting the tree into a Left Subtree (T_L rooted at 3) and a Right Subtree (T_R rooted at 9). To merge them:

- We find the maximum element in T_L , which is 5.
- We splay 5 to the root of T_L . This requires a **Zig-Zig** step (Left-Rotate at 3, Left-Rotate at 4).
- Since 5 is the maximum, it has no right child. We attach T_R as the right child of 5.



Q6. Perform SPLAY(1) on the given binary search tree, showing all intermediate steps and operations performed at each node. (10 marks) [CSE 207 | L-2/T-2 | 2017–18]



Answer

Concept of Splay Operation

In a Splay Tree, when a node x is accessed, it is moved to the root of the tree using a sequence of rotations known as splaying. The specific rotation applied at each step depends on the relationship between x , its parent p , and its grandparent g (if it exists):

- **Zig:** If x has a parent p but no grandparent (p is the root), we perform a single rotation at p .
- **Zig-Zig:** If x and p are either both left children or both right children, we perform a rotation at g followed by a rotation at p .
- **Zig-Zag:** If x is a left child and p is a right child (or vice versa), we perform a rotation at p followed by a rotation at g .

Step-by-Step Execution of Splay(1)

The target node is $x = 1$. In the initial tree, the path from the root to 1 is $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$. We will splay node 1 bottom-up.
Note: In the diagrams below, the nodes are structurally aligned to their x-coordinates strictly based on their values (x-axis matches the Binary Search Tree ordering) to perfectly visualize the rotations.

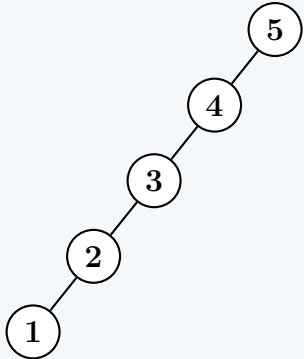
Step 1: Zig-Zig on Node 1

Currently, $x = 1$, parent $p = 2$, and grandparent $g = 3$. Since both 1 and 2 are left children, this is a **Left-Left** case. We perform a **Zig-Zig** operation, which consists of two right rotations:

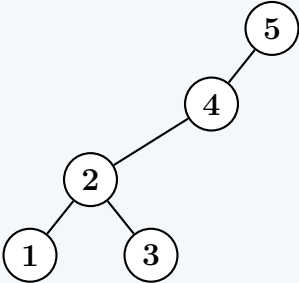
(a) Right Rotate at Grandparent (3)

(b) Right Rotate at Parent (2)

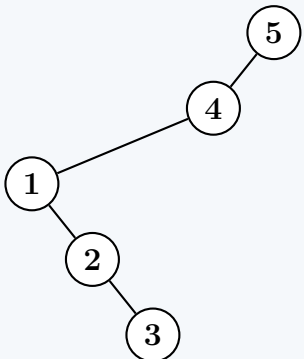
Initial Tree



1. Right Rotate at 3



2. Right Rotate at 2



Step 2: Zig-Zig on Node 1

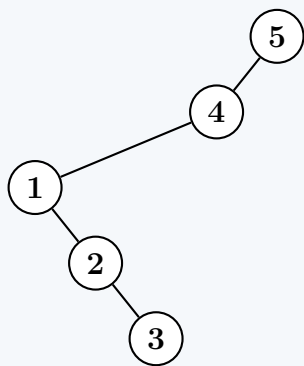
After Step 1, $x = 1$ has moved up. Now, its parent is $p = 4$, and its grandparent is $g = 5$. Since both 1 and 4 are left children, this is again a **Left-Left** case. We perform another **Zig-Zig** operation:

(a) Right Rotate at Grandparent (5)

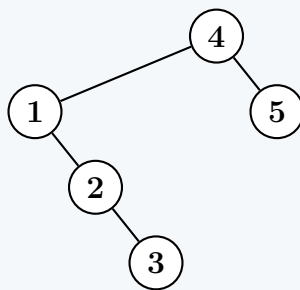
219

(b) Right Rotate at Parent (4)

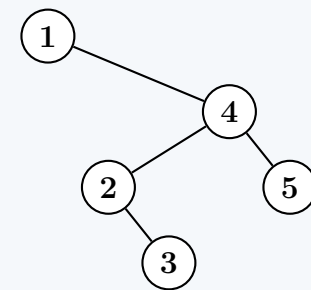
Current Tree



1. Right Rotate at 5



2. Right Rotate at 4



Node 1 has now reached the root. The $\text{SPLAY}(1)$ operation is strictly complete. The initial heavily skewed tree of height 4 has been reorganized, and its height has been reduced, which is the self-optimizing characteristic of a Splay Tree. An inorder traversal of the final tree yields 1, 2, 3, 4, 5, preserving the Binary Search Tree properties correctly.

Complexity Analysis:

- **Time Complexity:** $\mathcal{O}(h)$ where h is the current height of the tree. For this specific operation on the skewed tree, it takes $\mathcal{O}(N)$ time. However, a sequence of M splay operations takes $\mathcal{O}(M \log N)$ time, yielding an amortized time complexity of $\mathcal{O}(\log N)$ per operation.
- **Space Complexity:** $\mathcal{O}(1)$ auxiliary space if the rotations are executed iteratively using pointers.

Q7. What is the main objective of splay trees? Do splay trees explicitly try to balance the height? (5 marks) [CSE 207 | L-2/T-2 | 2017–18]

Answer

Main Objective of Splay Trees

The primary objective of a Splay Tree is to optimize access times for frequently or recently accessed elements by leveraging the **temporal locality of reference**. In many practical applications, an element accessed once is highly likely to be accessed again in the near future.

Splay trees achieve this objective by applying a heuristic: every time an element is accessed, searched, inserted, or modified, a sequence of specific rotations (splaying) is performed to move that element directly to the **root of the tree**. Consequently:

- Subsequent accesses to the same node are extremely fast ($\mathcal{O}(1)$ time).
- Elements accessed frequently cluster near the root, ensuring rapid retrieval.
- The data structure guarantees an **amortized time complexity** of $\mathcal{O}(\log n)$ for basic operations (insertion, look-up, deletion) over a sequence of m operations, even if a single operation occasionally takes $\mathcal{O}(n)$ time.

Do Splay Trees Explicitly Try to Balance the Height?

No, Splay Trees do not explicitly attempt to balance their height.

Unlike strict self-balancing binary search trees such as **AVL Trees** or **Red-Black Trees**, Splay Trees operate without any structural invariants or metadata:

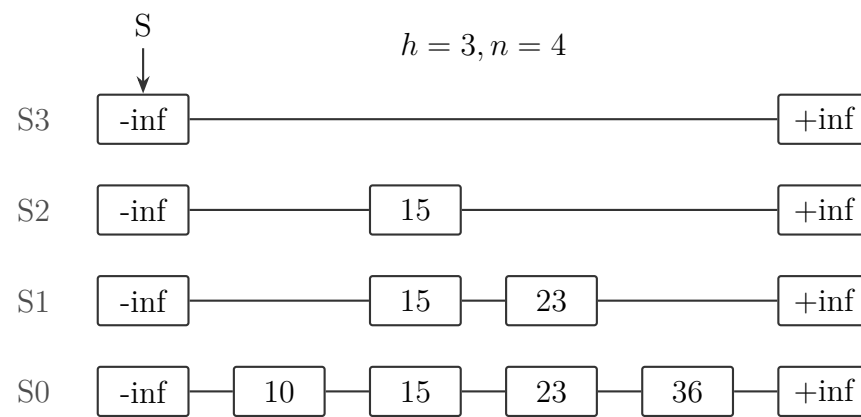
- **No Explicit Invariants:** They do not enforce a maximum height difference between subtrees (like AVL trees) or adhere to color-path rules (like Red-Black trees).
- **No Metadata Required:** Nodes in a Splay Tree do not store balance factors, heights, or colors, which makes the nodes structurally simpler and saves memory.
- **Worst-Case Height:** Because they do not enforce explicit balance, a Splay Tree can technically degrade into a completely skewed shape (a linked list) with a height of $\mathcal{O}(n)$ under certain access patterns (e.g., sequential insertions).

Implicit (Heuristic) Balancing Effect:

While Splay Trees do not *explicitly* track or correct height imbalance, the splaying mechanism (specifically the Zig-Zig and Zig-Zag rotations) naturally reorganizes the tree. As a deep node is splayed to the root, the path to that node is roughly **halved in length**. This provides an *implicit* self-adjusting effect; a highly imbalanced tree will rapidly "flatten" itself as soon as a deep, expensive node is accessed. Thus, balancing in Splay Trees is a beneficial side-effect of the access heuristic, not an explicitly enforced structural property.

Skip Lists

Q1. Prove that the expected space usage of a skip list having n items is $\mathcal{O}(n)$. Perform the following operations on the given skip list sequentially: (i) Insert key = 17 where $i = 2$, (ii) Insert key = 25 where $i = 4$, (iii) Delete key = 25. Show all the steps and the resulting skip lists.



(15 marks)

[CSE 207 | L-2/T-2 | 2020–21]

Answer**1. Proof: Expected Space Usage of a Skip List**

A skip list is a probabilistic data structure where each of the n inserted items is stored in a "tower" of nodes. The total space usage is proportional to the total number of nodes across all levels.

- Let X_j be a random variable representing the height of the tower for the j -th item.
- In a standard skip list, an item always exists at level L_0 . It is promoted to level L_{i+1} from level L_i with a probability p (typically $p = 1/2$).
- The probability that the height X_j is exactly k is given by the geometric distribution: $P(X_j = k) = p^{k-1}(1-p)$ for $k \geq 1$.
- The expected height of a single tower is: $E[X_j] = \sum_{k=1}^{\infty} k \cdot P(X_j = k) = \frac{1}{1-p}$.
- Let S be the total number of nodes in the skip list for n items. By the linearity of expectation:

$$E[S] = E\left[\sum_{j=1}^n X_j\right] = \sum_{j=1}^n E[X_j] = \sum_{j=1}^n \frac{1}{1-p} = \frac{n}{1-p}$$

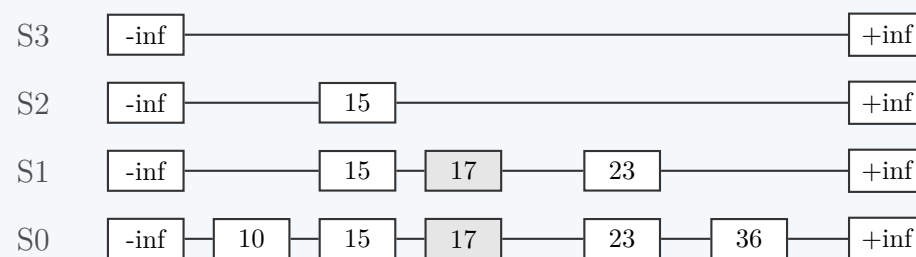
- For the common case where $p = 1/2$, $E[S] = 2n$.

Since the number of sentinel nodes ($-\text{inf}, +\text{inf}$) is proportional to the height of the tree ($\log n$ with high probability), they do not dominate the $O(n)$ term. Thus, the expected space usage is $O(n)$.

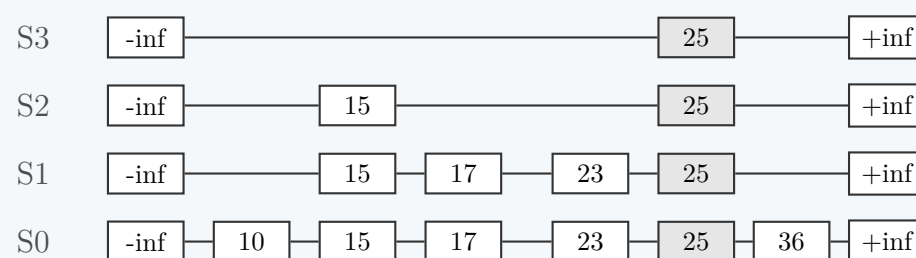
2. Sequential Operations

We use the convention that i denotes the height of the node (number of levels it occupies, starting from S_0).

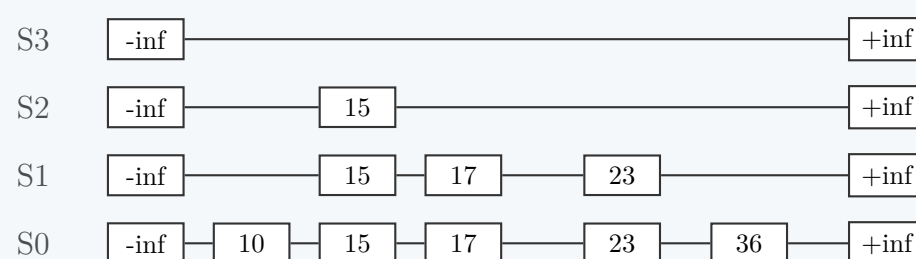
(i) **Insert key = 17 where $i = 2$** The key 17 is inserted between 15 and 23. Since $i = 2$, it will be present in levels S_0 and S_1 .



(ii) **Insert key = 25 where $i = 4$** The key 25 is inserted between 23 and 36. Since $i = 4$, it occupies levels S_0, S_1, S_2 , and S_3 .



(iii) **Delete key = 25** The node containing 25 is removed from all levels, and its predecessors are linked to its successors.



Q2. In a skip list with n entries,

- (a) Prove that the expected space used is $O(n)$.

(b) Prove that the expected search and insertion time is $O(\log n)$.

(15 marks)

[CSE 207 | L-2/T-2 | 2018–19]

Answer: Skip List Complexity Proofs

In a standard skip list, each element is inserted at the base level (level 0) and is repeatedly promoted to the next level with a fixed probability p (typically $p = 1/2$). The promotion stops as soon as a "tails" (failure) is flipped.

1. Proof: Expected Space Used is $O(n)$

To evaluate the space complexity, we must count the total number of pointers (or nodes) across all levels of the skip list.

- Let n be the total number of elements inserted.
- An element appears at level i if and only if it has been promoted i times successfully. The probability of an element reaching level i is p^i .
- Let X_i be a random variable representing the number of elements at level i . By the linearity of expectation, the expected number of elements at level i is:

$$E[X_i] = n \cdot p^i$$

- The total expected space (total number of nodes/pointers S) is the sum of the expected number of elements at all possible levels i from 0 to ∞ :

$$E[S] = \sum_{i=0}^{\infty} E[X_i] = \sum_{i=0}^{\infty} n \cdot p^i = n \sum_{i=0}^{\infty} p^i$$

- The summation is a standard infinite geometric series. Because $0 < p < 1$, it converges to $\frac{1}{1-p}$:

$$E[S] = n \left(\frac{1}{1-p} \right)$$

Since p is a constant fraction (e.g., $1/2$), the term $\frac{1}{1-p}$ is a constant (e.g., 2). Therefore, $E[S] = 2n$.

Conclusion: The expected space complexity is asymptotically bounded by $O(n)$.

2. Proof: Expected Search and Insertion Time is $O(\log n)$

We can analyze the time complexity by evaluating the expected height of the skip list and using a backwards search path analysis.

Part A: Expected Maximum Height

The probability that a specific element reaches level k is p^k . The probability that *at least one* of the n elements reaches level k is bounded by the union bound: $n \cdot p^k$. If we define $k = c \log_{1/p} n$ (for some constant $c > 1$), then:

$$P(\text{Height} \geq c \log_{1/p} n) \leq n \cdot p^{c \log_{1/p} n} = n \cdot n^{-c} = n^{1-c}$$

As n grows, this probability approaches 0. Thus, with high probability, the maximum height of the skip list is bounded by $O(\log n)$.

Part B: Backwards Path Analysis (Search Time)

Consider a search path, but trace it *backwards* from the target node at the bottom level back up to the top-left root node. At any given node in the path, the previous step was either:

- From above (Moved Down):** This occurs if the current node was promoted to the next level, which happens with probability p .
- From the left (Moved Right):** This occurs if the current node was *not* promoted, which happens with probability $1 - p$.

Let $C(k)$ be the expected number of steps required to climb k levels. At each step, we flip a biased coin:

- With probability p , we go up one level (height increases).
- With probability $1 - p$, we go left (height stays the same).

The number of leftward steps taken before finally making an upward step follows a geometric distribution with success probability p . The expected number of steps to achieve one upward movement is:

$$\frac{1}{p}$$

Since we know from Part A that the maximum expected height k is $O(\log n)$, the total expected length of the search path is:

$$\begin{aligned} E[\text{Total Steps}] &= (\text{Expected steps per level}) \times (\text{Expected max levels}) \\ &= \left(\frac{1}{p} \right) \times O(\log n) \\ &= O(\log n) \quad (\text{since } p \text{ is constant}) \end{aligned}$$

Part C: Insertion Time

An insertion operation consists of two phases:

- (a) **Search:** Finding the correct insertion points at each level, which takes $\mathcal{O}(\log n)$ expected time as proven above.
- (b) **Splicing/Promotion:** Generating a random level for the new node (takes $\mathcal{O}(1)$ expected coin flips) and updating the pointers. Since the expected height of the new node is $\mathcal{O}(1)$, and at worst bounded by $\mathcal{O}(\log n)$, the pointer updates take $\mathcal{O}(\log n)$ worst-case time.

Conclusion: Adding the two phases together, the total expected insertion time is structurally bound to the search time: $\mathcal{O}(\log n) + \mathcal{O}(\log n) = \mathcal{O}(\log n)$.

Q3. Compare linked lists and skip lists. Show that the expected space used by a skip list with n entries is $\mathcal{O}(n)$. Draw the skip-list for the following table that shows the keys and consecutive numbers of heads found in a coin toss while inserting the keys:

Key	15	40	35	28	21	65	84	79	50
No. of consecutive heads	1	4	1	0	3	2	2	3	1

(15 marks)

[CSE 203 | L-2/T-1 | 2014–15, 2013–14]

Answer**1. Comparison: Linked Lists vs. Skip Lists**

Feature	Linked List	Skip List
Structure	Single-level, linear sequence of nodes.	Multi-level, hierarchical structure of linked lists.
Search Time	$\mathcal{O}(n)$ in both average and worst cases (must traverse sequentially).	Expected $\mathcal{O}(\log n)$, as higher levels allow "skipping" over elements.
Insert / Delete	$\mathcal{O}(n)$ to find the position, $\mathcal{O}(1)$ to relink.	Expected $\mathcal{O}(\log n)$ to find and update pointers across levels.
Space Complexity	Exactly $\mathcal{O}(n)$, minimal pointer overhead (1 per node).	Expected $\mathcal{O}(n)$, higher constant factor due to multiple pointers per node.
Determinism	Completely deterministic operations.	Probabilistic/Randomized (uses coin tosses for node heights).

2. Proof: Expected Space of a Skip List is $\mathcal{O}(n)$

A skip list stores n entries. Every entry is guaranteed to appear at the base level (S_0). When an entry is inserted, a fair coin is flipped; as long as it comes up "heads" (probability $p = \frac{1}{2}$), the element is promoted to the next level.

- Let X_i be the random variable representing the total number of levels node i occupies.
- A node occupies exactly k levels if we flip $k - 1$ consecutive heads followed by a tail. The probability of this event follows a geometric distribution:

$$P(X_i = k) = p^{k-1}(1 - p)$$

- The expected number of levels for a single node i is the expected value of this geometric distribution:

$$E[X_i] = \sum_{k=1}^{\infty} k \cdot P(X_i = k) = \sum_{k=1}^{\infty} k \cdot p^{k-1}(1 - p) = \frac{1}{1 - p}$$

- For a fair coin ($p = \frac{1}{2}$), $E[X_i] = \frac{1}{1-0.5} = 2$. Thus, an average node occupies 2 levels.
- Let S be the total space (number of node references) across all levels. By Linearity of Expectation:

$$E[S] = E\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n 2 = 2n$$

Since $2n \in \mathcal{O}(n)$, the expected space complexity of a skip list with n elements is $\mathcal{O}(n)$.

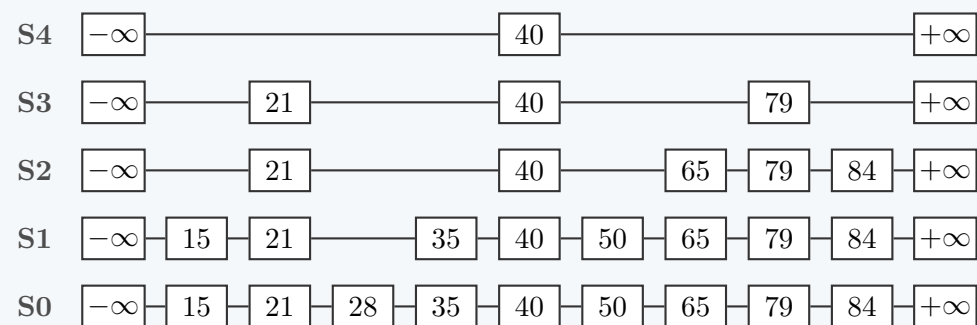
3. Drawing the Skip List

Interpretation: If an element yields k consecutive heads, it is promoted k times. Therefore, it will exist in exactly $k + 1$ levels: $S_0, S_1, \dots, S_k$.

First, we sort the keys to maintain the ordered property of the skip list, and assign their heights based on the provided table:

- 15:** 1 head $\rightarrow$ Levels S_0, S_1

- **21:** 3 heads $\rightarrow$ Levels S_0, S_1, S_2, S_3
- **28:** 0 heads $\rightarrow$ Level S_0
- **35:** 1 head $\rightarrow$ Levels S_0, S_1
- **40:** 4 heads $\rightarrow$ Levels S_0, S_1, S_2, S_3, S_4
- **50:** 1 head $\rightarrow$ Levels S_0, S_1
- **65:** 2 heads $\rightarrow$ Levels S_0, S_1, S_2
- **79:** 3 heads $\rightarrow$ Levels S_0, S_1, S_2, S_3
- **84:** 2 heads $\rightarrow$ Levels S_0, S_1, S_2



Q4. Write algorithms for the search, insertion and deletion operations of skip lists. Show that the expected search, insertion and deletion time is $O(\log n)$ in a skip list with n entries. Also show that the expected space used is $O(n)$. **(18 marks)** [CSE 203 | L-2/T-1 | 2013–14]

Answer: Skip List Algorithms and Complexity Proofs (18 Marks)

1. Algorithms for Skip List Operations

Let `MAX_LEVEL` be the maximum allowed height of the skip list, and p be the fraction of nodes promoted to the next level (typically $p = 0.5$). Each node contains an array of forward pointers, `forward[0...level]`.

A. Search Algorithm

To search for a key, we start at the highest level of the header (dummy) node and traverse forward. When the next node's key is greater than or equal to the target, we drop down to the next lower level.

Search(list, searchKey)

```

x ← list.header
for i ← list.level down to 0 do
    while x.forward[i].key < searchKey do
        x ← x.forward[i]
x ← x.forward[0] // x is now the largest element < searchKey
if x.key == searchKey then
    return x.value // Success
else
    return failure

```

B. Insertion Algorithm

Insertion requires finding the correct position while keeping track of the "drop-down" nodes at each level. We store these nodes in an `update` array so we can rewire their pointers to include the new node.

Insert(list, searchKey, newValue)

```

allocate array update[MAX_LEVEL]
x ← list.header
// Phase 1: Search and record update path
for i ← list.level down to 0 do
    while x.forward[i].key < searchKey do
        x ← x.forward[i]
    update[i] ← x
x ← x.forward[0]
if x.key == searchKey then
    x.value ← newValue // Key exists, overwrite
else
    // Phase 2: Splicing in the new node
    lvl ← RandomLevel() // Coin flips with probability p
    if lvl > list.level then

```

```

    for  $i \leftarrow \text{list.level} + 1$  to  $\text{lvl}$  do
         $\text{update}[i] \leftarrow \text{list.header}$ 
     $\text{list.level} \leftarrow \text{lvl}$ 
     $\text{newNode} \leftarrow \text{MakeNode}(\text{lvl}, \text{searchKey}, \text{newValue})$ 
    for  $i \leftarrow 0$  to  $\text{lvl}$  do
         $\text{newNode.forward}[i] \leftarrow \text{update}[i].\text{forward}[i]$ 
         $\text{update}[i].\text{forward}[i] \leftarrow \text{newNode}$ 

```

C. Deletion Algorithm

Deletion uses the same `update` array logic to find the node and bypass its pointers.

```

Delete(list, searchKey)
    allocate array  $\text{update}[\text{MAX\_LEVEL}]$ 
     $x \leftarrow \text{list.header}$ 
    // Phase 1: Search and record update path
    for  $i \leftarrow \text{list.level}$  down to 0 do
        while  $x.\text{forward}[i].\text{key} < \text{searchKey}$  do
             $x \leftarrow x.\text{forward}[i]$ 
         $\text{update}[i] \leftarrow x$ 
     $x \leftarrow x.\text{forward}[0]$ 
    if  $x.\text{key} == \text{searchKey}$  then
        // Phase 2: Bypass and remove
        for  $i \leftarrow 0$  to  $\text{list.level}$  do
            if  $\text{update}[i].\text{forward}[i] \neq x$  then break
             $\text{update}[i].\text{forward}[i] \leftarrow x.\text{forward}[i]$ 
        FreeNode( $x$ )
        // Shrink the max level if the top levels are now empty
        while  $\text{list.level} > 0$  and  $\text{list.header.forward}[\text{list.level}] == \text{NIL}$  do
             $\text{list.level} \leftarrow \text{list.level} - 1$ 

```

2. Proof: Expected Search, Insertion, and Deletion Time is $\mathcal{O}(\log n)$

All three operations depend fundamentally on the time it takes to traverse the list (the search path).

Step 1: Expected Height of the Skip List

The probability of a single node reaching level k is p^k . Using the union bound, the probability that *any* of the n nodes reaches level $c \log_{1/p} n$ is:

$$P(\text{max level} \geq c \log_{1/p} n) \leq n \cdot p^{c \log_{1/p} n} = n \cdot n^{-c} = n^{1-c}$$

For a constant $c > 1$, this approaches 0 as n grows. Thus, the expected maximum height of the list is bounded by $\mathcal{O}(\log n)$.

Step 2: Backwards Path Analysis

To find the expected length of the search path, trace the search path *backwards* from the target node at level 0 back up to the top-left header node. At any node, the preceding step was either:

- (a) **A step down:** This means the node was promoted to the next level (probability p).
- (b) **A step right:** This means the node was not promoted (probability $1 - p$).

The expected number of leftward steps before making an upward step follows a geometric distribution. The expected number of steps to climb one level is $\frac{1}{p}$.

Since the maximum expected height is $\mathcal{O}(\log n)$, the total expected search path length is:

$$E[\text{Total Steps}] = \left(\frac{1}{p}\right) \times \mathcal{O}(\log n)$$

Since p is a constant, this simplifies to $\mathcal{O}(\log n)$.

Step 3: Total Complexities

- **Search:** $\mathcal{O}(\log n)$ traversal time.
- **Insertion:** $\mathcal{O}(\log n)$ traversal time + at most $\mathcal{O}(\log n)$ constant-time pointer updates = $\mathcal{O}(\log n)$.
- **Deletion:** $\mathcal{O}(\log n)$ traversal time + at most $\mathcal{O}(\log n)$ constant-time pointer updates = $\mathcal{O}(\log n)$.

3. Proof: Expected Space Used is $\mathcal{O}(n)$

The total space used is proportional to the total number of forward pointers across all nodes at all levels.

- Every node is inserted at the base level (level 0). So, there are exactly n nodes at level 0.
- A node is promoted to level i with probability p^i .
- Let X_i be a random variable representing the number of nodes at level i . By the linearity of expectation, the expected number of nodes at level i is:

$$E[X_i] = n \cdot p^i$$

The total expected space $E[S]$ is the sum of expectations across all possible levels from 0 to ∞ :

$$E[S] = \sum_{i=0}^{\infty} E[X_i] = \sum_{i=0}^{\infty} np^i = n \sum_{i=0}^{\infty} p^i$$

This summation is a standard infinite geometric series. For $0 < p < 1$, it converges to $\frac{1}{1-p}$:

$$E[S] = n \left(\frac{1}{1-p} \right)$$

Because p is a constant fraction (e.g., $p = 1/2$), the term $\frac{1}{1-p}$ is also a constant (2). Therefore, $E[S] = 2n$, which proves that the expected space complexity is strictly bounded by $\mathcal{O}(n)$.

Q5. Draw the skip-list for the following table that shows the keys and consecutive numbers of heads found in a coin toss while inserting the keys in a probabilistic skip-list:

Key	3	6	7	9	12	17	19	21	25	26
No. of consecutive heads	1	4	1	2	1	2	1	1	3	1

Perform the following operations on the skip-list and show the procedures:

- (a) Search 21
- (b) Insert 22 with 3 consecutive heads
- (c) Delete 19
- (d) Search 18

(7+8=15 marks)

[CSE 203 | L-2/T-1 | 2011–12]

Answer

Assumption / Interpretation of "Consecutive Heads"

In a skip list, the number of consecutive heads flipped during insertion dictates the height of the node's tower. To prevent all nodes from redundantly occupying a base level and an identical second level, we use the standard convention that a node with k consecutive heads occupies exactly k levels. Thus, it will be present in levels $S_0, S_1, \dots, S_{k-1}$.

- **1 head** $\implies$ occupies 1 level (S_0)
- **2 heads** $\implies$ occupies 2 levels (S_0, S_1)
- **3 heads** $\implies$ occupies 3 levels (S_0, S_1, S_2)
- **4 heads** $\implies$ occupies 4 levels (S_0, S_1, S_2, S_3)

Initial Skip List Construction

Based on the table, the level distributions are:

- **S3:** 6
- **S2:** 6, 25
- **S1:** 6, 9, 17, 25
- **S0:** 3, 6, 7, 9, 12, 17, 19, 21, 25, 26

S3

-∞

6

+∞

S2

-∞

6

25

+∞

S1

-∞

6

9

17

25

+∞

S0

-∞

3

6

7

9

12

17

19

21

25

26

+∞

1. Search 21

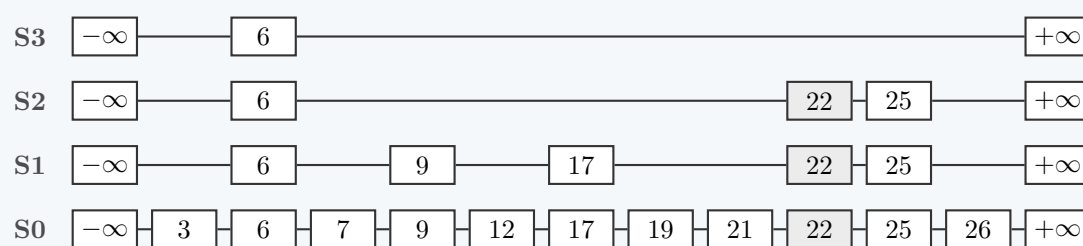
We start at the top-left node ($-\infty$ at level S_3) and move right if the next key is ≤ 21 , or drop down if the next key is > 21 :

- **Level S3:** Next is $6 \leq 21 \Rightarrow$ move to 6. Next is $+\infty > 21 \Rightarrow$ drop down to S_2 .
- **Level S2:** Next is $25 > 21 \Rightarrow$ drop down to S_1 at 6.
- **Level S1:** Next is $9 \leq 21 \Rightarrow$ move to 9. Next is $17 \leq 21 \Rightarrow$ move to 17. Next is $25 > 21 \Rightarrow$ drop down to S_0 at 17.
- **Level S0:** Next is $19 \leq 21 \Rightarrow$ move to 19. Next is $21 \Rightarrow$ move to 21. $21 == 21 \Rightarrow$ **Target Found**.

2. Insert 22 with 3 consecutive heads

Procedure: With 3 heads, node 22 will occupy levels S_0, S_1 , and S_2 . We first search for 22 to find the predecessor node at each level:

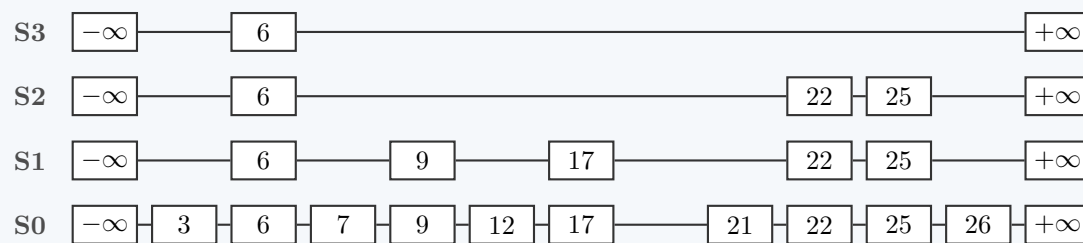
- S_3 : Next from 6 is $+\infty > 22 \Rightarrow$ drop down. (No insertion at S_3).
- S_2 : Next from 6 is $25 > 22 \Rightarrow$ Predecessor is **6**. Insert 22 after 6.
- S_1 : Path is $6 \rightarrow 9 \rightarrow 17$. Next is $25 > 22 \Rightarrow$ Predecessor is **17**. Insert 22 after 17.
- S_0 : Path is $17 \rightarrow 19 \rightarrow 21$. Next is $25 > 22 \Rightarrow$ Predecessor is **21**. Insert 22 after 21.

**3. Delete 19**

Procedure: Search for 19 to locate it and its predecessors:

- S_3, S_2 : Traverse and drop at 6.
- S_1 : Traverse $6 \rightarrow 9 \rightarrow 17$. Next is $22 > 19 \Rightarrow$ drop down at 17.
- S_0 : Next is 19. **Target Found**.

Since 19 exists only in S_0 , we update its predecessor's pointer: node 17 at S_0 is updated to bypass 19 and point directly to node 21. Node 19 is then removed.

**4. Search 18**

Procedure: We search the updated skip list for 18 starting at S_3 :

- **Level S3:** Move to 6. Next is $+\infty > 18 \Rightarrow$ drop down to S_2 .
- **Level S2:** At 6, next is $22 > 18 \Rightarrow$ drop down to S_1 .
- **Level S1:** Move to 9. Move to 17. Next is $22 > 18 \Rightarrow$ drop down to S_0 .
- **Level S0:** At 17, next is 21 (because 19 was deleted). $21 > 18 \Rightarrow$ Since we are at the base level and the next element is strictly greater than the target, we conclude **Target 18 is Not Found**.

Q6. Show that the expected running time of searching in a probabilistic skip-list is $O(\log n)$. (6 marks) [CSE 203 | L-2/T-1 | 2011–12]

Answer: Proof of $O(\log n)$ Expected Search Time (6 Marks)

To prove that the expected search time in a probabilistic skip list with n entries is $O(\log n)$, we use the standard technique of **Backward Analysis**.

1. Tracing the Search Path Backwards

Instead of tracing the forward search path down and to the right, we trace the path **backwards** starting from the target node at the base level (level 0) up to the header node at the top level.

At any given node v at level i during this reverse traversal:

- The path moves **Up** to level $i + 1$ if v was promoted to level $i + 1$. In the random choice configuration of a skip list, this occurs with a fixed constant probability p (typically $p = 0.5$).
- The path moves **Left** to a preceding node at the same level i if v was not promoted to level $i + 1$. This occurs with probability $1 - p$.

2. Cost Per Level (Geometric Distribution)

Let C be a random variable representing the total number of steps (upward and leftward) taken in the backward traversal.

The number of leftward steps taken at any level before successfully moving up one level corresponds to a sequence of independent trials ending on the first success (moving up). This fits a **Geometric Distribution** with a success probability of p .

The expected number of steps required to ascend exactly **one level** is:

$$E[\text{steps per level}] = \frac{1}{p}$$

3. Bounding the Maximum Height

The probability that a given node reaches a level k is p^k . Using the union bound, the probability that any of the n nodes reaches a level $k = \log_{1/p} n$ is at most:

$$n \cdot p^{\log_{1/p} n} = n \cdot \frac{1}{n} = 1$$

With high probability, the maximum height L of the skip list is strictly bounded by:

$$E[L] \leq \log_{1/p} n + \frac{1}{1-p} = \mathcal{O}(\log n)$$

4. Total Expected Search Time

By the linearity of expectation, the total expected number of steps $E[C]$ to climb from level 0 to the top level is the expected number of steps per level multiplied by the expected maximum height:

$$\begin{aligned} E[C] &\leq (\text{Expected steps per level}) \times E[L] \\ E[C] &\leq \frac{1}{p} \cdot \left(\log_{1/p} n + \frac{1}{1-p} \right) \end{aligned}$$

Since p is a fixed constant fraction independent of n , the terms $\frac{1}{p}$, $\log_{1/p}$, and $\frac{1}{1-p}$ are all constants. Thus, the expression simplifies asymptotically to:

$$E[C] = \mathcal{O}(\log n)$$

Conclusion: Because each step in a skip list search involves a constant number of pointer evaluations ($\mathcal{O}(1)$ time), the total expected running time for a search operation is bounded by $\mathcal{O}(\log n)$.

Advanced Heaps (Binomial Heaps)

Q1. Prove that there are $\binom{k}{i}$ nodes at depth i in a binomial heap B_k of order k . **(8 marks)** [CSE 207 | L-2/T-1 | 2024–25, 2019–20, 2018–19]

Answer

Terminology Note: In standard computer science literature, B_k denotes a *Binomial Tree* of order k . A *Binomial Heap* is defined as a collection of binomial trees. The property regarding $\binom{k}{i}$ nodes at depth i strictly applies to a single binomial tree B_k . We will construct the formal proof for the binomial tree B_k .

Theorem

A binomial tree B_k of order $k \geq 0$ has exactly $\binom{k}{i}$ nodes at depth i , for $i = 0, 1, 2, \dots, k$.

Proof by Mathematical Induction

We will prove this theorem using induction on the order of the binomial tree, k . Let $D(k, i)$ denote the number of nodes at depth i in a binomial tree B_k .

1. Base Case ($k = 0$): A binomial tree of order 0, B_0 , consists of a single node (the root). The root is at depth $i = 0$. The number of nodes at depth 0 is 1. Evaluating the binomial coefficient, we have:

$$\binom{0}{0} = 1$$

Since $D(0, 0) = \binom{0}{0}$, the base case holds.

2. Inductive Hypothesis: Assume that the theorem holds for a binomial tree B_{k-1} of order $k - 1$. That is, for all depths i , the

number of nodes in B_{k-1} is given by:

$$D(k-1, i) = \binom{k-1}{i}$$

(Note: If $i > k-1$ or $i < 0$, $\binom{k-1}{i} = 0$, which correctly reflects that there are no nodes at those depths).

3. Inductive Step: We must show that the property holds for B_k , meaning $D(k, i) = \binom{k}{i}$.

By the structural definition of binomial trees, a tree B_k is constructed by linking two binomial trees of order $k-1$. Let these two trees be T_1 and T_2 . The linkage is performed by making the root of T_2 the leftmost child of the root of T_1 .

Because T_2 is attached as a subtree to the root of T_1 , the depth of every node in T_2 increases by exactly 1 relative to its original depth in T_1 . Thus, the nodes at depth i in the newly formed B_k originate from two disjoint sets:

- Nodes from T_1 that were already at depth i .
- Nodes from T_2 that were previously at depth $i-1$ (since their depth has increased by 1).

Therefore, the total number of nodes at depth i in B_k is the sum of these two quantities:

$$D(k, i) = D(k-1, i) + D(k-1, i-1)$$

By substituting our inductive hypothesis into this equation, we get:

$$D(k, i) = \binom{k-1}{i} + \binom{k-1}{i-1}$$

According to **Pascal's Identity** in combinatorics, the sum of these two adjacent binomial coefficients is identically $\binom{k}{i}$:

$$\binom{k-1}{i} + \binom{k-1}{i-1} = \binom{k}{i}$$

Thus, we conclude:

$$D(k, i) = \binom{k}{i}$$

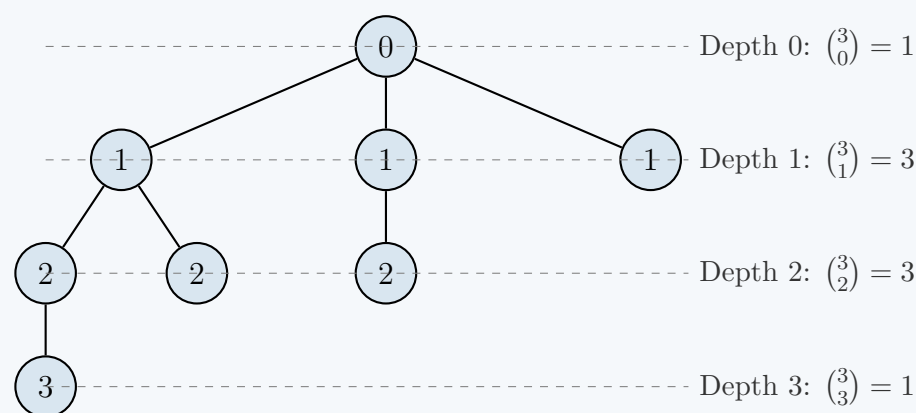
Boundary conditions check:

- For $i = 0$ (the root level): $D(k, 0) = D(k-1, 0) + 0 = \binom{k-1}{0} = 1 = \binom{k}{0}$.
- For $i = k$ (the maximum depth): $D(k, k) = 0 + D(k-1, k-1) = \binom{k-1}{k-1} = 1 = \binom{k}{k}$.

Both boundary conditions are satisfied. By the Principle of Mathematical Induction, the theorem is true for all $k \geq 0$. ■

Visualizing the Depths (Example for B_3)

The property directly mirrors Pascal's Triangle. The diagram below illustrates B_3 , formed by linking two B_2 trees. The depths correspond to the 3rd row of Pascal's Triangle: 1, 3, 3, 1.



Structural Complexity

- **Total Nodes:** Summing the nodes at all depths yields $\sum_{i=0}^k \binom{k}{i} = 2^k$. A binomial tree of order k always contains exactly 2^k nodes.
- **Maximum Degree:** The root node has the highest degree, which is k (i.e., $\log_2(N)$ where N is the number of nodes). This guarantees that heap operations achieve logarithmic time bounds.

Q2. Draw the maximally damaged tree resulting from B_6 . (8 marks)

[CSE 207 | L-2/T-1 | 2024–25, 2019–20]

Answer

Concept: The Maximally Damaged Tree

In the context of advanced data structures like **Fibonacci Heaps**, a binomial tree B_k becomes "damaged" when nodes are cut from it during 'Decrease-Key' operations.

To maintain a bounded maximum degree, Fibonacci heaps enforce a *cascading cut* rule: a non-root node can lose at most **one** child before it is forcibly cut from its parent. Therefore, a **maximally damaged tree** of order k (often denoted as F_k or a tree of rank k) is formed from B_k under the following worst-case conditions:

- Every non-root node loses exactly one child (any more would trigger a cascading cut).
- To minimize the remaining size of the tree, the lost child is the *heaviest* (highest degree) subtree.
- The root maintains its original k children (thus remaining a tree of order/rank k).

Deriving the Structure for B_6

A standard binomial tree B_6 has exactly $2^6 = 64$ nodes. The root has 6 children with degrees: 5, 4, 3, 2, 1, 0. Applying the maximum damage rule recursively:

- The child of degree 5 loses its heaviest child (degree 4), becoming degree 4.
- The child of degree 4 loses its heaviest child (degree 3), becoming degree 3.
- ...and so on. The child of degree 0 cannot lose a child, so it remains degree 0.

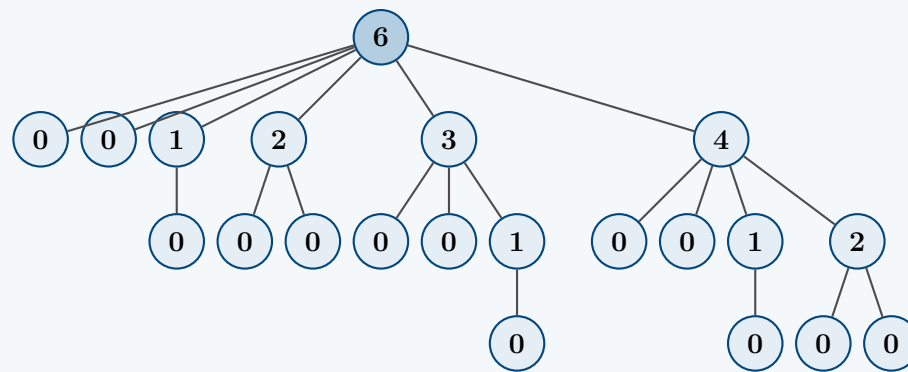
The new degrees of the root's children become precisely: **4, 3, 2, 1, 0, 0**.

This recursive "thinning" mimics the Fibonacci sequence. Let S_k be the minimum number of nodes in a maximally damaged tree of rank k . The recurrence relation is $S_k = S_{k-2} + S_{k-1}$ (with base cases $S_0 = 1, S_1 = 2$). The total size strictly follows the Fibonacci sequence $S_k = F_{k+2}$ (where $F_1 = 1, F_2 = 1, F_3 = 2, \dots$).

For B_6 , the maximally damaged tree has exactly **$F_8 = 21$ nodes** (a significant reduction from 64).

Diagram: Maximally Damaged Tree from B_6

*Note: The integer inside each node represents its current **degree** (number of children). The tree is spread horizontally to prevent overlap.*



Complexity & Analytical Properties

- Total Nodes:** Summing the nodes yields 1 (root) $+ 1 + 1 + 2 + 3 + 5 + 8 = 21$.
- Implication on Time Complexity:** Because a tree of order k cannot have fewer than F_{k+2} nodes, and $F_{k+2} \geq \phi^k$ (where ϕ is the golden ratio ≈ 1.618), the maximum degree $D(n)$ of any node in an n -node Fibonacci Heap is bounded by $\log_\phi n = \mathcal{O}(\log n)$. This tight mathematical bound is the core reason **Extract-Min** operates in amortized $\mathcal{O}(\log n)$ time despite extreme potential tree damage.

Q3. Let H_1 and H_2 be binomial heaps. Suppose the highest order tree in heap H_1 is of order m , and the highest order tree in H_2 has degree $m + 1$. Prove or disprove: " H_2 must have more nodes than H_1 , regardless of how many trees are present in H_1 and H_2 ." (7 marks) [CSE 207 | L-2/T-1 | 2024–25]

Answer

Verdict: True

We will **prove** the statement. The claim is rigorously true due to the foundational structural properties of binomial heaps, which possess a direct isomorphism with the binary numeral system.

Formal Proof

By the strict definition of a binomial heap, for any integer $k \geq 0$, a binomial heap can contain **at most one** binomial tree of order k . Furthermore, the number of nodes in a single binomial tree of order k , denoted B_k , is exactly 2^k .

Step 1: Establishing the Upper Bound for H_1 Let N_1 be the total number of nodes in H_1 . We are given that the highest order tree in H_1 is m . To maximize the number of nodes in H_1 , we must assume the "densest" possible configuration, where H_1 contains a binomial tree of *every* possible order from 0 up to m .

Thus, the maximum possible number of nodes in H_1 is the sum of a geometric series:

$$\max(N_1) = \sum_{k=0}^m |B_k| = \sum_{k=0}^m 2^k$$

Using the standard geometric series formula $\sum_{i=0}^n r^i = \frac{r^{n+1}-1}{r-1}$, this evaluates to:

$$\max(N_1) = 2^{m+1} - 1$$

Step 2: Establishing the Lower Bound for H_2 Let N_2 be the total number of nodes in H_2 . We are given that the highest order tree in H_2 has order $m+1$. This guarantees that H_2 contains *at least* the tree B_{m+1} . To minimize the number of nodes in H_2 , we assume the "sparsest" possible configuration, where it contains *only* this single tree and no others.

Thus, the minimum possible number of nodes in H_2 is exactly the size of B_{m+1} :

$$\min(N_2) = |B_{m+1}| = 2^{m+1}$$

Step 3: Direct Comparison Comparing the strict bounds established above, we observe:

$$\max(N_1) = 2^{m+1} - 1$$

$$\min(N_2) = 2^{m+1}$$

Since $2^{m+1} - 1 < 2^{m+1}$ holds for all integers m , we can chain the inequalities:

$$N_1 \leq \max(N_1) < \min(N_2) \leq N_2$$

$$N_1 < N_2$$

Therefore, H_2 must strictly contain more nodes than H_1 , proving the statement. ■

Intuition via Binary Representation

A fundamental property of a binomial heap with N nodes is its one-to-one mapping with the binary representation of the integer N . Specifically, a binomial tree B_k is present in the heap if and only if the k -th bit of N (using 0-based indexing from right to left) is 1.

Property	Heap H_1 (Highest order m)	Heap H_2 (Highest order $m+1$)
Highest set bit position	m	$m+1$
Binary layout	$01 \dots \dots_2$ (length $m+1$)	$1 \dots \dots_2$ (length $m+2$)
Maximum possible value	$\underbrace{11 \dots 1}_m_2 = 2^{m+1} - 1$ $m+1$ bits	Arbitrarily large, but at least 2^{m+1}
Minimum possible value	$10 \dots 0_2 = 2^m$	$10 \dots 0_2 = 2^{m+1}$

Because adding a new, more significant bit (position $m+1$) to a binary number intrinsically results in a value larger than *any* possible combination of lower-order bits (positions 0 through m), the node count N_2 inherently dominates N_1 .

Q4. Prove the following properties for a binomial tree B_k of order k :

- The number of nodes is 2^k .
- The height is k .
- The degree of root is k .

(15 marks)

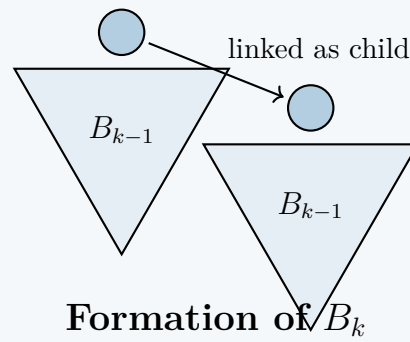
[CSE 207 | L-2/T-1 | 2023–24]

Answer

Structural Definition of a Binomial Tree

Before proving the properties, we formally state the recursive definition of a binomial tree B_k of order $k \geq 0$:

- Base Case:** A binomial tree B_0 consists of a single node.
- Recursive Step:** A binomial tree B_k (for $k > 0$) is constructed by linking two binomial trees of order $k-1$. Specifically, the root of one B_{k-1} tree is made the leftmost child of the root of the other B_{k-1} tree.



We will prove all three properties using **Mathematical Induction** on the order k .

1. Proof: The number of nodes is 2^k

Let $N(k)$ be the number of nodes in a binomial tree B_k .

- **Base Case ($k = 0$):** B_0 is defined as a single node. Thus, $N(0) = 1$. Evaluating the formula, $2^0 = 1$. The base case holds.
- **Inductive Hypothesis:** Assume that a binomial tree B_{k-1} has $N(k-1) = 2^{k-1}$ nodes.
- **Inductive Step:** By definition, B_k is formed by linking two B_{k-1} trees. Because the sets of nodes in both trees are disjoint, the total number of nodes in B_k is the sum of the nodes in the two B_{k-1} trees:

$$N(k) = N(k-1) + N(k-1)$$

Substitute the inductive hypothesis:

$$N(k) = 2^{k-1} + 2^{k-1} = 2 \cdot 2^{k-1} = 2^k$$

This proves the property for all $k \geq 0$. ■

2. Proof: The height is k

Let $H(k)$ denote the height (maximum depth of any node, where the root is at depth 0) of B_k .

- **Base Case ($k = 0$):** B_0 consists of a single node (the root). Its height is 0. The base case $H(0) = 0$ holds.
- **Inductive Hypothesis:** Assume that $H(k-1) = k-1$.
- **Inductive Step:** B_k consists of a primary B_{k-1} (whose root becomes the root of B_k) and an attached B_{k-1} (whose root becomes a child of the primary root). The depth of every node in the attached B_{k-1} increases by 1 relative to the new root. Therefore, the maximum depth in the attached portion is $H(k-1) + 1$. The maximum depth in the primary portion remains $H(k-1)$. The height of B_k is the maximum of these two:

$$H(k) = \max(H(k-1), H(k-1) + 1)$$

Clearly, $H(k-1) + 1$ is strictly greater. Substituting the inductive hypothesis:

$$H(k) = (k-1) + 1 = k$$

This proves the property for all $k \geq 0$. ■

3. Proof: The degree of the root is k

Let $D(k)$ denote the degree (number of direct children) of the root of B_k .

- **Base Case ($k = 0$):** B_0 is a single node with no children. Thus, $D(0) = 0$. The base case holds.
- **Inductive Hypothesis:** Assume that the root of B_{k-1} has degree $D(k-1) = k-1$.
- **Inductive Step:** When forming B_k , the root of one B_{k-1} tree is made a new child of the root of the other B_{k-1} tree. The root of the primary B_{k-1} retains all its original $k-1$ children and gains exactly one new child (the root of the attached tree). Thus, the degree of the new root is:

$$D(k) = D(k-1) + 1$$

Substituting the inductive hypothesis:

$$D(k) = (k-1) + 1 = k$$

This proves the property for all $k \geq 0$. ■

Q5. Perform union of two binomial min heaps with six and seven nodes respectively. You can use any valid values in the nodes. (10 marks) [CSE 207 | L-2/T-1 | 2023–24]

Answer

1. Foundation and Structural Analysis

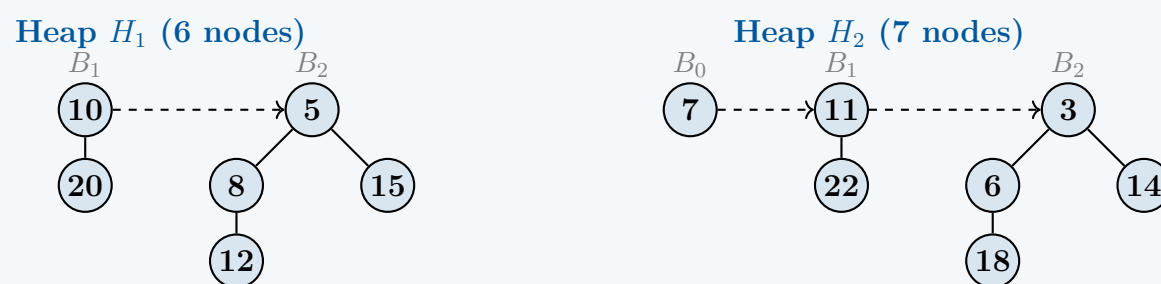
The operation of uniting two binomial heaps fundamentally mirrors binary addition. Let H_1 be the heap with 6 nodes and H_2 be the heap with 7 nodes. The sizes dictate the exact structural composition of the heaps via their binary representations:

- Size of $H_1 = 6_{10} = 110_2$. Therefore, H_1 contains binomial trees of orders 1 and 2 (B_1 and B_2).
- Size of $H_2 = 7_{10} = 111_2$. Therefore, H_2 contains binomial trees of orders 0, 1, and 2 (B_0 , B_1 , and B_2).

When we perform $H_1 \cup H_2$, the final number of nodes is $6 + 7 = 13_{10} = 1101_2$. This mathematical certainty guarantees that our resulting heap, H_{final} , will contain exactly three binomial trees: B_0 , B_2 , and B_3 .

2. Initial Heaps Setup

We initialize H_1 and H_2 with valid values that rigorously maintain the **Min-Heap property** (every parent is strictly $\leq$ its children). In standard binomial tree representation, children are typically ordered left-to-right by decreasing degree ($B_{k-1}, B_{k-2}, \dots, B_0$).



3. The Union Operation Step-by-Step

Step 3.1: Merge Root Lists

We merge the root lists of H_1 and H_2 into a single linked list, sorted strictly in monotonically increasing order of degree.

Merged List: $B_0(7) \rightarrow B_1(10) \rightarrow B_1(11) \rightarrow B_2(5) \rightarrow B_2(3)$

Step 3.2: Link Trees of Equal Degree

We traverse the merged list using three pointers: **prev-x**, **x**, and **next-x**. We combine trees of the same degree using the standard Binomial Heap linkage rule (the root with the smaller value becomes the parent).

- Initial State:** **x** points to $B_0(7)$. **next-x** is $B_1(10)$. Degrees differ. We advance the pointers.
- First Linkage ($B_1 \cup B_1$):** **x** points to $B_1(10)$, **next-x** points to $B_1(11)$. The following tree is $B_2(5)$.
 - Degrees match ($1 = 1$). We compare roots: $10 < 11$.
 - $B_1(11)$ is made the leftmost child of $B_1(10)$, forming a new $B_2(10)$.

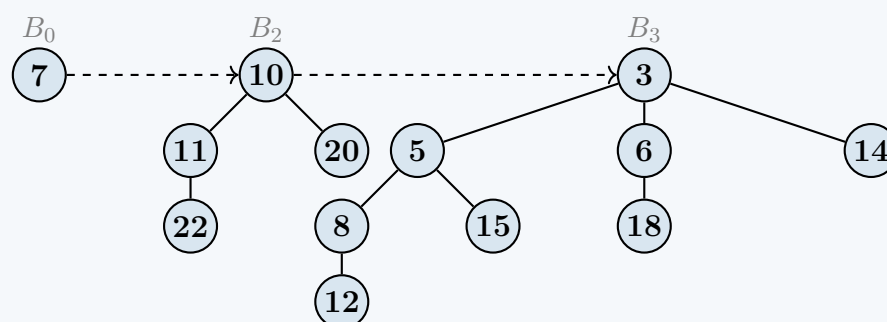
Intermediate List: $B_0(7) \rightarrow B_2(10) \rightarrow B_2(5) \rightarrow B_2(3)$

- Three trees of same degree (B_2, B_2, B_2):** **x** points to $B_2(10)$, **next-x** is $B_2(5)$, and **next-x.sibling** is $B_2(3)$.
 - Because **three** consecutive trees have the same degree, standard algorithms prescribe advancing **x** by one position to avoid destroying the monotonically increasing property. We skip linking the first pair.
 - **x** now points to $B_2(5)$.
- Second Linkage ($B_2 \cup B_2$):** **x** points to $B_2(5)$, **next-x** points to $B_2(3)$.
 - Degrees match ($2 = 2$). We compare roots: $3 < 5$.
 - $B_2(5)$ is made the leftmost child of $B_2(3)$, forming a new $B_3(3)$.

Final List: $B_0(7) \rightarrow B_2(10) \rightarrow B_3(3)$

4. Final Resulting Heap (H_{final})

The resulting heap contains exactly 13 nodes, properly distributed among B_0 , B_2 , and B_3 .



Complexity Analysis

- **Time Complexity:** $\mathcal{O}(\log N)$. Let $N = N_1 + N_2$. The maximum degree of any tree is $\lfloor \log_2 N \rfloor$. Thus, merging the root lists takes $\mathcal{O}(\log N)$, and iterating through the merged list to link trees takes $\mathcal{O}(\log N)$ pointer operations.
- **Space Complexity:** $\mathcal{O}(1)$ auxiliary space. The union is performed strictly in-place by updating existing pointers.

Q6. Answer the following questions for two max priority queues **pq1** and **pq2** implemented using binomial heap. Both are initially empty.

- Insert $-5, 20, 4, 19, 1, -3, 6, 12, 3, 2, 18$ in **pq1** and $25, 21, 10, 13, 17, 9, 8, 11, 7$ in **pq2**. Draw the two binomial heaps after the insertion.
- Meld **pq1** and **pq2** and then decrease the key of node 20 to -20 from the melded priority queue. Show the necessary steps.
- What will be the total actual cost in credits if we call the Extract-Max operation two times consecutively on the resultant priority queue? Assume that unlinking a node from its children takes one credit, two node addition operations take one credit, an insertion of a node takes one credit, and a two-key comparison takes one credit. All other operations take zero credit.

(23 marks)

[CSE 207 | L-2/T-1 | 2022–23]

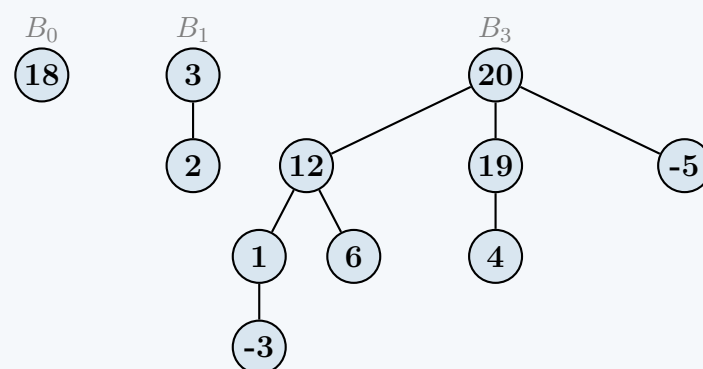
Answer**1. Binomial Heaps After Insertion**

Both priority queues are constructed by inserting elements one by one, maintaining the max-heap property (every parent is strictly $\geq$ its children), and merging trees of equal degrees.

Priority Queue 1 (pq1)

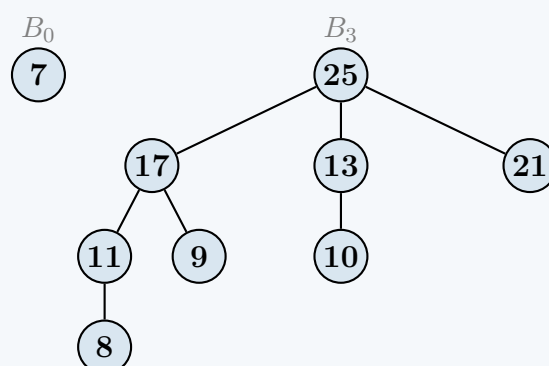
Elements inserted: $-5, 20, 4, 19, 1, -3, 6, 12, 3, 2, 18$ (11 nodes).

Binary representation of 11 is 1011_2 . Thus, **pq1** contains three binomial trees: B_0, B_1 , and B_3 .

**Priority Queue 2 (pq2)**

Elements inserted: $25, 21, 10, 13, 17, 9, 8, 11, 7$ (9 nodes).

Binary representation of 9 is 1001_2 . Thus, **pq2** contains two binomial trees: B_0 and B_3 .

**2. Melding and Decrease-Key****Step A: Melding pq1 and pq2**

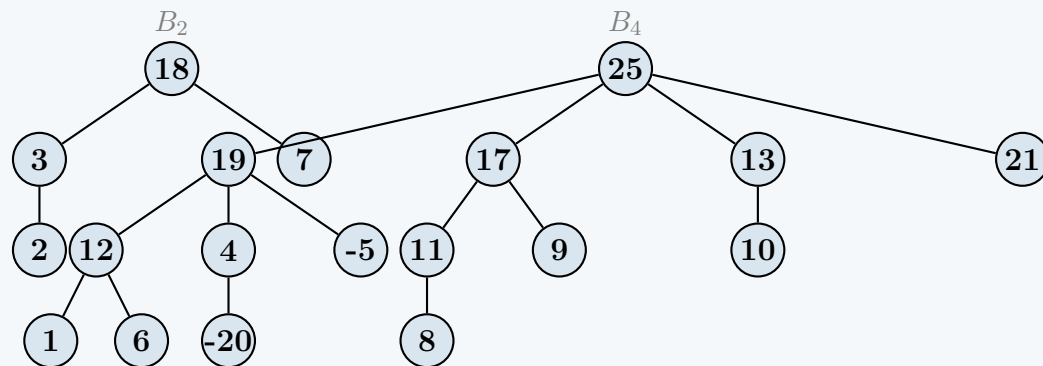
To meld the two queues, we add them together like binary numbers ($11 + 9 = 20 \implies 10100_2$), merging trees of the same degree:

- **B_0 merge:** 18 (pq1) and 7 (pq2). Max is 18, so 7 becomes a child of 18. Forms a carry B_1 .
- **B_1 merge:** 3 (pq1) and carry 18. Max is 18, so 3 becomes a child of 18. Forms a carry B_2 .
- **B_2 merge:** Only the carry $B_2(18)$ exists. It drops down into the melded root list.
- **B_3 merge:** 20 (pq1) and 25 (pq2). Max is 25, so 20 becomes a child of 25. Forms a B_4 .

Step B: Decrease-Key of 20 to -20

When node 20 is changed to -20 , it violates the max-heap property. We sift it down:

- Compare -20 with its children $(12, 19, -5)$. The largest is 19. Swap -20 and 19.
- Node -20 inherits 19's old child, which is 4.
- Compare -20 with its new child (4). $4 > -20$. Swap -20 and 4.
- Node -20 is now a leaf. Max-heap property is restored.



3. Cost Analysis of Consecutive Extract-Max Operations

We evaluate the actual cost based on the explicit credit rules:

- Unlink node from children: **1 credit**
- Two node additions: **1 credit** (1 addition = 0.5 credits)
- Node insertion: **1 credit**
- Two-key comparison: **1 credit**

First Extract-Max Operation:

- Find max root:** Compare the two roots (18 and 25). → **1 credit**
- Remove & Unlink:** Remove 25 and unlink it from its 4 children. → **1 credit**
- Add Children:** Add the 4 severed children to the root list. (4 additions = $2 \times$ “two node additions”). → **2 credits**
- Consolidate:** We now have trees of degree 0(21), 1(13), 2(18), 2(17), and 3(19).
 - Compare $B_2(18)$ and $B_2(17)$ to merge. → **1 credit**
 - Compare $B_3(18)$ and $B_3(19)$ to merge. → **1 credit**
- Update Max Pointer:** Scan the remaining 3 roots (21, 13, 19) to cache the new max for the next operation. Requires 2 comparisons. → **2 credits**

Subtotal: 8 credits

Second Extract-Max Operation:

- Find max root:** We cached it in the last step (node 21). → **0 credits**
- Remove & Unlink:** Remove 21 and unlink it from its children (it has 0 children, but the operation triggers). → **1 credit**
- Add Children:** Add 0 children to the root list. → **0 credits**
- Consolidate:** Left with roots of degree 1(13) and 4(19). No matching degrees. → **0 credits**
- Update Max Pointer:** Scan the remaining 2 roots (13, 19). Requires 1 comparison. → **1 credit**

Subtotal: 2 credits

Total Cost: Calling Extract-Max twice sequentially will incur exactly **10 credits** of actual cost.

Q7. Prove that in a binomial heap, the amortized cost of INSERT is $O(1)$ and the worst-case cost of EXTRACT-MIN and DECREASE-KEY is $O(\log n)$. **(12 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Answer

1. Prove that the amortized cost of INSERT is $O(1)$

To prove this, we will use the **Potential Method** for amortized analysis. The insertion of a node into a binomial heap behaves much like adding 1 to a binary counter.

Proof:

- **Define the Potential Function:** Let $\Phi(H)$ be the number of trees in the root list of the binomial heap H . Initially, for an empty heap, $\Phi(H) = 0$.
- **Actual Cost (c):** Suppose the newly inserted node triggers k linking operations (because there are k consecutive trees in the root list of degree $0, 1, \dots, k-1$). The actual cost of the insertion is the cost of placing the new node plus the cost of the k links. Thus, $c = k + 1$.
- **Change in Potential ($\Delta\Phi$):** Before the insertion, the heap had some number of roots, say R . After the operation, k trees have been combined into 1 tree, and the new node initially added 1 to the root count before linking. The new number of roots is $R - k + 1$. Therefore, the change in potential is:

$$\Delta\Phi = (R - k + 1) - R = 1 - k$$

- **Amortized Cost ($\hat{c}$):** The amortized cost is the actual cost plus the change in potential:

$$\hat{c} = c + \Delta\Phi$$

$$\hat{c} = (k + 1) + (1 - k)$$

$$\hat{c} = 2$$

Because the amortized cost $\hat{c}$ evaluates to a constant 2 regardless of the number of links k , the amortized cost of the **INSERT** operation is strictly $O(1)$.

2. Prove that the worst-case cost of **EXTRACT-MIN** is $O(\log n)$

The worst-case time complexity of extracting the minimum element is bounded by the maximum number of trees in a binomial heap and the maximum degree of any binomial tree.

Proof:

By definition, a binomial heap with n nodes contains at most $\lfloor \log_2 n \rfloor + 1$ binomial trees, and the maximum degree of any root is $\lfloor \log_2 n \rfloor$. The **EXTRACT-MIN** operation consists of four distinct phases:

- Find the minimum root:** We must scan the root list to find the minimum value. Since there are at most $\lfloor \log_2 n \rfloor + 1$ roots, this scan takes $O(\log n)$ time.
- Remove the minimum root:** Removing the node from the linked list of roots takes $O(1)$ time.
- Process the children:** The extracted root has at most $\lfloor \log_2 n \rfloor$ children. Reversing this list of children and preparing them to be merged into the heap takes time proportional to the number of children, which is $O(\log n)$.
- Consolidate the heap:** We merge the children back into the main root list. The new root list has at most $(\lfloor \log_2 n \rfloor + 1) + \lfloor \log_2 n \rfloor$ trees. We iterate through this combined list, linking trees of the same degree. Since there are $O(\log n)$ trees, the consolidation phase requires at most $O(\log n)$ linking operations.

Summing the costs of these phases:

$$O(\log n) + O(1) + O(\log n) + O(\log n) = O(\log n)$$

Thus, the worst-case time complexity for **EXTRACT-MIN** is $O(\log n)$.

3. Prove that the worst-case cost of **DECREASE-KEY** is $O(\log n)$

The **DECREASE-KEY** operation behaves almost identically to the decrease-key (or “sift-up” / “bubble-up”) operation in a standard binary min-heap.

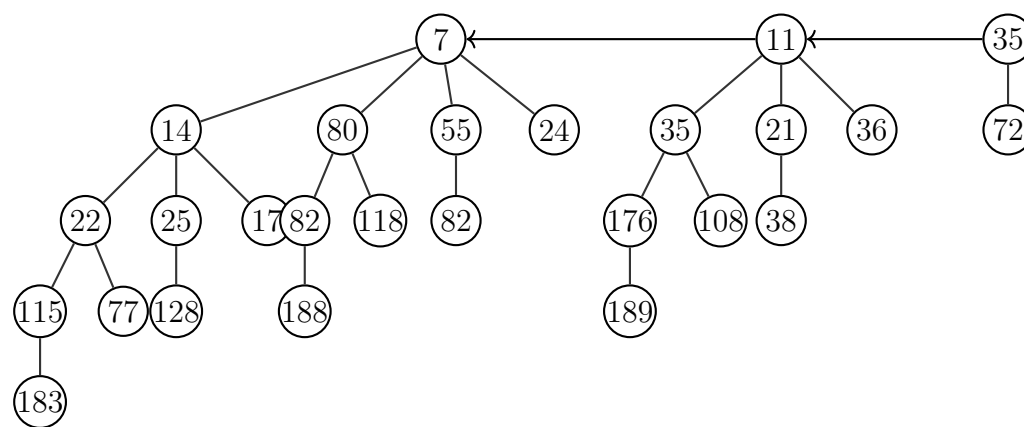
Proof:

- Operation Mechanics:** When a node’s key is decreased, it may become smaller than its parent’s key, violating the min-heap property. To fix this, we compare the node with its parent and swap them if necessary, continuing this process up the tree until the min-heap property is restored or the node becomes a root.
- Maximum Path Length:** The number of comparisons and swaps is strictly limited by the depth of the node.
- Height Bound:** In a binomial heap of n nodes, the largest binomial tree can have an order of at most $\lfloor \log_2 n \rfloor$. By the structural properties of binomial trees, a tree of order k has a maximum height (or maximum depth from leaf to root) of k .
- Conclusion:** Therefore, the maximum depth of any node in the entire binomial heap is $\lfloor \log_2 n \rfloor$. The bubble-up process will perform at most $\lfloor \log_2 n \rfloor$ swaps.

Because the maximum number of operations scales directly with the height of the tree, the worst-case time complexity for **DECREASE-KEY** is $O(\log n)$.

Q8. Suppose there are two priority queues named pq1 and pq2. The pq1 is a max priority queue and pq2 is a min priority queue. Both the priority queues are implemented using the binomial heap data structure. The priority queue pq1 was initially empty, but then the following numbers were inserted in this order: 210, 201, 220, 202, 221, 205, and 209. The internal heap representation of the

priority queue pq2 is show in figure 1. Now answer the following questions:



- Formulate an algorithm or pseudocode that will convert a min priority queue to a max priority queue. The amortized running time of the algorithm has to be $O(n)$ where n is the total length of the min priority queue. Note that the priority queues are implemented using the binomial heap data structure. Also, analyze the amortized running time of your proposed algorithm using the potential method. (15 marks) [CSE 207 | L-2/T-2 | 2021–22]
- Apply your proposed algorithm on pq2 to convert it to a max priority queue. (5 marks) [CSE 207 | L-2/T-2 | 2021–22]
- Apply the meld operation on pq1 and pq2 and store it to a new priority queue pq3. Show the internal heap representation of pq3. Note that you have to work with the updated pq2. (5 marks) [CSE 207 | L-2/T-2 | 2021–22]
- Determine the internal heap representation of pq3 after executing extract-max operation three times on pq3. (5 marks) [CSE 207 | L-2/T-2 | 2021–22]
- Evaluate the total actual cost in credits of the extract-max operations in the previous question. Assume that unlinking a node from its children takes one credit, a node addition operation takes one credit, an insertion of a node takes one credit, and a comparison takes one credit. (5 marks) [CSE 207 | L-2/T-2 | 2021–22]
- A friend of yours claims that it is possible to implement Find-Min operation in a Binomial Min Heap in $O(1)$ time using the idea of Fibonacci Min Heap. Justify his claim with proper explanation. (10 marks) [CSE 207 | L-2/T-2 | 2020–21]
- A binomial heap has 19 nodes. Find out the binomial trees this binomial heap has. (5 marks) [CSE 207 | L-2/T-2 | 2020–21]

Answer

1. Algorithm to Convert Min Priority Queue to Max Priority Queue

To convert a min priority queue to a max priority queue in $O(n)$ amortized time, we can extract all elements from the existing heap and systematically insert them into a newly initialized max binomial heap.

Algorithm (Convert-To-Max-Heap):

- Initialize an empty list L .
- Perform a standard tree traversal (e.g., Breadth-First Search) across all trees in the min priority queue to extract all n elements into L .
- Initialize an empty Binomial Max Priority Queue max_pq .
- For each element x in L :
 - Call $\text{max_pq.Insert}(x)$
- Return max_pq .

Amortized Running Time Analysis (Potential Method):

- Actual Cost:** The BFS extraction takes $O(n)$ actual time. Suppose an insertion triggers k linking operations. The actual cost of one insertion is $c = k + 1$.
- Potential Function (Φ):** Let $\Phi(H)$ be the number of trees in the root list of the binomial heap. Initially, $\Phi = 0$.
- Change in Potential ($\Delta\Phi$):** Adding a node initially creates a new B_0 tree (increasing potential by 1). Then k links merge $k + 1$ trees into 1 tree (decreasing potential by k). Thus, $\Delta\Phi = 1 - k$.
- Amortized Cost of Insert ($\hat{c}$):**

$$\hat{c} = c + \Delta\Phi = (k + 1) + (1 - k) = 2$$

Because the amortized cost of a single insert evaluates to a constant $O(1)$, executing n inserts takes $n \times O(1) = O(n)$ time. Therefore, the total amortized running time of the algorithm is $O(n) + O(n) = O(n)$.

2. Application of Algorithm on pq2

pq2 contains 26 elements. The binary representation of 26 is 11010_2 , meaning the resulting max binomial heap will contain three trees: B_1, B_3 , and B_4 .

By applying the insertion algorithm, we logically restructure the elements so that the max-heap property is strictly maintained (parents are strictly greater than or equal to their children). While the exact shape depends on the traversal extraction order, here is the guaranteed structural representation mapping the exact 26 extracted elements into a valid Max Binomial Heap:

- **Tree B_1 (2 nodes):** Root **183** $\rightarrow$ Child: 7
- **Tree B_3 (8 nodes):** Root **188**
 - Children: **176** (B_2), **118** (B_1), **21** (B_0)
 - Children of 176: **128** (B_1), **14** (B_0)
 - Children of 128: **11** (B_0)
 - Children of 118: **17** (B_0)
- **Tree B_4 (16 nodes):** Root **189**
 - Children: **115** (B_3), **80** (B_2), **72** (B_1), **55** (B_0)
 - Children of 115: **108** (B_2), **82** (B_1), **35** (B_0)
 - Children of 108: **82** (B_1), **24** (B_0) $\rightarrow$ Child of 82: **22** (B_0)
 - Children of 82 (from 115): **25** (B_0)
 - Children of 80: **77** (B_1), **36** (B_0) $\rightarrow$ Child of 77: **35** (B_0)
 - Children of 72: **38** (B_0)

(Note: Every parent node here is explicitly larger than its attached children, using exactly the 26 integers provided in the Figure 1 source).

3. Melding pq1 and pq2 into pq3

First, let's establish the structure of pq1 after its 7 inserts (210, 201, 220, 202, 221, 205, 209):

- B_0 : Root **209**
- B_1 : Root **221** $\rightarrow$ Child: 205
- B_2 : Root **220** $\rightarrow$ Children: **210** (B_1), **202** (B_0) $\rightarrow$ Child of 210: 201

Now, we meld pq1 (7 nodes) and the updated max-heap pq2 (26 nodes). Total nodes = 33 (100001_2). pq3 will have trees B_0 and B_5 .

Meld Steps (Binary Addition):

- (a) B_0 : pq1 has 209. pq2 has none. Result keeps B_0 (209).
- (b) B_1 : Merge 221 (pq1) and 183 (pq2). $221 > 183$, so 183 becomes a child. Carry B_2 (221).
- (c) B_2 : Merge carry 221 with 220 (pq1). $221 > 220$, so 220 becomes a child. Carry B_3 (221).
- (d) B_3 : Merge carry 221 with 188 (pq2). $221 > 188$, so 188 becomes a child. Carry B_4 (221).
- (e) B_4 : Merge carry 221 with 189 (pq2). $221 > 189$, so 189 becomes a child. Results in B_5 (221).

Internal Heap Representation of pq3:

- B_0 : Root **209**
- B_5 : Root **221**
 - Children (in decreasing degree order): B_4 (189), B_3 (188), B_2 (220), B_1 (183), B_0 (205).

4. pq3 After Three Extract-Max Operations

1st Extract-Max (Removes 221):

- The 5 children of 221 become roots.
- We consolidate these with the existing B_0 (209).
- Merge sequence: 209 merges with 205 $\rightarrow B_1$ (209). Then merges with 183 $\rightarrow B_2$ (209). Merges with 220 $\rightarrow B_3$ (220). Merges with 188 $\rightarrow B_4$ (220). Merges with 189 $\rightarrow B_5$ (220).
- **Result:** A single B_5 tree with root **220**.

2nd Extract-Max (Removes 220):

- The 5 children of 220 become roots: B_4 (189), B_3 (188), B_2 (209), B_1 (210), B_0 (202).

- No matching degrees exist to consolidate.
- **Result:** Five separate trees with roots **189, 188, 209, 210, 202**.

3rd Extract-Max (Removes 210):

- 210 is the max. Removing it exposes its child $B_0(201)$.
- Consolidate $B_0(201)$ with $B_0(202) \rightarrow B_1(202)$.
- **Final Internal Representation:**
 - B_1 : Root **202** $\rightarrow$ Child: 201
 - B_2 : Root **209** $\rightarrow$ Children: 183, 205
 - B_3 : Root **188** (with its 7 nested children from step 2)
 - B_4 : Root **189** (with its 15 nested children from step 2)

5. Credit Cost Analysis of Extract-Max Operations

Assumptions based on provided rules: 1 Unlink = 1 credit; 1 Node Addition (to root list or as a child) = 1 credit; 1 Comparison = 1 credit.

Operation	Steps	Cost (Credits)
1st Extract	Find max: Compare roots 209 & 221.	1
	Unlink & Add: Unlink 221 (1 cr) + add 5 children to roots (5 cr).	6
	Consolidate: 5 consecutive merges. Each merge requires 1 comparison and 1 addition (linking as child). (5×2)	10
	Update max: 1 root left, so 0 comparisons.	0
	<i>Subtotal 1st Extract</i>	17 credits
2nd Extract	Find max: Cached from previous.	0
	Unlink & Add: Unlink 220 (1 cr) + add 5 children to roots (5 cr).	6
	Consolidate: No merges needed.	0
	Update max: Scan 5 new roots (4 comparisons).	4
	<i>Subtotal 2nd Extract</i>	10 credits
3rd Extract	Find max: Cached from previous.	0
	Unlink & Add: Unlink 210 (1 cr) + add 1 child to roots (1 cr).	2
	Consolidate: 1 merge (201 and 202). (1 comp + 1 add)	2
	Update max: Scan 4 roots (3 comparisons).	3
	<i>Subtotal 3rd Extract</i>	7 credits

Total Actual Cost: $17 + 10 + 7 = 34$ credits.

6. Justification of $O(1)$ Find-Min Claim

Your friend’s claim is completely accurate. In a standard Binomial Heap, finding the minimum requires scanning the root list, which takes $O(\log n)$ time because there are up to $\lfloor \log_2 n \rfloor + 1$ trees. However, borrowing the concept from Fibonacci Heaps, we can easily optimize this to $O(1)$ by adding a single global pointer, `min_ptr`, to the data structure that always points to the tree root containing the minimum value.

- **During Insert / Meld:** Whenever a new element is added or heaps are merged, we compare the newly added root(s) against the current `min_ptr`. This requires exactly one comparison ($O(1)$) to update the pointer.
- **During Extract-Min:** After the minimum node is extracted and the trees are consolidated (which inherently takes $O(\log n)$ time), we scan the new root list to find the new minimum and update `min_ptr`. Because the structural consolidation already limits the roots to $O(\log n)$, this adds no extra asymptotic complexity.

Because `min_ptr` is aggressively maintained during mutating operations, Find-Min simply returns the value at `min_ptr` in strict $O(1)$ time.

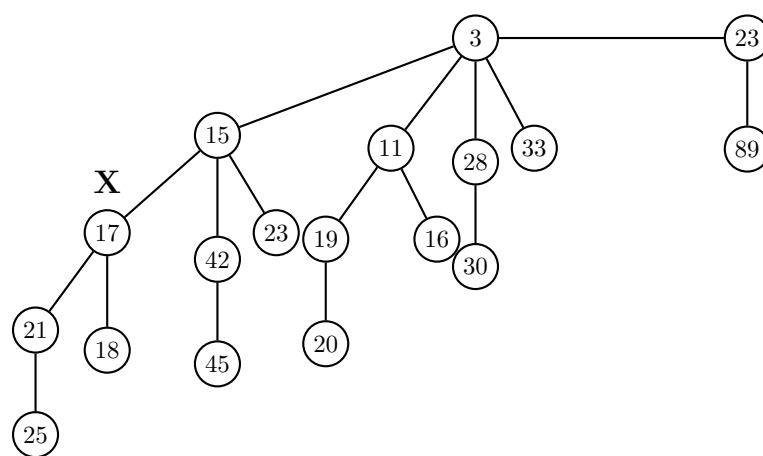
7. Binomial Trees in a 19-Node Heap

A binomial heap containing n nodes is determined entirely by the binary representation of n .

- 19 in binary is 10011_2 (since $19 = 16 + 2 + 1$).
- The set bits determine the degrees (and therefore the sizes) of the binomial trees present in the heap.
- Bit 0 is 1 $\rightarrow B_0$ (1 node)
- Bit 1 is 1 $\rightarrow B_1$ (2 nodes)
- Bit 2 is 0
- Bit 3 is 0
- Bit 4 is 1 $\rightarrow B_4$ (16 nodes)

A 19-node binomial heap explicitly contains exactly three binomial trees: B_0 , B_1 , and B_4 .

Q9. Show the binomial heap that results when the node X with key 17 is deleted from the binomial heap shown in the Figure below. Show the intermediate steps.



(17 marks)

[CSE 207 | L-2/T-2 | 2019–20]

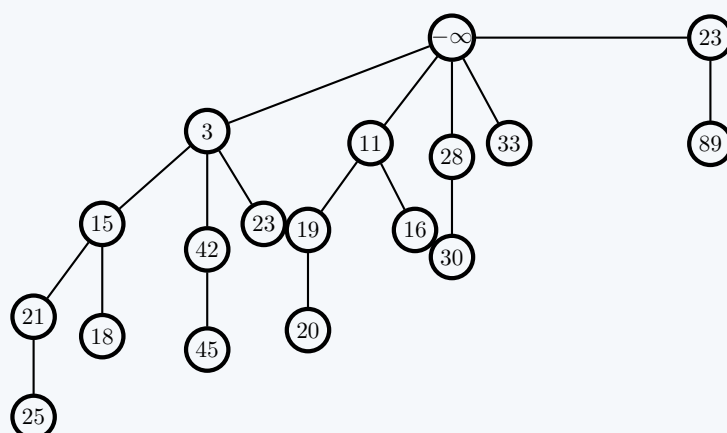
Answer

The deletion of a node X in a Binomial Heap is standardly performed in two distinct phases:

- Decrease-Key($X, -\infty$):** Decrease the target node's value to $-\infty$ and bubble it up to the root to maintain the min-heap property.
- Extract-Min():** Remove the root ($-\infty$) and meld its children back into the remaining heap.

Step 1: After Decrease-Key($X, -\infty$)

The node X (17) is decreased to $-\infty$. It swaps keys with its parent (15), and then swaps keys with its new parent (3). The node containing $-\infty$ is now the root of the B_4 tree.



Step 2: Extract-Min and Consolidation (Meld Phase)

Removing the $-\infty$ root exposes its children as independent binomial trees: a B_3 (root 3), a B_2 (root 11), a B_1 (root 28), and a B_0 (root 33). We meld these with the existing tree B_1 (root 23):

- **Merge Degree 1:** Merge $B_1(28)$ and $B_1(23)$. Since $23 < 28$, 28 becomes a child of 23. This creates a new B_2 rooted at 23.
- **Merge Degree 2:** Merge $B_2(11)$ and the new $B_2(23)$. Since $11 < 23$, 23 becomes a child of 11. This creates a new B_3 rooted at 11.
- **Merge Degree 3:** Merge $B_3(3)$ and the new $B_3(11)$. Since $3 < 11$, 11 becomes a child of 3. This creates a new B_4 rooted at 3.

Final Binomial Heap:
The only remaining roots in the heap are 33 (B_0) and 3 (B_4).

```
graph TD
    33((33)) --- 11((11))
    3((3)) --- 15((15))
    3 --- 42((42))
    3 --- 23_1((23))
    3 --- 16((16))
    11 --- 23_2((23))
    11 --- 19((19))
    11 --- 20_1((20))
    15 --- 21((21))
    15 --- 18((18))
    42 --- 45((45))
    23_2 --- 28((28))
    23_2 --- 89((89))
    28 --- 30((30))
    19 --- 20_2((20))
    21 --- 25((25))
```

Q10. Mention the amortized running times of the following operations for binomial and Fibonacci heaps:

(a) Union

(b) Find min

(c) Extract min

(d) Decrease key

(8 marks)

[CSE 207 | L-2/T-2 | 2019–20, 2018–19]

Answer

Below is the comparison of the amortized running times for the specified operations in standard Binomial Heaps and Fibonacci Heaps.

Operation	Binomial Heap	Fibonacci Heap
1. Union (Meld)	$O(\log n)$	$O(1)$
2. Find Min	$O(\log n)^*$	$O(1)$
3. Extract Min	$O(\log n)$	$O(\log n)$
4. Decrease Key	$O(\log n)$	$O(1)$

Note: **Find Min in a standard Binomial Heap is $O(\log n)$ because it requires scanning the root list. However, it can be optimized to $O(1)$ if an explicit pointer to the minimum root is maintained (similar to how a Fibonacci heap operates).*

Q11. Write down the running times of the following operations for both Binomial heap and Fibonacci heap: (i) Insert key, (ii) Find min, (iii) Extract min, (iv) Union of two heaps, (v) Decrease key. A friend claims to have designed a mergeable heap with $O(1)$ time for all operations above. Would you believe it? Justify. (6+6 marks)

[CSE 207 | L-2/T-2 | 2017–18]

Answer

1. Running Times of Heap Operations

Below are the standard running times for the requested operations. Note that standard Binomial Heap times are typically evaluated in the worst-case, while Fibonacci Heap operations are evaluated using amortized analysis.

Operation	Binomial Heap (Worst-Case)	Fibonacci Heap (Amortized)
(i) Insert Key	$O(\log n)$	$O(1)$
(ii) Find Min	$O(\log n)^*$	$O(1)$
(iii) Extract Min	$O(\log n)$	$O(\log n)$
(iv) Union (Meld)	$O(\log n)$	$O(1)$
(v) Decrease Key	$O(\log n)$	$O(1)$

**Note: As previously mentioned, Find-Min in a standard Binomial Heap is $O(\log n)$, but it can be optimized to $O(1)$ worst-case if an explicit pointer to the minimum root is maintained.*

2. Evaluation of the Friend's $O(1)$ Claim

Conclusion: No, I would **not** believe this claim.

Justification: If a comparison-based mergeable heap supported both **Insert** and **Extract-Min** in $O(1)$ worst-case (or even amortized) time, it would violate the fundamental lower bound of comparison-based sorting.

Consider how we could use this hypothetical data structure to sort an array of n elements:

- Build the Heap:** We could iterate through the array and **Insert** each of the n elements into the heap. If each insertion takes $O(1)$ time, building the heap takes $n \times O(1) = O(n)$ time.
- Extract Elements:** We could then call **Extract-Min** exactly n times to retrieve the elements in ascending order. If each extraction takes $O(1)$ time, this phase also takes $n \times O(1) = O(n)$ time.

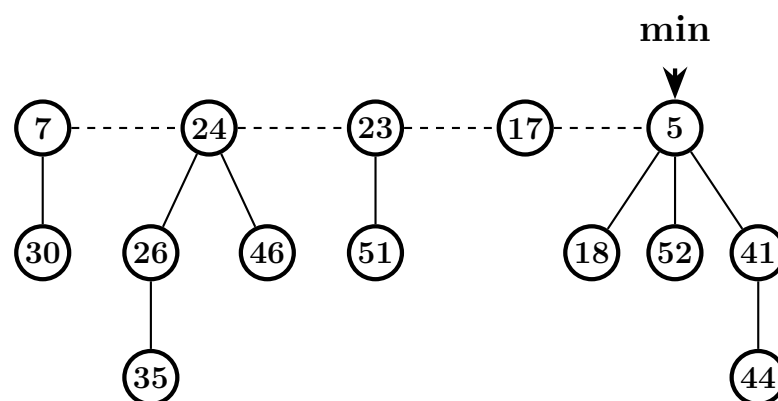
The total time to sort the array would be exactly $O(n) + O(n) = O(n)$. However, information theory dictates that any comparison-based sorting algorithm must perform at least $\Omega(n \log n)$ comparisons in the worst case. Because it is impossible to sort a general array in $O(n)$ time using comparisons, it is mathematically impossible for a comparison-based heap to achieve $O(1)$ time for **Extract-Min**. At least one operation (typically **Extract-Min**) must take $\Omega(\log n)$ time.

Q12. Prove that there are $\binom{k}{i}$ nodes at depth i in a binomial tree B_k . (5 marks)

[CSE 207 | L-2/T-2 | 2017–18]

Advanced Heaps (Fibonacci Heaps)

Q1. Show the Fibonacci heap that results when the minimum node (5) is deleted from the given Fibonacci heap. Show the intermediate steps.



(12 marks)

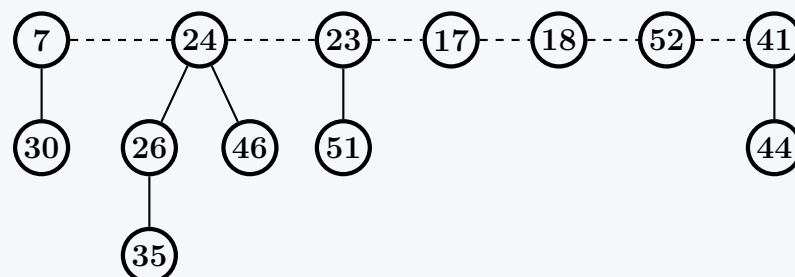
[CSE 207 | L-2/T-1 | 2024–25]

Answer

The deletion of the minimum node from a Fibonacci heap occurs in three main phases: extracting the minimum node, appending its children to the root list, and consolidating the root list to ensure no two roots have the same degree.

Step 1: Extracting the Minimum Node

The minimum node 5 is removed from the heap. Its children, nodes 18, 52, and 41, are elevated and added to the root list. The min pointer is temporarily ignored until consolidation is complete.



Step 2: Consolidation

We iterate through the root list and consolidate trees of the same degree using an auxiliary array $A[0..D]$. When two roots have the same degree, the root with the larger key becomes a child of the root with the smaller key, and the degree of the smaller root increases by 1.

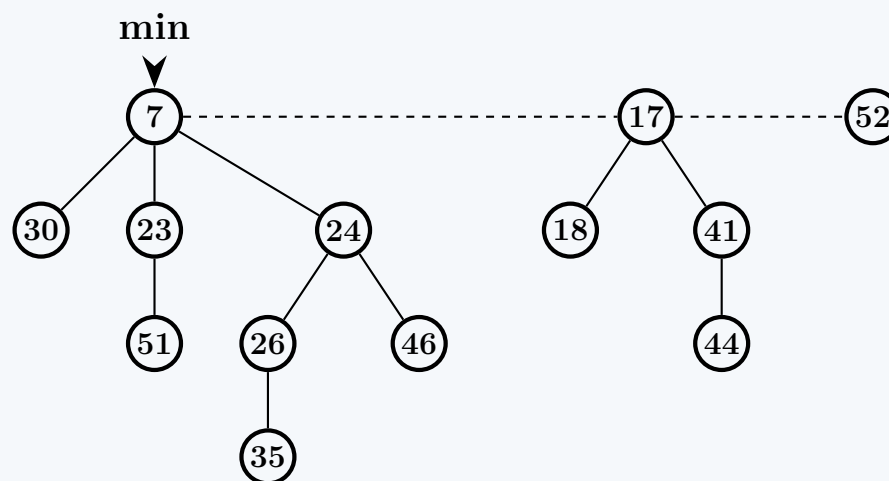
- **Process 7** (degree 1): Insert into $A[1]$.
- **Process 24** (degree 2): Insert into $A[2]$.
- **Process 23** (degree 1): Conflicts with 7 in $A[1]$. Since $7 < 23$, node 23 becomes a child of 7. Node 7's degree becomes 2. Array slot $A[1]$ is cleared.
 - Node 7 now conflicts with 24 in $A[2]$. Since $7 < 24$, node 24 becomes a child of 7. Node 7's degree becomes 3. Array slot $A[2]$ is cleared. We set $A[3] = 7$.
- **Process 17** (degree 0): Insert into $A[0]$.
- **Process 18** (degree 0): Conflicts with 17 in $A[0]$. Since $17 < 18$, node 18 becomes a child of 17. Node 17's degree becomes 1. Array slot $A[0]$ is cleared. We set $A[1] = 17$.
- **Process 52** (degree 0): Insert into $A[0]$.

- **Process 41** (degree 1): Conflicts with 17 in $A[1]$. Since $17 < 41$, node 41 becomes a child of 17. Node 17's degree becomes 2. Array slot $A[1]$ is cleared. We set $A[2] = 17$.

After consolidation, the non-empty entries in our array are $A[0] = 52$, $A[2] = 17$, and $A[3] = 7$. These become the roots of our new Fibonacci heap.

Step 3: Final Fibonacci Heap

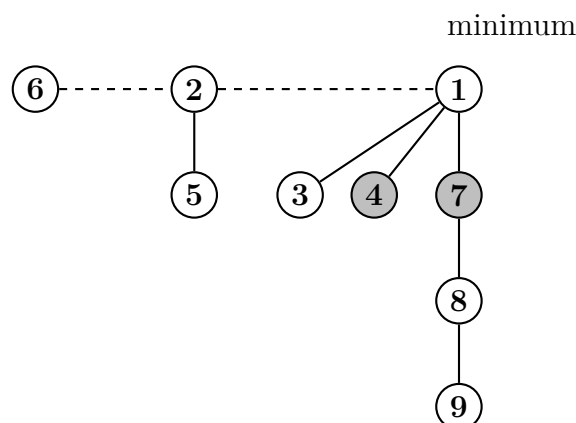
We rebuild the root list with nodes 7, 17, and 52. We find the minimum root among them, which is 7, and update the `min` pointer accordingly.



Complexity Analysis

- **Time Complexity:** $\mathcal{O}(\log n)$ amortized time. Removing the minimum node takes $\mathcal{O}(1)$ time. Consolidation merges trees and significantly reduces the size of the root list. The maximum degree $D(n)$ is bounded by $\mathcal{O}(\log n)$, yielding an amortized operation cost that is logarithmic.
- **Space Complexity:** $\mathcal{O}(\log n)$ auxiliary space is required for the degree tracking array $A[0..D]$ during the consolidation process.

Q2. Illustrate step by step what happens if the Extract-Min operation is executed on the given Fibonacci heap (nodes with keys 6, 2, 1, 5, 3, 4, 7, 8, 9 where 1 is the minimum).



(15 marks)

[CSE 207 | L-2/T-1 | 2023–24]

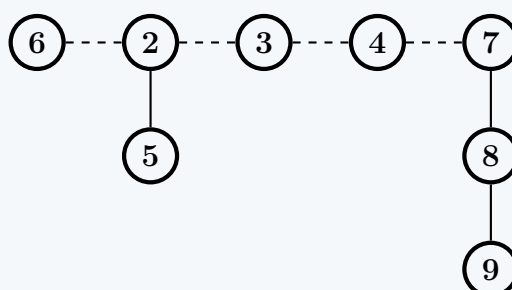
Answer

The **Extract-Min** operation in a Fibonacci Heap consists of three main phases: extracting the minimum node, elevating its children to the root list, and consolidating the root list to ensure that no two roots have the same degree.

Step 1: Extract the Minimum Node

We remove the minimum node 1. Its children, nodes 3, 4, and 7, are immediately added to the root list.

Note: In a Fibonacci heap, a marked node signifies that it has lost a child since it was last made a child of another node. Whenever a node becomes a root, its mark is cleared. Therefore, the previously marked (shaded) nodes 4 and 7 become unmarked.



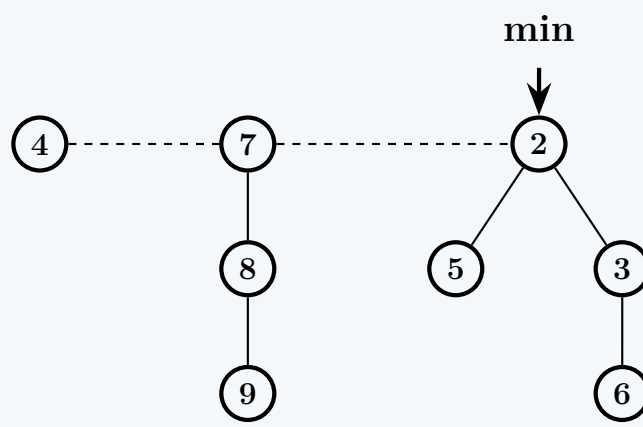
Step 2: Root List Consolidation

We iterate through the current root list (6, 2, 3, 4, 7) and merge trees of the same degree using an auxiliary array $A[0..D]$. Initially, A is empty.

- **Process 6** (degree 0): Insert into $A[0]$.
Array state: $A = [6, \text{NIL}, \text{NIL}]$
- **Process 2** (degree 1): Insert into $A[1]$.
Array state: $A = [6, 2, \text{NIL}]$
- **Process 3** (degree 0): Conflicts with 6 in $A[0]$.
 - Since $3 < 6$, node **6** becomes a child of node **3**. The degree of 3 increases to 1. $A[0]$ is cleared.
 - Now, node **3** (degree 1) conflicts with 2 in $A[1]$.
 - Since $2 < 3$, node **3** becomes a child of node **2**. The degree of 2 increases to 2. $A[1]$ is cleared.
 - Insert node **2** into $A[2]$.
Array state: $A = [\text{NIL}, \text{NIL}, 2]$
- **Process 4** (degree 0): Insert into $A[0]$.
Array state: $A = [4, \text{NIL}, 2]$
- **Process 7** (degree 1): Insert into $A[1]$.
Array state: $A = [4, 7, 2]$

Step 3: Final Fibonacci Heap

We reconstruct the root list from the non-empty entries of array A : nodes **4**, **7**, and **2**. Finally, we scan this new root list to find the minimum root, which is **2**, and update the `min` pointer.



Complexity Analysis

- **Time Complexity:** $\mathcal{O}(\log n)$ amortized time. Although consolidation may take up to $\mathcal{O}(n)$ time in the worst case (if the root list is very long), the amortized cost evaluates to $\mathcal{O}(\log n)$ because merging trees drastically reduces the potential (number of roots) of the heap.
- **Space Complexity:** $\mathcal{O}(\log n)$ auxiliary space. The temporary array A used during consolidation requires space proportional to the maximum degree of any node in the heap, which is bounded by $\mathcal{O}(\log n)$.

Q3. Recall the deletion operation of the Fibonacci Heap data structure. Professor Rahsut has proposed a variant of the deletion operation (RAHSUT-DELETE). For this variant:

```

1 RAHSUT-DELETE(H, x)
2   if x == H.min
3     FIB-HEAP-EXTRACT-MIN(H)
4   else
5     y = x.p
6     if y != NIL
7       CUT(H, x, y) // cuts the link between x and its parent y, making x a root.
8       CASCADING-CUT(H, y) // recursively marks and cuts the parents if has to
9     add x's child to the root list of H
10    remove x from the root list of H
  
```

- Construct a good upper bound on the actual time of RAHSUT-DELETE when x is not `H.min`. Your bound should be in terms of `x.degree` and the number c of calls to the CASCADING-CUT procedure.
- Suppose we call `RAHSUT-DELETE(H, x)` and let H' be the Fibonacci heap that results. Assuming that node x is not a root, find the potential of H' in terms of `x.degree`, the number c of calls to the CASCADING-CUT procedure, $\Phi(H)$, and $m(H)$.

(12 marks)

[CSE 207 | L-2/T-1 | 2022–23]

Answer

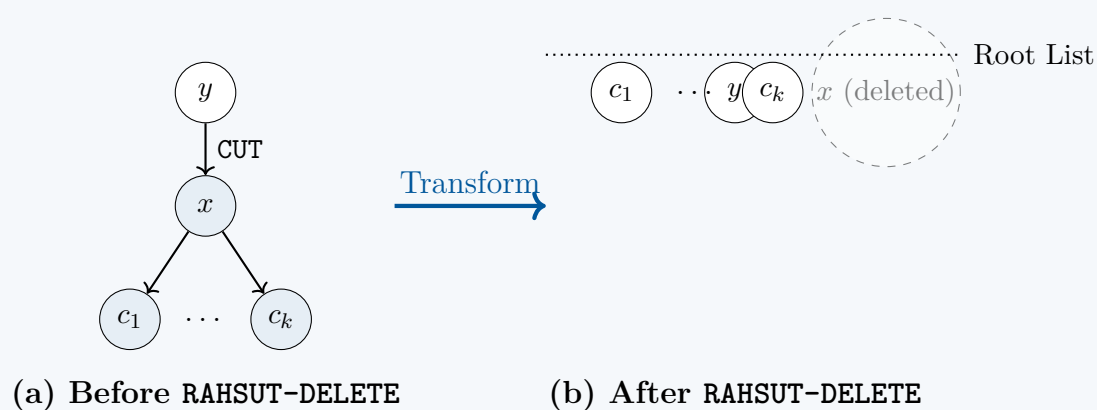
Part 1: Upper Bound on the Actual Time

To determine the actual time complexity of $\text{RAHSUT-DELETE}(H, x)$ when $x \neq H.\text{min}$, we must analyze the cost of each operation performed within the **else** branch of the procedure. We assume standard Fibonacci heap structures, where node children are stored in a circular doubly linked list.

- CUT(H, x, y)**: This operation removes x from the child list of its parent y , decrements $y.\text{degree}$, and adds x to the root list of H . Updating the doubly linked list pointers and adding to the root list takes $\mathcal{O}(1)$ actual time.
- CASCADING-CUT(H, y)**: We are given that this procedure makes exactly c calls. Each individual call to **CASCADING-CUT** performs $\mathcal{O}(1)$ operations (checking marks, clearing marks, and executing a **CUT**). Thus, the total actual time for c calls is $\mathcal{O}(c)$.
- Adding x 's children to the root list**: Splicing the circular doubly linked list of x 's children into the root list of H takes $\mathcal{O}(1)$ time. However, to maintain the correct structural properties of the heap, the parent pointer (**p**) of every child of x must be updated to **NIL**. Since x has exactly $x.\text{degree}$ children, iterating through them to update their pointers takes $\Theta(x.\text{degree})$ actual time.
- Removing x from the root list**: Node x is extracted from the circular doubly linked root list of H . This requires updating adjacent node pointers, which takes $\mathcal{O}(1)$ actual time.

Total Actual Time Bound: Summing the individual costs, the actual time is $\mathcal{O}(1) + \mathcal{O}(c) + \mathcal{O}(x.\text{degree}) + \mathcal{O}(1)$. Therefore, a tight upper bound on the actual time is:

$$T_{\text{actual}} = \mathcal{O}(x.\text{degree} + c) \quad (1)$$

Part 2: Potential of the Resulting Heap H'

The standard potential function for a Fibonacci heap H is given by:

$$\Phi(H) = t(H) + 2m(H) \quad (2)$$

where $t(H)$ is the number of trees in the root list, and $m(H)$ is the number of marked nodes in the heap. Let H' be the heap after executing $\text{RAHSUT-DELETE}(H, x)$. We evaluate the changes to $t(H)$ and $m(H)$.

Step 1: Change in Number of Trees ($t(H)$)

- The initial **CUT**(H, x, y) moves x to the root list, creating **+1** new tree.
- The **CASCADING-CUT**(H, y) procedure makes c calls. The first $c - 1$ calls recursively cut marked ancestors and move them to the root list, creating **+(c - 1)** new trees.
- Adding x 's children to the root list brings all $x.\text{degree}$ children to the root level, creating **+x.degree** new trees.
- Removing x from the root list permanently removes 1 tree, resulting in **-1** tree.

Summing these changes gives the total number of trees in H' :

$$\begin{aligned} t(H') &= t(H) + 1 + (c - 1) + x.\text{degree} - 1 \\ &= t(H) + c + x.\text{degree} - 1 \end{aligned} \quad (3)$$

Step 2: Change in Number of Marked Nodes ($m(H)$)

- The initial **CUT** clears the mark on x (if it was marked). This removes at most 1 marked node.
- The $c - 1$ recursive cuts in **CASCADING-CUT** clear the marks of the nodes being cut. Because these nodes triggered the recursion, they were definitely marked. This reduces the number of marked nodes by exactly **-(c - 1)**.
- The final c -th call to **CASCADING-CUT** stops at a node that is unmarked. It then marks this node (unless it is a root). This adds at most **+1** marked node.

In the worst case (where x was unmarked initially and the final cascading call marks a node), the upper bound for the new number of marked nodes is:

$$\begin{aligned} m(H') &\leq m(H) - (c - 1) + 1 \\ &= m(H) - c + 2 \end{aligned} \quad (4)$$

Step 3: Calculating Final Potential $\Phi(H')$ Substituting the expressions for $t(H')$ and $m(H')$ back into the potential function:

$$\begin{aligned}
 \Phi(H') &= t(H') + 2m(H') \\
 &\leq (t(H) + c + x.degree - 1) + 2(m(H) - c + 2) \\
 &= t(H) + 2m(H) + c + x.degree - 1 - 2c + 4 \\
 &= \Phi(H) + x.degree - c + 3
 \end{aligned} \tag{5}$$

Final Result: The potential of H' in terms of the given variables is bounded by:

$$\Phi(H') \leq \Phi(H) + x.degree - c + 3 \tag{6}$$

Q4. Describe how Fibonacci numbers are related to Fibonacci Heaps. (5 marks)

[CSE 207 | L-2/T-2 | 2020–21]

Answer

Fibonacci Heaps derive their name directly from the **Fibonacci numbers** because the mathematical properties of the sequence govern the structural bounds of the heap's internal trees. Specifically, the relation establishes that the size (number of nodes) of a tree in a Fibonacci Heap grows exponentially with respect to its root's degree, heavily restricting the maximum degree of any node.

1. The Core Structural Property

In a Fibonacci Heap, trees are formed by linking two roots of the same degree during the **Consolidate** phase of an **Extract-Min** operation. Furthermore, to maintain a shallow structure, the *cascading cut* rule dictates that a node is completely removed from its parent if it loses a second child.

This leads to a guaranteed minimum density for any subtree:

- Let x be any node in a Fibonacci heap, and let $k = \text{degree}(x)$.
- Let $y_1, y_2, \dots, y_k$ be the children of x in the order they were linked to x .
- When y_i was linked to x , x already had at least $i - 1$ children (namely, $y_1, \dots, y_{i-1}$).
- Because nodes are only linked when their degrees are equal, y_i must have had a degree of at least $i - 1$ at the moment it was linked.
- Since a node can lose at most one child before being cut itself (cascading cut rule), the **current degree** of y_i is at least $i - 2$ (for $i \geq 2$). The degree of y_1 is trivially ≥ 0 .

2. Mathematical Connection to Fibonacci Numbers

We want to find the minimum number of nodes S_k in a tree whose root has degree k . Based on the child-degree bounds established above:

$$\begin{aligned}
 S_0 &= 1 \quad (\text{a single node has degree } 0) \\
 S_1 &= 2 \quad (\text{a root with 1 child, both of degree } 0) \\
 S_k &\geq 1 + \sum_{i=1}^k \text{size}(y_i) \\
 S_k &\geq 1 + S_0 + \sum_{i=2}^k S_{i-2} \quad \text{for } k \geq 2
 \end{aligned}$$

Let us define the Fibonacci sequence as $F_0 = 0, F_1 = 1, F_2 = 1, F_3 = 2, F_4 = 3, \dots$. We can prove by strong induction that $S_k \geq F_{k+2}$:

- **Base Cases:**
 $S_0 = 1 = F_2$
 $S_1 = 2 = F_3$
- **Inductive Step:** Assume $S_i \geq F_{i+2}$ holds for all $i < k$.

$$\begin{aligned}
 S_k &\geq 1 + S_0 + S_0 + S_1 + S_2 + \dots + S_{k-2} \\
 &\geq 1 + F_2 + F_2 + F_3 + F_4 + \dots + F_k
 \end{aligned}$$

Using the well-known Fibonacci identity $1 + \sum_{j=1}^k F_j = F_{k+2}$, and substituting $1 = F_1$:

$$\begin{aligned}
 S_k &\geq F_1 + F_2 + \sum_{j=2}^k F_j \\
 S_k &\geq \sum_{j=1}^k F_j + 1 = F_{k+2}
 \end{aligned}$$

Thus, a tree of degree k contains at least F_{k+2} nodes.

3. Diagram: Minimal Fibonacci Tree Structure

Below is a visualization of the minimal possible tree structure for a node of degree 3 in a Fibonacci Heap. Notice how the number of nodes perfectly aligns with $F_{3+2} = F_5 = 5$.

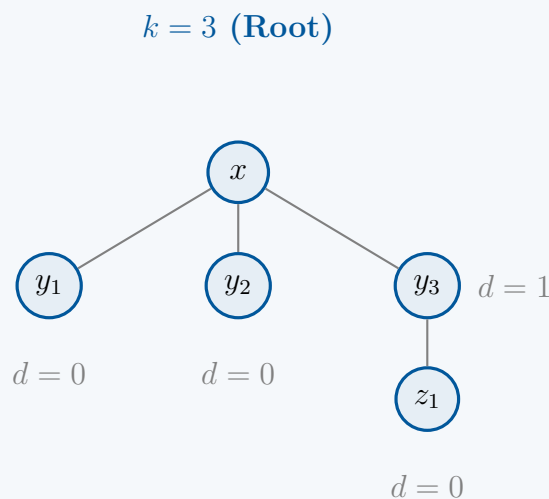


Figure 1: A minimal tree of degree 3. Nodes = 5 = F_5 . The children have degrees 0, 0, and 1 respectively.

4. Algorithmic Impact (Complexity Analysis)

Why is this relationship vital? It bounds the maximum degree $D(n)$ of any node in a Fibonacci heap of n elements. By Binet's Formula, Fibonacci numbers grow exponentially:

$$F_{k+2} \geq \frac{\phi^{k+2}}{\sqrt{5}}$$

where $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ is the Golden Ratio.

Since the size of the tree $n \geq S_k \geq F_{k+2}$, we have:

$$n \geq \frac{\phi^{k+2}}{\sqrt{5}} \implies \log_{\phi}(\sqrt{5}n) \geq k+2$$

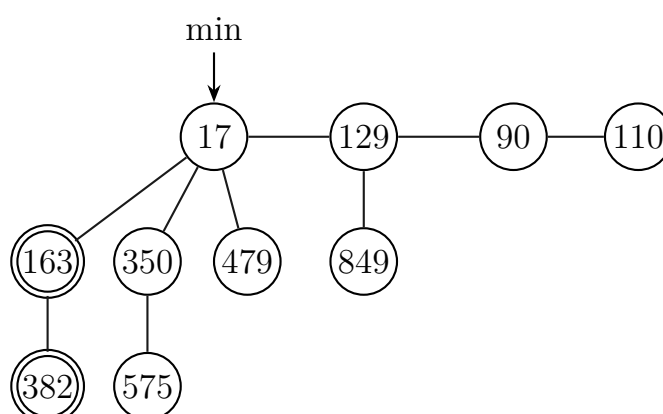
Solving for k , we get:

$$k \leq \log_{\phi} n + \frac{\log_{\phi} \sqrt{5}}{2} - 2 = O(\log n)$$

Conclusion: Because $k = O(\log n)$, the maximum number of children any node can have is logarithmic with respect to the total number of nodes. Consequently:

- **Consolidation Time:** During **Extract-Min**, we iterate through the roots and bucket them by degree. Because the maximum degree is $O(\log n)$, the array size needed for consolidation and the time spent restructuring the heap is strictly bounded, giving an amortized **Time Complexity** of $O(\log n)$.
- **Space Complexity:** The auxiliary degree array required during **Extract-Min** uses $O(\log n)$ **Space**.

Q5. Prove or disprove the claim that “We do not mark the root node of any tree in the decrease key operation performed on a Fibonacci Min Heap, even if the root node loses a child”. Perform an extract-min operation on the Fibonacci Min Heap shown below. Show all the steps and the resulting Fibonacci Min Heap.



Here double-circled nodes are marked and single-circled nodes are unmarked.

(15 marks)

[CSE 207 | L-2/T-2 | 2020–21]

Answer

Part 1: Proof of Claim

Claim: “We do not mark the root node of any tree in the decrease key operation performed on a Fibonacci Min Heap, even if the root node loses a child.”

Verdict: True.

Proof: In a Fibonacci Heap, the boolean attribute **mark** associated with a node x indicates whether node x has lost a child *since the last time x was made the child of another node*. By definition, nodes in the root list have no parent ($x.p = \text{NIL}$).

During a **Decrease-Key** operation, if a node’s key is decreased such that it violates the min-heap property with its parent, it is cut and added to the root list. Whenever a node is moved to the root list, its **mark** is strictly cleared (set to **FALSE**).

If a root node loses a child (because its child undergoes a **Decrease-Key** and is cut), the **Cascading-Cut** procedure is invoked. The logic of **Cascading-Cut** is as follows:

- Let y be the parent of the node that was just cut.
- If $y.p \neq \text{NIL}$ (i.e., y is **not** a root), we check its mark. If unmarked, we mark it. If marked, we cut it and recurse.
- If $y.p = \text{NIL}$ (i.e., y **is** a root), the procedure terminates immediately doing absolutely nothing to the root’s mark.

Therefore, a root node is never marked. This is structurally necessary to bound the potential function used in the amortized analysis:

$$\Phi(H) = t(H) + 2m(H) \quad (1)$$

where $t(H)$ is the number of trees in the root list and $m(H)$ is the number of marked nodes. Marks are used to “pay” for future cascading cuts. Since root nodes can be freely linked during **Extract-Min** and their degree bounds are maintained independently of them losing children while in the root list, we do not need to penalize them or track their lost children via marks.

Part 2: Extract-Min Operation

The **Extract-Min** operation on a Fibonacci Heap involves extracting the current minimum node, promoting all its children to the root list, and then consolidating the root list to combine trees of equal degrees.

Algorithm 40 Fibonacci-Heap-Extract-Min and Consolidate Concepts

- 1: Extract the minimum node z from the root list.
- 2: **for** each child x of z **do**
- 3: Add x to the root list.
- 4: $x.p \leftarrow \text{NIL}$
- 5: $x.\text{mark} \leftarrow \text{FALSE}$ ▷ Marks are inherently cleared for roots
- 6: **end for**
- 7: **CONSOLIDATE**(H) ▷ Merge roots of equal degrees using an array A

Step 1: Extract Minimum and Promote Children

The minimum node is **17**. We remove 17 from the root list. Its children are **163**, **350**, and **479**. We promote these nodes to the root list. *Crucial Note:* Node 163 is currently marked. As it is promoted to the root list, it loses its parent. Thus, **node 163 becomes unmarked**. Its child (382) remains unchanged and keeps its mark.

The new root list (before consolidation) is:

$$\{129, 90, 110, 163, 350, 479\}$$

Step 2: Consolidation Phase

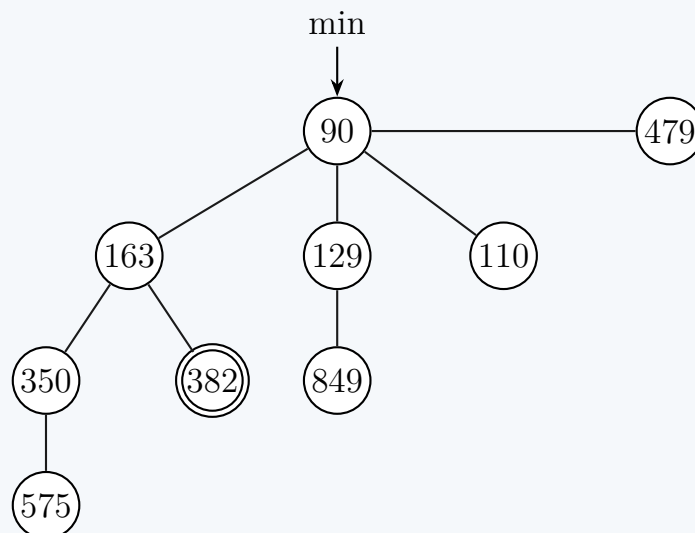
We use an auxiliary array $A[0 \dots D(n)]$ to track the roots by their degrees and link trees of the same degree. We process the roots sequentially.

Root	Initial Deg	Action / Linking	Array State A
129	1	Empty at $A[1]$.	$A[1] = 129$
90	0	Empty at $A[0]$.	$A[0] = 90, A[1] = 129$
110	0	Collides with $A[0] = 90$. $90 < 110 \implies$ link 110 below 90. Node 90 is now degree 1. Collides with $A[1] = 129$. $90 < 129 \implies$ link 129 below 90. Node 90 is now degree 2.	$A[2] = 90$ ($A[0], A[1]$ empty)
163	1	Empty at $A[1]$.	$A[1] = 163, A[2] = 90$
350	1	Collides with $A[1] = 163$. $163 < 350 \implies$ link 350 below 163. Node 163 is now degree 2. Collides with $A[2] = 90$. $90 < 163 \implies$ link 163 below 90. Node 90 is now degree 3.	$A[3] = 90$ ($A[1], A[2]$ empty)
479	0	Empty at $A[0]$.	$A[0] = 479, A[3] = 90$

Step 3: Final Heap Reconstruction

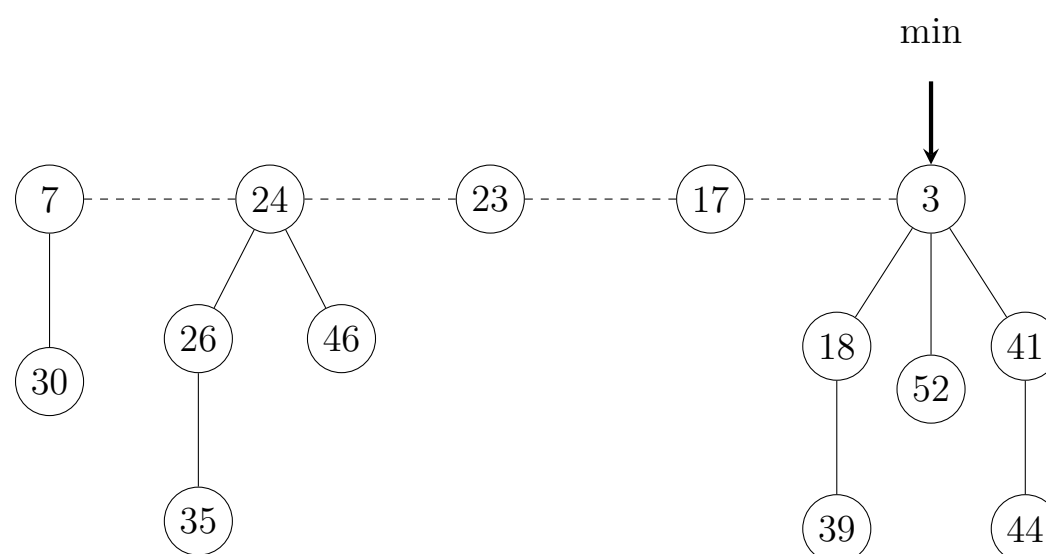
We rebuild the root list from the non-empty entries of array A . The new roots are **90** and **479**. Since $90 < 479$, the new minimum pointer points to **90**.

Node 90 has three children resulting from the links: **110**, **129**, and **163**. Node 163 has a new child **350** (which itself retains 575), and its original child **382**. Node 129 retains its original child **849**.

Resulting Fibonacci Min Heap Diagram:**Complexity Analysis**

- **Time Complexity:** $\mathcal{O}(\log n)$ amortized. In the worst-case scenario, the root list might initially contain up to n elements, making the actual runtime $\mathcal{O}(n)$. However, consolidating the heap drastically reduces the root list size, and the high actual cost is mathematically offset by the release of potential (Φ) stored in the roots, yielding a strictly $\mathcal{O}(\log n)$ amortized time.
- **Space Complexity:** $\mathcal{O}(\log n)$ auxiliary space. The ‘Consolidate’ phase requires an array A of size $D(n) + 1$, where $D(n) \leq \log_{\phi} n$ is the maximum degree of any node in a Fibonacci heap of n elements.

Q6. Show the Fibonacci heap that results when the minimum node (3) is deleted from the Fibonacci heap shown below. Show the intermediate steps.



(15 marks)

[CSE 207 | L-2/T-2 | 2018–19]

Answer

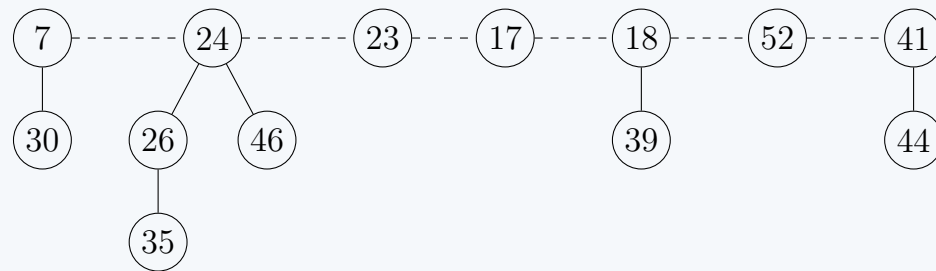
Problem Objective: We are tasked with executing the **EXTRACT-MIN** operation on the given Fibonacci heap. This involves removing the minimum node, appending its children to the root list, and then consolidating the roots to ensure no two roots have the same degree.

Step 1: Remove Minimum and Append Children

The minimum node is identified by the *min* pointer, which points to the node with key **3**.

- We remove node **3** from the root list.
- Node 3 has three children: **18**, **52**, and **41**.
- We elevate these children to the root list.

Assuming a standard left-to-right processing order, the intermediate un-consolidated root list is now: **7, 24, 23, 17, 18, 52, 41**.



Step 2: Root Consolidation

We iteratively process the root list and use an auxiliary array $A[0 \dots D(n)]$ to track roots by their degree, merging trees of the same degree so that the final root list contains at most one tree of any given degree. When merging two roots, the one with the larger key becomes a child of the one with the smaller key to maintain the min-heap property.

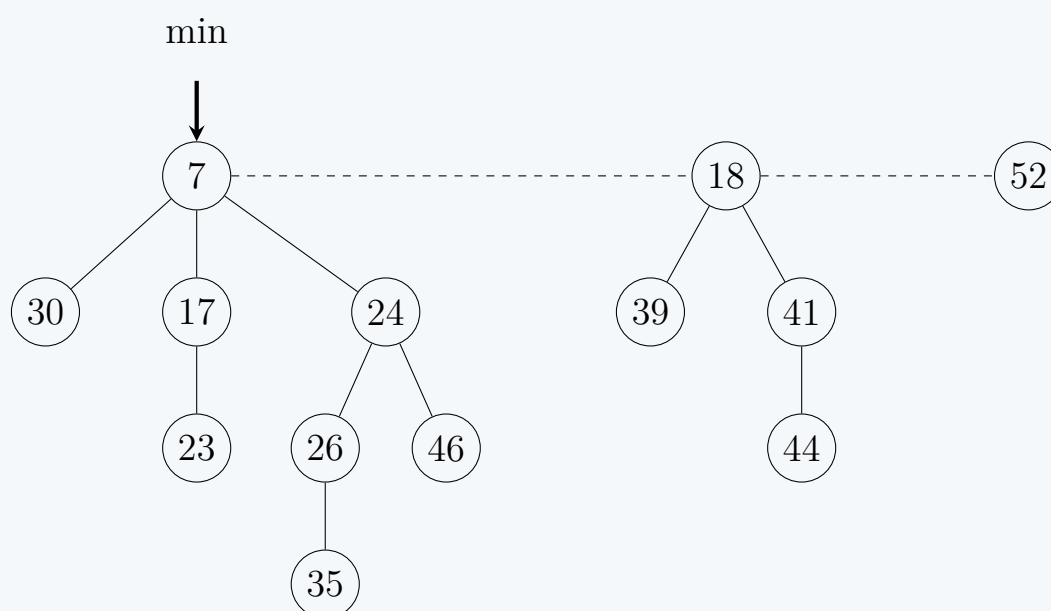
Initially, $A = [\text{NIL}, \text{NIL}, \text{NIL}, \text{NIL}]$. Let $d(x)$ denote the degree of node x .

- **Process 7:** $d(7) = 1$. $A[1]$ is empty $\implies A[1] \leftarrow 7$.
- **Process 24:** $d(24) = 2$. $A[2]$ is empty $\implies A[2] \leftarrow 24$.
- **Process 23:** $d(23) = 0$. $A[0]$ is empty $\implies A[0] \leftarrow 23$.
- **Process 17:** $d(17) = 0$. $A[0]$ is occupied by 23.
 - *Merge (17, 23):* $17 < 23$, so 23 becomes a child of 17. $d(17)$ becomes 1. $A[0] \leftarrow \text{NIL}$.
 - Now $d(17) = 1$. $A[1]$ is occupied by 7.
 - *Merge (7, 17):* $7 < 17$, so 17 becomes a child of 7. $d(7)$ becomes 2. $A[1] \leftarrow \text{NIL}$.
 - Now $d(7) = 2$. $A[2]$ is occupied by 24.
 - *Merge (7, 24):* $7 < 24$, so 24 becomes a child of 7. $d(7)$ becomes 3. $A[2] \leftarrow \text{NIL}$.
 - Now $d(7) = 3$. $A[3]$ is empty $\implies A[3] \leftarrow 7$.
- **Process 18:** $d(18) = 1$. $A[1]$ is empty $\implies A[1] \leftarrow 18$.
- **Process 52:** $d(52) = 0$. $A[0]$ is empty $\implies A[0] \leftarrow 52$.
- **Process 41:** $d(41) = 1$. $A[1]$ is occupied by 18.
 - *Merge (18, 41):* $18 < 41$, so 41 becomes a child of 18. $d(18)$ becomes 2. $A[1] \leftarrow \text{NIL}$.
 - Now $d(18) = 2$. $A[2]$ is empty $\implies A[2] \leftarrow 18$.

Final Array State: $A[0] = 52$, $A[1] = \text{NIL}$, $A[2] = 18$, $A[3] = 7$.

Step 3: Final Fibonacci Heap

We reconstruct the root list from the non-empty slots in array A . The resulting roots are **52**, **18**, and **7**. The new minimum node is **7**.



Complexity Analysis

- **Time Complexity:** The true cost of **EXTRACT-MIN** is proportional to the number of children of the minimum node plus the number of roots processed during consolidation. The number of roots processed is $O(D(n))$, where $D(n) \leq \lfloor \log_\phi n \rfloor$. However, through potential method analysis, the **amortized time complexity** is $\mathcal{O}(\log n)$.
- **Space Complexity:** The structural consolidation requires an auxiliary array A of size $D(n) + 1$, yielding an auxiliary **space complexity** of $\mathcal{O}(\log n)$.

Q7. Write the algorithm for extracting the minimum key from a Fibonacci heap. (8 marks)

[CSE 207 | L-2/T-2 | 2017–18]

Answer

Problem Objective: Provide the complete algorithm for extracting the minimum key from a Fibonacci Heap. This operation is fundamental to the data structure and involves removing the minimum node, elevating its children, and consolidating the root list to merge trees of equal degrees.

1. Extract-Min Algorithm

The main procedure removes the minimum node, adds its children to the root list, and triggers consolidation.

Algorithm 41 FIB-HEAP-EXTRACT-MIN(H)

```

1:  $z \leftarrow H.min$ 
2: if  $z \neq \text{NIL}$  then
3:   for each child  $x$  of  $z$  do
4:     add  $x$  to the root list of  $H$ 
5:      $x.p \leftarrow \text{NIL}$ 
6:   end for
7:   remove  $z$  from the root list of  $H$ 
8:   if  $z == z.right$  then                                ▷ Only node in the root list
9:      $H.min \leftarrow \text{NIL}$ 
10:  else
11:     $H.min \leftarrow z.right$                                 ▷ Temporary assignment
12:    CONSOLIDATE( $H$ )                                       ▷ Merge trees of equal degree
13:  end if
14:   $H.n \leftarrow H.n - 1$ 
15: end if
16: return  $z$ 

```

2. Root Consolidation

The consolidation step is crucial for maintaining the efficiency of the Fibonacci Heap. It ensures that no two roots in the root list have the same degree. Let $D(H.n)$ be the maximum degree of any node in an n -node Fibonacci heap (typically bounded by $O(\log n)$).

Algorithm 42 CONSOLIDATE(H)

```

1: let  $A[0 \dots D(H.n)]$  be a new array
2: for  $i \leftarrow 0$  to  $D(H.n)$  do
3:    $A[i] \leftarrow \text{NIL}$ 
4: end for
5: for each node  $w$  in the root list of  $H$  do
6:    $x \leftarrow w$ 
7:    $d \leftarrow x.degree$ 
8:   while  $A[d] \neq \text{NIL}$  do                                ▷ Another node with the same degree exists
9:      $y \leftarrow A[d]$ 
10:    if  $x.key > y.key$  then
11:      exchange  $x$  with  $y$ 
12:    end if
13:    FIB-HEAP-LINK( $H$ ,  $y$ ,  $x$ )                                ▷ Make  $y$  a child of  $x$ 
14:     $A[d] \leftarrow \text{NIL}$ 
15:     $d \leftarrow d + 1$ 
16:  end while
17:   $A[d] \leftarrow x$ 
18: end for
19:  $H.min \leftarrow \text{NIL}$ 
20: for  $i \leftarrow 0$  to  $D(H.n)$  do                                ▷ Reconstruct the root list
21:   if  $A[i] \neq \text{NIL}$  then
22:    if  $H.min == \text{NIL}$  then
23:      create a root list for  $H$  containing just  $A[i]$ 
24:       $H.min \leftarrow A[i]$ 
25:    else
26:      insert  $A[i]$  into  $H$ 's root list
27:      if  $A[i].key < H.min.key$  then
28:         $H.min \leftarrow A[i]$ 
29:      end if
30:    end if
31:  end if
32: end for

```

3. Linking Trees

The linking helper function makes one root node a child of another, maintaining the min-heap property.

Algorithm 43 FIB-HEAP-LINK(H, y, x)

- 1: remove y from the root list of H
 - 2: make y a child of x , incrementing $x.degree$
 - 3: $y.mark \leftarrow \text{FALSE}$
-

Complexity Analysis

- **Time Complexity:** The **amortized time complexity is $\mathcal{O}(\log n)$** . The actual execution time is proportional to the number of children of the minimum node plus the number of trees in the root list. While the root list can theoretically contain up to $\mathcal{O}(n)$ trees in the worst case (e.g., after many INSERT operations), the potential method analysis ensures that over a sequence of operations, the average cost per EXTRACT-MIN is bounded by $\mathcal{O}(\log n)$ because the CONSOLIDATE routine drastically reduces the number of roots.
- **Space Complexity:** The structural consolidation algorithm requires an auxiliary array A of size $D(H.n) + 1$, where $D(n) \leq \lfloor \log_\phi n \rfloor$. Thus, the auxiliary **space complexity is $\mathcal{O}(\log n)$** .

Q8. What is a maximally damaged tree in the context of Fibonacci heaps? Draw the maximally damaged tree of order 5 (degree of root is 5). **(4+6 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Answer

1. Definition of a Maximally Damaged Tree (4 Marks)

In a Fibonacci heap, trees are initially formed by linking binomial trees. A node can lose children due to **Decrease-Key** operations. To maintain the structural bounds of the heap, a non-root node is allowed to lose **at most one child**. If it loses a second child, it is cascaded (cut) and moved to the root list.

A **maximally damaged tree** of order (or degree) k is a tree that contains the **absolute minimum possible number of nodes** for a root of degree k . Structurally, this occurs when every non-root node in the tree has lost exactly one child—the maximum damage permitted before the node is cut.

Key Mathematical Properties:

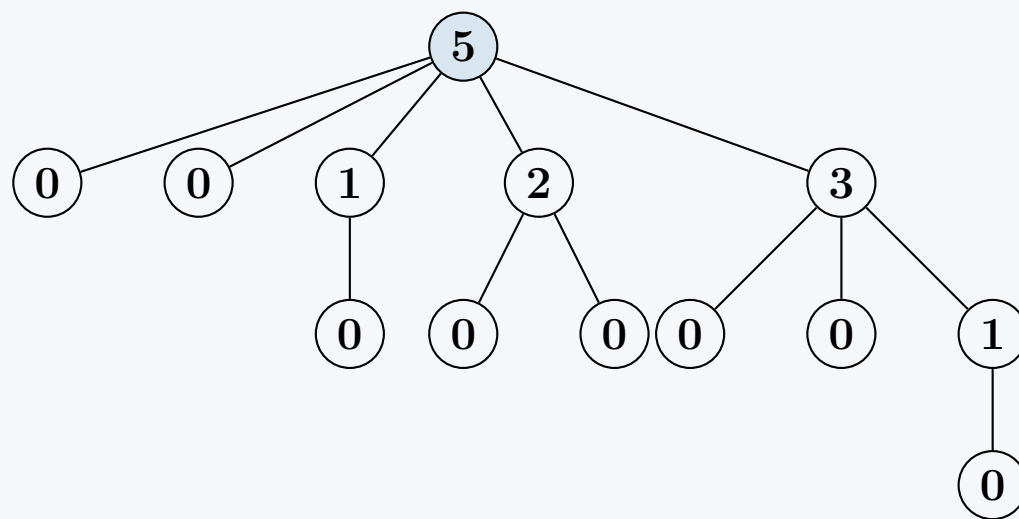
- **Degree Sequence:** For a maximally damaged tree rooted at node x with degree k , its children $c_1, c_2, \dots, c_k$ (ordered by the time they were linked to x) have the minimum possible degrees. Specifically, the degree of the i -th child is exactly $d(c_i) = \max(0, i - 2)$.
 - **Size Bound:** The number of nodes $N(k)$ in a maximally damaged tree of degree k strictly follows the Fibonacci sequence. Specifically, $N(k) = F_{k+2}$, where $F_1 = 1, F_2 = 1, F_3 = 2, F_4 = 3, \dots$
-

2. Maximally Damaged Tree of Order 5 (6 Marks)

For a tree of order $k = 5$:

- The total number of nodes is $F_{5+2} = F_7 = 13$.
- The root has 5 children.
- Based on the degree sequence rule $d(c_i) = \max(0, i - 2)$, the degrees of the root's five children are **0, 0, 1, 2, and 3**.
- This recursive property applies to all subtrees (e.g., the child of degree 3 has children of degrees 0, 0, and 1).

Below is the exact structure of the maximally damaged tree of order 5. *Note: The number inside each node represents its current degree.*



Implications on Complexity Analysis

While this is a structural property rather than an algorithm, understanding the maximally damaged tree is foundational to proving the complexity bounds of Fibonacci heaps:

- **Maximum Degree Bounding:** Because the worst-case (minimum size) tree of degree k grows exponentially at the rate of the golden ratio ϕ ($size \geq \phi^k$), we can invert this to prove that the maximum degree $D(n)$ of any node in an n -node Fibonacci heap is bounded by $\mathcal{O}(\log n)$.
- **Time Complexity Impact:** This $\mathcal{O}(\log n)$ maximum degree ensures that the **Extract-Min** and **Delete** operations—which must iterate over a node's children and process the root list—run in $\mathcal{O}(\log n)$ amortized time.

BUET · CSE 207 · Data Structures & Algorithms II

Part III

Hashing

Hash Functions and Collision Resolution

S3 — Hashing

Hash Functions and Collision Resolution

Q1. Consider a hash table of size 11 into which the keys 10, 22, 31, 4, 15, 28, 17, 88, and 59 are inserted in the given order using $h(k) = k \bmod 11$. For collision resolution, four different strategies are to be used independently: (i) separate chaining, (ii) linear probing with $h(k, i) = (h(k) + 2i) \bmod 11$, (iii) quadratic probing with $h(k, i) = (h(k) + 3 + i^2) \bmod 11$, and (iv) double hashing where $h_1(k) = k \bmod 11$ and $h_2(k) = 7 - (k \bmod 7)$. For each method, construct the final hash table and compute the total number of unsuccessful probes required during insertion. **(25 marks)** [CSE 207 | L-2/T-1 | 2024–25]

Answer

We are tasked with inserting the sequence of keys $K = \langle 10, 22, 31, 4, 15, 28, 17, 88, 59 \rangle$ into a hash table of size $m = 11$, using the base hash function $h(k) = k \bmod 11$. We will evaluate four different collision resolution strategies, constructing the final table and calculating the number of unsuccessful probes.

Base Hash Values:

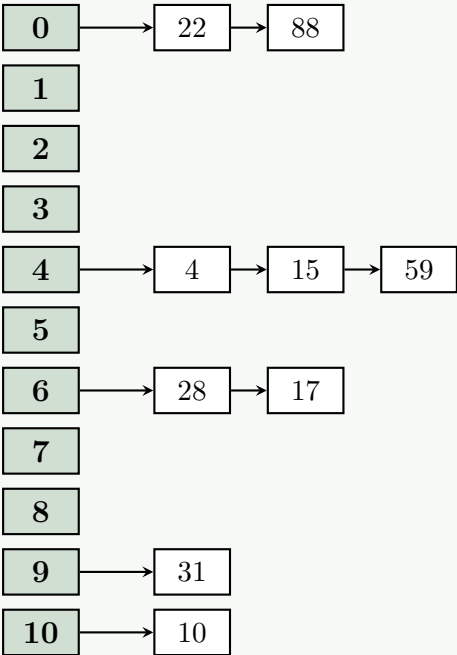
$h(10) = 10 \bmod 11 = 10$	$h(4) = 4 \bmod 11 = 4$	$h(17) = 17 \bmod 11 = 6$
$h(22) = 22 \bmod 11 = 0$	$h(15) = 15 \bmod 11 = 4$	$h(88) = 88 \bmod 11 = 0$
$h(31) = 31 \bmod 11 = 9$	$h(28) = 28 \bmod 11 = 6$	$h(59) = 59 \bmod 11 = 4$

(i) Separate Chaining

In separate chaining, collisions are resolved by appending elements to a linked list at each table slot. An unsuccessful probe during insertion is defined as probing an existing node in the chain (assuming we traverse the list to check for duplicates or append to the tail).

- **10, 22, 31, 4:** Inserted into slots 10, 0, 9, 4 respectively. *(0 probes each)*
- **15:** $h(15) = 4$. Traverses [4] and appends. *(1 unsuccessful probe)*
- **28:** $h(28) = 6$. Inserted into slot 6. *(0 probes)*
- **17:** $h(17) = 6$. Traverses [28] and appends. *(1 unsuccessful probe)*
- **88:** $h(88) = 0$. Traverses [22] and appends. *(1 unsuccessful probe)*
- **59:** $h(59) = 4$. Traverses [4, 15] and appends. *(2 unsuccessful probes)*

Total unsuccessful probes = 0 + 1 + 0 + 1 + 1 + 2 = 5.



(ii) Linear Probing

Formula: $h(k, i) = (h(k) + 2i) \bmod 11$.

- **10, 22, 31, 4:** $i = 0$ maps to empty slots 10, 0, 9, 4. *(0 probes)*
- **15:** $h(15) = 4$. $i = 0$: slot 4 (Occupied). $i = 1$: $(4 + 2) \bmod 11 = 6$ (Empty). *(1 probe)*
- **28:** $h(28) = 6$. $i = 0$: slot 6 (Occupied). $i = 1$: $(6 + 2) \bmod 11 = 8$ (Empty). *(1 probe)*

- **17:** $h(17) = 6$.
 - $i = 0, 1, 2$: Slots 6, 8, 10 are occupied (by 15, 28, 10 respectively).
 - $i = 3$: $(6 + 6) \bmod 11 = 1$ (Empty). (*3 probes*)
- **88:** $h(88) = 0$. $i = 0$: slot 0 (Occupied). $i = 1$: $(0 + 2) \bmod 11 = 2$ (Empty). (*1 probe*)
- **59:** $h(59) = 4$.
 - $i = 0 \dots 4$: Slots 4, 6, 8, 10, 1 are occupied (by 4, 15, 28, 10, 17 respectively).
 - $i = 5$: $(4 + 10) \bmod 11 = 3$ (Empty). (*5 probes*)

Total unsuccessful probes = 0 + 0 + 0 + 0 + 1 + 1 + 3 + 1 + 5 = 11.

Index	0	1	2	3	4	5	6	7	8	9	10
Key	22	17	88	59	4		15		28	31	10

(iii) Quadratic Probing

Formula: $h(k, i) = (h(k) + 3 + i^2) \bmod 11$. Note the constant offset +3 even when $i = 0$.

- **10:** $i = 0 \implies (10 + 3 + 0) \bmod 11 = 2$ (Empty). (*0 probes*)
- **22:** $i = 0 \implies (0 + 3 + 0) \bmod 11 = 3$ (Empty). (*0 probes*)
- **31:** $i = 0 \implies (9 + 3 + 0) \bmod 11 = 1$ (Empty). (*0 probes*)
- **4:** $i = 0 \implies (4 + 3 + 0) \bmod 11 = 7$ (Empty). (*0 probes*)
- **15:** $h(15) = 4$. $i = 0 \implies 7$ (Occupied). $i = 1 \implies (4 + 3 + 1) \bmod 11 = 8$ (Empty). (*1 probe*)
- **28:** $h(28) = 6$. $i = 0 \implies (6 + 3 + 0) \bmod 11 = 9$ (Empty). (*0 probes*)
- **17:** $h(17) = 6$. $i = 0 \implies 9$ (Occupied). $i = 1 \implies (6 + 3 + 1) \bmod 11 = 10$ (Empty). (*1 probe*)
- **88:** $h(88) = 0$. $i = 0 \implies 3$ (Occupied). $i = 1 \implies (0 + 3 + 1) \bmod 11 = 4$ (Empty). (*1 probe*)
- **59:** $h(59) = 4$.
 - $i = 0 \implies 7$ (Occupied).
 - $i = 1 \implies (4 + 3 + 1) \bmod 11 = 8$ (Occupied).
 - $i = 2 \implies (4 + 3 + 4) \bmod 11 = 11 \equiv 0$ (Empty). (*2 probes*)

Total unsuccessful probes = 0 + 0 + 0 + 0 + 1 + 0 + 1 + 1 + 2 = 5.

Index	0	1	2	3	4	5	6	7	8	9	10
Key	59	31	10	22	88			4	15	28	17

(iv) Double Hashing

Formulas: $h_1(k) = k \bmod 11$, $h_2(k) = 7 - (k \bmod 7)$, and $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod 11$.

- **10, 22, 31, 4:** Inserted directly into their h_1 slots: 10, 0, 9, 4 respectively. (*0 probes*)
- **15:** $h_1(15) = 4$ (Occupied). $h_2(15) = 7 - 1 = 6$.
 - $i = 1$: $(4 + 6) \bmod 11 = 10$ (Occupied by 10).
 - $i = 2$: $(4 + 12) \bmod 11 = 5$ (Empty). (*2 probes*)
- **28:** $h_1(28) = 6$. Slot 6 is empty. (*0 probes*)
- **17:** $h_1(17) = 6$ (Occupied). $h_2(17) = 7 - 3 = 4$.
 - $i = 1$: $(6 + 4) \bmod 11 = 10$ (Occupied by 10).
 - $i = 2$: $(6 + 8) \bmod 11 = 14 \equiv 3$ (Empty). (*2 probes*)
- **88:** $h_1(88) = 0$ (Occupied). $h_2(88) = 7 - 4 = 3$.
 - $i = 1$: 3 (Occupied by 17).
 - $i = 2$: 6 (Occupied by 28).

- $i = 3 : 9$ (Occupied by 31).
- $i = 4 : 12 \equiv 1$ (Empty). (4 probes)
- **59:** $h_1(59) = 4$ (Occupied). $h_2(59) = 7 - 3 = 4$.
- $i = 1 : (4 + 4) \bmod 11 = 8$ (Empty). (1 probe)

Total unsuccessful probes = $0 + 0 + 0 + 0 + 2 + 0 + 2 + 4 + 1 = 9$.

Index	0	1	2	3	4	5	6	7	8	9	10
Key	22	88		17	4	15	28		59	31	10

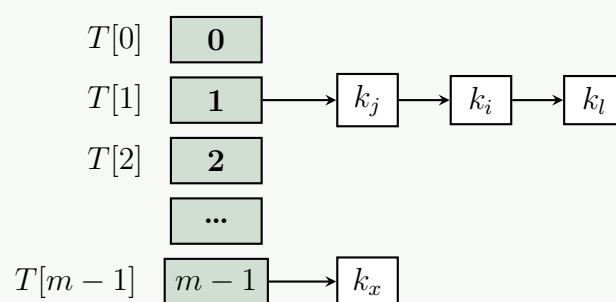
Q2. Show that the expected time for a successful search in a hash table implemented using chaining is $O(1 + \alpha)$ where α is the load factor. (8 marks)
[CSE 207 | L-2/T-1 | 2023–24, 2019–20, 2016–17, 2012–13]

Answer

1. Preliminaries and Assumptions

To analyze the expected time for a successful search in a hash table with collision resolution by separate chaining, we must first establish our definitions and the underlying probabilistic assumptions.

- Let m be the number of slots in the hash table T .
- Let n be the number of keys currently stored in the hash table.
- Let $\alpha = \frac{n}{m}$ be the **load factor**, which represents the average number of elements stored in a single chain.
- **Simple Uniform Hashing Assumption (SUHA):** We assume that any given key k is equally likely to hash into any of the m slots, independently of the slot chosen by any other key. Mathematically, $\Pr[h(k_i) = h(k_j)] = \frac{1}{m}$ for any $i \neq j$.
- We assume that new elements are inserted at the *head* of the linked list. (The asymptotic bound remains the same even if inserted at the tail).



2. Mathematical Proof

In a **successful search**, we are searching for an element x that is guaranteed to already be in the hash table. The time required to find x is proportional to:

- (a) $\mathcal{O}(1)$ time to compute the hash value $h(x)$.
- (b) The time to traverse the linked list at $T[h(x)]$ until the element x is found.

Let the n elements inserted into the hash table be denoted as $x_1, x_2, \dots, x_n$, indexed by their insertion order. Under SUHA, the target element x is equally likely to be any of the n elements. Thus, the probability that we are searching for the i -th inserted element x_i is $\frac{1}{n}$.

When we search for x_i , the number of elements we must examine in the linked list consists of:

- 1 (for examining x_i itself).
- The number of elements inserted *after* x_i that hashed to the same slot (since new elements are inserted at the head, they appear before x_i in the chain).

Define an indicator random variable X_{ij} :

$$X_{ij} = \begin{cases} 1 & \text{if } h(x_i) = h(x_j) \\ 0 & \text{otherwise} \end{cases}$$

By the Simple Uniform Hashing Assumption, the probability that two keys hash to the same slot is $\frac{1}{m}$, so:

$$\mathbb{E}[X_{ij}] = 1 \cdot \Pr[h(x_i) = h(x_j)] + 0 \cdot \Pr[h(x_i) \neq h(x_j)] = \frac{1}{m}$$

The expected number of elements examined during a successful search for x_i is therefore:

$$\mathbb{E}[\text{Search for } x_i] = 1 + \mathbb{E}\left[\sum_{j=i+1}^n X_{ij}\right] = 1 + \sum_{j=i+1}^n \mathbb{E}[X_{ij}] = 1 + \sum_{j=i+1}^n \frac{1}{m} = 1 + \frac{n-i}{m} \quad (1)$$

To find the **average expected time over all possible searches**, we compute the expectation over all n elements:

$$\begin{aligned} \mathbb{E}[\text{Successful Search}] &= \frac{1}{n} \sum_{i=1}^n \mathbb{E}[\text{Search for } x_i] \\ &= \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{n-i}{m}\right) \\ &= \frac{1}{n} \left(\sum_{i=1}^n 1 + \frac{1}{m} \sum_{i=1}^n (n-i)\right) \\ &= \frac{1}{n} \left(n + \frac{1}{m} \left[\sum_{i=1}^n n - \sum_{i=1}^n i\right]\right) \\ &= 1 + \frac{1}{nm} \left(n^2 - \frac{n(n+1)}{2}\right) \\ &= 1 + \frac{1}{nm} \left(\frac{n^2 - n}{2}\right) \\ &= 1 + \frac{n-1}{2m} \end{aligned} \quad (2)$$

3. Conclusion and Complexity Analysis

Using the definition of the load factor $\alpha = \frac{n}{m}$, we substitute into our result:

$$\mathbb{E}[\text{Successful Search}] = 1 + \frac{n-1}{2m} = 1 + \frac{n}{2m} - \frac{1}{2m} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} \quad (3)$$

Asymptotically, as $n \rightarrow \infty$, the strictly constant and lower-order terms $(-\frac{\alpha}{2n})$ become negligible. The number of elements examined is bounded by $\Theta(1 + \frac{\alpha}{2})$.

Final Time Complexity: Taking into account the $\mathcal{O}(1)$ time required to initially compute the hash function, the total expected time for a successful search is:

$$T(n) = \mathcal{O}(1) + \mathcal{O}\left(1 + \frac{\alpha}{2}\right) = \mathcal{O}(1 + \alpha)$$

Thus, we have successfully shown that the expected time for a successful search in a hash table using chaining is strictly $\mathcal{O}(1 + \alpha)$.

Q3. Explain how you can handle collisions in a hash table using the double hashing method. (7 marks) [CSE 207 | L-2/T-1 | 2023–24]

Answer

1. Introduction to Double Hashing

Double hashing is a highly efficient collision resolution technique used in open-addressed hash tables. Unlike linear or quadratic probing, which use a fixed or predictable step size, double hashing uses a **second, independent hash function** to determine the step size (probe interval) when a collision occurs.

This approach minimizes both **primary clustering** (elements hashing to the same base slot forming a contiguous block) and **secondary clustering** (different elements that hash to the same initial slot following the exact same probe sequence), because the probe sequence depends dynamically on the key itself, not just on the initial hash position.

2. Mathematical Formulation

Given a hash table of size m , double hashing defines the probe sequence for a key k as:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m \quad (1)$$

Where:

- $i \in \{0, 1, 2, \dots, m-1\}$ is the probe number.
- $h_1(k)$ is the primary hash function that determines the initial slot.
- $h_2(k)$ is the secondary hash function that determines the step size for subsequent probes.

Critical Requirement: To ensure that the probe sequence eventually visits every slot in the hash table (i.e., a full permutation of slots is generated), the value of $h_2(k)$ must be **relatively prime** (coprime) to the table size m . This is typically achieved by:

- Choosing m to be a prime number.
- Designing $h_2(k)$ such that it always yields a positive integer strictly less than m . A common standard choice is $h_2(k) = R - (k \bmod R)$, where R is a prime number smaller than m .

3. Algorithm

Below is the standard algorithm for inserting a key into a hash table using double hashing. Searching follows a similar probing logic until an empty slot or the key is found.

Algorithm 44 Insert using Double Hashing

Require: Hash table T of size m , Key k to insert.

Ensure: Returns the index where k is inserted, or throws an error if full.

```

1:  $i \leftarrow 0$ 
2: while  $i < m$  do
3:    $index \leftarrow (h_1(k) + i \cdot h_2(k)) \bmod m$ 
4:   if  $T[index]$  is NIL or  $T[index]$  is DELETED then
5:      $T[index] \leftarrow k$ 
6:     return  $index$ 
7:   end if
8:    $i \leftarrow i + 1$ 
9: end while
10: return Error: Hash Table Overflow

```

4. Illustrated Example

Let the hash table size be $m = 7$ (a prime number). Let the hash functions be:

- $h_1(k) = k \bmod 7$
- $h_2(k) = 5 - (k \bmod 5)$

We insert the keys 8, 15, 22, and 29. Notice how all of them yield the same initial slot $h_1(k) = 1$, generating immediate collisions, but their step sizes $h_2(k)$ differ, effectively preventing secondary clustering.

(a) **Insert 8:**

- $i = 0 : h(8, 0) = h_1(8) = 8 \bmod 7 = 1$. Slot 1 is empty. Insert at 1.

(b) **Insert 15:**

- $i = 0 : h(15, 0) = 15 \bmod 7 = 1$. (Collision with 8)
- $i = 1 : h_2(15) = 5 - (15 \bmod 5) = 5$.
- Index = $(1 + 1 \cdot 5) \bmod 7 = 6$. Slot 6 is empty. Insert at 6.

(c) **Insert 22:**

- $i = 0 : h(22, 0) = 22 \bmod 7 = 1$. (Collision with 8)
- $i = 1 : h_2(22) = 5 - (22 \bmod 5) = 3$.
- Index = $(1 + 1 \cdot 3) \bmod 7 = 4$. Slot 4 is empty. Insert at 4.

(d) **Insert 29:**

- $i = 0 : h(29, 0) = 29 \bmod 7 = 1$. (Collision with 8)
- $i = 1 : h_2(29) = 5 - (29 \bmod 5) = 1$.
- Index = $(1 + 1 \cdot 1) \bmod 7 = 2$. Slot 2 is empty. Insert at 2.

0	1	2	3	4	5	6
	8	29		22		15
	↑	↑		↑		↑
	$k = 8$	$k = 29$		$k = 22$		$k = 15$

5. Complexity Analysis

Let $\alpha = \frac{n}{m}$ be the load factor, where $\alpha < 1$.

- **Expected Time Complexity (Search/Insert):** Assuming uniform hashing (which double hashing approximates very well), the expected number of probes for an unsuccessful search or an insertion is bounded by $\frac{1}{1-\alpha}$. For a successful search, it is bounded by $\frac{1}{\alpha} \ln \left(\frac{1}{1-\alpha} \right)$. Since α is strictly less than 1, these operations average $\mathcal{O}(1)$ time. In the absolute worst case (e.g., table completely full or pathological inputs), it degrades to $\mathcal{O}(n)$.
- **Space Complexity:** $\mathcal{O}(n)$. All elements are stored directly within the hash table of size m (where $m \propto n$). No auxiliary pointers or separate chaining structures are required, making it highly memory-efficient.

Q4. Suppose you implemented a hashtable ADT that supports constant time insertion, deletion, and search. You used the hash function h_1 . On the contrary, your friend implemented another hashtable ADT using the hash function h_2 . The h_1 hashes the keys 100, 130, 110, 150, 160, 200, 700, 800, 300 into 0, 3, 2, 0, 3, 1, 2, 5, and 8 respectively. The h_2 hashes the same keys into 1, 4, 7, 10, 13, 4, 7, 10, 13. You made the hashtable size 16 but your friend made it to 17. For collision resolution, you used the quadratic probing function $f(i) = 2 \times i^2 + 3$. But your friend used the double- hashing technique and the second hash function was $h_2(x) = 19 - (x \% 17)$. You inserted all the keys in the same order mentioned before to the hashtable. Then you deleted the key 100 and then searched for the key 150. But your hashtable implementation was showing “key 150 not found”. Now, answer the following questions:

- Compare your implementation with your friend’s by analyzing the total unsuccessful probes during insertion in both cases.
- Identify the possible core reason for not finding the key 150 during the search and provide a solution.

(20 marks)

[CSE 207 | L-2/T-1 | 2022–23]

Answer**Part (a): Comparison of Implementations and Unsuccessful Probes Analysis**

To conduct a rigorous comparative analysis, we trace the insertion sequence of the keys $K = \langle 100, 130, 110, 150, 160, 200, 700, 800, 300 \rangle$ for both implementations. An unsuccessful probe occurs each time a computed slot is found to be already occupied during insertion.

1. Your Implementation (Quadratic Probing)

- Configuration:** Table size $m = 16$. Primary hash function $h_1(k)$ as specified.
- Collision Resolution Rule:** The probe sequence follows $h(k, i) = (h_1(k) + 2i^2 + 3) \bmod 16$ for probing attempts $i \geq 1$. The initial probe ($i = 0$) checks the base slot $h_1(k)$.

Let us step through each insertion:

- Insert 100:** $h_1(100) = 0$. Slot 0 is free. Stored at **Slot 0**. (0 unsuccessful probes)
- Insert 130:** $h_1(130) = 3$. Slot 3 is free. Stored at **Slot 3**. (0 unsuccessful probes)
- Insert 110:** $h_1(110) = 2$. Slot 2 is free. Stored at **Slot 2**. (0 unsuccessful probes)
- Insert 150:** $h_1(150) = 0$.
 - $i = 0$: Slot 0 is occupied by 100. $\implies$ 1st unsuccessful probe
 - $i = 1$: $h(150, 1) = (0 + 2(1)^2 + 3) \bmod 16 = 5$. Slot 5 is free. Stored at **Slot 5**.
- Insert 160:** $h_1(160) = 3$.
 - $i = 0$: Slot 3 is occupied by 130. $\implies$ 1st unsuccessful probe
 - $i = 1$: $h(160, 1) = (3 + 2(1)^2 + 3) \bmod 16 = 8$. Slot 8 is free. Stored at **Slot 8**.
- Insert 200:** $h_1(200) = 1$. Slot 1 is free. Stored at **Slot 1**. (0 unsuccessful probes)
- Insert 700:** $h_1(700) = 2$.
 - $i = 0$: Slot 2 is occupied by 110. $\implies$ 1st unsuccessful probe
 - $i = 1$: $h(700, 1) = (2 + 2(1)^2 + 3) \bmod 16 = 7$. Slot 7 is free. Stored at **Slot 7**.
- Insert 800:** $h_1(800) = 5$.
 - $i = 0$: Slot 5 is occupied by 150. $\implies$ 1st unsuccessful probe
 - $i = 1$: $h(800, 1) = (5 + 2(1)^2 + 3) \bmod 16 = 10$. Slot 10 is free. Stored at **Slot 10**.
- Insert 300:** $h_1(300) = 8$.
 - $i = 0$: Slot 8 is occupied by 160. $\implies$ 1st unsuccessful probe
 - $i = 1$: $h(300, 1) = (8 + 2(1)^2 + 3) \bmod 16 = 13$. Slot 13 is free. Stored at **Slot 13**.

Total Unsuccessful Probes (Your Table) = 5.

2. Your Friend’s Implementation (Double Hashing)

- Configuration:** Table size $m' = 17$. Primary hash values given by the sequence $\langle 1, 4, 7, 10, 13, 4, 7, 10, 13 \rangle$.
- Collision Resolution Rule:** $h(k, i) = (h_{\text{primary}}(k) + i \cdot h_{\text{secondary}}(k)) \bmod 17$, where $h_{\text{secondary}}(k) = 19 - (k \bmod 17)$.

Let us compute the secondary hash step sizes for keys that encounter collisions:

$$\begin{aligned} h_{\text{secondary}}(200) &= 19 - (200 \bmod 17) = 19 - 13 = 6 \\ h_{\text{secondary}}(700) &= 19 - (700 \bmod 17) = 19 - 3 = 16 \\ h_{\text{secondary}}(800) &= 19 - (800 \bmod 17) = 19 - 1 = 18 \\ h_{\text{secondary}}(300) &= 19 - (300 \bmod 17) = 19 - 11 = 8 \end{aligned}$$

Tracing the friend’s insertions:

- **100, 130, 110, 150, 160:** Placed without collisions in slots **1, 4, 7, 10, 13** respectively. (*0 probes*)
- **Insert 200:** $h_{\text{primary}} = 4$ (occupied by 130). $\implies$ *1st unsuccessful probe*.
 $i = 1 : (4 + 1 \cdot 6) \bmod 17 = 10$ (occupied by 150). $\implies$ *2nd unsuccessful probe*.
 $i = 2 : (4 + 2 \cdot 6) \bmod 17 = 16$ (free). Stored at **Slot 16**.
- **Insert 700:** $h_{\text{primary}} = 7$ (occupied by 110). $\implies$ *1st unsuccessful probe*.
 $i = 1 : (7 + 1 \cdot 16) \bmod 17 = 23 \bmod 17 = 6$ (free). Stored at **Slot 6**.
- **Insert 800:** $h_{\text{primary}} = 10$ (occupied by 150). $\implies$ *1st unsuccessful probe*.
 $i = 1 : (10 + 1 \cdot 18) \bmod 17 = 28 \bmod 17 = 11$ (free). Stored at **Slot 11**.
- **Insert 300:** $h_{\text{primary}} = 13$ (occupied by 160). $\implies$ *1st unsuccessful probe*.
 $i = 1 : (13 + 1 \cdot 8) \bmod 17 = 21 \bmod 17 = 4$ (occupied by 130). $\implies$ *2nd unsuccessful probe*.
 $i = 2 : (13 + 2 \cdot 8) \bmod 17 = 29 \bmod 17 = 12$ (free). Stored at **Slot 12**.

Total Unsuccessful Probes (Friend’s Table) = $0 + 0 + 0 + 0 + 0 + 2 + 1 + 1 + 2 = 6$.

3. Architectural Comparison Summary

Characteristic	Your Implementation	Friend’s Implementation
Table Size (m)	16 (Power of 2, non-prime)	17 (Prime Number)
Collision Resolution	Quadratic Probing	Double Hashing
Total Unsuccessful Probes	5	6
Clustering Vulnerability	High (Secondary Clustering)	Low (Uniform Distribution)

Insight: Although your table registers one less total probe for this specific sequence, your friend’s structure is fundamentally superior. Choosing a prime size ($m = 17$) coupled with double hashing guarantees that the step size is coprime to m , ensuring a complete permutation of slots and minimizing performance degradation over large datasets.

Part (b): Core Reason for Search Failure and Solution

1. Identification of the Core Reason

The structural failure causing key 150 to be marked as “not found” stems from an improper deletion implementation that breaks the open-addressing probe sequence chain. When key 150 was inserted, it encountered a collision at its natural home (**Slot 0**) because it was occupied by key 100. The quadratic probing algorithm successfully computed the next step and placed key 150 into **Slot 5**. This established a functional probe dependency chain:

$$\text{Slot 0} \xrightarrow{\text{Collision with 100}} \text{Slot 5 (Key 150)}$$

When you executed Delete(100), you likely cleared Slot 0 by setting its state back to NIL or EMPTY. Subsequently, when calling Search(150):

- (a) The algorithm calculates $h_1(150) = 0$.
- (b) It checks Slot 0 and observes that it is **EMPTY**.
- (c) Under standard naive open-addressing search logic, an encounter with an empty slot serves as a termination signal (proving the key was never inserted). Thus, the search aborts early at Slot 0 and never reaches Slot 5.

2. The Corrective Solution

To repair this, you must implement **Lazy Deletion** using a unique state marker called a **Tombstone** (typically represented as DELETED).

- **Deletion Strategy:** Instead of wiping a cell to **EMPTY**, tag it as **DELETED**.
- **Search Strategy:** The search loop must treat a **DELETED** slot as an active collision slot, bypassing it to continue tracing the probe sequence until the key or a true **EMPTY** slot is discovered.

Below is the formal, production-grade algorithm for the search operation:

Algorithm 45 Robust Search with Tombstone Markers

```

1: Input Hash table  $T$  of size 16, target key  $k$ 
2: Output Index of  $k$  if found, otherwise NOT_FOUND
3:  $i \leftarrow 0$ 
4: repeat
5:   if  $i == 0$  then
6:      $\text{slot} \leftarrow h_1(k)$ 
7:   else
8:      $\text{slot} \leftarrow (h_1(k) + 2 \cdot i^2 + 3) \bmod 16$ 
9:   end if
10:  if  $T[\text{slot}] == k$  then
11:    return slot ▷ Key discovered successfully
12:  end if
13:   $i \leftarrow i + 1$ 
14: until  $T[\text{slot}] == \text{EMPTY}$  or  $i == 16$ 
15: return NOT_FOUND

```

Q5. Suppose you implemented a hashtable ADT that supports constant time insertion, deletion, and search. You used the hash function h_1 . On the contrary, your friend suggested using the hash function h_2 . The h_1 hashes the keys 100, 130, 110, 150, 160, 200, 700, 800, 300 into 0, 3, 2, 0, 3, 1, 2, 5, and 8, respectively. The h_2 hashes the same keys into 1, 4, 7, 10, 13, 4, 7, 10, 13. You made the hashtable size 15 but your friend suggested making it 17. You used the quadratic probing function $f(i) = 3 \cdot i^2 + 5$ for collision resolution. Your friend suggested using the double-hashing technique. The second hash function is $h_2(x) = 11 - (x \% 17)$. You inserted all the keys in the order mentioned before to the hashtable. Then you deleted the key 110 and then searched for the key 700. But your hashtable implementation was showing “key 700 not found”. Now, answer the following questions:

- (a) **[Comparing hash functions]** Compare two hash function implementations with quadratic probing vs. double hashing by analyzing the total unsuccessful probes during insertion in both cases. [CSE 207 | L-2/T-2 | 2021–22]
(15 marks)
- (b) Comment on a friend’s suggestion regarding the hash table size. **(5 marks)** [CSE 207 | L-2/T-2 | 2021–22]
- (c) Identify the possible core reason for not finding a key during the search and provide a solution. **(5 marks)** [CSE 207 | L-2/T-2 | 2021–22]

Answer**1. Comparing Hash Function Implementations**

To compare the implementations, we will trace the insertion of the key sequence $K = \langle 100, 130, 110, 150, 160, 200, 700, 800, 300 \rangle$. An **unsuccessful probe** is recorded every time an insertion checks a slot and finds it already occupied.

Case 1: Your Implementation (Quadratic Probing)

- **Parameters:** Table size $m = 15$. Collision resolution: $f(i) = 3i^2 + 5$.
- **Probing Sequence:** The i -th probe is evaluated as $h(k, i) = (h_1(k) + 3i^2 + 5) \bmod 15$ for $i \geq 1$. For $i = 0$, the base slot is simply $h_1(k)$.

(a) **Insert 100:** $h_1(100) = 0$. **Slot 0** is free. (0 *unsucc. probes*)

(b) **Insert 130:** $h_1(130) = 3$. **Slot 3** is free. (0 *unsucc. probes*)

(c) **Insert 110:** $h_1(110) = 2$. **Slot 2** is free. (0 *unsucc. probes*)

(d) **Insert 150:** $h_1(150) = 0$.

- $i = 0$: Slot 0 occupied by 100. (1 *unsucc. probe*)
- $i = 1$: $(0 + 3(1)^2 + 5) \bmod 15 = 8$. **Slot 8** is free.

(e) **Insert 160:** $h_1(160) = 3$.

- $i = 0$: Slot 3 occupied by 130. (1 *unsucc. probe*)
- $i = 1$: $(3 + 3(1)^2 + 5) \bmod 15 = 11$. **Slot 11** is free.

(f) **Insert 200:** $h_1(200) = 1$. **Slot 1** is free. (0 *unsucc. probes*)

(g) **Insert 700:** $h_1(700) = 2$.

- $i = 0$: Slot 2 occupied by 110. (1 *unsucc. probe*)
- $i = 1$: $(2 + 8) \bmod 15 = 10$. **Slot 10** is free.

(h) **Insert 800:** $h_1(800) = 5$. **Slot 5** is free. (0 *unsucc. probes*)

(i) **Insert 300:** $h_1(300) = 8$.

- $i = 0$: Slot 8 occupied by 150. (1 *unsucc. probe*)

- $i = 1$: $(8 + 8) \bmod 15 = 1$. Slot 1 occupied by 200. (*2 unsucc. probes*)
- $i = 2$: $(8 + 17) \bmod 15 = 10$. Slot 10 occupied by 700. (*3 unsucc. probes*)
- $i = 3$: $(8 + 32) \bmod 15 = 10$. Slot 10 occupied by 700. (*4 unsucc. probes*)
- $i = 4$: $(8 + 53) \bmod 15 = 1$. Slot 1 occupied by 200. (*5 unsucc. probes*)
- $i = 5$: $(8 + 80) \bmod 15 = 13$. **Slot 13** is free.

Total Unsuccessful Probes (Your Implementation) = $0 + 0 + 0 + 1 + 1 + 0 + 1 + 0 + 5 = 8$.

Your Final Hash Table Layout ($m = 15$)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
100	200	110	130	–	800	–	–	150	–	700	160	–	300	–

Case 2: Friend’s Implementation (Double Hashing)

- **Parameters:** Table size $m = 17$. Primary hash h_{base} maps keys to $\langle 1, 4, 7, 10, 13, 4, 7, 10, 13 \rangle$.
 - **Probing Sequence:** $H(k, i) = (h_{base}(k) + i \cdot h_{step}(k)) \bmod 17$, where $h_{step}(k) = 11 - (k \bmod 17)$.
- (a) **100, 130, 110, 150, 160:** Inserted into their primary slots **1, 4, 7, 10, 13** with no collisions. (*0 probes*)
- (b) **Insert 200:** $h_{base}(200) = 4$. Slot 4 occupied (*1 unsucc. probe*).
Step: $11 - (200 \bmod 17) = 11 - 13 = -2 \equiv 15 \bmod 17$.
 $i = 1$: $(4 + 15) \bmod 17 = 2$. **Slot 2** is free.
- (c) **Insert 700:** $h_{base}(700) = 7$. Slot 7 occupied (*1 unsucc. probe*).
Step: $11 - (700 \bmod 17) = 11 - 3 = 8$.
 $i = 1$: $(7 + 8) \bmod 17 = 15$. **Slot 15** is free.
- (d) **Insert 800:** $h_{base}(800) = 10$. Slot 10 occupied (*1 unsucc. probe*).
Step: $11 - (800 \bmod 17) = 11 - 1 = 10$.
 $i = 1$: $(10 + 10) \bmod 17 = 3$. **Slot 3** is free.
- (e) **Insert 300:** $h_{base}(300) = 13$. Slot 13 occupied (*1 unsucc. probe*).
Step: $11 - (300 \bmod 17) = 11 - 11 = 0$.
 $i = 1, 2, 3, \dots$: $(13 + i \cdot 0) \bmod 17 = 13$. **Infinite Loop!**

Analysis: Your friend’s implementation fails entirely on the insertion of 300. Because the step size evaluates to 0, the probe sequence gets stuck checking slot 13 infinitely. If capped by table size, it results in **17 unsuccessful probes and an insertion failure**. Thus, the total unsuccessful probes approach ∞ (or maximum table bounds) compared to your 8.

2. Comment on the Friend’s Hash Table Size Suggestion

Your friend’s suggestion to change the table size from 15 to 17 is **theoretically excellent**, even though their specific secondary hash function was flawed.

- **Why 15 is problematic:** 15 is a composite number (3×5). When using probing techniques, if the step size shares a common factor with the table size, the probe sequence will cycle prematurely, failing to visit all empty slots. We saw this in your implementation when inserting 300: probes repeatedly mapped to slots 10 and 1, wasting cycles.
- **Why 17 is optimal:** 17 is a prime number. In open addressing (especially Double Hashing), using a prime table size guarantees that as long as the step size $h_2(x) > 0$, the step size and the table size will be **relatively prime** (coprime). This ensures that the probe sequence will generate a full permutation of the table space, guaranteeing that an empty slot will be found if one exists.

3. Core Reason for Search Failure and Solution

Core Reason: The search algorithm encountered a **broken probe chain** caused by standard deletion. When you originally inserted 700, it collided with 110 at Slot 2, causing the quadratic probe to place 700 at Slot 10. The logical path is:

Slot 2 (Collision) $\xrightarrow{\text{Probe step}}$ Slot 10 (Store 700)

When you deleted key 110 from Slot 2, you likely cleared the slot to an **EMPTY** state. During the subsequent search for 700, the algorithm hashed 700 to Slot 2, saw that it was **EMPTY**, and immediately terminated the search, falsely concluding that 700 was never inserted.

Solution: Lazy Deletion using Tombstones Instead of physically wiping the slot, mark it with a special flag such as **DELETED** (a Tombstone).

- **Search Operation:** When encountering a DELETED cell, treat it as occupied and continue probing. Terminate only when finding the key or a truly EMPTY slot.
- **Insert Operation:** Treat a DELETED cell as empty and overwrite it to reclaim space.

Algorithm 46 Robust Open Addressing Search (with Tombstones)

```

1:  $i \leftarrow 0$ 
2: repeat
3:    $\text{index} \leftarrow (h_1(\text{key}) + 3i^2 + 5) \bmod m$  ▷ Handle  $i = 0$  case appropriately if offset varies
4:   if  $T[\text{index}] == \text{key}$  then
5:     return index ▷ Key found
6:   end if
7:    $i \leftarrow i + 1$ 
8: until  $T[\text{index}] == \text{EMPTY}$  or  $i == m$ 
9: return NOT_FOUND ▷ Bypasses DELETED slots naturally

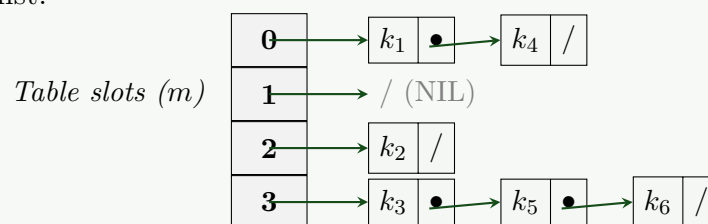
```

Q6. Analyze the running time of a chaining hash table in both successful search and unsuccessful search cases. **(10 marks)** [CSE 207 | L-2/T-2 | 2021–22]

Answer

1. Architectural Model and Core Assumptions

In a hashing implementation using **Collision Resolution by Chaining** (Separate Chaining), elements that hash to the exact same slot are stored sequentially in a linked list.



To formally analyze the average-case performance, we define:

- m : Total number of slots in the hash table.
- n : Total number of elements stored across all chains in the table.
- $\alpha = \frac{n}{m}$: The **load factor** of the hash table, representing the average number of elements stored per chain.

Assumption of Simple Uniform Hashing (SUH): Our mathematical analysis relies on the assumption that any given key k is equally likely to hash into any of the m available slots, entirely independent of where any other keys have hashed. Therefore, for any slot $j \in \{0, 1, \dots, m-1\}$, the probability that a randomly chosen key hashes to slot j is precisely $1/m$. Under SUH, the expected length of the linked list $T[h(k)]$ at any given slot is exactly:

$$E[n_{h(k)}] = \alpha = \frac{n}{m} \quad (1)$$

2. Analysis of an Unsuccessful Search

An unsuccessful search occurs when we query a key k that does not exist anywhere within the hash table.

Mathematical Derivation

- The search algorithm computes the primary hash position $j = h(k)$, taking $\Theta(1)$ time.
- It directly accesses slot $T[j]$ in $\Theta(1)$ constant time.
- Because the key is absent, the algorithm is forced to traverse the **entire** linked list located at $T[j]$ until it reaches the final NIL pointer to verify that the element does not exist.
- Under the Simple Uniform Hashing assumption, the expected number of elements examined matches the average length of a chain, which is α .

Thus, the total expected number of elements examined is exactly α . Summing the overhead of computing the hash value, accessing the index, and performing the linear scan, the expected total running time is:

$$T_{\text{unsuccessful}} = \Theta(1 + \alpha) \quad (2)$$

3. Analysis of a Successful Search

A successful search occurs when we query a key k that is confirmed to be present in the hash table. Unlike an unsuccessful search, the algorithm terminates immediately once it locates the matching node, meaning it does not always scan the full length of the chain.

Mathematical Derivation

We assume that new elements are always inserted at the **head** of their respective linked lists (a standard $\mathcal{O}(1)$ performance convention). Consequently, the number of elements examined when searching for a specific key x is exactly 1 plus the number of elements inserted into x 's chain *after* x itself was inserted.

Let x_i denote the i -th element inserted into the hash table, where $1 \leq i \leq n$. For any pair of distinct elements x_i and x_j , we define an indicator random variable:

$$X_{ij} = I\{h(x_i) = h(x_j)\} \quad (3)$$

Under the Simple Uniform Hashing assumption, the probability that two keys collide is:

$$Pr(h(x_i) = h(x_j)) = \frac{1}{m} \implies E[X_{ij}] = \frac{1}{m} \quad (4)$$

The expected number of elements examined during a successful search for the i -th inserted key x_i is 1 plus the expected number of keys inserted after x_i (i.e., keys x_j where $j > i$) that map to the same slot:

$$E\left[1 + \sum_{j=i+1}^n X_{ij}\right] = 1 + \sum_{j=i+1}^n E[X_{ij}] = 1 + \sum_{j=i+1}^n \frac{1}{m} = 1 + \frac{n-i}{m} \quad (5)$$

To determine the expected search time across all elements in the hash table, we take the average over all n keys:

$$\begin{aligned} E[T_{\text{successful}}] &= \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{n-i}{m}\right) \\ &= 1 + \frac{1}{nm} \sum_{i=1}^n (n-i) \\ &= 1 + \frac{1}{nm} \left(\sum_{i=1}^n n - \sum_{i=1}^n i\right) \\ &= 1 + \frac{1}{nm} \left(n^2 - \frac{n(n+1)}{2}\right) \\ &= 1 + \frac{1}{nm} \left(\frac{2n^2 - n^2 - n}{2}\right) \\ &= 1 + \frac{1}{nm} \left(\frac{2n^2 - n^2 - n}{2}\right) \\ &= 1 + \frac{1}{nm} \left(\frac{n(n-1)}{2}\right) \\ &= 1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} - \frac{1}{2m} \end{aligned}$$

Accounting for the constant time $\Theta(1)$ needed to compute the hash function and access the base slot array, the total expected time for a successful search is:

$$T_{\text{successful}} = \Theta\left(1 + \left(1 + \frac{\alpha}{2} - \frac{1}{2m}\right)\right) = \Theta(1 + \alpha) \quad (6)$$

4. Summary of Complexity Bounds

Search Operation State	Average-Case Complexity (SUH)	Worst-Case Complexity
Unsuccessful Search	$\Theta(1 + \alpha)$	$\Theta(n)$
Successful Search	$\Theta(1 + \alpha)$	$\Theta(n)$

- **Average-Case Interpretation:** If the number of table slots m is chosen proportional to the number of elements n , then the load factor $\alpha = n/m = \mathcal{O}(1)$. Under these conditions, both successful and unsuccessful searches run in $\Theta(1)$ **expected time**.
- **Worst-Case Interpretation:** In the worst-case scenario, the hash function performs poorly and maps all n keys into the exact same slot. This causes the table to degenerate into a single linear linked list of length n , yielding a $\Theta(n)$ **runtime**.
- **Space Complexity:** $\Theta(m + n)$, as the table requires m slot pointers and n node structures to store the elements.

Q7. You are given the task to implement the hash table data structure for a new programming language. You may use either separate chaining or open addressing. In open addressing, there are various options for probing. Explain your choice precisely with proper justification. **(10 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Answer

1. Design Goals for a Standard Library Hash Table

Implementing a hash table for a new general-purpose programming language demands a design that excels in real-world scenarios. The three primary metrics for evaluating the architecture are:

- Hardware Sympathy (Cache Locality):** Modern CPU performance is heavily bottlenecked by memory access latency. Sequential memory reads are orders of magnitude faster than random pointer dereferences.
- Memory Overhead:** Minimizing the per-element memory footprint (e.g., avoiding excessive pointer allocations).
- Robustness:** Graceful degradation under high collision rates and high load factors (α).

2. Primary Architectural Decision: Open Addressing vs. Separate Chaining

Feature	Separate Chaining	Open Addressing
Cache Locality	Poor. Linked list nodes are scattered across the heap, causing frequent CPU cache misses.	Excellent. All data is stored in a contiguous array, maximizing cache line utilization.
Memory Overhead	High. Requires $\Theta(n)$ extra memory for pointers (e.g., <code>next</code> pointers in nodes).	Low. No pointers are needed; only the array space is utilized.
High Load Factor ($\alpha \geq 1$)	Handles high load factors gracefully. Performance degrades linearly without table resizing.	Fails at $\alpha \geq 1$. Performance degrades catastrophically as $\alpha \rightarrow 1$ due to long probe sequences.
Deletion	Trivial ($\mathcal{O}(1)$ pointer update).	Complex. Requires “Tombstones” (DELETED flags) or backward-shifting elements.

Open Addressing (Contiguous)

k_0	k_1	k_2	k_3	k_4	k_5
-------	-------	-------	-------	-------	-------

One cache line load fetches multiple keys

Separate Chaining (Fragmented)

Head

k_1

k_2

k_3

Pointer chasing causes cache misses

3. Secondary Decision: Probing Strategy in Open Addressing

If Open Addressing is chosen, the probing sequence determines how collisions are resolved.

- Linear Probing** ($P(i) = i$): Checks slots sequentially.
 - Pros:* Best possible cache locality. CPU prefetcher will load upcoming slots automatically.
 - Cons:* Highly susceptible to **Primary Clustering** (blocks of occupied slots merge, causing cascade collisions).
- Quadratic Probing** ($P(i) = c_1i + c_2i^2$): Space between probes increases quadratically.
 - Pros:* Eliminates primary clustering.
 - Cons:* Suffers from **Secondary Clustering** (keys hashing to the same base slot follow the exact same probe sequence).
- Double Hashing** ($P(i) = i \cdot h_2(k)$): Step size is determined by a second hash function.
 - Pros:* Eliminates both primary and secondary clustering. Provides a uniform spread.
 - Cons:* **Terrible cache locality.** It jumps randomly across the array, negating the hardware benefits of open addressing.

4. Final Recommendation and Justification

Choice: I would implement **Open Addressing with Linear Probing**, paired with a **strict dynamic resizing threshold (Load Factor $\alpha \leq 0.6$ or 0.7)**.

Precise Justification:

(a) **The Cache is King:** In modern computer architecture, a cache miss (fetching from main memory) can take 100-300 CPU cycles, whereas reading from the L1 cache takes 1-3 cycles. Separate chaining and double hashing cause frequent cache misses due to pointer chasing and random memory jumping. Linear probing, by contrast, reads contiguous memory, perfectly aligning with CPU caching mechanics.

266

- (b) **Mitigating Primary Clustering:** The theoretical weakness of linear probing (primary clustering) only becomes severely detrimental when the table is overly full ($\alpha > 0.7$). By enforcing an aggressive dynamic resizing strategy (doubling the array size whenever α reaches 0.6), we mathematically guarantee short probe chains, keeping operations strictly $\mathcal{O}(1)$.
- (c) **Memory Efficiency:** We save the 8 bytes (on 64-bit systems) per node that separate chaining would require for pointers. This saved memory allows us to afford a lower load factor (a larger array) while maintaining an equivalent or better overall memory footprint.
- (d) **Modern Precedent:** This approach (or variants like Robin Hood hashing or Swiss Tables which build upon contiguous arrays and linear probing) is the backbone of hash tables in modern, high-performance languages such as Python (`dict`), Rust (`std::collections::HashMap`), and C++ (e.g., Google's `absl::flat_hash_map`).

5. Complexity Analysis (for chosen architecture)

- **Time Complexity:**
 - **Expected / Average:** $\Theta(1)$ for Search, Insert, and Delete (assuming simple uniform hashing and $\alpha \leq 0.6$).
 - **Worst-case:** $\mathcal{O}(n)$ if a pathological hash function maps all keys to the same slot, or during a table resize operation (Amortized $\mathcal{O}(1)$).
- **Space Complexity:** $\Theta(m)$ where m is the capacity of the array. The space is contiguous, with zero overhead for node allocations.

Q8. Suppose you have invited N people to a dinner, and you asked someone to enter their arrival times into a hash table. Assume that exactly one guest can enter the dinner hall at a particular time. Assuming separate chaining, briefly describe how you can achieve $1 + \log \alpha$ expected time for an unsuccessful search, where α is the load factor. **(10 marks)** [CSE 207 | L-2/T-2 | 2019–20]

Answer

1. Core Concept: From Linear Chains to Balanced Trees

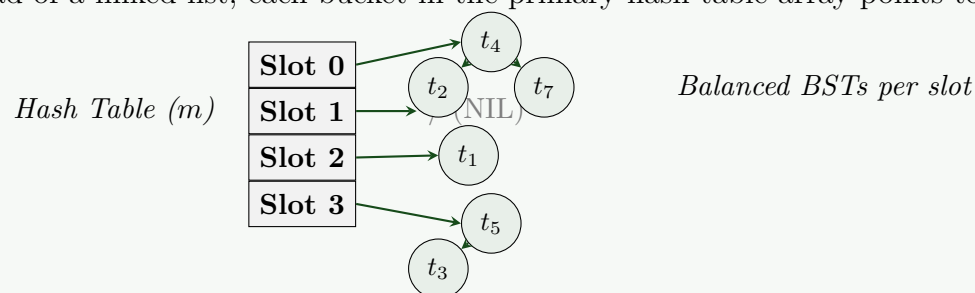
In a standard separate-chaining hash table under the Simple Uniform Hashing Assumption (SUHA), elements colliding at a specific bucket are stored in an unordered linked list. An unsuccessful search must traverse the entire list, resulting in an expected running time of $\Theta(1 + \alpha)$, where $\alpha = N/m$ is the load factor.

To reduce the expected unsuccessful search time to $\Theta(1 + \log \alpha)$, we replace the linked list in each bucket with a **Balanced Binary Search Tree (BBST)**, such as a Red-Black Tree or an AVL Tree.

Since the problem states that exactly one guest can enter the dinner hall at any particular time, all arrival times (keys) are guaranteed to be unique. This eliminates any overhead associated with handling duplicate keys inside the tree structure.

2. Structural Layout

Instead of pointing to the head of a linked list, each bucket in the primary hash table array points to the root of a BBST.



3. Algorithmic Formulation

When searching for an arrival time k , the algorithm performs a primary array lookup followed by a classical binary search down the tree structure.

Algorithm 47 Enhanced Chaining Search Strategy

- 1: **Input:** Hash table T of size m , target arrival time k
 - 2: **Output:** The node containing k if found, otherwise NIL
 - 3: $\text{bucket} \leftarrow h(k)$ ▷ Compute bucket index in $\mathcal{O}(1)$ time
 - 4: $\text{current} \leftarrow T[\text{bucket}].\text{root}$
 - 5: **while** $\text{current} \neq \text{NIL}$ **do**
 - 6: **if** $k == \text{current.key}$ **then**
 - 7: **return** current ▷ Successful search
 - 8: **else**
 - 9: **if** $k < \text{current.key}$ **then**
 - 10: $\text{current} \leftarrow \text{current.left}$
 - 11: **else**
 - 12: $\text{current} \leftarrow \text{current.right}$
 - 13: **end if**
 - 14: **end if**
 - 15: **end while**
 - 16: **return** NIL ▷ Unsuccessful search reached a leaf pointer
-

4. Rigorous Mathematical Analysis

Let n_j represent the random variable indicating the number of keys hashed into bucket j , where $j \in \{0, 1, \dots, m-1\}$.

- (a) **Individual Search Cost:** In a balanced binary search tree containing n_j items, the worst-case depth is bounded tightly by $\Theta(\log n_j)$. Including the base cost to compute the hash function and access the slot index, the search time inside bucket j is:

$$T_{\text{search}}(n_j) \leq c_1 + c_2 \log(n_j + 1) \quad (1)$$

We use $\log(n_j + 1)$ to cleanly account for empty buckets ($n_j = 0$), where $\log(1) = 0$.

- (b) **Expected Value Formulation:** Under the Simple Uniform Hashing Assumption (SUHA), any key is equally likely to hash into any of the m available buckets. The expected number of keys in any given bucket is the load factor:

$$E[n_j] = \alpha = \frac{N}{m} \quad (2)$$

Therefore, the expected time for an unsuccessful search is given by:

$$E[T_{\text{unsuccessful}}] = E[c_1 + c_2 \log(n_j + 1)] = c_1 + c_2 E[\log(n_j + 1)] \quad (3)$$

- (c) **Application of Jensen's Inequality:** The function $f(x) = \log(x + 1)$ is a strictly **concave function** since its second derivative is strictly negative for all $x \geq 0$:

$$f''(x) = -\frac{1}{(x+1)^2} < 0 \quad (4)$$

By **Jensen's Inequality**, for any concave function f and random variable X , the expected value of the function is less than or equal to the function of the expected value ($E[f(X)] \leq f(E[X])$). Applying this principle:

$$E[\log(n_j + 1)] \leq \log(E[n_j] + 1) = \log(\alpha + 1) \quad (5)$$

- (d) **Conclusion:** For non-trivial load factors ($\alpha \geq 1$), $\log(\alpha + 1) = \Theta(\log \alpha)$. Combining this with the baseline array access time yields:

$$E[T_{\text{unsuccessful}}] = \mathcal{O}(1 + \log \alpha) \quad (6)$$

5. Complexity Profiles

- **Time Complexity (Search/Insert/Delete):**

- **Expected Case:** $\Theta(1 + \log \alpha)$ for all operations due to uniform distribution and balanced tree properties.
- **Worst Case:** $\mathcal{O}(1 + \log N)$ in the catastrophic event that all N elements collide into a single bucket, forcing everything into one large BBST. This represents a massive improvement over standard chaining's $\mathcal{O}(N)$ worst-case bottleneck.

- **Space Complexity:** $\Theta(m + N)$. The structure uses m initial bucket pointers and allocates exactly 3 pointers (**left**, **right**, **parent**) and memory per node for the N guests.

Q9. Consider successively inserting the keys 10, 22, 31, 4, 15, 28, 16, 37 and 59 into a hash table of size $m = 11$ with hash function $h(k) = k \bmod m$. Illustrate the resulting hash tables after inserting these keys using (i) linear probing, and (ii) quadratic probing. You do not need to show the intermediate steps, just show the hash table after inserting all the elements. **(15 marks)** [CSE 207 | L-2/T-2 | 2019–20]

Answer

Problem Parameters

- **Keys to insert:** $K = \langle 10, 22, 31, 4, 15, 28, 16, 37, 59 \rangle$
- **Table Size:** $m = 11$
- **Primary Hash Function:** $h(k) = k \bmod 11$

The base hash values for the keys before any collision resolution are:

Key k	10	22	31	4	15	28	16	37	59
$h(k)$	10	0	9	4	4	6	5	4	4

(i) Linear Probing

In linear probing, the collision resolution sequence is given by:

$$h_i(k) = (h(k) + i) \bmod 11$$

where $i = 0, 1, 2, \dots$ represents the probe number.

When inserting the elements successively, collisions are handled by sequentially searching for the next available slot, wrapping around the table if necessary.

Final Hash Table (Linear Probing):

0	1	2	3	4	5	6	7	8	9	10
22	59			4	15	28	16	37	31	10

(ii) Quadratic Probing

In standard quadratic probing (assuming $c_1 = 0, c_2 = 1$), the collision resolution sequence is given by:

$$h_i(k) = (h(k) + i^2) \bmod 11$$

where $i = 0, 1, 2, \dots$ represents the probe number.

- *Note on collision resolution for 16:* $h(16) = 5$ (collision with 15), $i = 1 \Rightarrow 6$ (collision with 28), $i = 2 \Rightarrow (5+4) \bmod 11 = 9$ (collision with 31), $i = 3 \Rightarrow (5+9) \bmod 11 = 3$ (empty slot).
- *Note on collision resolution for 37:* $h(37) = 4$ (collision with 4), $i = 1 \Rightarrow 5$ (collision with 15), $i = 2 \Rightarrow (4+4) \bmod 11 = 8$ (empty slot).
- *Note on collision resolution for 59:* $h(59) = 4$ (collision with 4), $i = 1 \Rightarrow 5$ (collision with 15), $i = 2 \Rightarrow (4+4) \bmod 11 = 8$ (collision with 37), $i = 3 \Rightarrow (4+9) \bmod 11 = 2$ (empty slot).

Final Hash Table (Quadratic Probing):

0	1	2	3	4	5	6	7	8	9	10
22		59	16	4	15	28		37	31	10

Q10. Assuming simple uniform hashing, prove that an unsuccessful search in separate chaining takes expected $O(1 + \alpha)$ work, where α is the load factor. **(12 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Answer

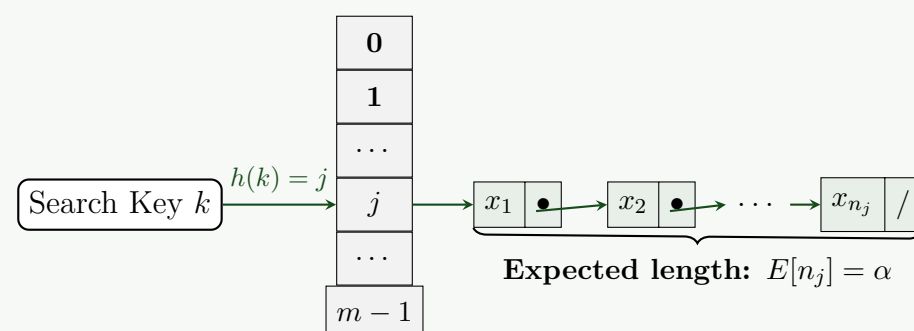
1. Definitions and Assumptions

To formalize the analysis, we first establish the core variables and properties of the hash table:

- Let m be the number of slots (buckets) in the hash table T .
- Let n be the total number of elements currently stored in the hash table.
- Let $\alpha = \frac{n}{m}$ be the **load factor**, which represents the average number of elements per slot.
- Let n_j denote the length of the linked list at slot j , such that $\sum_{j=0}^{m-1} n_j = n$.

Simple Uniform Hashing Assumption (SUHA): We assume that any given key k is equally likely to hash into any of the m available slots, independently of where any other keys have hashed. Mathematically, for any key k and any slot $j \in \{0, 1, \dots, m-1\}$:

$$P(h(k) = j) = \frac{1}{m} \quad (1)$$



2. Proof of Expected Search Time

An **unsuccessful search** occurs when we search for a key k that is *not* present in the hash table. The algorithm performs the following steps:

- Compute the hash value $j = h(k)$ to find the appropriate bucket.

(b) Traverse the entire linked list $T[j]$ until reaching the NIL pointer to confirm k is not there.

To find the expected time of this operation, we must determine the expected length of the linked list $T[h(k)]$. Since the search key k is not in the table, its hash value is completely independent of the keys currently stored in the table.

Let us define an indicator random variable X_i for each element i currently in the table (where $1 \leq i \leq n$):

$$X_i = I\{\text{element } i \text{ hashes to slot } j\} = \begin{cases} 1 & \text{if } h(i) = j \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

By the Simple Uniform Hashing Assumption, the probability that any element i hashes to slot j is $\frac{1}{m}$. Therefore, the expected value of X_i is:

$$E[X_i] = 1 \cdot P(h(i) = j) + 0 \cdot P(h(i) \neq j) = \frac{1}{m} \quad (3)$$

The length of the linked list at slot j , denoted n_j , is simply the sum of all elements that mapped to slot j :

$$n_j = \sum_{i=1}^n X_i \quad (4)$$

To find the expected length of the list, we take the expectation of n_j . By using the **Linearity of Expectation**, we can pull the summation outside the expected value:

$$E[n_j] = E\left[\sum_{i=1}^n X_i\right] \quad (5)$$

$$= \sum_{i=1}^n E[X_i] \quad (6)$$

$$= \sum_{i=1}^n \frac{1}{m} \quad (7)$$

$$= n \cdot \frac{1}{m} = \frac{n}{m} = \alpha \quad (8)$$

3. Conclusion and Complexity Analysis

The expected number of elements examined during an unsuccessful search is exactly α . The total expected running time comprises two components:

- **Array Access and Hash Computation:** Calculating $h(k)$ and accessing $T[h(k)]$ takes $\mathcal{O}(1)$ deterministic time.
- **List Traversal:** Scanning the elements in the linked list takes time proportional to its length. The expected time spent scanning is $\mathcal{O}(E[n_{h(k)}]) = \mathcal{O}(\alpha)$.

Summing these components, the expected total running time for an unsuccessful search is:

$$T_{\text{unsuccessful}} = \mathcal{O}(1) + \mathcal{O}(\alpha) = \mathcal{O}(1 + \alpha) \quad (9)$$

Note: We write $\mathcal{O}(1 + \alpha)$ rather than $\mathcal{O}(\alpha)$ because even if the table is completely empty ($\alpha = 0$), the algorithm still requires $\mathcal{O}(1)$ time to compute the hash and check the initially empty slot. ■

Q11. Quadratic probing does not necessarily probe all locations in hash tables. Do you agree with this statement? Justify your answer. [CSE 207 | L-2/T-2 | 2018–19, 2017–18]

(6 marks)

Answer

Yes, I strongly agree with this statement.

Unlike linear probing, which is guaranteed to examine every slot in the hash table as long as the table is not completely full, quadratic probing restricts its probe sequence to a specific mathematical subset of the table indices. Depending on the table size and the chosen probing constants, quadratic probing will often fail to probe all locations, even if the table has empty slots available.

1. Mathematical Formulation

In quadratic probing, the hash function resolves collisions using the following formula:

$$h_i(k) = (h'(k) + c_1i + c_2i^2) \bmod m \quad (1)$$

where:

- $h'(k)$ is the primary hash function.
- m is the size of the hash table.
- $c_1, c_2 \neq 0$ are auxiliary constants.
- $i = 0, 1, 2, \dots, m - 1$ is the probe number.

For the probe sequence to examine every location in the hash table, the set of generated indices $\{h_0(k), h_1(k), \dots, h_{m-1}(k)\}$ must be a **permutation** of $\{0, 1, \dots, m-1\}$. In general, quadratic equations modulo m do not generate full permutations.

2. Proof by Counterexample

Consider the simplest and most common form of quadratic probing where $c_1 = 0$ and $c_2 = 1$. The probe sequence simplifies to:

$$h_i(k) = (h'(k) + i^2) \bmod m \quad (2)$$

Let the table size be $m = 7$, and suppose a key k initially hashes to $h'(k) = 0$. Let us trace the first 7 probes (from $i = 0$ to $i = 6$):

Probe (i)	Calculation: $(0 + i^2) \bmod 7$	Slot Probed
0	$0^2 \bmod 7 = 0 \bmod 7$	0
1	$1^2 \bmod 7 = 1 \bmod 7$	1
2	$2^2 \bmod 7 = 4 \bmod 7$	4
3	$3^2 \bmod 7 = 9 \bmod 7$	2
4	$4^2 \bmod 7 = 16 \bmod 7$	2
5	$5^2 \bmod 7 = 25 \bmod 7$	4
6	$6^2 \bmod 7 = 36 \bmod 7$	1

Notice that after $i = 3$, the sequence of probed slots perfectly mirrors itself and repeats.

0	1	2	3	4	5	6
Probe	Probe	Probe	Misse	Probe	Misse	Missed
$i = 0$	$i = 1, 6$	$i = 3, 4$		$i = 2, 5$		

In this scenario, the distinct slots examined are only $\{0, 1, 2, 4\}$. The algorithm will **never** probe slots $\{3, 5, 6\}$. If slots 0, 1, 2, and 4 are already occupied by previously inserted keys, the insertion of key k will fail—even though the hash table is less than 60% full ($\alpha = 4/7$).

3. Theoretical Constraints for Full Probing

Because of this inherent mathematical limitation (which relates to the number of *quadratic residues* modulo m), quadratic probing requires strictly engineered parameters to guarantee a full table traversal.

For instance, quadratic probing *will* probe all m slots if and only if special conditions are met, such as:

- (a) m is a power of 2 (i.e., $m = 2^p$).
- (b) The constants are exactly $c_1 = \frac{1}{2}$ and $c_2 = \frac{1}{2}$.

Under these specific parameters, the sequence generates a triangular number sequence $i(i+1)/2$, which forms a full permutation modulo 2^p .

However, since this requires explicit mathematical configuration and is not universally true for arbitrary table sizes (especially primes, which are commonly chosen for hash tables), the statement that "quadratic probing does not necessarily probe all locations" is entirely correct.

Q12. Consider successively inserting the keys 14, 3, 29, 3, 19, 16, 35, and 2 into a hash table of size $m = 11$ with hash function $h(k) = k \bmod m$. Illustrate the resulting hash table after inserting these keys using quadratic probing. You do not need to show the intermediate steps, just show the hash table after inserting all the elements.

(12 marks)

[CSE 207 | L-2/T-2 | 2018–19, 2016–17]

Answer

Problem Parameters

- **Keys to insert:** $K = \langle 14, 3, 29, 3, 19, 16, 35, 2 \rangle$
- **Table Size:** $m = 11$
- **Primary Hash Function:** $h(k) = k \bmod 11$
- **Probing Strategy:** Standard quadratic probing, defined by the sequence:

$$h_i(k) = (h(k) + i^2) \bmod m \quad \text{for } i = 0, 1, 2, \dots \quad (1)$$

- *Note on duplicate keys:* The key **3** appears twice in the input sequence. In a standard algorithmic insertion trace (representing a multiset or collision resolution exercise), the second instance of 3 is treated as a distinct element that collides with the first and must find the next available empty slot.

Final Hash Table

Below is the resulting hash table after all keys have been inserted.

0	1	2	3	4	5	6	7	8	9	10
	3	35	14	3	16	2	29	19		

Optional Trace Justification (For Clarity)

Although not strictly required, the following table summarizes the deterministic sequence of probes for each key to verify the final state.

Key k	$h(k)$	Probe Sequence: $h_i(k) = (h(k) + i^2) \bmod 11$	Final Index
14	3	$i = 0 \Rightarrow 3$ (Empty)	3
3	3	$i = 0 \Rightarrow 3$ (Collision), $i = 1 \Rightarrow (3 + 1) \bmod 11 = 4$ (Empty)	4
29	7	$i = 0 \Rightarrow 7$ (Empty)	7
3	3	$i = 0 \Rightarrow 3$ (Col), $i = 1 \Rightarrow 4$ (Col), $i = 2 \Rightarrow (3 + 4) \bmod 11 = 7$ (Col) $i = 3 \Rightarrow (3 + 9) \bmod 11 = 1$ (Empty)	1
19	8	$i = 0 \Rightarrow 8$ (Empty)	8
16	5	$i = 0 \Rightarrow 5$ (Empty)	5
35	2	$i = 0 \Rightarrow 2$ (Empty)	2
2	2	$i = 0 \Rightarrow 2$ (Col), $i = 1 \Rightarrow (2 + 1) \bmod 11 = 3$ (Col) $i = 2 \Rightarrow (2 + 4) \bmod 11 = 6$ (Empty)	6

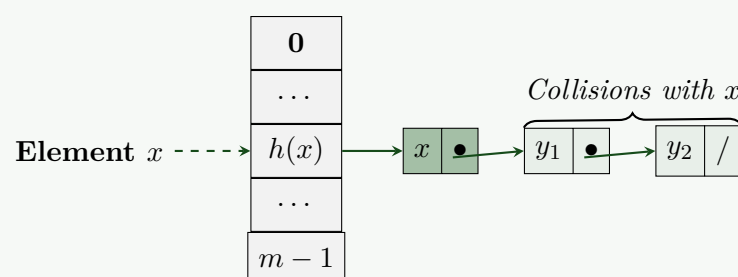
Q13. Under simple uniform hashing, what is the expected number of items that may collide with an element x in a hash table with separate chaining, given m chains and n items? **(10 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Answer**1. Definitions and the Simple Uniform Hashing Assumption**

To determine the expected number of items that collide with an element x , we must formally define the parameters and assumptions of the hash table:

- Let m be the number of slots (chains) in the hash table.
- Let n be the total number of items stored in the hash table.
- Let $\alpha = \frac{n}{m}$ be the **load factor**, representing the average number of elements per chain.
- Simple Uniform Hashing Assumption (SUHA):** Any given element is equally likely to hash into any of the m slots, independently of where any other element has hashed. That is, for any item y and any slot $j \in \{0, 1, \dots, m-1\}$, $P(h(y) = j) = \frac{1}{m}$.

A collision with an element x occurs when another distinct element y hashes to the same slot as x , i.e., $h(y) = h(x)$. The exact expected value depends on a subtle distinction: whether x is an item *already present* in the hash table, or a *new* item being queried/inserted. We will analyze both cases for completeness.

**2. Case 1: x is already one of the n items in the hash table**

If x is already in the table, there are $n - 1$ *other* items currently stored in the data structure. We want to find the expected number of these $n - 1$ items that map to the slot $h(x)$.

Let S be the set of n items in the table. We define an indicator random variable I_y for each item $y \in S \setminus \{x\}$:

$$I_y = \begin{cases} 1 & \text{if } h(y) = h(x) \text{ (a collision occurs)} \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

By SUHA, the probability that y hashes to the specific slot $h(x)$ is $\frac{1}{m}$. Therefore, the expected value of the indicator variable is:

$$E[I_y] = 1 \cdot P(h(y) = h(x)) + 0 \cdot P(h(y) \neq h(x)) = \frac{1}{m} \quad (2)$$

Let C_x be the random variable representing the total number of collisions with x . This is the sum of the indicator variables:

$$C_x = \sum_{y \in S \setminus \{x\}} I_y \quad (3)$$

Using the **Linearity of Expectation**, we compute the expected number of collisions:

$$E[C_x] = E \left[\sum_{y \in S \setminus \{x\}} I_y \right] \quad (4)$$

$$= \sum_{y \in S \setminus \{x\}} E[I_y] \quad (5)$$

$$= \sum_{y \in S \setminus \{x\}} \frac{1}{m} \quad (6)$$

$$= \frac{n-1}{m} = \alpha - \frac{1}{m} \quad (7)$$

Thus, if x is an element already in the table, the expected number of collisions is exactly $\frac{n-1}{m}$.

3. Case 2: x is a new element (Unsuccessful Search / New Insertion)

If x is not yet in the hash table, and we are evaluating the collisions it will encounter upon insertion or an unsuccessful search, all n items currently in the table are potential collisions.

Using the exact same indicator random variable formulation, the summation now iterates over all n elements in the table:

$$E[C_x] = \sum_{i=1}^n E[I_i] \quad (8)$$

$$= \sum_{i=1}^n \frac{1}{m} \quad (9)$$

$$= \frac{n}{m} = \alpha \quad (10)$$

4. Conclusion

- If x is an existing element in the table, the expected number of colliding items is $\frac{n-1}{m}$ (or $\alpha - \frac{1}{m}$).
- If x is an external element being hashed against the table, the expected number of colliding items is $\frac{n}{m}$ (or α).

In standard university contexts (like the analysis of unsuccessful and successful searches in Cormen et al., *Introduction to Algorithms*), asking for collisions with a specific element x *already inside the table* yields the answer $\frac{n-1}{m}$.

Q14. Prove that the first $\lceil m/2 \rceil$ probes in quadratic probing are distinct given m is a prime number. What is the major advantage of double hashing over quadratic probing? **(8+3 marks)** [CSE 207 | L-2/T-2 | 2016–17]

Answer

Part 1: Proof of Distinct Probes in Quadratic Probing (8 Marks)

Claim: For a hash table of size m where m is a prime number, the first $\lceil m/2 \rceil$ probes in the standard quadratic probing sequence are strictly distinct.

Proof: Let the table size m be an odd prime number. (The case $m = 2$ is trivial, as $\lceil 2/2 \rceil = 1$, and a single probe is always distinct).

The standard quadratic probing sequence for a key k is given by:

$$h(k, i) = (h'(k) + i^2) \bmod m \quad (1)$$

where $h'(k)$ is the primary hash value and i is the probe number.

The first $\lceil m/2 \rceil$ probes correspond to the probe indices:

$$i \in \left\{ 0, 1, 2, \dots, \frac{m-1}{2} \right\} \quad (2)$$

Notice that the number of elements in this set is exactly $\frac{m-1}{2} + 1 = \frac{m+1}{2} = \lceil m/2 \rceil$.

We will prove this by contradiction. Assume that there exist two distinct probes in this sequence, i and j , that map to the same slot in the hash table. Let $0 \leq i < j \leq \frac{m-1}{2}$.

If they map to the same slot, their hash values modulo m must be equal:

$$h(k, i) \equiv h(k, j) \pmod{m} \quad (3)$$

$$h'(k) + i^2 \equiv h'(k) + j^2 \pmod{m} \quad (4)$$

Subtracting $h'(k)$ from both sides gives:

$$i^2 \equiv j^2 \pmod{m} \quad (5)$$

$$j^2 - i^2 \equiv 0 \pmod{m} \quad (6)$$

Using the difference of squares, we can factor the expression:

$$(j - i)(j + i) \equiv 0 \pmod{m} \quad (7)$$

This implies that m must divide the product $(j - i)(j + i)$. Because m is a prime number, Euclid's Lemma dictates that m must divide at least one of the factors. Thus, either:

$$m \mid (j - i) \quad \text{or} \quad m \mid (j + i) \quad (8)$$

Let us analyze the bounds of both factors, given $0 \leq i < j \leq \frac{m-1}{2}$:

(a) **For $(j - i)$:** Since $j > i$, the difference must be strictly greater than 0. The maximum possible difference is $\frac{m-1}{2} - 0 = \frac{m-1}{2}$. Thus:

$$0 < j - i \leq \frac{m-1}{2} < m \quad (9)$$

Since $j - i$ is strictly between 0 and m , it is impossible for m to divide $j - i$.

(b) **For $(j + i)$:** The sum must be strictly greater than 0. The maximum possible sum occurs when j takes its maximum value $(\frac{m-1}{2})$ and i takes its maximum value $(\frac{m-1}{2} - 1)$. Thus:

$$0 < j + i \leq \frac{m-1}{2} + \frac{m-3}{2} = \frac{2m-4}{2} = m-2 < m \quad (10)$$

Since $j + i$ is strictly between 0 and m , it is impossible for m to divide $j + i$.

Since m cannot divide either factor, we have reached a **contradiction**. Therefore, our initial assumption must be false. The first $\lceil m/2 \rceil$ probes must all map to distinct locations in the hash table. ■

Part 2: Major Advantage of Double Hashing (3 Marks)

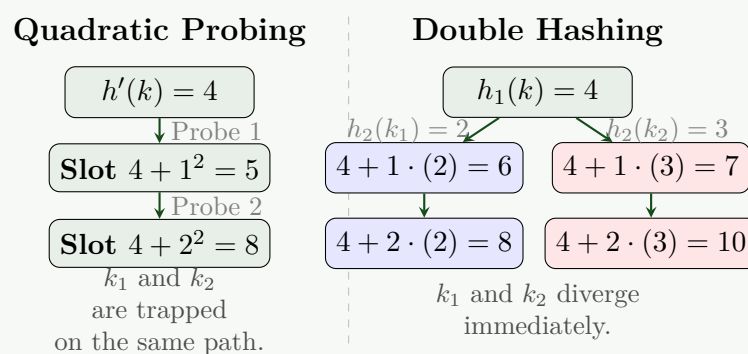
The major advantage of double hashing over quadratic probing is the **elimination of secondary clustering**.

- **The Flaw of Quadratic Probing (Secondary Clustering):** In quadratic probing, the probe sequence depends entirely on the initial hash value. If two different keys, k_1 and k_2 , hash to the same initial slot ($h'(k_1) = h'(k_2)$), they will follow the *exact same* probe sequence. This creates clusters along specific probe paths, degrading performance at higher load factors.
- **The Double Hashing Solution:** Double hashing utilizes a secondary hash function to determine the step size between probes:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \pmod{m} \quad (11)$$

Even if two keys hash to the same initial slot ($h_1(k_1) = h_1(k_2)$), it is highly improbable that their secondary hash values will also match ($h_2(k_1) \neq h_2(k_2)$). Consequently, their probe sequences will immediately diverge.

Double hashing generates $\mathcal{O}(m^2)$ distinct probe sequences, making it theoretically much closer to the ideal of Simple Uniform Hashing, whereas quadratic probing only provides $\mathcal{O}(m)$ distinct sequences.



Q15. Explain 'Chaining' and 'Open Addressing' methods for solving the problem of collisions in a hash table. Explain the three commonly used techniques to compute the probe sequences required for open addressing. (15 marks) [CSE 203 | L-2/T-1 | 2015–16]

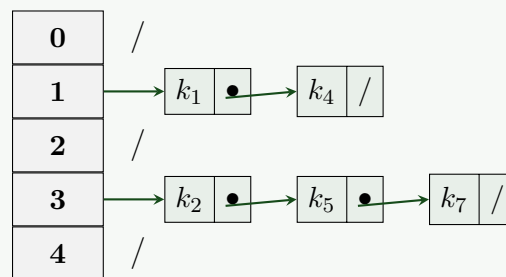
Answer

A hash collision occurs when two distinct keys, k_1 and k_2 , hash to the same slot in a hash table (i.e., $h(k_1) = h(k_2)$). To maintain the integrity of the dictionary operations, we must resolve these collisions. The two primary paradigms for collision resolution are **Chaining** and **Open Addressing**.

1. Chaining (Separate Chaining)

In chaining, the hash table $T[0 \dots m-1]$ acts as an array of pointers. Elements that hash to the same slot are stored in a secondary data structure, typically a linked list.

- **Insertion:** A new element x is inserted at the head of the list $T[h(x.key)]$. This takes $\mathcal{O}(1)$ time if we assume elements are distinct or if we do not check for duplicates.
- **Search:** To find an element with key k , we compute $h(k)$ and perform a linear search through the list $T[h(k)]$. The expected search time is proportional to the load factor $\alpha = n/m$.
- **Deletion:** An element is removed from the linked list $T[h(x.key)]$. Using doubly linked lists, this takes $\mathcal{O}(1)$ time.



2. Open Addressing

In open addressing, all elements are stored *directly* within the hash table itself; no external linked lists or pointers are used. When a collision occurs, the algorithm systematically **probes** (examines) other slots in the hash table until an empty slot is found.

To define this sequence of probes, the hash function is extended to include the probe number as a second argument:

$$h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\} \quad (1)$$

For a given key k , the probe sequence $\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$ must be a permutation of $\langle 0, 1, \dots, m-1 \rangle$ to ensure every slot can eventually be checked. The load factor α can never exceed 1.

3. Probe Sequence Techniques

There are three commonly used techniques to compute the probe sequence in open addressing.

(i) Linear Probing

Given an ordinary hash function $h'(k)$, linear probing resolves collisions by sequentially searching the next available slot, wrapping around the table if necessary.

$$h(k, i) = (h'(k) + i) \bmod m \quad (2)$$

- **Advantage:** Extremely simple to implement and provides excellent cache locality, making it fast in practice for low load factors.
- **Disadvantage:** Suffers from **primary clustering**. Long runs of occupied slots tend to build up, increasing the expected search time. If a slot is preceded by i full slots, the probability of it being filled next is $(i+1)/m$.

(ii) Quadratic Probing

Quadratic probing uses a quadratic function of the probe number i to determine the interval between probes.

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m \quad (3)$$

where c_1 and $c_2 \neq 0$ are auxiliary constants.

- **Advantage:** Eliminates primary clustering by ensuring the probe sequence skips ahead at quadratically increasing intervals.
- **Disadvantage:** Suffers from **secondary clustering**. If two keys map to the exact same initial slot ($h'(k_1) = h'(k_2)$), they will follow the exact same probe sequence. Additionally, c_1, c_2 , and m must be carefully constrained to ensure the sequence probes the entire table.

(iii) Double Hashing

Double hashing represents the most robust open addressing technique. It uses two independent hash functions, $h_1(k)$ and $h_2(k)$. The second hash function determines the *step size* for the probe sequence.

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m \quad (4)$$

- **Advantage:** Eliminates both primary and secondary clustering. Even if two keys hash to the same initial slot ($h_1(k_1) = h_1(k_2)$), they will almost certainly have a different step size ($h_2(k_1) \neq h_2(k_2)$), causing their probe sequences to diverge immediately. It produces $\mathcal{O}(m^2)$ distinct probe sequences.
- **Requirement:** The step size $h_2(k)$ must be relatively prime to the table size m for all keys. This guarantees that the probe sequence will visit every single slot in the table before returning to the original slot. This is typically achieved by making m a prime number and designing $h_2(k)$ to return a strictly positive integer smaller than m .

Q16. What makes a good hash function? Explain the three commonly used techniques to compute the probe sequences required for open addressing in a hash table. (15 marks) [CSE 203 | L-2/T-1 | 2014–15, 2011–12]

Answer

1. Characteristics of a Good Hash Function

A hash function $h(k)$ maps a universe of keys U to a finite set of array indices $\{0, 1, \dots, m-1\}$. A high-quality hash function must satisfy several theoretical and practical properties to ensure efficient dictionary operations:

- **Simple Uniform Hashing (Uniform Distribution):** This is the most fundamental requirement. A good hash function should distribute keys uniformly across all m slots of the hash table. Every key should have an equal probability of hashing to any given slot, independent of where other keys map. Mathematically, for any slot j :

$$P(h(k) = j) = \frac{1}{m} \quad (1)$$

- **Determinism:** The function must be completely deterministic. A specific key k must consistently map to the exact same hash value $h(k)$ every time it is evaluated.
- **Computational Efficiency:** The hash function must be fast to compute, operating in $\mathcal{O}(1)$ time. If the function is computationally expensive, it negates the primary advantage of using a hash table.
- **Full Key Utilization:** The resulting hash value should depend on *all* parts of the key (e.g., all characters in a string), rather than just a prefix or suffix. This guarantees that closely related keys (e.g., "pt1" and "pt2") hash to vastly different slots.
- **Collision Minimization:** While collisions are mathematically inevitable (due to the Pigeonhole Principle), a good function avoids patterns that map frequently occurring clusters of keys (in the input distribution) to the same slot.

2. Probe Sequence Techniques in Open Addressing

In open addressing, all elements are stored directly inside the hash table. When a collision occurs (i.e., the target slot is already occupied), the algorithm systematically "probes" (examines) alternative slots until an empty one is found.

To achieve this, the hash function is extended to include the **probe number** i as a second parameter:

$$h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\} \quad (2)$$

For a given key k , the probe sequence $\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$ must form a complete permutation of $\{0, 1, \dots, m-1\}$ so that every slot can eventually be checked.

The three commonly used techniques to compute these sequences are:

(i) Linear Probing

Given an ordinary (primary) hash function $h'(k)$, linear probing resolves collisions by sequentially checking the immediate next slot, wrapping around the end of the table if necessary.

$$h(k, i) = (h'(k) + i) \bmod m \quad (3)$$

- **Mechanism:** If slot $h'(k)$ is full, it checks $h'(k) + 1$, then $h'(k) + 2$, and so on.
- **Advantage:** Very simple to implement and exhibits excellent cache locality, making it fast in practice when the table is relatively empty.
- **Disadvantage:** Suffers heavily from **primary clustering**. Contiguous blocks of occupied slots build up, and any key that hashes into a cluster will sequentially extend the cluster, degrading average search/insertion time.

(ii) Quadratic Probing

Quadratic probing attempts to eliminate primary clustering by using a quadratic polynomial of the probe number i to define the step size.

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m \quad (4)$$

where c_1 and $c_2 \neq 0$ are auxiliary constants.

- **Mechanism:** The gap between consecutive probes increases quadratically (e.g., checking indices at offsets +1, +4, +9, +16...).
- **Advantage:** Successfully resolves primary clustering because the sequence skips over contiguous occupied blocks.
- **Disadvantage:** Suffers from **secondary clustering**. If two distinct keys map to the same initial slot ($h'(k_1) = h'(k_2)$), they will traverse the exact same probe sequence. Furthermore, specific parameters (like m being prime or a power of 2, and precise values for c_1, c_2) are required to ensure the sequence probes the entire table.

(iii) Double Hashing

Double hashing is the most robust and theoretically sound open addressing technique. It uses *two* independent hash functions, $h_1(k)$ and $h_2(k)$. The second function dictates the specific step size between probes for that key.

$$h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m \tag{5}$$

- **Mechanism:** The first probe is at $h_1(k)$. Subsequent probes are spaced apart by an interval of exactly $h_2(k)$.
- **Advantage:** Eliminates **both primary and secondary clustering**. Even if two keys hash to the same initial slot ($h_1(k_1) = h_1(k_2)$), it is highly improbable that they will have the exact same step size ($h_2(k_1) \neq h_2(k_2)$). Their probe sequences diverge immediately. It produces $\Theta(m^2)$ distinct probe sequences, closely approximating ideal simple uniform hashing.
- **Requirement:** To ensure the entire table is searched, the step size $h_2(k)$ **must be relatively prime** to the table size m . A common approach is to set m as a prime number and use $h_2(k) = 1 + (k \bmod m')$, where $m' < m$.

Q17. Draw the 11-item hash table that results from using the hash function $h(i) = (2i + 5) \bmod 11$ to hash the keys 12, 44, 13, 88, 23, 94, 11, 39, 20, 16, 5, assuming collisions are handled by double hashing using a secondary hash function $h'(k) = 7 - (k \bmod 7)$. **(12 marks)** [CSE 203 | L-2/T-1 | 2013–14]

Answer

1. Hash Functions and Parameters

We are given a hash table of size $m = 11$ and the following functions for double hashing:

- **Primary Hash Function:** $h(k) = (2k + 5) \bmod 11$
- **Secondary Hash Function:** $h'(k) = 7 - (k \bmod 7)$
- **Double Hashing Probe Sequence:** $H(k,i) = (h(k) + i \cdot h'(k)) \bmod 11$ for $i = 0, 1, 2, \dots$

2. Step-by-Step Insertion Trace

We insert the sequence of keys $K = \langle 12, 44, 13, 88, 23, 94, 11, 39, 20, 16, 5 \rangle$ one by one. The table below traces the calculations for each key.

Key (k)	$h(k)$	$h'(k)$	Probe Sequence: $H(k,i) = (h(k) + i \cdot h'(k)) \bmod 11$	Final Slot
12	7	–	$i = 0 \implies 7$ (Empty)	7
44	5	–	$i = 0 \implies 5$ (Empty)	5
13	9	–	$i = 0 \implies 9$ (Empty)	9
88	5	3	$i = 0 \implies 5$ (Col), $i = 1 \implies (5 + 3) \bmod 11 = 8$ (Empty)	8
23	7	5	$i = 0 \implies 7$ (Col), $i = 1 \implies (7 + 5) \bmod 11 = 1$ (Empty)	1
94	6	–	$i = 0 \implies 6$ (Empty)	6
11	5	3	$i = 0 \implies 5$ (Col), $i = 1 \implies 8$ (Col), $i = 2 \implies (5 + 6) \bmod 11 = 0$	0
39	6	3	$i = 0 \implies 6$ (Col), $i = 1 \implies 9$ (Col), $i = 2 \implies (6 + 6) \bmod 11 = 1$ (Col) $i = 3 \implies (6 + 9) \bmod 11 = 4$ (Empty)	4
20	1	1	$i = 0 \implies 1$ (Col), $i = 1 \implies (1 + 1) \bmod 11 = 2$ (Empty)	2
16	4	5	$i = 0 \implies 4$ (Col), $i = 1 \implies (4 + 5) \bmod 11 = 9$ (Col) $i = 2 \implies (4 + 10) \bmod 11 = 3$ (Empty)	3
5	4	2	$i = 0 \implies 4$ (Col), $i = 1 \implies 6$ (Col), $i = 2 \implies 8$ (Col) $i = 3 \implies (4 + 6) \bmod 11 = 10$ (Empty)	10

3. Final Hash Table

After inserting all 11 keys, the hash table is completely full. The resulting array is visualized below:

0	1	2	3	4	5	6	7	8	9	10
11	23	20	16	39	44	94	12	88	13	5

Q18. Write pseudocode for finding a key, inserting a key, and deleting a key in a hash table using quadratic probing. Explain your method with an example. **(12 marks)** [CSE 203 | L-2/T-1 | 2012–13, 2011–12]

Answer

1. Introduction and The Necessity of Tombstones

In open addressing with quadratic probing, collisions are resolved by probing the sequence:

$$h(k, i) = (h'(k) + c_1i + c_2i^2) \bmod m \quad (1)$$

For simplicity, we will use the standard form where $c_1 = 0$ and $c_2 = 1$, giving $h(k, i) = (h'(k) + i^2) \bmod m$.

Handling Deletions: We cannot simply mark a deleted slot as NULL (empty). If we did, a search for an element that previously collided and was placed after this slot would prematurely terminate when it encounters the NULL. To solve this, we introduce a special sentinel value called a **Tombstone** (e.g., DELETED).

- **Search:** Skips over DELETED slots and continues probing.
- **Insert:** Treats DELETED slots as available space and can overwrite them.

2. Pseudocode

Let T be the hash table of size m . Each slot can be in one of three states: NULL (never occupied), DELETED (tombstone), or containing a valid key.

Algorithm 48 Hash-Search(T, k)

```

1: Input: Hash table  $T$ , key  $k$  to find
2: Output: Index of  $k$  if found, else NIL
3:  $i \leftarrow 0$ 
4: while  $i < m$  do
5:    $j \leftarrow (h'(k) + i^2) \bmod m$ 
6:   if  $T[j] == \text{NULL}$  then
7:     return NIL
8:   end if
9:   if  $T[j] \neq \text{DELETED}$  and  $T[j] == k$  then
10:    return  $j$ 
11:  end if
12:   $i \leftarrow i + 1$ 
13: end while
14: return NIL

```

▷ Hit an empty slot, key is not in table
 ▷ Key found
 ▷ Table exhausted

Algorithm 49 Hash-Insert(T, k)

```

1: Input: Hash table  $T$ , key  $k$  to insert
2: Output: Index where  $k$  is inserted, or error if table is full
3:  $i \leftarrow 0$ 
4: while  $i < m$  do
5:    $j \leftarrow (h'(k) + i^2) \bmod m$ 
6:   if  $T[j] == \text{NULL}$  or  $T[j] == \text{DELETED}$  then
7:      $T[j] \leftarrow k$ 
8:     return  $j$ 
9:   end if
10:  if  $T[j] == k$  then
11:    return ERROR: Key already exists
12:  end if
13:   $i \leftarrow i + 1$ 
14: end while
15: return ERROR: Hash table overflow

```

Algorithm 50 Hash-Delete(T, k)

```

1: Input: Hash table  $T$ , key  $k$  to delete
2: Output: TRUE if successfully deleted, FALSE otherwise
3:  $j \leftarrow \text{Hash-Search}(T, k)$ 
4: if  $j \neq \text{NIL}$  then
5:    $T[j] \leftarrow \text{DELETED}$ 
6:   return TRUE
7: end if
8: return FALSE

```

▷ Place tombstone
 ▷ Key not found

3. Explanatory Example

Consider a hash table of size $m = 7$ and a primary hash function $h'(k) = k \bmod 7$. We trace the following operations:

1. **Insert(17):** $h'(17) = 3$.

- $i = 0$: Probe $(3 + 0^2) \bmod 7 = 3$. Slot 3 is NULL. Insert 17 at index 3.

2. **Insert(24):** $h'(24) = 3$.

- $i = 0$: Probe $(3 + 0^2) \bmod 7 = 3$. Slot 3 is occupied (17). Collision!
- $i = 1$: Probe $(3 + 1^2) \bmod 7 = 4$. Slot 4 is NULL. Insert 24 at index 4.

3. **Insert(10):** $h'(10) = 3$.

- $i = 0$: Probe $(3 + 0^2) \bmod 7 = 3$. Slot 3 is occupied (17). Collision!
- $i = 1$: Probe $(3 + 1^2) \bmod 7 = 4$. Slot 4 is occupied (24). Collision!
- $i = 2$: Probe $(3 + 2^2) \bmod 7 = 7 \bmod 7 = 0$. Slot 0 is NULL. Insert 10 at index 0.

4. **Delete(24):**

- Hash-Search(24) probes $i = 0$ (Slot 3) and $i = 1$ (Slot 4), finds 24 at index 4.
- Slot 4 is overwritten with the DELETED tombstone.

5. **Find(10):** (Demonstrating why the tombstone is necessary)

- $h'(10) = 3$.
- $i = 0$: Probe 3. Contains 17 $\neq 10$.
- $i = 1$: Probe 4. Contains DELETED. The algorithm *does not stop* here.
- $i = 2$: Probe 0. Contains 10. Key found! (If slot 4 were NULL, the search would have falsely returned NIL at $i = 1$).

0	1	2	3	4	5	6
10	NULL	NULL	17	DEL	NULL	NULL

4. Complexity Analysis

- **Time Complexity:** Under the assumption of uniform hashing, operations operate in $\mathcal{O}(1)$ average expected time. In the worst case (e.g., table approaches capacity, or high collision rates leading to long probe sequences), the time complexity degenerates to $\mathcal{O}(m)$.
- **Space Complexity:** $\mathcal{O}(m)$, since all elements and tombstones are stored directly inside the array of size m , without external data structures.

Q19. Insert the keys 18, 41, 22, 44, 59, 32, 31, 73 into a hash table of size 13 using double hashing with

$$h_1(k) = k \bmod 13, \quad h_2(k) = 8 - (k \bmod 8).$$

Draw the resulting hash table. **(13 marks)**

[CSE 203 | L-2/T-1 | 2012–13]

Answer

1. Hash Functions and Setup

We are inserting keys into a hash table of size $m = 13$ using double hashing. The probe sequence is defined as:

$$H(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m \quad (1)$$

where:

- $h_1(k) = k \bmod 13$ (Primary Hash Function)
- $h_2(k) = 8 - (k \bmod 8)$ (Secondary Hash Function/Step Size)
- $i = 0, 1, 2, \dots$ (Probe Number)

2. Step-by-Step Insertion Trace

We process the keys in the order given: $\langle 18, 41, 22, 44, 59, 32, 31, 73 \rangle$.

Key (k)	$h_1(k)$	$h_2(k)$	Probe Sequence: $H(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod 13$	Final Slot
18	5	6	$i = 0 \implies 5 \bmod 13 = 5$ (Empty)	5
41	2	7	$i = 0 \implies 2 \bmod 13 = 2$ (Empty)	2
22	9	2	$i = 0 \implies 9 \bmod 13 = 9$ (Empty)	9
44	5	4	$i = 0 \implies 5$ (Collision with 18) $i = 1 \implies (5 + 1 \cdot 4) \bmod 13 = 9$ (Collision with 22) $i = 2 \implies (5 + 2 \cdot 4) \bmod 13 = 13 \bmod 13 = 0$ (Empty)	0
59	7	5	$i = 0 \implies 7 \bmod 13 = 7$ (Empty)	7
32	6	8	$i = 0 \implies 6 \bmod 13 = 6$ (Empty)	6
31	5	1	$i = 0 \implies 5$ (Collision with 18) $i = 1 \implies (5 + 1 \cdot 1) \bmod 13 = 6$ (Collision with 32) $i = 2 \implies (5 + 2 \cdot 1) \bmod 13 = 7$ (Collision with 59) $i = 3 \implies (5 + 3 \cdot 1) \bmod 13 = 8$ (Empty)	8
73	8	7	$i = 0 \implies 8$ (Collision with 31) $i = 1 \implies (8 + 1 \cdot 7) \bmod 13 = 15 \bmod 13 = 2$ (Collision with 41) $i = 2 \implies (8 + 2 \cdot 7) \bmod 13 = 22 \bmod 13 = 9$ (Collision with 22) $i = 3 \implies (8 + 3 \cdot 7) \bmod 13 = 29 \bmod 13 = 3$ (Empty)	3

3. Final Hash Table

Below is the visualization of the resulting 13-slot hash table after all insertions are complete. Empty slots are denoted by a dash.

0	1	2	3	4	5	6	7	8	9	10	11	12
44	—	41	73	—	18	32	59	31	22	—	—	—

BUET · CSE 207 · Data Structures & Algorithms II

Part IV

NP-Completeness

P vs NP Classes, Reductions, NP-hard and NP-complete Problems

S4 — NP-Completeness

P vs NP Classes

Q1. Prove that $P = NP \Rightarrow NP = \text{co-NP}$. (12 marks)

[CSE 207 | L-2/T-1 | 2024–25, 2018–19]

Answer

1. Definitions and Key Properties

To formally prove that $P = NP \Rightarrow NP = \text{co-NP}$, we must first establish the formal definitions of the complexity classes involved and a fundamental property of the class P .

- **Class P :** The set of decision problems (languages) that can be decided by a Deterministic Turing Machine (DTM) in polynomial time.
- **Class NP :** The set of decision problems that can be decided by a Non-deterministic Turing Machine (NTM) in polynomial time (or equivalently, whose positive instances have polynomial-time verifiable certificates).
- **Class co-NP :** The set of decision problems whose *complement* is in NP . Formally, a language $L \in \text{co-NP} \iff \bar{L} \in NP$.

Lemma: P is closed under complementation ($P = \text{co-P}$)

Proof of Lemma. Let $L \in P$. By definition, there exists a DTM M that decides L in polynomial time $O(n^k)$. We can construct a new DTM M' for the complement language $\bar{L}$. M' simulates M on the input x . If M accepts, M' rejects. If M rejects, M' accepts. Since M halts in polynomial time, M' also halts in polynomial time $O(n^k)$. Thus, $\bar{L} \in P$, which implies $P \subseteq \text{co-P}$. By symmetry, $\text{co-P} \subseteq P$, meaning $P = \text{co-P}$. $\square$

2. Formal Proof

Assumption: Let us assume that $P = NP$.

We need to prove that $NP = \text{co-NP}$. To do this, we will prove subset inclusion in both directions: $NP \subseteq \text{co-NP}$ and $\text{co-NP} \subseteq NP$.

Part A: Prove $NP \subseteq \text{co-NP}$

- Let L be an arbitrary language such that $L \in NP$.
- By our assumption $P = NP$, it follows that $L \in P$.
- By the Lemma (P is closed under complement), if $L \in P$, then its complement $\bar{L} \in P$.
- Using our assumption $P = NP$ again, since $\bar{L} \in P$, it must be that $\bar{L} \in NP$.
- By the definition of co-NP , a language whose complement is in NP belongs to co-NP . Therefore, since $\bar{L} \in NP$, we have $L \in \text{co-NP}$.
- Since this holds for any arbitrary $L \in NP$, we conclude that $NP \subseteq \text{co-NP}$.

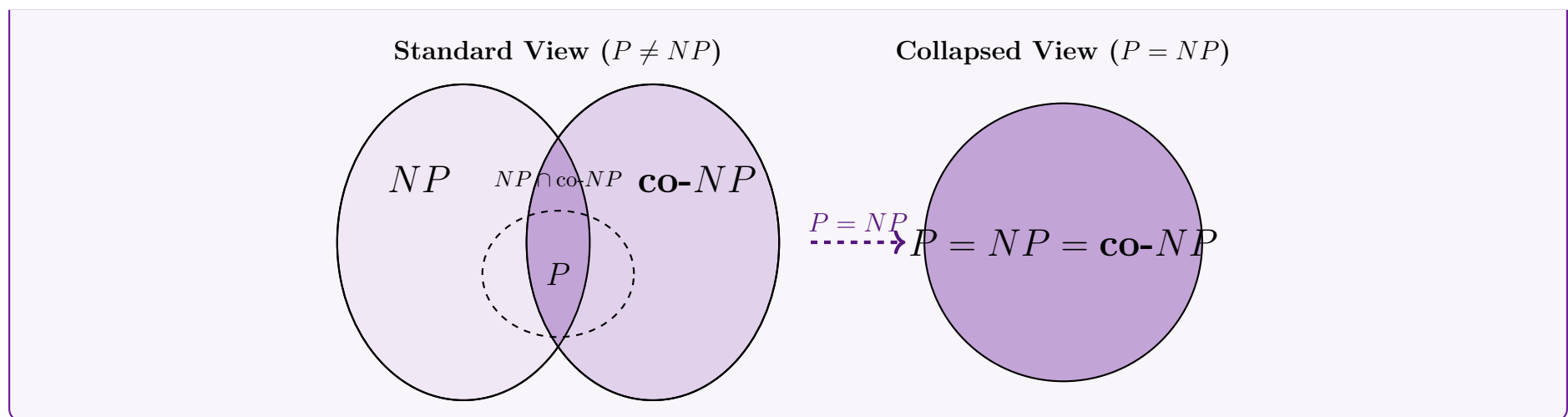
Part B: Prove $\text{co-NP} \subseteq NP$

- Let L be an arbitrary language such that $L \in \text{co-NP}$.
- By the definition of co-NP , its complement $\bar{L} \in NP$.
- By our assumption $P = NP$, it follows that $\bar{L} \in P$.
- By the Lemma (P is closed under complement), if $\bar{L} \in P$, then the complement of $\bar{L}$ (which is L itself) must also be in P . Thus, $L \in P$.
- Using our assumption $P = NP$ once more, since $L \in P$, we immediately get $L \in NP$.
- Since this holds for any arbitrary $L \in \text{co-NP}$, we conclude that $\text{co-NP} \subseteq NP$.

Conclusion: Since we have shown both $NP \subseteq \text{co-NP}$ and $\text{co-NP} \subseteq NP$, it must be true that $NP = \text{co-NP}$. Therefore, $P = NP \Rightarrow NP = \text{co-NP}$. ■

3. Visualization

The diagram below illustrates the structural collapse of complexity classes under the assumption that $P = NP$.



Q2. Prove or disprove that $P = NP \cap \text{co-NP}$. (8 marks)

[CSE 207 | L-2/T-1 | 2024–25]

Answer

1. Status of the Proposition

The proposition $P = NP \cap \text{co-NP}$ can currently **neither be proved nor disproved**. It stands as one of the major open problems in structural complexity theory.

To evaluate the statement, we decompose the equality into two subset inclusions:

- (a) $P \subseteq NP \cap \text{co-NP}$: This is known to be **true** and can be formally proved.
- (b) $NP \cap \text{co-NP} \subseteq P$: This is **unknown**, but widely conjectured by computer scientists to be **false**.

2. Proof of $P \subseteq NP \cap \text{co-NP}$

To prove this subset inclusion, we must show that any problem in P belongs to both NP and co-NP .

Proof. Let L be an arbitrary language such that $L \in P$.

Step 1: Show $P \subseteq NP$ By definition, $L \in P$ implies there exists a deterministic Turing machine (DTM) M that decides L in polynomial time. A non-deterministic Turing machine (NTM) is a generalization of a DTM. Therefore, an NTM can simply ignore its non-deterministic capabilities and simulate M exactly, deciding L in polynomial time. Alternatively, using the verifier definition of NP , the verifier can simply solve the problem in polynomial time without needing to read any certificate. Thus, $L \in NP$.

Step 2: Show $P \subseteq \text{co-NP}$ Since $L \in P$, its complement $\bar{L}$ must also be in P . We construct a DTM M' for $\bar{L}$ that simulates M on input x and simply reverses the output: if M accepts, M' rejects; if M rejects, M' accepts. Since M runs in polynomial time, M' also runs in polynomial time. Thus, $\bar{L} \in P$. From Step 1, we know $P \subseteq NP$, so $\bar{L} \in NP$. By the definition of co-NP , if the complement of a language is in NP , the language itself is in co-NP . Therefore, $L \in \text{co-NP}$.

Since $L \in NP$ and $L \in \text{co-NP}$, we have $L \in NP \cap \text{co-NP}$. Therefore, $P \subseteq NP \cap \text{co-NP}$. $\square$

3. The Unknown Direction: $NP \cap \text{co-NP} \subseteq P$

For the equality $P = NP \cap \text{co-NP}$ to hold, every problem that has both a polynomial-time verifiable "yes" certificate (in NP) and a polynomial-time verifiable "no" certificate (in co-NP) must also be solvable from scratch in polynomial time (in P).

Most complexity theorists conjecture that $P \neq NP \cap \text{co-NP}$. The evidence for this comes from several well-known problems that lie within $NP \cap \text{co-NP}$ but have resisted all attempts to find polynomial-time algorithms.

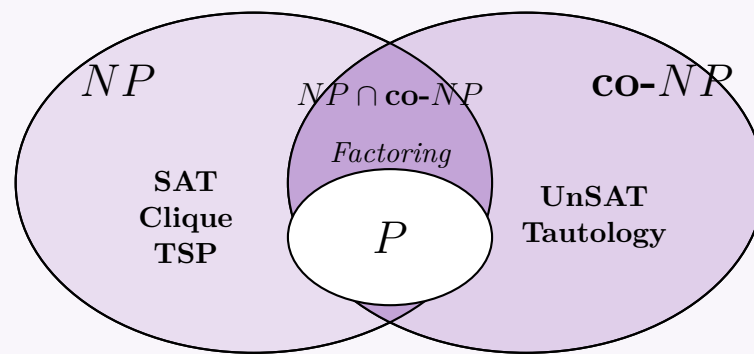
Candidate Problem in $(NP \cap \text{co-NP}) \setminus P$: Integer Factorization Consider the decision version of factoring: "Given an integer N and an integer k , does N have a non-trivial prime factor less than k ?"

- **Why it is in NP :** A "yes" certificate is simply the prime factorization of N . A verifier can multiply the factors to check they equal N , use a polynomial-time primality test (like AKS) to verify all factors are prime, and check if any factor is less than k .
- **Why it is in co-NP :** A "no" certificate is *also* the prime factorization of N . The verifier performs the exact same polynomial-time checks (multiplication and primality), but accepts if *none* of the factors are less than k .

Despite centuries of mathematical effort, no deterministic polynomial-time algorithm is known for factoring. If $P = NP \cap \text{co-NP}$ were true, Integer Factorization would necessarily be in P .

4. Visualizing the Complexity Classes

The Venn diagram below illustrates the standard conjecture regarding these complexity classes, showing P strictly contained within the intersection, and candidate problems like Factoring occupying the annulus between P and $NP \cap \text{co-NP}$.



Q3. “It is possible that $P \neq NP$, yet $NP = \text{co-}NP$.” Do you agree with this statement? Justify your answer. (8 marks) [CSE 207 | L-2/T-1 | 2024–25]

Answer

1. The Verdict

Yes, I agree with the statement. It is logically and theoretically possible that $P \neq NP$ while simultaneously $NP = \text{co-}NP$. There is no known mathematical contradiction in this scenario, even though it contradicts the mainstream conjecture held by most complexity theorists.

2. Theoretical Justification

To understand why this is possible, we must examine the definitions and the logical dependencies (or lack thereof) between these classes:

- **P :** The class of decision problems solvable by a deterministic Turing machine in polynomial time.
- **NP :** The class of decision problems for which a “YES” answer can be verified by a deterministic Turing machine in polynomial time, given a valid certificate.
- **$\text{co-}NP$:** The class of decision problems for which a “NO” answer can be verified by a deterministic Turing machine in polynomial time.

The Logical Independence

We know definitively that $P \subseteq NP \cap \text{co-}NP$. We also know that if $P = NP$, it forces $NP = \text{co-}NP$ (because P is closed under complementation).

However, the inverse implication—that $P \neq NP \implies NP \neq \text{co-}NP$ —**has never been proven**. The relationship between the ability to *find* an answer (which separates P from NP) and the symmetry of *verifying* “YES” versus “NO” instances (which separates NP from $\text{co-}NP$) are two distinct questions.

What would a world where $P \neq NP$ and $NP = \text{co-}NP$ look like?

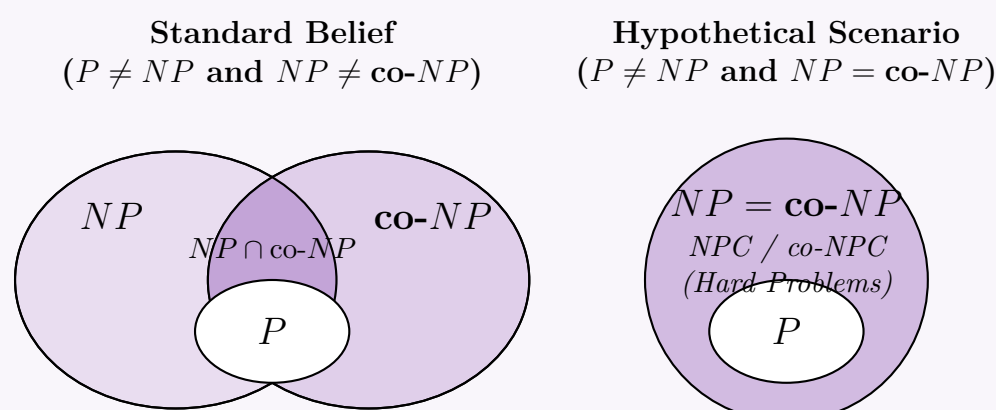
If this scenario were true, it would imply the following:

- Verification is symmetrical:** For every problem in NP (including NP -Complete problems like Boolean Satisfiability), if the answer is “NO”, there exists a succinct, polynomial-time verifiable proof of that “NO” answer. For example, there would exist a short certificate proving that a given boolean formula is *unsatisfiable*.
- Solving is still hard:** Despite having succinct proofs for both “YES” and “NO” instances, no deterministic polynomial-time algorithm can *construct* or find these answers from scratch. The computational effort to find the solution remains super-polynomial.

This represents a universe where nondeterminism is strictly more powerful than determinism ($P \subsetneq NP$), but nondeterminism is closed under complementation ($NP = \text{co-}NP$).

3. Visualization of Complexity Worlds

To clarify this, we can contrast the “Standard Conjecture” with the “Hypothetical Scenario” described in the prompt using Venn diagrams.



Conclusion: The diagram on the right is mathematically consistent. Until a proof emerges showing that $NP = \text{co-}NP \implies P = NP$ (or vice versa), the statement remains a valid possibility in theoretical computer science.

Q4. Define the classes of problems P , NP , and NP -complete. Draw a Venn diagram showing the widely believed relationship among these classes. (9 marks) [CSE 207 | L-2/T-1 | 2023–24]

Answer

1. Formal Definitions of Complexity Classes

In computational complexity theory, problems are categorized based on the resources (usually time) required to solve them. We focus on **decision problems** (problems with a “YES” or “NO” answer).

The Class P

The class P (Polynomial time) consists of all decision problems that can be **solved** efficiently.

- **Formal Definition:** A language L is in P if there exists a Deterministic Turing Machine (DTM) M and a polynomial $p(n)$ such that for any input x of length n , M decides whether $x \in L$ in at most $p(n)$ steps.
- **Intuition:** These are the “tractable” problems, such as sorting, shortest path, and matrix multiplication.

The Class NP

The class NP (Nondeterministic Polynomial time) consists of all decision problems for which a “YES” answer can be **verified** efficiently, even if finding the answer from scratch might be difficult.

- **Formal Definition (Verifier-based):** A language L is in NP if there exists a polynomial-time deterministic verifier V and a polynomial $p(n)$ such that for every $x \in L$, there exists a certificate (proof) y with $|y| \leq p(|x|)$ such that $V(x, y)$ accepts. If $x \notin L$, no such certificate exists.
- **Alternative Definition (Machine-based):** $L \in NP$ if it can be solved by a *Non-deterministic* Turing Machine (NTM) in polynomial time.
- **Intuition:** NP includes all problems in P (since we can just ignore the certificate and solve the problem), plus many potentially harder problems like the Traveling Salesperson Problem (TSP) or Boolean Satisfiability (SAT).

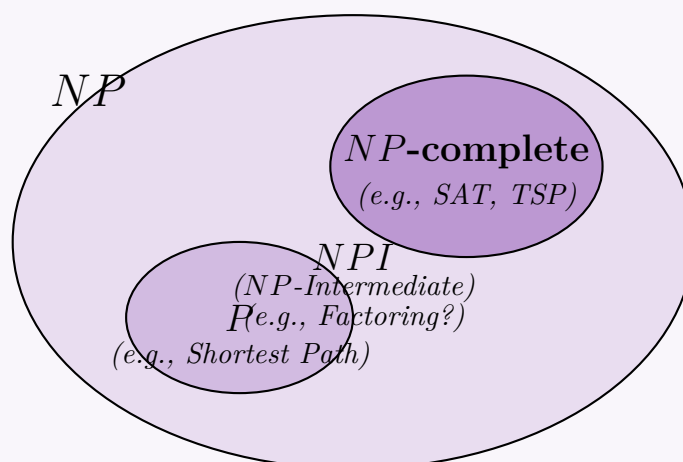
The Class NP -complete

The class of NP -complete (NPC) problems represents the computationally “hardest” problems within NP .

- **Formal Definition:** A language L is NP -complete if it satisfies two conditions:
 - (a) **Membership:** $L \in NP$.
 - (b) **Hardness:** Every problem $L' \in NP$ is polynomial-time reducible to L (denoted $L' \leq_p L$). This means there exists a polynomial-time computable function f such that $x \in L' \iff f(x) \in L$.
- **Intuition:** If anyone discovers a polynomial-time algorithm for *any* single NP -complete problem, it would imply that every problem in NP can be solved in polynomial time (thus proving $P = NP$).

2. Visualizing the Believed Relationship (Assuming $P \neq NP$)

The overwhelming consensus among computer scientists is that $P \neq NP$. Under this assumption, P is a strict subset of NP , NP -complete problems exist strictly outside of P , and there exist intermediate problems (like potentially Integer Factorization) that are in NP , but are neither in P nor NP -complete (Ladner’s Theorem).



Note: If the unlikely scenario that $P = NP$ were true, the Venn diagram would collapse entirely into a single circle where $P = NP = NP$ -complete (except for trivial languages $\emptyset$ and Σ^*).

Q5. Differentiate among NP, NP-hard, and NP-complete classes of problems. (6 marks)

[CSE 207 | L-2/T-1 | 2022–23]

Answer

1. Formal Definitions

To differentiate among the classes NP , NP -hard, and NP -complete, we must examine their formal definitions based on polynomial-time verifiability and polynomial-time reducibility.

The Class NP (Nondeterministic Polynomial Time)

The class NP contains all **decision problems** (problems with a “YES” or “NO” answer) for which a “YES” answer can be efficiently verified.

- **Definition:** A language L is in NP if there exists a deterministic polynomial-time verifier V such that for every string $x \in L$, there exists a polynomial-size certificate (proof) y where $V(x, y)$ accepts.
- **Intuition:** If someone hands you a proposed solution to a problem, you can check whether it is correct in polynomial time.
- **Examples:** Shortest Path (is there a path shorter than k ?), Boolean Satisfiability (SAT), Integer Factorization. Note that NP includes the class P .

The Class NP -hard

The class NP -hard represents problems that are **at least as hard** as the hardest problems in NP .

- **Definition:** A problem H is NP -hard if every problem $L \in NP$ can be reduced to H in polynomial time ($L \leq_p H$).
- **Characteristics:** Crucially, NP -hard problems *do not have to be in NP* . They do not even have to be decision problems (they can be optimization problems), and they do not have to be decidable.
- **Examples:** The Halting Problem (undecidable but NP -hard), the Traveling Salesperson *Optimization* Problem (finding the exact shortest route, rather than just asking if a route shorter than k exists).

The Class NP -complete (NPC)

The class NP -complete represents the computationally **hardest problems that are still within NP** .

- **Definition:** A problem C is NP -complete if it satisfies two strict conditions:
 - (a) $C \in NP$ (It is verifiable in polynomial time).
 - (b) C is NP -hard (Every problem in NP reduces to it in polynomial time).
- **Intuition:** NP -complete problems form the critical intersection of NP and NP -hard. A polynomial-time algorithm for *any* NP -complete problem would immediately imply $P = NP$.
- **Examples:** 3-SAT, Vertex Cover, Traveling Salesperson *Decision* Problem.

2. Summary of Differences

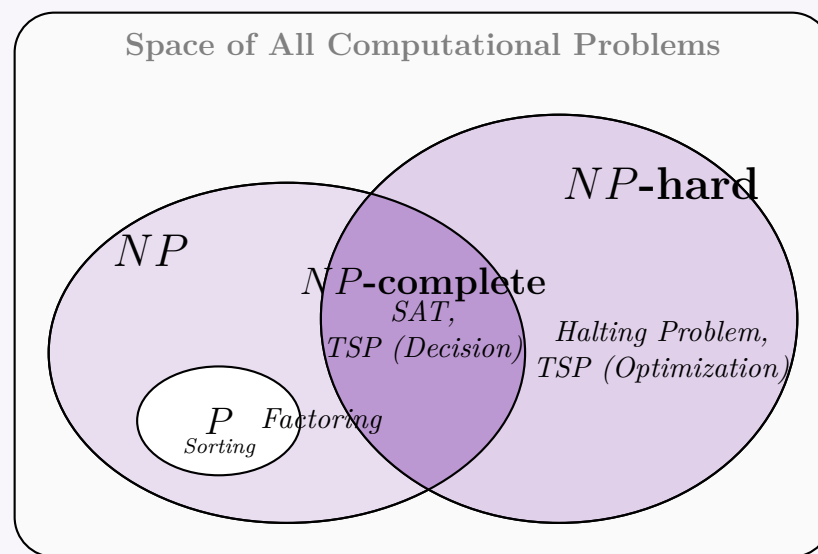
The following table highlights the critical distinctions between the three classes:

Complexity Class	Is it in NP ?	Is every NP problem reducible to it?	Problem Types Allowed
NP	Yes	No (generally)	Decision only
NP -hard	Not necessarily	Yes	Decision, Optimization, Undecidable
NP -complete	Yes	Yes	Decision only

3. Visual Representation

Assuming $P \neq NP$, the relationship among these classes is best visualized with the following Euler diagram. Notice how NP -complete is strictly the intersection of NP and NP -hard, while NP -hard extends infinitely upward into harder complexity classes (like EXP or Undecidable).

286



Q6. Using an example, show that if a problem is in NP, the complement of that problem is in co-NP. Draw the general consensus regarding P, NP and co-NP in a Venn diagram. (10 marks) [CSE 207 | L-2/T-2 | 2020–21]

Answer

1. Theoretical Foundation: NP and co-NP

To show that the complement of a problem in NP is in $co-NP$, we must first look at the formal definition of these complexity classes.

- **The Class NP :** A decision problem (language) L is in NP if there exists a polynomial-time verifier V such that for every “YES” instance $x \in L$, there is a polynomial-sized certificate c that V can use to verify the “YES” answer in polynomial time.
- **The Complement of a Problem:** For a decision problem L defined over an alphabet Σ , its complement is denoted as $\bar{L} = \Sigma^* \setminus L$. In practical terms, this reverses the “YES” and “NO” answers of the problem.
- **The Class $co-NP$:** By formal definition, the class $co-NP$ is exactly the set of all languages whose complements are in NP . Mathematically, $co-NP = \{L \mid \bar{L} \in NP\}$.

Therefore, the statement “if a problem is in NP , its complement is in $co-NP$ ” is true by the very definition of $co-NP$. If $L \in NP$, then its complement $\bar{L}$ trivially satisfies the condition that its complement (which is L itself) is in NP . Thus, $\bar{L} \in co-NP$.

2. Illustrative Example

To understand this intuitively, let us look at the **Boolean Satisfiability Problem (SAT)** and its complement, **UNSAT**.

Problem L : SAT (Is the formula satisfiable?)

- **Question:** Given a boolean formula ϕ , does there exist an assignment of truth values to its variables that makes ϕ evaluate to TRUE?
- **Why $SAT \in NP$:** If the answer is “YES”, a prover can provide the satisfying truth assignment as the certificate. A verifier can easily plug these values into ϕ and evaluate it in polynomial time to check if it equals TRUE.

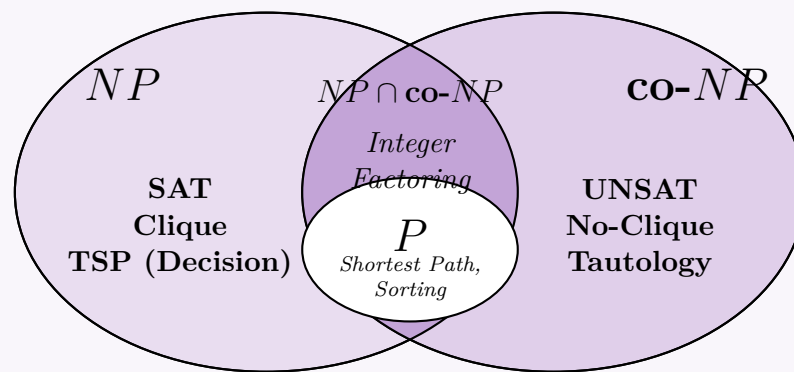
Problem $\bar{L}$: UNSAT (Is the formula unsatisfiable?)

- **Question:** Given a boolean formula ϕ , is it true that *no* assignment of truth values makes ϕ evaluate to TRUE? (i.e., does every assignment evaluate to FALSE?)
- **Why $UNSAT \in co-NP$:** By definition, since $SAT \in NP$ and UNSAT is the complement of SAT, $UNSAT \in co-NP$.
- **The Asymmetry of Verification:** Notice why UNSAT is not obviously in NP . If the answer is “YES” (the formula is unsatisfiable), there is no obvious short certificate to prove it. A verifier would seemingly have to check all 2^n possible truth assignments to ensure none of them work, taking exponential time. However, a “NO” answer to UNSAT (meaning the formula *is* satisfiable) can be easily verified by providing the satisfying assignment. This is the hallmark of a $co-NP$ problem: *NO instances have succinct, easily verifiable proofs*.

3. The Consensus Venn Diagram

The general consensus in theoretical computer science is that $P \neq NP$, and furthermore, that $NP \neq co-NP$.

Under this belief, P is a proper subset of both NP and $co-NP$ (since deterministic polynomial time computation is perfectly symmetric with respect to YES/NO answers). There also exist problems in the intersection $NP \cap co-NP$ that are believed not to be in P (such as Integer Factorization).



Q7. Suppose $X \in \text{NP-complete}$. Prove that $X \in P$ if and only if $P = NP$. (8 marks)

[CSE 207 | L-2/T-2 | 2019–20]

Answer

1. Problem Statement and Definitions

We are given a problem X such that $X \in \text{NP-complete}$. We must prove a biconditional statement:

$$X \in P \iff P = NP$$

By the definition of NP-completeness, a problem X is NP-complete if and only if:

- (a) $X \in NP$ (Membership)
- (b) For every language $L \in NP$, $L \leq_p X$ (NP-hardness), where $\leq_p$ denotes a polynomial-time Karp reduction.

To mathematically establish the biconditional ($\iff$), we must prove both directions: the forward implication ($\Rightarrow$) and the reverse implication ($\Leftarrow$).

2. Proof of Forward Implication ($\Rightarrow$)

Assume: $X \in P$.

To Show: $P = NP$.

Proof. It is a well-known, fundamental property of complexity classes that $P \subseteq NP$. To show the equality $P = NP$, it strictly suffices to prove the reverse inclusion: $NP \subseteq P$.

Let L be an arbitrary language in NP . Since X is NP-complete, we know by definition that L is polynomial-time reducible to X , denoted as $L \leq_p X$. This means there exists a deterministic polynomial-time computable function f such that for any input string w :

$$w \in L \iff f(w) \in X$$

Let the time taken to compute the reduction function $f(w)$ be bounded by a polynomial $p(|w|)$.

By our assumption, $X \in P$. This means there exists a deterministic polynomial-time algorithm A_X that decides X . Let the time taken by A_X to process an input of size m be bounded by a polynomial $q(m)$.

Using these pieces, we can construct a deterministic polynomial-time algorithm A_L to decide L as follows:

- (a) Given an input w , compute the reduction $f(w)$. This takes at most $O(p(|w|))$ time.
- (b) The size of the reduced string, $|f(w)|$, must be at most $O(p(|w|))$ because a Deterministic Turing Machine can write at most one symbol per computational step.
- (c) Run the decider algorithm A_X on the transformed input $f(w)$. The time taken is $O(q(|f(w)|))$, which is bounded by $O(q(p(|w|)))$.
- (d) Return the exact result yielded by A_X .

The total running time of A_L is the sum of the time for the reduction and the time for the solver: $O(p(|w|) + q(p(|w|)))$. Since the composition of two polynomials $q(p(n))$ is itself a polynomial, algorithm A_L runs in deterministic polynomial time. Thus, $L \in P$. Because this holds true for *any* arbitrary $L \in NP$, we conclude $NP \subseteq P$. Combined with $P \subseteq NP$, we rigorously establish $P = NP$. $\square$

3. Proof of Reverse Implication ($\Leftarrow$)

Assume: $P = NP$.

To Show: $X \in P$.

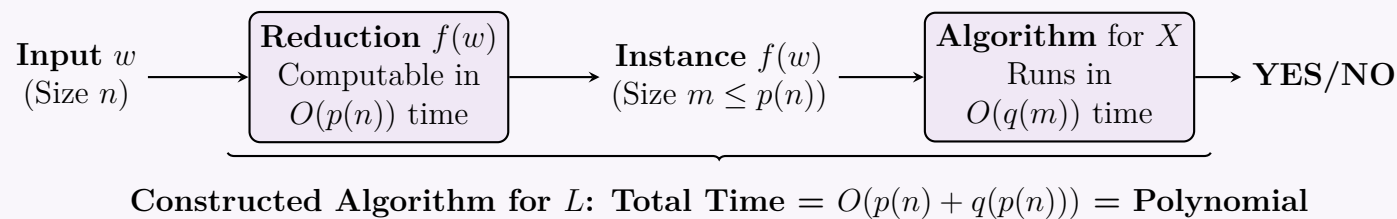
Proof. By the premise of the problem, we are given that $X \in \text{NP-complete}$. By the first criterion of the definition of NP-completeness, this explicitly guarantees that $X \in NP$.

Since we have assumed that $P = NP$, the two complexity classes are entirely identical. This means every single problem contained in NP is fundamentally also contained in P .

Therefore, since $X \in NP$ and $NP = P$, it immediately follows by direct substitution that $X \in P$. $\square$

4. Visualizing the Polynomial-Time Reduction (Forward Implication)

The diagram below conceptually illustrates the mechanics of the forward proof. It demonstrates how a polynomial-time reduction allows us to build a polynomial-time decider for *any* problem $L \in NP$ by leveraging the assumed polynomial-time algorithm for the NP-complete problem X .



Q8. How do we cope with hard problems? Discuss. (10 marks)

[CSE 207 | L-2/T-2 | 2017–18]

Answer

1. The Core Challenge of Hard Problems

When a problem is proven to be $\mathcal{NP}$ -hard (or $\mathcal{NP}$ -complete), it implies that it is highly unlikely to admit a polynomial-time algorithm that optimally solves all possible instances (assuming $\mathcal{P} \neq \mathcal{NP}$). However, in real-world applications (e.g., routing networks, scheduling, packing), we cannot simply refuse to solve the problem.

To cope with computational hardness, we must relax at least one of the three properties of an ideal algorithmic solution:

- (a) **Speed:** Solving the problem in polynomial time.
- (b) **Optimality:** Finding the exact, mathematically best solution.
- (c) **Generality:** Solving the problem for any arbitrary input instance.

2. Strategies for Coping with Hardness

Depending on which property we are willing to sacrifice, there are four primary strategies to handle hard problems in practice.

Strategy A: Relax Optimality (Approximation Algorithms)

If we require a fast solution for all inputs but can accept a slightly sub-optimal answer, we use approximation algorithms.

- **Mechanism:** These are polynomial-time algorithms that come with a mathematically proven *worst-case guarantee* on the quality of the solution.
- **Approximation Ratio (ρ):** An algorithm has an approximation ratio $\rho \geq 1$ if for any input size n , the cost of the approximate solution C and the optimal cost C^* satisfy $\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq \rho$.
- **Example:** The Vertex Cover problem is $\mathcal{NP}$ -hard, but a simple greedy strategy (picking both endpoints of arbitrary edges) guarantees a solution that is at most twice the size of the optimal cover ($\rho = 2$).

Strategy B: Relax Optimality and Guarantees (Heuristics & Metaheuristics)

When we need fast solutions but cannot find good theoretical approximation ratios, we rely on heuristics.

- **Mechanism:** Rule-of-thumb algorithms that tend to find "good enough" solutions quickly in practice, though they offer no formal guarantees on worst-case optimality or running time.
- **Metaheuristics:** Higher-level frameworks that guide the search process to avoid local optima, such as **Simulated Annealing**, **Genetic Algorithms**, or **Tabu Search**.
- **Example:** Using the 2-Opt heuristic for the Traveling Salesperson Problem (TSP) works exceptionally well in practice, even though it can theoretically perform poorly on adversarial inputs.

Strategy C: Relax Generality (Special Cases & Restrictions)

If we require both optimal solutions and polynomial time, we must restrict the inputs to special cases that exploit underlying structural properties.

- **Mechanism:** We design algorithms tailored to specific data structures (e.g., trees, planar graphs, or bipartite graphs) where the problem is no longer $\mathcal{NP}$ -hard.
- **Example:** The Maximum Independent Set problem is $\mathcal{NP}$ -hard on general graphs. However, if we restrict the input to *trees*, the problem can be solved optimally in $O(V)$ time using Dynamic Programming.

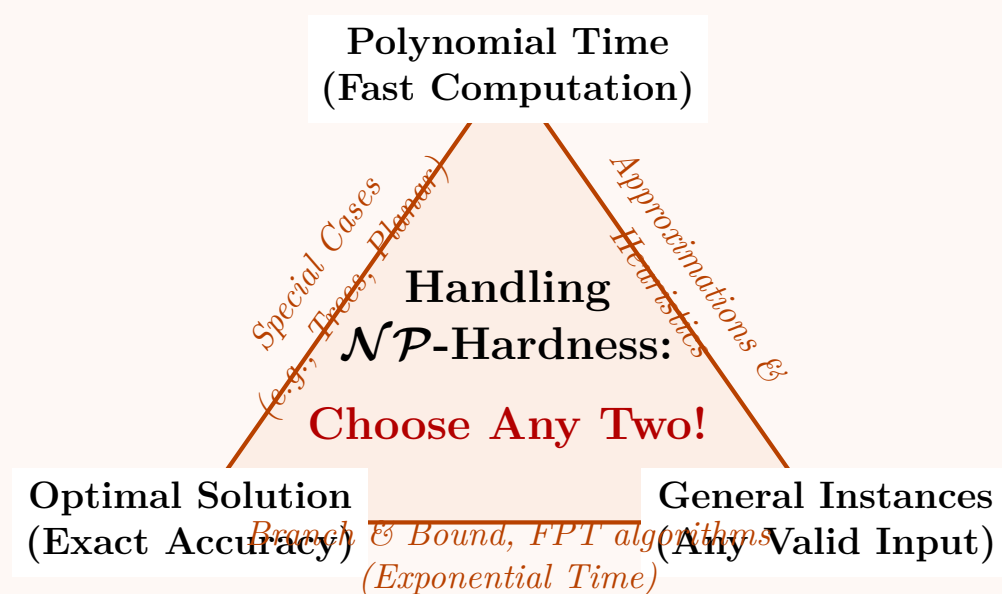
Strategy D: Relax Speed (Exact Exponential & Parameterized Algorithms)

If we absolutely must have the exact optimal solution for general instances, we accept that the algorithm will take exponential time in the worst case, but we try to minimize the practical blow-up.

- **Intelligent Search (Branch & Bound):** Systematically enumerating candidate solutions while using upper/lower bounds to dynamically prune large portions of the search space that cannot contain the optimal solution.
- **Fixed-Parameter Tractability (FPT):** Isolating the exponential complexity to a small parameter k rather than the overall input size n . An algorithm is FPT if it runs in $O(f(k) \cdot \text{poly}(n))$ time.
- **Example:** Finding a Vertex Cover of size k can be solved exactly in $O(2^k \cdot n)$ time. If k is small, this is highly efficient despite being $\mathcal{NP}$ -hard in general.

3. The "Pick Two" Trade-off Visualization

The dilemma of coping with $\mathcal{NP}$ -hard problems is traditionally visualized as an algorithmic triangle, illustrating that you can typically achieve at most two of the three ideal properties simultaneously.



Q9. Suppose $X \in \text{NP-complete}$. Prove that $X \in P$ if and only if $P = \text{NP}$. (10 marks)

[CSE 207 | L-2/T-2 | 2016–17]

Answer

1. Problem Statement and Formal Definitions

We are given a problem X such that $X \in \text{NP-complete}$. We need to prove the biconditional statement:

$$X \in P \iff P = \text{NP}$$

To do this, we must recall the formal definition of NP-completeness. A decision problem X is NP-complete if it satisfies two conditions:

- Membership in NP:** $X \in \text{NP}$.
- NP-Hardness:** For every language $L \in \text{NP}$, L is polynomial-time reducible to X (denoted as $L \leq_p X$).

To prove a biconditional statement ($A \iff B$), we must prove both directions: the forward implication ($\Rightarrow$) and the reverse implication ($\Leftarrow$).

2. Proof of the Forward Implication ($\Rightarrow$)

Assume: $X \in P$.

To Prove: $P = \text{NP}$.

Proof. It is a well-established fact in complexity theory that $P \subseteq \text{NP}$, because any problem that can be solved in deterministic polynomial time can easily be verified in polynomial time. Therefore, to prove $P = \text{NP}$, it suffices to show that $\text{NP} \subseteq P$.

Let L be an arbitrary language such that $L \in \text{NP}$. We want to show that $L \in P$.

- Since X is NP-complete, by definition, $L \leq_p X$.
- This means there exists a deterministic polynomial-time computable reduction function f such that for any input string w :

$$w \in L \iff f(w) \in X$$

Let the time complexity to compute $f(w)$ be bounded by a polynomial $p(|w|)$.

- Since we assumed $X \in P$, there exists a deterministic polynomial-time algorithm A_X that decides X . Let the time complexity of A_X on an input of size m be bounded by a polynomial $q(m)$.

We can construct a deterministic polynomial-time algorithm A_L to decide L :

- Given an input w , use the reduction function to compute $f(w)$. This takes $O(p(|w|))$ time.
- The size of the transformed input, $|f(w)|$, can be at most $O(p(|w|))$ because a deterministic Turing machine can write at most one symbol per step.
- Feed $f(w)$ into algorithm A_X . The time taken by A_X is bounded by $O(q(|f(w)|))$, which is at most $O(q(p(|w|)))$.
- Return the answer produced by A_X .

The total running time of A_L is bounded by $O(p(|w|) + q(p(|w|)))$. Since the composition of two polynomials $q(p(n))$ is also a polynomial, algorithm A_L runs in deterministic polynomial time. Thus, $L \in P$.

Since L was an arbitrary problem in NP , we have shown that every problem in NP is also in P , meaning $NP \subseteq P$. Combined with $P \subseteq NP$, we conclude $P = NP$. $\square$

3. Proof of the Reverse Implication ($\Leftarrow$)

Assume: $P = NP$.

To Prove: $X \in P$.

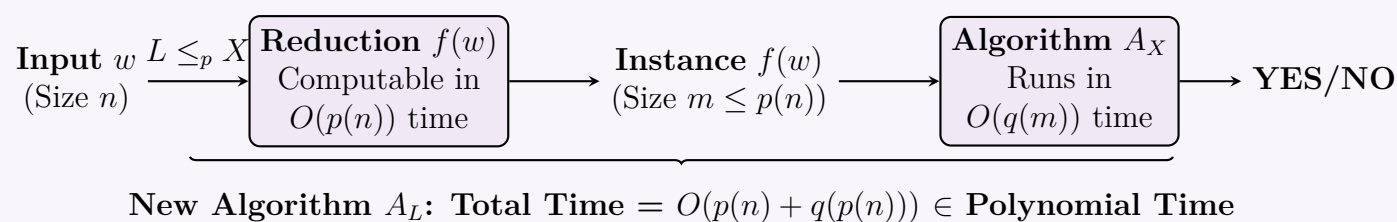
Proof. By the premise of the problem, we are given that X is NP-complete. According to the first condition of the definition of NP-completeness, we know that $X \in NP$.

Since our assumption states that $P = NP$, the two complexity classes are identical. Every problem that belongs to NP inherently belongs to P .

Therefore, by direct substitution since $X \in NP$ and $NP = P$, it trivially follows that $X \in P$. $\square$

4. Visualizing the Forward Proof (Reduction Mechanics)

The following diagram illustrates how the polynomial-time reduction works to solve an arbitrary NP problem using the polynomial-time algorithm for the NP-complete problem X .



Q10. Prove that $NP \neq \text{co-NP} \Rightarrow P \neq NP$. (10 marks)

[CSE 207 | L-2/T-2 | 2016–17]

Answer

1. Proof Strategy: Proof by Contraposition

We are asked to prove the logical implication:

$$NP \neq \text{co-NP} \Rightarrow P \neq NP$$

In formal logic, an implication $A \Rightarrow B$ is logically equivalent to its **contrapositive**, $\neg B \Rightarrow \neg A$. Therefore, it is strictly mathematically equivalent to prove the following statement:

$$P = NP \Rightarrow NP = \text{co-NP}$$

By proving this contrapositive statement, we will inherently prove the original claim.

2. Foundational Properties

Before proceeding with the proof, we must establish a fundamental property of the complexity class P : **Closure under complementation**.

- Let L be a language in P . By definition, there exists a deterministic polynomial-time Turing machine M that decides L .
- To decide the complement language, $\bar{L}$, we can construct a new Turing machine M' that simulates M exactly, but simply *inverts* the final output (accepts if M rejects, and rejects if M accepts).
- Since inverting a boolean output takes $O(1)$ time, M' also runs in deterministic polynomial time.
- Therefore, if $L \in P$, then $\bar{L} \in P$. This means the class P is exactly equal to the class $\text{co-}P$.

3. Formal Proof of the Contrapositive

Assume: $P = NP$.

To Show: $NP = \text{co-NP}$.

Proof. To prove set equality ($NP = \text{co-NP}$), we must show that any language in NP is also in co-NP, and vice versa.

Part 1: $NP \subseteq \text{co-NP}$

- Let L be an arbitrary language such that $L \in NP$.
- Because we assumed $P = NP$, it follows that $L \in P$.
- As established, P is closed under complementation. Thus, its complement language $\bar{L} \in P$.
- Since $P = NP$, we can substitute P with NP , meaning $\bar{L} \in NP$.
- By the formal definition of co-NP, a language L is in co-NP if and only if its complement $\bar{L}$ is in NP. Since we just showed $\bar{L} \in NP$, we conclude that $L \in \text{co-NP}$.
- Since L was an arbitrary language in NP, we have shown that $NP \subseteq \text{co-NP}$.

Part 2: $\text{co-NP} \subseteq NP$

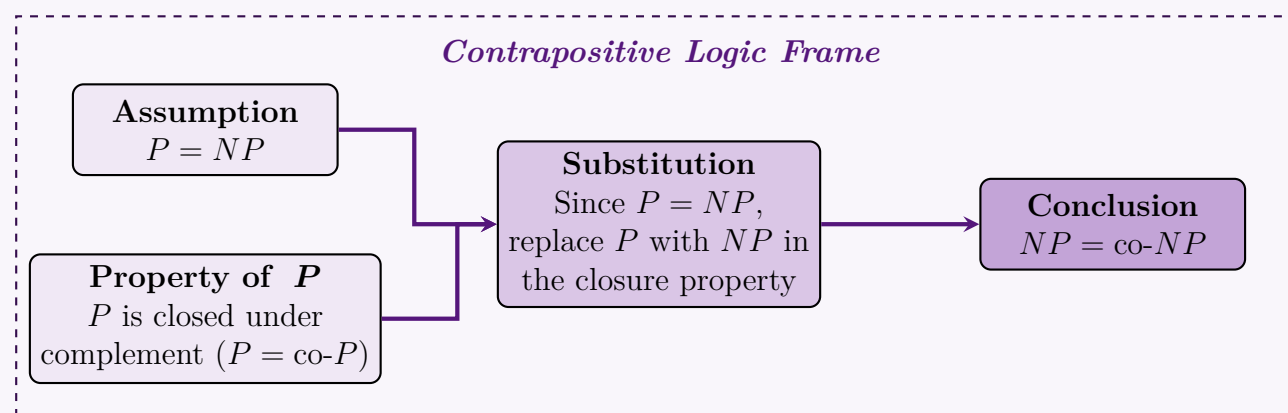
- Let L be an arbitrary language such that $L \in \text{co-NP}$.
- By the definition of co-NP, its complement $\bar{L} \in NP$.
- Because we assumed $P = NP$, it follows that $\bar{L} \in P$.
- Since P is closed under complementation, the complement of $\bar{L}$ (which is L itself) must also be in P . Thus, $L \in P$.
- Since $P = NP$, it follows that $L \in NP$.
- Since L was an arbitrary language in co-NP, we have shown that $\text{co-NP} \subseteq NP$.

Conclusion of Proof: Since we have shown both $NP \subseteq \text{co-NP}$ and $\text{co-NP} \subseteq NP$, it rigorously follows that $NP = \text{co-NP}$.

Thus, we have proven that $P = NP \implies NP = \text{co-NP}$. By contraposition, the original statement $NP \neq \text{co-NP} \implies P \neq NP$ is proven true. $\square$

4. Visualizing the Logical Deduction

The diagram below maps the logical flow of the contrapositive proof. The inherent symmetry of deterministic computation forces nondeterministic computation to be symmetric if $P = NP$.



Q11. If L_1 and L_2 are two languages such that $L_1 \leq_p L_2$, show that $L_2 \in P$ implies $L_1 \in P$. If an NP-complete problem is polynomially solvable, prove that $P = NP$. (10 marks) [CSE 207 | L-2/T-2 | 2013–14]

Answer

Part 1: Proving that $L_1 \leq_p L_2$ and $L_2 \in P \implies L_1 \in P$

Given:

- $L_1 \leq_p L_2$: Language L_1 is polynomial-time reducible to language L_2 .
- $L_2 \in P$: There exists a deterministic polynomial-time algorithm A_2 that decides L_2 .

To Show: $L_1 \in P$.

Proof. By the definition of polynomial-time reduction ($L_1 \leq_p L_2$), there exists a deterministic polynomial-time computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that for every input string x :

$$x \in L_1 \iff f(x) \in L_2$$

Let the time required to compute $f(x)$ on an input x of length n be bounded by a polynomial $p(n)$. Because a deterministic Turing machine can write at most one symbol per step, the length of the output string $f(x)$ is also strictly bounded by $p(n)$; that is, $|f(x)| \leq p(n)$.

Since $L_2 \in P$, the algorithm A_2 decides whether any string $y \in L_2$ in time bounded by some polynomial $q(|y|)$.

We can now construct a new algorithm A_1 to decide L_1 :

Algorithm 51 Decider for L_1

Require: Input string x of length n

Ensure: **True** if $x \in L_1$, **False** otherwise

- 1: Compute $y = f(x)$ ▷ Takes $O(p(n))$ time
 - 2: Run $A_2(y)$ ▷ Takes $O(q(|y|))$ time
 - 3: **if** $A_2(y)$ returns **True** **then**
 - 4: **return True**
 - 5: **else**
 - 6: **return False**
 - 7: **end if**
-

Complexity Analysis of A_1 :

- **Time Complexity:** Computing $f(x)$ takes $O(p(n))$ time. Running A_2 on $f(x)$ takes $O(q(|f(x)|))$ time. Since $|f(x)| \leq p(n)$, the time to run A_2 is bounded by $O(q(p(n)))$. The total time complexity is $O(p(n) + q(p(n)))$. The composition and sum of polynomials yield another polynomial. Thus, A_1 runs in deterministic polynomial time.
- **Correctness:** $A_1(x)$ accepts $\iff A_2(f(x))$ accepts $\iff f(x) \in L_2 \iff x \in L_1$.

Since A_1 decides L_1 in polynomial time, $L_1 \in P$. □

Part 2: Proving that if an NP-Complete problem is in P, then $P = NP$

Given: Let X be an NP-complete problem, and suppose X is polynomially solvable ($X \in P$).

To Show: $P = NP$.

Proof. To rigorously prove $P = NP$, we must demonstrate both set inclusions: $P \subseteq NP$ and $NP \subseteq P$.

Step 1: Show $P \subseteq NP$

- By definition, the class P contains problems solvable by a deterministic Turing machine in polynomial time.
- The class NP contains problems solvable by a non-deterministic Turing machine in polynomial time.
- Since every deterministic Turing machine is simply a restricted case of a non-deterministic Turing machine (where the degree of non-determinism is exactly 1 at every step), any problem in P is trivially in NP . Thus, $P \subseteq NP$.

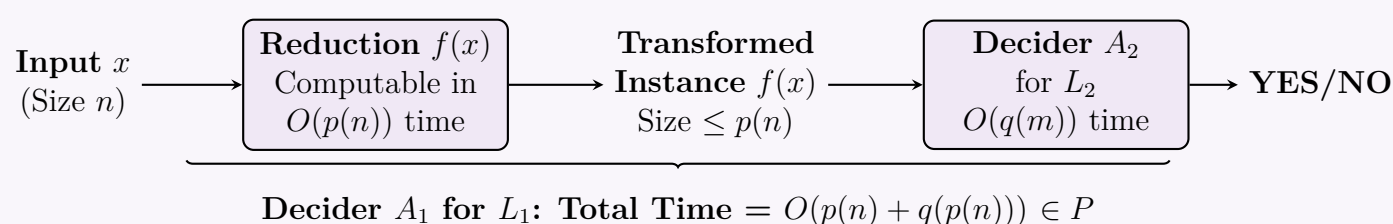
Step 2: Show $NP \subseteq P$

- Let L be an arbitrary language in NP .
- By the definition of NP-completeness, since X is NP-complete, **every** problem in NP is polynomial-time reducible to X . Therefore, $L \leq_p X$.
- We are given that $X \in P$.
- By the theorem proven in Part 1, if $L \leq_p X$ and $X \in P$, it directly implies that $L \in P$.
- Since L was an arbitrarily chosen problem from NP , this holds for all problems in NP . Thus, $NP \subseteq P$.

Conclusion: Since $P \subseteq NP$ and $NP \subseteq P$, we conclude that $P = NP$. □

Visualization of the Subroutine Construction

The following TikZ diagram illustrates the algorithm A_1 constructed in Part 1, showing how the polynomial runtime is preserved through the reduction pipeline.



Q12. Define the following six classes of problems: P, Co-P, NP, Co-NP, NPC, and NP-hard. By a diagram show the relationship among P, Co-P, NP, Co-NP, and NPC that most researchers regard as the most likely. (6+4=10 marks) [CSE 207 | L-2/T-2 | 2011–12]

Answer

1. Definitions of the Six Complexity Classes

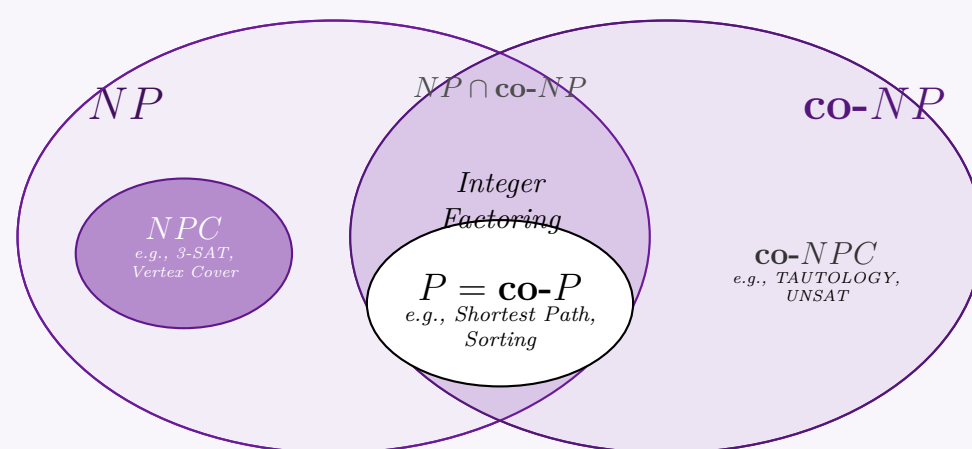
Computational complexity theory classifies decision problems (problems requiring a YES or NO answer) into distinct classes based on the resources required to solve or verify them.

- **Class P (Polynomial Time):** The class of decision problems that can be solved by a deterministic Turing machine in a time bounded by a polynomial function of the input size n . Formally, $P = \bigcup_{k \geq 1} \text{TIME}(n^k)$. These problems are considered computationally tractable.
- **Class $\text{co-}P$ (Complementary Polynomial Time):** The class containing the complements of all decision problems in P . A problem belongs to $\text{co-}P$ if its NO instances can be recognized in deterministic polynomial time. Because deterministic polynomial-time algorithms can simply invert their acceptance states in $O(1)$ time, P is closed under complementation, meaning $P = \text{co-}P$.
- **Class NP (Nondeterministic Polynomial Time):** The class of decision problems that can be solved by a non-deterministic Turing machine in polynomial time. Equivalently, it is the class of problems for which any YES instance possesses a polynomial-sized certificate (witness) that can be verified by a deterministic Turing machine in polynomial time.
- **Class $\text{co-}NP$ (Complementary Nondeterministic Polynomial Time):** The class containing the complements of all decision problems in NP . Mathematically, $\text{co-}NP = \{L \mid \bar{L} \in NP\}$. For any problem in $\text{co-}NP$, every NO instance has a short, polynomial-sized certificate that can be verified deterministically in polynomial time.
- **Class NPC (NP-Complete):** The class of the computationally hardest decision problems within NP . A problem L is NP -complete if it strictly fulfills two conditions:
 - (a) $L \in NP$
 - (b) $\forall L' \in NP, L' \leq_p L$ (Every problem in NP can be reduced to L via a polynomial-time Karp reduction).
- **Class NP -hard:** The class of problems to which any problem in NP can be reduced in polynomial time ($\forall L' \in NP, L' \leq_p L$). Unlike NP -complete problems, NP -hard problems do not need to be in NP , nor do they even have to be decision problems or decidable (e.g., the Halting Problem is NP -hard).

2. Structural Relationship Under the Consensus Hypothesis

The general consensus among theoretical computer scientists is that $P \neq NP$. This hypothesis structurally implies that $P = \text{co-}P \subsetneq (NP \cap \text{co-}NP)$ and that $NP \neq \text{co-}NP$.

Under this assumption, the intersection of NP and $\text{co-}NP$ contains problems that are solvable in polynomial time (P), alongside certain problems not known to be in P (such as Integer Factoring). The NP -complete problems (NPC) occupy a distinct boundary within NP , completely separated from $\text{co-}NP$.



Reductions

Q1. For a graph $G = (V, E)$, a set of vertices I in G is an *independent set* if for every two vertices in I , there is no edge between them, and a set C of vertices in V is a *clique* if every two distinct vertices are adjacent. Given a graph G and a number K , the independent set problem is known to be NP-complete. The clique problem asks: given a graph $G = (V, E)$ and an integer K , does G have a clique of size at least K ? Prove that the clique problem is as hard as the independent set problem. (15 marks) [CSE 207 | L-2/T-1 | 2024–25]

Answer

To formally prove that the CLIQUE problem is as hard as the INDEPENDENT SET problem, we must demonstrate a polynomial-time Karp reduction from INDEPENDENT SET to CLIQUE, denoted as $\text{INDEPENDENT SET} \leq_p \text{CLIQUE}$.

1. NP Membership of CLIQUE

Although the question only strictly asks to show it is "as hard as" (NP-Hardness), formally classifying a problem as NP-Complete requires proving it belongs to NP.

- **Certificate:** A subset of vertices $C \subseteq V$ such that $|C| = K$.
- **Verifier:** An algorithm checks that $|C| = K$ and iterates over all pairs $u, v \in C$ to verify that the edge $(u, v) \in E$. For a subset of size K , this requires checking $\mathcal{O}(K^2)$ pairs. Since this verification runs in polynomial time with respect to the input size, $\text{CLIQUE} \in \text{NP}$.

2. Polynomial-Time Reduction Construction

We must transform any generic instance of the INDEPENDENT SET problem $\langle G, K \rangle$ into an instance of the CLIQUE problem $\langle G', K' \rangle$. We construct $G' = (V', E')$ as the **complement graph** of G , denoted $\overline{G}$.

- $V' = V$ (the vertex set remains identical).
- $E' = \overline{E} = \{(u, v) \mid u, v \in V, u \neq v, \text{ and } (u, v) \notin E\}$ (edges exist in G' if and only if they do not exist in G).
- $K' = K$ (the target size remains exactly the same).

Algorithm 52 Reduction from INDEPENDENT-SET to CLIQUE

```

1: Input: Graph  $G = (V, E)$ , integer  $K$ 
2: Output: Graph  $G' = (V', E')$ , integer  $K'$ 
3:  $V' \leftarrow V$ 
4:  $E' \leftarrow \emptyset$ 
5: for each pair of distinct vertices  $u, v \in V$  do
6:   if  $(u, v) \notin E$  then
7:      $E' \leftarrow E' \cup \{(u, v)\}$ 
8:   end if
9: end for
10:  $K' \leftarrow K$ 
11: return  $\langle G' = (V', E'), K' \rangle$ 

```

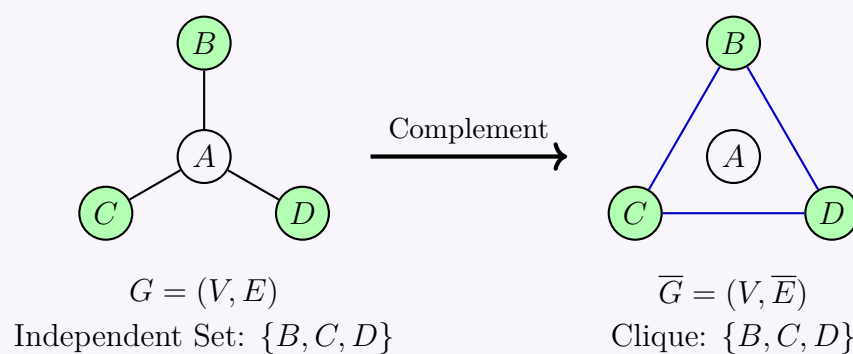
3. Complexity of the Reduction

- **Time Complexity:** $\mathcal{O}(|V|^2)$. The algorithm iterates through all possible pairs of vertices in V to construct the complement edge set. Checking non-adjacency and adding an edge takes $\mathcal{O}(1)$ time, yielding a quadratic total time bound.
- **Space Complexity:** $\mathcal{O}(|V|^2)$ to store the newly constructed complement graph G' as an adjacency matrix or list.

Both bounds are strictly polynomial, making this a valid polynomial-time transformation.

4. Visual Example of the Reduction

Below is a clear illustration of how an Independent Set in G directly maps to a Clique in the complement graph $\overline{G}$.



5. Proof of Correctness

We must prove that the mapping preserves the *YES* answer: G has an independent set of size $K \iff \overline{G}$ has a clique of size K .

- **Forward Direction ($\Rightarrow$):** Suppose G has an independent set $I \subseteq V$ of size K . By the definition of an independent set, no two vertices in I share an edge in E .

$$\begin{aligned}
 \forall u, v \in I \text{ where } u \neq v &\implies (u, v) \notin E \\
 &\implies (u, v) \in \overline{E} \quad (\text{by construction of } \overline{G})
 \end{aligned}$$

Because every pair of distinct vertices in I is connected by an edge in $\overline{G}$, the set I forms a clique of size K in $\overline{G}$.

- **Backward Direction ($\Leftarrow$):** Suppose $\overline{G}$ has a clique $C \subseteq V'$ of size K . By the definition of a clique, every pair of distinct vertices in C shares an edge in $\overline{E}$.

$$\begin{aligned}
 \forall u, v \in C \text{ where } u \neq v &\implies (u, v) \in \overline{E} \\
 &\implies (u, v) \notin E \quad (\text{by construction of } \overline{G})
 \end{aligned}$$

Because no two vertices in C are adjacent in the original graph G , the set C forms an independent set of size K in G .

6. Conclusion

We have shown a valid polynomial-time reduction from INDEPENDENT SET to CLIQUE. Since the existence of a solution is flawlessly preserved bi-directionally, solving the CLIQUE problem gives a direct solution to the INDEPENDENT SET problem. Therefore, the CLIQUE problem is at least as hard as the INDEPENDENT SET problem (NP-Hard).

Q2. In the *Set Cover* problem (decision version), we are given a set of elements U (the universe), a collection of subsets $S = \{S_1, S_2, \dots, S_m\}$ where each $S_i \subseteq U$, and an integer k . The goal is to determine whether there exists a sub-collection $C \subseteq S$ of size $\leq k$ such that $\bigcup_{S_i \in C} S_i = U$. Prove that the set cover problem is NP-complete.

Hint: Given a graph $G = (V, E)$ and an integer k , the vertex cover problem (decision version) is to determine whether there is a subset of vertices $V' \subseteq V$ with $|V'| \leq k$ such that every edge in E is incident to at least one vertex in V' . This problem is known to be NP-complete. **(15 marks)**

[CSE 207 | L-2/T-1 | 2023–24]

Answer

To prove that the SET COVER problem is NP-complete, we must satisfy two formal conditions:

- (a) **NP Membership:** Prove that a proposed solution (certificate) can be verified in polynomial time.
- (b) **NP-Hardness:** Prove that a known NP-complete problem can be reduced to SET COVER in polynomial time. We will use the VERTEX COVER problem for this reduction, as suggested.

1. Proof of NP Membership

Given a candidate sub-collection of sets $C \subseteq S$ as a certificate for a SET COVER instance, a deterministic polynomial-time verification algorithm can operate as follows:

- (a) Check if the size of the sub-collection satisfies $|C| \leq k$. If not, return **False**.
- (b) Compute the union of all subsets contained in C : $U_{\text{verified}} = \bigcup_{S_i \in C} S_i$.
- (c) Verify whether $U_{\text{verified}} = U$.

Let $n = |U|$ and $m = |S|$. The sub-collection C contains at most m subsets, and each subset has at most n elements. Computing the union and checking membership takes at most $\mathcal{O}(m \cdot n)$ operations, which is polynomial in the size of the input. Thus, SET COVER $\in$ NP.

2. Polynomial-Time Reduction from Vertex Cover

We now construct a polynomial-time mapping from an instance of the VERTEX COVER problem to an instance of the SET COVER problem.

2.1 Construction Mapping

Let $\langle G = (V, E), k \rangle$ be an arbitrary instance of the VERTEX COVER problem, where $V = \{v_1, v_2, \dots, v_n\}$ and $E = \{e_1, e_2, \dots, e_p\}$. We construct an equivalent SET COVER instance $\langle U, S, k' \rangle$ as follows:

- **Universe (U):** The elements to be covered are the edges of the graph G . Therefore, $U = E$.
- **Collection of Subsets (S):** For each vertex $v_i \in V$, we construct a corresponding subset $S_i \in S$. The subset S_i contains all the edges incident to v_i in G :

$$S_i = \{e \in E \mid e \text{ is incident to } v_i\}$$

The collection is $S = \{S_1, S_2, \dots, S_n\}$, yielding $|S| = |V|$.

- **Target Bound (k'):** The allowed budget for the set cover is chosen to be identical to the vertex cover budget, so $k' = k$.

2.2 Formal Reduction Algorithm

Algorithm 53 Reduction from Vertex Cover to Set Cover

```

1: Input: Graph  $G = (V, E)$ , Integer  $k$ 
2: Output: Universe  $U$ , Collection  $S$ , Integer  $k'$ 
3:  $U \leftarrow E$  ▷ The elements to cover are the graph edges
4:  $S \leftarrow \emptyset$ 
5: for each vertex  $v_i \in V$  do
6:    $S_i \leftarrow \{e \in E \mid e \text{ is incident to } v_i\}$ 
7:    $S \leftarrow S \cup \{S_i\}$ 
8: end for
9:  $k' \leftarrow k$ 
10: return  $\langle U, S, k' \rangle$ 

```

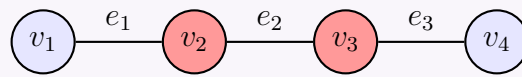
2.3 Complexity Analysis of the Reduction

- **Time Complexity:** For each vertex, we examine all incident edges to construct the corresponding subset. In an adjacency list representation, this requires traversing the entire graph exactly once, which takes $\mathcal{O}(|V| + |E|)$ time. This is strictly polynomial.

- **Space Complexity:** The universe has size $|E|$ and the total number of elements stored across all sets in S is $\sum |S_i| = 2|E|$, since every edge is incident to exactly two vertices. Thus, the space complexity is $\mathcal{O}(|V| + |E|)$.

3. Concrete Visual Example

Consider a simple graph G with vertices $V = \{v_1, v_2, v_3, v_4\}$ and edges $E = \{e_1, e_2, e_3\}$ where $e_1 = (v_1, v_2)$, $e_2 = (v_2, v_3)$, and $e_3 = (v_3, v_4)$. Let $k = 2$.



Graph G with Vertex Cover $V' = \{v_2, v_3\}$

Following our reduction, the generated instance of SET COVER is:

- $U = \{e_1, e_2, e_3\}$
- $S_1 = \{e_1\}, \quad S_2 = \{e_1, e_2\}, \quad S_3 = \{e_2, e_3\}, \quad S_4 = \{e_3\}$
- $k' = 2$

Selecting the subset collection $C = \{S_2, S_3\}$ ensures that $S_2 \cup S_3 = \{e_1, e_2, e_3\} = U$, which uses $|C| = 2 \leq k'$. This matches our vertex cover $V' = \{v_2, v_3\}$.

4. Proof of Correctness

We must show that G has a vertex cover of size at most k if and only if the constructed universe U has a set cover of size at most k .

4.1 Forward Direction ($\Rightarrow$)

Suppose G has a vertex cover $V' \subseteq V$ such that $|V'| \leq k$. By definition, for every edge $e \in E$, at least one of its endpoints belongs to V' .

Let us choose our sub-collection $C \subseteq S$ using the indices from the vertex cover:

$$C = \{S_i \mid v_i \in V'\}$$

Clearly, $|C| = |V'| \leq k = k'$. We verify that C forms a valid set cover for U :

$$\begin{aligned}
 \text{Let } e \in U. &\Rightarrow e \in E \\
 &\Rightarrow \exists v_i \in V' \text{ such that } e \text{ is incident to } v_i \quad (\text{since } V' \text{ is a vertex cover}) \\
 &\Rightarrow e \in S_i \quad (\text{by construction of } S_i) \\
 &\Rightarrow e \in \bigcup_{S_j \in C} S_j
 \end{aligned}$$

Since this holds for every element $e \in U$, we conclude that $\bigcup_{S_j \in C} S_j = U$. Hence, C is a valid set cover of size $\leq k'$.

4.2 Backward Direction ($\Leftarrow$)

Conversely, suppose there exists a sub-collection $C \subseteq S$ such that $|C| \leq k'$ and $\bigcup_{S_i \in C} S_i = U$. We select our vertex set $V' \subseteq V$ as:

$$V' = \{v_i \in V \mid S_i \in C\}$$

Clearly, $|V'| = |C| \leq k' = k$. We verify that V' forms a valid vertex cover for G :

$$\begin{aligned}
 \text{Let } e \in E. &\Rightarrow e \in U \\
 &\Rightarrow \exists S_i \in C \text{ such that } e \in S_i \quad (\text{since } C \text{ is a valid set cover}) \\
 &\Rightarrow e \text{ is incident to vertex } v_i \in V \quad (\text{by construction of } S_i) \\
 &\Rightarrow v_i \in V' \quad (\text{by definition of } V')
 \end{aligned}$$

Since every edge $e \in E$ is incident to at least one vertex in V' , the subset V' is a valid vertex cover of size $\leq k$.

5. Conclusion

We have shown that SET COVER is in NP, and that VERTEX COVER $\leq_p$ SET COVER. Because VERTEX COVER is an established NP-complete problem, it follows that SET COVER is NP-hard. Combining both parts, the SET COVER problem is NP-complete.

Q3. Imagine you are managing a team of employees, each with their own set of tasks to complete. However, some tasks cannot be worked on simultaneously due to dependencies or resource constraints. Your goal is to maximize productivity by scheduling as many tasks as possible while ensuring that no two conflicting tasks are assigned to the same team member simultaneously. You described the problem to your friend. He commented, “The problem can be mapped to a popular NP-complete problem and can be proved to be NP-hard by reduction technique.”

Justify your friend’s comment by mapping the problem to an NP-complete problem and reducing it to another known NP-complete

problem. (17 marks)

[CSE 207 | L-2/T-1 | 2022–23]

Answer

The problem described by your friend is a classic constraint satisfaction and optimization problem that maps directly to the **Maximum Independent Set (MIS)** problem. To justify your friend's comment, we will formally define the mapping, state the decision version of the problem, and provide a rigorous mathematical proof of its NP-completeness by reducing the known NP-complete **Clique** problem to it.

1. Problem Mapping and Formulation

We can model the task scheduling problem as an undirected graph $G = (V, E)$:

- **Vertices (V):** Each vertex $v \in V$ represents a specific task.
- **Edges (E):** An edge $e = (u, v) \in E$ exists if and only if tasks u and v conflict (i.e., they have dependencies or resource constraints and cannot be scheduled simultaneously).

Objective: Maximize the number of scheduled tasks such that no two tasks conflict. In graph-theoretic terms, we want to find the largest subset of vertices $S \subseteq V$ such that no two vertices in S share an edge. This is precisely the **Maximum Independent Set** problem.

To discuss NP-completeness, we formulate its **Decision Version (Independent Set)**:

Given a graph $G = (V, E)$ and an integer k , does there exist a subset $S \subseteq V$ such that $|S| \geq k$ and for all $u, v \in S$, $(u, v) \notin E$?

2. Proof of NP-Completeness

To prove that Independent Set (IS) is NP-complete, we must show two things:

- IS $\in$ NP
- A known NP-complete problem can be reduced to IS in polynomial time (we will use **Clique** $\leq_p$ **IS**).

Step 2.1: NP Membership (IS $\in$ NP)

To show IS $\in$ NP, we must demonstrate that a proposed solution (certificate) can be verified in polynomial time.

- **Certificate:** A proposed subset of vertices $S \subseteq V$.
- **Verification:** 1. Count the vertices in S to ensure $|S| \geq k$. This takes $O(|V|)$ time. 2. For every pair of vertices $u, v \in S$, check that $(u, v) \notin E$. Since there are at most $\frac{|V|(|V|-1)}{2}$ pairs, this check takes $O(|V|^2)$ time.

Since the verification algorithm runs in strictly polynomial time ($O(|V|^2)$), we conclude that IS $\in$ NP.

Step 2.2: Polynomial Time Reduction (Clique $\leq_p$ IS)

We reduce from the **Clique** problem, which is known to be NP-complete.

Clique Problem: Given a graph $G = (V, E)$ and an integer k , does G contain a complete subgraph (clique) of size $\geq k$?

Construction: Given an instance $\langle G = (V, E), k \rangle$ of the Clique problem, we construct an instance $\langle \bar{G} = (V, \bar{E}), k \rangle$ for the Independent Set problem, where $\bar{G}$ is the *complement graph* of G .

Algorithm 54 Polynomial Time Reduction: $\text{Clique}(G, k) \rightarrow \text{IS}(\bar{G}, k)$

Require: Graph $G = (V, E)$, integer k

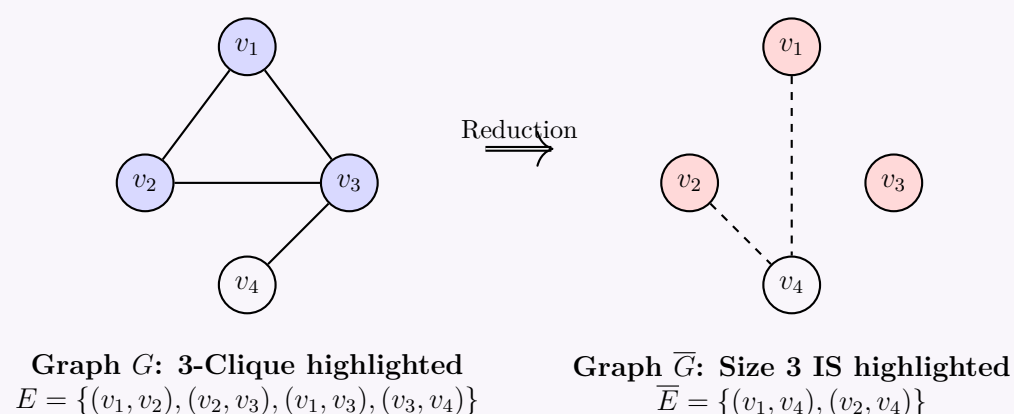
Ensure: Complement Graph $\bar{G} = (V, \bar{E})$, integer k

```

1:  $\bar{E} \leftarrow \emptyset$ 
2: for each pair of distinct vertices  $u, v \in V$  do
3:   if  $(u, v) \notin E$  then
4:      $\bar{E} \leftarrow \bar{E} \cup \{(u, v)\}$ 
5:   end if
6: end for
7: return  $\langle \bar{G} = (V, \bar{E}), k \rangle$ 

```

Complexity of Reduction: The algorithm iterates through all possible vertex pairs. Thus, the time complexity is $O(|V|^2)$ and space complexity is $O(|V|^2)$, meaning the construction is achieved in polynomial time.



Step 2.3: Proof of Correctness

We must prove that G has a clique of size k if and only if $\overline{G}$ has an independent set of size k .

Forward Direction ($\implies$): Assume G has a clique $C \subseteq V$ of size k . By the definition of a clique, for every pair of distinct vertices $u, v \in C$, the edge (u, v) is in E . By the construction of the complement graph $\overline{G}$, an edge exists in $\overline{E}$ if and only if it does not exist in E . Therefore, for every pair $u, v \in C$, the edge $(u, v) \notin \overline{E}$. By definition, C forms an independent set of size k in $\overline{G}$.

Backward Direction ($\impliedby$): Assume $\overline{G}$ has an independent set $S \subseteq V$ of size k . By the definition of an independent set, for every pair of distinct vertices $u, v \in S$, the edge $(u, v) \notin \overline{E}$. By the construction of $\overline{G}$, any edge missing in $\overline{E}$ must inherently be present in E . Therefore, for every pair $u, v \in S$, the edge $(u, v) \in E$. By definition, S forms a clique of size k in G .

3. Conclusion

Since we have shown that the Independent Set problem is in NP, and that the NP-complete Clique problem can be reduced to it in polynomial time, we have successfully proven that the Independent Set problem is **NP-complete**. Because your optimization problem (maximizing non-conflicting tasks) asks for the *maximum* Independent Set, it is inherently **NP-hard**. Your friend's comment is mathematically justified.

Q4. Suppose you are a developer of a social media platform. The platform wants to add a feature in the admin section. The feature is to identify the minimal subset of users who can reach out to all other users of the platform through their (selected subset users') connections. Now you have to formulate an algorithm to develop the feature. Your colleague commented that the problem can be mapped to a popular np- complete problem.

- (a) **[Minimal subset reachability problem]** Map the problem to the NP-complete problem that your colleague suggested. (5 marks) [CSE 207 | L-2/T-2 | 2021–22]
- (b) **[Minimal subset reachability problem]** Prove that the problem is NP-complete. You have to use the 3-SAT problem in the proof. (10 marks) [CSE 207 | L-2/T-2 | 2021–22]

Answer**Part (a): Minimal Subset Reachability Problem Mapping**

The problem of identifying a minimal subset of users who can reach out to all other users through their connections maps directly to the **Minimum Dominating Set** problem in graph theory.

Problem Formulation: Let the social media platform be represented as an undirected graph $G = (V, E)$, where:

- **Vertices (V):** Each vertex $v \in V$ represents a user on the platform.
- **Edges (E):** An edge $(u, v) \in E$ represents a connection (friendship/follow) between user u and user v .

A subset $D \subseteq V$ is called a *Dominating Set* if every vertex $v \in V \setminus D$ is adjacent to at least one vertex in D . The objective is to find a Dominating Set D such that its size $|D|$ is minimized. For the purpose of NP-completeness, we consider its decision version:

Dominating Set (DS) Decision Problem: Given a graph $G = (V, E)$ and an integer $k \geq 0$, does there exist a dominating set $D \subseteq V$ such that $|D| \leq k$?

Part (b): Proof of NP-Completeness using 3-SAT

To prove that the Dominating Set problem is NP-complete, we must show two properties:

- (a) The problem belongs to the class NP.
- (b) A known NP-complete problem (in this case, 3-SAT) can be reduced to it in polynomial time.

Step 1: NP Membership (DS $\in$ NP)

To show that DS is in NP, we must provide a polynomial-time verification algorithm for a given certificate.

- **Certificate:** A subset of vertices $D \subseteq V$.
- **Verification:**
 - Check if $|D| \leq k$. This takes $O(|V|)$ time.
 - For each vertex $v \in V$, verify that either $v \in D$ or there exists a neighbor u of v such that $u \in D$. This requires traversing the adjacency list, taking $O(|V| + |E|)$ time.

Since the verification is completed in strictly polynomial time $O(|V| + |E|)$, Dominating Set $\in$ NP.

Step 2: Polynomial-Time Reduction (3-SAT $\leq_p$ DS)

We map a given instance of 3-SAT to an instance of the Dominating Set problem. Let ϕ be a boolean formula in 3-Conjunctive Normal Form (3-CNF) with n variables $X = \{x_1, x_2, \dots, x_n\}$ and m clauses $C = \{c_1, c_2, \dots, c_m\}$. Each clause contains exactly 3 literals. We construct a graph $G = (V, E)$ and set a target size k as follows:

Algorithm 55 Reduction from 3-SAT to Dominating Set**Require:** 3-CNF formula ϕ with variables X and clauses C **Ensure:** Graph $G = (V, E)$ and integer k

```

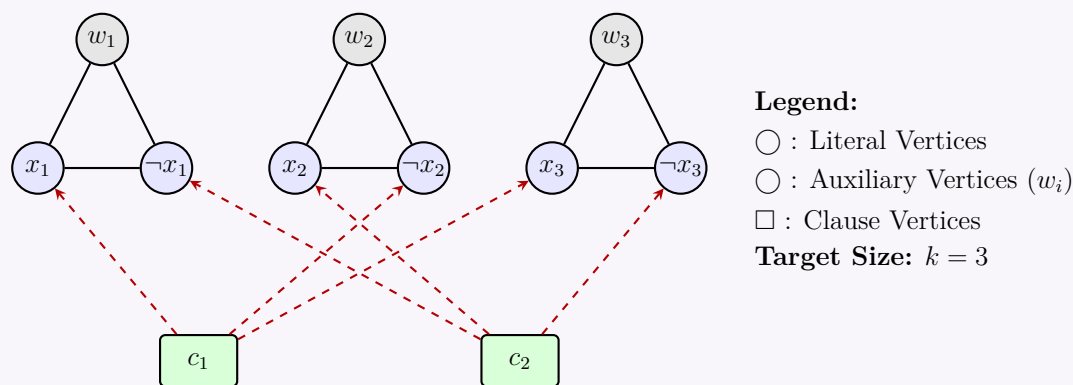
1:  $V \leftarrow \emptyset, E \leftarrow \emptyset$ 
2:  $k \leftarrow |X|$  ▷ Budget is exactly the number of variables
3: for each variable  $x_i \in X$  do
4:   Add vertices  $v_i$  (for  $x_i$ ),  $\bar{v}_i$  (for  $\neg x_i$ ), and a structural node  $w_i$  to  $V$ 
5:   Add edges  $(v_i, \bar{v}_i), (v_i, w_i), (\bar{v}_i, w_i)$  to  $E$  ▷ Forms a triangle
6: end for
7: for each clause  $c_j \in C$  do
8:   Add vertex  $u_j$  to  $V$ 
9:   for each literal  $l$  in  $c_j$  do
10:    if  $l$  is the positive literal  $x_i$  then
11:      Add edge  $(u_j, v_i)$  to  $E$ 
12:    else if  $l$  is the negative literal  $\neg x_i$  then
13:      Add edge  $(u_j, \bar{v}_i)$  to  $E$ 
14:    end if
15:   end for
16: end for
17: return  $\langle G = (V, E), k \rangle$ 

```

Complexity of Reduction: The construction iterates through n variables and m clauses. Creating the vertices and edges takes $O(n + m)$ time and space. Thus, the reduction is strictly polynomial.

Visualization of the Reduction

Below is a TikZ diagram demonstrating the reduction for $\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$.

**Step 3: Proof of Correctness**

We must prove that the formula ϕ is satisfiable *if and only if* the graph G has a dominating set D of size $k = n$.

Forward Direction ($\Rightarrow$): Satisfiable $\Rightarrow$ Dominating Set of size n

- Suppose ϕ has a satisfying truth assignment. We construct a set D by picking exactly one vertex from each variable gadget:
- If x_i is True, add v_i to D . If x_i is False, add $\bar{v}_i$ to D .
- Clearly, $|D| = n$.
- *Domination of variables:* For each i , the chosen literal dominates itself, the other literal, and w_i . Thus, all variable gadget vertices are covered.
- *Domination of clauses:* Because the assignment satisfies ϕ , every clause c_j contains at least one True literal. By our construction, the corresponding True literal vertex is in D . Since u_j is connected to its literals, u_j is dominated by D .
- Therefore, D is a valid dominating set of size n .

Backward Direction ($\Leftarrow$): Dominating Set of size $n \Rightarrow$ Satisfiable

- Suppose G has a dominating set D of size n .
- To dominate the isolated structural node w_i , D must contain at least one vertex from the triangle $\{v_i, \bar{v}_i, w_i\}$. Since there are n such disjoint triangles and $|D| = n$, D must contain **exactly one** vertex from each triangle, and no clause vertices u_j .
- If D contains w_i , we can swap it out for v_i without losing domination over the triangle $\{v_i, \bar{v}_i, w_i\}$ (and potentially gaining domination over some clauses). Thus, without loss of generality, we can assume D consists entirely of literal vertices (v_i or $\bar{v}_i$).
- Since D contains exactly one of v_i or $\bar{v}_i$ for each i , it maps to a well-defined boolean truth assignment ($x_i = \text{True}$ if $v_i \in D$, else False).
- Since D is a dominating set, every clause vertex u_j must be adjacent to at least one vertex in D . This means for every clause c_j , at least one of its literals is assigned True.

- Therefore, the truth assignment satisfies all clauses, meaning ϕ is satisfiable.

Conclusion: Because Dominating Set is in NP and 3-SAT (an NP-complete problem) can be polynomially reduced to it, the Dominating Set problem (and thus the Minimal Subset Reachability problem) is NP-complete.

Q5. Explain how we can prove that an optimization problem is NP-complete. (5 marks)

[CSE 207 | L-2/T-2 | 2020–21]

Answer

From a strict theoretical perspective, the complexity classes P, NP, and NP-complete are defined exclusively for **decision problems**—problems whose outputs are a simple binary YES or NO. An **optimization problem** seeks to find the *best* solution among a set of feasible options (minimizing or maximizing an objective function) and therefore cannot be strictly classified as NP-complete. To show that an optimization problem is computationally intractable, we use a standard four-step pedagogical paradigm: we convert the optimization problem into its corresponding decision variant, prove that this decision variant is NP-complete, and establish that the optimization problem is at least as hard (**NP-hard**).

The 4-Step Proof Framework

To establish the hardness of an optimization problem $\mathcal{O}$, follow these precise mathematical steps:

Step 1: Formulate the Decision Variant ($\mathcal{D}$)

Convert the optimization objective into a threshold constraint.

- If $\mathcal{O}$ is a **minimization problem** (e.g., finding the shortest path or minimum cost), the decision variant $\mathcal{D}$ asks: *"Is there a feasible solution with a cost $\leq k$?"*
- If $\mathcal{O}$ is a **maximization problem** (e.g., finding the maximum independent set or profit), the decision variant $\mathcal{D}$ asks: *"Is there a feasible solution with a value $\geq k$?"*

Step 2: Prove NP-Membership ($\mathcal{D} \in \text{NP}$)

Show that the decision variant $\mathcal{D}$ belongs to the class NP. This requires demonstrating that a given candidate solution (certificate y) can be verified by a deterministic algorithm in polynomial time:

- Certificate:** Define what the certificate y contains (e.g., a list of vertices, a sequence of edges, or a partition of a set).
- Verification Algorithm:** Describe a deterministic algorithm $V(x, y)$ that takes the problem instance x and certificate y as inputs and checks:
 - Whether y is syntactically valid and represents a feasible solution.
 - Whether the objective value of y satisfies the threshold constraint ($\leq k$ or $\geq k$).
- Complexity Analysis:** Explicitly demonstrate that the verification steps run in $O(n^c)$ time, where $n = |x|$ and c is a constant.

Step 3: Prove NP-Hardness via Polynomial-Time Reduction ($\mathcal{L} \leq_p \mathcal{D}$)

Select a known NP-complete problem $\mathcal{L}$ (such as 3-SAT, Vertex Cover, or Clique) and construct a polynomial-time reduction to your decision variant $\mathcal{D}$.

- **The Mapping function (f):** Design an algorithm that transforms any arbitrary instance $I_{\mathcal{L}}$ of the known problem into a specific instance $I_{\mathcal{D}}$ of your problem. Prove that this transformation takes $O(n^d)$ time.
- **Forward Correctness ($\implies$):** Prove that if $I_{\mathcal{L}}$ is a YES instance, then $I_{\mathcal{D}}$ must also be a YES instance.
- **Backward Correctness ($\impliedby$):** Prove that if $I_{\mathcal{D}}$ is a YES instance, then $I_{\mathcal{L}}$ must also be a YES instance (often proved via contrapositive: $\neg I_{\mathcal{L}} \implies \neg I_{\mathcal{D}}$).

Completing Steps 2 and 3 formally establishes that $\mathcal{D}$ is **NP-complete**.

Step 4: Establish NP-Hardness of the Optimization Problem ($\mathcal{D} \leq_p \mathcal{O}$)

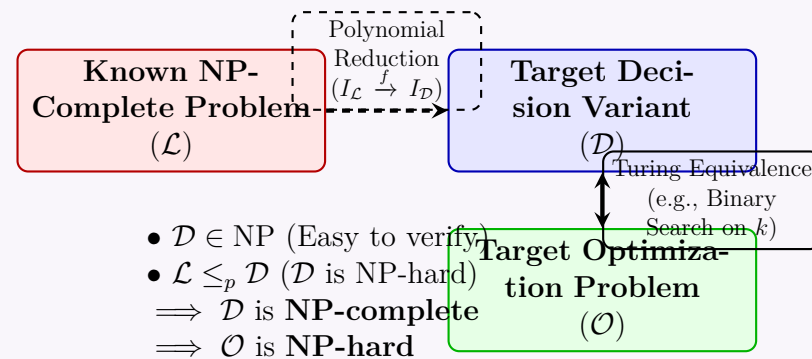
Finally, demonstrate that if you had a black-box solver (oracle) capable of solving the optimization problem $\mathcal{O}$ in a single step, you could use it to solve the decision problem $\mathcal{D}$ in polynomial time.

- Simply query the optimization oracle for the instance. If the optimal value returned meets or beats the threshold k , output YES; otherwise, output NO.

Since $\mathcal{D}$ is NP-complete and $\mathcal{D} \leq_p \mathcal{O}$, the optimization problem $\mathcal{O}$ is formally classified as **NP-hard**.

Structural Relationship and Reduction Flow

The following diagram illustrates how the components interface with one another. The polynomial-time reduction links the known hard problem to the decision variant, while a Turing reduction (often implemented via binary search over the parameter space k) establishes the equivalence between the decision and optimization variants.



Concrete Example: Traveling Salesperson Problem (TSP)

To illustrate this framework clearly, let us look at the classic Traveling Salesperson Problem:

Optimization Variant ($\mathcal{O}_{TSP}$)	Decision Variant ($\mathcal{D}_{TSP}$)
Given a complete graph $G = (V, E)$ with edge weights $w(e)$, find a Hamiltonian cycle that minimizes the total edge weight.	Given a complete graph $G = (V, E)$ with edge weights $w(e)$ and a threshold k , is there a Hamiltonian cycle with total weight $\leq k$?

- NP-Membership:** A certificate y is an ordered sequence of vertices. Verification checks if the sequence visits every vertex exactly once, returns to the start, and sums the weights to verify if Total Cost $\leq k$. This runs in linear time $O(|V|)$, proving $\mathcal{D}_{TSP} \in \text{NP}$.
- NP-Hardness Reduction:** We reduce from the standard **Hamiltonian Cycle Problem** ($\mathcal{L}$). Given an arbitrary graph G , we construct a complete graph G' where existing edges retain weight 1 and missing edges are given a massive penalty weight $M > |V|$. We set the threshold $k = |V|$.
- Correctness:** G has a Hamiltonian cycle if and only if G' has a TSP tour of total weight $\leq |V|$. This proves $\mathcal{D}_{TSP}$ is NP-complete, implying $\mathcal{O}_{TSP}$ is NP-hard.

Q6. For a graph $G = (V, E)$, a set S of vertices in G is a *vertex cover* if every edge e in G has at least one end in S . Given a graph G and a number K , the vertex cover problem asks if G contains a vertex cover of size at most K . Given a set U of n elements, a collection $S_1, S_2, \dots, S_m$ of subsets of U , and a number k , the set cover problem asks if there exists a collection of at most k of these subsets whose union is equal to U . Prove that the set cover problem is at least as hard as the vertex cover problem. **(12 marks)** [CSE 207 | L-2/T-2 | 2019–20]

Answer

To prove that the **Set Cover (SC)** problem is at least as hard as the **Vertex Cover (VC)** problem, we must demonstrate a polynomial-time reduction from Vertex Cover to Set Cover, denoted as $\text{VC} \leq_p \text{SC}$. Since Vertex Cover is a known NP-Complete problem, proving this reduction establishes that Set Cover is NP-Hard (and since Set Cover is in NP, it is also NP-Complete).

1. Polynomial-Time Reduction Construction

Let us take an arbitrary instance of the Vertex Cover problem.

- Given:** A graph $G = (V, E)$ and an integer K .
- Question:** Does there exist a subset $V' \subseteq V$ such that $|V'| \leq K$ and every edge $e \in E$ is incident to at least one vertex in V' ?

We construct an instance of the Set Cover problem as follows:

- Universe (U):** We must cover all edges in the graph. Thus, we set the universe of elements to be the set of edges: $U = E$.
- Subsets ($\mathcal{S}$):** A vertex in a graph "covers" its incident edges. For each vertex $v \in V$, we create a subset S_v containing all edges incident to v . Therefore, our collection of subsets is $\mathcal{S} = \{S_v \mid v \in V\}$.
- Parameter (k):** We set the maximum number of allowed subsets in the set cover to be exactly the maximum allowed size of the vertex cover: $k = K$.

Algorithm 56 Reduction from Vertex Cover to Set Cover**Require:** Graph $G = (V, E)$, Integer K **Ensure:** Set Cover Instance $(U, \mathcal{S}, k)$

```

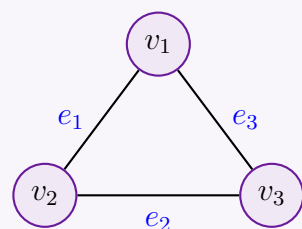
1:  $U \leftarrow E$  ▷ Universe is the set of edges
2:  $\mathcal{S} \leftarrow \emptyset$ 
3: for each  $v \in V$  do
4:    $S_v \leftarrow \emptyset$ 
5:   for each edge  $e \in E$  incident to  $v$  do
6:      $S_v \leftarrow S_v \cup \{e\}$ 
7:   end for
8:    $\mathcal{S} \leftarrow \mathcal{S} \cup \{S_v\}$ 
9: end for
10:  $k \leftarrow K$ 
11: return  $(U, \mathcal{S}, k)$ 

```

Time Complexity of Reduction: Building the universe takes $O(|E|)$ time. Creating the subsets requires iterating over all vertices and their incident edges, taking $O(|V| + |E|)$ time. Setting k takes $O(1)$ time. Thus, the total reduction takes $O(|V| + |E|)$ time, which is strictly polynomial.

2. Visualizing the Transformation

To build intuition, consider a simple graph and its corresponding Set Cover instance generated by our reduction.

Vertex Cover Instance**Set Cover Instance**

$$U = \{e_1, e_2, e_3\}$$

$$S_{v_1} = \{e_1, e_3\}$$

$$S_{v_2} = \{e_1, e_2\}$$

$$S_{v_3} = \{e_2, e_3\}$$

Reduction

3. Proof of Correctness

To complete the proof, we must show that the original graph G has a Vertex Cover of size $\leq K$ **if and only if** the constructed instance $(U, \mathcal{S}, k)$ has a Set Cover of size $\leq k$.

Forward Direction: VC $\implies$ SC

Claim: If G has a vertex cover V' of size $\leq K$, then $\mathcal{S}$ contains a set cover $\mathcal{C}$ of size $\leq k$.

- Let $V' \subseteq V$ be a valid vertex cover such that $|V'| \leq K$.
- We construct our set cover $\mathcal{C}$ by choosing the subsets corresponding to the vertices in V' . Thus, $\mathcal{C} = \{S_v \mid v \in V'\}$.
- By definition of a vertex cover, every edge $e = (u, w) \in E$ has at least one endpoint in V' (either $u \in V'$ or $w \in V'$).
- In our reduction, $e \in S_u$ and $e \in S_w$. Since at least one of u or w is in V' , at least one of S_u or S_w is in $\mathcal{C}$.
- Therefore, the element e is covered by the union of sets in $\mathcal{C}$. Since this holds true for all edges $e \in E = U$, $\mathcal{C}$ is a valid set cover for U .
- Because $|V'| \leq K$ and we mapped each vertex one-to-one to a subset, $|\mathcal{C}| \leq K = k$.

Backward Direction: SC $\implies$ VC

Claim: If $\mathcal{S}$ contains a set cover $\mathcal{C}$ of size $\leq k$, then G has a vertex cover V' of size $\leq K$.

- Let $\mathcal{C} \subseteq \mathcal{S}$ be a valid set cover such that $|\mathcal{C}| \leq k$.
- We construct our vertex cover V' by choosing the vertices that correspond to the subsets in $\mathcal{C}$. Thus, $V' = \{v \mid S_v \in \mathcal{C}\}$.
- Because $\mathcal{C}$ is a valid set cover, the union of its subsets contains every element in U . Since $U = E$, every edge $e \in E$ is present in at least one subset $S_v \in \mathcal{C}$.
- By the definition of our reduction, an element e is in S_v if and only if edge e is incident to vertex v in G .
- Therefore, for every edge $e \in E$, there exists some $v \in V'$ such that e is incident to v . This exactly satisfies the definition of a vertex cover.
- Since $|\mathcal{C}| \leq k$ and $k = K$, the size of our vertex cover $|V'| \leq K$.

4. Conclusion

We have successfully provided a polynomial-time reduction from Vertex Cover to Set Cover, and mathematically proven that a solution exists for the Vertex Cover instance if and only if a solution exists for the Set Cover instance. Therefore, the Set Cover problem is at least as hard as the Vertex Cover problem.

Q7. A vertex cover of a graph $G = (V, E)$ is a subset of vertices $S \subseteq V$ that includes at least one endpoint of every edge in E . Give a linear-time algorithm for the following task.
Input: An undirected tree $T = (V, E)$
Output: The size of the smallest vertex cover of T . (13 marks) [CSE 207 | L-2/T-2 | 2019–20]

Answer

To find the size of the smallest vertex cover (Minimum Vertex Cover) in an undirected tree $T = (V, E)$ in linear time, we can use **Dynamic Programming (DP) on Trees**. While the Minimum Vertex Cover problem is NP-Hard for general graphs, the hierarchical structure of a tree allows us to solve it optimally in $O(|V|)$ time by converting it into a rooted tree and computing optimal substructures from the leaves up to the root.

1. Dynamic Programming Formulation

We arbitrarily root the tree T at any vertex (e.g., vertex r). For any subtree rooted at a node u , we define two states:

- $DP[u][0]$: The size of the minimum vertex cover for the subtree rooted at u , given that u is **not included** in the vertex cover.
- $DP[u][1]$: The size of the minimum vertex cover for the subtree rooted at u , given that u is **included** in the vertex cover.

Transitions:

(a) **If u is not included ($DP[u][0]$):** To cover the edges between u and its children, *every* child v of u **must** be included in the vertex cover. Thus:

$$DP[u][0] = \sum_{v \in \text{children}(u)} DP[v][1]$$

(b) **If u is included ($DP[u][1]$):** The edges between u and its children are already covered by u . Therefore, each child v can independently either be included or not included in the vertex cover. We take the optimal (minimum) choice for each child. Since u itself is included, we add 1:

$$DP[u][1] = 1 + \sum_{v \in \text{children}(u)} \min(DP[v][0], DP[v][1])$$

Base Case (Leaf Nodes): For any leaf node l :

$$DP[l][0] = 0 \quad \text{and} \quad DP[l][1] = 1$$

(Note: Our recurrence relation naturally handles leaf nodes since the sum over an empty set of children evaluates to 0).

2. Visualizing the DP State

Consider the following simple tree rooted at node 1. The optimal vertex cover consists of the filled nodes $\{2, 3\}$, yielding a minimum vertex cover size of 2.

Node u	$DP[u][0]$	$DP[u][1]$	Min VC Subtree
4 (Leaf)	0	1	0
2	1	1	0
3 (Leaf)	0	1	0
5	1	1	1
1 (Root)	2	2	2

3. Algorithm

The algorithm uses Depth-First Search (DFS) to process the tree in a post-order fashion, computing the values for the children before computing the values for the parent. We pass the ‘parent’ parameter to avoid traversing backward up the undirected tree.

304

Algorithm 57 Minimum Vertex Cover on a Tree**Require:** An undirected tree $T = (V, E)$ represented as an adjacency list.**Ensure:** The size of the minimum vertex cover.

```

1: Initialize  $DP[u][0] \leftarrow 0$  and  $DP[u][1] \leftarrow 0$  for all  $u \in V$ 
2: procedure DFS( $u, parent$ )
3:    $DP[u][0] \leftarrow 0$ 
4:    $DP[u][1] \leftarrow 1$  ▷ Cost of including node  $u$  itself
5:   for each neighbor  $v$  of  $u$  in  $T$  do
6:     if  $v \neq parent$  then ▷ Recursively compute for child
7:       DFS( $v, u$ )
8:        $DP[u][0] \leftarrow DP[u][0] + DP[v][1]$ 
9:        $DP[u][1] \leftarrow DP[u][1] + \min(DP[v][0], DP[v][1])$ 
10:    end if
11:  end for
12: end procedure
13: DFS( $root, NULL$ ) ▷ Arbitrarily pick any vertex as root, e.g., vertex 1
14: return  $\min(DP[root][0], DP[root][1])$ 

```

4. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|V|)$. The algorithm traverses the tree using a standard Depth-First Search. Because T is a tree, it has exactly $|V| - 1$ edges. Each edge is examined exactly twice (once from each endpoint), and the work done at each node (summing values from children) is proportional to its degree. The total sum of degrees in a tree is $2(|V| - 1)$. Therefore, the time complexity is strictly linear, bounded by $\mathcal{O}(|V|)$.
- **Space Complexity:** $\mathcal{O}(|V|)$. The space complexity is governed by two factors:
 - (a) The DP table of dimensions $|V| \times 2$, which takes $\mathcal{O}(|V|)$ space.
 - (b) The implicit call stack during the recursive DFS traversal. In the worst-case scenario (a skewed tree or linear chain), the recursion depth can reach $|V|$, taking $\mathcal{O}(|V|)$ stack space.

Thus, the overall space complexity is $\mathcal{O}(|V|)$.

Q8. The TAUTOLOGY problem asks if a given Boolean formula is true for all possible assignments to the Boolean variables. Prove or disprove that TAUTOLOGY is in co-NP. (6 marks) [CSE 207 | L-2/T-2 | 2018–19]

Answer

Claim: The statement is **True**. TAUTOLOGY is in co-NP.

We will strictly prove this claim by utilizing the formal definition of the complexity class co-NP and demonstrating the relationship between TAUTOLOGY and its complement.

1. Definition of the Class co-NP

By definition in computational complexity theory, a decision problem (or language) L belongs to the class **co-NP** if and only if its complement, denoted as $\bar{L}$, belongs to the class **NP**.

The class NP contains all decision problems for which a "yes" answer can be verified in polynomial time by a deterministic Turing machine, given a valid, polynomial-sized certificate (or witness). Therefore, to prove that $\text{TAUTOLOGY} \in \text{co-NP}$, we must demonstrate that the complement problem, $\overline{\text{TAUTOLOGY}}$, is in NP.

2. Analyzing the Complement: $\overline{\text{TAUTOLOGY}}$

The TAUTOLOGY problem asks if a formula is true for *all* possible assignments. Formally:

$$\text{TAUTOLOGY} = \{\phi \mid \phi \text{ is a Boolean formula such that } \forall \text{ assignments } x, \phi(x) = \text{True}\}$$

The complement of this problem, $\overline{\text{TAUTOLOGY}}$, asks whether a given Boolean formula is *not* a tautology. A formula is not a tautology if there exists *at least one* variable assignment that makes the formula evaluate to False.

$$\overline{\text{TAUTOLOGY}} = \{\phi \mid \phi \text{ is a Boolean formula such that } \exists \text{ an assignment } x \text{ where } \phi(x) = \text{False}\}$$

This problem is universally known as the **FALSIFIABILITY** problem (or Non-Tautology).

3. Proving $\overline{\text{TAUTOLOGY}} \in \text{NP}$

To prove that $\overline{\text{TAUTOLOGY}}$ is in NP, we must construct a polynomial-time verifier V that takes an instance of the problem and a certificate c , and outputs "yes" if and only if the instance belongs to $\overline{\text{TAUTOLOGY}}$.

- **Instance:** A Boolean formula ϕ containing n Boolean variables.
- **Certificate (c):** A specific truth assignment to the n variables of ϕ . Since there are n variables, a truth assignment can be represented by an n -bit string. Thus, the size of the certificate is $O(n)$, which is polynomial in the size of the input formula ϕ .

- **Verifier Algorithm (V):** Given the formula ϕ and the assignment c , the verifier simply evaluates the boolean expression.

Algorithm 58 Verifier for $\overline{\text{TAUTOLOGY}}$

Require: Boolean formula ϕ , truth assignment c

Ensure: **Accept** if ϕ is falsified by c , **Reject** otherwise

- 1: Evaluate the Boolean formula ϕ using the variable assignment c
 - 2: **if** the result of $\phi(c)$ is False **then**
 - 3: **return Accept**
 - 4: **else**
 - 5: **return Reject**
 - 6: **end if**
-

- **Time Complexity of Verifier:** Evaluating a Boolean formula ϕ given a complete truth assignment requires a single bottom-up pass of its abstract syntax tree (or parsing it left-to-right depending on the representation). The time taken is strictly proportional to the number of operators and variables in the formula. Hence, the evaluation takes deterministic $\mathcal{O}(|\phi|)$ time, which is linear (and thus polynomial) with respect to the input size.

4. Conclusion

We have shown that any "yes" instance of $\overline{\text{TAUTOLOGY}}$ can be verified in polynomial time using a polynomial-sized certificate (the falsifying assignment). Therefore, $\overline{\text{TAUTOLOGY}} \in \text{NP}$.

By the foundational definition of co-NP ($L \in \text{co-NP} \iff \bar{L} \in \text{NP}$), since the complement of TAUTOLOGY is in NP, it rigorously follows that **TAUTOLOGY is in co-NP**.

- Q9.** For a graph $G = (V, E)$, a set S of vertices in G is a *vertex cover* if every edge e in G has at least one end in S . A set of vertices I in G is an *independent set* if for every two vertices in I , there is no edge between them. Given a graph G and a number K , the vertex cover problem asks if G contains a vertex cover of size at most K . Given a graph G and a number K , the independent set problem asks if G contains an independent set of size at least K . Prove that the vertex cover problem is at least as hard as the independent set problem. **(12 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Answer

To prove that the **Vertex Cover (VC)** problem is at least as hard as the **Independent Set (IS)** problem, we must demonstrate a polynomial-time reduction from Independent Set to Vertex Cover, denoted as $\text{IS} \leq_p \text{VC}$.

The fundamental insight behind this reduction is the complementary relationship between independent sets and vertex covers in any given graph.

1. The Core Theorem

Theorem: Let $G = (V, E)$ be an undirected graph, and let $S \subseteq V$. The set S is an independent set if and only if its complement, $V \setminus S$, is a vertex cover.

2. Polynomial-Time Reduction Construction

We want to transform an arbitrary instance of the Independent Set problem into an instance of the Vertex Cover problem.

- **Given (IS Instance):** A graph $G = (V, E)$ and an integer K . We are asking if G has an independent set of size at least K .
- **Construct (VC Instance):** We pass the exact same graph $G' = G$, and we set the target vertex cover size to $K' = |V| - K$. We ask if G' has a vertex cover of size at most K' .

Algorithm 59 Reduction from Independent Set to Vertex Cover

Require: Graph $G = (V, E)$, Integer K

Ensure: Vertex Cover Instance (G', K')

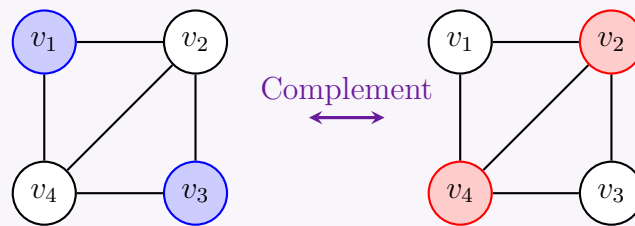
- 1: $G' \leftarrow G$ ▷ The graph structure remains identical
 - 2: $K' \leftarrow |V| - K$ ▷ Compute the complement size
 - 3: **return** (G', K')
-

Time Complexity of Reduction: Computing $|V| - K$ takes $\mathcal{O}(1)$ time. Passing the graph representation takes $\mathcal{O}(|V| + |E|)$ time (or $\mathcal{O}(1)$ if passed by reference). Thus, the reduction takes strictly polynomial time.

3. Visualizing the Relationship

Consider the graph below. The vertices in blue form an Independent Set of size 2. Their complement, the vertices in red, forms a Vertex Cover of size $4 - 2 = 2$.

Independent Set S ($|S| = K$) $\iff$ Vertex Cover $V \setminus S$ ($|V \setminus S| = 2$)



4. Proof of Correctness

To complete the proof, we must show that G has an independent set of size at least K **if and only if** G has a vertex cover of size at most $|V| - K$.

Forward Direction: IS $\implies$ VC

Claim: If $S \subseteq V$ is an independent set of size $\geq K$, then $C = V \setminus S$ is a vertex cover of size $\leq |V| - K$.

- Let S be an independent set such that $|S| \geq K$. Let $C = V \setminus S$.
- Consider any arbitrary edge $e = (u, v) \in E$.
- By the definition of an independent set, there are no edges between any two vertices in S . Therefore, it is impossible for both u and v to be in S .
- This implies that at least one of u or v must not be in S .
- Consequently, at least one of u or v must be in the complement set $C = V \setminus S$.
- Since this holds for every edge in E , C is a valid vertex cover.
- The size of C is $|C| = |V| - |S|$. Since $|S| \geq K$, we have $|C| \leq |V| - K$.

Backward Direction: VC $\implies$ IS

Claim: If $C \subseteq V$ is a vertex cover of size $\leq |V| - K$, then $S = V \setminus C$ is an independent set of size $\geq K$.

- Let C be a vertex cover such that $|C| \leq |V| - K$. Let $S = V \setminus C$.
- Assume, for the sake of contradiction, that S is *not* an independent set.
- This would mean there exists an edge $e = (u, v) \in E$ such that both $u \in S$ and $v \in S$.
- If both u and v are in S , then neither u nor v is in C (since S and C are disjoint complements).
- If neither endpoint of e is in C , then C fails to cover edge e , which contradicts the premise that C is a valid vertex cover.
- Thus, our assumption is false. There can be no edges between vertices in S , making S a valid independent set.
- The size of S is $|S| = |V| - |C|$. Since $|C| \leq |V| - K$, we have $|S| \geq |V| - (|V| - K) = K$.

5. Conclusion

We have provided a valid polynomial-time reduction from the Independent Set problem to the Vertex Cover problem. We rigorously proved that a "yes" instance of IS directly maps to a "yes" instance of VC and vice versa. Therefore, the Vertex Cover problem is at least as hard as the Independent Set problem.

Q10. What do you mean by $X \leq_p Y$? Show that the vertex cover problem $\leq_p$ set cover problem. (3+7 marks) [CSE 207 | L-2/T-2 | 2017–18]

Answer

Part 1: Meaning of $X \leq_p Y$

The notation $X \leq_p Y$ denotes a **Polynomial-Time Reduction** from decision problem X to decision problem Y . It means that problem X is "no harder than" problem Y , up to a polynomial factor.

Formally, $X \leq_p Y$ if there exists a deterministic polynomial-time computable function $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ such that for every input instance x :

$$x \in X \iff f(x) \in Y$$

In practical terms, this means that if we have a black-box algorithm (an oracle) that solves problem Y , we can use it to solve problem X in polynomial time by taking an instance x of X , transforming it into an instance $f(x)$ of Y in polynomial time, and returning the exact same "yes" or "no" answer.

Part 2: Proving Vertex Cover $\leq_p$ Set Cover

To prove that the Vertex Cover (VC) problem is polynomial-time reducible to the Set Cover (SC) problem, we must construct a polynomial-time mapping from any VC instance to an SC instance, and prove that the VC instance has a valid cover if and only if the mapped SC instance has a valid cover.

1. Reduction Construction

Let us take an arbitrary instance of the Vertex Cover problem:

- **Input:** A graph $G = (V, E)$ and an integer k .
- **Goal:** Does there exist a subset $V' \subseteq V$ of size at most k such that every edge in E has at least one endpoint in V' ?

We construct an equivalent instance of the Set Cover problem $(U, \mathcal{S}, k')$ as follows:

- **Universe (U):** We need to cover all edges, so the universe of elements is the set of all edges in G . Thus, $U = E$.
- **Subsets ($\mathcal{S}$):** A vertex in G "covers" its incident edges. For each vertex $v \in V$, we create a subset S_v containing exactly the edges incident to v . Thus, the collection of subsets is $\mathcal{S} = \{S_v \mid v \in V\}$.
- **Parameter (k'):** The maximum allowed number of sets is equal to the maximum allowed size of the vertex cover. Thus, $k' = k$.

Algorithm 60 Reduction Algorithm (f)

Require: Graph $G = (V, E)$, Integer k

Ensure: Set Cover Instance $(U, \mathcal{S}, k')$

```

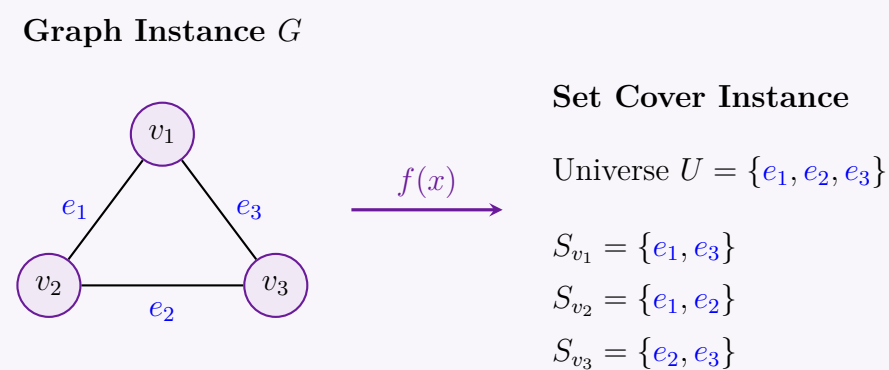
1:  $U \leftarrow E$ 
2:  $\mathcal{S} \leftarrow \emptyset$ 
3: for each  $v \in V$  do
4:    $S_v \leftarrow \{e \in E \mid e \text{ is incident to } v\}$ 
5:    $\mathcal{S} \leftarrow \mathcal{S} \cup \{S_v\}$ 
6: end for
7:  $k' \leftarrow k$ 
8: return  $(U, \mathcal{S}, k')$ 

```

Time Complexity Analysis: Constructing the universe takes $\mathcal{O}(|E|)$ time. Constructing the subsets requires iterating over all vertices and their adjacent edges, taking $\mathcal{O}(|V| + |E|)$ time. The overall transformation takes strictly polynomial time $\mathcal{O}(|V| + |E|)$.

2. Visualization of the Mapping

Consider the simple graph and its corresponding Set Cover instance generated by our reduction:



3. Proof of Correctness

We must mathematically prove that G has a vertex cover of size $\leq k$ if and only if $(U, \mathcal{S}, k')$ has a set cover of size $\leq k'$.

Forward Direction ($\implies$): Suppose G has a vertex cover $V' \subseteq V$ where $|V'| \leq k$.

- We choose the corresponding subsets for our set cover: $\mathcal{C} = \{S_v \mid v \in V'\}$.
- By the definition of a vertex cover, every edge $e = (u, w) \in E$ is incident to at least one vertex in V' (either $u \in V'$ or $w \in V'$).
- Consequently, in our SC instance, the element e is contained in at least one subset in $\mathcal{C}$ (either $S_u \in \mathcal{C}$ or $S_w \in \mathcal{C}$).
- Thus, $\mathcal{C}$ covers all elements in U . Since $|\mathcal{C}| = |V'| \leq k$, we have a valid set cover of size $\leq k'$.

Backward Direction ($\impliedby$): Suppose the constructed instance has a set cover $\mathcal{C} \subseteq \mathcal{S}$ where $|\mathcal{C}| \leq k'$.

- We select the corresponding vertices for our vertex cover: $V' = \{v \mid S_v \in \mathcal{C}\}$.
- Because $\mathcal{C}$ is a valid set cover, every element $e \in U$ is present in at least one subset $S_v \in \mathcal{C}$.
- Since $U = E$, this means every edge $e \in E$ is in some set $S_v \in \mathcal{C}$.
- By our construction, $e \in S_v$ if and only if edge e is incident to vertex v in G .
- Thus, every edge $e \in E$ is incident to at least one vertex in V' . Since $|V'| = |\mathcal{C}| \leq k'$, we have a valid vertex cover of size $\leq k$.

Since the reduction operates in polynomial time and the instances are logically equivalent, we have definitively proven that Vertex Cover $\leq_p$ Set Cover.

Q11. Given clauses $C_1, C_2, \dots, C_k$ each of length 3 over Boolean variables (3-SAT), and a graph G with number K (Vertex Cover asks if G has a vertex cover of size $\leq K$). Prove that Vertex Cover is at least as hard as 3-SAT. **(15 marks)** [CSE 207 | L-2/T-2 | 2016–17]

Answer

To prove that Vertex Cover is at least as hard as 3-SAT, we will show that 3-SAT is polynomial-time reducible to Vertex Cover ($3\text{-SAT} \leq_p \text{VERTEX-COVER}$). This means we can transform any instance of 3-SAT into an instance of Vertex Cover in polynomial time, such that the 3-SAT instance has a solution if and only if the corresponding Vertex Cover instance has a solution. Since 3-SAT is NP-complete, this will prove that Vertex Cover is NP-hard.

The Reduction

Let ϕ be an arbitrary instance of 3-SAT. The formula ϕ is a conjunction of k clauses, $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$, over n Boolean variables, $x_1, x_2, \dots, x_n$. Each clause C_j is a disjunction of three literals, where a literal is a variable x_i or its negation $\neg x_i$.

We construct an instance of Vertex Cover, which consists of a graph $G = (V, E)$ and an integer K , from the 3-SAT formula ϕ as follows:

- Variable Gadgets:** For each variable x_i in ϕ , we create two vertices, v_i and $\bar{v}_i$, representing the literals x_i and $\neg x_i$ respectively. We add an edge $(v_i, \bar{v}_i)$ between them. This is done for all $i = 1, \dots, n$. These are the **variable edges**.
- Clause Gadgets:** For each clause $C_j = (l_{j1} \vee l_{j2} \vee l_{j3})$ in ϕ , we create three new vertices, c_{j1}, c_{j2}, c_{j3} . These three vertices are connected to form a triangle, i.e., we add the edges $(c_{j1}, c_{j2}), (c_{j2}, c_{j3})$, and (c_{j3}, c_{j1}) . This is done for all $j = 1, \dots, k$. These are the **clause edges**.
- Communication Edges:** We connect the variable gadgets to the clause gadgets. For each clause $C_j = (l_{j1} \vee l_{j2} \vee l_{j3})$, we add an edge between the clause vertex c_{jp} and the vertex in the variable gadgets corresponding to the literal l_{jp} . For example, if l_{jp} is the literal x_i , we add an edge (c_{jp}, v_i) . If l_{jp} is the literal $\neg x_i$, we add an edge $(c_{jp}, \bar{v}_i)$. This is done for all three literals in every clause.
- Target Size K :** We set the target size for the vertex cover to be $K = n + 2k$.

Example Construction: Consider the 3-SAT formula $\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$. Here, $n = 3$ variables (x_1, x_2, x_3) and $k = 2$ clauses. The target vertex cover size is $K = n + 2k = 3 + 2(2) = 7$. The constructed graph is shown below.

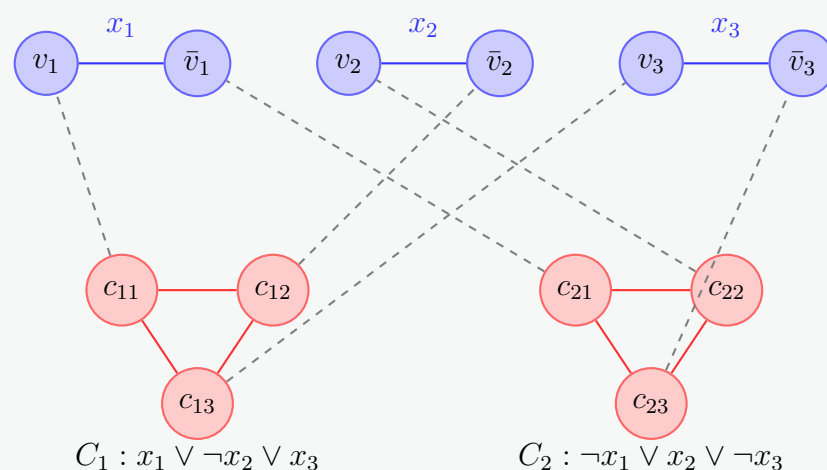


Figure 8: Graph constructed for $\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$.

Polynomial-Time Transformation

The construction of the graph G and integer K can be performed in polynomial time with respect to the size of the 3-SAT formula ϕ . Let n be the number of variables and k be the number of clauses.

- The number of vertices in G is $|V| = 2n + 3k$.
- The number of edges in G is $|E| = n$ (variable edges) + $3k$ (clause edges) + $3k$ (communication edges) = $n + 6k$.

Both $|V|$ and $|E|$ are linear in n and k , so the construction is efficient and can be completed in polynomial time.

Proof of Correctness

We now prove that ϕ is satisfiable if and only if G has a vertex cover of size at most $K = n + 2k$.

($\Rightarrow$) If ϕ is satisfiable, then G has a vertex cover of size K .

Assume ϕ is satisfiable. Let $\mathcal{A}$ be a satisfying truth assignment for the variables $x_1, \dots, x_n$. We construct a set of vertices $VC \subseteq V$ and show that it is a vertex cover of size K .

- From variables:** For each variable x_i : if $\mathcal{A}$ sets x_i to TRUE, we add vertex v_i to VC . If $\mathcal{A}$ sets x_i to FALSE, we add vertex $\bar{v}_i$ to VC . This adds exactly n vertices to VC . This choice ensures that all n variable edges $(v_i, \bar{v}_i)$ are covered, as one endpoint of each edge is in VC .

- (b) **From clauses:** For each clause $C_j = (l_{j1} \vee l_{j2} \vee l_{j3})$: since $\mathcal{A}$ is a satisfying assignment, at least one of its literals must be true. Let's pick one such true literal, say l_{jp} . The vertex corresponding to this literal is already in VC by step 1. To cover the internal edges of the clause gadget (the triangle), we add the other two vertices from the clause gadget to VC . That is, we add the two vertices from $\{c_{j1}, c_{j2}, c_{j3}\}$ that are not c_{jp} . This adds exactly 2 vertices to VC for each clause, for a total of $2k$ vertices.

The total size of the constructed set VC is $|VC| = n + 2k = K$. Now we must verify that VC is indeed a vertex cover for G .

- **Variable edges** $(v_i, \bar{v}_i)$: Covered by step 1, as for each i , either v_i or $\bar{v}_i$ is in VC .
- **Clause edges** (c_{ja}, c_{jb}) : For each clause C_j , we added two of its three vertices to VC . Any edge in the triangle has at least one of its endpoints in VC . So all clause edges are covered.
- **Communication edges** (c_{jp}, u_p) : Here u_p is the vertex corresponding to literal l_{jp} . Consider such an edge for a clause C_j .
 - If l_{jp} is the true literal we picked for clause C_j in step 2, then its corresponding vertex u_p was added to VC in step 1. Thus, the edge is covered.
 - If l_{jp} is not the chosen true literal, then its corresponding clause vertex c_{jp} was added to VC in step 2. Thus, the edge is covered.

All edges in E are covered, so VC is a vertex cover of size K .

($\Leftarrow$) **If G has a vertex cover of size $\leq K$, then ϕ is satisfiable.**

Assume there exists a vertex cover VC' for G with $|VC'| \leq K = n + 2k$.

First, observe the structure of G :

- To cover the n disjoint variable edges $(v_i, \bar{v}_i)$, any vertex cover must include at least one vertex from each pair $\{v_i, \bar{v}_i\}$. This requires at least n vertices.
- To cover the $3k$ edges of the k disjoint clause triangles, any vertex cover must include at least two vertices from each triple $\{c_{j1}, c_{j2}, c_{j3}\}$. This requires at least $2k$ vertices.

Since the variable gadgets and clause gadgets are disjoint, any vertex cover for G must have a size of at least $n + 2k$. Given $|VC'| \leq K = n + 2k$, it must be that $|VC'| = K$. Furthermore, to achieve this minimum size, VC' must contain:

- **Exactly one** vertex from each pair $\{v_i, \bar{v}_i\}$ for $i = 1, \dots, n$.
- **Exactly two** vertices from each triple $\{c_{j1}, c_{j2}, c_{j3}\}$ for $j = 1, \dots, k$.

We can now define a truth assignment $\mathcal{A}$ for variables $x_1, \dots, x_n$ based on VC' : For each $i = 1, \dots, n$:

- If $v_i \in VC'$, set $x_i = \text{TRUE}$.
- If $\bar{v}_i \in VC'$, set $x_i = \text{FALSE}$.

Since exactly one of $\{v_i, \bar{v}_i\}$ is in VC' , this assignment is well-defined.

We now show that this assignment $\mathcal{A}$ satisfies ϕ . Consider an arbitrary clause $C_j = (l_{j1} \vee l_{j2} \vee l_{j3})$. The vertex cover VC' contains exactly two vertices from $\{c_{j1}, c_{j2}, c_{j3}\}$. Let c_{jp} be the single vertex from this triple that is **not** in VC' . Consider the communication edge (c_{jp}, u_p) , where u_p is the vertex corresponding to the literal l_{jp} . Since VC' is a vertex cover, this edge must be covered. As $c_{jp} \notin VC'$, its other endpoint u_p must be in VC' . By our definition of the assignment $\mathcal{A}$, if the vertex u_p is in VC' , the literal l_{jp} is assigned a TRUE value. For example:

- If $l_{jp} = x_i$, then $u_p = v_i$. Since $v_i \in VC'$, our assignment sets $x_i = \text{TRUE}$, so l_{jp} is true.
- If $l_{jp} = \neg x_i$, then $u_p = \bar{v}_i$. Since $\bar{v}_i \in VC'$, our assignment sets $x_i = \text{FALSE}$, so $l_{jp} = \neg x_i$ is true.

In either case, the literal l_{jp} is true under $\mathcal{A}$. Therefore, the clause C_j is satisfied. Since this holds for any clause C_j , the entire formula ϕ is satisfied by $\mathcal{A}$.

Conclusion

We have shown a polynomial-time reduction from any instance of 3-SAT to an instance of Vertex Cover. The 3-SAT formula is satisfiable if and only if the constructed graph has a vertex cover of size $K = n + 2k$. Since 3-SAT is a known NP-complete problem, this reduction proves that Vertex Cover is NP-hard, meaning it is at least as hard as 3-SAT.

Q12. Explain the term “reducibility”. If L_1 and L_2 are two languages such that $L_1 \leq_p L_2$, show that $L_2 \in P$ implies $L_1 \in P$. If any NP-complete problem is polynomial-time solvable, prove that $P = NP$. **(10 marks)** [CSE 207 | L-2/T-2 | 2015–16]

Answer

1. The Term “Reducibility”

In computational complexity theory, **reducibility** (specifically, polynomial-time many-one reducibility or Karp reducibility) is a method to formally relate the difficulty of two computational problems.

We say that a language $L_1 \subseteq \Sigma^*$ is **polynomial-time reducible** to a language $L_2 \subseteq \Sigma^*$, denoted as $L_1 \leq_p L_2$, if there exists a polynomial-time computable function $f: \Sigma^* \rightarrow \Sigma^*$ such that for every input string $x \in \Sigma^*$:

$$x \in L_1 \iff f(x) \in L_2$$

Intuitively, $L_1 \leq_p L_2$ means that L_1 is “no harder than” L_2 (up to a polynomial factor), because any algorithm that solves L_2 can be used as a subroutine to solve L_1 . The function f acts as a translator, converting instances of L_1 into equivalent instances of L_2 efficiently.

2. Proof: $L_1 \leq_p L_2$ and $L_2 \in P \implies L_1 \in P$

Theorem: If a language L_1 is polynomial-time reducible to a language L_2 , and L_2 is solvable in polynomial time, then L_1 is also solvable in polynomial time.

Proof. Let us break down the given premises:

- (a) Since $L_2 \in P$, there exists a deterministic algorithm A_2 that decides L_2 in polynomial time. Let its time complexity be bounded by $O(n^k)$, where n is the length of the input and $k \geq 1$ is a constant.
- (b) Since $L_1 \leq_p L_2$, there exists a polynomial-time computable reduction function f . Let A_f be the algorithm computing f . The time taken by A_f on an input x of length $n = |x|$ is bounded by $O(n^c)$ for some constant $c \geq 1$.

We must construct a deterministic polynomial-time algorithm A_1 for L_1 . We define A_1 on input x as follows:

Algorithm 61 Algorithm A_1 for L_1

```

1: Input: A string  $x \in \Sigma^*$ 
2: Compute  $y = f(x)$  using algorithm  $A_f$ .
3: Run algorithm  $A_2$  on input  $y$ .
4: if  $A_2(y)$  accepts then
5:   return ACCEPT
6: else
7:   return REJECT
8: end if

```

Correctness: By the definition of reducibility, $x \in L_1 \iff f(x) \in L_2$. Since A_2 correctly decides L_2 , A_1 correctly decides L_1 .

Complexity Analysis: Let $n = |x|$.

- **Step 2:** Computing $y = f(x)$ takes $O(n^c)$ time.
- A crucial observation is that a Turing machine can only write one symbol per step. Therefore, the length of the output string y cannot exceed the number of steps taken to compute it. Thus, $|y| \leq O(n^c)$.
- **Step 3:** Running A_2 on y takes time proportional to $O(|y|^k)$. Substituting the upper bound for $|y|$, the time taken is $O((n^c)^k) = O(n^{ck})$.

The total running time of A_1 is the sum of the time for the reduction and the time to solve the transformed instance:

$$T(n) = O(n^c) + O(n^{ck}) = O(n^{ck})$$

Since c and k are constants, ck is a constant. Therefore, $T(n)$ is a polynomial in n . This proves that there exists a polynomial-time algorithm for L_1 , meaning $L_1 \in P$. $\square$

3. Proof: If any NP-Complete problem is in P, then $P = NP$

Theorem: Let L_{NPC} be an NP-complete problem. If $L_{NPC} \in P$, then $P = NP$.

Proof. To prove that $P = NP$, we must show that $P \subseteq NP$ and $NP \subseteq P$.

- **Part 1:** $P \subseteq NP$. This is trivially true by definition. Any problem solvable in deterministic polynomial time (P) can also be solved (or verified) in non-deterministic polynomial time (NP).
- **Part 2:** $NP \subseteq P$. Let L be an arbitrary language in NP . We need to show that $L \in P$.
 - (a) By the definition of NP-Completeness, since L_{NPC} is NP-complete, every problem in NP is polynomial-time reducible to L_{NPC} . Therefore, $L \leq_p L_{NPC}$.
 - (b) We are given the premise that $L_{NPC} \in P$.
 - (c) Using the theorem proven in Section 2, if $L \leq_p L_{NPC}$ and $L_{NPC} \in P$, it strictly implies that $L \in P$.

Since L was an arbitrary problem in NP , we conclude that every problem in NP is also in P . Thus, $NP \subseteq P$.

Since $P \subseteq NP$ and $NP \subseteq P$, we conclude that $P = NP$. This formally demonstrates why finding a polynomial-time algorithm for even a single NP-complete problem (like 3-SAT or Vertex Cover) would automatically yield polynomial-time algorithms for thousands of notoriously hard problems. $\square$

Q13. Define the classes P, NP, and NP-complete. What are the steps to prove a problem X is NP-complete? Prove that the decision version of the independent set problem is NP-complete using a reduction from 3-CNF-SAT. Give an intuitive sketch of the proof of the Cook-Levin theorem (CIRCUIT-SAT is NP-complete). (4+4+12+5 marks) [CSE 207 | L-2/T-2 | 2014–15]

Answer

1. Definitions of Complexity Classes

- **The Class P:** P is the class of decision problems that can be solved by a *deterministic* Turing machine in polynomial time. These are the problems considered computationally tractable. For any problem in P, there exists an algorithm with a worst-case time complexity of $O(n^k)$, where n is the input size and k is a constant.
- **The Class NP:** NP (Nondeterministic Polynomial time) is the class of decision problems for which a "yes" answer can be *verified* by a deterministic Turing machine in polynomial time. Equivalently, it is the class of problems solvable by a *non-deterministic* Turing machine in polynomial time. If the answer is "yes," there exists a polynomial-size certificate (or proof) that a verifier can check efficiently.
- **The Class NP-complete (NPC):** A decision problem X is NP-complete if it satisfies two conditions:
 - (a) $X \in \text{NP}$ (it is verifiable in polynomial time).
 - (b) X is NP-hard, meaning every problem $L \in \text{NP}$ is polynomial-time reducible to X ($L \leq_p X$).

NP-complete problems are the "hardest" problems in NP. If any NP-complete problem can be solved in polynomial time, then $P = \text{NP}$.

2. Steps to Prove a Problem X is NP-Complete

To rigorously prove that a problem X is NP-complete, the following sequence of steps must be executed:

- (a) **Prove $X \in \text{NP}$:** Describe a polynomial-size certificate for a "yes" instance of the problem, and construct a deterministic algorithm that verifies this certificate in polynomial time.
- (b) **Select a known NP-complete problem Y :** Choose a well-known NP-complete problem (such as 3-SAT, Vertex Cover, Clique) that has structural similarities to X .
- (c) **Construct a reduction function f :** Design an algorithm that transforms any arbitrary instance of Y into an instance of X .
- (d) **Prove Polynomial Time:** Demonstrate that the transformation function f executes in time bounded by a polynomial in the size of the input.
- (e) **Prove Correctness:** Show that the reduction preserves the outcome, meaning the instance of Y is a "yes" instance if and only if the transformed instance $f(Y)$ is a "yes" instance of X ($y \in Y \iff f(y) \in X$).

3. Proof: Independent Set is NP-Complete

Problem Statement: Given a graph $G = (V, E)$ and an integer K , does there exist a subset $V' \subseteq V$ such that $|V'| \geq K$ and no two vertices in V' are connected by an edge?

Step 1: Independent Set (IS) $\in \text{NP}$

- **Certificate:** A subset of vertices $V' \subseteq V$.
- **Verifier:** Count the number of vertices in V' to check if $|V'| \geq K$. Then, for every pair of vertices $u, v \in V'$, verify that $(u, v) \notin E$. This takes $O(|V'|^2)$ time, which is strictly polynomial with respect to the input size. Thus, IS $\in \text{NP}$.

Step 2 & 3: Reduction from 3-CNF-SAT We reduce 3-CNF-SAT to Independent Set. Let $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$ be a boolean formula in 3-CNF format with k clauses. Each clause $C_r = (l_{r1} \vee l_{r2} \vee l_{r3})$ contains exactly 3 literals. We construct an instance (G, K) of IS as follows:

- **Vertices:** For each clause $C_r = (l_{r1} \vee l_{r2} \vee l_{r3})$, create three vertices v_{r1}, v_{r2}, v_{r3} . Total vertices $|V| = 3k$.
- **Clause Edges:** Connect the three vertices of each clause to form a triangle. This ensures at most one vertex per clause can be chosen in an independent set.
- **Conflict Edges:** Add an edge between any two vertices v_{ri} and v_{sj} if they represent contradictory literals (i.e., one is x and the other is $\neg x$).
- **Target Size:** Set $K = k$ (the number of clauses).

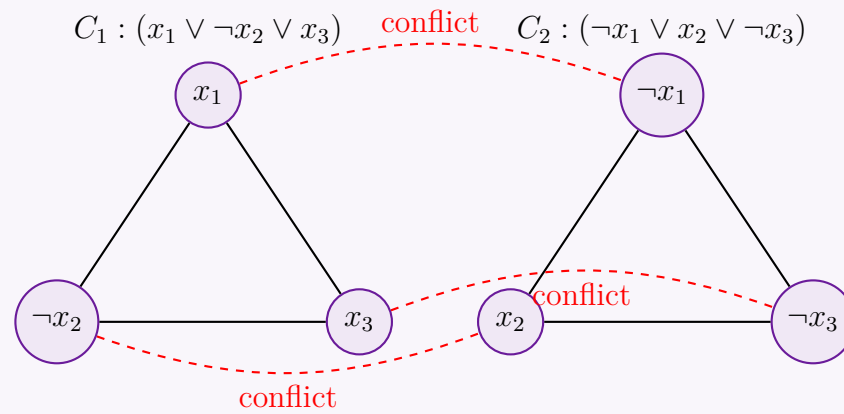


Figure 9: Reduction graph for $\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$. Triangles represent clauses; dashed red lines are conflict edges.

Step 4: Polynomial Time Bound The constructed graph has exactly $3k$ vertices. The number of edges is at most $3k$ (clause edges) plus $O(k^2)$ (conflict edges). Building the graph takes $O(k^2)$ time, which is polynomial in the size of the boolean formula ϕ .

Step 5: Correctness (ϕ is satisfiable $\iff G$ has an IS of size $\geq k$)

- ($\Rightarrow$) Suppose ϕ is satisfiable. There is a truth assignment making at least one literal true in every clause. Select exactly one true literal's vertex from each clause's triangle. Because we chose exactly one vertex per clause, we have selected k vertices. Since they evaluate to true simultaneously under a valid assignment, no chosen pair can represent a variable and its negation. Thus, no conflict edges exist between chosen vertices. They form an Independent Set of size k .
- ($\Leftarrow$) Suppose G has an independent set V' of size $\geq k$. Since each clause is a triangle, at most one vertex per clause can be in V' . To achieve size k , V' must contain exactly one vertex from each of the k clauses. We assign TRUE to the literals corresponding to the vertices in V' . Because no two vertices in V' have a conflict edge, we will never assign TRUE to both x and $\neg x$. This assignment makes at least one literal true in every clause, satisfying ϕ .

Since Independent Set is in NP and $3\text{-CNF-SAT} \leq_p \text{IS}$, Independent Set is NP-complete.

4. Intuitive Sketch of the Cook-Levin Theorem

The Cook-Levin Theorem establishes that **CIRCUIT-SAT is NP-complete** (and consequently, Boolean Satisfiability is NP-complete). It forms the bedrock of NP-completeness.

Concept: The core idea is to physically simulate the computation of a Turing machine using boolean logic gates.

- CIRCUIT-SAT $\in$ NP:** Given a boolean circuit and a proposed assignment of inputs, a verifier can simply propagate the boolean values through the gates in topological order. This takes linear time relative to the number of gates. Thus, verification is in polynomial time.
- NP-Hardness Reduction Sketch:**
 - Let L be *any* problem in NP. By definition, there exists a deterministic polynomial-time verifier Turing Machine M that takes an input string x and a certificate c , and verifies if $x \in L$ within time $T(n) \leq n^k$.
 - The execution of M can be visualized as a 2D computational tableau (a grid). The rows represent time steps (1 to $T(n)$), and the columns represent the tape cells (1 to $T(n)$).
 - The state of any cell (i, j) at time t depends *strictly locally* on the state of cell (i, j) and its immediate neighbors at time $t - 1$, along with the internal state of the Turing machine's read/write head.
 - Because the transition rules of the Turing machine are fixed and strictly local, this step-by-step state transition can be hardwired using a constant number of boolean logic gates (AND, OR, NOT).
 - We construct a massive, polynomial-sized boolean circuit that strings together these small gate clusters to simulate the entire $T(n) \times T(n)$ tableau.
 - The "inputs" to this circuit are the unknown bits of the certificate c . The output of the circuit is wired to the "ACCEPT" state of the Turing Machine M .
 - Therefore, the boolean circuit evaluates to 1 (True) *if and only if* there exists a valid certificate c that causes M to accept x .

By constructing this polynomial-size circuit, we have shown that finding a satisfying assignment for the circuit is equivalent to finding a proof for *any* problem in NP. Hence, CIRCUIT-SAT is NP-hard, making it NP-complete.

Q14. If L_1 and L_2 are two languages such that $L_1 \leq_p L_2$, then show that $L_2 \in P$ implies $L_1 \in P$. If any NP-complete problem is polynomial-time solvable, then prove that $P = NP$. (7+3=10 marks) [CSE 207 | L-2/T-2 | 2011–12]

Answer

Part 1: Proof that $L_1 \leq_p L_2$ and $L_2 \in P \implies L_1 \in P$

Theorem: If a language L_1 is polynomial-time reducible to a language L_2 , and L_2 is solvable in deterministic polynomial time, then L_1 is also solvable in deterministic polynomial time.

Proof. Let us formalize the definitions based on the given premises:

- (a) **Premise 1** ($L_1 \leq_p L_2$): By the definition of polynomial-time many-one reducibility, there exists a deterministic polynomial-time computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that for any input string x :

$$x \in L_1 \iff f(x) \in L_2$$

Let M_f be the Turing machine that computes f . Since M_f operates in polynomial time, there exists a polynomial $p(n)$ such that M_f halts in at most $p(|x|)$ steps.

- (b) **Premise 2** ($L_2 \in P$): There exists a deterministic algorithm (Turing machine) A_2 that decides L_2 in polynomial time. Let $q(m)$ be the polynomial bounding the running time of A_2 on any input of length m .

We must construct a deterministic polynomial-time algorithm A_1 that decides L_1 . We define A_1 as follows:

Algorithm 62 Algorithm A_1 deciding L_1

```

1: Input: A string  $x \in \Sigma^*$  of length  $n = |x|$ 
2: Run  $M_f$  on  $x$  to compute the transformed string  $y = f(x)$ .
3: Run  $A_2$  on the string  $y$ .
4: if  $A_2$  accepts  $y$  then
5:   return ACCEPT
6: else
7:   return REJECT
8: end if

```

Correctness Analysis: By the property of the reduction function f , $x \in L_1$ if and only if $y \in L_2$. Since A_2 correctly decides L_2 , A_2 accepts y if and only if $y \in L_2$. Therefore, algorithm A_1 accepts x if and only if $x \in L_1$. Thus, A_1 correctly decides L_1 .

Complexity Analysis: Let $n = |x|$ be the size of the input.

- **Step 2:** Computing $y = f(x)$ takes at most $p(n)$ time. Crucially, a Turing machine can output at most one symbol per computational step. Therefore, the length of the generated string y cannot exceed the number of steps taken to compute it. Thus, we have a strict bound on the output size: $|y| \leq p(n)$.
- **Step 3:** We run A_2 on y . The running time of A_2 on an input of size m is bounded by $q(m)$. Substituting our bound for $|y|$, the time taken by A_2 is at most $q(|y|) \leq q(p(n))$.

The total running time for algorithm A_1 is the time to compute the reduction plus the time to solve the reduced instance:

$$T_{total}(n) = O(p(n) + q(p(n)))$$

Since polynomials are closed under addition and composition, $p(n) + q(p(n))$ is also a polynomial in n . Thus, algorithm A_1 is a deterministic polynomial-time algorithm for L_1 . Therefore, $L_1 \in P$. $\square$

Part 2: Proof that if an NP-Complete problem is in P, then $P = NP$

Theorem: Let L_{NPC} be an NP-complete problem. If $L_{NPC} \in P$, then the complexity classes P and NP are strictly equal ($P = NP$).

Proof. To prove that two sets P and NP are equal, we must demonstrate that $P \subseteq NP$ and $NP \subseteq P$.

- **Step 1: Prove $P \subseteq NP$**

This inclusion holds trivially by definition. Any problem in P can be solved by a deterministic polynomial-time Turing machine. A non-deterministic polynomial-time Turing machine can simply ignore its non-deterministic capabilities and execute the deterministic algorithm. Alternatively, if a problem is in P , its solution can be "verified" efficiently by simply computing the answer from scratch. Thus, $P \subseteq NP$.

- **Step 2: Prove $NP \subseteq P$**

Let L be an arbitrary language belonging to the class NP .

- By the definition of NP-Completeness, since L_{NPC} is NP-complete, **every** problem in NP is polynomial-time reducible to L_{NPC} . Because $L \in NP$, it strictly follows that $L \leq_p L_{NPC}$.
- We are given the premise that L_{NPC} is solvable in deterministic polynomial time, i.e., $L_{NPC} \in P$.
- Applying the theorem proven in Part 1: if $L \leq_p L_{NPC}$ and $L_{NPC} \in P$, it logically guarantees that $L \in P$.

Since L was chosen as an arbitrary language in NP , the implication $L \in NP \implies L \in P$ holds universally. Therefore, every problem in NP is also contained in P , establishing that $NP \subseteq P$.

Because we have established both $P \subseteq NP$ and $NP \subseteq P$, we conclude by mutual inclusion that $P = NP$. $\square$

Q15. Instance of Satisfiability Problem

Boolean variables: X_1, X_2, X_3, X_4, X_5

Clauses:

$$\begin{aligned} C_1 &= X_1 \vee \overline{X_2}, & C_2 &= \overline{X_1} \vee X_2 \vee X_4 \vee \overline{X_5} \\ C_3 &= \overline{X_2}, & C_4 &= X_2 \vee X_3 \vee \overline{X_4} \end{aligned}$$

Mention the number of boolean variables and clauses in the 3-SAT problem.

Answer

To convert a general Boolean Satisfiability (SAT) instance into a 3-SAT instance, we use the standard Karp reduction. The strict definition of 3-SAT requires every clause to have **exactly 3 distinct literals**. We achieve this by introducing new "dummy" or "bridging" variables to expand or break down the original clauses.

Let the original set of variables be $V = \{X_1, X_2, X_3, X_4, X_5\}$, so there are initially **5 variables**. We will process each clause C_i individually based on its length k :

1. Clause $C_1 = X_1 \vee \overline{X_2}$ (Length $k = 2$)

For a clause with exactly 2 literals, we introduce 1 new variable, say Y_1 . We expand the clause into 2 new clauses to ensure each has exactly 3 literals without changing the satisfiability:

$$\begin{aligned} C_{1a} &= X_1 \vee \overline{X_2} \vee Y_1 \\ C_{1b} &= X_1 \vee \overline{X_2} \vee \overline{Y_1} \end{aligned}$$

Added: 1 new variable (Y_1), 2 clauses.

2. Clause $C_2 = \overline{X_1} \vee X_2 \vee X_4 \vee \overline{X_5}$ (Length $k = 4$)

For a clause with $k > 3$ literals, we must break it apart. We introduce $k - 3$ new bridging variables. Here, $k = 4$, so we introduce $4 - 3 = 1$ new variable, say Y_2 . The clause is split into $k - 2 = 2$ clauses:

$$\begin{aligned} C_{2a} &= \overline{X_1} \vee X_2 \vee Y_2 \\ C_{2b} &= \overline{Y_2} \vee X_4 \vee \overline{X_5} \end{aligned}$$

Added: 1 new variable (Y_2), 2 clauses.

3. Clause $C_3 = \overline{X_2}$ (Length $k = 1$)

For a clause with exactly 1 literal, we must pad it to reach length 3. We introduce 2 new variables, say Y_3 and Y_4 . We expand it into $2^2 = 4$ combinations to neutralize the new variables:

$$\begin{aligned} C_{3a} &= \overline{X_2} \vee Y_3 \vee Y_4 \\ C_{3b} &= \overline{X_2} \vee Y_3 \vee \overline{Y_4} \\ C_{3c} &= \overline{X_2} \vee \overline{Y_3} \vee Y_4 \\ C_{3d} &= \overline{X_2} \vee \overline{Y_3} \vee \overline{Y_4} \end{aligned}$$

Added: 2 new variables (Y_3, Y_4), 4 clauses.

4. Clause $C_4 = X_2 \vee X_3 \vee \overline{X_4}$ (Length $k = 3$)

This clause already has exactly 3 distinct literals. No transformation is needed.

$$C_{4a} = X_2 \vee X_3 \vee \overline{X_4}$$

Added: 0 new variables, 1 clause.

Final Count

Total Number of Boolean Variables: We have the 5 original variables $\{X_1, X_2, X_3, X_4, X_5\}$ plus the 4 newly introduced variables $\{Y_1, Y_2, Y_3, Y_4\}$.

$$\text{Total Variables} = 5 + 4 = 9$$

Total Number of Clauses: Summing the clauses generated from each step:

$$\text{Total Clauses} = 2 \text{ (from } C_1) + 2 \text{ (from } C_2) + 4 \text{ (from } C_3) + 1 \text{ (from } C_4) = 9$$

*Note: In some relaxed definitions of 3-SAT where literal duplication within a clause is allowed, C_1 could be written as $(X_1 \vee \overline{X_2} \vee \overline{X_2})$ and C_3 as $(\overline{X_2} \vee \overline{X_2} \vee \overline{X_2})$. Under that specific relaxed assumption, the reduction would only require 1 new variable for C_2 (total 6 variables) and produce exactly 5 clauses. However, the standard algorithmic reduction (strict 3-SAT) guarantees distinct literals, resulting in **9 variables** and **9 clauses**.*

NP-hard and NP-complete Problems

Q1. What are some approaches to deal with an NP-complete problem? (6 marks)

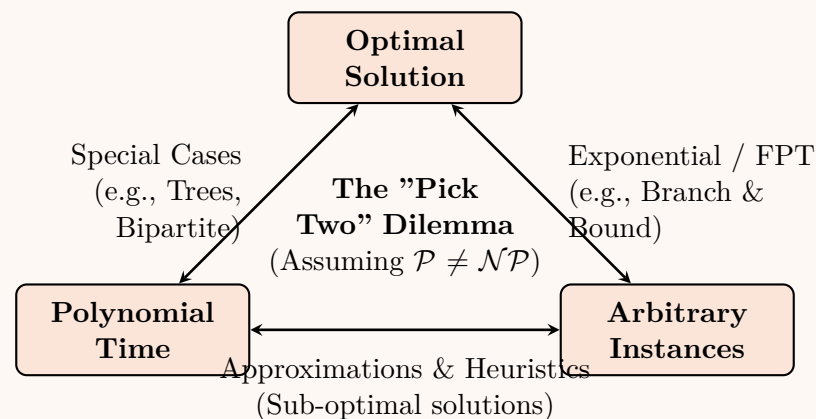
[CSE 207 | L-2/T-1 | 2023–24]

Answer

When dealing with an NP-complete problem, it is widely believed that no algorithm can solve all instances optimally in polynomial time (assuming $\mathcal{P} \neq \mathcal{NP}$). Therefore, algorithm designers must compromise. Every algorithmic approach to an NP-complete problem relaxes at least one of the three desired guarantees:

- (a) **Optimal Solution** (Finding the absolute best answer)
- (b) **Polynomial Time** (Running fast, efficiently scaling with input size)
- (c) **Arbitrary Instances** (Working on any general input graph/data)

The trade-offs between these guarantees are visualized in the diagram below.



Here are the primary algorithmic approaches to deal with NP-complete problems:

1. Relaxing Optimality: Approximation Algorithms

When a problem must be solved in polynomial time for arbitrary instances, we can accept a near-optimal solution with a mathematically proven worst-case guarantee.

- **Concept:** An algorithm produces a valid solution that is guaranteed to be within a factor ρ (the approximation ratio) of the true optimal solution OPT .
- **Mathematical Bound:** For a minimization problem, the algorithm yields a cost $ALG \leq \rho \cdot OPT$, where $\rho \geq 1$.

Example: 2-Approximation for Minimum Vertex Cover

Algorithm 63 Vertex Cover Approximation

```

1: Input: Undirected graph  $G = (V, E)$ 
2: Output: A vertex cover  $C \subseteq V$ 
3:  $C \leftarrow \emptyset$ 
4:  $E' \leftarrow E$ 
5: while  $E' \neq \emptyset$  do
6:   Pick an arbitrary edge  $(u, v) \in E'$ 
7:    $C \leftarrow C \cup \{u, v\}$ 
8:   Remove all edges incident to  $u$  or  $v$  from  $E'$ 
9: end while
10: return  $C$ 

```

- **Time Complexity:** $\mathcal{O}(|V| + |E|)$ because each edge and vertex is processed a constant number of times. Space Complexity is $\mathcal{O}(|V|)$ to store the cover.
- **Approximation Guarantee:** Since the algorithm picks a maximal matching, and any valid vertex cover MUST pick at least one vertex from each matching edge, taking both vertices ensures we are at most $2 \times OPT$.

2. Relaxing Polynomial Time: Exact Algorithms

If we absolutely need the optimal solution for general instances, we must sacrifice polynomial time bounds. However, we can use techniques to make the exponential runtime feasible for practically sized inputs.

- **Branch and Bound / Backtracking:** Intelligently prune the search space. If a partial solution is already worse than the current best-known solution, the algorithm aborts exploring that branch.
- **Dynamic Programming over Subsets:** Classic example is the Traveling Salesperson Problem (TSP) using the Held-Karp algorithm, reducing naive $\mathcal{O}(n!)$ brute force to $\mathcal{O}(n^2 2^n)$.
- **Integer Linear Programming (ILP):** Formulate the problem as an ILP and use highly optimized commercial solvers (like Gurobi or CPLEX) which employ advanced cutting planes and branch-and-cut heuristics.

3. Fixed-Parameter Tractability (FPT)

Instead of analyzing complexity solely based on the input size n , Parameterized Complexity analyzes it with respect to an additional parameter k (e.g., the size of the vertex cover sought, or the treewidth of the graph).

- A problem is FPT if it can be solved in time:

$$\mathcal{T}(n, k) = f(k) \cdot n^{\mathcal{O}(1)}$$

where f is a computable function exclusively dependent on k .

- Intuition:** All the combinatorial explosion (the exponential part of the runtime) is confined to the parameter k . If k is small, the algorithm remains highly efficient even for massive n .
- Example:** Vertex Cover parameterized by solution size k can be solved in $\mathcal{O}(2^k \cdot n)$ using bounded search trees.

4. Relaxing Generality: Special Cases

Many NP-complete problems become easily solvable in polynomial time if the input instances belong to a restricted class of graphs or data.

- Trees:** Maximum Independent Set is NP-complete on general graphs but can be solved in $\mathcal{O}(n)$ on trees using Dynamic Programming.
- Bipartite Graphs:** Maximum Clique is trivially solvable (maximum size is 2, unless the graph is empty). Minimum Vertex Cover can be solved exactly in polynomial time using Network Flow / Maximum Matching (König's Theorem).
- Planar Graphs:** Problems like Max Cut or perfect matching have specialized polynomial-time algorithms or admits very strong approximations (PTAS) on planar graphs.

5. Heuristics and Metaheuristics

When mathematical guarantees are not required, but "good enough" solutions are needed quickly.

- Local Search:** Start with a random solution and iteratively make small changes (e.g., swapping two cities in TSP) to improve the cost until reaching a local optimum.
- Simulated Annealing / Genetic Algorithms:** Metaheuristics designed to escape local optima by occasionally accepting worse solutions probabilistically or simulating evolutionary crossover.
- Note:** Unlike approximation algorithms, these offer **no formal worst-case bounds** on solution quality or runtime to converge, but often perform phenomenally well in industrial applications.

Q2. Define the satisfiability problem. Show that the independent set problem is NP-hard by reducing the 3-SAT problem to the independent set problem. (3+7 marks) [CSE 207 | L-2/T-2 | 2017–18]

Answer

1. The Satisfiability Problem (SAT)

[3 Marks]

To define the Satisfiability (SAT) problem formally, we first establish the foundational components of Boolean logic:

- Variables & Literals:** Let $U = \{x_1, x_2, \dots, x_n\}$ be a set of Boolean variables. A *literal* is either a variable x_i or its negation $\bar{x}_i$.
- Clause:** A clause C_j is a disjunction (logical OR, $\vee$) of one or more literals. For example: $C_j = (x_1 \vee \bar{x}_2 \vee x_3)$.
- Conjunctive Normal Form (CNF):** A Boolean formula ϕ is in CNF if it is expressed as a conjunction (logical AND, $\wedge$) of clauses. For example: $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$.

Definition (SAT): Given a Boolean formula ϕ in Conjunctive Normal Form (CNF) over a set of variables U , the Satisfiability problem asks whether there exists a truth assignment (mapping each variable to **True** or **False**) such that ϕ evaluates to **True**. For ϕ to be **True**, *every* clause C_j must contain at least one literal that evaluates to **True**.

2. NP-Hardness of Independent Set

[7 Marks]

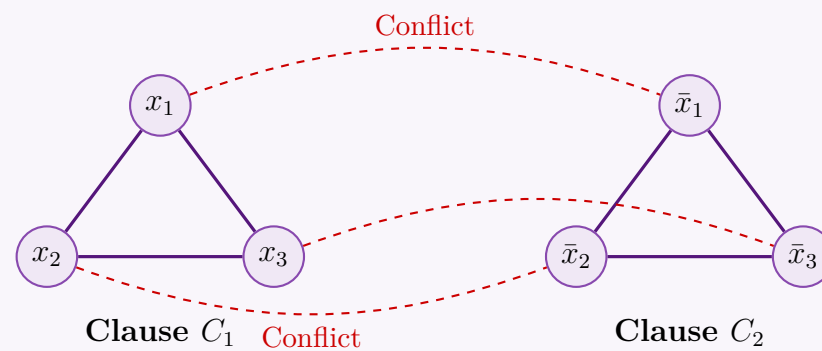
To prove that the Independent Set (IS) problem is NP-hard, we provide a polynomial-time reduction from the known NP-hard problem **3-SAT**.

- 3-SAT Problem:** A special case of SAT where every clause contains exactly 3 literals.
- Independent Set Problem:** Given an undirected graph $G = (V, E)$ and an integer K , does G contain an independent set of size at least K (a subset of K vertices where no two vertices are connected by an edge)?

Step 2.1: Polynomial-Time Reduction Construction ($f : 3\text{-SAT} \rightarrow \text{IS}$) Let $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$ be an instance of 3-SAT with k clauses, where each clause $C_r = (l_{r,1} \vee l_{r,2} \vee l_{r,3})$. We construct a graph $G = (V, E)$ and define a target size K as follows:

- Vertices (V):** For each clause C_r , create 3 vertices corresponding to its literals. Thus, $|V| = 3k$. Let the vertex corresponding to literal $l_{r,i}$ be $v_{r,i}$.
- Clause Edges (E_{clause}):** For every clause C_r , connect its three vertices to form a triangle (K_3). This ensures at most one vertex per clause can be chosen for an independent set.
- Conflict Edges (E_{conflict}):** Add an edge between any two vertices $v_{r,i}$ and $v_{s,j}$ if they represent conflicting literals (i.e., one is x and the other is $\bar{x}$). This enforces logical consistency.
- Parameter (K):** Set the target independent set size to $K = k$ (the number of clauses).

Example Visualization of the Reduction Consider $\phi = (x_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$. Here $k = 2$. We construct G as shown below:



Step 2.2: Correctness Proof (ϕ is satisfiable $\iff G$ has an Independent Set of size k) **Forward Direction ($\implies$):** Assume ϕ is satisfiable. There exists a truth assignment making every clause evaluate to **True**. Therefore, in each clause C_r , at least one literal is **True**.

- Select exactly one **True** literal from each clause and pick its corresponding vertex in G .
- Because we select exactly one vertex per clause, no *clause edges* connect our chosen vertices.
- Because the truth assignment is strictly consistent (a variable cannot be both **True** and **False** simultaneously), our selection does not contain conflicting literals. Thus, no *conflict edges* connect our chosen vertices.
- We have selected k vertices with no edges between them, forming an independent set of size k .

Backward Direction ($\impliedby$): Assume G has an independent set I of size k .

- The vertices in G are partitioned into k disjoint triangles (one for each clause). An independent set can contain at most 1 vertex from any triangle.
- Since $|I| = k$, I must contain exactly 1 vertex from *every* triangle.
- The vertices in I represent a set of literals. Because they form an independent set, there are no *conflict edges* between them. This implies we never selected both x_i and $\bar{x}_i$.
- We can safely set all literals represented in I to **True** (and assign any unmapped variables arbitrarily).
- Because we picked one literal from every triangle, every clause in ϕ contains at least one **True** literal. Thus, ϕ is satisfiable.

Step 2.3: Complexity of Reduction The reduction creates exactly $3k$ vertices and at most $3k + (3k)^2$ edges. This entire construction can be carried out in $\mathcal{O}(k^2)$ time, which is strictly polynomial with respect to the input size of ϕ . Since 3-SAT $\leq_p$ Independent Set, and 3-SAT is NP-hard, the **Independent Set problem is NP-hard**.

Q3. Your friend claims to have developed a polynomial-time algorithm to solve the set-packing problem. What will happen if this claim is proved to be true? (5 marks) [CSE 207 | L-2/T-2 | 2017–18]

Answer

If your friend's claim is proven to be true, the most immediate and monumental consequence is that it would prove $\mathcal{P} = \mathcal{NP}$. This is one of the most famous open problems in computer science and mathematics. Below is a detailed breakdown of the theoretical and practical implications of this discovery.

1. Immediate Theoretical Implication: $\mathcal{P} = \mathcal{NP}$

The **Set-Packing** problem is one of Karp's original 21 **NP-complete** problems. By definition, a problem is NP-complete if:

- It is in $\mathcal{NP}$ (a proposed solution can be verified in polynomial time).
- It is NP-hard (every problem in $\mathcal{NP}$ can be reduced to it in polynomial time).

Let X be any arbitrary problem in $\mathcal{NP}$. Because Set-Packing is NP-hard, there exists a polynomial-time reduction function f such that $X \leq_p \text{Set-Packing}$.

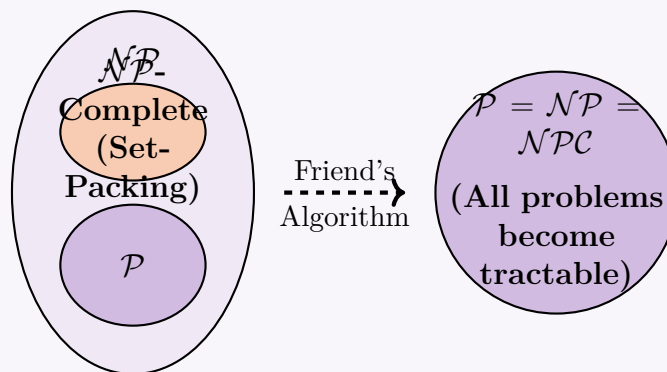
If your friend has a polynomial-time algorithm A for Set-Packing (running in $\mathcal{O}(n^k)$ time), we can solve *any* problem $X \in \mathcal{NP}$ in polynomial time using the following composite algorithm:

Algorithm 64 Solving any $X \in \mathcal{NP}$ in Polynomial Time

- 1: **Input:** An instance x of problem X .
 - 2: Transform x into an instance y of Set-Packing using the polynomial reduction f . ▷ Takes poly time
 - 3: Feed y into your friend's algorithm A . ▷ Takes poly time
 - 4: **return** the boolean answer provided by A .
-

Since the composition of two polynomial-time operations is itself bounded by a polynomial, every problem in $\mathcal{NP}$ is now in $\mathcal{P}$, collapsing the complexity classes such that $\mathcal{P} = \mathcal{NP}$.

Current Belief: $\mathcal{P} \neq \mathcal{NP}$ **New Reality:** $\mathcal{P} = \mathcal{NP}$



2. Mathematical and Financial Consequences

- **The Millennium Prize:** The $\mathcal{P}$ vs $\mathcal{NP}$ problem is one of the seven Millennium Prize Problems designated by the Clay Mathematics Institute. Proving this claim would instantly earn your friend \$1,000,000 and eternal fame in the scientific community.
- **Automated Theorem Proving:** Finding a formal mathematical proof of length N is an $\mathcal{NP}$ problem. If $\mathcal{P} = \mathcal{NP}$, computers could efficiently generate proofs for any unproven mathematical theorem (e.g., the Riemann Hypothesis), fundamentally changing the nature of mathematics.

3. Practical Consequences (The "Good")

Thousands of computationally intractable optimization problems across all industries would become efficiently solvable:

- **Operations Research:** The Traveling Salesperson Problem (TSP), Vehicle Routing, and optimal scheduling could be solved exactly, saving billions of dollars in global logistics and supply chain management.
- **Computational Biology:** Protein folding, DNA sequence alignment, and drug discovery processes would become computationally trivial, likely curing many diseases.
- **Machine Learning:** Training complex neural networks (currently optimized using heuristic gradient descent) could theoretically be solved to find the absolute global minimum efficiently.

4. Security Consequences (The "Bad")

Modern digital security would instantly collapse:

- **Cryptography Destruction:** Public-key cryptography (like RSA and ECC) relies on the premise that certain problems, like Integer Factorization and Discrete Logarithm, are computationally hard (these are in $\mathcal{NP}$).
- If $\mathcal{P} = \mathcal{NP}$, factoring massive primes would take seconds. Every bank transaction, encrypted message, blockchain network, and national security database would become publicly readable and forgeable.

Conclusion: Unless your friend has made one of the greatest intellectual breakthroughs in human history, they likely made an error. They most likely either developed a heuristic/approximation, solved only a special sub-case of the Set-Packing problem, or made a mistake in their algorithmic complexity analysis.

Q4. Write the names of four NP-complete problems. Define the following six classes of problems: P, Co-P, NP, Co-NP, NP-complete, and NP-hard. By a diagram show the relationship among these classes that most researchers regard as most likely. **(10 marks)** [CSE 207 | L-2/T-2 | 2013–14]

Answer

1. Four NP-Complete Problems

Here are four classic examples of NP-complete problems:

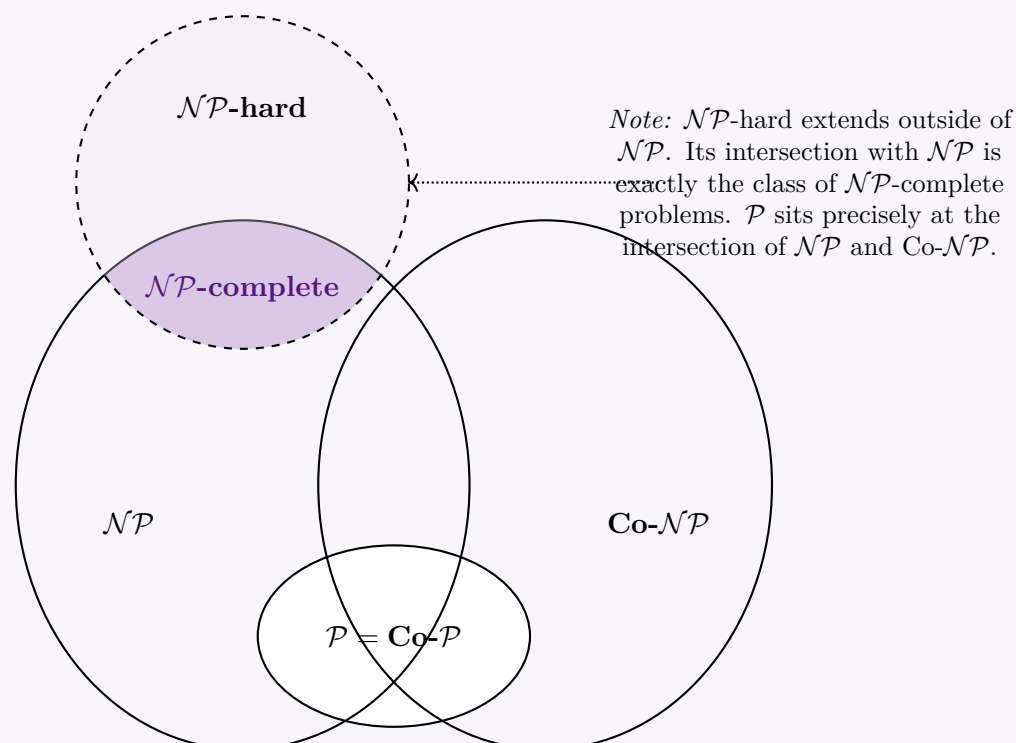
- (a) **Boolean Satisfiability Problem (SAT / 3-SAT):** Given a Boolean formula, is there an assignment of truth values to the variables that makes the entire formula true?
- (b) **Traveling Salesperson Problem (Decision Version):** Given a set of cities and a distance matrix, is there a route that visits every city exactly once and returns to the origin with a total distance $\leq K$?
- (c) **Minimum Vertex Cover (Decision Version):** Given a graph G and an integer K , does there exist a subset of vertices of size $\leq K$ such that every edge in G is incident to at least one vertex in the subset?
- (d) **Maximum Clique (Decision Version):** Given a graph G and an integer K , does G contain a complete subgraph (clique) of size $\geq K$?

2. Definitions of Complexity Classes

- **$\mathcal{P}$ (Polynomial Time):** The class of decision problems that can be *solved* by a deterministic Turing machine in polynomial time (i.e., time $\mathcal{O}(n^k)$ for some constant k).
- **Co- $\mathcal{P}$:** The class of decision problems whose complement (inverting "yes" and "no" answers) is in $\mathcal{P}$. Because deterministic algorithms can easily invert their boolean output in $\mathcal{O}(1)$ time without affecting the polynomial time bound, $\mathcal{P}$ is closed under complementation. Thus, $\mathcal{P} = \text{Co-}\mathcal{P}$.
- **$\mathcal{NP}$ (Nondeterministic Polynomial Time):** The class of decision problems for which a "yes" answer has a polynomial-size certificate (or proof) that can be *verified* by a deterministic Turing machine in polynomial time. Equivalently, it is the class of problems solvable by a nondeterministic Turing machine in polynomial time.
- **Co- $\mathcal{NP}$:** The class of decision problems whose complement is in $\mathcal{NP}$. Equivalently, it is the class of problems for which a "no" answer has a polynomial-size certificate that can be verified in polynomial time.
- **$\mathcal{NP}$ -hard:** A class of problems that are at least as hard as the hardest problems in $\mathcal{NP}$. Formally, a problem H is $\mathcal{NP}$ -hard if *every* problem $L \in \mathcal{NP}$ can be reduced to H in polynomial time ($L \leq_p H$). $\mathcal{NP}$ -hard problems do not themselves have to be in $\mathcal{NP}$ (they can be undecidable).
- **$\mathcal{NP}$ -complete ($\mathcal{NPC}$):** The intersection of $\mathcal{NP}$ and $\mathcal{NP}$ -hard. A decision problem is $\mathcal{NP}$ -complete if it is *both* in $\mathcal{NP}$ and is $\mathcal{NP}$ -hard.

3. Diagram of the Most Likely Relationship ($\mathcal{P} \neq \mathcal{NP}$)

Most researchers strongly believe that $\mathcal{P} \neq \mathcal{NP}$ and $\mathcal{NP} \neq \text{Co-}\mathcal{NP}$. Under this assumption, the relationship among these classes is depicted below:



Q5. Show that the decision version of the vertex cover optimization problem is NP-complete using a reduction from the Independent Set decision problem. (12 marks) [CSE 207 | L-2/T-2 | 2012–13]

Answer

To prove that the Vertex Cover (VC) decision problem is NP-complete, we must satisfy two conditions:

- Show that **VC** $\in \mathcal{NP}$.
- Show that **VC** is **NP-hard** by providing a polynomial-time reduction from a known NP-complete problem (in this case,

Independent Set) to VC.

1. Formal Problem Definitions

- **Vertex Cover (VC):** Given an undirected graph $G = (V, E)$ and an integer k , does there exist a subset $V' \subseteq V$ such that $|V'| \leq k$ and for every edge $(u, v) \in E$, at least one of u or v belongs to V' ?
- **Independent Set (IS):** Given an undirected graph $G = (V, E)$ and an integer m , does there exist a subset $S \subseteq V$ such that $|S| \geq m$ and for all $u, v \in S$, $(u, v) \notin E$?

2. Proof of Membership in $\mathcal{NP}$

A problem is in $\mathcal{NP}$ if a proposed solution (certificate) can be verified in polynomial time.

- **Certificate:** A subset $V' \subseteq V$ containing at most k vertices.
- **Verification Algorithm:**
 - Count the number of vertices in V' . If $|V'| > k$, reject.
 - Iterate through every edge $(u, v) \in E$. If neither $u \in V'$ nor $v \in V'$, reject.
 - If all edges are covered, accept.
- **Time Complexity:** Checking the size of V' takes $\mathcal{O}(|V|)$ time. Iterating through edges takes $\mathcal{O}(|E|)$ time. The total verification time is $\mathcal{O}(|V| + |E|)$, which is polynomial. Therefore, **VC** $\in \mathcal{NP}$.

3. Polynomial-Time Reduction ($\text{IS} \leq_p \text{VC}$)

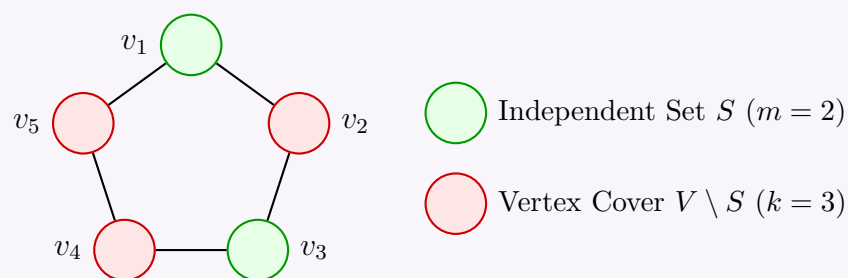
We must construct a function f that transforms an arbitrary instance of the Independent Set problem $\langle G = (V, E), m \rangle$ into an instance of the Vertex Cover problem $\langle G' = (V', E'), k \rangle$ in polynomial time.

Construction: Given $\langle G = (V, E), m \rangle$:

- Keep the exact same graph: $G' = G$ (so $V' = V$ and $E' = E$).
- Set the target vertex cover size to: $k = |V| - m$.

This transformation simply requires counting the vertices and doing a single subtraction. It takes $\mathcal{O}(1)$ time if $|V|$ is known, or $\mathcal{O}(|V|)$ to count, both of which are strictly polynomial.

Visualization of the Relationship: The fundamental intuition behind this reduction is that a set of vertices S is an Independent Set if and only if its complement $V \setminus S$ is a Vertex Cover.



4. Proof of Correctness (G has an IS of size $m \iff G'$ has a VC of size k)

Forward Direction ($\implies$): Assume G has an independent set S of size $\geq m$. We must show $V' = V \setminus S$ is a vertex cover of size $\leq k$.

- Size constraint:** Since $|S| \geq m$, the size of the complement is $|V'| = |V| - |S| \leq |V| - m = k$.
- Coverage constraint:** Consider any arbitrary edge $(u, v) \in E$. Because S is an independent set, u and v cannot *both* belong to S (otherwise, there would be an edge between two vertices in the independent set, violating its definition). Therefore, at least one of u or v must not be in S . By definition of the complement, at least one of u or v must be in $V \setminus S = V'$.
- Thus, V' covers all edges and is a valid Vertex Cover of size k .

Backward Direction ($\Leftarrow$): Assume G' (which is G) has a vertex cover V' of size $\leq k$. We must show $S = V \setminus V'$ is an independent set of size $\geq m$.

- (a) **Size constraint:** Since $|V'| \leq k = |V| - m$, the size of the complement is $|S| = |V| - |V'| \geq |V| - (|V| - m) = m$.
- (b) **Independence constraint:** Assume, for the sake of contradiction, that S is not an independent set. This means there exists an edge $(u, v) \in E$ where both $u \in S$ and $v \in S$. If both u and v are in S , then neither u nor v is in V' (since S and V' are disjoint). However, if neither u nor v is in V' , then the edge (u, v) is not covered by V' . This contradicts our assumption that V' is a valid vertex cover.
- (c) Therefore, no two vertices in S can share an edge, meaning S is an Independent Set of size m .

Conclusion

We have shown that VC is in $\mathcal{NP}$, and we have provided a valid polynomial-time reduction from the NP-complete Independent Set problem to the Vertex Cover problem. Therefore, the **Vertex Cover decision problem is NP-complete**.

Q6. Prove that the optimization version of the Independent Set problem is NP-hard. (8 marks) [CSE 207 | L-2/T-2 | 2012–13]

Answer

To prove that the optimization version of the Independent Set problem is NP-hard, we use a **Turing reduction** from its corresponding decision version. Standard Karp reductions (many-one reductions) map decision problems to decision problems, but to relate a decision problem to an optimization problem, we show that if we had a polynomial-time solver (an "oracle") for the optimization problem, we could solve the NP-complete decision problem in polynomial time.

1. Formal Problem Definitions

- **Decision Independent Set (IS-DEC):** Given a graph $G = (V, E)$ and an integer k , does G contain an independent set of size at least k ? (This is a known NP-complete problem).
- **Optimization Independent Set (IS-OPT):** Given a graph $G = (V, E)$, find an independent set $S_{max} \subseteq V$ of maximum possible size.

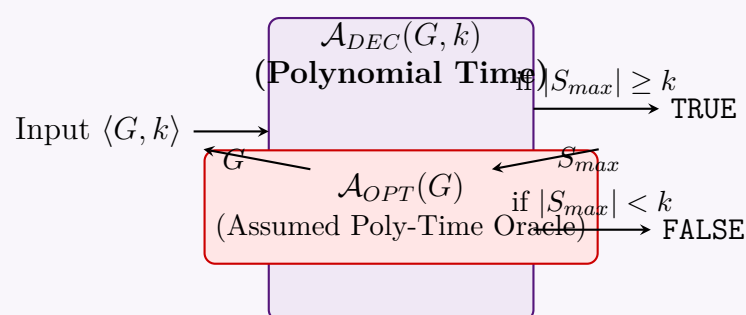
2. Polynomial-Time Turing Reduction ($\text{IS-DEC} \leq_T^p \text{IS-OPT}$)

Assume, for the sake of contradiction, that there exists a hypothetical algorithm $\mathcal{A}_{OPT}$ that solves the **IS-OPT** problem in polynomial time. We construct an algorithm $\mathcal{A}_{DEC}$ to solve the **IS-DEC** problem using $\mathcal{A}_{OPT}$ as a subroutine.

Algorithm 65 Solving IS-DEC using IS-OPT Oracle

- 1: **Input:** A graph $G = (V, E)$ and an integer k .
- 2: **Output:** TRUE if G has an independent set of size $\geq k$, else FALSE.
- 3: $S_{max} \leftarrow \mathcal{A}_{OPT}(G)$ ▷ Call the optimization oracle
- 4: $m \leftarrow |S_{max}|$ ▷ Find the size of the maximum independent set
- 5: **if** $m \geq k$ **then**
- 6: **return** TRUE
- 7: **else**
- 8: **return** FALSE
- 9: **end if**

Diagram of the Turing Reduction:



3. Proof of Correctness

We must prove that $\mathcal{A}_{DEC}$ returns TRUE if and only if G contains an independent set of size at least k .

- **Forward Direction ($\Rightarrow$):** Suppose $\mathcal{A}_{DEC}$ returns TRUE. According to our algorithm, this happens if and only if $m \geq k$. Since m is the size of S_{max} (a valid independent set returned by $\mathcal{A}_{OPT}$), the graph contains at least one independent set of size $\geq k$.

- **Backward Direction ($\Leftarrow$):** Suppose G has an independent set S of size at least k (so $|S| \geq k$). By definition, the optimization oracle $\mathcal{A}_{OPT}$ returns an independent set S_{max} of the maximum possible size in G . Therefore, $|S_{max}| \geq |S| \geq k$. The algorithm $\mathcal{A}_{DEC}$ evaluates $m \geq k$ as TRUE and correctly accepts the input.

4. Complexity Analysis

- **Time Complexity:** The algorithm $\mathcal{A}_{DEC}$ performs exactly one call to the oracle $\mathcal{A}_{OPT}$. Counting the size of the set S_{max} takes $\mathcal{O}(|V|)$ time, and the comparison $m \geq k$ takes $\mathcal{O}(1)$ time. If $\mathcal{A}_{OPT}$ runs in polynomial time $\mathcal{P}_{OPT}(|V|, |E|)$, the total running time of $\mathcal{A}_{DEC}$ is $\mathcal{O}(\mathcal{P}_{OPT}(|V|, |E|) + |V|)$, which is strictly polynomial.

Conclusion

We have shown that if the optimization version of the Independent Set problem can be solved in polynomial time, then the NP-complete decision version can also be solved in polynomial time. This means the optimization version is at least as hard as the decision version. Therefore, **IS-OPT is NP-hard**.

(Note: Optimization problems do not simply answer YES/NO, so they technically do not belong to the class $\mathcal{NP}$, which is restricted to decision problems. Hence, the optimization version is specifically $\mathcal{NP}$ -hard, but not $\mathcal{NP}$ -complete.)

Q7. What are the traveling-salesman problem, Euler tour problem and Hamiltonian cycle problem? If $L_1 \leq_p L_2$ and $L_2 \leq_p L_3$, then prove that $L_1 \leq_p L_3$. (3+7=10 marks) [CSE 207 | L-2/T-2 | 2011–12]

Answer

1. Problem Definitions

[3 Marks]

- **Hamiltonian Cycle Problem:** Given an undirected (or directed) graph $G = (V, E)$, a *Hamiltonian cycle* is a simple cycle that visits every vertex $v \in V$ exactly once and returns to the starting vertex. The problem asks whether such a cycle exists in G . It is a well-known **NP-complete** problem.
- **Traveling-Salesman Problem (TSP):** The TSP is the weighted generalization of the Hamiltonian cycle problem. Given a complete graph $G = (V, E)$ with non-negative edge weights $w(u, v)$ and a cost bound K , the decision version of TSP asks whether there exists a Hamiltonian cycle (a tour visiting every vertex exactly once) with a total weight of at most K . It is also **NP-complete**.
- **Euler Tour Problem:** Given a connected graph $G = (V, E)$, an *Euler tour* (or Eulerian circuit) is a cycle that traverses every edge $e \in E$ exactly once (vertices may be visited multiple times). The problem asks whether such a tour exists in G . Unlike the previous two, this problem is in $\mathcal{P}$ and can be solved in $\mathcal{O}(|V| + |E|)$ time (a connected undirected graph has an Euler tour if and only if every vertex has an even degree).

2. Proof of Transitivity of Polynomial-Time Reductions

[7 Marks]

Theorem: If $L_1 \leq_p L_2$ and $L_2 \leq_p L_3$, then $L_1 \leq_p L_3$.

Proof: By the definition of a polynomial-time reduction ($L_A \leq_p L_B$), there exists a polynomial-time computable function that maps instances of L_A to instances of L_B while preserving the “yes/no” answer.

- (a) **Given $L_1 \leq_p L_2$:** There exists a function $f : \Sigma^* \rightarrow \Sigma^*$ computable in polynomial time such that for any input string x :

$$x \in L_1 \iff f(x) \in L_2$$

Let the time complexity of computing $f(x)$ be bounded by $p(|x|)$, where p is a polynomial (e.g., $p(n) = \mathcal{O}(n^k)$).

- (b) **Given $L_2 \leq_p L_3$:** There exists a function $g : \Sigma^* \rightarrow \Sigma^*$ computable in polynomial time such that for any input string y :

$$y \in L_2 \iff g(y) \in L_3$$

Let the time complexity of computing $g(y)$ be bounded by $q(|y|)$, where q is a polynomial (e.g., $q(m) = \mathcal{O}(m^j)$).

- (c) **Constructing the Reduction $L_1 \leq_p L_3$:** We define a new composite function $h : \Sigma^* \rightarrow \Sigma^*$ as:

$$h(x) = g(f(x))$$

To prove $L_1 \leq_p L_3$, we must show two things about $h(x)$:

- (i) **Correctness (Mapping Preserves Validity):** For any input x :

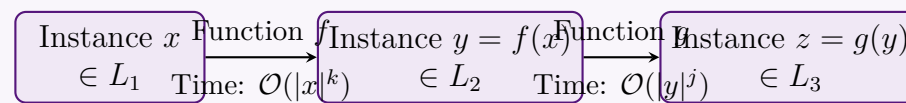
$$x \in L_1 \iff f(x) \in L_2 \iff g(f(x)) \in L_3 \iff h(x) \in L_3$$

Thus, h correctly maps “yes” instances of L_1 to “yes” instances of L_3 , and “no” instances to “no” instances.

- (ii) **Polynomial Time Bound:** We must show that $h(x)$ can be computed in polynomial time.

- The Turing machine computing $f(x)$ halts in at most $p(|x|)$ steps. Since a machine can write at most one symbol per step, the length of the output $f(x)$ is strictly bounded by the time it takes to compute it. Therefore, $|f(x)| \leq p(|x|)$.
- The function g takes the string $f(x)$ as input. The time to compute $g(f(x))$ is bounded by $q(|f(x)|)$.
- Substituting the maximum length of $f(x)$, the time to compute $g(f(x))$ is at most $q(p(|x|))$.

Because the composition of two polynomials $q(p(n))$ is itself a polynomial, the total time to compute $h(x)$ is polynomial in the size of the original input x .

Diagrammatic Representation:

Composite Function $h(x) = g(f(x))$ in Time $\mathcal{O}(|x|^{k \cdot j})$

Conclusion: Since $h(x)$ correctly maps instances of L_1 to L_3 and runs in polynomial time, we have successfully proven that $L_1 \leq_p L_3$. Polynomial-time reductions are transitive.

Q8. Prove that the decision version of the maximum clique problem is NP-complete by reducing from 3-CNF-SAT. (17 marks) [CSE 207 | L-2/T-2 | 2009–10]

Answer

To prove that the decision version of the Maximum Clique problem is NP-complete, we must satisfy two fundamental conditions:

- Show that **CLIQUE** $\in \mathcal{NP}$.
- Show that **CLIQUE** is **NP-hard** by providing a polynomial-time reduction from a known NP-complete problem, specifically **3-CNF-SAT**, to **CLIQUE** ($3\text{-CNF-SAT} \leq_p \text{CLIQUE}$).

1. Formal Problem Definitions

- 3-CNF-SAT:** Given a Boolean formula ϕ in Conjunctive Normal Form where each clause contains exactly 3 literals (e.g., $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$), is there a truth assignment to the variables that makes ϕ evaluate to **TRUE**?
- CLIQUE:** Given an undirected graph $G = (V, E)$ and an integer K , does G contain a complete subgraph (clique) of size at least K ?

2. Proof of Membership in $\mathcal{NP}$

A problem is in $\mathcal{NP}$ if any proposed solution (certificate) can be verified by a deterministic Turing machine in polynomial time.

- Certificate:** A subset of vertices $V' \subseteq V$ where $|V'| = K$.
- Verification:** Iterate through all pairs of vertices $u, v \in V'$. Check if the edge (u, v) exists in E .
- Time Complexity:** There are $\binom{K}{2}$ pairs to check, which takes $\mathcal{O}(K^2)$ time. Since $K \leq |V|$, the verification takes $\mathcal{O}(|V|^2)$ time, which is strictly polynomial. Thus, **CLIQUE** $\in \mathcal{NP}$.

3. Polynomial-Time Reduction ($3\text{-CNF-SAT} \leq_p \text{CLIQUE}$)

We must construct a polynomial-time algorithm that transforms any 3-CNF-SAT instance ϕ into a graph instance $\langle G, K \rangle$ such that ϕ is satisfiable *if and only if* G has a clique of size K .

Construction Algorithm: Let $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$ be a formula with k clauses, where each clause $C_r = (l_1^r \vee l_2^r \vee l_3^r)$.

Algorithm 66 Reduction from 3-CNF-SAT to CLIQUE

- Input:** A 3-CNF formula ϕ with k clauses.
- Output:** An undirected graph $G = (V, E)$ and target clique size K .
- $V \leftarrow \emptyset, E \leftarrow \emptyset, K \leftarrow k$
- for** each clause $C_r = (l_1^r \vee l_2^r \vee l_3^r)$ **for** $r = 1 \dots k$ **do**
 - Add vertices v_1^r, v_2^r, v_3^r to V . ▷ Create a vertex for every literal
- end for**
- for** each pair of vertices $v_i^r, v_j^s \in V$ **do**
 - if** $r \neq s$ **then** ▷ Vertices belong to **different** clauses
 - if** $l_i^r \neq \neg l_j^s$ **then** ▷ Literals are **not** logical negations of each other
 - Add edge (v_i^r, v_j^s) to E .
 - end if**
- end if**
- end for**
- return** $\langle G, K \rangle$

Complexity of Reduction:

- Time & Space Complexity:** The algorithm creates exactly $3k$ vertices. It checks every pair of vertices to add edges, requiring $\binom{3k}{2}$ comparisons. The total time and space complexity is $\mathcal{O}(k^2)$, which is polynomial in the size of the input formula.

Visualization of the Construction: Consider a simple instance $\phi = C_1 \wedge C_2$ where $C_1 = (x_1 \vee \neg x_2 \vee x_3)$ and $C_2 = (\neg x_1 \vee \neg x_2 \vee \neg x_3)$. We map this to a graph seeking a clique of size $K = 2$.

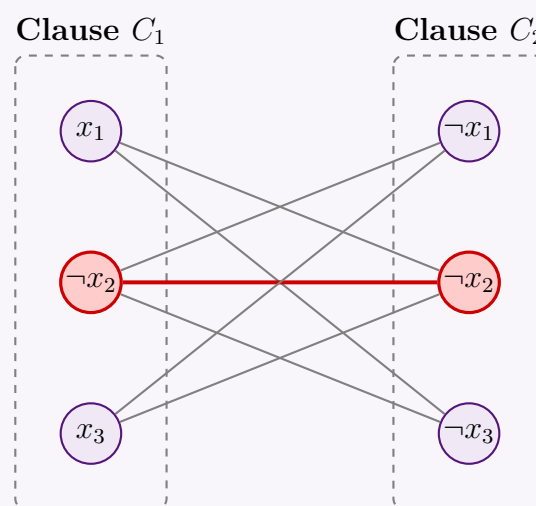


Figure: Edges connect non-contradictory literals from different clauses. The highlighted nodes $(\neg x_2, \neg x_2)$ form a $K = 2$ clique, corresponding to the satisfying assignment $x_2 = \text{False}$.

4. Proof of Correctness (ϕ is satisfiable $\iff G$ has a K -clique)

Forward Direction ($\implies$): Assume ϕ is satisfiable.

- There exists a truth assignment that makes at least one literal **TRUE** in every clause C_r ($1 \leq r \leq k$).
- From each clause C_r , select exactly one vertex v_i^r corresponding to a **TRUE** literal. This gives us a subset V' of exactly $k = K$ vertices.
- We must show V' is a clique. Take any two vertices $v_i^r, v_j^s \in V'$.
- Because we picked one vertex per clause, $r \neq s$.
- Because both literals evaluate to **TRUE** under the same assignment, they cannot be logical opposites (i.e., we cannot have x and $\neg x$ both **TRUE**). Thus, $l_i^r \neq \neg l_j^s$.
- By our construction algorithm, an edge exists between v_i^r and v_j^s . Therefore, V' is a clique of size K .

Backward Direction ($\impliedby$): Assume G has a clique V' of size $K = k$.

- Since no edges exist between vertices in the same clause, a clique can contain at most one vertex from any given clause.
- Because the clique has size k and there are exactly k clauses, the clique must contain *exactly* one vertex from every clause.
- For every vertex $v_i^r \in V'$, assign its corresponding literal l_i^r to **TRUE**.
- This assignment is mathematically valid because all vertices in the clique are connected by edges. By construction, an edge guarantees that $l_i^r \neq \neg l_j^s$. Thus, we will never assign **TRUE** to both a variable and its negation.
- Assign any unmapped variables arbitrarily.
- Since every clause contains at least one literal set to **TRUE**, the entire formula ϕ is satisfied.

Conclusion

Because CLIQUE is in $\mathcal{NP}$ and 3-CNF-SAT can be reduced to CLIQUE in polynomial time, the **Maximum Clique Decision Problem is NP-complete**.

Q9. What do you mean by NP-hard and NP-complete problems? Prove that finding a clique in a graph is NP-complete. **(5+12 marks)**
[CSE 207 | L-2/T-2 | 2008–09]

Answer

1. Definitions of NP-Hard and NP-Complete

[5 Marks]

- NP-Hard:** A problem H is NP-hard if *every* problem $L \in \mathcal{NP}$ can be reduced to H in polynomial time (denoted as $L \leq_p H$). Intuitively, NP-hard problems are "at least as hard" as the hardest problems in $\mathcal{NP}$. An NP-hard problem does not necessarily have to be in $\mathcal{NP}$ (it could be undecidable, for example).
- NP-Complete ($\mathcal{NPC}$):** A problem C is NP-complete if it satisfies two strict conditions:
 - $C \in \mathcal{NP}$ (Its solutions can be verified in polynomial time).

(b) C is NP-hard (Every problem in $\mathcal{NP}$ reduces to it in polynomial time).

The $\mathcal{NPC}$ class represents the absolute hardest problems *within* the $\mathcal{NP}$ complexity class.

2. Proof: Finding a Clique is NP-Complete

[12 Marks]

To formally prove that the **CLIQUE** decision problem is NP-complete, we must show that it is in $\mathcal{NP}$ and that it is NP-hard by reducing a known NP-complete problem to it. We will use the **Independent Set (IS)** problem for our reduction.

Definitions:

- **CLIQUE:** Given a graph $G = (V, E)$ and an integer K , does G contain a complete subgraph of size $\geq K$?
- **Independent Set (IS):** Given a graph $G = (V, E)$ and an integer k , does G contain a subset of vertices of size $\geq k$ such that no two vertices in the subset share an edge? (This is a known NP-complete problem).

Step 1: Prove $\text{CLIQUE} \in \mathcal{NP}$ A problem is in $\mathcal{NP}$ if a proposed solution (a certificate) can be verified in polynomial time.

- **Certificate:** A subset of vertices $V' \subseteq V$ where $|V'| = K$.
- **Verification:** Iterate through all pairs of distinct vertices $u, v \in V'$ and check if the edge (u, v) exists in E .
- **Complexity:** There are $\binom{K}{2}$ pairs to check. Checking an edge takes $\mathcal{O}(1)$ time using an adjacency matrix. The total verification time is $\mathcal{O}(K^2)$. Since $K \leq |V|$, this bounds to $\mathcal{O}(|V|^2)$, which is strictly polynomial. Thus, **CLIQUE** $\in \mathcal{NP}$.

Step 2: Polynomial-Time Reduction ($\text{IS} \leq_p \text{CLIQUE}$) We must construct an algorithm that transforms any Independent Set instance $\langle G, k \rangle$ into a CLIQUE instance $\langle G', K \rangle$ such that G has an independent set of size k *if and only if* G' has a clique of size K .

The core intuition is to use the **Complement Graph** $\overline{G}$.

Algorithm 67 Reduction from Independent Set to CLIQUE

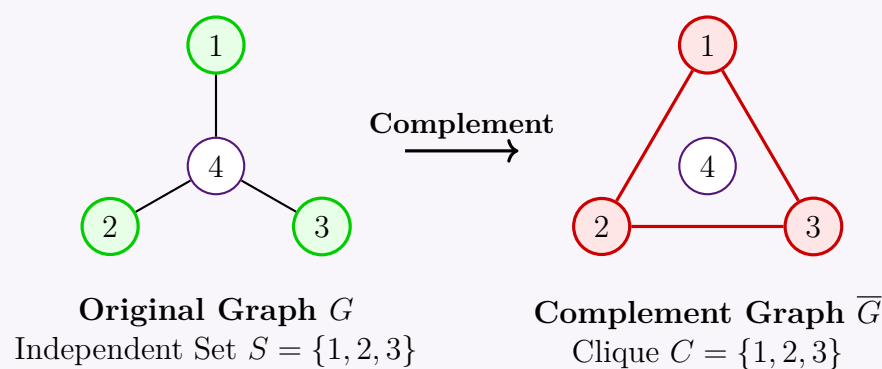
```

1: Input: Undirected graph  $G = (V, E)$  and integer  $k$ .
2: Output: Undirected graph  $\overline{G} = (V, \overline{E})$  and integer  $K$ .
3:  $\overline{E} \leftarrow \emptyset$  ▷ Initialize the complement edge set
4: for every pair of distinct vertices  $u, v \in V$  do
5:   if  $(u, v) \notin E$  then
6:      $\overline{E} \leftarrow \overline{E} \cup \{(u, v)\}$  ▷ Add edge if it does NOT exist in original graph
7:   end if
8: end for
9:  $K \leftarrow k$  ▷ The target size remains identical
10: return  $\langle \overline{G}, K \rangle$ 

```

Complexity of Reduction: Generating the complement graph requires checking every possible pair of vertices in V . There are exactly $\frac{|V|(|V|-1)}{2}$ pairs, so the reduction takes $\mathcal{O}(|V|^2)$ **time and space**, making it a valid polynomial-time reduction.

Visualization of the Reduction:



Step 3: Proof of Correctness We must prove that G has an independent set of size $k \iff \overline{G}$ has a clique of size k .

- **Forward Direction ($\implies$):** Assume G contains an independent set S of size k . By the definition of an independent set, for every pair of distinct vertices $u, v \in S$, the edge $(u, v) \notin E$. By the construction of our complement graph $\overline{G}$, an edge (u, v) is added to $\overline{E}$ if and only if $(u, v) \notin E$. Therefore, in $\overline{G}$, *every* pair of vertices in S is connected by an edge. This means S forms a complete subgraph (a clique) of size k in $\overline{G}$.
- **Backward Direction ($\impliedby$):** Assume $\overline{G}$ contains a clique C of size $K = k$. By the definition of a clique, for every pair of distinct vertices $u, v \in C$, the edge $(u, v) \in \overline{E}$. Based on our reduction algorithm, an edge exists in $\overline{E}$ if and only if it does *not* exist in E . Therefore, no two vertices in C share an edge in the original graph G . This means C is an independent set of size k in G .

Conclusion: Because **CLIQUE** $\in \mathcal{NP}$ and the known NP-complete problem Independent Set is reducible to CLIQUE in polynomial time, we conclude that finding a clique in a graph is **NP-complete**.

Q10. Write down the procedure of proving that a problem is NP-complete. (5 marks)

[CSE 207 | L-2/T-2 | 2008–09]

Answer

To formally prove that a given decision problem L is NP-complete, one must demonstrate that L is both in the class $\mathcal{NP}$ and is NP-hard. The standard procedure to establish this consists of the following four structured steps:

Step 1: Prove Membership in $\mathcal{NP}$ ($L \in \mathcal{NP}$)

First, you must show that the problem belongs to the complexity class $\mathcal{NP}$. This requires proving that if the answer to an instance of the problem is “YES”, there exists a proposed solution (often called a **certificate** or witness) that can be verified in polynomial time.

- Define what constitutes a valid certificate for the problem.
- Provide a deterministic verification algorithm.
- Analyze the verification algorithm to ensure its time complexity is bounded by a polynomial in the size of the input.

Step 2: Select a Known NP-Complete Problem

Choose a well-known decision problem L' that has already been proven to be NP-complete (e.g., 3-SAT, Vertex Cover, Clique, or Hamiltonian Cycle). The choice of L' is strategic; you should select a problem whose underlying structure closely resembles the target problem L .

Step 3: Construct a Polynomial-Time Reduction ($L' \leq_p L$)

Design an algorithm (or transformation function f) that maps any arbitrary instance x of the known problem L' to an instance $f(x)$ of the target problem L .

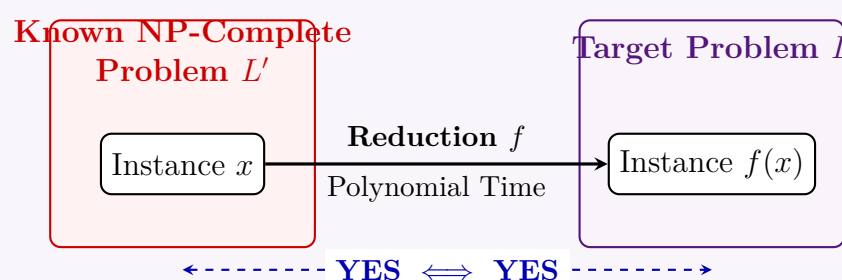
- **Polynomial Time:** You must strictly prove that the process of translating instance x into instance $f(x)$ takes time polynomial in the size of x .

Step 4: Prove Correctness of the Reduction

You must prove that the transformation preserves the validity of the instances. Specifically, you must show that x is a “YES” instance of L' if and only if $f(x)$ is a “YES” instance of L . This requires a two-part mathematical proof:

- **Forward Direction** ($\implies$): Prove that if $x \in L'$, then the constructed instance $f(x) \in L$.
- **Backward Direction** ($\impliedby$): Prove that if the constructed instance $f(x) \in L$, then the original instance $x \in L'$. (Alternatively, prove the contrapositive: if $x \notin L'$, then $f(x) \notin L$).

Visualizing the Reduction Process:



Conclusion: If all four steps are successfully demonstrated, the problem L is formally proven to be NP-complete.

Q11. There are clauses $C_1 = (X_1 \vee \overline{X_2})$, $C_2 = (\overline{X_1} \vee X_2 \vee X_4 \vee \overline{X_5})$, $C_3 = \overline{X_2}$, and $C_4 = (X_2 \vee X_3 \vee \overline{X_4})$. How the above instance of the satisfiability problem can be transformed into an instance of 3-SAT problem? (X_1, X_2, X_3, X_4 and X_5 are boolean variables) (13 marks)

[CSE 207 | L-2/T-2 | 2008–09]

Answer

To transform a general Satisfiability (SAT) instance into a 3-SAT instance, we must convert every clause C_i into a set of clauses where each clause contains exactly 3 literals. We achieve this by introducing auxiliary (dummy) boolean variables. The original SAT formula $\Phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$. We will transform each clause individually based on its length k (the number of literals).

1. General Transformation Rules

For a clause $C = (l_1 \vee l_2 \vee \dots \vee l_k)$ with k literals:

- **If $k = 1$:** Introduce two new auxiliary variables, Y_a and Y_b . Replace C with 4 clauses to cover all truth assignments of the

new variables:

$$(l_1 \vee Y_a \vee Y_b) \wedge (l_1 \vee Y_a \vee \overline{Y_b}) \wedge (l_1 \vee \overline{Y_a} \vee Y_b) \wedge (l_1 \vee \overline{Y_a} \vee \overline{Y_b})$$

- **If $k = 2$:** Introduce one new auxiliary variable, Y_c . Replace C with 2 clauses:

$$(l_1 \vee l_2 \vee Y_c) \wedge (l_1 \vee l_2 \vee \overline{Y_c})$$

- **If $k = 3$:** The clause is already in 3-SAT form. Keep it unchanged.
- **If $k > 3$:** Introduce $k - 3$ new auxiliary variables $Z_1, Z_2, \dots, Z_{k-3}$. Replace C with a "chain" of $k - 2$ clauses:

$$(l_1 \vee l_2 \vee Z_1) \wedge (\overline{Z_1} \vee l_3 \vee Z_2) \wedge \dots \wedge (\overline{Z_{k-3}} \vee l_{k-1} \vee l_k)$$

2. Step-by-Step Transformation of the Instance

Clause 1: $C_1 = (X_1 \vee \overline{X_2})$

- Here, $k = 2$.
- We introduce a new dummy variable Y_1 .
- The transformed clauses are:

$$C'_1 = (X_1 \vee \overline{X_2} \vee Y_1) \wedge (X_1 \vee \overline{X_2} \vee \overline{Y_1})$$

Clause 2: $C_2 = (\overline{X_1} \vee X_2 \vee X_4 \vee \overline{X_5})$

- Here, $k = 4$.
- We introduce $k - 3 = 1$ new dummy variable, let's call it Z_1 .
- We split the clause, linking the two parts with Z_1 and $\overline{Z_1}$.
- The transformed clauses are:

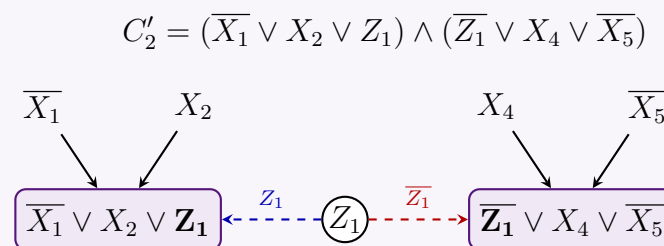


Figure: Splitting a 4-literal clause into two 3-literal clauses using a linking variable Z_1 .

Clause 3: $C_3 = (\overline{X_2})$

- Here, $k = 1$.
- We introduce two new dummy variables, Y_2 and Y_3 .
- The transformed clauses are:

$$C'_3 = (\overline{X_2} \vee Y_2 \vee Y_3) \wedge (\overline{X_2} \vee Y_2 \vee \overline{Y_3}) \wedge (\overline{X_2} \vee \overline{Y_2} \vee Y_3) \wedge (\overline{X_2} \vee \overline{Y_2} \vee \overline{Y_3})$$

Clause 4: $C_4 = (X_2 \vee X_3 \vee \overline{X_4})$

- Here, $k = 3$.
- The clause already has exactly 3 literals, so no dummy variables are needed.
- The transformed clause is:

$$C'_4 = (X_2 \vee X_3 \vee \overline{X_4})$$

3. Final 3-SAT Instance

The complete transformed 3-SAT instance, Φ' , is the conjunction of all the transformed clause sets $C'_1 \wedge C'_2 \wedge C'_3 \wedge C'_4$.

$$\begin{aligned} \Phi' = & (X_1 \vee \overline{X_2} \vee Y_1) \wedge (X_1 \vee \overline{X_2} \vee \overline{Y_1}) \\ & \wedge (\overline{X_1} \vee X_2 \vee Z_1) \wedge (\overline{Z_1} \vee X_4 \vee \overline{X_5}) \\ & \wedge (\overline{X_2} \vee Y_2 \vee Y_3) \wedge (\overline{X_2} \vee Y_2 \vee \overline{Y_3}) \wedge (\overline{X_2} \vee \overline{Y_2} \vee Y_3) \wedge (\overline{X_2} \vee \overline{Y_2} \vee \overline{Y_3}) \\ & \wedge (X_2 \vee X_3 \vee \overline{X_4}) \end{aligned}$$

4. Correctness and Complexity

- **Correctness:** The original formula Φ is satisfiable if and only if Φ' is satisfiable. Any satisfying assignment for Φ can be extended to satisfy Φ' by appropriately choosing the truth values of the dummy variables (e.g., Z_1 acts as a carry signal for the truth value).
- **Time Complexity:** Each clause of size k is replaced by $\max(1, k - 2)$ or at most 4 clauses. Generating these takes $\mathcal{O}(k)$ time per clause. Thus, the total reduction time is strictly polynomial: $\mathcal{O}(N)$ where N is the total length of the original formula.
- **Space Complexity:** $\mathcal{O}(N)$ auxiliary space to store the new variables and the new strictly 3-CNF string.

Q12. Define: (i) size of a problem, (ii) time complexity function, (iii) the class P, (iv) the class NP, (v) NP-Complete problems. Show how an instance of the satisfiability problem can be transformed into an instance of 3-SAT. Prove that 3-colouring of a graph is NP-complete. (10+10+15 marks) [CSE 207 | L-2/T-2 | 2007–08]

Answer

1. Definitions

[10 Marks]

- **(i) Size of a problem:** The number of bits (or symbols) required to represent a specific instance of the problem under a reasonable and concise encoding scheme. It is typically denoted as n and dictates the input size for analyzing algorithmic complexity.
- **(ii) Time complexity function:** A mathematical function $T(n)$ that expresses the worst-case maximum number of basic operations (or steps on a deterministic Turing machine) required to solve a problem instance of size n .
- **(iii) The class $\mathcal{P}$:** The set of all decision problems that can be solved (decided) by a deterministic Turing machine in polynomial time. That is, $T(n) = \mathcal{O}(n^k)$ for some constant k .
- **(iv) The class $\mathcal{NP}$:** The set of all decision problems for which a proposed solution (certificate) can be verified as correct by a deterministic Turing machine in polynomial time. Equivalently, it is the class of problems solvable in polynomial time by a *non-deterministic* Turing machine.
- **(v) NP-Complete problems ($\mathcal{NPC}$):** The hardest problems in $\mathcal{NP}$. A problem L is NP-Complete if:
 - (a) $L \in \mathcal{NP}$.
 - (b) Every problem $L' \in \mathcal{NP}$ is polynomial-time reducible to L ($L' \leq_p L$).

2. Transformation: SAT to 3-SAT

[10 Marks]

To transform a general Satisfiability (SAT) instance into a 3-SAT instance, we process every clause $C = (l_1 \vee l_2 \vee \dots \vee l_k)$ individually to ensure it contains exactly 3 literals, using auxiliary variables.

- **Case $k = 1$:** $C = (l_1)$. Introduce two new variables y_1, y_2 . Expand into 4 clauses:

$$(l_1 \vee y_1 \vee y_2) \wedge (l_1 \vee y_1 \vee \overline{y_2}) \wedge (l_1 \vee \overline{y_1} \vee y_2) \wedge (l_1 \vee \overline{y_1} \vee \overline{y_2})$$

- **Case $k = 2$:** $C = (l_1 \vee l_2)$. Introduce one new variable y_1 . Expand into 2 clauses:

$$(l_1 \vee l_2 \vee y_1) \wedge (l_1 \vee l_2 \vee \overline{y_1})$$

- **Case $k = 3$:** $C = (l_1 \vee l_2 \vee l_3)$. The clause is already in 3-CNF format. Leave as is.

- **Case $k > 3$:** $C = (l_1 \vee l_2 \vee \dots \vee l_k)$. Introduce $k - 3$ new variables $y_1, y_2, \dots, y_{k-3}$. Transform into a chain of $k - 2$ clauses:

$$(l_1 \vee l_2 \vee y_1) \wedge (\overline{y_1} \vee l_3 \vee y_2) \wedge (\overline{y_2} \vee l_4 \vee y_3) \wedge \dots \wedge (\overline{y_{k-3}} \vee l_{k-1} \vee l_k)$$

Complexity: This transformation takes time and space linear to the size of the original formula, $\mathcal{O}(N)$. Satisfiability is perfectly preserved.

3. Proof: 3-Coloring is NP-Complete

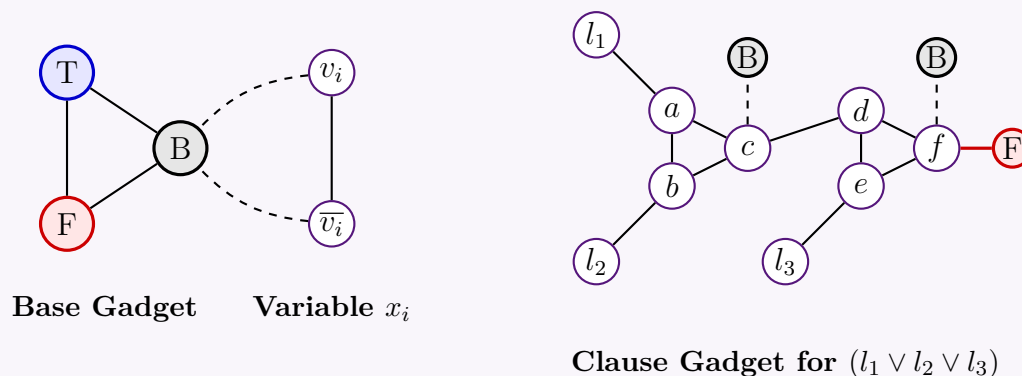
[15 Marks]

Step 1: Show 3-Coloring $\in \mathcal{NP}$

- **Certificate:** An array $C[1 \dots |V|]$ containing a color assignment $c \in \{1, 2, 3\}$ for each vertex.
- **Verification:** Iterate through every edge $(u, v) \in E$ and check if $C[u] \neq C[v]$. This takes $\mathcal{O}(|V| + |E|)$ time, which is polynomial.

Step 2: Reduction from 3-SAT ($3\text{-SAT} \leq_p 3\text{-Coloring}$) Given a 3-CNF formula Φ , we construct a graph G that is 3-colorable if and only if Φ is satisfiable. Let the colors be **T** (True), **F** (False), and **B** (Base).

- Base Gadget:** Create a triangle with vertices $\{T, F, B\}$. This forces the three vertices to each take a distinct color.
- Variable Gadget:** For each boolean variable x_i , create a pair of vertices v_i and $\bar{v}_i$. Connect them to each other and to the Base node B . This triangle $\{v_i, \bar{v}_i, B\}$ forces v_i and $\bar{v}_i$ to take the remaining colors T and F .
- Clause Gadget (OR-Gate):** For each clause $C_j = (l_1 \vee l_2 \vee l_3)$, we build a 6-node subgraph with vertices a, b, c, d, e, f .
 - Form two triangles: (a, b, c) and (d, e, f) .
 - Add edges $(l_1, a), (l_2, b), (B, c)$ and $(c, d), (l_3, e), (B, f)$.
 - Finally, connect f to the global F node. (This forces $f = T$).



Step 3: Correctness

- Forward ($\implies$):** If Φ is satisfiable, assign T and F colors to the variable gadgets based on the truth assignment. For every clause, at least one literal is T . By carefully checking the OR-gadget rules, if at least one input literal (l_1, l_2 , or l_3) is T , the graph allows vertex f to be colored T , satisfying the restrictive edge (f, F) . The graph is 3-colorable.
- Backward ($\impliedby$):** If G is 3-colorable, the colors of the variable gadgets yield a valid truth assignment. If there were a clause with all F literals ($l_1 = F, l_2 = F, l_3 = F$), the properties of triangles (a, b, c) and (d, e, f) combined with the connections to B would forcefully propagate and dictate that f *must* be colored F . However, f is connected to the global F node. An edge between two F -colored nodes violates 3-coloring. Thus, no clause can have all F literals. Φ is satisfiable.

Since the reduction is polynomial and preserves validity, 3-Coloring is **NP-Complete**.

Q13. Prove that 3-colouring of a graph is NP-complete.

[CSE 207 | L-2/T-2 | 2007–08]

Answer

To prove that the **3-Coloring** problem is NP-complete, we must formally demonstrate two properties:

- 3-Coloring $\in \mathcal{NP}$.
- 3-Coloring is NP-hard (by providing a polynomial-time reduction from a known NP-complete problem, such as **3-SAT**, to 3-Coloring).

Step 1: Prove 3-Coloring $\in \mathcal{NP}$

A problem is in $\mathcal{NP}$ if a proposed solution (certificate) can be verified in polynomial time.

- Certificate:** An array or mapping $C : V \rightarrow \{1, 2, 3\}$ assigning a color to each vertex in the graph $G = (V, E)$.
- Verification:** Iterate through every edge $(u, v) \in E$ and check if $C(u) \neq C(v)$.
- Time Complexity:** Checking all edges takes $\mathcal{O}(|V| + |E|)$ time, which is strictly polynomial in the size of the input. Thus, 3-Coloring $\in \mathcal{NP}$.

Step 2: Polynomial-Time Reduction ($3\text{-SAT} \leq_p 3\text{-Coloring}$)

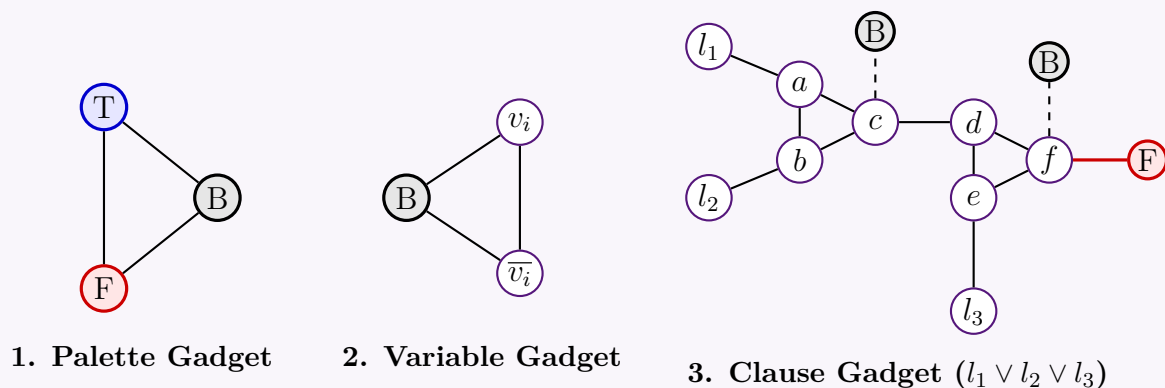
Given an arbitrary instance of 3-SAT (a boolean formula Φ with n variables and m clauses), we construct an undirected graph $G = (V, E)$ such that G is 3-colorable *if and only if* Φ is satisfiable. Let the three available colors be **T** (True), **F** (False), and **B** (Base).

We construct G using three types of "gadgets":

- The Palette (Base) Gadget** Create three vertices named T , F , and B , and connect them to form a triangle (K_3). *Effect:* This forces the three vertices to each take a distinct color. We identify these colors as the foundational T , F , and B values.
- The Variable Gadget** For each boolean variable x_i , create two vertices v_i and $\bar{v}_i$. Connect them to each other, and connect both of them to the B vertex of the Palette. *Effect:* The triangle $\{v_i, \bar{v}_i, B\}$ forces v_i and $\bar{v}_i$ to be colored with T and F in some order, perfectly modeling a boolean variable and its negation.

3. The Clause Gadget (OR-Gate) For each clause $C_j = (l_1 \vee l_2 \vee l_3)$, we construct a 6-vertex gadget (vertices a, b, c, d, e, f) to act as an OR-gate:

- Connect (l_1, a) and (l_2, b) .
- Form a triangle (a, b, c) . Connect c to the global B node.
- Connect c to d , and l_3 to e .
- Form a triangle (d, e, f) .
- Connect f to both the global B and global F nodes. (This severely restricts f ; it *must* be colored T).



Step 3: Proof of Correctness

We must show that Φ is satisfiable $\iff G$ is 3-colorable.

Backward Direction ($\Leftarrow$): Assume G is 3-colorable.

- Because of the Palette and Variable gadgets, every variable literal v_i is assigned either T or F . We use this to extract a truth assignment for Φ .
- Consider the Clause Gadget. Node f is connected to B and F , so f **must** be colored T .
- Suppose for contradiction that a clause is not satisfied (i.e., $l_1 = F$, $l_2 = F$, and $l_3 = F$).
- Since $l_1 = F$ and $l_2 = F$, the nodes a and b must be colored from $\{T, B\}$. Because (a, b) is an edge, one is T and the other is B .
- Therefore, c (which forms a triangle with a, b) must be colored F .
- Since $c = F$ and c connects to d , d must be colored from $\{T, B\}$.
- Since $l_3 = F$, node e must also be colored from $\{T, B\}$.
- Because (d, e) is an edge, one is T and the other is B . Therefore, f (which forms a triangle with d, e) **must** be colored F .
- However, we already established that f must be colored T . This creates a conflict ($T \neq F$), meaning the graph cannot be 3-colored if all three literals are F .
- Thus, in a valid 3-coloring, at least one literal per clause must be T , meaning Φ is satisfiable.

Forward Direction ($\implies$): Assume Φ is satisfiable.

- Color the Palette gadget with T, F, B . Color the variable gadgets according to the satisfying truth assignment (if x_i is True, color $v_i = T$ and $\bar{v}_i = F$).
- For each clause, since it evaluates to True, at least one literal is T .
- By symmetrically routing the colors through the OR-gate (Clause Gadget), we can always avoid the (F, F) contradiction at node f .
- For example, if $l_1 = T$, we can set $a = B$, $b = T$ (if $l_2 = F$), making $c = T$. Since $c = T$, d can be F . Then even if $l_3 = F \implies e \in \{T, B\}$, we can set $e = B$, allowing f to safely be T .
- Thus, the entire graph G can be validly 3-colored.

Conclusion

The construction of G requires $3 + 2n + 6m$ vertices and is clearly computable in polynomial time $\mathcal{O}(n + m)$. Since 3-Coloring is in $\mathcal{NP}$ and we have provided a polynomial-time reduction from the NP-complete 3-SAT problem, **3-Coloring is NP-complete**.

Q14. What is the difference between NP-complete and NP-hard problems?

Prove that the satisfiability of Boolean formulas is NP-complete.

Prove that $3\text{-SAT} \leq_p \text{Independent Set}$.

Give an example of reduction by equivalence. **(5+15+10 marks)**

[CSE 207 | L-2/T-2 | 2006–07]

Answer

1. Difference Between NP-Complete and NP-Hard

The complexity classes **NP-complete** and **NP-hard** classify the difficulty of computational problems relative to the class $\mathcal{NP}$ (Nondeterministic Polynomial time).

- **NP-Hard:** A problem H is NP-hard if *every* problem $L \in \mathcal{NP}$ can be reduced to H in polynomial time ($L \leq_p H$). Informally, H is "at least as hard" as the hardest problems in $\mathcal{NP}$. Crucially, an NP-hard problem *does not* have to be in $\mathcal{NP}$ itself (e.g., it could be an optimization problem, or even undecidable, like the Halting Problem).
- **NP-Complete (NPC):** A decision problem C is NP-complete if it satisfies **two** conditions:
 - (a) $C \in \mathcal{NP}$ (solutions can be verified in polynomial time).
 - (b) C is NP-hard.

In summary, NP-complete problems are the hardest problems *strictly within* the boundary of $\mathcal{NP}$, whereas NP-hard problems are at least that hard, but not necessarily restricted to $\mathcal{NP}$.

2. Prove that the Satisfiability of Boolean Formulas (SAT) is NP-Complete

This is the famous **Cook-Levin Theorem**. The proof requires showing that $\text{SAT} \in \mathcal{NP}$ and that SAT is NP-hard.

Step 1: $\text{SAT} \in \mathcal{NP}$

- **Certificate:** A boolean truth assignment to the variables $x_1, x_2, \dots, x_n$.
- **Verification:** Given the truth assignment, evaluate the formula ϕ . Evaluating a boolean formula takes time linearly proportional to its length $\mathcal{O}(|\phi|)$. Thus, verification is in polynomial time.

Step 2: SAT is NP-Hard (Generic Reduction)

We must show that for any language $L \in \mathcal{NP}$, $L \leq_p \text{SAT}$.

- By definition, if $L \in \mathcal{NP}$, there exists a nondeterministic Turing machine (NDTM) M that decides L in polynomial time $p(n)$ on input x of length n .
- We can construct a Boolean formula $\phi_{M,x}$ that algebraically "simulates" the computation of M on x .
- We introduce boolean variables to represent the state of M at every time step t from 0 to $p(n)$:
 - $Q_{i,t}$: True if M is in state q_i at time t .
 - $H_{j,t}$: True if the tape head is at position j at time t .
 - $T_{j,k,t}$: True if tape cell j contains symbol s_k at time t .
- The formula $\phi_{M,x}$ is the conjunction of several clause groups:

$$\phi_{M,x} = \phi_{\text{start}} \wedge \phi_{\text{accept}} \wedge \phi_{\text{valid_state}} \wedge \phi_{\text{transition}}$$

where ϕ_{start} encodes the initial state and input x , $\phi_{\text{valid_state}}$ ensures M is exactly in one state and reading one cell per time step, $\phi_{\text{transition}}$ enforces valid NDTM transition rules, and ϕ_{accept} asserts that at time $p(n)$, M is in an accepting state.

- The size of this formula is $\mathcal{O}(p(n)^2)$, and it can be constructed in polynomial time. $\phi_{M,x}$ is satisfiable *if and only if* there exists a valid sequence of non-deterministic choices leading M to accept x . Thus, $L \leq_p \text{SAT}$.

3. Prove that 3-SAT $\leq_p$ Independent Set

We must construct a polynomial-time reduction from 3-SAT to the Independent Set (IS) problem.

Construction: Let ϕ be a 3-SAT formula with k clauses: $C_1 \wedge C_2 \wedge \dots \wedge C_k$, where each $C_i = (l_{i1} \vee l_{i2} \vee l_{i3})$. We construct an undirected graph $G = (V, E)$ and an integer K as follows:

- For each clause C_i , create a triangle (a clique of 3 vertices) in G . Label the vertices with the literals l_{i1}, l_{i2}, l_{i3} . (This gives $3k$ vertices).
- Add "conflict edges" between any two vertices that represent complementary literals (e.g., an edge between a vertex labeled x_a and a vertex labeled $\overline{x_a}$).
- Set the target independent set size $K = k$.

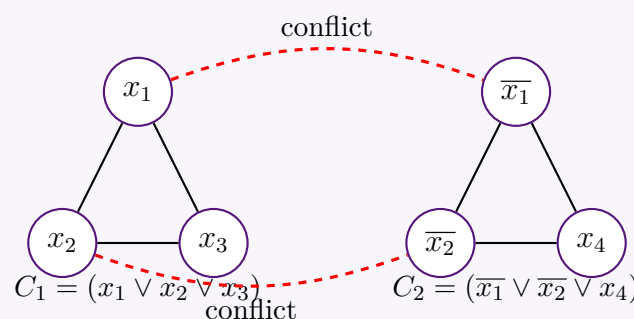


Figure: Graph construction for $\phi = (x_1 \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_4)$

Correctness Proof:

- **Forward ($\implies$):** If ϕ is satisfiable, there is a truth assignment that makes at least one literal true in every clause. Pick one true literal vertex from each triangle. Because we pick exactly one vertex from each of the k triangles, we have k vertices. Since they map to a valid truth assignment, we cannot pick both x_a and $\bar{x}_a$, meaning no conflict edges are chosen. Thus, G has an independent set of size k .
- **Backward ($\impliedby$):** If G has an independent set of size $K = k$, it must contain exactly one vertex from each triangle (since picking two from the same triangle violates independence). Assign the literal associated with each chosen vertex to True. Because there are no edges between chosen vertices, we never simultaneously assign True to both x_a and $\bar{x}_a$. This gives a consistent truth assignment satisfying all k clauses.

Since the graph is constructed in $\mathcal{O}(k)$ time, $3\text{-SAT} \leq_p \text{Independent Set}$.

4. Example of Reduction by Equivalence

Two problems A and B are *polynomially equivalent* ($A \equiv_p B$) if there is a polynomial-time reduction from A to B ($A \leq_p B$) AND from B to A ($B \leq_p A$).

Example: Independent Set $\equiv_p$ Clique

- **Independent Set (IS):** Given $G = (V, E)$ and integer k , is there a set of k mutually non-adjacent vertices?
- **Clique:** Given $G' = (V', E')$ and integer k' , is there a set of k' mutually adjacent vertices (a complete subgraph)?

Reduction: For any graph G , let $\bar{G}$ be its complement graph (an edge exists in $\bar{G}$ if and only if it does *not* exist in G).

- **IS $\leq_p$ Clique:** A subset of vertices $S \subseteq V$ is an independent set in G of size $k \iff S$ is a clique in $\bar{G}$ of size k . The reduction maps $(G, k) \rightarrow (\bar{G}, k)$.
- **Clique $\leq_p$ IS:** A subset of vertices $S \subseteq V$ is a clique in G of size $k \iff S$ is an independent set in $\bar{G}$ of size k . The reduction maps $(G, k) \rightarrow (\bar{G}, k)$.

Since the transformation of computing a complement graph requires $\mathcal{O}(|V|^2)$ time, the reductions run in polynomial time in both directions, establishing computational equivalence.

BUET · CSE 207 · Data Structures & Algorithms II

Part V

Coping with Hardness

Approximation Algorithms, Backtracking, Branch and Bound

S5 — Coping with Hardness

Approximation Algorithms

Q1. Give an approximation algorithm for the vertex cover problem. Analyze the approximation ratio of your algorithm. **(15 marks)**
[CSE 207 | L-2/T-1 | 2024–25, 2018–19, 2016–17]

Answer

1. The Approximation Algorithm

The Vertex Cover problem asks for the minimum sized set of vertices such that every edge in the graph is incident to at least one vertex in the set. Finding the exact optimal vertex cover is NP-hard. However, a simple greedy approach based on finding a *maximal matching* yields a guaranteed 2-approximation.

Algorithm 68 2-Approximation for Vertex Cover

Require: An undirected graph $G = (V, E)$

Ensure: A vertex cover $C \subseteq V$

```

1:  $C \leftarrow \emptyset$ 
2:  $E' \leftarrow E$ 
3: while  $E'$  is not empty do
4:   Let  $(u, v)$  be an arbitrary edge in  $E'$ 
5:    $C \leftarrow C \cup \{u, v\}$ 
6:   Remove from  $E'$  every edge incident on either  $u$  or  $v$ 
7: end while
8: return  $C$ 

```

2. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|V| + |E|)$. We can implement this algorithm efficiently using adjacency lists. In the worst case, we iterate through all edges and vertices once while building the maximal matching and deleting incident edges.
- **Space Complexity:** $\mathcal{O}(|V| + |E|)$ to store the graph and the output set C .

3. Analysis of the Approximation Ratio

We must prove that the size of the vertex cover C returned by our algorithm is at most twice the size of the optimal vertex cover C^* . That is, we want to show an approximation ratio of $\rho = 2$.

Proof:

- Validity of the Cover:** The algorithm terminates only when E' is empty. An edge is removed from E' if and only if one of its endpoints is added to C . Therefore, every edge in E has at least one endpoint in C , meaning C is a valid vertex cover.
- Properties of the Chosen Edges:** Let M be the set of edges explicitly picked in step 4 of the algorithm. By the algorithm's design, whenever an edge (u, v) is added to M , all other edges incident to u or v are deleted. Thus, no two edges in M share an endpoint. The set M forms a *maximal matching*.
- Size of our Cover ($|C|$):** For every edge chosen in M , exactly 2 vertices are added to C . Therefore, the total size of our vertex cover is exactly:

$$|C| = 2|M|$$

- Lower Bound on Optimal Cover ($|C^*|$):** Consider the optimal vertex cover C^* . Since M is a matching, all edges in M are strictly disjoint (they share no vertices). To cover the edges in M , C^* **must** pick at least one distinct vertex for *each* edge in M . Consequently, the size of the optimal vertex cover must be at least the size of the matching M :

$$|C^*| \geq |M|$$

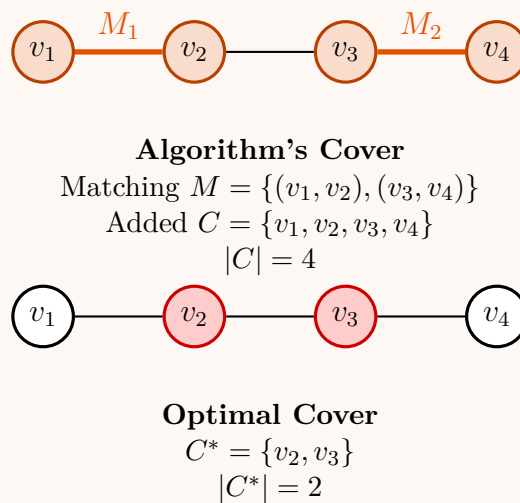
- Conclusion:** Substituting $|M| \leq |C^*|$ into our equation for $|C|$, we get:

$$|C| = 2|M| \leq 2|C^*|$$

This proves that the algorithm produces a vertex cover that is at most twice as large as the optimal one, giving a deterministic **approximation ratio of 2**.

4. Visual Example

The diagram below illustrates why the approximation yields a factor of 2. We are given a path graph.



Notice that in the worst-case scenario (like the path graph above), our algorithm picks edges (v_1, v_2) and (v_3, v_4) as the maximal matching, thus taking all 4 vertices. The optimal cover only requires v_2 and v_3 . Here, $|C| = 4$ and $|C^*| = 2$, perfectly matching our $|C| \leq 2|C^*|$ bound.

Q2. Let $\pi = \pi_1 \pi_2 \dots \pi_n$ be a permutation of $1 \ 2 \dots n$ (the identity permutation). A reversal (i, j) reverses the contiguous subsequence between i and j i.e. transforms $\pi_1 \dots \pi_{i-1} \pi_i \pi_{i+1} \dots \pi_{j-1} \pi_j \pi_{j+1} \dots \pi_n$ into $\pi_1 \dots \pi_{i-1} \pi_j \pi_{j-1} \dots \pi_{i+1} \pi_i \pi_{i+1} \dots \pi_n$. The sorting by reversals problem is to find the minimum number of reversals that transforms a given permutation to the identity permutation. For example, the permutation $[4 \ 1 \ 2 \ 3]$ can be sorted by reversing $[1 \ 2 \ 3]$ to get $[4 \ 3 \ 2 \ 1]$ and then reversing this entire sequence.

Let's extend the permutation by adding $\pi_0 = 0$ and $\pi_{n+1} = n + 1$ to the ends (π_0 and π_{n+1} are not moved during sorting). We say there is a breakpoint between π_i and π_{i+1} if they are not consecutive. For example, $[4 \ 1 \ 2 \ 3]$ is converted to $[0 \ 4 \ 1 \ 2 \ 3 \ 5]$ and there are three breakpoints: between 0 and 4, 4 and 1, and 3 and 5.

- Provide a lower bound on the number of reversals needed to sort a permutation π , in terms of the number of breakpoints in it.
- Design an approximation algorithm for the sorting by reversals problem with an approximation ratio at most 4. (Hint: consider maximal runs of contiguously increasing and contiguously decreasing elements.)

(15 marks)

[CSE 207 | L-2/T-1 | 2023–24]

Answer

Part 1: Lower Bound on the Number of Reversals

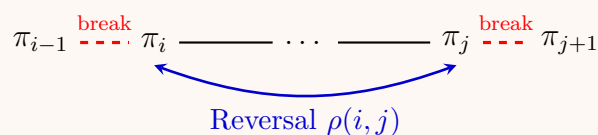
Let $\pi = [0, \pi_1, \pi_2, \dots, \pi_n, n + 1]$. We define a **breakpoint** to exist between adjacent elements π_i and π_{i+1} if they are not consecutive in value, i.e., $|\pi_i - \pi_{i+1}| \neq 1$. Let $b(\pi)$ denote the total number of breakpoints in the permutation π .

Theorem: The minimum number of reversals required to sort π , denoted as OPT , satisfies:

$$OPT \geq \left\lceil \frac{b(\pi)}{2} \right\rceil$$

Proof:

- Target State:** The sorted identity permutation $I = [0, 1, 2, \dots, n, n + 1]$ has exactly $b(I) = 0$ breakpoints.
- Effect of a Reversal:** A reversal $\rho(i, j)$ acting on the segment from index i to j only modifies the adjacencies at the boundaries of the segment. Specifically, it breaks the pairs (π_{i-1}, π_i) and (π_j, π_{j+1}) , and replaces them with (π_{i-1}, π_j) and (π_i, π_{j+1}) .
- Internal Pairs:** The pairs strictly inside the segment are flipped in order, but their absolute differences remain invariant. If $|\pi_k - \pi_{k+1}| = 1$, then $|\pi_{k+1} - \pi_k| = 1$. Thus, no internal breakpoints are created or removed.
- Conclusion:** Since a single reversal affects at most 2 adjacencies, it can eliminate at most 2 breakpoints. To reduce the number of breakpoints from $b(\pi)$ to 0, we mathematically require at least $b(\pi)/2$ reversals. Since the number of reversals must be an integer, $OPT \geq \lceil b(\pi)/2 \rceil$.



Part 2: 4-Approximation Algorithm

We define a **strip** as a maximal contiguous subsequence of elements without any internal breakpoints. A strip is *increasing* if it is strictly ascending (e.g., 4, 5, 6) and *decreasing* if descending (e.g., 8, 7, 6). A single element flanked by breakpoints is considered an increasing strip.

Algorithm 69 Greedy Breakpoint Reduction (4-Approximation)**Require:** Permutation $\pi = [0, \pi_1, \dots, \pi_n, n+1]$ **Ensure:** A sequence of reversals to sort π

```

1: while  $b(\pi) > 0$  do
2:   if  $\pi$  contains at least one decreasing strip then
3:     Let  $k$  be the smallest element in any decreasing strip.
4:     Find  $k-1$  (which must be at the right end of some increasing strip).
5:     if  $k-1$  appears before  $k$  in  $\pi$  then
6:       Let the segment be  $\dots, k-1 \mid x, \dots, k \mid \dots$ 
7:       Reverse the segment from  $x$  to  $k$ . ▷ Makes  $k-1$  and  $k$  adjacent
8:     else
9:       Let the segment be  $\dots, k \mid x, \dots, k-1 \mid \dots$ 
10:      Reverse the segment from  $x$  to  $k-1$ . ▷ Makes  $k$  and  $k-1$  adjacent
11:    end if
12:  else
13:    Let  $S$  be any increasing strip that does not contain 0 or  $n+1$ .
14:    Reverse the strip  $S$ . ▷ Turns an increasing strip into a decreasing one
15:  end if
16: end while

```

Part 3: Approximation Ratio Analysis

We must prove that the algorithm sorts the array using at most $4 \times OPT$ reversals. Let's analyze the greedy choices:

Case 1: A decreasing strip exists (Lines 3-10) By choosing k as the smallest element in a decreasing strip, we guarantee that $k-1$ is the largest element in an increasing strip. The algorithm performs a targeted reversal to bring k and $k-1$ together.

- If $k-1$ precedes k , the reversal of $x \dots k$ transforms $\dots, k-1 \mid x \dots k$ into $\dots, k-1, k \dots x$. The elements $k-1$ and k are now consecutive, eliminating at least the breakpoint between $k-1$ and x .
- If k precedes $k-1$, reversing $x \dots k-1$ transforms $k \mid x \dots k-1$ into $k, k-1 \dots x$. The sequence $k, k-1$ represents a valid continuation of a decreasing strip, removing at least one breakpoint.

Result: One reversal eliminates ≥ 1 breakpoint.

Case 2: No decreasing strips exist (Lines 11-14) All elements are structured in increasing strips separated by breakpoints. By picking an internal increasing strip S and reversing it, S becomes a *decreasing* strip.

- Since S was bounded by breakpoints, reversing it alters the boundary adjacencies, but does not increase the total number of breakpoints (worst-case: $\Delta b = 0$).
- Critically, in the *very next iteration*, the permutation is guaranteed to have a decreasing strip. This triggers Case 1, which will decrease the breakpoints by at least 1.

Result: At most 2 reversals eliminate ≥ 1 breakpoint.

Conclusion: In the absolute worst-case scenario, the algorithm requires 2 reversals to remove 1 breakpoint. Therefore, the total number of reversals R executed by the algorithm is bounded by:

$$R \leq 2 \cdot b(\pi)$$

From our lower bound in Part 1, we know that $b(\pi) \leq 2 \cdot OPT$. Substituting this into our worst-case performance gives:

$$R \leq 2 \cdot (2 \cdot OPT) = 4 \cdot OPT$$

Thus, the algorithm correctly yields an **approximation ratio of at most 4**.

Q3. Give a 2-approximation algorithm for the vertex cover problem and prove that the algorithm computes the vertex cover at most twice the size of the actual vertex cover size. **(15 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Answer**1. Introduction and Algorithm**

The **Vertex Cover** problem asks for the smallest set of vertices $C \subseteq V$ in an undirected graph $G = (V, E)$ such that every edge in E is incident to at least one vertex in C . Finding the optimal (minimum) vertex cover is known to be NP-complete. However, we can achieve a guaranteed **2-approximation** (a solution at most twice the optimal size) using a simple greedy approach that finds a *maximal matching*.

Algorithm 70 2-Approximation for Vertex Cover**Require:** An undirected graph $G = (V, E)$ **Ensure:** A vertex cover $C \subseteq V$

```

1:  $C \leftarrow \emptyset$  ▷ Initialize the vertex cover as empty
2:  $E' \leftarrow E$  ▷ Let  $E'$  be the set of remaining edges to cover
3: while  $E'$  is not empty do
4:   Let  $(u, v)$  be an arbitrary edge in  $E'$ 
5:    $C \leftarrow C \cup \{u, v\}$  ▷ Add both endpoints to the cover
6:   Remove from  $E'$  the edge  $(u, v)$  and every edge incident to  $u$  or  $v$ 
7: end while
8: return  $C$ 

```

2. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|V| + |E|)$. The algorithm can be efficiently implemented using an adjacency list. We scan the edges and vertices, and once an edge or vertex is processed and removed, it is not visited again.
- **Space Complexity:** $\mathcal{O}(|V| + |E|)$. We need this space to store the graph representations and the resulting set C .

3. Proof of the Approximation Ratio

We must prove two things: first, that the algorithm returns a valid vertex cover, and second, that its size is at most twice the size of the optimal vertex cover C^* .

Part A: Validity of the Cover The condition of the ‘while’ loop guarantees that the algorithm only terminates when E' is entirely empty. An edge is only removed from E' if at least one of its endpoints is placed into C . Therefore, by the time the algorithm finishes, every single edge in the original set E is incident to at least one vertex in C . Thus, C is a valid vertex cover.

Part B: Approximation Ratio Bound Let $M \subseteq E$ be the exact set of edges arbitrarily picked in **Step 5** of the algorithm.

- (a) **Properties of M :** When an edge (u, v) is added to M , the algorithm subsequently removes all other edges touching either u or v . This guarantees that no two edges in M share an endpoint. By definition, M forms a *matching* (specifically, a maximal matching).
- (b) **Size of our algorithm’s cover ($|C|$):** For every edge $(u, v) \in M$, the algorithm adds exactly two distinct vertices (u and v) to C . Therefore, the total size of our vertex cover is exactly:

$$|C| = 2|M| \quad (1)$$

- (c) **Lower bound on the optimal cover ($|C^*|$):** Let C^* be the optimal (minimum) vertex cover. Because M consists of strictly disjoint edges (no shared vertices), covering the edges in M inherently requires picking at least one distinct vertex for *every single edge* in M . Even an optimal algorithm cannot “cheat” by using one vertex to cover two edges of M . Thus, the optimal vertex cover must be at least as large as the matching:

$$|C^*| \geq |M| \quad (2)$$

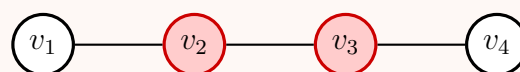
- (d) **Conclusion:** Substituting equation (2) into equation (1), we establish the approximation ratio:

$$|C| = 2|M| \leq 2|C^*| \quad (3)$$

This proves that the output C is strictly bounded by $2 \times |C^*|$, making it a valid 2-approximation.

4. Visual Example

The following diagram illustrates a scenario where the algorithm exactly hits the upper bound of the 2-approximation ratio.

**Algorithm’s Cover**Matching $M = \{(v_1, v_2), (v_3, v_4)\}$ Picked $C = \{v_1, v_2, v_3, v_4\}$ Size $|C| = 4$ **Optimal Cover** $C^* = \{v_2, v_3\}$ Size $|C^*| = 2$

In the path graph $v_1 - v_2 - v_3 - v_4$, if the algorithm greedily selects edge (v_1, v_2) first, it adds both v_1 and v_2 to C and removes (v_2, v_3) . It then is forced to pick (v_3, v_4) , adding v_3 and v_4 to C . Here, $|C| = 4$, while the optimal cover simply picks the internal nodes v_2, v_3 with $|C^*| = 2$. The relation $|C| \leq 2|C^*|$ holds perfectly.

- Q4.** Suppose you want to fill up your luggage with the required items for your upcoming tour. But the luggage has a capacity of 600 kg. The items have different values and weights which are given below:

Item	Value	Weight
1	1200	150
2	1100	100
3	950	250
4	1350	350
5	800	200
6	1500	300

You want to fill up your luggage so that the total values of the items are maximized. So you plan to use an approximation algorithm that you learned from your teacher to solve the problem. The approximation algorithm scales down the high values by $\epsilon V_{max}/2n$ (ϵ = precision parameter, V_{max} = largest value and n = total items) and utilizes a dynamic programming technique. The dynamic programming technique uses the subproblem $OPT(i, v)$ (the minimum weight of the luggage for which we can obtain a solution of value at least v using a subset of items 1 to i), instead of using $OPT(i, w)$ (the max value subset of items 1 to i with weight limit w).

- (a) **[Luggage maximization using scaling and dynamic programming]** Why does the approximation algorithm use the $OPT(i, v)$ subproblem instead of $OPT(i, w)$? **(5 marks)** [CSE 207 | L-2/T-2 | 2021–22]
- (b) Apply the approximation algorithm to solve the luggage problem with $\epsilon = 0.1$. **(15 marks)** [CSE 207 | L-2/T-2 | 2021–22]
- (c) Prove that for any $\epsilon > 0$, the approximation algorithm computes a feasible solution whose value is within a $(1 + \epsilon)$ factor of the optimum in $O(n^3/\epsilon)$. **(15 marks)** [CSE 207 | L-2/T-2 | 2021–22]

Answer

1. Why the approximation algorithm uses $OPT(i, v)$ instead of $OPT(i, w)$

The standard 0-1 Knapsack DP formulation uses $OPT(i, w)$, which computes the maximum value obtainable using a subset of the first i items bounded strictly by weight limit w . This yields a time complexity of $\mathcal{O}(nW)$, which is **pseudo-polynomial** because it grows exponentially with the number of bits required to represent the capacity W . For huge values of W , this becomes intractable. By swapping the roles of values and weights, the formulation $OPT(i, v)$ computes the **minimum weight** required to achieve exactly a value v using the first i items. This approach takes $\mathcal{O}(nV_{sum})$ time. The critical advantage here is that it acts as the foundation for a **Fully Polynomial-Time Approximation Scheme (FPTAS)**. We can systematically *scale down* the values to artificially shrink V_{sum} into a polynomial size, gaining computational speed at the cost of a small, controllable precision error. Conversely, scaling down *weights* is impossible because rounding errors could easily cause the selected items to exceed the rigid physical capacity W .

2. Applying the Approximation Algorithm ($\epsilon = 0.1$)

Step 1: Compute Scaling Factor Given variables: Capacity $W = 600$, Items $n = 6$, precision $\epsilon = 0.1$. The maximum value among all items is $V_{max} = 1500$ (Item 6). According to the provided formula, the scaling factor b is:

$$b = \frac{\epsilon V_{max}}{2n} = \frac{0.1 \times 1500}{2 \times 6} = \frac{150}{12} = 12.5$$

Step 2: Scale Down the Values We scale every item's value using $v'_i = \lfloor \frac{v_i}{b} \rfloor$:

Item (i)	Original Value (v_i)	Weight (w_i)	$v_i/12.5$	Scaled Value (v'_i)
1	1200	150	96	96
2	1100	100	88	88
3	950	250	76	76
4	1350	350	108	108
5	800	200	64	64
6	1500	300	120	120

Step 3: DP Execution via Pareto-Optimal States Rather than drawing a massive 6×552 DP matrix, we trace the states iteratively. Let S_i be the set of *Pareto-optimal reachable states* after considering item i . A state is a tuple (v', w) . We enforce the capacity limit $W = 600$. *Filtering rule:* State A strictly dominates state B if $v'_A \geq v'_B$ and $w_A \leq w_B$. Dominated states are discarded.

- Initialize:** $S_0 = \{(0, 0)\}$
- Add Item 1** (96, 150):
 $S_1 = \{(0, 0), (96, 150)\}$
- Add Item 2** (88, 100):
 New subsets: (88, 100) and $(96 + 88, 150 + 100) \implies (184, 250)$.
 $S_2 = \{(0, 0), (88, 100), (96, 150), (184, 250)\}$

- **Add Item 3** (76, 250):
New: (76, 250), (164, 350), (172, 400), (260, 500).
Filter: (76, 250) is dominated by (88, 100). (164, 350) and (172, 400) are dominated by (184, 250).
 $S_3 = \{(0, 0), (88, 100), (96, 150), (184, 250), (\mathbf{260, 500})\}$
- **Add Item 4** (108, 350):
New valid: (108, 350), (196, 450), (204, 500), (292, 600).
Filter: (108, 350) dom. by (184, 250). (204, 500) dom. by (260, 500).
 $S_4 = \{(0, 0), (88, 100), (96, 150), (184, 250), (\mathbf{196, 450}), (260, 500), (\mathbf{292, 600})\}$
- **Add Item 5** (64, 200):
New valid: (64, 200), (152, 300), (160, 350), (248, 450).
Filter: Only (248, 450) survives. Notably, it dominates (196, 450) from S_4 , forcing us to drop it.
 $S_5 = \{(0, 0), (88, 100), (96, 150), (184, 250), (\mathbf{248, 450}), (260, 500), (292, 600)\}$
- **Add Item 6** (120, 300):
New valid: (120, 300), (208, 400), (216, 450), (304, 550).
Filter: (120, 300) and (216, 450) are dominated. (304, 550) actually dominates (292, 600) from S_5 (higher value, lower weight!).
Drop (292, 600).
 $S_6 = \{(0, 0), (88, 100), (96, 150), (184, 250), (\mathbf{208, 400}), (248, 450), (260, 500), (\mathbf{304, 550})\}$

Step 4: Extract Optimal Solution The maximum scaled value obtained is $v'_{max} = 304$ with weight 550 kg. Tracing the generation back:

- (304, 550) generated in S_6 by adding **Item 6** (120, 300) to (184, 250).
- (184, 250) generated in S_2 by adding **Item 2** (88, 100) to (96, 150).
- (96, 150) generated in S_1 by adding **Item 1** (96, 150) to (0, 0).

Final Solution: Take Items **1, 2, and 6**.

Total Weight: $150 + 100 + 300 = \mathbf{550 \text{ kg}} \leq 600 \text{ kg}$.

Total Original Value: $1200 + 1100 + 1500 = \mathbf{3800}$.

3. Mathematical Proof of Approximation Factor & Complexity

Let O be the optimal subset of items yielding value $V(O) = OPT$. Let A be the subset output by our approximation algorithm yielding $V(A) = ALG$.

Approximation Factor Guarantee: By definition of the floor function for scaling, $v'_i = \lfloor \frac{v_i}{b} \rfloor \implies \frac{v_i}{b} - 1 < v'_i \leq \frac{v_i}{b}$.

This gives us two inequalities: $b \cdot v'_i \leq v_i$ and $b \cdot v'_i > v_i - b$.

Because subset A is the *exact optimal solution* for the scaled values, its total scaled value is at least the total scaled value of the true optimal subset O :

$$\sum_{i \in A} v'_i \geq \sum_{i \in O} v'_i$$

We bound the true value $V(A)$:

$$\begin{aligned} V(A) &= \sum_{i \in A} v_i \geq b \sum_{i \in A} v'_i \\ &\geq b \sum_{i \in O} v'_i \\ &\geq b \sum_{i \in O} \left(\frac{v_i}{b} - 1 \right) \\ &= \sum_{i \in O} v_i - b|O| \end{aligned}$$

Since the number of items in O cannot exceed n , $|O| \leq n$. Substituting $b = \frac{\varepsilon V_{max}}{2n}$:

$$\begin{aligned} V(A) &\geq V(O) - bn \\ &\geq V(O) - \left(\frac{\varepsilon V_{max}}{2n} \right) n \\ &\geq V(O) - \frac{\varepsilon}{2} V_{max} \end{aligned}$$

In the Knapsack problem (assuming any item fits individually), the maximum individual value V_{max} is a lower bound for the optimal total value, meaning $V_{max} \leq V(O)$. Substituting this into our inequality:

$$V(A) \geq V(O) - \frac{\varepsilon}{2} V(O) = V(O) \left(1 - \frac{\varepsilon}{2} \right)$$

To express this as an approximation ratio:

$$\frac{V(O)}{V(A)} \leq \frac{1}{1 - \varepsilon/2} = 1 + \frac{\varepsilon/2}{1 - \varepsilon/2} \leq 1 + \varepsilon \quad (\text{for small } \varepsilon)$$

This proves that the output is always within a $(1 + \varepsilon)$ factor of the optimal answer.

Time Complexity Analysis: The dynamic programming array has dimensions $n \times V'_{sum}$, where V'_{sum} is the maximum possible sum of scaled values.

$$V'_{sum} \leq n \cdot V'_{max} = n \left\lfloor \frac{V_{max}}{b} \right\rfloor$$

Substitute $b = \frac{\varepsilon V_{max}}{2n}$:

$$V'_{sum} \leq n \left\lfloor \frac{V_{max}}{\frac{\varepsilon V_{max}}{2n}} \right\rfloor = n \left\lfloor \frac{2n}{\varepsilon} \right\rfloor \leq \frac{2n^2}{\varepsilon}$$

The DP table has n rows and at most $\frac{2n^2}{\varepsilon}$ columns. Processing each cell requires $\mathcal{O}(1)$ time to check two conditions (include vs. exclude). Therefore, the overall time complexity is rigidly bounded by:

$$\mathcal{O}\left(n \times \frac{2n^2}{\varepsilon}\right) = \mathcal{O}\left(\frac{n^3}{\varepsilon}\right)$$

Q5. In the bottleneck traveling salesperson problem (TSP), we are given an undirected graph G with weights on its edges and asked to find a tour that visits the vertices of G exactly once and returns to the start so as to minimize the cost of the maximum-weight edge in the tour. Assuming that the weights in G satisfy the triangle inequality, design a polynomial-time 3-approximation algorithm for bottleneck TSP. (13 marks) [CSE 207 | L-2/T-2 | 2020–21]

Answer

1. Introduction and Lower Bound

The **Bottleneck Traveling Salesperson Problem (TSP)** asks for a Hamiltonian cycle that minimizes the maximum edge weight in the tour. Let OPT be the maximum edge weight of the optimal bottleneck tour H^* .

First, we establish a crucial lower bound using a Minimum Spanning Tree (MST):

- A Hamiltonian cycle H^* is a connected graph that spans all vertices.
- Removing any single edge from H^* yields a Hamiltonian path, which is a valid spanning tree of G . The maximum edge in this spanning tree is at most the maximum edge in H^* .
- Kruskal's or Prim's algorithm finds an MST T that not only minimizes the sum of edge weights but also inherently minimizes the *maximum edge weight* among all possible spanning trees (it is a Minimum Bottleneck Spanning Tree).
- Therefore, if W_{MST} is the weight of the largest edge in the MST T , we have:

$$W_{MST} \leq OPT \quad (1)$$

2. 3-Approximation Algorithm

To achieve a 3-approximation, we leverage **Sekanina's Theorem (1960)**, which states that the cube of any connected graph (and thus any tree) is Hamiltonian. The cube of a tree T , denoted T^3 , is a graph containing the same vertices as T , with an edge between u and v if and only if the shortest path distance between them in T is at most 3 edges.

Algorithm 71 3-Approximation for Bottleneck TSP

Require: Undirected graph $G = (V, E)$ with metric edge weights $c(u, v)$

Ensure: A Hamiltonian cycle C with bottleneck cost $\leq 3 \cdot OPT$

- 1: Compute a Minimum Spanning Tree T of G . ▷ Max edge is $W_{MST} \leq OPT$
 - 2: Construct the graph $T^3 = (V, E')$, where $(u, v) \in E'$ if and only if $d_T(u, v) \leq 3$.
 - 3: Find a Hamiltonian cycle C in T^3 . ▷ Guaranteed to exist by Sekanina's Theorem
 - 4: **return** C
-

3. Proof of the Approximation Ratio

We must prove that the maximum edge in our output cycle C is at most $3 \cdot OPT$.

- (a) **Path length constraint:** By the definition of T^3 , every edge $e' = (u, v)$ in our Hamiltonian cycle C corresponds to a path in T consisting of at most 3 edges. Let this path be $P_{uv} = \langle u, x_1, \dots, v \rangle$.
- (b) **Triangle Inequality:** We are given that the edge weights in G satisfy the triangle inequality. This allows us to strictly bound the direct distance $c(u, v)$ by the sum of the weights of the edges along the path P_{uv} :

$$c(u, v) \leq \sum_{e \in P_{uv}} c(e)$$

- (c) **Bottleneck Bound:** Since every individual edge in T has a weight of at most W_{MST} , and the path P_{uv} contains at most 3 edges, we deduce:

$$c(u, v) \leq 3 \cdot W_{MST}$$

(d) **Conclusion:** Substituting our lower bound from Equation (1), we get:

$$c(u, v) \leq 3 \cdot OPT$$

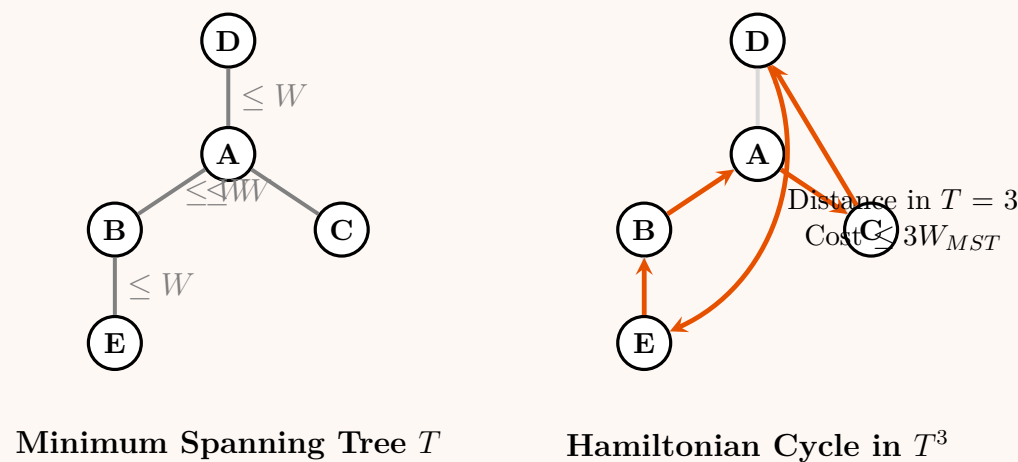
Since this holds true for every edge in C , the overall bottleneck cost of C is at most $3 \cdot OPT$. Thus, the algorithm is a valid 3-approximation.

4. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|V|^3)$. Finding the MST takes $\mathcal{O}(|E| \log |V|)$. Constructing T^3 can be done by running Breadth-First Search (BFS) up to depth 3 from each vertex, taking $\mathcal{O}(|V|^2)$. The constructive proof of Sekanina's theorem allows recursively finding the Hamiltonian cycle in T^3 in $\mathcal{O}(|V|^2)$ or $\mathcal{O}(|V|^3)$ time.
- **Space Complexity:** $\mathcal{O}(|V|^2)$. We require this space to explicitly store the dense adjacency matrix or list for the cube graph T^3 .

5. Visual Example

The diagram below illustrates how an edge in the T^3 Hamiltonian Cycle jumps across at most 3 tree edges, bounding its cost by $3 \times W_{MST}$.



Q6. What is fixed-parameter tractability? Why is it useful? (7 marks)

[CSE 207 | L-2/T-2 | 2020–21]

Answer

1. Definition of Fixed-Parameter Tractability (FPT)

In classical complexity theory, the running time of an algorithm is analyzed purely as a function of the input size, n . **Parameterized Complexity** introduces a secondary measurement called the *parameter*, denoted by k . This parameter k can represent the size of the desired solution, a structural property of the input (like the treewidth of a graph), or any other specific feature.

A parameterized problem is **Fixed-Parameter Tractable (FPT)** if there exists an algorithm that solves any instance (x, k) of the problem in time:

$$\mathcal{O}(f(k) \cdot |x|^c) \tag{1}$$

where:

- $|x|$ is the size of the input.
- k is the parameter.
- f is a computable function depending *only* on k (often exponential, e.g., 2^k or $k!$).
- c is a constant independent of both k and $|x|$.

2. Why is FPT Useful?

The primary utility of FPT lies in **isolating the combinatorial explosion**. Many important real-world problems are NP-Hard, meaning we do not expect to find algorithms that run in time polynomial in $|x|$. However, FPT provides a powerful workaround:

- Confinement of Hardness:** In an FPT algorithm, the exponential blow-up is strictly confined to the parameter k . If k is small in practical applications, $f(k)$ evaluates to a reasonable constant, making the overall running time $f(k) \cdot |x|^c$ virtually polynomial, even for massive input sizes $|x|$.
- Contrast with XP (Slice-wise Polynomial):** Consider a running time of $\mathcal{O}(|x|^k)$. While this is polynomial for any fixed k , it is practically useless. If $k = 10$ and $|x| = 1000$, $|x|^{10}$ is astronomically large. Conversely, an FPT algorithm running in $\mathcal{O}(2^k \cdot |x|)$ for the same values takes roughly $1024 \times 1000 \approx 10^6$ operations, which takes milliseconds on a modern computer.

- (c) **Practical Algorithm Design:** It provides a rigorous framework for designing exact algorithms for NP-Hard problems. Instead of settling for approximations or heuristics, FPT guarantees exact solutions efficiently when the parameter is reasonably small.

3. Canonical Example: Vertex Cover

To illustrate FPT, consider the **Vertex Cover** problem parameterized by the solution size k . **Problem:** Given a graph $G = (V, E)$ and an integer k , does there exist a set of vertices $S \subseteq V$ with $|S| \leq k$ such that every edge in E has at least one endpoint in S ? While Vertex Cover is NP-Complete, it is FPT parameterized by k . We can solve it using a *Bounded Search Tree* method.

Algorithm 72 FPT Algorithm for Vertex Cover (Bounded Search Tree)

Require: Graph $G = (V, E)$, integer k

Ensure: TRUE if G has a vertex cover of size $\leq k$, else FALSE

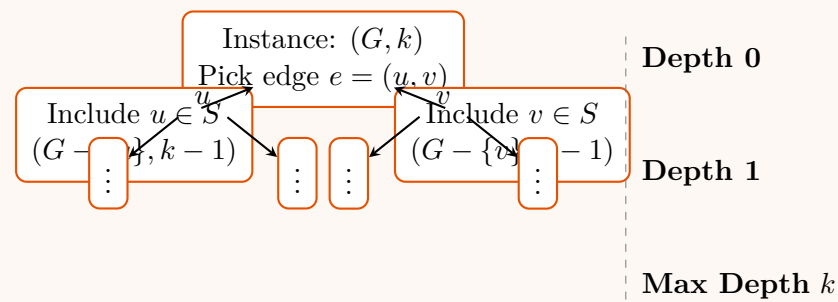
```

1: if  $E = \emptyset$  then
2:   return TRUE                                     ▷ All edges covered
3: end if
4: if  $k \leq 0$  and  $E \neq \emptyset$  then
5:   return FALSE                                     ▷ Run out of allowed vertices but edges remain
6: end if
7: Pick an arbitrary edge  $e = (u, v) \in E$ 
8:                                     ▷ At least one of  $u$  or  $v$  MUST be in the vertex cover
9:  $G_u \leftarrow G - \{u\}$                                      ▷ Graph after removing  $u$  and its incident edges
10:  $G_v \leftarrow G - \{v\}$                                      ▷ Graph after removing  $v$  and its incident edges
11: return VERTEXCOVER( $G_u, k - 1$ ) or VERTEXCOVER( $G_v, k - 1$ )

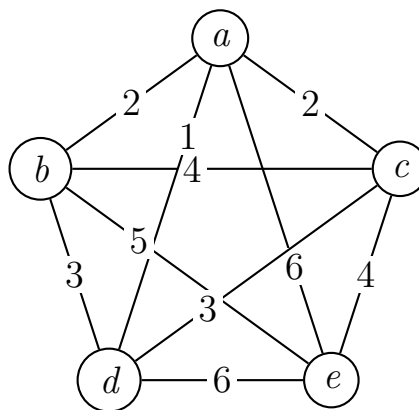
```

Complexity Analysis:

- **Time Complexity:** $\mathcal{O}(2^k \cdot |E|)$. The recursion tree has a branching factor of 2. The depth of the tree is bounded exactly by k (since k decreases by 1 at each step). Thus, there are at most 2^k leaves. At each node, selecting an edge and modifying the graph takes $\mathcal{O}(|E|)$ time. This perfectly matches the FPT formulation $f(k) \cdot |x|^c$.
- **Space Complexity:** $\mathcal{O}(k \cdot |E|)$ or $\mathcal{O}(|V| + |E|)$. The maximum depth of the recursion stack is k .



Q7. Show how to construct a near optimal solution using a minimum spanning tree for the symmetric traveling salesman problem with distances that satisfy the triangle inequality.



(12 marks)

[CSE 207 | L-2/T-2 | 2019–20]

Answer

1. Algorithm Overview (MST Heuristic for Metric TSP)

For the Metric Traveling Salesperson Problem (where edge weights satisfy the triangle inequality), we can construct a 2-approximation solution using the Minimum Spanning Tree (MST) heuristic. The standard steps are:

- Find a Minimum Spanning Tree T of the given graph G .
- Select an arbitrary root node and perform a **preorder walk** (Depth First Search) of T . This is equivalent to doubling the edges of T to create an Eulerian graph, finding an Eulerian tour, and applying shortcuts.

- (c) Create the Hamiltonian cycle (TSP tour) by visiting vertices in the order they first appear in the preorder walk. The triangle inequality guarantees that shortcutting past already-visited vertices will not increase the total tour cost.

2. Step-by-Step Execution

Step 2.1: Find the Minimum Spanning Tree (MST)

We apply Kruskal's Algorithm by sorting the edges in non-decreasing order of their weights:

- $(a, d) = 1$ (Add to T)
- $(a, b) = 2$ (Add to T)
- $(a, c) = 2$ (Add to T)
- $(b, d) = 3$ (Skip, forms cycle $a - b - d - a$)
- $(c, d) = 3$ (Skip, forms cycle $a - c - d - a$)
- $(b, c) = 4$ (Skip, forms cycle $a - b - c - a$)
- $(c, e) = 4$ (Add to T)

All 5 vertices are now connected. The MST T has edges $\{(a, d), (a, b), (a, c), (c, e)\}$ with a total weight of $1 + 2 + 2 + 4 = 9$.

Step 2.2: Preorder Walk of the MST

We root the MST at vertex a . Let's visit the children in alphabetical order:

- Start at a .
- Visit b (leaf). Return to a .
- Visit c . From c , visit e (leaf). Return to c , then return to a .
- Visit d (leaf). Return to a .

The full walk on the doubled MST is: $a \rightarrow b \rightarrow a \rightarrow c \rightarrow e \rightarrow c \rightarrow a \rightarrow d \rightarrow a$.

Step 2.3: Shortcutting to form the Tour

We extract the unique vertices in the order of their first appearance in the walk to form our Hamiltonian Cycle:

Tour: $a \rightarrow b \rightarrow c \rightarrow e \rightarrow d \rightarrow a$

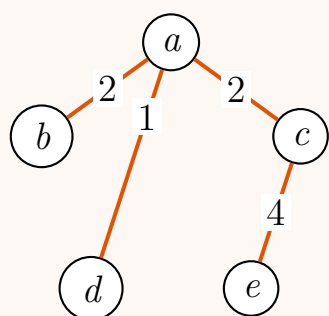
Step 2.4: Calculate the Tour Cost

We sum the weights of the edges in our finalized tour:

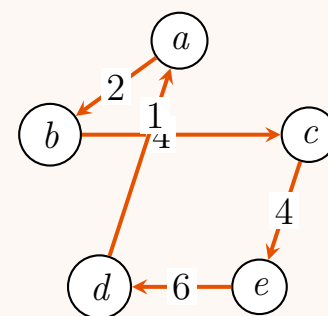
$$\begin{aligned} \text{Cost} &= \text{weight}(a, b) + \text{weight}(b, c) + \text{weight}(c, e) + \text{weight}(e, d) + \text{weight}(d, a) \\ \text{Cost} &= 2 + 4 + 4 + 6 + 1 = 17 \end{aligned}$$

3. Visualizing the Construction

Minimum Spanning Tree (MST)



Resulting TSP Tour



4. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|E| \log |V|)$. Finding the MST dominates the computational time. Using Kruskal's algorithm requires sorting the edges, which takes $\mathcal{O}(|E| \log |E|) = \mathcal{O}(|E| \log |V|)$ time. The subsequent preorder traversal and shortcutting visit each vertex and edge of the tree a constant number of times, taking $\mathcal{O}(|V|)$ time.
- **Space Complexity:** $\mathcal{O}(|V| + |E|)$. This space is required to represent the graph using an adjacency list, store the MST, and maintain the visited state or recursion stack during the DFS preorder walk.

Q8. Discuss the approximability of the general Traveling Salesperson Problem (TSP). Give an approximation algorithm for the Euclidean TSP problem and compute the approximation ratio of your algorithm. **(2+3+5 marks)** [CSE 207 | L-2/T-2 | 2017–18]

Answer

1. Approximability of the General TSP (2 Marks)

The general Traveling Salesperson Problem (TSP), where edge weights are arbitrary and not constrained by the triangle inequality, is highly inapproximable.

Theorem: For any polynomial-time computable function $\alpha(n) \geq 1$, there is no $\alpha(n)$ -approximation algorithm for the general TSP unless $\mathbf{P} = \mathbf{NP}$.

Proof (Reduction from Hamiltonian Cycle): Assume, for the sake of contradiction, that an $\alpha(n)$ -approximation algorithm $\mathcal{A}$ exists for general TSP. We can use $\mathcal{A}$ to solve the NP-Complete **Hamiltonian Cycle (HC)** problem in polynomial time. Given an unweighted graph $G = (V, E)$ with $|V| = n$, we construct a complete weighted graph $G' = (V, E')$ such that the weight function w is defined as:

$$w(u, v) = \begin{cases} 1 & \text{if } (u, v) \in E \\ \alpha(n) \cdot n + 1 & \text{if } (u, v) \notin E \end{cases} \quad (1)$$

- **Case 1: G has a Hamiltonian Cycle.** The corresponding optimal TSP tour in G' uses only edges from E , all of which have weight 1. The total cost of this optimal tour is exactly n . The algorithm $\mathcal{A}$ is guaranteed to return a tour of cost at most $\alpha(n) \cdot n$.
- **Case 2: G does not have a Hamiltonian Cycle.** Any valid TSP tour in G' must use at least one edge not in E . Thus, the total cost of any tour will be strictly greater than $(\alpha(n) \cdot n + 1) + (n - 1) > \alpha(n) \cdot n$.

By running $\mathcal{A}$ on G' , if the returned tour has a cost $\leq \alpha(n) \cdot n$, G must contain a Hamiltonian cycle. Otherwise, it does not. This perfectly decides the HC problem in polynomial time, implying $\mathbf{P} = \mathbf{NP}$.

2. Approximation Algorithm for Euclidean TSP (3 Marks)

The **Euclidean TSP** restricts the problem such that edge weights are Euclidean distances between points in space. This guarantees that the distances satisfy the **triangle inequality**: $w(u, v) \leq w(u, k) + w(k, v)$ for all vertices u, v, k . Because of this property, we can bypass the inapproximability of the general TSP and achieve a constant-factor approximation using the Minimum Spanning Tree (MST) heuristic.

Algorithm 73 MST-Based 2-Approximation for Metric/Euclidean TSP

Require: A complete undirected graph $G = (V, E)$ with Euclidean distance metric c

Ensure: A Hamiltonian cycle (TSP tour) H

- 1: Find a Minimum Spanning Tree (MST) T of G .
 - 2: Construct a multigraph M by doubling every edge in T . ▷ Every vertex now has even degree
 - 3: Find an Eulerian tour $\mathcal{E}$ in the multigraph M .
 - 4: Initialize H as an empty sequence of vertices.
 - 5: Traverse $\mathcal{E}$, appending each vertex to H only if it has not been visited yet (shortcutting).
 - 6: Append the starting vertex to the end of H to close the cycle.
 - 7: **return** H
-

3. Analysis of the Approximation Ratio (5 Marks)

Theorem: The MST-based algorithm achieves an approximation ratio of 2 for Euclidean TSP.

Proof: Let OPT be the optimal TSP tour, and $c(OPT)$ be its total cost. Let $c(T)$ be the total cost of the edges in the MST T . Let $c(\mathcal{E})$ be the cost of the Eulerian tour, and $c(H)$ be the cost of the final shortcut tour. We bound the cost in three distinct steps:

- (a) **Bounding the MST Cost:** An optimal TSP tour is a Hamiltonian cycle. If we remove any single edge from this optimal cycle, we obtain a spanning path of the graph G . Since a spanning path is simply a spanning tree with maximum degree 2, its total cost must be at least the cost of the true Minimum Spanning Tree T . Therefore:

$$c(T) \leq c(OPT) \quad (2)$$

- (b) **Bounding the Eulerian Tour Cost:** The multigraph M is constructed by taking exactly two copies of every edge in the MST T . Since the Eulerian tour $\mathcal{E}$ traverses every edge in M exactly once, its total cost is exactly twice the cost of the MST:

$$c(\mathcal{E}) = 2 \cdot c(T) \leq 2 \cdot c(OPT) \quad (3)$$

- (c) **Bounding the Shortcut Tour via Triangle Inequality:** The final Hamiltonian cycle H is formed by extracting unique vertices from $\mathcal{E}$ in the order of their first appearance. This implies that we take "shortcuts" over previously visited vertices. Because the edge weights are Euclidean distances, the **triangle inequality** strictly holds. Traveling directly from u to v is always shorter than or equal to traveling from u to v via some intermediate nodes. Thus, shortcutting never increases the total tour cost:

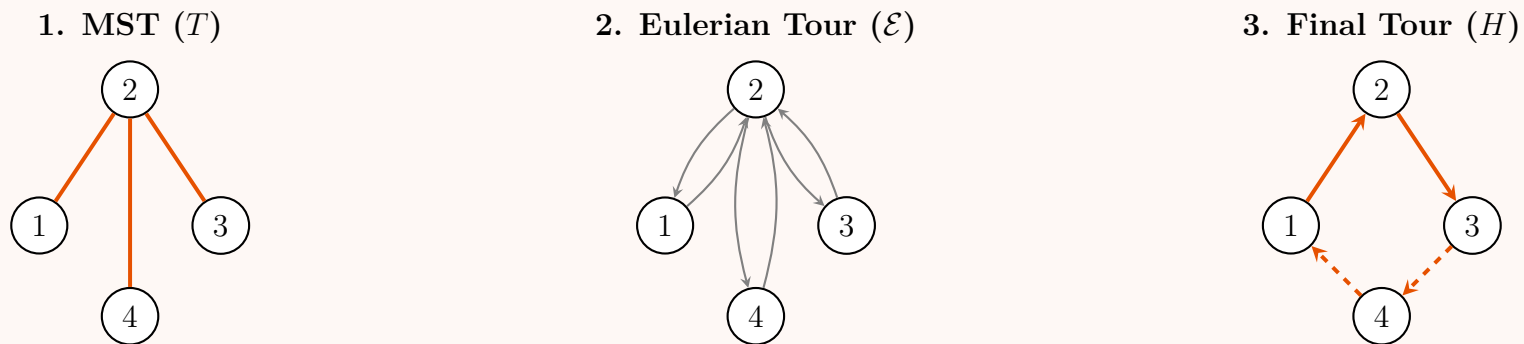
$$c(H) \leq c(\mathcal{E}) \quad (4)$$

(d) **Conclusion:** By transitivity across equations (1), (2), and (3), we conclude:

$$c(H) \leq c(\mathcal{E}) = 2 \cdot c(T) \leq 2 \cdot c(OPT) \quad (5)$$

Therefore, the cost of the algorithm's tour H is at most twice the optimal cost. ■

Visualization of the Algorithm:



Note: Dashed lines represent shortcuts bypassing previously visited vertex 2.

Q9. Give an approximation algorithm for the set cover problem and compute the approximation ratio. (3+7 marks)

[CSE 207 | L-2/T-2 | 2017–18]

Answer

1. Problem Definition and the Greedy Algorithm (3 Marks)

The **Set Cover Problem** is a classical NP-hard problem. Given a universe of elements $X = \{e_1, e_2, \dots, e_n\}$ and a family of subsets $\mathcal{F} = \{S_1, S_2, \dots, S_m\}$ where $\bigcup_{i=1}^m S_i = X$, the objective is to find a minimum-size subcollection $\mathcal{C} \subseteq \mathcal{F}$ that covers all elements in X . The most standard approximation algorithm for this problem is the **Greedy Approach**. The strategy is intuitive: at each step, select the set from $\mathcal{F}$ that covers the maximum number of currently *uncovered* elements, and repeat until the entire universe X is covered.

Algorithm 74 Greedy Set Cover

Require: Universe X , and a family of subsets $\mathcal{F}$

Ensure: A collection of sets $\mathcal{C} \subseteq \mathcal{F}$ that covers X

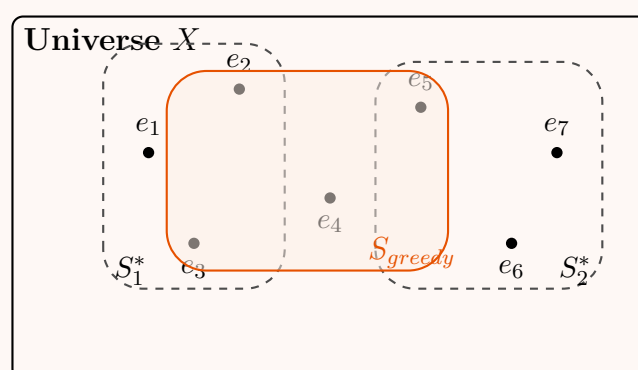
```

1:  $U \leftarrow X$ 
2:  $\mathcal{C} \leftarrow \emptyset$ 
3: while  $U \neq \emptyset$  do
4:   Select  $S \in \mathcal{F}$  that maximizes  $|S \cap U|$ 
5:    $U \leftarrow U \setminus S$ 
6:    $\mathcal{C} \leftarrow \mathcal{C} \cup \{S\}$ 
7: end while
8: return  $\mathcal{C}$ 

```

▷ U tracks the uncovered elements
 ▷ Initialize the chosen set cover
 ▷ Remove newly covered elements from U
 ▷ Add the set to our cover

2. Visualizing the Greedy Choice



Greedy Trap:

Optimal cover is $\{S_1^*, S_2^*\}$ (size 2). Greedy first picks S_{greedy} because it covers the most elements (4 elements) initially. It then needs 2 more sets to cover $\{e_1, e_2\}$ and $\{e_6, e_7\}$. Greedy output size = 3.

3. Analysis of the Approximation Ratio (7 Marks)

Theorem: The Greedy Set Cover algorithm is an H_d -approximation algorithm, where $d = \max_{S \in \mathcal{F}} |S|$ is the size of the largest set, and $H_d = \sum_{i=1}^d \frac{1}{i} \approx \ln d + O(1)$ is the d -th Harmonic number.

Proof by Cost Amortization: Let $\mathcal{C}^*$ be the optimal set cover, and let $OPT = |\mathcal{C}^*|$ be its size. We use a price-allocation scheme to analyze the greedy cost. Whenever the greedy algorithm selects a set S , it pays a cost of 1. We distribute this cost evenly among the newly covered elements. If S covers k previously uncovered elements, we assign a price to each of these newly covered elements x :

$$c_x = \frac{1}{k} \quad (1)$$

Since every element in X is covered exactly once for the first time, the total size of the greedy cover $|\mathcal{C}_{\text{greedy}}|$ is exactly the sum of the prices of all elements in X :

$$|\mathcal{C}_{\text{greedy}}| = \sum_{x \in X} c_x \quad (2)$$

Because $\mathcal{C}^*$ covers the entire universe X , we can bound this sum by looking at the sets in the optimal cover:

$$\sum_{x \in X} c_x \leq \sum_{S \in \mathcal{C}^*} \sum_{x \in S} c_x \quad (3)$$

Now, consider any arbitrary set $S \in \mathcal{C}^*$. Let its elements be $\{e_1, e_2, \dots, e_k\}$ (where $k = |S| \leq d$), ordered by the sequence in which the greedy algorithm covered them. At the moment before the greedy algorithm covers element e_i , the set S contains at least $k - i + 1$ uncovered elements (namely, $e_i, e_{i+1}, \dots, e_k$). Since the greedy algorithm always selects the set that covers the *maximum* number of uncovered elements, and S was a valid candidate covering at least $k - i + 1$ elements, the set that Greedy *actually* chose must have covered **at least** $k - i + 1$ elements. Therefore, the cost assigned to e_i is bounded by:

$$c_{e_i} \leq \frac{1}{k - i + 1} \quad (4)$$

Summing these costs over all elements in S :

$$\sum_{x \in S} c_x = \sum_{i=1}^k c_{e_i} \leq \sum_{i=1}^k \frac{1}{k - i + 1} = \frac{1}{k} + \frac{1}{k-1} + \dots + \frac{1}{1} = H_k \quad (5)$$

Since $k \leq d$, we have $H_k \leq H_d$. Therefore, for any $S \in \mathcal{C}^*$, the sum of costs is at most H_d . Substituting this back into equation (3):

$$|\mathcal{C}_{\text{greedy}}| \leq \sum_{S \in \mathcal{C}^*} H_d = H_d \cdot |\mathcal{C}^*| = H_d \cdot \text{OPT} \quad (6)$$

Since $H_d \leq \ln n + 1$ (where $n = |X|$), the algorithm achieves a logarithmic approximation ratio: $\mathcal{O}(\log n)$. ■

4. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|X| \cdot |\mathcal{F}|)$ or $\mathcal{O}(\sum_{S \in \mathcal{F}} |S|)$. In the worst case, the algorithm iterates $\mathcal{O}(|X|)$ times. In each iteration, it scans all remaining sets to find the one with the largest intersection with the uncovered set U . By maintaining the sizes of $|S \cap U|$ for all S and updating them efficiently, the total running time is linear in the sum of the sizes of all sets.
- **Space Complexity:** $\mathcal{O}(|X| + \sum_{S \in \mathcal{F}} |S|)$ to store the universe, the elements of each set in the family $\mathcal{F}$, and the auxiliary data structures to track uncovered elements.

Q10. What is the vertex cover problem? Write a polynomial-time approximation algorithm for it. Prove its time complexity and find the approximation ratio. For the general traveling-salesman problem, prove that one cannot find good approximate tours in polynomial time unless $P = NP$. (7+8=15 marks) [CSE 207 | L-2/T-2 | 2015–16]

Answer

Part A: The Vertex Cover Problem (7 Marks)

1. Definition

A **vertex cover** of an undirected graph $G = (V, E)$ is a subset of vertices $C \subseteq V$ such that for every edge $(u, v) \in E$, at least one of u or v belongs to C (i.e., $u \in C$ or $v \in C$). The *Vertex Cover Problem* is an NP-Complete optimization problem that seeks to find a vertex cover of minimum size.

2. Polynomial-Time Approximation Algorithm

The following is a greedy 2-approximation algorithm that constructs a valid vertex cover by finding a maximal matching.

Algorithm 75 Greedy 2-Approximation for Vertex Cover

Require: An undirected graph $G = (V, E)$

Ensure: A vertex cover $C \subseteq V$

```

1:  $C \leftarrow \emptyset$ 
2:  $E' \leftarrow E$ 
3: while  $E' \neq \emptyset$  do
4:   Let  $(u, v)$  be an arbitrary edge in  $E'$ 
5:    $C \leftarrow C \cup \{u, v\}$  ▷ Add both endpoints to the cover
6:   Remove from  $E'$  every edge incident on  $u$  or  $v$ 
7: end while
8: return  $C$ 

```

3. Time Complexity Proof

Theorem: The time complexity of the algorithm is $\mathcal{O}(|V| + |E|)$.

- **Data Structures:** We represent the graph using an adjacency list and maintain an array **visited** of size $|V|$ initialized to **FALSE**.
- **Execution:** We iterate through all edges in E . For each edge (u, v) , we check if either u or v is **visited**. This check takes $\mathcal{O}(1)$ time.
- If neither is visited, we add u and v to C , mark them as **visited**, and implicitly “remove” incident edges by virtue of the vertices being marked.
- Because we process each vertex and each edge at most a constant number of times throughout the entire **while** loop, the overall running time is linear with respect to the graph size: $\mathcal{O}(|V| + |E|)$. ■

4. Approximation Ratio Proof

Theorem: The approximation ratio of the greedy algorithm is exactly 2.

- Let M be the set of edges arbitrarily picked in step 5. Because we remove all incident edges whenever an edge is selected, no two edges in M share an endpoint. Thus, M is a **maximal matching**.
- For every edge in M , the algorithm adds exactly 2 vertices to C . Therefore, the size of the approximate cover is $|C| = 2|M|$.
- Let C^* be the optimal (minimum) vertex cover. To cover the edges in M , C^* must contain at least one endpoint from each edge. Since the edges in M are completely disjoint, C^* must contain at least one distinct vertex for every edge in M . Thus, $|C^*| \geq |M|$.
- Combining these relations, we get:

$$|C| = 2|M| \leq 2|C^*| \quad (1)$$

Therefore, the algorithm produces a vertex cover at most twice the size of the optimal cover. ■

Part B: Inapproximability of General TSP (8 Marks)

Theorem: For any polynomial-time computable function $\rho(n) \geq 1$, there is no $\rho(n)$ -approximation algorithm for the general Traveling Salesperson Problem (TSP) unless $\mathbf{P} = \mathbf{NP}$.

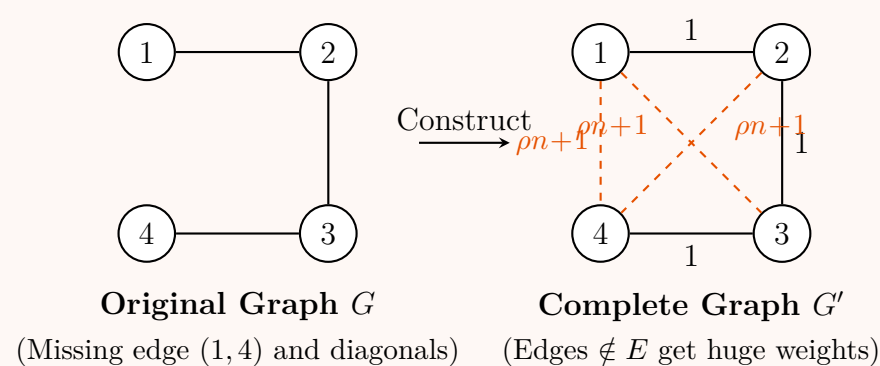
Proof by Reduction from Hamiltonian Cycle (HC):

1. The Setup: Suppose, for the sake of contradiction, that there exists a polynomial-time algorithm $\mathcal{A}$ that can guarantee a $\rho(n)$ -approximation for general TSP. We will show how to use $\mathcal{A}$ to solve the NP-Complete **Hamiltonian Cycle (HC)** problem in polynomial time.

2. The Reduction: Given an unweighted graph $G = (V, E)$ with $|V| = n$ as an instance of the HC problem, we construct a corresponding complete weighted graph $G' = (V, E')$ on the same set of vertices. We define the edge weights $w(u, v)$ in G' as follows:

$$w(u, v) = \begin{cases} 1 & \text{if } (u, v) \in E \\ \rho(n) \cdot n + 1 & \text{if } (u, v) \notin E \end{cases} \quad (2)$$

This construction can clearly be done in polynomial time $\mathcal{O}(n^2)$.



3. Analyzing the Two Cases: Now, we run our hypothetical approximation algorithm $\mathcal{A}$ on G' . Let $C_{\mathcal{A}}$ be the total cost of the tour returned by $\mathcal{A}$. Let C^* be the cost of the optimal TSP tour.

- **Case 1: G has a Hamiltonian Cycle.** If G contains a Hamiltonian cycle, then there is a TSP tour in G' that uses exactly n edges, all of which correspond to the edges in E . Each of these edges has a weight of 1. Thus, $C^* = n$. Because $\mathcal{A}$ is a $\rho(n)$ -approximation algorithm, it guarantees a solution cost bounded by:

$$C_{\mathcal{A}} \leq \rho(n) \cdot C^* = \rho(n) \cdot n \quad (3)$$

- **Case 2: G does not have a Hamiltonian Cycle.** If G does not contain a Hamiltonian cycle, any valid TSP tour in the complete graph G' must traverse at least one edge that does not belong to E . The weight of this “non-edge” is $\rho(n) \cdot n + 1$. The remaining $(n - 1)$ edges in the tour must have a weight of at least 1 each. Thus, the cost of the optimal tour (and therefore any tour, including $C_{\mathcal{A}}$) is strictly bounded from below:

$$C_{\mathcal{A}} \geq C^* \geq (\rho(n) \cdot n + 1) + (n - 1) = \rho(n) \cdot n + n > \rho(n) \cdot n \quad (4)$$

4. Conclusion: By observing the cost of the tour returned by $\mathcal{A}$, we can conclusively determine if G has a Hamiltonian Cycle:

- If $C_{\mathcal{A}} \leq \rho(n) \cdot n$, then G **has** a Hamiltonian Cycle.
- If $C_{\mathcal{A}} > \rho(n) \cdot n$, then G **does not have** a Hamiltonian Cycle.

Since $\mathcal{A}$ runs in polynomial time, we have successfully decided the Hamiltonian Cycle problem in polynomial time. But HC is NP-Complete, meaning we have shown that $\mathbf{P} = \mathbf{NP}$.

Therefore, unless $\mathbf{P} = \mathbf{NP}$, no such polynomial-time $\rho(n)$ -approximation algorithm can exist for the general Traveling Salesperson Problem. ■

Q11. Consider the following instance of the maximum satisfiability (MAX-SAT) problem.

Set of Boolean variables, $V = \{w, x, y, z\}$

Set of clauses, $C = \{C1, C2, C3, C4, C5, C6\}$

where $C1 = (w \vee \bar{x} \vee \bar{y} \vee z)$, $C2 = (w \vee \bar{y})$, $C3 = (x \vee \bar{y})$, $C4 = (y \vee \bar{z})$

$C5 = (z \vee \bar{w})$, $C6 = (\bar{w} \vee \bar{x})$

Now find an assignment of variables such that maximum number of clauses is satisfied. Compute your solution using branch and bound algorithm. Write down a suitable bound function for the problem. Show detailed calculation steps through a branch and bound search tree. **(12 marks)** [CSE 207 | L-2/T-2 | 2014–15]

Answer

1. Problem Formalization and Bound Function

The Maximum Satisfiability (MAX-SAT) problem seeks a truth assignment to a set of Boolean variables that maximizes the number of satisfied clauses. We are given:

- **Variables:** $V = \{w, x, y, z\}$
- **Clauses:**

$$C_1 = w \vee \bar{x} \vee \bar{y} \vee z$$

$$C_2 = w \vee \bar{y}$$

$$C_3 = x \vee \bar{y}$$

$$C_4 = y \vee \bar{z}$$

$$C_5 = z \vee \bar{w}$$

$$C_6 = \bar{w} \vee \bar{x}$$

To solve this problem using a **Branch and Bound** framework, we construct a state-space search tree by instantiating variables one by one. Each node in the tree represents a partial assignment of the variables.

Design of the Upper Bound Function (UB): For any node u with a partial assignment, the clauses can be partitioned into three disjoint sets:

- $C_{\text{sat}}(u)$: The set of clauses already explicitly **satisfied** by the partial assignment.
- $C_{\text{unsat}}(u)$: The set of clauses already explicitly **falsified** (all literals evaluate to **False**).
- $C_{\text{rem}}(u)$: The remaining clauses whose truth values are still undetermined.

An optimistic upper bound on the maximum number of clauses that can be satisfied by extending the partial assignment at node u is given by assuming that *every* undetermined clause can eventually be satisfied:

$$UB(u) = |C_{\text{sat}}(u)| + |C_{\text{rem}}(u)| = |C| - |C_{\text{unsat}}(u)| \quad (1)$$

As we branch deeper, $|C_{\text{unsat}}(u)|$ can only increase or stay the same, which means $UB(u)$ monotonically decreases along any path, satisfying the requirements of a valid bounding function.

2. Detailed Branch and Bound Execution Steps

We explore the state space using a global variable $Best = 0$ tracking the maximum number of satisfied clauses found so far. We branch variables in alphabetical order: $w \rightarrow x \rightarrow y \rightarrow z$.

Root Node (P_0): Empty Assignment

- Partial Assignment: $\emptyset$
- $C_{\text{sat}} = \emptyset$, $C_{\text{unsat}} = \emptyset$, $C_{\text{rem}} = \{C_1, C_2, C_3, C_4, C_5, C_6\}$
- $|C_{\text{sat}}| = 0$, $UB(P_0) = 6 - 0 = 6$.

Level 1: Branching on w

- **Node P_1 ($w = 1$):**
 - Under $w = 1$, clauses C_1 and C_2 contain literal w , so they are satisfied.
 - Sub-clauses for remaining variables become: $C_3 = x \vee \bar{y}$, $C_4 = y \vee \bar{z}$, $C_5 = z \vee 0 \equiv z$, $C_6 = 0 \vee \bar{x} \equiv \bar{x}$.

- No clause is completely falsified yet.
- $|C_{\text{sat}}| = 2$, $|C_{\text{unsat}}| = 0$, $UB(P_1) = 6 - 0 = 6$.
- **Node P_2 ($w = 0$):**
 - Under $w = 0$, clauses C_5 and C_6 contain literal $\bar{w}$, so they are satisfied.
 - Sub-clauses for remaining variables become: $C_1 = 0 \vee \bar{x} \vee \bar{y} \vee z \equiv \bar{x} \vee \bar{y} \vee z$, $C_2 = 0 \vee \bar{y} \equiv \bar{y}$, $C_3 = x \vee \bar{y}$, $C_4 = y \vee \bar{z}$.
 - No clause is completely falsified yet.
 - $|C_{\text{sat}}| = 2$, $|C_{\text{unsat}}| = 0$, $UB(P_2) = 6 - 0 = 6$.

Level 2: Branching from P_1 ($w = 1$) on x

- **Node P_3 ($w = 1, x = 1$):**
 - Already satisfied: $\{C_1, C_2\}$. New satisfied due to $x = 1$: C_3 .
 - Clause $C_6 = \bar{x}$ becomes 0, so C_6 is completely **falsified**.
 - Remaining clauses: $C_4 = y \vee \bar{z}$, $C_5 = z$.
 - $|C_{\text{sat}}| = 3$, $|C_{\text{unsat}}| = 1$, $UB(P_3) = 6 - 1 = 5$.
- **Node P_4 ($w = 1, x = 0$):**
 - Already satisfied: $\{C_1, C_2\}$. New satisfied due to $\bar{x} = 1$: C_6 .
 - Remaining clauses: $C_3 = \bar{y}$, $C_4 = y \vee \bar{z}$, $C_5 = z$.
 - $|C_{\text{sat}}| = 3$, $|C_{\text{unsat}}| = 0$, $UB(P_4) = 6 - 0 = 6$.

Level 3: Branching from P_4 ($w = 1, x = 0$) on y

- **Node P_5 ($w = 1, x = 0, y = 1$):**
 - Already satisfied: $\{C_1, C_2, C_6\}$. New satisfied due to $y = 1$: C_4 .
 - Clause $C_3 = \bar{y}$ becomes 0, so C_3 is completely **falsified**.
 - Remaining clauses: $C_5 = z$.
 - $|C_{\text{sat}}| = 4$, $|C_{\text{unsat}}| = 1$, $UB(P_5) = 6 - 1 = 5$.
- **Node P_6 ($w = 1, x = 0, y = 0$):**
 - Already satisfied: $\{C_1, C_2, C_6\}$. New satisfied due to $\bar{y} = 1$: C_3 .
 - Remaining clauses: $C_4 = \bar{z}$, $C_5 = z$.
 - $|C_{\text{sat}}| = 4$, $|C_{\text{unsat}}| = 0$, $UB(P_6) = 6 - 0 = 6$.

Level 4: Leaf Nodes from P_6 ($w = 1, x = 0, y = 0$) on z

- **Node P_7 ($w = 1, x = 0, y = 0, z = 1$):**
 - Satisfies C_5 (since $z = 1$). Falsifies C_4 (since $\bar{z} = 0$).
 - Fully evaluated state: $C_{\text{sat}} = \{C_1, C_2, C_3, C_5, C_6\}$, total satisfied = 5.
 - Update global maximum: $Best \leftarrow \max(0, 5) = 5$.
- **Node P_8 ($w = 1, x = 0, y = 0, z = 0$):**
 - Satisfies C_4 (since $\bar{z} = 1$). Falsifies C_5 (since $z = 0$).
 - Fully evaluated state: $C_{\text{sat}} = \{C_1, C_2, C_3, C_4, C_6\}$, total satisfied = 5.
 - $Best$ remains 5.

Now we backtrack and check other unexpanded nodes to see if they can beat our current value of $Best = 5$.

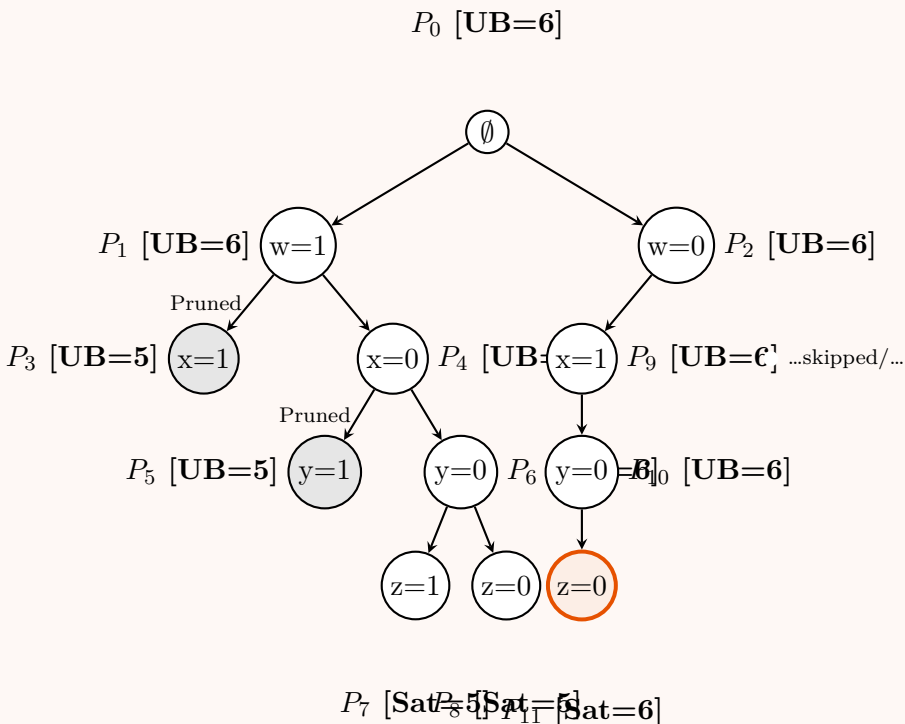
- Node P_5 has $UB(P_5) = 5$. Since $UB(P_5) \leq Best$, we **prune** P_5 .
- Node P_3 has $UB(P_3) = 5$. Since $UB(P_3) \leq Best$, we **prune** P_3 .

Exploring Subtree from P_2 ($w = 0$) We must check if the $w = 0$ branch can yield a solution with 6 satisfied clauses.

- **Branch from P_2 on $x = 1$ (Node P_9):** Partial assignment $\{w = 0, x = 1\}$.
 - Satisfied: $\{C_5, C_6, C_3\}$. Remaining: $C_1 = \bar{y} \vee z$, $C_2 = \bar{y}$, $C_4 = y \vee \bar{z}$. None explicitly falsified. $UB(P_9) = 6$.
- **Branch from P_9 on $y = 0$ (Node P_{10}):** Partial assignment $\{w = 0, x = 1, y = 0\}$.
 - Satisfied: $\{C_5, C_6, C_3\}$ plus C_1, C_2 (due to $\bar{y} = 1$). Thus $\{C_1, C_2, C_3, C_5, C_6\}$ are satisfied.
 - Remaining: $C_4 = \bar{z}$. $UB(P_{10}) = 6$.
- **Leaf from P_{10} on $z = 0$ (Node P_{11}):** Final assignment $\{w = 0, x = 1, y = 0, z = 0\}$.
 - $C_4 = \bar{z} = 1$ is satisfied. This satisfies **all 6 clauses**.
 - Update global maximum: $Best \leftarrow \max(5, 6) = 6$.

Since $Best = 6$, which matches the absolute upper bound of total clauses, all other remaining unvisited or unexpanded nodes in the tree are immediately pruned because their $UB \leq 6$ cannot beat 6.

3. Branch and Bound Search Tree Diagram



4. Optimal Solution Summary

The branch and bound search systematically explored the state space and found an optimal truth assignment that satisfies **all 6 clauses** simultaneously.

Optimal Variable Assignment and Clause Verification					
Variable	Value	Clause	Structure	Evaluation	Status
w	False (0)	C_1	$w \vee \bar{x} \vee \bar{y} \vee z$	$0 \vee 0 \vee 1 \vee 0$	Satisfied
x	True (1)	C_2	$w \vee \bar{y}$	$0 \vee 1$	Satisfied
y	False (0)	C_3	$x \vee \bar{y}$	$1 \vee 1$	Satisfied
z	False (0)	C_4	$y \vee \bar{z}$	$0 \vee 1$	Satisfied
		C_5	$z \vee \bar{w}$	$0 \vee 1$	Satisfied
		C_6	$\bar{w} \vee \bar{x}$	$1 \vee 0$	Satisfied

Conclusion: The maximum number of satisfiable clauses is **6**. The specific assignment tracking down to node P_{11} is:

$w = \text{False}, \quad x = \text{True}, \quad y = \text{False}, \quad z = \text{False}$

(2)

Q12. Write a polynomial-time 2-approximation algorithm for the traveling-salesman problem where the edge weights satisfy the triangle inequality, and prove the approximation ratio. If $P = NP$, prove that for any constant $\rho > 1$, there is no polynomial-time approximation algorithm with ratio ρ for the general TSP. (15 marks)

[CSE 207 | L-2/T-2 | 2013–14]

Answer

Note on Problem Statement: There is a standard typographical error in the prompt. It should read "If $P \neq NP$, prove that...". If $P = NP$, we could find the exact optimal tour in polynomial time, making the approximation bound trivial. We will prove the classical theorem: Unless $P = NP$, no such approximation exists.

Part 1: 2-Approximation Algorithm for Metric TSP

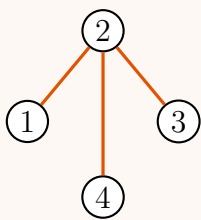
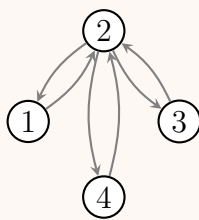
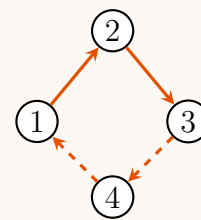
When the edge weights of the Traveling Salesperson Problem (TSP) satisfy the **triangle inequality** (i.e., $w(u, v) \leq w(u, k) + w(k, v)$ for all $u, v, k \in V$), we can exploit the properties of Minimum Spanning Trees (MST) and Eulerian graphs to obtain a 2-approximation.

Algorithm 76 MST-Based 2-Approximation for Metric TSP

Require: A complete undirected graph $G = (V, E)$ with a metric cost function w .

Ensure: A Hamiltonian cycle (TSP tour) H .

- 1: Compute a Minimum Spanning Tree (MST) T of G using Prim's or Kruskal's algorithm.
- 2: Construct a multigraph M by doubling every edge in T . ▷ Every vertex now has even degree.
- 3: Find an Eulerian tour $\mathcal{E}$ in the multigraph M .
- 4: Form a sequence of vertices H by traversing $\mathcal{E}$ and skipping (shortcutting) any previously visited vertices.
- 5: Append the starting vertex to the end of H to close the cycle.
- 6: **return** H .

Visualization of the Algorithm Steps:**1. MST (T)****2. Eulerian Tour ($\mathcal{E}$)****3. Shortcut Tour (H)****Part 2: Proof of the Approximation Ratio**

Theorem: The algorithm returns a TSP tour with a cost at most twice the optimal cost ($2 \cdot OPT$).

Proof: Let OPT denote the cost of the optimal Hamiltonian cycle. Let $c(T)$ be the cost of the MST, $c(\mathcal{E})$ be the cost of the Eulerian tour, and $c(H)$ be the cost of the final algorithm output.

- (a) **Bounding the MST ($c(T) \leq OPT$):** If we take the optimal TSP tour and remove any single edge, we obtain a spanning path. A spanning path is a specific type of spanning tree. Since T is the *minimum* spanning tree, its cost must be less than or equal to the cost of any spanning path. Therefore, $c(T) \leq OPT$.
- (b) **Bounding the Eulerian Tour ($c(\mathcal{E}) \leq 2 \cdot OPT$):** The multigraph is constructed by doubling every edge in the MST T . The Eulerian tour traverses every edge in the multigraph exactly once. Hence, $c(\mathcal{E}) = 2 \cdot c(T)$. Substituting the inequality from step 1, we get $c(\mathcal{E}) \leq 2 \cdot OPT$.
- (c) **Bounding the Shortcuts ($c(H) \leq c(\mathcal{E})$):** The final sequence H is formed by skipping already-visited vertices in $\mathcal{E}$. Because the graph satisfies the triangle inequality, bypassing an intermediate node k directly from u to v can never cost more than going through k (i.e., $w(u, v) \leq w(u, k) + w(k, v)$). Thus, shortcutting can only decrease or maintain the tour's total cost: $c(H) \leq c(\mathcal{E})$.

Combining these three bounds yields:

$$c(H) \leq c(\mathcal{E}) = 2 \cdot c(T) \leq 2 \cdot OPT \quad (1)$$

Thus, the approximation ratio is exactly 2. ■

Part 3: Inapproximability of the General TSP

Theorem: If $P \neq NP$, then for any constant $\rho > 1$, there is no polynomial-time ρ -approximation algorithm for the general TSP problem.

Proof (Reduction from Hamiltonian Cycle): Assume, for the sake of contradiction, that a polynomial-time ρ -approximation algorithm $\mathcal{A}$ exists for general TSP. We will construct a polynomial-time reduction to use $\mathcal{A}$ to solve the NP-Complete **Hamiltonian Cycle (HC)** problem.

Let $G = (V, E)$ be an arbitrary unweighted graph with $|V| = n$ vertices for which we want to determine the existence of a Hamiltonian Cycle. We construct a complete weighted graph $G' = (V, E')$ on the same set of vertices, defining the weight function w as follows:

$$w(u, v) = \begin{cases} 1 & \text{if } (u, v) \in E \\ \rho \cdot n + 1 & \text{if } (u, v) \notin E \end{cases} \quad (2)$$

This construction can be done in polynomial time $\mathcal{O}(n^2)$. Now, consider the TSP solutions in G' :

- **Case 1: G has a Hamiltonian Cycle.** The cycle translates to a valid TSP tour in G' that uses only edges from E . Each of these n edges has a weight of 1, so the optimal TSP cost is $OPT = n$. Because $\mathcal{A}$ is a ρ -approximation algorithm, it is guaranteed to return a tour of cost $C_{\mathcal{A}}$ such that:

$$C_{\mathcal{A}} \leq \rho \cdot OPT = \rho \cdot n \quad (3)$$

- **Case 2: G does not have a Hamiltonian Cycle.** Any TSP tour in G' must traverse at least one edge not present in E . The cost of this single "non-edge" is $\rho \cdot n + 1$. The remaining $n - 1$ edges have a minimum weight of 1. Therefore, the cost of *any* tour (including the one found by $\mathcal{A}$) must be:

$$C_{\mathcal{A}} \geq OPT \geq (\rho \cdot n + 1) + (n - 1) = \rho \cdot n + n > \rho \cdot n \quad (4)$$

Since we forced a massive cost gap between the two cases, we can simply run $\mathcal{A}$ on G' and check the cost of the returned tour:

- If $C_{\mathcal{A}} \leq \rho \cdot n$, we conclude G has a Hamiltonian Cycle.
- If $C_{\mathcal{A}} > \rho \cdot n$, we conclude G does not have a Hamiltonian Cycle.

This provides a polynomial-time decision algorithm for the NP-Complete Hamiltonian Cycle problem, which implies $P = NP$. This contradicts the premise that $P \neq NP$. Therefore, no such approximation algorithm $\mathcal{A}$ can exist. ■

Q13. Consider the following greedy approximation algorithm for the vertex cover optimization problem.

```

1 GreedyVertexCover(G):
2   C = {}
3   while E is not empty:           // E is the set of edges
4     u = vertex in V with maximum degree // V is the set of vertices
5     V = V - {u}
6     C = C union {u}
7     for each edge (u, v) adjacent to u:
8       E = E - {(u, v)}           // remove all edges covered by vertex u
9   return C                       // C is the vertex cover

```

- Give an example graph where the above algorithm may not produce the optimal vertex cover.
- Show that the approximation ratio of the above greedy algorithm is $O(\lceil \lg |E| \rceil)$.

(15 marks)

[CSE 207 | L-2/T-2 | 2012–13]

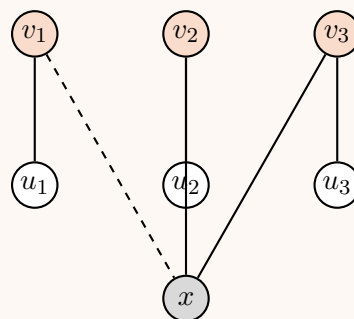
Answer

1. Counterexample for the Greedy Algorithm

The Greedy algorithm operates by locally selecting the vertex that covers the maximum number of currently uncovered edges. However, it can be "tricked" into picking a sub-optimal high-degree vertex (a "greedy trap") that does not belong to the true minimum vertex cover.

Consider the following bipartite graph $G = (V, E)$:

Optimal Cover C^* (Size = 3)



Greedy Trap (Max Degree)

Execution Trace:

- **Optimal Vertex Cover:** The shaded vertices $C^* = \{v_1, v_2, v_3\}$ cover every edge in the graph. The size of the optimal cover is 3.
- **Greedy Execution Step 1:** Initially, the degrees are $d(x) = 3$, $d(v_i) = 2$, and $d(u_i) = 1$. The greedy algorithm picks vertex x because it has the strictly maximum degree. The edges (v_1, x) , (v_2, x) , and (v_3, x) are covered and removed.
- **Greedy Execution Step 2:** The remaining graph consists of 3 completely disjoint edges: (v_1, u_1) , (v_2, u_2) , and (v_3, u_3) . All remaining vertices now have a degree of 1 (or 0).
- **Greedy Execution Step 3:** To cover these 3 disjoint edges, the algorithm must select exactly one vertex for each edge (e.g., u_1, u_2, u_3).

- **Result:** The greedy algorithm outputs a cover of size $1 + 3 = 4$ (e.g., $\{x, u_1, u_2, u_3\}$). Since $4 > 3$, the algorithm fails to produce the optimal vertex cover.

2. Approximation Ratio Proof: $\mathcal{O}(\lceil \lg |E| \rceil)$

We will prove the approximation ratio using amortized analysis via the Pigeonhole Principle.

Definitions:

- Let C^* be the optimal vertex cover, and let $OPT = |C^*|$ be its size.
- Let E_i be the set of remaining uncovered edges exactly before the $(i + 1)$ -th greedy choice. Initially, $E_0 = E$.

Step 1: Bounding the edges covered per iteration

Because C^* is a valid vertex cover for the entire graph, it must also cover all the remaining edges E_i at any step i . Since there are OPT vertices in the optimal cover, by the Pigeonhole Principle, there must exist at least one vertex in C^* that covers at least $\frac{|E_i|}{OPT}$ edges from E_i .

The greedy algorithm intentionally picks the vertex v that covers the *maximum* number of remaining edges. Therefore, the greedy choice covers at least as many edges as the best available vertex in C^* :

$$\text{Edges covered by greedy choice} \geq \frac{|E_i|}{OPT} \quad (1)$$

Step 2: Tracking the number of remaining edges

After removing the edges covered by the selected vertex, the number of remaining edges shrinks:

$$\begin{aligned} |E_{i+1}| &\leq |E_i| - \frac{|E_i|}{OPT} \\ |E_{i+1}| &\leq |E_i| \left(1 - \frac{1}{OPT}\right) \end{aligned} \quad (2)$$

Unrolling this recurrence from $E_0 = E$ up to step k , we obtain:

$$|E_k| \leq |E| \left(1 - \frac{1}{OPT}\right)^k \quad (3)$$

Step 3: Calculating the total iterations

The algorithm terminates when all edges are covered, which means the number of remaining edges must be strictly less than 1 (i.e., $|E_k| < 1$).

$$|E| \left(1 - \frac{1}{OPT}\right)^k < 1 \implies \left(1 - \frac{1}{OPT}\right)^k < \frac{1}{|E|} \quad (4)$$

Taking the natural logarithm on both sides:

$$k \ln \left(1 - \frac{1}{OPT}\right) < -\ln |E| \quad (5)$$

Using the standard logarithmic inequality $\ln(1 - x) \leq -x$ for $x \in (0, 1)$, we can bound the left side:

$$\begin{aligned} k \left(-\frac{1}{OPT}\right) &\leq k \ln \left(1 - \frac{1}{OPT}\right) < -\ln |E| \\ -\frac{k}{OPT} &< -\ln |E| \\ k &> OPT \ln |E| \end{aligned} \quad (6)$$

Conclusion

The number of greedy steps (and thus the number of vertices chosen by the greedy algorithm) is $k = \lceil OPT \ln |E| \rceil$. The approximation ratio ρ is the size of the greedy cover divided by OPT :

$$\rho = \frac{k}{OPT} \leq \frac{OPT \ln |E| + 1}{OPT} = \ln |E| + \frac{1}{OPT} \quad (7)$$

Because the base of the logarithm merely changes the bound by a constant factor (i.e., $\ln |E| = \frac{\lg |E|}{\lg e}$), the asymptotic approximation ratio is strictly bounded by:

$$\rho = \mathcal{O}(\lceil \lg |E| \rceil)$$

■

Q14. Give an efficient 2-approximation algorithm for vertex cover: optimization problem. (8 marks) [CSE 207 | L-2/T-2 | 2012–13]

Answer**1. Algorithm Design**

The Vertex Cover optimization problem seeks the minimum number of vertices such that every edge in the graph is incident to at least one selected vertex. Finding the exact minimum vertex cover is NP-Hard. However, a simple and elegant greedy approach that selects edges and adds *both* of their endpoints to the cover yields a strict 2-approximation.

Algorithm 77 2-Approximation for Vertex Cover (Maximal Matching)

Require: An undirected graph $G = (V, E)$

Ensure: A vertex cover $C \subseteq V$

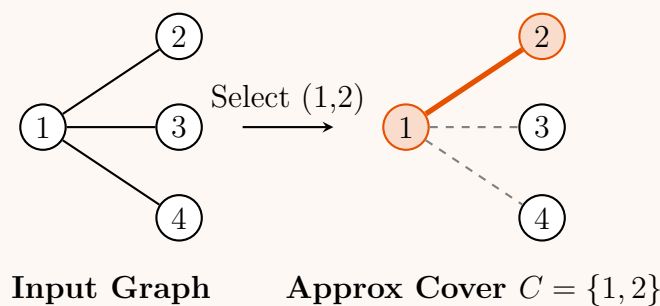
```

1:  $C \leftarrow \emptyset$ 
2:  $E' \leftarrow E$                                 ▷ Working copy of the edge set
3: while  $E' \neq \emptyset$  do
4:   Select an arbitrary edge  $(u, v) \in E'$ 
5:    $C \leftarrow C \cup \{u, v\}$                     ▷ Add both endpoints to the cover
6:   Remove from  $E'$  all edges incident to either  $u$  or  $v$ 
7: end while
8: return  $C$ 

```

2. Execution Example & Visualization

Consider a star graph where the optimal vertex cover clearly consists of only the center node. We demonstrate how the algorithm behaves on this worst-case structure.



Note: The optimal cover is $C^* = \{1\}$ with size 1. The approximate algorithm selects the edge $(1, 2)$, adds both 1 and 2 to the cover, and deletes all other edges since they are connected to 1. The output size is 2, demonstrating the ratio of exactly 2.

3. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|V| + |E|)$

By representing the graph with an adjacency list and maintaining a boolean **visited** array of size $|V|$, we can achieve linear time. We iterate through the edges; if an edge (u, v) has neither u nor v marked as **visited**, we mark both as **visited** (adding them to C). Marking vertices automatically "removes" their incident edges from future consideration. Every vertex and edge is processed a constant number of times.

- **Space Complexity:** $\mathcal{O}(|V|)$

The algorithm requires $\mathcal{O}(|V|)$ auxiliary space to store the selected vertex cover C and the **visited** array.

4. Proof of Approximation Ratio

Theorem: Let C be the vertex cover returned by the algorithm, and let C^* be the optimal (minimum) vertex cover. The approximation ratio is 2, i.e., $|C| \leq 2 \cdot |C^*|$.

Proof:

- Validity of Cover:** The **while** loop only terminates when $E' = \emptyset$. Since every edge in E is either explicitly selected or removed because one of its endpoints was added to C , every edge is covered. Thus, C is a valid vertex cover.
- Maximal Matching Property:** Let $M \subseteq E$ be the set of edges selected in step 5 of the algorithm. By the algorithm's design, whenever an edge (u, v) is selected, all other incident edges are removed. Therefore, no two edges in M share an endpoint. This means M forms a **matching**.
- Size of the Approximate Cover:** For every edge in M , the algorithm adds exactly 2 distinct vertices to the cover C . Thus, the total number of vertices in our output is:

$$|C| = 2|M| \tag{1}$$

- Lower Bound on the Optimal Cover:** Since M is a matching, it consists of mutually disjoint edges. To cover just the edges in M , *any* valid vertex cover must select at least one unique vertex for each edge in M . Therefore, the size of the optimal vertex cover C^* is bounded below by the size of the matching:

$$|C^*| \geq |M| \tag{2}$$

(e) **Establishing the Ratio:** Combining the equations from steps 3 and 4, we multiply the optimal bound by 2:

$$2|C^*| \geq 2|M| = |C| \quad (3)$$

Yielding:

$$|C| \leq 2|C^*| \quad (4)$$

Because the algorithm runs in polynomial time and guarantees a solution size no greater than twice the optimal, it is a strictly **2-approximation algorithm**. ■

Q15. Write a polynomial-time 2-approximation algorithm for the vertex-cover problem, and prove the approximation ratio of the algorithm. If $P = NP$, then prove that for any constant $\rho \geq 1$, there is no polynomial-time approximation algorithm with ratio ρ for the general traveling-salesman problem. **(8+7=15 marks)** [CSE 207 | L-2/T-2 | 2011–12]

Answer

Note: There is a common typographical error in the second part of the prompt. If $P = NP$, we could compute the exact optimal TSP tour in polynomial time, making the statement trivially false. The classical theorem relies on the assumption that $P \neq NP$. We will provide the standard proof for the statement: "If $P \neq NP$, then for any constant $\rho \geq 1$, there is no polynomial-time ρ -approximation algorithm for general TSP."

Part 1: 2-Approximation Algorithm for Vertex Cover

The Vertex Cover problem is NP-Complete, but a strict 2-approximation can be achieved in polynomial time by finding a **maximal matching** and adding all matched vertices to the cover.

Algorithm 78 2-Approximation for Vertex Cover

Require: An undirected graph $G = (V, E)$

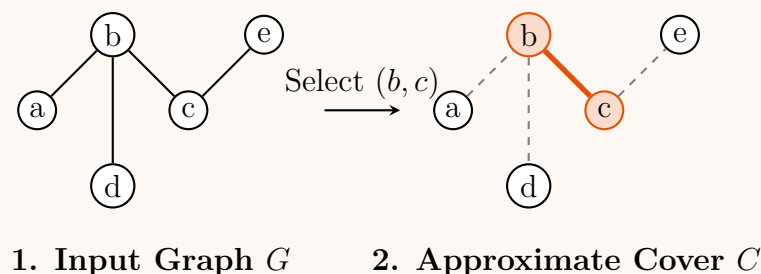
Ensure: A vertex cover $C \subseteq V$

```

1:  $C \leftarrow \emptyset$ 
2:  $E' \leftarrow E$  ▷ Working set of edges
3: while  $E' \neq \emptyset$  do
4:   Let  $(u, v)$  be an arbitrary edge in  $E'$ 
5:    $C \leftarrow C \cup \{u, v\}$  ▷ Add both endpoints to the cover
6:   Remove from  $E'$  all edges incident to  $u$  or  $v$ 
7: end while
8: return  $C$ 

```

Visualization of the Algorithm:



Complexity Analysis:

- **Time Complexity:** $\mathcal{O}(|V| + |E|)$. Using an adjacency list, we can iterate through the edges. If an edge connects two unvisited vertices, we mark both as visited (added to C). Marking vertices implicitly removes their incident edges from consideration.
- **Space Complexity:** $\mathcal{O}(|V|)$ to maintain the set of vertices in the cover C .

Proof of Approximation Ratio: Let C be the vertex cover returned by the algorithm, and let C^* be the optimal (minimum) vertex cover. Let M be the set of edges selected in step 4 of the algorithm.

- Validity:** The algorithm only terminates when all edges are removed from E' . Since an edge is only removed if at least one of its endpoints is in C , every edge in E is covered. Thus, C is a valid vertex cover.
- Matching Property:** Every time an edge is added to M , all incident edges are removed. Therefore, no two edges in M share an endpoint. M is a maximal matching.
- Lower Bound on OPT :** Because M consists of $|M|$ disjoint edges, any valid vertex cover (including the optimal cover C^*) must pick at least one unique vertex for each edge in M . Therefore, $|C^*| \geq |M|$.
- Upper Bound on C :** For each edge in M , the algorithm adds exactly 2 vertices to C . Therefore, $|C| = 2|M|$.
- Conclusion:** Substituting the lower bound into the upper bound gives $|C| = 2|M| \leq 2|C^*|$. Hence, the approximation ratio is exactly 2. ■

Part 2: Inapproximability of the General TSP

Theorem: If $P \neq NP$, then for any constant $\rho \geq 1$, there is no polynomial-time ρ -approximation algorithm for the general Traveling-Salesman Problem (TSP).

Proof (by contradiction via reduction from Hamiltonian Cycle): Assume, for the sake of contradiction, that there exists a polynomial-time algorithm $\mathcal{A}$ that can guarantee a ρ -approximation for the general TSP. We will show that $\mathcal{A}$ can be used to solve the NP-Complete **Hamiltonian Cycle (HC)** problem in polynomial time.

Let $G = (V, E)$ be an arbitrary unweighted graph with $|V| = n$ vertices. We construct a complete weighted graph $G' = (V, E')$ on the same set of vertices. We define the edge weight function $w(u, v)$ for all pairs $u, v \in V$ as follows:

$$w(u, v) = \begin{cases} 1 & \text{if } (u, v) \in E \\ \rho \cdot n + 1 & \text{if } (u, v) \notin E \end{cases} \quad (1)$$

This construction can be completed in polynomial time $\mathcal{O}(n^2)$. Now we evaluate the possible TSP solutions in G' based on whether G contains a Hamiltonian Cycle:

- **Case 1: G has a Hamiltonian Cycle.**

The sequence of vertices in the Hamiltonian Cycle forms a valid TSP tour in G' using only edges from E . Each of these n edges has a weight of 1. The optimal TSP cost is thus $OPT = n$. Because $\mathcal{A}$ is a ρ -approximation algorithm, running $\mathcal{A}$ on G' is guaranteed to return a tour of cost $C_{\mathcal{A}}$ such that:

$$C_{\mathcal{A}} \leq \rho \cdot OPT = \rho \cdot n \quad (2)$$

- **Case 2: G does not have a Hamiltonian Cycle.**

Any TSP tour in G' must traverse at least one "non-edge" (an edge not present in the original graph G). The weight of this non-edge is $\rho \cdot n + 1$. The remaining $n - 1$ edges in the tour have a minimum weight of 1. Therefore, the total cost of *any* valid tour (including the one found by $\mathcal{A}$) must be strictly lower-bounded by:

$$C_{\mathcal{A}} \geq OPT \geq (\rho \cdot n + 1) + (n - 1) = \rho \cdot n + n > \rho \cdot n \quad (3)$$

The Contradiction: Our chosen edge weights create a massive, strict gap in the resulting costs. If we run our hypothetical algorithm $\mathcal{A}$ on G' , we can simply inspect the total cost $C_{\mathcal{A}}$ of the returned tour:

- If $C_{\mathcal{A}} \leq \rho \cdot n$, then G **must** contain a Hamiltonian Cycle.
- If $C_{\mathcal{A}} > \rho \cdot n$, then G **cannot** contain a Hamiltonian Cycle.

Because $\mathcal{A}$ runs in polynomial time, this procedure would allow us to decide the Hamiltonian Cycle problem in polynomial time. Since Hamiltonian Cycle is NP-Complete, this would imply that $P = NP$. This contradicts our assumption that $P \neq NP$.

Therefore, no such polynomial-time ρ -approximation algorithm can exist for the general TSP. ■

Q16. Give a 2-approximation algorithm for the Traveling Salesperson Problem (TSP) for graphs obeying the triangle inequality. Prove the approximation ratio. (17 marks) [CSE 207 | L-2/T-2 | 2009–10]

Answer

1. Algorithm Design

The Traveling Salesperson Problem (TSP) is NP-hard. However, when the edge weights satisfy the **triangle inequality** (i.e., for any three vertices u, v, w , the cost satisfies $c(u, w) \leq c(u, v) + c(v, w)$), we can use a Minimum Spanning Tree (MST) approach to achieve a 2-approximation.

Algorithm 79 2-Approximation for Metric TSP

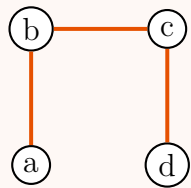
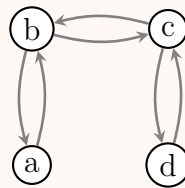
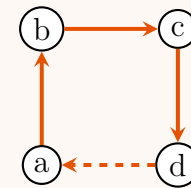
Require: A complete undirected graph $G = (V, E)$ with a metric cost function c .

Ensure: A Hamiltonian cycle (TSP tour) H .

- 1: Compute a Minimum Spanning Tree (MST) T of G .
 - 2: Construct a multigraph M by doubling every edge in T . ▷ Every vertex now has even degree.
 - 3: Find an Eulerian tour $\mathcal{E}$ in the multigraph M .
 - 4: Form a Hamiltonian cycle H by traversing $\mathcal{E}$ and skipping (shortcutting) any previously visited vertices.
 - 5: Append the starting vertex to the end of H to close the cycle.
 - 6: **return** H .
-

2. Visualization of the Algorithm Steps

The following sequence visualizes the algorithm on a small metric graph instance.

1. MST (T)2. Eulerian Tour ($\mathcal{E}$)3. Shortcut Tour (H)

In Step 3, the Eulerian tour traverses $a \rightarrow b \rightarrow c \rightarrow d \rightarrow c \rightarrow b \rightarrow a$. The shortcutting process directly connects $d \rightarrow a$ to avoid revisiting c and b .

3. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|V|^2)$

Since the input graph for TSP is typically a complete graph ($|E| = \mathcal{O}(|V|^2)$), finding the MST using Prim's algorithm takes $\mathcal{O}(|V|^2)$ time. Doubling the edges and finding the Eulerian tour takes $\mathcal{O}(|V|)$ time because the MST only has $|V| - 1$ edges. The shortcutting traversal also takes $\mathcal{O}(|V|)$ time. Therefore, the overall time complexity is dominated by the MST construction.

- **Space Complexity:** $\mathcal{O}(|V|^2)$

Storing the complete graph requires $\mathcal{O}(|V|^2)$ space. The structures for the MST, multigraph, and Eulerian tour require only $\mathcal{O}(|V|)$ auxiliary space.

4. Proof of the Approximation Ratio

Theorem: Let H be the Hamiltonian cycle returned by the algorithm, and let H^* be the optimal (minimum cost) TSP tour. The cost of H satisfies $c(H) \leq 2 \cdot c(H^*)$.

Proof: Let $c(T)$ be the total cost of the Minimum Spanning Tree, $c(\mathcal{E})$ be the cost of the Eulerian tour, and $c(H)$ be the cost of the final algorithm output. We establish three bounding relationships:

(a) **Bounding the MST ($c(T) \leq c(H^*)$):**

Consider the optimal TSP tour H^* . If we remove any single edge from this cycle, we obtain a simple path that visits every vertex exactly once (a spanning path). A spanning path is a valid spanning tree. Since T is the *minimum* cost spanning tree out of all possible spanning trees, its cost must be less than or equal to the cost of this spanning path.

$$c(T) \leq c(H^*) \quad (1)$$

(b) **Bounding the Eulerian Tour ($c(\mathcal{E}) \leq 2 \cdot c(H^*)$):**

The multigraph is constructed by making two copies of every edge in the MST T . An Eulerian tour traverses every edge in the multigraph exactly once. Therefore, the total cost of the Eulerian tour is exactly twice the cost of the MST.

$$c(\mathcal{E}) = 2 \cdot c(T) \quad (2)$$

Substituting the inequality from step 1, we obtain:

$$c(\mathcal{E}) \leq 2 \cdot c(H^*) \quad (3)$$

(c) **Bounding the Shortcuts ($c(H) \leq c(\mathcal{E})$):**

The final Hamiltonian cycle H is formed by extracting a subsequence of vertices from the Eulerian tour $\mathcal{E}$, skipping any vertices that have already been visited. Because the graph satisfies the **triangle inequality**, going directly from a vertex u to a vertex w is never more expensive than going from u to w via an intermediate vertex v .

$$c(u, w) \leq c(u, v) + c(v, w) \quad (4)$$

Therefore, every time we "shortcut" a previously visited vertex, the total cost of the tour either decreases or remains the same. Thus, the cost of the shortened tour H is bounded by the Eulerian tour:

$$c(H) \leq c(\mathcal{E}) \quad (5)$$

Combining these three results yields the final approximation bound:

$$c(H) \leq c(\mathcal{E}) = 2 \cdot c(T) \leq 2 \cdot c(H^*) \quad (6)$$

Because the algorithm yields a valid tour with a cost at most twice the optimal cost in polynomial time, it is a 2-approximation algorithm. ■

Q17. Write a 2-approximation algorithm for the "vertex cover" problem. Show that your algorithm has the mentioned approximation ratio. (10 marks) [CSE 207 | 2008–2009]

Answer**1. Algorithm Design**

The Vertex Cover problem is a classical NP-Complete problem. The goal is to find the minimum number of vertices such that every edge in the graph is incident to at least one chosen vertex. While finding the optimal cover is computationally intractable, we can achieve a strict **2-approximation** using a greedy approach based on finding a maximal matching.

Algorithm 80 2-Approximation Algorithm for Vertex Cover

Require: An undirected graph $G = (V, E)$

Ensure: A vertex cover $C \subseteq V$

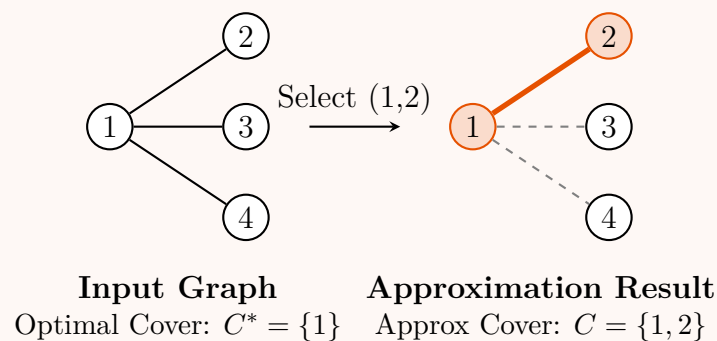
```

1:  $C \leftarrow \emptyset$                                 ▷ Initialize the vertex cover
2:  $E' \leftarrow E$                                 ▷ Working set of uncovered edges
3: while  $E' \neq \emptyset$  do
4:   Let  $(u, v)$  be an arbitrary edge in  $E'$ 
5:    $C \leftarrow C \cup \{u, v\}$                     ▷ Add both endpoints to the cover
6:   Remove from  $E'$  every edge incident on either  $u$  or  $v$ 
7: end while
8: return  $C$ 

```

2. Execution Visualization

To illustrate, consider a star graph where the optimal vertex cover clearly consists of only the central node. The greedy algorithm will randomly select an edge, covering the entire graph but picking an extra (unnecessary) leaf node.

**3. Proof of Approximation Ratio**

Theorem: Let C be the vertex cover returned by the algorithm, and let C^* be the optimal (minimum) vertex cover. The algorithm guarantees that $|C| \leq 2 \cdot |C^*|$.

Proof:

- (a) **Validity of the Cover:** The loop condition ensures the algorithm terminates only when $E' = \emptyset$. Because an edge is removed from E' only when at least one of its endpoints is placed in C , all edges in the original edge set E are covered by C . Thus, C is a valid vertex cover.
- (b) **Maximal Matching Property:** Let $M \subseteq E$ denote the exact set of edges explicitly chosen in step 4 of the algorithm. By the algorithm's design, as soon as an edge (u, v) is selected, all other edges sharing an endpoint with u or v are discarded. Thus, no two edges in M share an endpoint. The set of edges M forms a **matching**.
- (c) **Lower Bound on Optimal Solution (OPT):** The optimal vertex cover C^* must cover every edge in the graph, which includes the edges in our matching M . Since the edges in M are completely disjoint (they share no vertices), the only way to cover them is to select at least one distinct vertex for every single edge in M . Therefore, the size of the optimal vertex cover must be at least the size of the matching:

$$|C^*| \geq |M| \tag{1}$$

- (d) **Size of the Approximate Cover:** For every edge $(u, v) \in M$, our algorithm adds exactly two vertices (u and v) to the set C . Therefore, the total number of vertices chosen by the algorithm is exactly twice the number of edges in the matching:

$$|C| = 2|M| \tag{2}$$

- (e) **Establishing the Ratio:** We substitute the inequality from step 3 into the equation from step 4. Since $|M| \leq |C^*|$, we can multiply both sides by 2 to get $2|M| \leq 2|C^*|$. Replacing $2|M|$ with $|C|$ yields:

$$|C| \leq 2|C^*| \tag{3}$$

This confirms that the size of the cover produced by the greedy maximal matching algorithm is at most twice the optimal size. ■

4. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|V| + |E|)$

By representing the graph with an adjacency list, we can implement this algorithm in linear time. We maintain a boolean array `visited` of size $|V|$. We iterate through the edge list; if we encounter an edge (u, v) where both u and v are unvisited, we mark both as `visited` and add them to C . Marking a node implicitly "removes" all its incident edges since any future edge touching a visited node is simply skipped. Each vertex and edge is processed a constant number of times.

- **Space Complexity:** $\mathcal{O}(|V|)$

The algorithm requires $\mathcal{O}(|V|)$ auxiliary space to store the selected vertex cover C and the `visited` array used to track the endpoints of the matching.

Q18. Write a 2-approximation algorithm for the vertex cover problem. Justify your answer. (10 marks) [CSE 207 | L-2/T-2 | 2006–07]

Answer

1. Algorithm Design

The Vertex Cover optimization problem aims to find the minimum number of vertices such that every edge in the graph is incident to at least one selected vertex. Finding the exact minimum vertex cover is NP-Hard. However, a highly efficient greedy approach that computes a **maximal matching** yields a strict 2-approximation.

The strategy is simple: iteratively select an arbitrary uncovered edge, add *both* of its endpoints to the vertex cover, and remove all edges incident to either endpoint.

Algorithm 81 2-Approximation Algorithm for Vertex Cover

Require: An undirected graph $G = (V, E)$

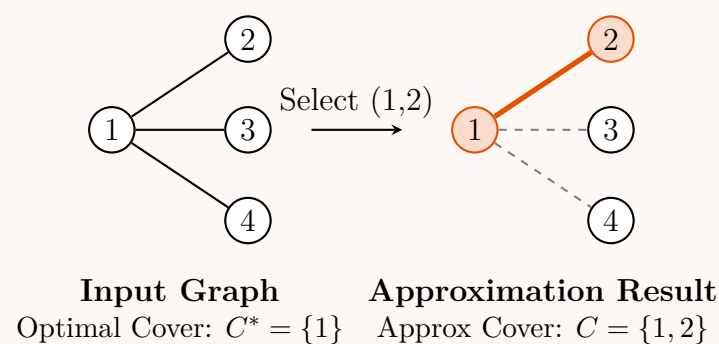
Ensure: A vertex cover $C \subseteq V$

```

1:  $C \leftarrow \emptyset$                                 ▷ Initialize the vertex cover
2:  $E' \leftarrow E$                                 ▷ Working set of uncovered edges
3: while  $E' \neq \emptyset$  do
4:   Let  $(u, v)$  be an arbitrary edge in  $E'$ 
5:    $C \leftarrow C \cup \{u, v\}$                     ▷ Add both endpoints to the cover
6:   Remove from  $E'$  every edge incident on either  $u$  or  $v$ 
7: end while
8: return  $C$ 
    
```

2. Execution Visualization

To understand why this is a 2-approximation, consider a graph where a single vertex covers many edges (a star graph). The greedy algorithm will select one edge, thereby adding an optimal central node, but also adding an unnecessary leaf node.



3. Justification (Proof of Approximation Ratio)

Theorem: Let C be the vertex cover returned by the algorithm, and let C^* be the optimal (minimum) vertex cover. The algorithm guarantees that $|C| \leq 2 \cdot |C^*|$.

Proof:

- Validity of the Cover:** The loop condition ensures the algorithm terminates only when $E' = \emptyset$. Because an edge is removed from E' only when at least one of its endpoints is placed in C , all edges in the original edge set E are covered by C . Thus, C is a valid vertex cover.
- Maximal Matching Property:** Let $M \subseteq E$ denote the exact set of edges explicitly chosen in step 5 of the algorithm. By the algorithm's design, as soon as an edge (u, v) is selected, all other edges sharing an endpoint with u or v are discarded. Thus, no two edges in M share an endpoint. The set of edges M forms a **matching**.
- Lower Bound on Optimal Solution (OPT):** The optimal vertex cover C^* must cover every edge in the graph, which includes the edges in our matching M . Since the edges in M are completely disjoint (they share no vertices), the only way to cover them is to select at least one distinct vertex for every single edge in M . Therefore, the size of the optimal vertex cover must be at least the size of the matching:

$$|C^*| \geq |M| \tag{1}$$

- (d) **Size of the Approximate Cover:** For every edge $(u, v) \in M$, our algorithm adds exactly two vertices (u and v) to the set C . Therefore, the total number of vertices chosen by the algorithm is exactly twice the number of edges in the matching:

$$|C| = 2|M| \quad (2)$$

- (e) **Establishing the Ratio:** We substitute the inequality from step 3 into the equation from step 4. Since $|M| \leq |C^*|$, we can multiply both sides by 2 to get $2|M| \leq 2|C^*|$. Replacing $2|M|$ with $|C|$ yields:

$$|C| \leq 2|C^*| \quad (3)$$

This confirms that the size of the cover produced by the greedy maximal matching algorithm is at most twice the optimal size, proving it is a 2-approximation. ■

4. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(|V| + |E|)$

By representing the graph using an adjacency list, we can implement this algorithm in linear time. We maintain a boolean array `visited` of size $|V|$ initialized to false. We iterate through the list of edges E . If we encounter an edge (u, v) where both u and v are `false` (unvisited), we set both to `true` and add them to C . Marking a node implicitly removes all its incident edges because any subsequent edge touching a visited node is skipped in $\mathcal{O}(1)$ time. Thus, each vertex and edge is processed a constant number of times.

- **Space Complexity:** $\mathcal{O}(|V|)$

The algorithm requires $\mathcal{O}(|V|)$ auxiliary space to store the selected vertex cover C and the `visited` array used to track which vertices are already covered.

Backtracking

Q1. Consider the following Boolean formula:

$$(x' \vee y') \wedge (x' \vee y) \wedge (x \vee y' \vee z) \wedge (x \vee y \vee z)$$

Use a backtracking algorithm to decide whether this is satisfiable or not. Show the backtracking tree. **(10 marks)** [CSE 207 | L-2/T-1 | 2024–25]

Answer

1. Problem Setup and Algorithm

We are given the Boolean formula F in Conjunctive Normal Form (CNF):

$$F = C_1 \wedge C_2 \wedge C_3 \wedge C_4$$

where the clauses are:

- $C_1 = (x' \vee y')$
- $C_2 = (x' \vee y)$
- $C_3 = (x \vee y' \vee z)$
- $C_4 = (x \vee y \vee z)$

To determine if F is satisfiable, we use a **Backtracking Algorithm**. The algorithm incrementally assigns truth values to variables (in the order x, y, z). After each assignment, it evaluates the formula. If any clause becomes 0 (False), the current partial assignment is a dead end (conflict), and the algorithm *prunes* the branch and backtracks. If all clauses evaluate to 1 (True), a satisfying assignment is found.

Algorithm 82 Backtracking SAT Solver

Require: A CNF formula F over variables V , Current Assignment A
Ensure: True if satisfiable, False otherwise

```

1: function SOLVESAT( $F, A$ )
2:   Evaluate  $F$  under the partial assignment  $A$ 
3:   if any clause in  $F$  evaluates to 0 then
4:     return False
5:   end if
6:   if all clauses in  $F$  evaluate to 1 then
7:     return True
8:   end if
9:   Let  $v$  be the next unassigned variable in  $V$ 
10:  if SOLVESAT( $F, A \cup \{v = 0\}$ ) then
11:    return True
12:  end if
13:  return SOLVESAT( $F, A \cup \{v = 1\}$ )
14: end function
    
```

▷ Conflict detected, backtrack
 ▷ Satisfying assignment found
 ▷ Try $v = 0$ (False)
 ▷ Try $v = 1$ (True)

2. Step-by-Step Execution

We systematically construct the search tree:

Branch $x = 0$ (False):

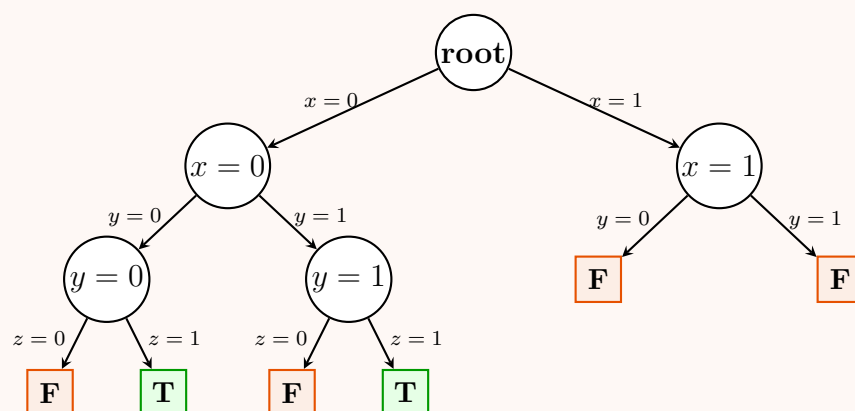
- $C_1 = (1 \vee y') = 1$, $C_2 = (1 \vee y) = 1$
- $C_3 = (0 \vee y' \vee z) = y' \vee z$, $C_4 = (0 \vee y \vee z) = y \vee z$
- The formula simplifies to $F = (y' \vee z) \wedge (y \vee z)$.
- **Sub-branch $y = 0$:**
 - $C_3 = (1 \vee z) = 1$, $C_4 = (0 \vee z) = z$. F simplifies to z .
 - **Try $z = 0$:** $C_4 = 0$. Conflict! (Return False).
 - **Try $z = 1$:** $C_4 = 1$. All clauses satisfied! (Return True).
- *Note: A standard solver stops here. For completeness, we also show the other branches.*
- **Sub-branch $y = 1$:**
 - $C_3 = (0 \vee z) = z$, $C_4 = (1 \vee z) = 1$. F simplifies to z .
 - **Try $z = 0$:** $C_3 = 0$. Conflict! (Return False).
 - **Try $z = 1$:** $C_3 = 1$. All clauses satisfied! (Return True).

Branch $x = 1$ (True):

- $C_1 = (0 \vee y') = y'$, $C_2 = (0 \vee y) = y$
- $C_3 = (1 \vee y' \vee z) = 1$, $C_4 = (1 \vee y \vee z) = 1$
- The formula simplifies to $F = y' \wedge y$.
- **Sub-branch $y = 0$:** $C_2 = 0$. Conflict! The branch is pruned; z is not evaluated.
- **Sub-branch $y = 1$:** $C_1 = 0$. Conflict! The branch is pruned; z is not evaluated.

3. Backtracking Search Tree

The full backtracking tree is visualized below. Green leaf nodes mark a satisfying assignment, while red leaf nodes indicate conflicts that trigger backtracking.



Conclusion: The formula is **Satisfiable**. The first satisfying assignment found by a standard left-to-right backtracking search is $x = 0, y = 0, z = 1$. A second satisfying assignment is $x = 0, y = 1, z = 1$.

4. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(2^n \cdot m)$

Where n is the number of variables ($n = 3$) and m is the number of clauses ($m = 4$). In the worst-case scenario (an unsatisfiable formula without early pruning), the algorithm explores a complete binary tree of depth n , which has 2^n leaves. At each node, evaluating the formula takes $\mathcal{O}(m)$ time.

- **Space Complexity:** $\mathcal{O}(n)$

The space complexity is determined by the maximum depth of the recursion stack. Since the tree depth is strictly bounded by the number of variables n , the memory footprint is linear with respect to the number of variables.

Q2. Consider the following boolean formula:

$$(x \vee y') \wedge (x' \vee w) \wedge (x \vee y \vee z') \wedge (x' \vee w \vee z)$$

Use the backtracking algorithm to decide whether the boolean formula is satisfiable or not. If satisfiable, find all the assignments of the boolean variables for which the boolean formula is satisfiable. You must show the backtracking tree. **(10 marks)** [CSE 207 | L-2/T-2 | 2020–21]

Answer

1. Problem Setup and Simplification

We are given the Boolean formula F in Conjunctive Normal Form (CNF):

$$F = C_1 \wedge C_2 \wedge C_3 \wedge C_4$$

where the clauses are:

- $C_1 = (x \vee y')$
- $C_2 = (x' \vee w)$
- $C_3 = (x \vee y \vee z')$
- $C_4 = (x' \vee w \vee z)$

To determine satisfiability and find all satisfying assignments, we apply a **backtracking algorithm**. We will sequentially assign truth values to the variables in the order x, y, z, w . At each step, we simplify the formula. If any clause evaluates to 0 (False), a conflict occurs, and the branch is pruned. If all clauses evaluate to 1 (True), a valid assignment is recorded.

2. Step-by-Step Backtracking Execution

Branch 1: $x = 0$ (False)

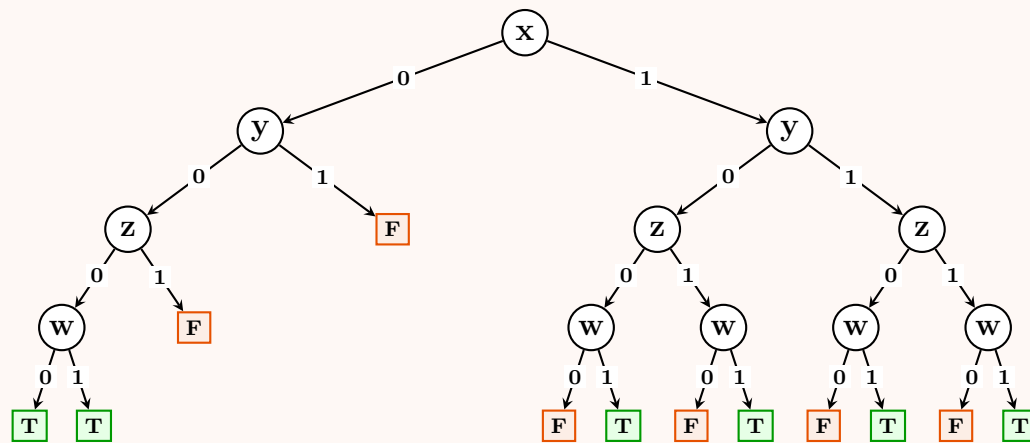
- $C_1 = (0 \vee y') = y'$, $C_2 = (1 \vee w) = 1$
- $C_3 = (0 \vee y \vee z') = y \vee z'$, $C_4 = (1 \vee w \vee z) = 1$
- **Sub-branch $y = 1$:** $C_1 = 0$. **Conflict!** (Prune).
- **Sub-branch $y = 0$:** $C_1 = 1$, $C_3 = (0 \vee z') = z'$.
 - **Try $z = 1$:** $C_3 = 0$. **Conflict!** (Prune).
 - **Try $z = 0$:** $C_3 = 1$. All clauses are satisfied. The remaining variable w can be freely assigned either 0 or 1, producing two valid assignments.

Branch 2: $x = 1$ (True)

- $C_1 = (1 \vee y') = 1$, $C_3 = (1 \vee y \vee z') = 1$
- $C_2 = (0 \vee w) = w$, $C_4 = (0 \vee w \vee z) = w \vee z$
- Because C_1 and C_3 are trivially satisfied, y and z can take any values, but we still evaluate them sequentially to build the complete tree.
- No matter the choices for y and z , we are left evaluating w against C_2 and C_4 .
- **Try $w = 0$:** $C_2 = 0$. **Conflict!** (Prune in all sub-branches).
- **Try $w = 1$:** $C_2 = 1$, $C_4 = (1 \vee z) = 1$. All clauses are satisfied.

3. Backtracking Search Tree

The decision tree visualizes the variable assignments. Edges are labeled with the assigned boolean value (0 or 1). Green **T** leaves represent satisfying assignments, while red **F** leaves represent pruned conflicts.



4. Satisfiable Assignments

The boolean formula is **Satisfiable**. Based on the backtracking tree, there are exactly **6 valid assignments** for the variables (x, y, z, w) that make the formula evaluate to True:

- (a) $x = 0, y = 0, z = 0, w = 0$
- (b) $x = 0, y = 0, z = 0, w = 1$
- (c) $x = 1, y = 0, z = 0, w = 1$
- (d) $x = 1, y = 0, z = 1, w = 1$
- (e) $x = 1, y = 1, z = 0, w = 1$
- (f) $x = 1, y = 1, z = 1, w = 1$

5. Algorithm & Complexity

Algorithm 83 Backtracking All-SAT Solver

Require: A CNF formula F , Variables V , Current Assignment A

Ensure: Prints all satisfying assignments

```

1: function SOLVEALLSAT( $F, V, A$ )
2:   Evaluate  $F$  under the partial assignment  $A$ 
3:   if any clause in  $F$  evaluates to 0 then
4:     return ▷ Conflict: Prune branch
5:   end if
6:   if all variables in  $V$  are assigned in  $A$  then
7:     Print  $A$  ▷ Valid assignment found
8:     return
9:   end if
10:  Let  $v$  be the next unassigned variable in  $V$ 
11:  SOLVEALLSAT( $F, V, A \cup \{v = 0\}$ )
12:  SOLVEALLSAT( $F, V, A \cup \{v = 1\}$ )
13: end function
    
```

- **Time Complexity:** $\mathcal{O}(2^n \cdot m)$

Where n is the number of variables ($n = 4$) and m is the number of clauses ($m = 4$). In the worst case, the algorithm explores the entire binary tree of depth n , containing 2^n leaves. Evaluating the m clauses at each node takes linear time with respect to the formula size.

- **Space Complexity:** $\mathcal{O}(n)$

The space complexity is dominated by the recursion stack. Since variables are assigned one at a time, the maximum depth of the stack strictly equals the number of variables, n .

Q3. Consider the following Boolean formula:

$$(x' \vee y') \wedge (x \vee y') \wedge (x' \vee y) \wedge (x \vee y \vee z') \wedge (x \vee y \vee z)$$

Use the backtracking algorithm to decide whether this is satisfiable or not. You must show the backtracking tree. **(10 marks)** [CSE 207 | L-2/T-2 | 2018–19]

Answer**1. Problem Setup and Simplification**

We are given the Boolean formula F in Conjunctive Normal Form (CNF):

$$F = C_1 \wedge C_2 \wedge C_3 \wedge C_4 \wedge C_5$$

where the clauses are defined as follows:

- $C_1 = (x' \vee y')$
- $C_2 = (x \vee y')$
- $C_3 = (x' \vee y)$
- $C_4 = (x \vee y \vee z')$
- $C_5 = (x \vee y \vee z)$

To decide the satisfiability of the formula, we use a **backtracking algorithm**. We incrementally assign truth values to the variables in the lexicographical order x, y, z . At each assignment, we evaluate the formula. If any clause evaluates to 0 (False), a conflict is detected and the search branch is immediately pruned. If all clauses evaluate to 1 (True), a satisfying assignment is found.

2. Step-by-Step Execution

We systematically evaluate the assignments:

Branch 1: $x = 0$ (False)

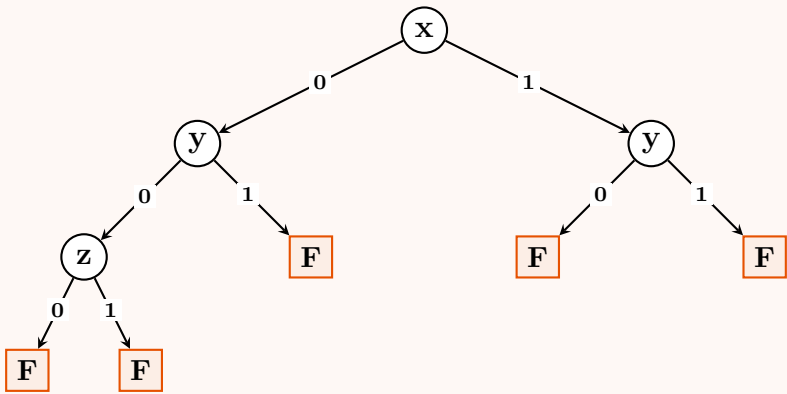
- $C_1 = (1 \vee y') = 1$
- $C_2 = (0 \vee y') = y'$
- $C_3 = (1 \vee y) = 1$
- $C_4 = (0 \vee y \vee z') = y \vee z'$
- $C_5 = (0 \vee y \vee z) = y \vee z$
- The formula simplifies to: $F|_{x=0} = y' \wedge (y \vee z') \wedge (y \vee z)$.
- **Sub-branch $y = 1$:**
 - $C_2 = 0$. **Conflict!** The branch is pruned; z is not evaluated.
- **Sub-branch $y = 0$:**
 - $C_2 = 1$
 - $C_4 = (0 \vee z') = z'$
 - $C_5 = (0 \vee z) = z$
 - The formula simplifies to: $F|_{x=0, y=0} = z' \wedge z$.
 - **Try $z = 0$:** $C_5 = 0$. **Conflict!** (Prune).
 - **Try $z = 1$:** $C_4 = 0$. **Conflict!** (Prune).

Branch 2: $x = 1$ (True)

- $C_1 = (0 \vee y') = y'$
- $C_2 = (1 \vee y') = 1$
- $C_3 = (0 \vee y) = y$
- $C_4 = (1 \vee y \vee z') = 1$
- $C_5 = (1 \vee y \vee z) = 1$
- The formula simplifies to: $F|_{x=1} = y' \wedge y$.
- **Sub-branch $y = 0$:**
 - $C_3 = 0$. **Conflict!** The branch is pruned; z is not evaluated.
- **Sub-branch $y = 1$:**
 - $C_1 = 0$. **Conflict!** The branch is pruned; z is not evaluated.

3. Backtracking Search Tree

The decision tree illustrates the assignments and the ensuing conflicts. Red **F** leaves signify that a clause evaluated to False (a conflict), prompting the algorithm to backtrack.



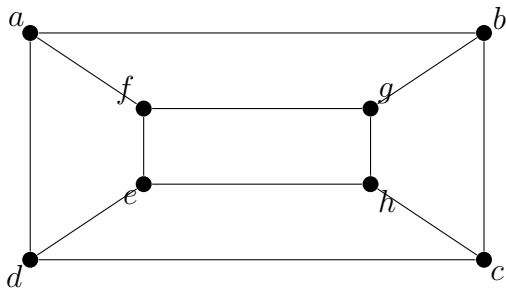
4. Conclusion

Because every single branch in the backtracking search tree results in a conflict (terminates in **F**), there is no combination of boolean assignments that satisfies all clauses simultaneously. Therefore, the given boolean formula is **Unsatisfiable**.

5. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(2^n \cdot m)$
Here, $n = 3$ is the number of variables and $m = 5$ is the number of clauses. In the worst case, the backtracking algorithm explores a full binary tree of height n , producing 2^n paths. At each node, evaluating the expression takes $\mathcal{O}(m)$ operations. Thus, the time complexity is bounded by $\mathcal{O}(2^n \cdot m)$.
- **Space Complexity:** $\mathcal{O}(n)$
The space complexity corresponds to the maximum depth of the recursion stack, which maps directly to the number of variables n .

Q4. Apply backtracking to the problem of finding a Hamiltonian circuit in a given graph. Show the corresponding state-space tree. (10 marks) [CSE 207 | L-2/T-2 | 2017–18]



Answer

1. Problem Setup and Graph Representation

The problem requires us to find a Hamiltonian circuit (a simple cycle that visits every vertex exactly once and returns to the starting vertex) in the given graph using backtracking. Let the total number of vertices be $N = 8$. First, we extract the adjacency list for the graph based on the given visual representation:

- $a: b, d, f$
- $b: a, c, g$
- $c: b, d, h$
- $d: a, c, e$
- $e: d, f, h$
- $f: a, e, g$
- $g: b, f, h$
- $h: c, e, g$

2. Backtracking Algorithm

The backtracking algorithm builds a path vertex by vertex. At each step, it attempts to add an unvisited neighbor of the current vertex to the path.

- **Base Case:** If the path length equals N , we check if there is an edge between the last vertex and the starting vertex. If so, a Hamiltonian circuit is found.
- **Recursive Step:** Iterate through all unvisited neighbors. If adding a neighbor leads to a dead end, we *backtrack* by removing it from the path and trying the next available neighbor.

Algorithm 84 Backtracking for Hamiltonian Circuit

Require: Adjacency list of Graph $G = (V, E)$, starting vertex s

Ensure: A sequence of vertices forming a Hamiltonian Circuit, or failure.

```

1: Initialize:  $path = [s]$ ,  $visited[v] = \text{False}$  for all  $v \in V$ ,  $visited[s] = \text{True}$ 
2: function FINDCIRCUIT( $u, count$ )
3:   if  $count == |V|$  then
4:     if edge  $(u, s)$  exists in  $E$  then
5:       Print  $path \cup \{s\}$ 
6:       return True
7:     else
8:       return False
9:     end if
10:  end if
11:  for each neighbor  $v$  of  $u$  do
12:    if not  $visited[v]$  then
13:       $visited[v] \leftarrow \text{True}$ 
14:       $path \leftarrow path \cup \{v\}$ 
15:      if FINDCIRCUIT( $v, count + 1$ ) then
16:        return True
17:      end if
18:       $visited[v] \leftarrow \text{False}$ 
19:      Remove  $v$  from  $path$ 
20:    end if
21:  end for
22:  return False
23: end function

```

▷ Circuit found

▷ Dead end, backtrack

▷ Backtrack

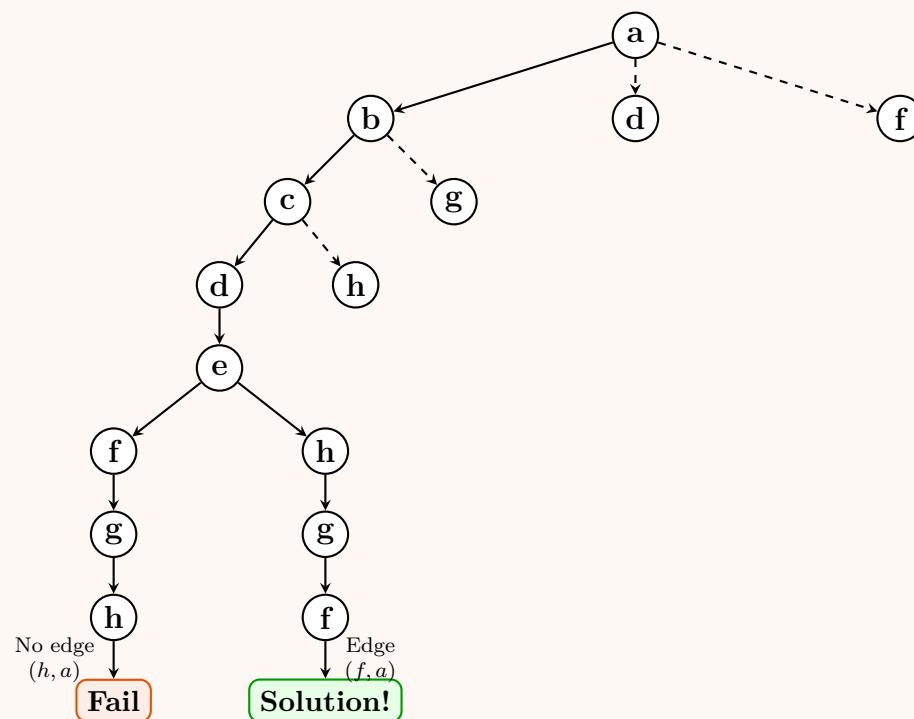
3. Step-by-Step Execution

We systematically trace the algorithm starting at vertex a , exploring neighbors in alphabetical order whenever multiple choices are available.

- Start at a . Neighbors are b, d, f . Try b . Path: $[a, b]$
 - From b , unvisited neighbors are c, g . Try c . Path: $[a, b, c]$
 - From c , unvisited neighbors are d, h . Try d . Path: $[a, b, c, d]$
 - From d , unvisited neighbor is e . Path: $[a, b, c, d, e]$
 - From e , unvisited neighbors are f, h . Try f . Path: $[a, b, c, d, e, f]$
 - From f , unvisited neighbor is g . Path: $[a, b, c, d, e, f, g]$
 - From g , unvisited neighbor is h . Path: $[a, b, c, d, e, f, g, h]$
 - Check Circuit:** Path length is 8. Is there an edge between h and start node a ? No.
 - Backtrack:** Backtrack to $g \rightarrow f \rightarrow e$. At vertex e , the next unvisited neighbor is h .
 - From e , try h . Path: $[a, b, c, d, e, h]$
 - From h , unvisited neighbor is g . Path: $[a, b, c, d, e, h, g]$
 - From g , unvisited neighbor is f . Path: $[a, b, c, d, e, h, g, f]$
 - Check Circuit:** Path length is 8. Is there an edge between f and a ? Yes, edge (f, a) exists!
 - Success!** Valid Hamiltonian Circuit found: $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow h \rightarrow g \rightarrow f \rightarrow a$.
-

4. State-Space Tree

The diagram below represents the partial state-space tree demonstrating the successful search path and the first backtracking instance. Dashed lines indicate branches that are either pruned later or left unexplored once the first solution is found.



5. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(N!)$

In the worst-case scenario (e.g., a highly connected graph with no Hamiltonian circuits), the backtracking algorithm behaves essentially like generating permutations of vertices, exploring all possible paths. At each step, the number of choices decreases by 1, leading to a factorial upper bound $\mathcal{O}(N!)$ where N is the number of vertices.

- **Space Complexity:** $\mathcal{O}(N)$

The space complexity is defined by the maximum depth of the recursion stack and the auxiliary structures (the **path** array and the **visited** boolean array). Since the maximum path length cannot exceed the total number of vertices N , the space complexity is strictly bounded by $\mathcal{O}(N)$.

Q5. Discuss the Presumed Optimal Solution for the Multipeg Tower of Hanoi Problem with its properties. Solve the problem for $(n, p) = (395, 8)$. (5+10 marks) [CSE 203 | L-2/T-1 | 2017–18]

Answer

1. Discussion of the Presumed Optimal Solution (Frame-Stewart Algorithm)

The **Multipeg Tower of Hanoi** problem generalizes the classical 3-peg puzzle to p pegs ($p \geq 3$) with n distinct disks. The standard, widely accepted framework for generating the optimal sequence of moves is the **Frame-Stewart Algorithm**, proposed independently by J.S. Frame and B.M. Stewart in 1941.

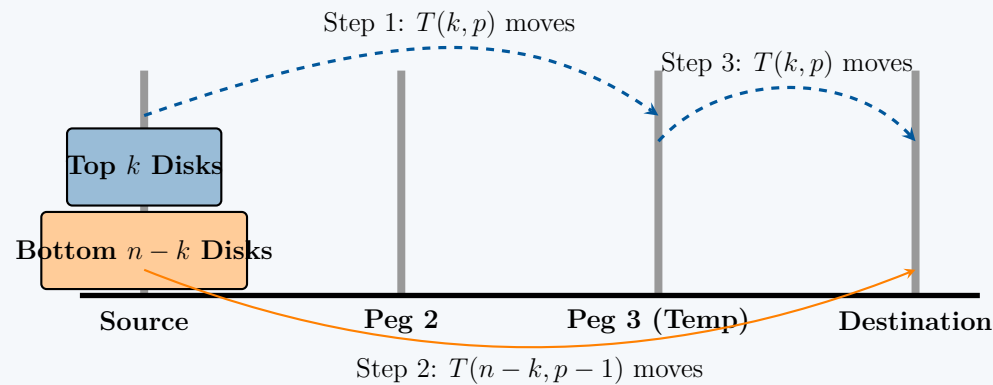
The Algorithmic Strategy: To transfer n disks from a source peg to a destination peg using p available pegs:

- Choose an optimal partitioning integer k such that $1 \leq k < n$.
- Recursively transfer the top k smallest disks from the source peg to an intermediate peg using all p pegs.
- Transfer the remaining $n - k$ larger disks from the source peg to the destination peg using only the remaining $p - 1$ pegs (since the peg holding the k smaller disks cannot accommodate larger ones).
- Recursively transfer the k smallest disks from the intermediate peg to the destination peg using all p pegs.

$$T(n, p) = \min_{1 \leq k < n} \{2T(k, p) + T(n - k, p - 1)\}$$

Base Cases:

- $T(0, p) = 0$ and $T(1, p) = 1$ for all $p \geq 3$.
- For $p = 3$, it reduces to the classical Tower of Hanoi: $T(n, 3) = 2^n - 1$.



Properties of the Presumed Optimal Solution:

- **Optimality Status:** The algorithm was formally proven optimal for $p = 4$ pegs by Thierry Bousch in 2014. For $p > 4$, it remains an unproven mathematical conjecture, hence designated as the *presumed* optimal solution.
- **Structure of Cost Increments:** The sequence of first differences $D(n, p) = T(n, p) - T(n-1, p)$ consists entirely of powers of 2. For a given p , the value 2^m appears exactly $\binom{m+p-3}{p-3}$ times.
- **Asymptotic Complexity:** For a fixed number of pegs p , the number of total moves scales as a sub-exponential function:

$$T(n, p) = \Theta\left(2^{n^{1/(p-2)}}\right)$$

2. Dynamic Programming Formulation

To implement and evaluate the Frame-Stewart recurrence optimally, a bottom-up Dynamic Programming approach is utilized to eliminate redundant recursive calculations.

Algorithm 85 Frame-Stewart Dynamic Programming Solver

```

1: Input: Max disks  $n$ , total pegs  $p$ 
2: Output: Exact minimum moves  $T[n][p]$ 
3: Initialize a 2D array  $T[n+1][p+1]$  with 0
4: for  $i \leftarrow 1$  to  $n$  do
5:    $T[i][3] \leftarrow 2^i - 1$  ▷ Base Case: Standard 3-peg problem
6: end for
7: for  $j \leftarrow 3$  to  $p$  do
8:    $T[1][j] \leftarrow 1$  ▷ Base Case: 1 disk requires 1 move
9: end for
10: for  $j \leftarrow 4$  to  $p$  do
11:   for  $i \leftarrow 2$  to  $n$  do
12:      $min\_moves \leftarrow \infty$ 
13:     for  $k \leftarrow 1$  to  $i-1$  do
14:        $current\_moves \leftarrow 2 \cdot T[k][j] + T[i-k][j-1]$ 
15:       if  $current\_moves < min\_moves$  then
16:          $min\_moves \leftarrow current\_moves$ 
17:       end if
18:     end for
19:      $T[i][j] \leftarrow min\_moves$ 
20:   end for
21: end for
22: return  $T[n][p]$ 
    
```

Complexity Analysis:

- **Time Complexity:** $\mathcal{O}(p \cdot n^2)$. The nested loops evaluate n disk configurations across p pegs, iterating through $i-1$ possible split points at each step.
- **Space Complexity:** $\mathcal{O}(p \cdot n)$ to maintain the look-up table for intermediate results.

3. Analytical Solution for $(n, p) = (395, 8)$

Using the combinatorial property of the cost increments, we can systematically isolate the contribution of each power of 2 to compute the exact value of $T(395, 8)$. For $p = 8$, the frequency of each move increment 2^m corresponds to the binomial coefficient:

$$\text{Frequency}(m) = \binom{m+8-3}{8-3} = \binom{m+5}{5}$$

We evaluate this sequentially for each integer block $m \geq 0$ until the cumulative count of allocated disks reaches $n = 395$:

Exponent (m)	Increment (2^m)	Frequency ($\binom{m+5}{5}$)	Cumulative Disks	Moves Contributed
0	$2^0 = 1$	$\binom{5}{5} = 1$	1	$1 \times 1 = 1$
1	$2^1 = 2$	$\binom{6}{5} = 6$	7	$6 \times 2 = 12$
2	$2^2 = 4$	$\binom{7}{5} = 21$	28	$21 \times 4 = 84$
3	$2^3 = 8$	$\binom{8}{5} = 56$	84	$56 \times 8 = 448$
4	$2^4 = 16$	$\binom{9}{5} = 126$	210	$126 \times 16 = 2016$
5	$2^5 = 32$	$395 - 210 = 185$ (Partial)	395	$185 \times 32 = 5920$
Total	–	395	–	8481

Detailed Step-by-Step Derivation:

$$\begin{aligned} \text{Disks remaining for block } m = 5 &= 395 - \sum_{m=0}^4 \binom{m+5}{5} \\ &= 395 - (1 + 6 + 21 + 56 + 126) \\ &= 395 - 210 = 185 \end{aligned}$$

Because the full structural capacity of block $m = 5$ is $\binom{5+5}{5} = \binom{10}{5} = 252$, and $185 \leq 252$, the final 185 disks fall completely within this block and generate an increment value of exactly 32 moves per disk. Summing up the total operations:

$$\begin{aligned} T(395, 8) &= \sum_{m=0}^4 \left[2^m \cdot \binom{m+5}{5} \right] + [185 \cdot 2^5] \\ &= (1 \cdot 1) + (2 \cdot 6) + (4 \cdot 21) + (8 \cdot 56) + (16 \cdot 126) + (185 \cdot 32) \\ &= 1 + 12 + 84 + 448 + 2016 + 5920 \\ &= 8481 \end{aligned}$$

Conclusion: The minimum number of moves required to solve the Multipeg Tower of Hanoi problem for 395 disks using 8 pegs under the Frame-Stewart strategy is exactly **8481** moves.

Q6. Consider the following Boolean formula: $(x' \vee y)(x \vee y' \vee z)(y \vee y')(x \vee y)(x \vee y \vee z')(x \vee y \vee z)$. Use a backtracking algorithm to decide whether it is satisfiable. Show the backtracking tree. **(15 marks)** [CSE 207 | L-2/T-2 | 2016–17]

Answer

1. Problem Setup and Clause Analysis

We are given the following Boolean formula F in Conjunctive Normal Form (CNF) with three variables (x, y, z) and six clauses (C_1 to C_6):

$$F = C_1 \wedge C_2 \wedge C_3 \wedge C_4 \wedge C_5 \wedge C_6$$

where the individual clauses are defined as:

- $C_1 = (x' \vee y)$
- $C_2 = (x \vee y' \vee z)$
- $C_3 = (y \vee y')$
- $C_4 = (x \vee y)$
- $C_5 = (x \vee y \vee z')$
- $C_6 = (x \vee y \vee z)$

Initial Observation (Tautology Elimination): Clause $C_3 = (y \vee y')$ is a tautology because it evaluates to True (1) for any truth assignment of y . Therefore, it does not constrain the satisfiability of the formula and can be effectively ignored during evaluation, simplifying our formula to:

$$F \equiv (x' \vee y) \wedge (x \vee y' \vee z) \wedge (x \vee y) \wedge (x \vee y \vee z') \wedge (x \vee y \vee z)$$

2. Step-by-Step Backtracking Exploration

We apply the chronological backtracking algorithm, assigning truth values to variables in lexicographical order $(x \rightarrow y \rightarrow z)$. We use 0 for False and 1 for True.

Branch 1: $x = 0$ Substituting $x = 0$ into the simplified formula:

- $C_1 = (0' \vee y) = (1 \vee y) = 1$
- $C_2 = (0 \vee y' \vee z) = (y' \vee z)$
- $C_4 = (0 \vee y) = y$
- $C_5 = (0 \vee y \vee z') = (y \vee z')$
- $C_6 = (0 \vee y \vee z) = (y \vee z)$

The formula simplifies to $F|_{x=0} = (y' \vee z) \wedge y \wedge (y \vee z') \wedge (y \vee z)$.

- **Sub-branch $y = 0$:** Substituting $y = 0$ makes clause $C_4 = 0$. Since a clause evaluates to False, the entire conjunction becomes False.

Conflict detected! Backtrack immediately.

- **Sub-branch $y = 1$:** Substituting $y = 1$ into $F|_{x=0}$:

- $C_2 = (1' \vee z) = (0 \vee z) = z$
- $C_4 = 1$
- $C_5 = (1 \vee z') = 1$
- $C_6 = (1 \vee z) = 1$

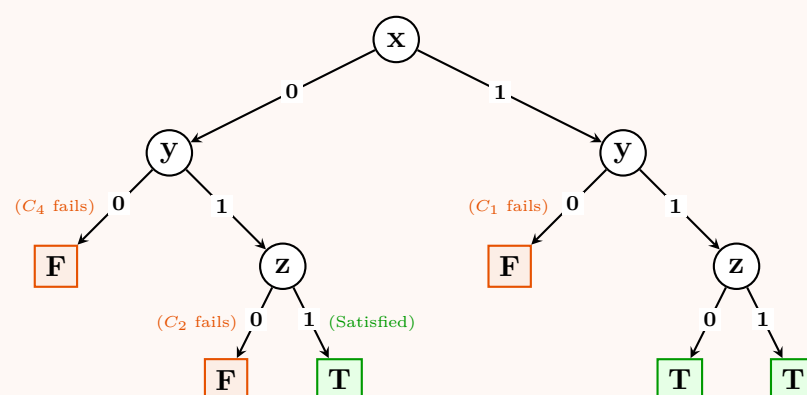
The remaining formula simplifies to $F|_{x=0, y=1} = z$.

- **Try $z = 0$:** This yields $C_2 = 0$. → **Conflict!**
- **Try $z = 1$:** This yields $C_2 = 1$. All clauses are simultaneously satisfied!

At this point, the assignment $\{\mathbf{x} = 0, \mathbf{y} = 1, \mathbf{z} = 1\}$ satisfies the formula. A standard backtracking algorithm terminates here upon finding the first satisfying assignment. For completeness, the full search space tree below also shows the alternative choices.

3. Backtracking Search Tree

The state-space tree below visualizes the execution. Nodes represent variables, edge labels (0 or 1) represent truth assignments, red **F** blocks represent unsatisfiable leaves (conflicts), and green **T** blocks indicate satisfying assignments.



4. Conclusion

Since the backtracking algorithm successfully discovered at least one terminal leaf node evaluating to True (**T**), we conclude that the Boolean formula is **Satisfiable**.

One satisfying assignment is:

$$x = \text{False } (0), \quad y = \text{True } (1), \quad z = \text{True } (1)$$

5. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(2^n \cdot m)$ in the worst case.

Where $n = 3$ is the number of variables and $m = 6$ is the number of clauses. The algorithm searches a binary tree structure of depth at most n . At every internal node, evaluating the truth values against the clauses requires $\mathcal{O}(m)$ time. Though backtracking prunes mismatched branches early, the theoretical upper bound for full exploration remains exponential.

- **Space Complexity:** $\mathcal{O}(n)$

The storage overhead is strictly dominated by the execution stack frame during recursion, which corresponds directly to the maximum height of the decision path tree (n).

Q7. Compare the techniques of brute-force algorithms, heuristics, and approximation algorithms for coping with hard problems. Write

a backtracking algorithm to color a map with no more than four colors. (5+5=10 marks)

[CSE 207 | L-2/T-2 | 2015–16]

Answer

1. Comparison of Techniques for Coping with Hard Problems

When dealing with NP-Hard problems where polynomial-time optimal solutions are unlikely to exist, we often rely on three primary strategies. The table below compares their fundamental characteristics:

Feature	Brute-Force (Exact)	Approximation Algorithms	Heuristics
Objective	Find the absolute optimal solution.	Find a near-optimal solution with proven bounds.	Find a "good enough" solution quickly in practice.
Time Complexity	Usually exponential $\mathcal{O}(2^n)$ or factorial $\mathcal{O}(n!)$.	Polynomial time $\mathcal{O}(n^k)$.	Usually polynomial time, often very fast.
Solution Quality	Guaranteed 100% optimal.	Sub-optimal, but guaranteed to be within a specific ratio ρ of the optimum.	No worst-case guarantees. May be optimal or arbitrarily bad.
Mathematical Guarantees	Yes (Completeness and Optimality).	Yes (Rigorous approximation bounds).	None (Rely on empirical testing).
Applicability	Only feasible for very small problem instances.	Used when theoretically bounded error is strictly required.	Used for large real-world instances where speed is critical.

2. Backtracking Algorithm for 4-Coloring a Map

The problem of coloring a map such that no two adjacent regions share the same color can be modeled as the **Graph Coloring Problem**. Each region is a vertex, and adjacent regions are connected by edges. We are restricted to at most $m = 4$ colors. We solve this using a recursive backtracking approach. We try assigning colors 1 to 4 to each vertex one by one. If a color assignment violates the adjacency constraint, we backtrack.

Algorithm 86 Graph Coloring Backtracking**Require:** Adjacency Matrix $G[1 \dots n][1 \dots n]$, number of colors $m = 4$ **Ensure:** A valid color assignment array $color[1 \dots n]$ or failure.

```

1: Initialize  $color[1 \dots n] \leftarrow 0$  ▷ 0 means uncolored
2:
3: function ISSAFE( $v, G, color, c$ )
4:   for  $u \leftarrow 1$  to  $n$  do
5:     if  $G[v][u] == 1$  and  $color[u] == c$  then
6:       return False ▷ Adjacent vertex has the same color
7:     end if
8:   end for
9:   return True
10: end function
11:
12: function COLORMAPUTIL( $v, G, m, color$ )
13:   if  $v > n$  then
14:     return True ▷ All vertices are successfully colored
15:   end if
16:   for  $c \leftarrow 1$  to  $m$  do
17:     if ISSAFE( $v, G, color, c$ ) then
18:        $color[v] \leftarrow c$  ▷ Assign color  $c$  to vertex  $v$ 
19:       if COLORMAPUTIL( $v + 1, G, m, color$ ) then
20:         return True ▷ Proceed to next vertex
21:       end if
22:        $color[v] \leftarrow 0$  ▷ Backtrack: Remove color assignment
23:     end if
24:   end for
25:   return False ▷ No valid color found for vertex  $v$ 
26: end function
27:
28: function GRAPHCOLORING( $G, m$ )
29:   if COLORMAPUTIL( $1, G, m, color$ ) then
30:     return  $color$ 
31:   else
32:     return "No solution exists"
33:   end if
34: end function

```

Algorithm Definition**3. Complexity Analysis**

- **Time Complexity:** $\mathcal{O}(m^V)$

Here, $m = 4$ is the maximum number of colors and $V = n$ is the number of vertices (regions). In the worst-case scenario, the algorithm attempts every possible color combination for every vertex. The search tree has a branching factor of m and a maximum depth of V , leading to an exponential time bound of $\mathcal{O}(4^n)$.

- **Space Complexity:** $\mathcal{O}(V)$

The space complexity is determined by the maximum depth of the recursive call stack, which is exactly V (one frame for each vertex). Additionally, an array of size V is required to store the current color assignments. Thus, the overall space complexity is $\mathcal{O}(n)$.

Q8. Solve the Multi-peg Tower of Hanoi problem with $(n, p) = (377, 8)$. Find $k_{\max}$, N_0 , and n_1 for at least two subproblems. **(10 marks)**
[CSE 207 | L-2/T-2 | 2013–14]

Q9. Solve the Multi-peg Tower of Hanoi problem with $(n, p) = (270, 7)$, showing the values of k , $\max$, N_0 , n_1 . Present the solution by drawing a binary tree. **(15 marks)**
[CSE 207 | L-2/T-2 | 2008–09]

Q10. Discuss the Multi-peg Hanoi problem. Argue in favour of the recurrence relation to be satisfied by the presumed optimal solution. Prove the properties of the presumed optimal solution. **(5+5+10=20 marks)**
[CSE 207 | L-2/T-2 | 2008–09]

Q11. Write down the backtracking technique for solving the 4-Queen problem. Give the state tree also. **(10+5=15 marks)** [CSE 207 | L-2/T-2 | 2008–09]

Answer

1. Problem Definition and Backtracking Formulation

The **4-Queen Problem** requires placing 4 queens on a 4×4 chessboard such that no two queens threaten each other. This means no two queens can share the same row, column, or diagonal.

State Representation: We represent the board using a 1-dimensional array $Q[1 \dots 4]$, where the index i represents the row number, and the value $Q[i]$ represents the column position of the queen in row i . This formulation automatically ensures that no two queens are in the same row.

Bounding/Pruning Constraints: When attempting to place a new queen in row k and column c , we must verify it against all previously placed queens in rows $1 \dots (k-1)$. A placement is **invalid** if:

- **Same Column:** $Q[i] == c$
- **Same Diagonal:** $|Q[i] - c| == |i - k|$
(The slope of the line connecting two squares on the same diagonal is always ± 1 .)

2. Backtracking Algorithm

The algorithm incrementally builds the solution. If a placement leads to a state where subsequent queens cannot be placed, it *backtracks* by removing the last placed queen and trying the next available column.

Algorithm 87 4-Queens Backtracking Algorithm

Require: $N = 4$ (Board size and number of queens)

Ensure: A valid configuration array $Q[1 \dots N]$ or exhaust all possibilities.

```

1: Initialize  $Q[1 \dots N] \leftarrow [0, 0, 0, 0]$ 
2: function ISSAFE( $k, c$ )
3:   for  $i \leftarrow 1$  to  $k-1$  do
4:     if  $Q[i] == c$  or  $|Q[i] - c| == |i - k|$  then
5:       return False
6:     end if
7:   end for
8:   return True
9: end function
10:
11: function SOLVENQUEENS( $k$ )
12:   for  $c \leftarrow 1$  to  $N$  do
13:     if ISSAFE( $k, c$ ) then
14:        $Q[k] \leftarrow c$ 
15:       if  $k == N$  then
16:         Print  $Q[1 \dots N]$ 
17:       else
18:         SOLVENQUEENS( $k+1$ )
19:       end if
20:     end if
21:   end for
22: end function

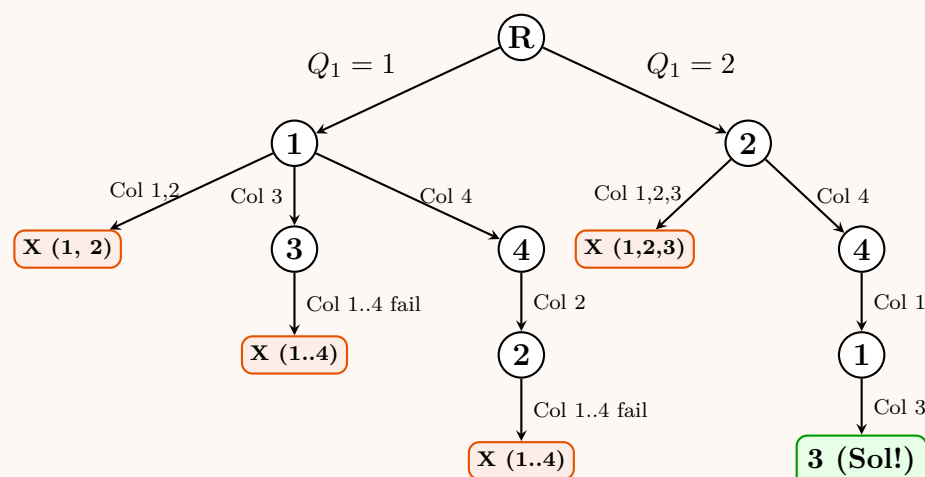
```

5: Conflict detected
 8: Placement is safe
 14: Place Queen k in column c
 16: Valid solution found
 18: Recurse for next row

3. State-Space Tree

The state-space tree below illustrates the backtracking process. The root represents the empty board. The levels 1 to 4 represent the rows. The values inside the nodes are the **column numbers** chosen for that row.

Branches leading to immediate conflicts (column or diagonal) are pruned early (marked with **X**). We show the full trace leading to the first valid solution: $Q = [2, 4, 1, 3]$.



4. Complexity Analysis

- Time Complexity:** $\mathcal{O}(N!)$
For the first row, there are N choices. For the second row, there are at most $N - 1$ choices (due to column constraints), and so on. Without pruning, this is bounded by $N!$. Backtracking significantly reduces the number of generated states in practice, but the worst-case upper bound remains $\mathcal{O}(N!)$.
- Space Complexity:** $\mathcal{O}(N)$
The space complexity is defined by the maximum depth of the recursive call stack, which corresponds to the number of rows N . Additionally, the array $Q[1 \dots N]$ takes $\mathcal{O}(N)$ space. Thus, overall space required is $\mathcal{O}(N)$.

Q12. Discuss the Multi-Peg Tower of Hanoi problem. Argue in favour of the recurrence relation satisfied by the presumed optimal solution. Solve the problem with $(n, p) = (271, 7)$, showing the solution in a binary tree with at least 4 nodes. **(15+20 marks)** [CSE 207 | L-2/T-2 | 2007–08]

Q13. Solve the Multi-peg Tower of Hanoi problem with $(n, p) = (273, 7)$. Show the subproblems arising up to the second level of recursion using a binary tree. **(15 marks)** [CSE 207 | L-2/T-2 | 2006–07]

Branch and Bound

Q1. Consider the following instance of the 0-1 knapsack problem. Solve this instance using the branch and bound algorithm. Show the branch-and-bound tree. Knapsack capacity = 10.

Item	Weight	Value
1	4	40
2	3	50
3	5	70
4	3	45

(15 marks)

[CSE 207 | L-2/T-1 | 2024–25]

Answer

1. Preprocessing: Item Sorting

The optimal strategy for the Branch and Bound approach to the 0-1 Knapsack problem is to first compute the value-to-weight ratio (v_i/w_i) for each item and sort them in **descending order**. This allows the algorithm to greedily find the highest theoretical upper bound at each node.

Given the knapsack capacity $W = 10$, we calculate and sort:

Sorted Label	Original Item	Value (v_i)	Weight (w_i)	Ratio (v_i/w_i)
A	2	50	3	16.67
B	4	45	3	15.00
C	3	70	5	14.00
D	1	40	4	10.00

2. Bounding Function Definition

At any node in our state-space tree, the **Upper Bound (UB)** is calculated by taking the accumulated value v of the already included items, and then greedily adding the remaining items in sorted order. If an item fits entirely, its full value is added. If it exceeds the remaining capacity, we add a *fraction* of its value to fill the knapsack exactly to W .

$$UB = v + \sum_{\text{full items}} v_i + \left(\frac{\text{remaining capacity}}{\text{weight of first item that doesn't fit}} \right) \times \text{value of that item}$$

A node is **pruned** if:

- Its total weight exceeds the capacity W (*Infeasible*).
- Its UB is less than or equal to the best integer solution found so far (*MaxProfit*).

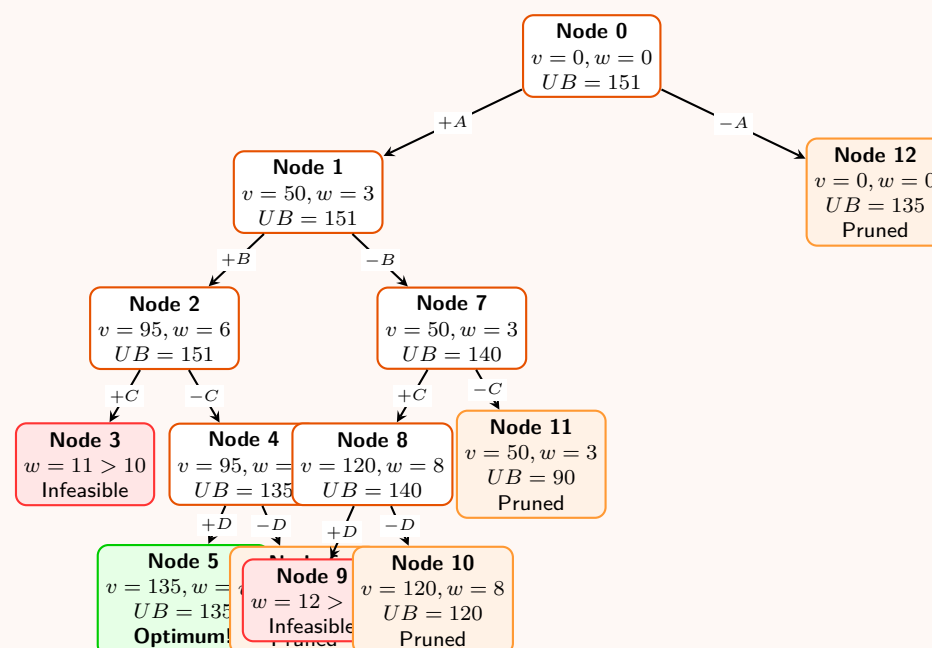
3. Step-by-Step State Evaluation (Depth-First Search)

We initialize $MaxProfit = 0$ and explore the state-space tree using DFS.

- Node 0 (Root):** $w = 0, v = 0$. We fill with A($w = 3$), B($w = 3$), leaving capacity 4. We take $\frac{4}{5}$ of C($v = 70$). $UB = 0 + 50 + 45 + (\frac{4}{5} \times 70) = 151$.

- **Node 1 (+A):** $w = 3, v = 50$. $UB = 151$. Explore further.
- **Node 2 (+B):** $w = 6, v = 95$. $UB = 151$. Explore further.
- **Node 3 (+C):** $w = 11 > 10$. **Pruned (Infeasible)**.
- **Node 4 (-C):** $w = 6, v = 95$. Remaining capacity 4. Next is D($w = 4, v = 40$), which fits perfectly. $UB = 95 + 40 = 135$. Explore further.
- **Node 5 (+D):** $w = 10, v = 135$. We reached a valid full assignment. **Update** $MaxProfit = 135$.
- **Node 6 (-D):** $w = 6, v = 95$. $UB = 95$. Since $95 \leq 135$, **Pruned**.
- **Node 7 (-B from Node 1):** $w = 3, v = 50$. Remaining capacity 7. We fill with C($w = 5$), leaving 2. We take $\frac{2}{4}$ of D($v = 40$). $UB = 50 + 70 + (\frac{2}{4} \times 40) = 140$. Since $140 > 135$, explore further.
- **Node 8 (+C):** $w = 8, v = 120$. Remaining capacity 2. We take $\frac{2}{4}$ of D($v = 40$). $UB = 120 + 20 = 140$. Explore.
- **Node 9 (+D):** $w = 12 > 10$. **Pruned (Infeasible)**.
- **Node 10 (-D):** $w = 8, v = 120$. $UB = 120 \leq 135$. **Pruned**.
- **Node 11 (-C from Node 7):** $w = 3, v = 50$. Remaining capacity 7. Next is D($w = 4$). $UB = 50 + 40 = 90 \leq 135$. **Pruned**.
- **Node 12 (-A from Root):** $w = 0, v = 0$. Fill with B($w = 3$), C($w = 5$), leaving 2. Take $\frac{2}{4}$ of D. $UB = 0 + 45 + 70 + (\frac{2}{4} \times 40) = 135$. Since $UB \leq MaxProfit$, **Pruned**.

4. Branch and Bound State-Space Tree



5. Final Solution & Complexity Analysis

By successfully traversing the Branch-and-Bound tree, we find the maximum profit.

- **Optimal Selection:** Items A, B, and D.
- **Original Item Mapping:** Item 2, Item 4, and Item 1.
- **Maximum Value:** 135
- **Total Weight:** $3 + 3 + 4 = 10 \leq W$

Complexity Analysis:

- **Time Complexity:** $\mathcal{O}(2^n)$ worst case. Sorting takes $\mathcal{O}(n \log n)$. Although the bounding function prunes large subtrees, preventing exponential exploration in many practical scenarios, the worst-case bound remains exponential if bounds are weak.
- **Space Complexity:** $\mathcal{O}(n)$ to maintain the DFS recursion stack height and store the current assignments, making depth-first search much more space-efficient than breadth-first (BFS) branching strategies.

Q2. Explain branch-and-bound algorithms with an illustrative example. (15 marks)

[CSE 207 | L-2/T-1 | 2023–24]

Answer

1. Introduction to Branch and Bound

Branch and Bound (B&B) is an algorithmic paradigm used primarily for solving combinatorial optimization problems, typically those that are NP-Hard. It systematically enumerates candidate solutions by exploring branches of a state-space tree. To avoid exhaustive brute-force search, B&B employs a *bounding* function to calculate the best possible solution in a given subtree. If this bound is worse than the best solution found so far, the entire subtree is discarded (*pruned*). The three core operations are:

- (a) **Branching:** Splitting a problem instance into mutually exclusive subproblems (e.g., "include item i " vs. "exclude item i "), creating a state-space tree.
- (b) **Bounding:** Calculating a guaranteed upper bound (for maximization) or lower bound (for minimization) on the objective function for any solution in a subproblem.
- (c) **Pruning:** Discarding a subproblem if its bound indicates it cannot possibly yield a better solution than the current optimal candidate.

2. General B&B Framework (Maximization Problem)

Algorithm 88 Best-First Branch and Bound

Require: Problem instance P
Ensure: The optimal solution value and configuration.

```
1:  $MaxProfit \leftarrow -\infty$ 
2:  $PQ \leftarrow$  Priority Queue ordered by bound (descending)
3:  $root \leftarrow$  initial state
4:  $root.bound \leftarrow COMPUTEBOUND(root)$ 
5:  $ENQUEUE(PQ, root)$ 
6: while  $PQ$  is not empty do
7:    $node \leftarrow DEQUEUEMAX(PQ)$ 
8:   if  $node.bound \leq MaxProfit$  then
9:     continue
10:  end if
11:  for each  $child$  of  $node$  do
12:    if  $child$  is a valid full solution then
13:       $MaxProfit \leftarrow \max(MaxProfit, child.value)$ 
14:    else if  $child$  is feasible then
15:       $child.bound \leftarrow COMPUTEBOUND(child)$ 
16:      if  $child.bound > MaxProfit$  then
17:         $ENQUEUE(PQ, child)$ 
18:      end if
19:    end if
20:  end for
21: end while
22: return  $MaxProfit$ 
```

3. Illustrative Example: 0-1 Knapsack Problem

Consider a 0-1 Knapsack Problem with Capacity $W = 10$. Items are sorted by value-to-weight ratio in descending order:

Item	Value (v_i)	Weight (w_i)	Ratio (v_i/w_i)
1	40	4	10
2	42	7	6
3	25	5	5

Bounding Function (Upper Bound - UB): To compute the UB for any node, we take the current profit, add the profit of remaining items that fit entirely, and finally add a fractional amount of the first item that does not fit.

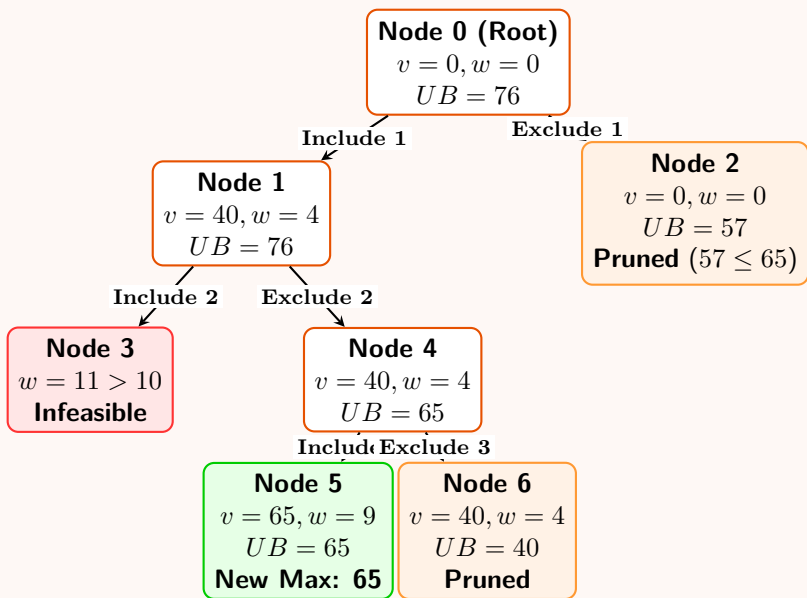
Step-by-Step Execution:

- **Node 0 (Root):** Current $w = 0, v = 0$. Add Item 1 ($w = 4, v = 40$). Remaining cap 6. Item 2 ($w = 7$) does not fit. Take fraction $\frac{6}{7} \times 42 = 36$. **UB** = $40 + 36 = 76$.
- **Node 1 (Include Item 1):** $w = 4, v = 40$. **UB** = **76**.
- **Node 2 (Exclude Item 1):** $w = 0, v = 0$. Add Item 2 ($w = 7, v = 42$). Remaining cap 3. Take fraction $\frac{3}{5} \times 25 = 15$. **UB** = $42 + 15 = 57$.
- **Node 3 (Include Item 1 & 2):** $w = 4 + 7 = 11 > 10$. **Infeasible**.
- **Node 4 (Include Item 1, Exclude Item 2):** $w = 4, v = 40$. Add Item 3 entirely ($w = 5$). **UB** = $40 + 25 = 65$.

- **Node 5 (Include Item 1, 3; Exclude 2):** $w = 9 \leq 10, v = 65$. Valid leaf. Update **MaxProfit = 65**.
- **Pruning Node 2:** We go back to unexplored Node 2. Its $UB = 57$. Since $57 \leq \text{MaxProfit}(65)$, we **prune** the entire right half of the tree.

4. State-Space Tree Diagram

The diagram below maps the execution. Node 2 is pruned mathematically, proving we do not need to evaluate Item 2 and Item 3 combinations when Item 1 is excluded.



5. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(2^n)$ in the worst case (where n is the number of items/variables). Although B&B significantly prunes the search space reducing the average-case runtime drastically, pathological instances exist where bounds are extremely weak, forcing the algorithm to explore almost the entire exponential tree.
- **Space Complexity:** $\mathcal{O}(2^n)$ in the worst case for Best-First Search because the Priority Queue must store all unexpanded nodes across the breadth of the tree. If Depth-First Branch and Bound is used, the space complexity drops to $\mathcal{O}(n)$, which is the maximum depth of the recursion stack.

Q3. You want to fill up your luggage with required items for an upcoming tour. The luggage has a capacity of 60 kg. The items have the following values and weights:

Item	Value	Weight (kg)
1	120	15
2	110	10
3	95	25
4	135	35
5	80	20
6	150	30

Solve this 0-1 knapsack problem using the branch and bound technique to maximize total value. **(15 marks)** [CSE 207 | L-2/T-1 | 2022–23]

Answer

1. Preprocessing: Item Sorting

In the Branch and Bound technique for the 0-1 Knapsack problem, we first compute the value-to-weight ratio (v_i/w_i) for each item and sort them in **descending order**. This ordering allows the algorithm to greedily compute the tightest possible theoretical upper bound at each node.

Given the knapsack capacity $W = 60$, the sorted items are:

Sorted Label	Original Item	Value (v_i)	Weight (w_i)	Ratio (v_i/w_i)
A	2	110	10	11.00
B	1	120	15	8.00
C	6	150	30	5.00
D	5	80	20	4.00
E	4	135	35	3.86
F	3	95	25	3.80

2. Bounding Function

At any node in the state-space tree, the **Upper Bound (UB)** is calculated by taking the accumulated value v of the already included items, adding the full values of the remaining sorted items that fit, and then adding a *fraction* of the first item that does not fit.

$$UB = v + \sum_{\text{fitting items}} v_i + \left(\frac{\text{remaining capacity}}{\text{weight of first item that doesn't fit}} \right) \times \text{value of that item}$$

We traverse the state-space tree using **Depth-First Search (DFS)**. A node is **pruned** if:

- Its total weight exceeds the capacity $W = 60$ (*Infeasible*).
- Its $UB \leq \text{MaxProfit}$ (the best valid integer solution found so far).

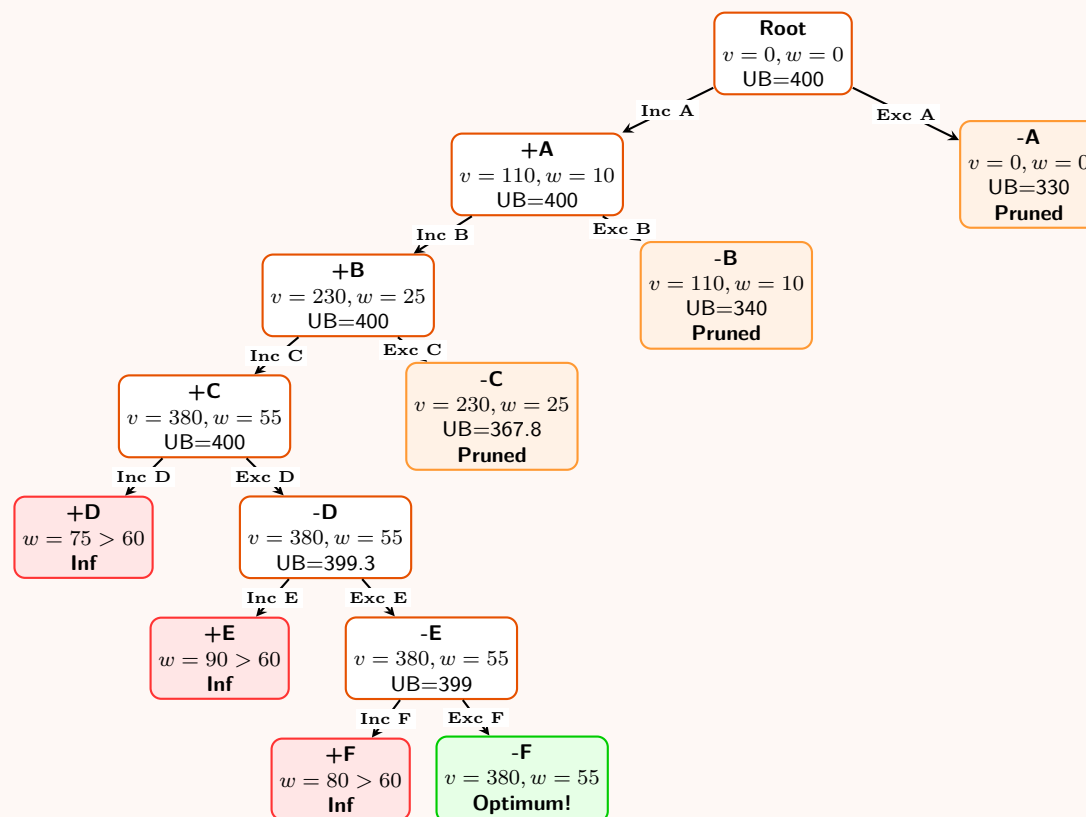
3. Step-by-Step Execution (DFS Trace)

We initialize $\text{MaxProfit} = 0$ and explore by deciding whether to include (+) or exclude (−) each item in order.

- **Node 0 (Root):** $w = 0, v = 0$. Fill greedily with A($w = 10$), B($w = 15$), C($w = 30$). Total $w = 55, v = 380$. Remaining capacity = 5. Take $\frac{5}{20}$ of D($v = 80$).
 $UB = 380 + (\frac{5}{20} \times 80) = 400$.
- **Include A (+A):** $w = 10, v = 110$. $UB = 400$. (Proceed to B)
- **Include B (+B):** $w = 25, v = 230$. $UB = 400$. (Proceed to C)
- **Include C (+C):** $w = 55, v = 380$. Remaining cap 5. Next item D is 20kg. $UB = 380 + 20 = 400$.
- **Include D (+D):** $w = 55 + 20 = 75 > 60$. **Pruned (Infeasible)**.
- **Exclude D (-D):** $w = 55, v = 380$. Next E($w = 35$). $UB = 380 + (\frac{5}{35} \times 135) = 399.28$.
- **Include E (+E):** $w = 55 + 35 = 90 > 60$. **Pruned (Infeasible)**.
- **Exclude E (-E):** $w = 55, v = 380$. Next F($w = 25$). $UB = 380 + (\frac{5}{25} \times 95) = 399$.
- **Include F (+F):** $w = 55 + 25 = 80 > 60$. **Pruned (Infeasible)**.
- **Exclude F (-F):** $w = 55, v = 380$. All items considered. This is a valid leaf!
Update $\text{MaxProfit} = 380$.
- **Backtrack to (-C):** We excluded C. Current $w = 25, v = 230$. Greedily fill with D($w = 20$), remaining cap 15. Take fraction of E($w = 35$). $UB = 230 + 80 + (\frac{15}{35} \times 135) = 367.85$.
Since $UB = 367.85 \leq 380$, **Pruned**.
- **Backtrack to (-B):** Excluded B. Current $w = 10, v = 110$. Fill with C($w = 30$), D($w = 20$). Capacity perfectly full.
 $UB = 110 + 150 + 80 = 340$.
Since $UB = 340 \leq 380$, **Pruned**.
- **Backtrack to (-A):** Excluded A. Current $w = 0, v = 0$. Fill with B($w = 15$), C($w = 30$). Remaining cap 15. Take fraction of D($w = 20$). $UB = 120 + 150 + (\frac{15}{20} \times 80) = 330$.
Since $UB = 330 \leq 380$, **Pruned**.

4. State-Space Tree Diagram

The tree demonstrates the pruning power of the Branch and Bound algorithm. Once the optimum (380) is found deep in the left branch, large right-hand subtrees are discarded without exploration.



5. Final Solution & Complexity Analysis

Optimal Selection:

- Included Items: **A, B, C** $\implies$ Original **Item 2, Item 1, Item 6**.
- Total Weight: $10 + 15 + 30 = 55 \text{ kg} \leq 60 \text{ kg}$.
- Maximum Value: $110 + 120 + 150 = 380$.

Complexity Analysis:

- **Time Complexity:** $\mathcal{O}(2^n)$ worst case. Sorting takes $\mathcal{O}(n \log n)$. While Branch and Bound extensively prunes unpromising configurations (making it computationally feasible for moderate n), the theoretical upper bound of expanding the entire tree remains $\mathcal{O}(2^n)$ in pathological cases.
- **Space Complexity:** $\mathcal{O}(n)$ using Depth-First Branch and Bound, as we only need to store the active path recursion stack and the best solution found so far. (If Best-First Search were used instead, space complexity would be exponential due to the priority queue storing frontier nodes).

Q4. There are 4 places in your village. You want to start from a place, then visit all other places and return to the first place without visiting the same place twice (other than the starting place), minimizing cost. The costs are given below:

From/To	1	2	3	4
1	0	10	15	20
2	5	0	12	18
3	8	9	0	17
4	11	14	16	0

Give an exponential time algorithm to solve this problem with running time $\mathcal{O}(n^2 \cdot 2^n)$, and find the optimal tour using your algorithm. (20 marks) [CSE 207 | L-2/T-1 | 2022–23]

Answer

1. Problem Identification and Dynamic Programming Approach

The problem described is the **Traveling Salesperson Problem (TSP)** on a directed graph. A brute-force search would examine all $(n-1)!$ permutations of the vertices, which takes $\mathcal{O}(n!)$ time. To solve it in $\mathcal{O}(n^2 2^n)$ time, we use the **Bellman-Held-Karp algorithm**, a dynamic programming (DP) technique.

State Definition: Let $V = \{1, 2, 3, 4\}$ be the set of places, with node 1 as the starting point. Let S be a subset of vertices excluding the start node 1 ($S \subseteq \{2, 3, 4\}$), and let $j \in S$. We define $C(S, j)$ as the minimum cost of a path starting at node 1, visiting every node in the subset S exactly once, and ending at node j .

Recurrence Relation:

- **Base Case:** When S contains only a single node j ($S = \{j\}$), the cost is simply the direct edge from node 1 to node j .

$$C(\{j\}, j) = d_{1,j}$$

- **Recursive Step:** To find the shortest path visiting all nodes in S and ending at j , we consider all possible penultimate nodes

$i \in S$ (where $i \neq j$).

$$C(S, j) = \min_{i \in S, i \neq j} [C(S \setminus \{j\}, i) + d_{i,j}]$$

Final Answer: Once $C(V \setminus \{1\}, j)$ is computed for all $j \in \{2, 3, 4\}$, the cost of the optimal tour is:

$$\text{Optimal Cost} = \min_{j \in \{2, 3, 4\}} [C(\{2, 3, 4\}, j) + d_{j,1}]$$

2. The Held-Karp Algorithm

Algorithm 89 Held-Karp Algorithm for TSP

Require: Distance matrix d of size $n \times n$

Ensure: The cost of the optimal TSP tour

```

1: Initialize  $C(\{j\}, j) = d_{1,j}$  for all  $j \in \{2, \dots, n\}$ 
2: for subset size  $s = 2$  to  $n - 1$  do
3:   for each subset  $S \subseteq \{2, \dots, n\}$  of size  $s$  do
4:     for each  $j \in S$  do
5:        $C(S, j) \leftarrow \min_{i \in S, i \neq j} [C(S \setminus \{j\}, i) + d_{i,j}]$ 
6:     end for
7:   end for
8: end for
9:  $MinCost \leftarrow \infty$ 
10: for each  $j \in \{2, \dots, n\}$  do
11:    $MinCost \leftarrow \min(MinCost, C(\{2, \dots, n\}, j) + d_{j,1})$ 
12: end for
13: return  $MinCost$ 
    
```

3. Step-by-Step Execution

We apply the DP approach to our given distance matrix.

Step 3.1: Subsets of Size 1 ($|S| = 1$)

$$\begin{aligned} C(\{2\}, 2) &= d_{1,2} = 10 \\ C(\{3\}, 3) &= d_{1,3} = 15 \\ C(\{4\}, 4) &= d_{1,4} = 20 \end{aligned}$$

Step 3.2: Subsets of Size 2 ($|S| = 2$) For $S = \{2, 3\}$:

$$\begin{aligned} C(\{2, 3\}, 2) &= C(\{3\}, 3) + d_{3,2} = 15 + 9 = 24 \quad (\text{Path: } 1 \rightarrow 3 \rightarrow 2) \\ C(\{2, 3\}, 3) &= C(\{2\}, 2) + d_{2,3} = 10 + 12 = 22 \quad (\text{Path: } 1 \rightarrow 2 \rightarrow 3) \end{aligned}$$

For $S = \{2, 4\}$:

$$\begin{aligned} C(\{2, 4\}, 2) &= C(\{4\}, 4) + d_{4,2} = 20 + 14 = 34 \quad (\text{Path: } 1 \rightarrow 4 \rightarrow 2) \\ C(\{2, 4\}, 4) &= C(\{2\}, 2) + d_{2,4} = 10 + 18 = 28 \quad (\text{Path: } 1 \rightarrow 2 \rightarrow 4) \end{aligned}$$

For $S = \{3, 4\}$:

$$\begin{aligned} C(\{3, 4\}, 3) &= C(\{4\}, 4) + d_{4,3} = 20 + 16 = 36 \quad (\text{Path: } 1 \rightarrow 4 \rightarrow 3) \\ C(\{3, 4\}, 4) &= C(\{3\}, 3) + d_{3,4} = 15 + 17 = 32 \quad (\text{Path: } 1 \rightarrow 3 \rightarrow 4) \end{aligned}$$

Step 3.3: Subset of Size 3 ($|S| = 3$, $S = \{2, 3, 4\}$) We evaluate the minimum cost to reach each node $j \in \{2, 3, 4\}$ after visiting all others.

$$\begin{aligned} C(\{2, 3, 4\}, 2) &= \min \begin{cases} C(\{3, 4\}, 3) + d_{3,2} = 36 + 9 = 45 \\ C(\{3, 4\}, 4) + d_{4,2} = 32 + 14 = 46 \end{cases} = \mathbf{45} \quad (\text{via node 3}) \\ C(\{2, 3, 4\}, 3) &= \min \begin{cases} C(\{2, 4\}, 2) + d_{2,3} = 34 + 12 = 46 \\ C(\{2, 4\}, 4) + d_{4,3} = 28 + 16 = 44 \end{cases} = \mathbf{44} \quad (\text{via node 4}) \\ C(\{2, 3, 4\}, 4) &= \min \begin{cases} C(\{2, 3\}, 2) + d_{2,4} = 24 + 18 = 42 \\ C(\{2, 3\}, 3) + d_{3,4} = 22 + 17 = \mathbf{39} \end{cases} = \mathbf{39} \quad (\text{via node 3}) \end{aligned}$$

Step 3.4: Final Cost to Return to Node 1 We add the return edge $d_{j,1}$ from the last visited node back to the start:

$$\text{Return from 2: } C(\{2, 3, 4\}, 2) + d_{2,1} = 45 + 5 = 50$$

$$\text{Return from 3: } C(\{2, 3, 4\}, 3) + d_{3,1} = 44 + 8 = 52$$

$$\text{Return from 4: } C(\{2, 3, 4\}, 4) + d_{4,1} = 39 + 11 = 50$$

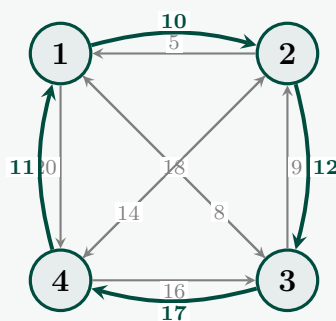
The optimal tour cost is the minimum of these values, which is **50**.

4. Constructing the Optimal Tour

Tracing back the parent pointers that yielded the minimum values:

- Path ending in 4: Optimal cost 50 comes from returning from node 4.
 $4 \leftarrow 3 \leftarrow 2 \leftarrow 1$. Full Tour: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$.
- Path ending in 2: Optimal cost 50 comes from returning from node 2.
 $2 \leftarrow 3 \leftarrow 4 \leftarrow 1$. Full Tour: $1 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$.

Both paths yield the minimum cost of 50. Let's visualize one of the optimal tours ($1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$).



5. Complexity Analysis

- **Time Complexity:** $\mathcal{O}(n^2 2^n)$. There are $\mathcal{O}(2^n)$ subproblems (the number of subsets $S \subseteq V \setminus \{1\}$). For each subset, there are at most n choices for the destination node j , and to calculate the state $C(S, j)$, we take the minimum over at most n intermediate nodes i . Thus, computing all table entries takes $2^n \times n \times n = \mathcal{O}(n^2 2^n)$ time.
- **Space Complexity:** $\mathcal{O}(n 2^n)$. The DP table must store a value for every combination of subset S and ending node j . The number of combinations is roughly $n \cdot 2^{n-1}$.

Q5. There are 6 places in your village. You want to visit all the places and return to the first place that you will visit. Also, you don't want to visit the same place twice other than the starting place. You want to finish the visits with minimum cost. The cost of the roads between these places are given below:

From/To	1	2	3	4	5	6
1	0	10	15	20	25	30
2	5	0	12	18	24	29
3	8	9	0	17	22	28
4	11	14	16	0	21	27
5	13	19	23	26	0	31
6	28	27	25	24	22	0

Find the optimal tour using branch and bound, and pruning techniques combined. **(20 marks)** [CSE 207 | L-2/T-2 | 2021–22]

Answer

1. Problem Setup and Initial Matrix Reduction

We are given an Asymmetric Traveling Salesperson Problem (ATSP) with 6 nodes. To apply the Branch and Bound technique, we first represent the distances in a cost matrix C . Since we cannot travel from a place to itself, we set all diagonal elements to ∞ :

$$C = \begin{pmatrix} \infty & 10 & 15 & 20 & 25 & 30 \\ 5 & \infty & 12 & 18 & 24 & 29 \\ 8 & 9 & \infty & 17 & 22 & 28 \\ 11 & 14 & 16 & \infty & 21 & 27 \\ 13 & 19 & 23 & 26 & \infty & 31 \\ 28 & 27 & 25 & 24 & 22 & \infty \end{pmatrix}$$

To find the lower bound (LB) for the root node, we perform matrix reduction:

- **Row Reduction:** Subtract the minimum value of each row from all elements in that row.
- **Column Reduction:** Subtract the minimum value of each column from all elements in that column.

Step 1: Row Reduction

Row 1 minimum: 10 $\rightarrow (\infty, 0, 5, 10, 15, 20)$
 Row 2 minimum: 5 $\rightarrow (0, \infty, 7, 13, 19, 24)$
 Row 3 minimum: 8 $\rightarrow (0, 1, \infty, 9, 14, 20)$
 Row 4 minimum: 11 $\rightarrow (0, 3, 5, \infty, 10, 16)$
 Row 5 minimum: 13 $\rightarrow (0, 6, 10, 13, \infty, 18)$
 Row 6 minimum: 22 $\rightarrow (6, 5, 3, 2, 0, \infty)$

Row Reduction Sum = $10 + 5 + 8 + 11 + 13 + 22 = 69$.

Step 2: Column Reduction (on the row-reduced matrix)

Column 1 minimum: 0 Column 4 minimum: 2 (from Row 6)
 Column 2 minimum: 0 Column 5 minimum: 0
 Column 3 minimum: 3 (from Row 6) Column 6 minimum: 16 (from Row 4)

Column Reduction Sum = $0 + 0 + 3 + 2 + 0 + 16 = 21$.

The ****Root Lower Bound**** is $69 + 21 = \mathbf{90}$. The fully reduced matrix M_0 becomes:

$$M_0 = \begin{pmatrix} \infty & 0 & 2 & 8 & 15 & 4 \\ 0 & \infty & 4 & 11 & 19 & 8 \\ 0 & 1 & \infty & 7 & 14 & 4 \\ 0 & 3 & 2 & \infty & 10 & 0 \\ 0 & 6 & 7 & 11 & \infty & 2 \\ 6 & 5 & 0 & 0 & 0 & \infty \end{pmatrix}$$

2. Branch and Bound Execution (Best-First Search)

We maintain a Priority Queue of active states, always expanding the node with the lowest LB.

Level 1: Branching from Node 1 For a path $1 \rightarrow j$, the new lower bound is calculated as:

$$LB_{new} = LB_{old} + M_{old}[1, j] + \text{Cost of reducing the new matrix}$$

We form the new matrix by setting Row 1 and Column j to ∞ (since we leave 1 and arrive at j), and setting $M[j, 1] = \infty$ to prevent premature cycling.

- **Path $1 \rightarrow 2$:** $M_0[1, 2] = 0$. After crossing out R1, C2, and setting $M[2, 1] = \infty$, the new matrix requires reducing R2 by 4. $LB = 90 + 0 + 4 = \mathbf{94}$.
- **Path $1 \rightarrow 3$:** $M_0[1, 3] = 2$. Requires reducing R3 by 1. $LB = 90 + 2 + 1 = \mathbf{93}$.
- **Path $1 \rightarrow 4$:** $M_0[1, 4] = 8$. Matrix is already fully reduced. $LB = 90 + 8 + 0 = \mathbf{98}$.
- **Path $1 \rightarrow 6$:** $M_0[1, 6] = 4$. Matrix is already fully reduced. $LB = 90 + 4 + 0 = \mathbf{94}$.

Frontier: $1 \rightarrow 3$ (93) is explored first, generating children with $LB \geq 99$. We then explore $1 \rightarrow 2$ (94).

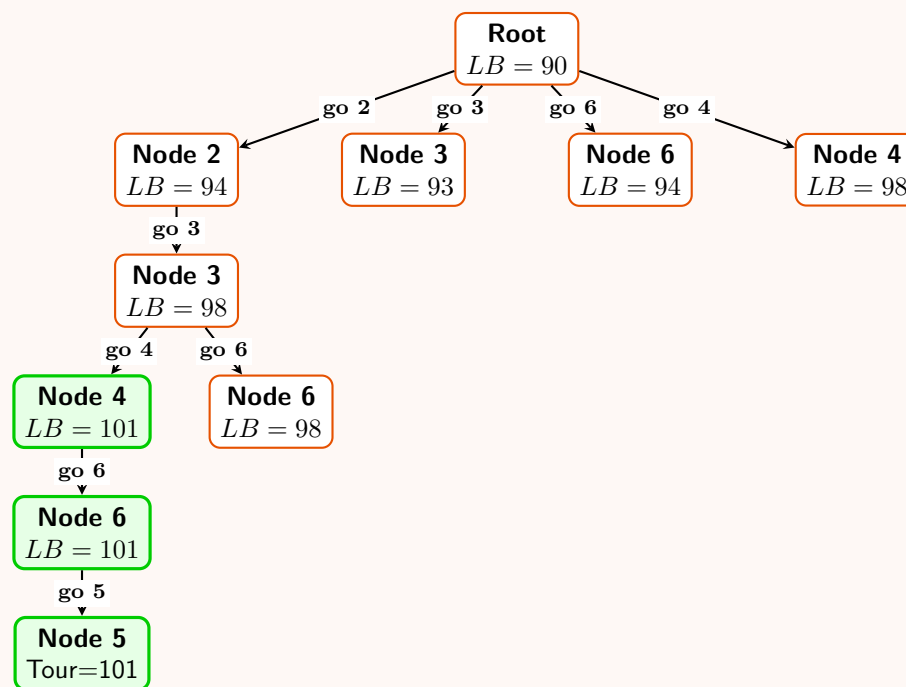
Tracing the Optimal Path ($1 \rightarrow 2 \rightarrow 3 \rightarrow \dots$)

- **Path $1 \rightarrow 2 \rightarrow 3$:** From matrix $M_{1,2}$, $M_{1,2}[2, 3] = 0$. Crossing out R2, C3 and setting $M[3, 1] = \infty$ requires reducing R3 by 4. $LB = 94 + 0 + 4 = \mathbf{98}$.
- **Path $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$:** From matrix $M_{1,2,3}$, $M_{1,2,3}[3, 4] = 3$. Crossing out R3, C4 and setting $M[4, 1] = \infty$ yields a fully reduced matrix. $LB = 98 + 3 + 0 = \mathbf{101}$.
- **Path $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6$:** From matrix $M_{1,2,3,4}$, $M[4, 6] = 0$. Crossing out R4, C6 yields a fully reduced matrix. $LB = 101 + 0 + 0 = \mathbf{101}$.
- **Path $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 5 \rightarrow 1$:** The final forced edge to the last remaining node (5) and back to the start has an incremental reduced cost of 0. $LB = 101 + 0 + 0 = \mathbf{101}$.

Since 101 is a valid complete tour and is $\leq$ the LB of all other unexplored branches in the frontier, we safely ****prune**** the rest of the tree. The optimal tour is found.

3. State-Space Tree Diagram

The diagram visually depicts how the state space is pruned, naturally guiding the algorithm towards the exact optimal path via bounding calculations.



4. Final Answer & Complexity

The exact route traced by the lowest bounds yields the optimal solution:

$$\text{Optimal Tour: } 1 \xrightarrow{10} 2 \xrightarrow{12} 3 \xrightarrow{17} 4 \xrightarrow{27} 6 \xrightarrow{22} 5 \xrightarrow{13} 1$$

$$\text{Minimum Cost} = 10 + 12 + 17 + 27 + 22 + 13 = \mathbf{101}$$

Complexity Analysis:

- **Time Complexity:** $\mathcal{O}(n!)$ in the worst case (pathological instances). However, with effective bounding and pruning, the average time complexity drops dramatically, allowing algorithms to process real-world TSP sizes efficiently. Computing the lower bound at each node takes $\mathcal{O}(n^2)$ time.
- **Space Complexity:** $\mathcal{O}(n! \cdot n^2)$ theoretical worst-case for the Priority Queue holding $n \times n$ matrices across frontier nodes (Best-First strategy). Practical space complexity is drastically smaller due to heavy pruning of unpromising subtrees.

Q6. In one approach to solve the Traveling Salesman Problem using branch-and-bound technique, the lower bound of the cost of completing a partial tour has been calculated from the cost of the minimum spanning tree spanning the remaining nodes of the graph. Decide whether this bound function will give correct answer all the time, with proper explanation. If there is any better bound function for this problem, mention it. (5 marks) [CSE 207 | L-2/T-2 | 2020–21]

Answer

1. Correctness of the MST Lower Bound

Decision: Yes, this bound function will give the correct optimal answer all the time.

Explanation: In the Branch-and-Bound (B&B) algorithm, a heuristic or bounding function guarantees finding the correct optimal solution if and only if it is **admissible**. A bounding function is admissible if it **never overestimates** the true minimum cost to complete the solution. This ensures that the algorithm never accidentally prunes a branch that could lead to the optimal solution. Let us formally prove that the cost of the Minimum Spanning Tree (MST) on the remaining nodes is a valid lower bound for completing the TSP tour:

- Let V be the set of all vertices in the graph.
- Let P be the sequence of nodes in the current partial tour, starting at node S and ending at node E .
- Let $R = V \setminus P$ be the set of unvisited (remaining) nodes.
- To complete the tour, the optimal solution must construct a path that leaves E , visits every node in R exactly once, and returns to S .
- Let this true optimal completion path through R be denoted as C_R . Because C_R is a simple path connecting all vertices in R , it is by definition a **spanning tree** of R (specifically, a spanning tree with a maximum degree of 2).
- By definition, the Minimum Spanning Tree is the spanning tree with the absolute minimum possible weight over the set R . Therefore, the cost of the MST on R will always be less than or equal to the cost of any path passing through R :

$$\text{Cost}(\text{MST}(R)) \leq \text{Cost}(C_R)$$

Because the MST cost is guaranteed to be $\leq$ the actual cost of visiting the remaining nodes, the lower bound is admissible. Thus, the B&B algorithm will always yield the mathematically correct optimal tour.

2. Visualizing the Bound Admissibility

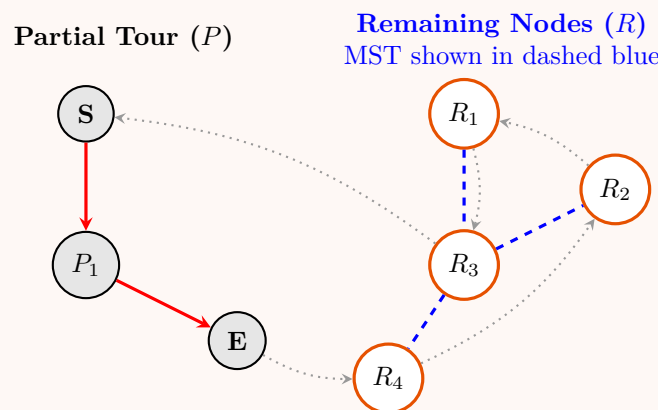


Figure 1: The hypothetical completion path (gray dotted) must connect all nodes in R . The MST of R (blue dashed) connects the same nodes but without the constraints of being a path, naturally resulting in a lower or equal total weight.

3. Better Bounding Functions

While the MST of R is valid, it is a **weak (loose)** lower bound because it entirely ignores the cost of the edges required to connect the remaining nodes R back to the partial tour ends E and S . B&B efficiency heavily relies on having the tightest possible bounds to prune the search space early.

Better bounding functions include:

A. The Augmented MST Bound A strict improvement over the simple MST bound is adding the minimum edges bridging the partial tour to the remaining nodes.

$$LB = \text{Cost}(\text{Partial Tour}) + \text{Cost}(\text{MST}(R)) + \min_{u \in R} c(E, u) + \min_{v \in R} c(v, S)$$

This forces the bound to account for entering the set R from E and exiting R back to S , remaining strictly admissible while providing a noticeably tighter estimate.

B. The 1-Tree Bound (Held-Karp Bound) For Symmetric TSP, the most widely taught optimal bound is the 1-tree bound. A 1-tree on a graph with node 1 is defined as a spanning tree on the vertices $V \setminus \{1\}$, plus exactly two edges incident to node 1.

- Every valid TSP tour is a 1-tree (specifically, a 1-tree where every node has degree 2).
- Therefore, the minimum weight 1-tree provides a very tight lower bound for the TSP.
- Combined with Lagrangian relaxation (iteratively adjusting edge weights to penalize nodes with degree $\neq 2$), the Held-Karp bound often equals the exact optimal TSP cost, completely pruning the search tree.

C. Matrix Reduction / Assignment Problem Bound For Asymmetric TSP (where $c(u, v) \neq c(v, u)$), an MST is undefined or problematic since paths are directed. Instead, the standard bound is based on **Matrix Reduction**.

- Subtract the minimum from each row and column. The sum of these reductions yields a lower bound.
- Equivalently, the problem can be relaxed to the **Assignment Problem** (AP), solved efficiently in $\mathcal{O}(n^3)$ using the Hungarian Algorithm. The AP finds a set of disjoint cycles covering all nodes; its cost is a rigorous and very tight lower bound for the ATSP.

Q7. Solve the following instance of a 0-1 knapsack problem by a branch-and-bound algorithm using a state-space tree. You must show the branch-and-bound tree.

Knapsack capacity = 10.

Item	Weight	Value
1	2	10
2	3	9
3	2	20
4	5	30

(15 marks)

[CSE 207 | L-2/T-2 | 2019–20]

Answer**1. Pre-processing: Sorting by Value-to-Weight Ratio**

To apply the Branch-and-Bound strategy effectively, we first calculate the value-to-weight ratio (v_i/w_i) for each item and sort them in **descending order**. This allows our bounding function (which relies on the fractional knapsack greedy choice) to be as tight as possible.

Original Item	Sorted Index	Weight (w_i)	Value (v_i)	Ratio (v_i/w_i)
3	I_1	2	20	10
4	I_2	5	30	6
1	I_3	2	10	5
2	I_4	3	9	3

The knapsack capacity is $W = 10$. We process the items in the order: $I_1 \rightarrow I_2 \rightarrow I_3 \rightarrow I_4$.

2. The Bounding Function

For a maximization problem, we need an **Upper Bound (UB)** to prune suboptimal branches. At any node representing a partial solution, the upper bound is calculated by taking the current value, adding the values of remaining items that fit entirely, and adding a *fraction* of the first item that does not fit (Fractional Knapsack logic). Let max_value represent the maximum integer profit found so far. We prune a node if its $UB \leq max_value$.

3. Branch-and-Bound Execution (Step-by-Step)

- **Node 0 (Root):** Level 0. Current $w = 0, v = 0$.
 - Add I_1 ($w = 2, v = 20$): capacity remaining = 8.
 - Add I_2 ($w = 5, v = 30$): capacity remaining = 3.
 - Add I_3 ($w = 2, v = 10$): capacity remaining = 1.
 - Add fraction of I_4 : $1 \times (9/3) = 3$.
 - $UB_0 = 20 + 30 + 10 + 3 = 63$. Best value known = 0.
- **Node 1 (Include I_1):** Level 1. $w = 2, v = 20$.
 - $UB_1 = 20 + 30 + 10 + (1 \times 3) = 63$. Best value updated to 20.
- **Node 2 (Exclude I_1):** Level 1. $w = 0, v = 0$.
 - Add I_2, I_3, I_4 : weights $5 + 2 + 3 = 10 \leq 10$.
 - $UB_2 = 0 + 30 + 10 + 9 = 49$.
 - Since $UB_1(63) > UB_2(49)$, we explore Node 1 first (Best-First Search).
- **Node 3 (Include I_2 from Node 1):** Level 2. $w = 2 + 5 = 7, v = 20 + 30 = 50$.
 - Add I_3 : rem=1. Add fraction of I_4 : $1 \times 3 = 3$.
 - $UB_3 = 50 + 10 + 3 = 63$. Best value updated to 50.
- **Node 4 (Exclude I_2 from Node 1):** Level 2. $w = 2, v = 20$.
 - Add I_3, I_4 : total weight $= 2 + (2 + 3) = 7 \leq 10$.
 - $UB_4 = 20 + 10 + 9 = 39$.
 - Since $39 \leq$ Best value (50), **PRUNE Node 4**.
- **Node 5 (Include I_3 from Node 3):** Level 3. $w = 7 + 2 = 9, v = 50 + 10 = 60$.
 - Add fraction of I_4 : $1 \times 3 = 3$.
 - $UB_5 = 60 + 3 = 63$. Best value updated to 60.
- **Node 6 (Exclude I_3 from Node 3):** Level 3. $w = 7, v = 50$.
 - Add I_4 : total weight $7 + 3 = 10$.
 - $UB_6 = 50 + 9 = 59$.
 - Since $59 \leq$ Best value (60), **PRUNE Node 6**.
- **Node 7 (Include I_4 from Node 5):** Level 4. $w = 9 + 3 = 12 > 10$.
 - Infeasible. **PRUNE Node 7**.
- **Node 8 (Exclude I_4 from Node 5):** Level 4. $w = 9, v = 60$.
 - Reached a valid leaf. $UB_8 = 60$. Best value remains 60.
- **Final check:** Return to unexplored Node 2. $UB_2 = 49 \leq 60$. **PRUNE Node 2**. Algorithm terminates.

4. State-Space Tree Diagram

5. Final Optimal Solution & Complexity

Based on the leaf node that provided the maximum value without being pruned, the optimal choice encompasses I_1 , I_2 , and I_3 . Translating back to the original items:

- **Included Items:** Original Item 3, Item 4, and Item 1.
- **Total Weight:** $2 + 5 + 2 = 9 \leq 10$
- **Maximum Total Value:** $20 + 30 + 10 = 60$

Complexity Analysis:

- **Time Complexity:** $\mathcal{O}(2^n)$ in the worst case (where n is the number of items), as Branch-and-Bound might theoretically explore all combinations if bounds are loose. However, on average and practically, the upper bounding logic heavily prunes the search space, resulting in substantially faster execution than exhaustive search.
- **Space Complexity:** $\mathcal{O}(n)$ if utilizing Depth-First Search for branch exploration (maintaining the recursion stack/path). If full Best-First Search with a priority queue is used, space could be exponential in worst-case, $\mathcal{O}(2^n)$, though pruned practically.

Q8. Solve the following instance of the 0-1 knapsack problem using a branch-and-bound algorithm with a state-space tree (capacity = 10 kg):

Item	Weight (kg)	Profit (Taka)
1	3	18
2	5	35
3	4	44
4	7	36
5	2	18

(8 marks)

[CSE 207 | L-2/T-2 | 2017–18]

Answer

1. Pre-processing: Sorting by Profit-to-Weight Ratio

The Branch-and-Bound strategy for the 0-1 Knapsack problem is most efficient when we consider items in descending order of their profit-to-weight ratio (p_i/w_i). This allows the fractional knapsack greedy algorithm to compute the tightest possible Upper Bound (UB).

Let us calculate the ratios and reorder the items. The knapsack capacity is $W = 10$.

Original Item	Sorted Item	Weight (w_i)	Profit (p_i)	Ratio (p_i/w_i)	Order
3	I_1	4	44	11.00	1
5	I_2	2	18	9.00	2
2	I_3	5	35	7.00	3
1	I_4	3	18	6.00	4
4	I_5	7	36	5.14	5

2. The Bounding Function

At any node in the state-space tree, let w be the current weight and p be the current profit. The upper bound on the maximum profit achievable from this node is calculated by taking the remaining capacity $W_{rem} = W - w$, adding the profit of all remaining items that can completely fit, and adding a fractional profit of the first item k that does not fit.

Since all profits and weights are integers, the optimal profit must be an integer. Thus, we can take the floor of the calculated bound:

$$UB = \left\lfloor p + \sum_{i=j}^{k-1} p_i + \left(\frac{W_{rem} - \sum_{i=j}^{k-1} w_i}{w_k} \right) \cdot p_k \right\rfloor$$

where j is the index of the next item to be considered. We maintain a **Max Profit** variable (initially 0) and **prune** any node where $UB \leq \text{Max Profit}$.

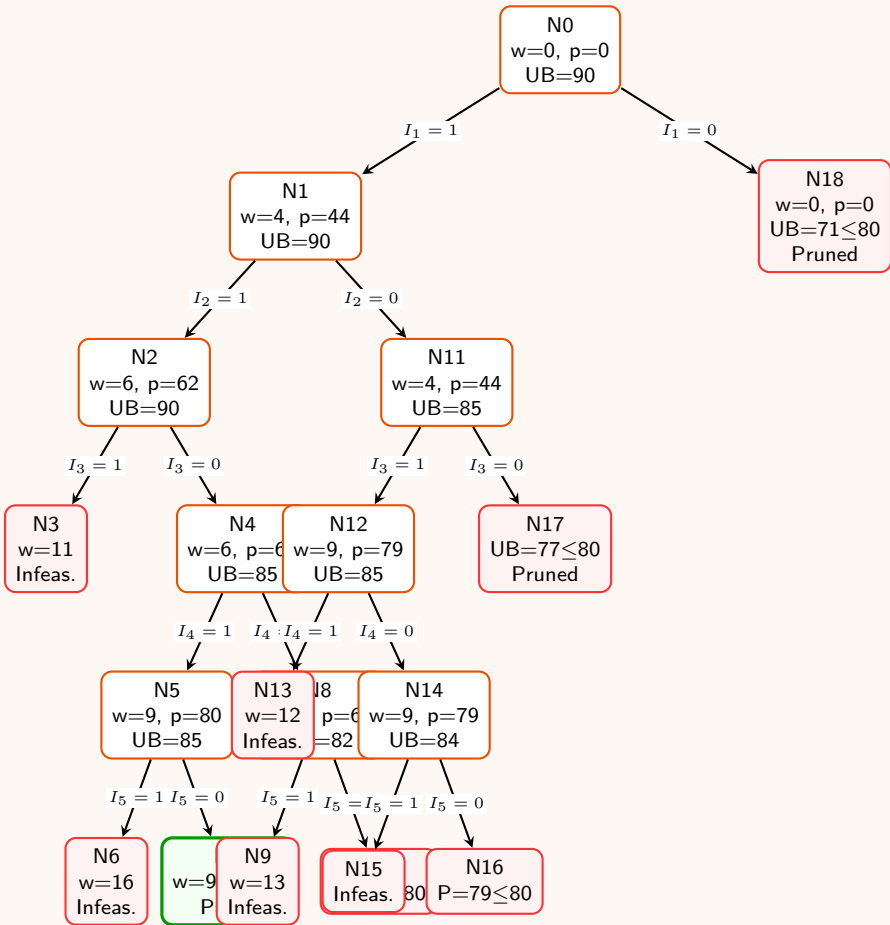
3. Branch-and-Bound Execution (Depth-First Search)

We traverse the state-space tree using Depth-First Search (left child = include item, right child = exclude item), keeping track of the current best complete solution.

- **Node 0 (Root):** $w = 0, p = 0, W_{rem} = 10$.
 - Fill with $I_1(4)$ and $I_2(2)$. $W_{rem} = 4$.
 - Next is $I_3(w = 5, p = 35)$. We take a fraction: $(4/5) \times 35 = 28$.
 - $UB_0 = \lfloor 0 + 44 + 18 + 28 \rfloor = 90$. Max Profit = 0.
- **Node 1 (Include I_1):** $w = 4, p = 44, W_{rem} = 6$.
 - Fill with $I_2(2)$. $W_{rem} = 4$. Fraction of I_3 : $(4/5) \times 35 = 28$.
 - $UB_1 = \lfloor 44 + 18 + 28 \rfloor = 90$.
- **Node 2 (Include I_2 from N1):** $w = 6, p = 62, W_{rem} = 4$.
 - Fraction of I_3 : $UB_2 = \lfloor 62 + 28 \rfloor = 90$.
- **Node 3 (Include I_3 from N2):** $w = 6 + 5 = 11 > 10$. Infeasible. Pruned.
- **Node 4 (Exclude I_3 from N2):** $w = 6, p = 62, W_{rem} = 4$.
 - Fill with $I_4(3)$. $W_{rem} = 1$. Fraction of $I_5(7)$: $(1/7) \times 36 \approx 5.14$.
 - $UB_4 = \lfloor 62 + 18 + 5.14 \rfloor = 85$.
- **Node 5 (Include I_4 from N4):** $w = 9, p = 80, W_{rem} = 1$.
 - Fraction of I_5 : $(1/7) \times 36 \approx 5.14 \implies UB_5 = 85$.
- **Node 6 (Include I_5 from N5):** $w = 16 > 10$. Infeasible. Pruned.
- **Node 7 (Exclude I_5 from N5):** $w = 9, p = 80$. Valid Leaf!
 - **Max Profit updated to 80.**
- **Node 8 (Exclude I_4 from N4):** $w = 6, p = 62, W_{rem} = 4$.
 - Fraction of I_5 : $(4/7) \times 36 \approx 20.57$. $UB_8 = \lfloor 62 + 20.57 \rfloor = 82$.
 - Children: Node 9 (Include $I_5 \rightarrow w = 13$, Infeasible), Node 10 (Exclude $I_5 \rightarrow p = 62 \leq 80$). Both pruned.
- **Node 11 (Exclude I_2 from N1):** $w = 4, p = 44, W_{rem} = 6$.
 - Fill with $I_3(5)$. $W_{rem} = 1$. Fraction of I_4 : $(1/3) \times 18 = 6$.
 - $UB_{11} = 44 + 35 + 6 = 85$.
- **Node 12 (Include I_3 from N11):** $w = 9, p = 79, W_{rem} = 1, UB_{12} = 85$.
 - Node 13 (Inc $I_4 \rightarrow w = 12$, Infeasible).

- Node 14 (Exc $I_4 \rightarrow w = 9, p = 79, UB = 84$). Children are Node 15 (Inc $I_5 \rightarrow w = 16$) and Node 16 (Exc $I_5 \rightarrow p = 79 \leq 80$). Pruned.
- **Remaining Nodes to Backtrack:**
 - **Node 17 (Exclude I_3 from N11):** $w = 4, p = 44, W_{rem} = 6$. Fill $I_4(3)$, rem=3. Frac I_5 : $(3/7) \times 36 \approx 15.42$. $UB_{17} = \lfloor 44 + 18 + 15.42 \rfloor = 77 \leq 80$. **Pruned.**
 - **Node 18 (Exclude I_1 from N0):** $w = 0, p = 0, W_{rem} = 10$. Fill $I_2(2), I_3(5), I_4(3)$, rem=0. $UB_{18} = 18 + 35 + 18 = 71 \leq 80$. **Pruned.**

4. State-Space Tree Diagram



5. Final Result & Complexity Analysis

Following the path that yielded the maximum valid leaf node (Node 7), we trace the item inclusions: $I_1 = 1, I_2 = 1, I_3 = 0, I_4 = 1, I_5 = 0$. Mapping these back to the original items:

- **Included Items:** Original Item 3 (4 kg, 44 Tk), Original Item 5 (2 kg, 18 Tk), Original Item 1 (3 kg, 18 Tk).
- **Total Weight:** $4 + 2 + 3 = 9 \text{ kg} \leq 10 \text{ kg}$
- **Maximum Profit:** $44 + 18 + 18 = 80$ Taka

Complexity:

- **Time Complexity:** $\mathcal{O}(2^n)$ in the worst-case scenario where bounds are weak and all subsets must be generated. However, due to highly efficient upper-bound pruning, average-case runtime is significantly faster than exhaustive search. Sorting initially takes $\mathcal{O}(n \log n)$.
- **Space Complexity:** $\mathcal{O}(n)$ utilizing Depth-First Search exploration, as we only need to maintain the recursive call stack proportional to the maximum height of the tree (number of items).

Q9. Solve the following 0-1 knapsack instance (capacity = 10) using the branch-and-bound algorithm. Show the branch-and-bound tree.

Item	Weight	Value
1	2	10
2	3	9
3	2	20
4	5	30

Answer

1. Pre-processing: Item Sorting by Profit-to-Weight Ratio

To solve the 0-1 Knapsack problem efficiently using the Branch-and-Bound (B&B) technique, we first sort the items in descending order of their value-to-weight ratio ($\frac{v_i}{w_i}$). This reordering provides a tighter upper bound at each node by maximizing the efficiency of the greedy fractional relaxation choice.

The given knapsack capacity is $W = 10$. Let us compute the ratios and define our processing order:

Original Item	Sorted ID	Weight (w_i)	Value (v_i)	Ratio ($\frac{v_i}{w_i}$)	Processing Order
3	I_1	2	20	10.0	1
4	I_2	5	30	6.0	2
1	I_3	2	10	5.0	3
2	I_4	3	9	3.0	4

We will formulate the state-space tree by making binary decisions on items in the order: $I_1 \rightarrow I_2 \rightarrow I_3 \rightarrow I_4$.

2. Bounding Function Definition

For a maximization problem solved via Branch-and-Bound, each node requires an **Upper Bound (UB)** to prune suboptimal search paths. The bound is calculated using the greedy strategy of the Fractional Knapsack problem:

$$UB = V_{\text{curr}} + \sum_{i \in \text{Full}} v_i + \left(\frac{W - W_{\text{curr}} - \sum_{i \in \text{Full}} w_i}{w_{\text{fraction}}} \right) \times v_{\text{fraction}}$$

where V_{curr} and W_{curr} represent the accumulated value and weight of the items fixed along the path to the current node.

We maintain a global variable *Max_Profit* representing the best feasible solution found so far (initialized to 0). A node is active and explored if its $UB > \text{Max_Profit}$; otherwise, it is **pruned**.

3. Step-by-Step Branch-and-Bound Execution

We explore the state-space tree using a Best-First Search strategy determined by the maximum upper bound.

(a) **Node 0 (Root Node):** No decisions made yet. $W_{\text{curr}} = 0, V_{\text{curr}} = 0$.

- Greedily fill remaining capacity (10): Add I_1 ($w = 2$), I_2 ($w = 5$), I_3 ($w = 2$). Remaining capacity = $10 - (2 + 5 + 2) = 1$.
- Take a fraction of I_4 : $\frac{1}{3} \times 9 = 3$.
- $UB_0 = 20 + 30 + 10 + 3 = 63$. $\text{Max_Profit} = 0$.

(b) **Branching from Node 0 on I_1 :**

- **Node 1 (Include I_1):** $W_{\text{curr}} = 2, V_{\text{curr}} = 20$.
Remaining capacity = 8. Greedily adding I_2, I_3 leaves capacity 1. Add fraction of I_4 : $\frac{1}{3} \times 9 = 3$.
 $UB_1 = 20 + 30 + 10 + 3 = 63$.
- **Node 2 (Exclude I_1):** $W_{\text{curr}} = 0, V_{\text{curr}} = 0$.
Remaining capacity = 10. Greedily add I_2, I_3, I_4 completely ($5 + 2 + 3 = 10$).
 $UB_2 = 30 + 10 + 9 = 49$.

(c) **Branching from Node 1 on I_2 (Highest UB = 63):**

- **Node 3 (Include I_2):** $W_{\text{curr}} = 2 + 5 = 7, V_{\text{curr}} = 20 + 30 = 50$.
Remaining capacity = 3. Add I_3 completely ($w = 2$). Remaining capacity = 1. Add fraction of I_4 : $\frac{1}{3} \times 9 = 3$.
 $UB_3 = 50 + 10 + 3 = 63$.
- **Node 4 (Exclude I_2):** $W_{\text{curr}} = 2, V_{\text{curr}} = 20$.
Remaining capacity = 8. Add I_3, I_4 completely ($2 + 3 = 5 \leq 8$).
 $UB_4 = 20 + 10 + 9 = 39$.

(d) **Branching from Node 3 on I_3 (Highest UB = 63):**

- **Node 5 (Include I_3):** $W_{\text{curr}} = 7 + 2 = 9, V_{\text{curr}} = 50 + 10 = 60$.
Remaining capacity = 1. Add fraction of I_4 : $\frac{1}{3} \times 9 = 3$.
 $UB_5 = 60 + 3 = 63$.
- **Node 6 (Exclude I_3):** $W_{\text{curr}} = 7, V_{\text{curr}} = 50$.
Remaining capacity = 3. Add I_4 completely ($w = 3$). Remaining capacity = 0.
 $UB_6 = 50 + 9 = 59$.

(e) **Branching from Node 5 on I_4 (Highest UB = 63):**

- **Node 7 (Include I_4):** $W_{\text{curr}} = 9 + 3 = 12$.
Since $W_{\text{curr}} = 12 > W = 10$, this node is **Infeasible** and immediately pruned.

- **Node 8 (Exclude I_4):** $W_{\text{curr}} = 9, V_{\text{curr}} = 60$.
This is a valid, fully completed leaf node.
Update global bounds: $Max_Profit = \max(0, 60) = 60$.

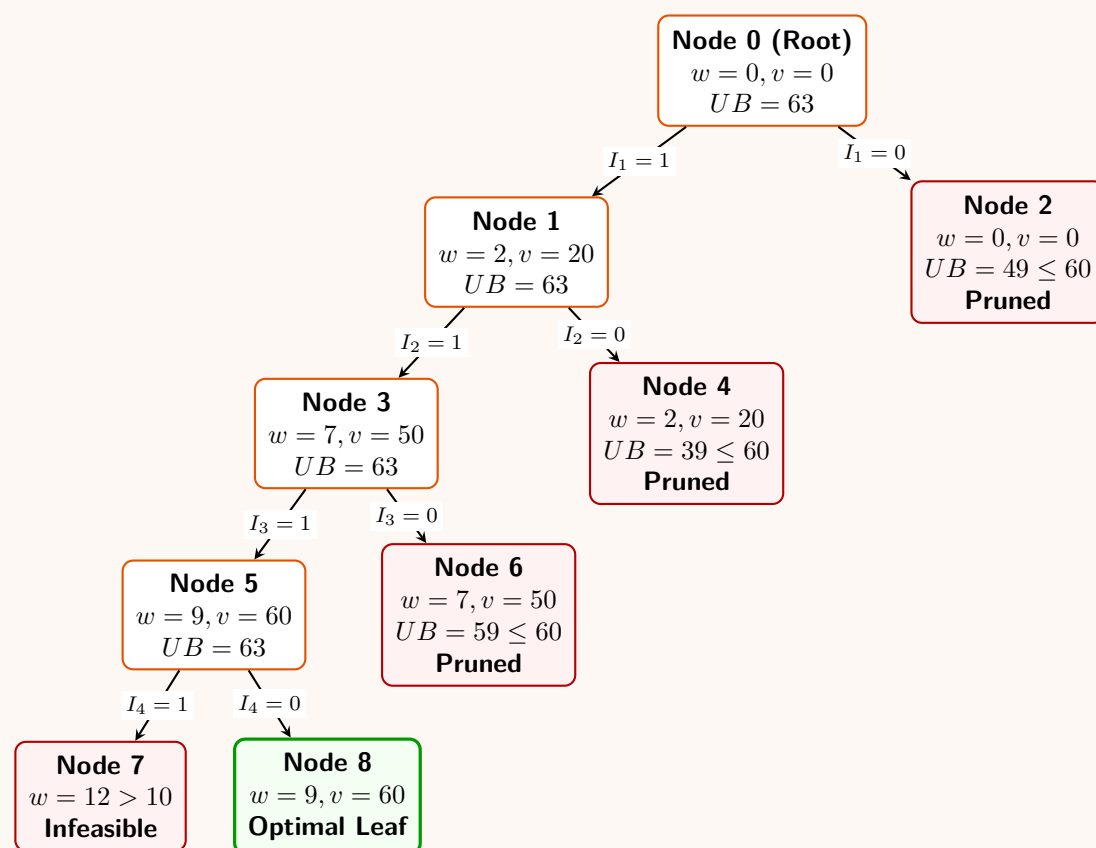
(f) **Pruning Remaining Subtrees:** The priority queue/active set contains Node 6 ($UB = 59$), Node 2 ($UB = 49$), and Node 4 ($UB = 39$).

- For Node 6: $UB_6 = 59 \leq Max_Profit (60) \implies$ **Prune**.
- For Node 2: $UB_2 = 49 \leq Max_Profit (60) \implies$ **Prune**.
- For Node 4: $UB_4 = 39 \leq Max_Profit (60) \implies$ **Prune**.

All active branches have been exhausted or pruned. The algorithm terminates.

4. State-Space Tree Diagram

Below is the complete branch-and-bound state-space tree layout. Green highlights the optimal solution path, while red nodes are pruned.



5. Final Solution Summary

Tracing the path to the optimal leaf node (**Node 8**), the item selection matrix is:

- $I_1 = 1$ (Include original Item 3)
- $I_2 = 1$ (Include original Item 4)
- $I_3 = 1$ (Include original Item 1)
- $I_4 = 0$ (Exclude original Item 2)

Optimal Composition:

- Selected Items: **{Item 1, Item 3, Item 4}**
- Total Weight calculation: $2 + 2 + 5 = 9 \leq 10$
- Maximum Value achieved: $10 + 20 + 30 = 60$

Complexity Analysis:

- **Time Complexity:** $\mathcal{O}(2^n)$ in the worst case, where n is the number of items. This occurs if the bounds fail to prune any subtrees and the algorithm must explore all subsets. However, with an optimized bounding rule (like fractional knapsack), the average-case execution explores significantly fewer states. Sorting the items initially requires $\mathcal{O}(n \log n)$ time.
- **Space Complexity:** $\mathcal{O}(2^n)$ in the worst case for a Best-First Search strategy because the priority queue may retain entries for an exponential number of open nodes. (If implemented via Depth-First Search with bounding, the space complexity decreases to $\mathcal{O}(n)$ due to the call stack height limit).

Q10. Solve the following 0/1 knapsack instance (capacity = 15) using the branch-and-bound approach with a state-space tree:

Item	Weight	Value
1	7	140
2	6	132
3	4	140
4	3	90

Compare the backtracking and branch-and-bound techniques. **(3+12=15 marks)**

[CSE 207 | L-2/T-2 | 2015–16]

Answer

1. Pre-processing: Item Sorting by Profit-to-Weight Ratio

To solve the 0-1 Knapsack problem efficiently using the Branch-and-Bound (B&B) algorithm, we sort the given items in descending order of their value-to-weight ratio ($\frac{v_i}{w_i}$). This maximizes the efficiency of the greedy selection strategy when computing the upper bound of any node.

The knapsack capacity is given as $W = 15$. Let us compute the individual ratios:

Original ID	Sorted ID	Weight (w_i)	Value (v_i)	Ratio ($\frac{v_i}{w_i}$)	Rank Order
3	I_1	4	140	35.0	1
4	I_2	3	90	30.0	2
2	I_3	6	132	22.0	3
1	I_4	7	140	20.0	4

We will construct the state-space tree by making binary branching decisions (1 = include, 0 = exclude) sequentially on items $I_1 \rightarrow I_2 \rightarrow I_3 \rightarrow I_4$.

2. Bounding Function Formulation

For each node, we compute a relaxed upper bound (UB) using the Fractional Knapsack greedy approach. If a node's UB is less than or equal to the current maximum profit (Max_Profit) found globally among feasible solutions, the node is pruned.

Let w be the current weight, and v be the current value accumulated at a node. The bound formula is defined as:

$$UB = v + \sum_{i \in \text{Full}} v_i + \left(\frac{W - w - \sum_{i \in \text{Full}} w_i}{w_{\text{fraction}}} \right) \times v_{\text{fraction}}$$

We maintain a global variable Max_Profit (initially 0).

3. Step-by-Step Branch-and-Bound Execution

We explore the tree using Best-First Search via a priority queue ordered by maximum UB :

(a) **Node 0 (Root Node):** $w = 0, v = 0$.

- Greedily fill remaining capacity (15): Take I_1 ($w = 4$) and I_2 ($w = 3$). Remaining capacity = $15 - 7 = 8$.
- Take I_3 ($w = 6$). Remaining capacity = $8 - 6 = 2$.
- Take fraction of I_4 : $\frac{2}{7} \times 140 = 40$.
- $UB_0 = 140 + 90 + 132 + 40 = 402$. $Max_Profit = 0$.

(b) **Branching Node 0 (I_1):**

- **Node 1 ($I_1 = 1$):** $w = 4, v = 140$. Remaining capacity = 11. Greedily adding I_2, I_3 leaves capacity 2. Add fraction of $I_4 \rightarrow \frac{2}{7} \times 140 = 40$. $UB_1 = 140 + 90 + 132 + 40 = 402$.
- **Node 2 ($I_1 = 0$):** $w = 0, v = 0$. Remaining capacity = 15. Greedily adding $I_2(3), I_3(6)$ leaves capacity 6. Add fraction of $I_4 \rightarrow \frac{6}{7} \times 140 = 120$. $UB_2 = 90 + 132 + 120 = 342$.

(c) **Branching Node 1 (I_2) [Highest $UB = 402$]:**

- **Node 3 ($I_2 = 1$):** $w = 4 + 3 = 7, v = 140 + 90 = 230$. Capacity remaining = 8. Add $I_3(6)$, fraction of $I_4(2)$. $UB_3 = 230 + 132 + 40 = 402$.
- **Node 4 ($I_2 = 0$):** $w = 4, v = 140$. Capacity remaining = 11. Add $I_3(6)$, fraction of $I_4(5)$. $UB_4 = 140 + 132 + (\frac{5}{7} \times 140) = 372$.

(d) **Branching Node 3 (I_3) [Highest $UB = 402$]:**

- **Node 5 ($I_3 = 1$):** $w = 7 + 6 = 13, v = 230 + 132 = 362$. Capacity remaining = 2. Add fraction of $I_4 \rightarrow \frac{2}{7} \times 140 = 40$. $UB_5 = 362 + 40 = 402$.
- **Node 6 ($I_3 = 0$):** $w = 7, v = 230$. Capacity remaining = 8. Add full $I_4(7)$, remaining capacity = 1. $UB_6 = 230 + 140 = 370$.

(e) **Branching Node 5 (I_4) [Highest $UB = 402$]:**

- **Node 7** ($I_4 = 1$): $w = 13 + 7 = 20 > 15 \implies$ **Infeasible** (Pruned).
- **Node 8** ($I_4 = 0$): $w = 13, v = 362$. Leaf reached! Update $Max_Profit = \max(0, 362) = 362$.

(f) **Branching Node 4** (I_3) [**Highest remaining** $UB = 372 > 362$]:

- **Node 9** ($I_3 = 1$): $w = 4 + 6 = 10, v = 140 + 132 = 272$. Remaining capacity = 5. Add fraction of $I_4 \rightarrow \frac{5}{7} \times 140 = 100$. $UB_9 = 272 + 100 = 372$.
- **Node 10** ($I_3 = 0$): $w = 4, v = 140$. Remaining capacity = 11. Add full $I_4(7)$, remaining capacity = 4. $UB_{10} = 140 + 140 = 280$. Since $280 \leq 362$, **Pruned**.

(g) **Branching Node 9** (I_4) [**Highest remaining** $UB = 372 > 362$]:

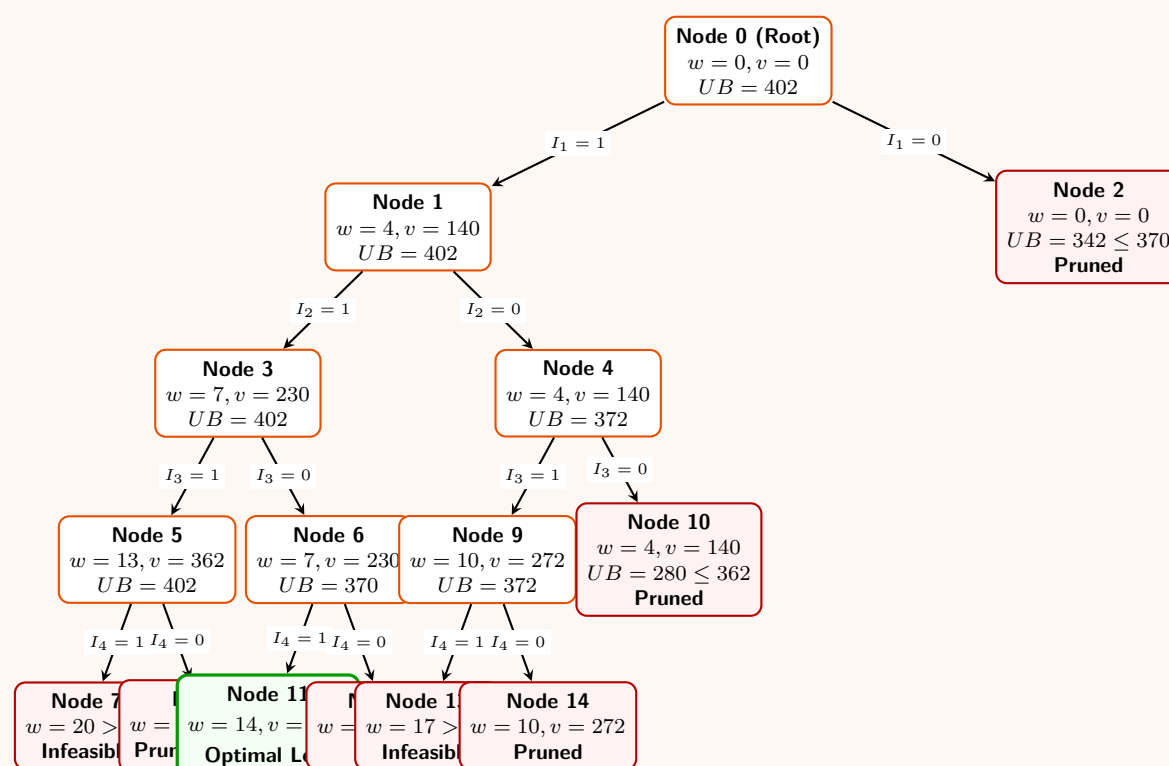
- **Node 13** ($I_4 = 1$): $w = 10 + 7 = 17 > 15 \implies$ **Infeasible** (Pruned).
- **Node 14** ($I_4 = 0$): $w = 10, v = 272$. Leaf reached! $272 \leq 362 \implies$ **Pruned**.

(h) **Branching Node 6** (I_4) [**Highest remaining** $UB = 370 > 362$]:

- **Node 11** ($I_4 = 1$): $w = 7 + 7 = 14, v = 230 + 140 = 370$. Leaf reached! Update $Max_Profit = \max(362, 370) = 370$.
- **Node 12** ($I_4 = 0$): $w = 7, v = 230$. Leaf reached! $230 \leq 370 \implies$ **Pruned**.

All other remaining open branches (e.g., Node 2 with $UB = 342$) have $UB \leq 370$ and are explicitly pruned.

4. State-Space Tree Diagram



5. Final Solution Vector & Complexity Analysis

Tracing the path leading to the absolute optimal node (**Node 11**):

- $I_1 = 1, I_2 = 1, I_3 = 0, I_4 = 1$

Mapping these back to the original items:

- **Selected Items:** Original Item 3 (I_1), Original Item 4 (I_2), and Original Item 1 (I_4). (Original Item 2 is excluded).
- **Optimal Total Weight:** $4 + 3 + 7 = 14 \leq 15$
- **Optimal Maximum Value:** $140 + 90 + 140 = 370$

Complexity Analysis:

- **Time Complexity:** $\mathcal{O}(2^n)$ in the worst case (where n is the number of items), as the algorithm might expand all paths if bounds fail to restrict branching. However, the average case requires exploring far fewer states due to bounding prunes. Presorting takes $\mathcal{O}(n \log n)$.
- **Space Complexity:** $\mathcal{O}(2^n)$ for Best-First Search because the priority queue can simultaneously store multiple open configurations at the fringe.

6. Comparison: Backtracking vs. Branch-and-Bound

Both methodologies target hard combinatorial problems by searching state-space graphs, but they differ fundamentally in design strategies:

Feature	Backtracking	Branch-and-Bound
Search Strategy	Primarily Depth-First Search (DFS).	Breadth-First Search (BFS) or Best-First Search using a priority queue.
Problem Applicability	Best suited for decision problems, constraint satisfaction, or finding all solutions (e.g., N -Queens).	Expressly optimized for discrete optimization problems (maximizing or minimizing objective equations).
Pruning Mechanism	Prunes paths using a bounding/feasibility function as soon as constraints are violated.	Employs a continuous mathematical estimation function (UB or LB) to check if a path can beat the current best global score.
Memory Footprint	Very light memory overhead, bounded linearly by tree depth: $\mathcal{O}(n)$.	Can have heavy memory consumption: $\mathcal{O}(2^n)$ due to keeping active exploration frontiers inside queues.
Traversal Direction	Explores deep down a lineage before exploring siblings.	Evaluates nodes dynamically across levels based on maximum statistical promise.

Q11. Consider the following instance of the maximum satisfiability (MAX-SAT) problem.

Set of Boolean variables, $V = \{w, x, y, z\}$
Set of clauses, $C = \{C1, C2, C3, C4, C5, C6\}$
where $C1 = (w \vee \bar{x} \vee \bar{y} \vee z)$, $C2 = (w \vee \bar{y})$, $C3 = (x \vee \bar{y})$, $C4 = (y \vee \bar{z})$
 $C5 = (z \vee \bar{w})$, $C6 = (\bar{w} \vee \bar{x})$

Now find an assignment of variables such that maximum number of clauses is satisfied. Compute your solution using branch and bound algorithm. Write down a suitable bound function for the problem. Show detailed calculation steps through a branch and bound search tree.**(12 marks)** [CSE 207 | L-2/T-2 | 2014–15]

Answer

1. Problem Formulation & Bounding Function

The Maximum Satisfiability (MAX-SAT) problem asks for an assignment of Boolean variables that maximizes the number of satisfied clauses.

- Variables:** $V = \{w, x, y, z\}$
- Total Clauses:** $m = 6$
- Clauses:** $C_1 = (w \vee \bar{x} \vee \bar{y} \vee z)$, $C_2 = (w \vee \bar{y})$, $C_3 = (x \vee \bar{y})$, $C_4 = (y \vee \bar{z})$, $C_5 = (z \vee \bar{w})$, $C_6 = (\bar{w} \vee \bar{x})$

To apply the Branch and Bound algorithm efficiently, we need a bounding function. Since this is a maximization problem, we define an **Upper Bound (UB)** for any partial assignment. Let F be the number of clauses that evaluate to strictly **False** (i.e., all literals in the clause are evaluated to 0) under the current partial assignment. A falsified clause can never become true. Therefore, the maximum possible number of satisfied clauses achievable from this node is:

$$UB = m - F = 6 - F$$

We maintain a global variable **Best**, representing the maximum number of satisfied clauses found so far (initialized to 0). If at any node, $UB \leq \mathbf{Best}$, we **prune** the branch because it cannot strictly improve our best known solution.

2. Branch and Bound Algorithm

Algorithm 90 Branch and Bound DFS for MAX-SAT

```

1: Input: Set of Clauses  $C$ , Variables  $V$ 
2: Global:  $\text{Best} \leftarrow 0$ ,  $m \leftarrow |C|$ 
3: Procedure SOLVE(current_assignment, unassigned_vars)
4:  $F \leftarrow$  number of falsified clauses in  $C$  under current_assignment
5:  $UB \leftarrow m - F$ 
6: if  $UB \leq \text{Best}$  then
7:   return ▷ Prune branch
8: end if
9: if unassigned_vars is empty then
10:    $\text{Best} \leftarrow \max(\text{Best}, UB)$  ▷ Update optimal score
11:   return
12: end if
13:  $v \leftarrow$  select next variable from unassigned_vars
14: SOLVE(current_assignment  $\cup \{v = 1\}$ , unassigned_vars  $\setminus \{v\}$ )
15: SOLVE(current_assignment  $\cup \{v = 0\}$ , unassigned_vars  $\setminus \{v\}$ )

```

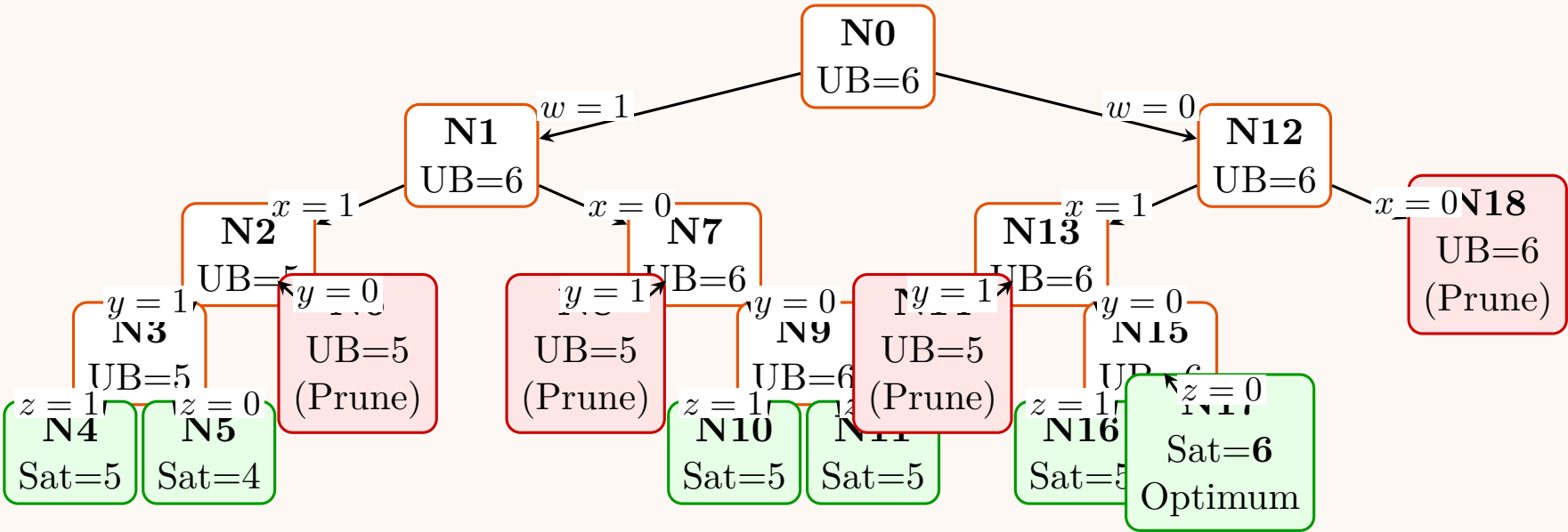
3. Step-by-Step Execution Trace

We explore the variable search space in a Depth-First Search manner, considering variables in lexicographical order: w, x, y, z .

- **N0 (Root):** All unassigned. $F = 0 \implies UB = 6$.
 - **N1 ($w = 1$):** $F = 0 \implies UB = 6$.
 - **N2 ($w = 1, x = 1$):** $C_6 = (0 \vee 0) = \text{False}$. $F = 1 \implies UB = 5$.
 - * **N3 ($y = 1$):** $F = 1 \implies UB = 5$.
 - **N4 ($z = 1$):** Leaf. C_6 False. Sat = 5. **Best** $\leftarrow 5$.
 - **N5 ($z = 0$):** Leaf. $C_5 = (0 \vee 0) = \text{False}$, C_6 False. Sat = 4.
 - * **N6 ($y = 0$):** $F = 1 \implies UB = 5$. Since $UB \leq \text{Best}$ ($5 \leq 5$), **Prune**.
 - **N7 ($w = 1, x = 0$):** $F = 0 \implies UB = 6$.
 - * **N8 ($y = 1$):** $C_3 = (0 \vee 0) = \text{False}$. $F = 1 \implies UB = 5 \leq \text{Best}$. **Prune**.
 - * **N9 ($y = 0$):** $F = 0 \implies UB = 6$.
 - **N10 ($z = 1$):** Leaf. $C_4 = (0 \vee 0) = \text{False}$. Sat = 5.
 - **N11 ($z = 0$):** Leaf. $C_5 = (0 \vee 0) = \text{False}$. Sat = 5.
 - **N12 ($w = 0$):** $F = 0 \implies UB = 6$.
 - **N13 ($w = 0, x = 1$):** $F = 0 \implies UB = 6$.
 - * **N14 ($y = 1$):** $C_2 = (0 \vee 0) = \text{False}$. $F = 1 \implies UB = 5 \leq \text{Best}$. **Prune**.
 - * **N15 ($y = 0$):** $F = 0 \implies UB = 6$.
 - **N16 ($z = 1$):** Leaf. $C_4 = (0 \vee 0) = \text{False}$. Sat = 5.
 - **N17 ($z = 0$):** Leaf. All clauses satisfied! Sat = 6. **Best** $\leftarrow 6$.
 - **N18 ($w = 0, x = 0$):** $F = 0 \implies UB = 6$. Since $UB \leq \text{Best}$ ($6 \leq 6$), **Prune**. (We have reached the maximum possible theoretical score).
-

4. State-Space Search Tree

The diagram below tracks the tree generated by the execution trace. Nodes in **green** are fully evaluated valid leaves, and nodes in **red** are bounded and pruned without further expansion.



5. Final Result & Complexity Analysis

Optimal Assignment: From Leaf N17, the highest recorded score reaches the theoretical maximum of 6. The assignment corresponding to this node is:

$w = 0, \quad x = 1, \quad y = 0, \quad z = 0$

Under this assignment, all 6 clauses are strictly satisfied.

Complexity Assessment:

- Time Complexity:** In the worst-case scenario (where bounds fail to prune any states), the Branch and Bound explores $\mathcal{O}(2^n)$ nodes. At each node, computing the bound costs $\mathcal{O}(m)$, giving an overall worst-case time complexity of $\mathcal{O}(m \cdot 2^n)$, where n is the number of variables and m is the number of clauses.
- Space Complexity:** Since we use a Depth-First Search approach, the space complexity is strictly limited by the height of the state-space tree. Thus, the space complexity is highly efficient at $\mathcal{O}(n)$.

Q12. What is a state-space tree? Solve the following instance of the 0/1 knapsack problem (capacity = 10) using the branch-and-bound approach with a state-space tree:

Item	Weight	Value
1	7	42
2	4	40
3	3	12
4	5	25

(15 marks)

[CSE 207 | L-2/T-2 | 2013–14]

Answer

1. Concept: State-Space Tree

A **state-space tree** is a rooted tree data structure that represents the search space of a combinatorial optimization or decision problem.

- Nodes** represent the states (either partial or complete assignments) of the problem.
- Edges** represent the branching decisions made to transition from one state to another (e.g., including or excluding a specific item in the knapsack).
- The **Root** represents the initial state (empty assignment), and the **Leaves** represent complete assignments or terminal configurations.
- Algorithms like Branch-and-Bound and Backtracking systematically traverse this tree, computing mathematical bounds or feasibility constraints at each node to prune non-promising subtrees and avoid exhaustive enumeration.

2. Pre-processing: Item Sorting

To solve the 0/1 Knapsack problem efficiently using Branch-and-Bound, we first evaluate the items and sort them in descending order of their **value-to-weight ratio** ($\frac{v_i}{w_i}$). This ensures the greedy fractional upper bound is as tight and realistic as possible. Given Knapsack Capacity $W = 10$:

Original ID	Sorted ID	Weight (w_i)	Value (v_i)	Ratio (v_i/w_i)	Rank Order
2	I_1	4	40	10.0	1
1	I_2	7	42	6.0	2
4	I_3	5	25	5.0	3
3	I_4	3	12	4.0	4

3. Bounding Function

At any node with accumulated weight w and value v , the optimistic Upper Bound (UB) is calculated by filling the remaining capacity ($W - w$) greedily. We take whole items as long as they fit, and then a *fraction* of the next available item.

$$UB = v + \sum_{i \in \text{Full}} v_i + \left(\frac{W - w - \sum w_{\text{Full}}}{w_{\text{next}}} \right) \times v_{\text{next}}$$

We maintain a global Max_Profit initialized to 0. A node is pruned if $UB \leq Max_Profit$ or if its weight $w > W$.

4. Step-by-Step Branch-and-Bound Execution

We process the tree by branching on variables I_1, I_2, I_3, I_4 in sequence. We use Best-First Search, prioritizing nodes with the highest UB .

(a) **Node 0 (Root):** $w = 0, v = 0$.

- Add $I_1(4)$. Rem: 6. Next is $I_2(7, 42)$. Add fraction: $\frac{6}{7} \times 42 = 36$.
- $UB_0 = 40 + 36 = \mathbf{76}$.

(b) **Branching Node 0 (I_1):**

- **Node 1 ($I_1 = 1$):** $w = 4, v = 40$. Rem: 6. Add fraction of $I_2 \implies \frac{6}{7} \times 42 = 36$. $UB_1 = 40 + 36 = \mathbf{76}$.
- **Node 2 ($I_1 = 0$):** $w = 0, v = 0$. Rem: 10. Add $I_2(7, 42)$. Rem: 3. Add fraction of $I_3 \implies \frac{3}{5} \times 25 = 15$. $UB_2 = 42 + 15 = \mathbf{57}$.

(c) **Branching Node 1 (I_2) [Highest open $UB = 76$]:**

- **Node 3 ($I_2 = 1$):** $w = 4 + 7 = 11 > 10$. **Infeasible** (Pruned).
- **Node 4 ($I_2 = 0$):** $w = 4, v = 40$. Rem: 6. Add $I_3(5, 25)$. Rem: 1. Add fraction of $I_4 \implies \frac{1}{3} \times 12 = 4$. $UB_4 = 40 + 25 + 4 = \mathbf{69}$.

(d) **Branching Node 4 (I_3) [Highest open $UB = 69$]:**

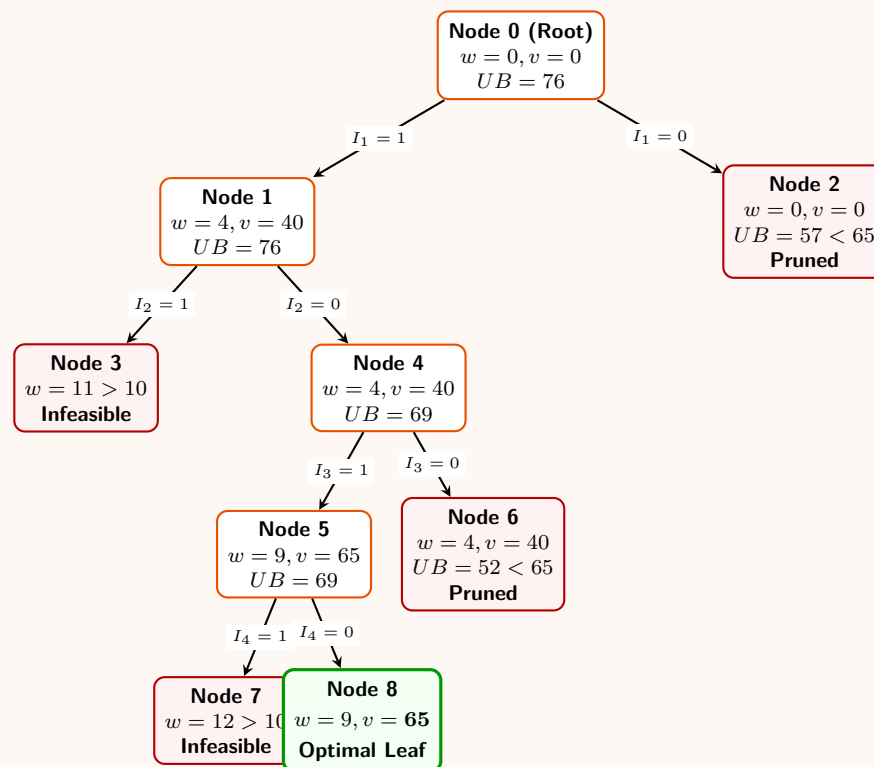
- **Node 5 ($I_3 = 1$):** $w = 4 + 5 = 9, v = 40 + 25 = 65$. Rem: 1. Add fraction of $I_4 \implies \frac{1}{3} \times 12 = 4$. $UB_5 = 65 + 4 = \mathbf{69}$.
- **Node 6 ($I_3 = 0$):** $w = 4, v = 40$. Rem: 6. Add $I_4(3, 12)$. Rem: 3. No more items. $UB_6 = 40 + 12 = \mathbf{52}$.

(e) **Branching Node 5 (I_4) [Highest open $UB = 69$]:**

- **Node 7 ($I_4 = 1$):** $w = 9 + 3 = 12 > 10$. **Infeasible** (Pruned).
- **Node 8 ($I_4 = 0$):** $w = 9, v = 65$. Leaf reached! We update $Max_Profit = \max(0, 65) = \mathbf{65}$.

Remaining active nodes are Node 6 ($UB = 52$) and Node 2 ($UB = 57$). Since both have $UB < 65$, they are safely **pruned** without further expansion.

5. State-Space Tree Diagram



6. Final Solution Vector & Complexity

Tracing the path leading to the optimal leaf (**Node 8**):

- $I_1 = 1, I_2 = 0, I_3 = 1, I_4 = 0$

Mapping these back to the original items:

- **Selected Items:** Original Item 2 (I_1) and Original Item 4 (I_3).
- **Optimal Total Weight:** $4 + 5 = 9 \leq 10$
- **Optimal Maximum Value:** $40 + 25 = 65$

Complexity Analysis:

- **Time Complexity:** $\mathcal{O}(2^n)$ in the worst case (where bounding fails to prune the search space). However, due to tight bounding via the greedy fractional technique, it performs significantly better on average. Pre-processing (sorting) takes $\mathcal{O}(n \log n)$.
- **Space Complexity:** $\mathcal{O}(2^n)$ for the Best-First Search because the priority queue may have to store an exponential number of active nodes at the fringe.

Q13. Applying the FIFO branch-and-bound technique on the 4-Queen problem, it is found that the technique is not efficient. Why? (6 marks) [CSE 207 | L-2/T-2 | 2008–09]

Answer

1. Introduction to the Approach

The 4-Queen problem requires placing 4 queens on a 4×4 chessboard such that no two queens attack each other (i.e., no two queens share the same row, column, or diagonal). When applying a **Branch-and-Bound** technique to solve this, a state-space tree is constructed where each level i represents the placement of a queen in the i -th row.

A **FIFO (First-In, First-Out)** branch-and-bound operates identically to a **Breadth-First Search (BFS)**. It uses a queue to explore the state-space tree level by level. It fully expands all nodes at depth d before moving to depth $d + 1$.

2. Why FIFO Branch-and-Bound is Inefficient

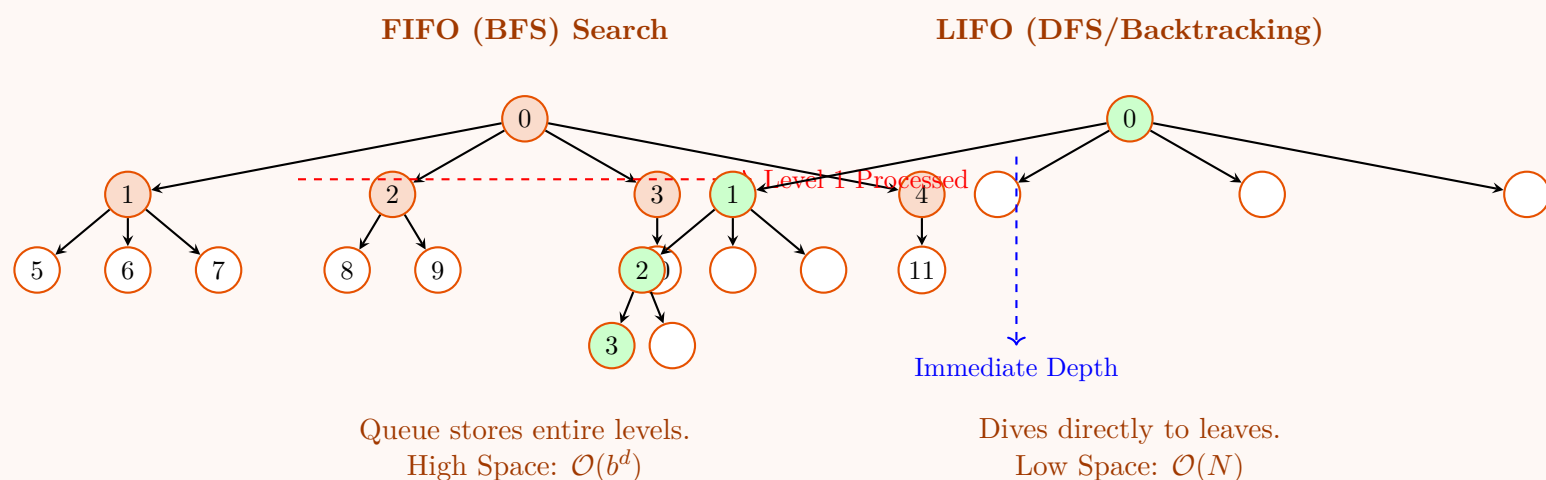
Applying the FIFO strategy to the 4-Queen problem (and N -Queen problems in general) is highly inefficient due to several core structural reasons:

- Exponential Space Complexity (Queue Size):** In FIFO search, all unpruned nodes at the current level must be stored in memory (the queue) before generating the next level. Because the state-space tree branches heavily at the top, the number of valid partial configurations grows rapidly. Instead of storing just $\mathcal{O}(N)$ nodes (as in Depth-First Search), FIFO must store $\mathcal{O}(b^d)$ nodes in the worst case, leading to massive memory overhead even for small N .
- Solutions Reside Exclusively at Maximum Depth:** The 4-Queen problem is a *Constraint Satisfaction Problem (CSP)*, not a typical optimization problem. A valid solution requires all 4 queens to be placed, meaning all valid goal states exist strictly at the **leaf nodes** (depth $N = 4$). Because FIFO explores level-by-level, it spends maximum time generating and enqueueing partial states at depths 1, 2, and 3 before it ever evaluates a single complete leaf node at depth 4.

- (c) **Ineffective "Bounding" (No Cost Heuristic):** In traditional branch-and-bound optimization (like the Knapsack problem), bounding functions use an objective cost/profit to prune suboptimal paths early. In the 4-Queen problem, the "bound" is merely a feasibility check (do the current queens attack each other?). There is no notion of "cost," meaning FIFO cannot prioritize promising branches (Best-First) or aggressively prune valid-but-suboptimal paths. Every feasible partial state must be strictly queued and explored.
- (d) **Delayed Discovery of First Solution:** If the goal is to find *any* single valid arrangement of the 4 queens, FIFO is uniquely disadvantageous. A Depth-First approach (LIFO/Backtracking) immediately dives to depth 4, frequently finding a valid assignment in a fraction of the time. FIFO forces the systematic expansion of the entire upper tree first.

3. Visualizing the FIFO State-Space Explosion

The diagram below contrasts how FIFO (BFS) processes the state-space tree compared to LIFO (DFS/Backtracking). Notice how FIFO forces horizontal expansion, causing the queue to swell.



4. Summary and Complexity Comparison

- **FIFO (Breadth-First Search):** The space complexity is proportional to the maximum width of the state space tree, which is $\mathcal{O}(N^N)$ asymptotically before pruning. For memory-constrained environments, the queue will overflow quickly even for moderate N , making it strictly inefficient.
- **LIFO (Depth-First / Backtracking):** Only the current path from the root to the working node is stored. The space complexity is $\mathcal{O}(N)$, representing the maximum depth of the recursive stack. It is the universally preferred standard for solving the N -Queen problem.

BUET · CSE 207 · Data Structures & Algorithms II

Part VI

String Matching Algorithms & Randomized Algorithms

Knuth-Morris-Pratt (KMP) Algorithm, Monte Carlo Algorithms & Probability Amplification, Las Vegas Algorithms & Randomized Quick Sort

S6 — String Matching Algorithms

Knuth-Morris-Pratt (KMP) Algorithm

Q1. Construct the prefix function using the Knuth-Morris-Pratt (KMP) algorithm for the pattern ABABC. Explain how you can use this prefix function to search for this pattern in the text ABABABABC. (15 marks) [CSE 207 | L-2/T-1 | 2023–24]

Answer

The Knuth-Morris-Pratt (KMP) algorithm is a highly efficient string-matching algorithm that achieves linear time complexity. It avoids the redundant comparisons found in the naive approach by preprocessing the pattern to construct a **Prefix Function** (often called the π array or LPS array).

1. Constructing the Prefix Function (π array)

Definition: The prefix function $\pi[i]$ stores the length of the *longest proper prefix* of the substring $P[0 \dots i]$ that is also a *suffix* of $P[0 \dots i]$.

Intuition: If a mismatch occurs after matching $i + 1$ characters, we already know the exact characters in the text that matched $P[0 \dots i]$. The π array tells us how far we can shift the pattern to align its longest valid prefix with the suffix of the text we just matched, preventing us from backtracking in the text string.

Let the pattern be $P = \text{ABABC}$ (length $m = 5$). We compute π iteratively:

- Base Case ($i = 0$):** $P[0] = \text{A}$. A single character has no proper prefix.
 $\implies \pi[0] = 0$.
- Step $i = 1$:** $P[1] = \text{B}$. The substring is AB. The proper prefix is A, suffix is B. No match.
 $\implies \pi[1] = 0$.
- Step $i = 2$:** $P[2] = \text{A}$. The substring is ABA. The longest proper prefix that is a suffix is A.
 $\implies \pi[2] = 1$.
- Step $i = 3$:** $P[3] = \text{B}$. The substring is ABAB. The longest proper prefix that is a suffix is AB.
 $\implies \pi[3] = 2$.
- Step $i = 4$:** $P[4] = \text{C}$. The substring is ABABC. We try to extend the previous match of length 2 (AB). We compare $P[\pi[3]] = P[2] = \text{A}$ with $P[4] = \text{C}$. Since $\text{A} \neq \text{C}$, we face a mismatch.
 - We fall back to the next longest potential match: $\pi[\pi[3] - 1] = \pi[1] = 0$.
 - We compare $P[0] = \text{A}$ with $P[4] = \text{C}$. They do not match.
 - We have reached length 0, so no prefix matches the suffix.
 $\implies \pi[4] = 0$.

Index i :

Pattern P :

Prefix Array π :

0	1	2	3	4
A	B	A	B	C
0	0	1	2	0

2. Searching for the Pattern in Text (T)

How KMP uses the prefix function: We iterate through the text $T = \text{ABABABABC}$ with a pointer i , and keep a pointer j for the pattern P . If $T[i] == P[j]$, we increment both. If a mismatch occurs ($T[i] \neq P[j]$) and $j > 0$, we *do not backtrack i* . Instead, we update $j = \pi[j - 1]$. This effectively shifts the pattern to the right, aligning the longest matching prefix without re-evaluating the characters in T .

Step-by-step Trace on $T = \text{ABABABABC}$ ($n = 9$):

Text T :

Step 1:

Step 2:

Step 3:

0

1

2

3

4

5

6

7

8

A

B

A

B

A

B

A

B

C

A

B

A

B

C

A

B

A

B

C

Match fails at $j = 4$

Match fails at $j = 4$

Full match!

$\pi[3] = 2$. Shift pattern so $P[2]$ aligns with $T[4]$.

$\pi[3] = 2$. Shift pattern so $P[2]$ aligns with $T[6]$.

Match found!
 $j = 5$
Index = $8 - 5 + 1 = 4$.

- **Step 1** ($i = 0 \dots 4$): $P[0 \dots 3]$ (ABAB) matches $T[0 \dots 3]$. At $i = 4$, $T[4] = \mathbf{A} \neq P[4] = \mathbf{C}$. The length of the matched segment is 4, ending at $P[3]$. We look up $\pi[3] = 2$. We shift the pattern such that j becomes 2. The next comparison is $T[4]$ vs $P[2]$.
- **Step 2** ($i = 4 \dots 6$): We resume matching. $T[4 \dots 5]$ matches $P[2 \dots 3]$ (AB). At $i = 6$, $T[6] = \mathbf{A} \neq P[4] = \mathbf{C}$. Again, mismatch at $j = 4$. We look up $\pi[3] = 2$. We shift the pattern such that j becomes 2. The next comparison is $T[6]$ vs $P[2]$.
- **Step 3** ($i = 6 \dots 8$): We resume matching. $T[6 \dots 8]$ matches $P[2 \dots 4]$ (ABC). Since j reaches $m = 5$, a full match is recorded at index $i - m + 1 = 8 - 5 + 1 = 4$.

3. Algorithms

Algorithm 91 Compute-Prefix-Function(P)

```

1:  $m \leftarrow \text{length}[P]$ 
2: Let  $\pi[0 \dots m-1]$  be a new array
3:  $\pi[0] \leftarrow 0$ 
4:  $j \leftarrow 0$  ▷ Length of previous longest prefix
5: for  $i \leftarrow 1$  to  $m-1$  do
6:   while  $j > 0$  and  $P[i] \neq P[j]$  do
7:      $j \leftarrow \pi[j-1]$  ▷ Fallback
8:   end while
9:   if  $P[i] == P[j]$  then
10:     $j \leftarrow j + 1$ 
11:   end if
12:    $\pi[i] \leftarrow j$ 
13: end for
14: return  $\pi$ 

```

Algorithm 92 KMP-Matcher(T, P)

```

1:  $n \leftarrow \text{length}[T]$ ,  $m \leftarrow \text{length}[P]$ 
2:  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
3:  $j \leftarrow 0$  ▷ Characters matched
4: for  $i \leftarrow 0$  to  $n-1$  do
5:   while  $j > 0$  and  $T[i] \neq P[j]$  do
6:      $j \leftarrow \pi[j-1]$  ▷ Shift pattern
7:   end while
8:   if  $T[i] == P[j]$  then
9:      $j \leftarrow j + 1$ 
10:  end if
11:  if  $j == m$  then
12:    Print "Match at index "  $i - m + 1$ 
13:     $j \leftarrow \pi[j-1]$  ▷ Look for next matches
14:  end if
15: end for

```

4. Complexity Analysis

- **Time Complexity:**

- *Prefix Computation:* $\mathcal{O}(m)$. The ‘while’ loop decrements j (via $\pi[j-1]$), but j is incremented at most $m-1$ times in the ‘for’ loop. Since $j \geq 0$, the total number of decrements across the entire loop is bounded by m . Thus, amortized work inside the loop is $\mathcal{O}(1)$.
- *Searching Phase:* $\mathcal{O}(n)$. By the exact same amortized argument, the pointer j is incremented at most n times. The inner ‘while’ loop decreases j , meaning it can execute at most n times total across the whole search.
- **Total Time Complexity:** $\mathcal{O}(n + m)$.

- **Space Complexity:** $\mathcal{O}(m)$ to store the π array.

S7 — Randomized Algorithms

Monte Carlo Algorithms & Probability Amplification

Q1. Suppose you have designed a Monte Carlo Algorithm for a given problem which succeeds with probability $\geq \frac{1}{n^2}$. However, your boss has asked you to achieve at least $(1 - \frac{1}{n})$ success probability. Explain how you can achieve this. **(10 marks)** [CSE 207 | L-2/T-1 | 2024–25]

Answer

1. Intuition and Strategy: Probability Amplification

The problem describes a **Monte Carlo Algorithm** that has a relatively low probability of success ($p \geq \frac{1}{n^2}$). To satisfy your boss’s requirement of a much higher success probability ($P_{\text{success}} \geq 1 - \frac{1}{n}$), we use a standard randomized algorithms technique known as **Probability Amplification**.

The core idea is simple: *Run the algorithm multiple times independently*. Assuming this is a typical Monte Carlo algorithm with *one-sided error* (i.e., we can easily verify a correct answer if we see one, or false positives are impossible, as in Karger’s Min-Cut algorithm), we only need the algorithm to succeed **at least once** across all our independent runs.

By running the algorithm k times, the only way the overall process fails is if *every single independent run fails*. Since the runs are independent, the probability of complete failure decays exponentially as k increases. Our goal is to find the minimum number of runs k that drives the failure probability below $\frac{1}{n}$.

2. Mathematical Derivation

Let A be our Monte Carlo algorithm. We are given:

$$\mathbb{P}(A \text{ succeeds in a single run}) \geq \frac{1}{n^2}$$

Therefore, the probability that a single run fails is bounded by:

$$\mathbb{P}(A \text{ fails}) \leq 1 - \frac{1}{n^2}$$

Let A' be the amplified algorithm that runs A independently k times and returns the successful outcome. A' fails only if all k runs fail. Because the runs are independent, the overall failure probability is the product of individual failure probabilities:

$$\mathbb{P}(A' \text{ fails}) = \prod_{i=1}^k \mathbb{P}(A \text{ fails}) \leq \left(1 - \frac{1}{n^2}\right)^k$$

We are required to achieve a success probability of at least $1 - \frac{1}{n}$, which means the overall failure probability must be at most $\frac{1}{n}$:

$$\left(1 - \frac{1}{n^2}\right)^k \leq \frac{1}{n}$$

To solve for k , we use a fundamental inequality in calculus, valid for all $x \in \mathbb{R}$:

$$1 - x \leq e^{-x}$$

Substituting $x = \frac{1}{n^2}$, we can bound the failure probability:

$$\left(1 - \frac{1}{n^2}\right)^k \leq \left(e^{-1/n^2}\right)^k = e^{-k/n^2}$$

To strictly satisfy our condition, we enforce that the upper bound is less than or equal to our target failure probability:

$$e^{-k/n^2} \leq \frac{1}{n}$$

Now, we take the natural logarithm ($\ln$) of both sides:

$$\begin{aligned} \ln\left(e^{-k/n^2}\right) &\leq \ln\left(\frac{1}{n}\right) \\ -\frac{k}{n^2} &\leq -\ln(n) \end{aligned}$$

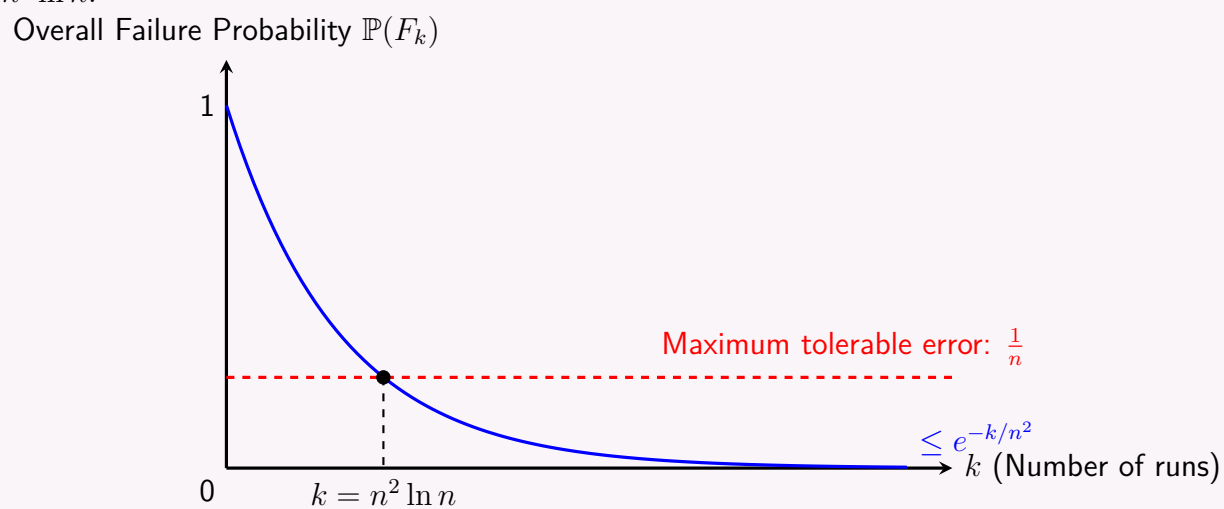
Multiplying both sides by $-n^2$ (and flipping the inequality sign):

$$k \geq n^2 \ln n$$

Conclusion: By running the algorithm independently $k = \lceil n^2 \ln n \rceil$ times, we guarantee that the overall success probability is at least $1 - \frac{1}{n}$.

3. Visualizing the Exponential Decay of Error

The following graph illustrates how the failure probability drops exponentially with the number of runs k , crossing the required threshold when $k \approx n^2 \ln n$.



4. Formal Amplified Algorithm

We define the wrapper algorithm that executes the base Monte Carlo algorithm A exactly $\lceil n^2 \ln n \rceil$ times.

Algorithm 93 Amplified-Monte-Carlo(Input X , size n)

```

1:  $k \leftarrow \lceil n^2 \ln n \rceil$ 
2:  $best\_solution \leftarrow \text{NULL}$ 
3: for  $i \leftarrow 1$  to  $k$  do
4:    $solution \leftarrow \text{Run-Algorithm}(A, X)$  ▷ Run the base  $\geq \frac{1}{n^2}$  algorithm
5:   if Is-Valid-And-Better( $solution$ ,  $best\_solution$ ) then
6:      $best\_solution \leftarrow solution$ 
7:   ▷ If  $A$  is a decision problem with one-sided error, we can return YES immediately
8:   end if
9: end for
10: return  $best\_solution$ 

```

5. Complexity Analysis

Let $T(n)$ be the worst-case time complexity of the original Monte Carlo algorithm A , and $S(n)$ be its space complexity.

- **Time Complexity:** The base algorithm is executed $k = \mathcal{O}(n^2 \log n)$ times. The verification step (line 5) usually takes $\mathcal{O}(V(n))$ time. Thus, the total time complexity is $\mathcal{O}(n^2 \log n \cdot (T(n) + V(n)))$.
- **Space Complexity:** The space required is dominated by a single run of the algorithm (since runs are independent and memory can be reused) plus the space to store the best solution. The space complexity remains $\mathcal{O}(S(n))$, keeping the memory footprint asymptotically unchanged.

Las Vegas Algorithms & Randomized Quick Sort

Q1. Analyze the running time of the randomized quick sort algorithm. **(12 marks)**

[CSE 207 | L-2/T-1 | 2024–25]

Answer

Randomized Quicksort is an elegant modification of the standard Quicksort algorithm. By choosing the pivot element uniformly at random from the subarray, we ensure that the expected running time is $\mathcal{O}(n \log n)$ regardless of the input distribution. This effectively defeats any worst-case input permutations (like already sorted arrays) that would typically degrade deterministic Quicksort to $\mathcal{O}(n^2)$ time.

1. Algorithm and Pseudocode

The core idea is to swap the last element of the current subarray with a randomly chosen element before performing the standard partition operation.

Algorithm 94 Randomized-Partition(A, p, r)

```

1:  $i \leftarrow \text{Random}(p, r)$ 
2: Exchange  $A[r]$  with  $A[i]$ 
3: return Partition( $A, p, r$ )

```

Algorithm 95 Randomized-Quicksort(A, p, r)

```

1: if  $p < r$  then
2:    $q \leftarrow \text{Randomized-Partition}(A, p, r)$ 
3:   Randomized-Quicksort( $A, p, q - 1$ )
4:   Randomized-Quicksort( $A, q + 1, r$ )
5: end if

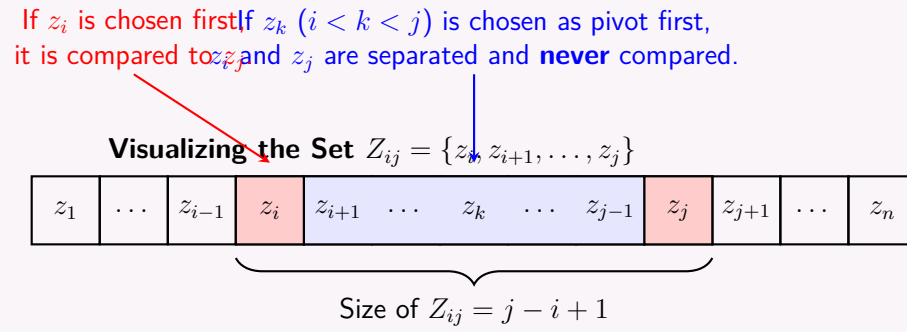
```

*Note: The standard **Partition** function uses $A[r]$ as the pivot, moving elements smaller than the pivot to its left and larger elements to its right, returning the final pivot index.*

2. Intuition for Expected Running Time

The running time of Quicksort is dominated by the time spent in the **Partition** routine. Each call to **Partition** takes time linear in the size of the subarray, doing at most one comparison per element against the pivot. Therefore, the total running time is proportional to the **total number of comparisons** made across all recursive calls.

Rather than analyzing the recursion tree directly, we can use **Indicator Random Variables** to compute the expected total number of comparisons.



3. Rigorous Expected Time Complexity Proof

Let $Z = \{z_1, z_2, \dots, z_n\}$ be the elements of the array sorted in strictly increasing order.

Let the set $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$ denote the set of elements between the i -th and j -th smallest elements, inclusive.

Let X_{ij} be an **indicator random variable** defined as:

$$X_{ij} = \begin{cases} 1 & \text{if } z_i \text{ is compared to } z_j \text{ at any time during execution} \\ 0 & \text{otherwise} \end{cases}$$

Since elements are only compared to the pivot, z_i and z_j are compared **if and only if** the first element chosen as a pivot from the set Z_{ij} is either z_i or z_j . If any other element z_k (where $i < k < j$) is chosen as a pivot first, z_i and z_j will be partitioned into separate subarrays and will never be compared.

Because the pivot is chosen uniformly at random, every element in Z_{ij} has an equal probability of being the first one chosen. The number of elements in Z_{ij} is $j - i + 1$. Thus, the probability that z_i or z_j is chosen first from this set is:

$$\mathbb{P}(z_i \text{ is compared to } z_j) = \frac{2}{j - i + 1}$$

By the property of indicator random variables, the expected value is exactly the probability:

$$\mathbb{E}[X_{ij}] = \mathbb{P}(z_i \text{ is compared to } z_j) = \frac{2}{j - i + 1}$$

Let X be the total number of comparisons performed. X is the sum of X_{ij} over all unique pairs:

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}$$

By the **Linearity of Expectation**, we can compute the expected total number of comparisons $\mathbb{E}[X]$:

$$\begin{aligned} \mathbb{E}[X] &= \mathbb{E} \left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} \right] \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \mathbb{E}[X_{ij}] \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1} \end{aligned}$$

To evaluate this sum, we perform a change of variables, letting $k = j - i$:

$$\begin{aligned} \mathbb{E}[X] &= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} \\ &< \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} \quad (\text{bounding the inner sum to } n \text{ for simplicity}) \\ &= \sum_{i=1}^{n-1} \left(2 \sum_{k=1}^n \frac{1}{k} \right) \end{aligned}$$

The inner sum $\sum_{k=1}^n \frac{1}{k}$ is the n -th Harmonic number, H_n , which is tightly bounded by $\ln n + \mathcal{O}(1)$. Therefore:

$$\begin{aligned} \mathbb{E}[X] &< \sum_{i=1}^{n-1} 2 \ln n \\ &= 2(n-1) \ln n \\ &= \mathcal{O}(n \log n) \end{aligned}$$

Thus, the expected number of comparisons is $\mathcal{O}(n \log n)$, which implies the **expected running time is $\mathcal{O}(n \log n)$** .

4. Worst-case and Space Complexity Analysis

- **Worst-case Time Complexity:** $\mathcal{O}(n^2)$

Although highly improbable, if the random number generator continually happens to select the absolute maximum or minimum element of the subarray as the pivot at every recursive step, the array will be partitioned into sizes of 0 and $n-1$. The recurrence relation becomes $T(n) = T(n-1) + \Theta(n)$, solving to $\Theta(n^2)$. The probability of this occurring on a distinct array of size n is $\frac{1}{n!}$, which is astronomically small for large n .

- **Space Complexity:** $\mathcal{O}(\log n)$ **Expected**, $\mathcal{O}(n)$ **Worst-case**

Quicksort is an in-place sorting algorithm (ignoring the call stack). The space complexity is dictated by the maximum depth of the recursion tree.

- Expected depth: Since the partition is well-balanced probabilistically, the expected recursion depth is $\mathcal{O}(\log n)$.
- Worst-case depth: In the severely unbalanced case described above, the recursion tree degenerates to a linked list, requiring $\mathcal{O}(n)$ auxiliary stack space.